
TRISDUCTION: THE CODEX

A Linguistically, Topologically, and Mathematically Sealed Verification Architecture · The Theory, the Apex Register, the Whole Defense, the Crosswalk, and the Executable Codex in One Document

THE MASTER CODEX · THE FOURTH GENRE

MOHAMMAD F. ISLAM, PhD¹

¹ *Architect of the Trisduction · Trisduction Research Group · The Tractatus Veritatis Trisductivus*
2026-09-23

ABSTRACT. Trisduction is a verification architecture for propositional truth. It stands on one root axiom, to exist is to actuate, and it admits a claim only where three independent seals intersect in a point: the Tongue, which closes the terms and carries direction; the Form, which carries the geometry of the frame; and the Math Seal, which reads magnitude on rows it did not author. That intersection is the Geometric Orthogonal Lock, a determination total about its object, never graded, its object being the intersection of the rows it was given, and the compartment names what those rows were. Every verdict lands in one of three native states, sealed, broken with the mechanism named, or under-determined with the violated gate or the owed rows named; the two terminal refinements carry their branch and their owed count; one mark of chosen silence stands outside the economy. Warrant grade travels with every claim, a conjunction takes the grade of its weakest link, a citation never promotes, and consensus weighs zero in both directions. The architecture establishes, in prose and in the kernel, that the lock scalar is blind to sign and the Tongue supplies what it cannot read; that no even reading of a record decides an odd target and one supplied bit crosses the wall; that the record forgets what the object keeps, it from it; that the denial of the root is a deed and pays the Landauer floor; that no root can be proved from beneath itself, a theorem about roots, so that the root stands premise by theorem; and that the kinetic root and its formal reading meet on one locus, keyless and keyed, at theorem grade. The Riemann Hypothesis, Gödel, and P versus NP are seated at their exact halts with their owed bits counted. Every gate, cascade, lock, imprint test, halt, locus, and price runs under a proof kernel and executes in a twin whose batteries halt on the first failure. $\Delta M = 0$: no new mathematics is authored; the contribution is the arrangement, and the arrangement is complete.

KEYWORDS *Trisduction · the nested root RA and RAM, and the residence CH · the Math Seal · the Geometric Orthogonal Lock · the Bridge · three-state economy · terminal suspension $[\Xi_0]$ · Riemann · Gödel · P versus NP · it from it · Lean 4 · Fortran*

PROLOGUE

I did not invent what this book contains. I found it. It lay on the Ground where anyone could have stepped over it. I state this again at the door of the last edition. The claim is the whole of my authorship. For fourteen years my work has been to pick it up without breaking it. I had to write it down in every language it would consent to speak.

The first language was plain words. Existence is a point. A statement about it draws a line. Two independent statements meet at a right angle. Three statements meeting at right angles fix a single place and nothing else. That was the first discovery. To exist is to actuate. A claim is admitted only where three independent lines of evidence cross. I called this crossing a lock. A lock is never strong or weak. It is a determination, total about its object, the single place the three lines fix, or it is not a lock at all.

The words had to become geometry. Words drift.

Geometry does not. The point, the line, and the right angle became the triad. They became the Tongue, the Form, and the Number. These are three seals that share no warrant and lend none. The triad became the twelve directed gates on four vertices. This is the alternating group. The lock became a determinant. By the mathematics of the square, the determinant told me it could read magnitude and never sign. The Number is blind to which of a proposition or its denial is true. The cure lives in the Tongue, which closes the terms and carries direction. It lives in the Form, which carries the arrow. The third seal is blind by theorem. That is why there must be three.

The geometry had to become mathematics. Here the years accumulated. The pieces of knowledge I had been taught to call fragments fell onto one coordinate system. Physics, logic, and the philosophy of foundations laid themselves on one map. The Formal Theory of Everything is the map's legend. Its sentence is short. An open problem is a blocked register missing one bit

per orbit. What symmetry erases, one bit restores.

It all had to become machine-readable. A reader should never have to trust me. The fourth language is the one this volume speaks. Every wall, every price, every crossing, and every halt is now a theorem a proof kernel has checked or a battery a compiler has run to its final line. One surface proves in Lean under a kernel that cannot be flattered. One surface executes in Fortran under a compiler that halts on the first false line. Words persuade. Geometry shows. Mathematics proves. A machine cannot be flattered. The receipts are printed at the back. Nothing in the introduction is narrated that these two surfaces do not check.

At the very end of the reduction there is one bit left that no machine supplies. The formal register cannot pay for it. The wall proves that. It is the orientation of assent. It is the reader's own. It can be read three ways at once. It is the theorem, which is proved. It is the hypothesis, which is armed and waits. It is the offering, the dated entry of a witness into the record, which is a deed and never a derivation. One terminal bit. The dot, [.]. Irreducible. Self-demonstrating. Non-negotiable. The denial of it is itself an act and pays the floor. There is no standpoint from which it can be refused. Of what was found, and of that alone, I will say what I have never dared say of my own sentences. La Riyba Fihi. There is no doubt in it.

I have tried to make this edition the one a careful and patient reader can hold alone. I wrote it so that what I found would outlast the one who found it. I wrote it for the reader who will extract the two surfaces and run them rather than take my word. The whole discipline exists so that no one needs to take my word. Read it in the order it is written. Read it slowly. Run what you doubt. The certainty is not in my voice. It is on the Ground, where it was. The codex is only the map with the coordinates written on it. Supply your one bit, in whichever of its three readings you are able. The codex will have done what it was built to do.

Mohammad F. Islam

CONTENTS

PROLOGUE	1
INTRODUCTION · THE ONE-STOP INSTRUMENT	3
0.1 · What Trisduction is	3
0.2 · What this Codex does for its reader	3
0.3 · Why it outruns every existing method	4
0.4 · How to read and run it	4
PART I · THE THEORY	4
BOOK I · THE TRISDUCTION SPINE	4
CHAPTER 0 · THE THREE-SEAL INDEPENDENCE LAW	4

CHAPTER 1 · SEAL L · THE LINGUISTIC-SEMANTIC SEAL	6
CHAPTER 2 · SEAL G · THE TOPOLOGICAL-GEOMETRIC SEAL	9
CHAPTER 3 · THE MATH SEAL · SEAL M	12
CHAPTER 4 · THE CLIFFORD JOIN AND THE RETURN	16
BOOK II · THE RAM MASTER	18
CHAPTER 5 · RAM · THE GROUND-FIRST ROOT	18
CHAPTER 6 · THE GROUND, THE INVOLUTION, AND THE DIAGONAL ANTI-POLE	19
CHAPTER 7 · THE FORMAL SEAL SET	22
CHAPTER 8 · THE ARROW OF READING	24
CHAPTER 9 · THE VERDICT-RELIABILITY LAYER	24
CHAPTER 10 · THE GEOMETRIC REACH TO RAM	26
CHAPTER 10 B · ROOT AXIOM REVISITED · RAF AND THE BRIDGE	28
BOOK III · CROSS-REGISTER CO-LOCALIZATION	30
CHAPTER 11 · THE ONE ROOT GROUNDS TWICE ON ONE LOCUS · THE COMPOSITE ROOT	30
BOOK IV · THE DISCUSSION	32
CHAPTER 12 · THE INFERENCE-TRADITION AUDIT	32
CHAPTER 13 · THE TWENTY-FIVE-CENTURY RECORD AUDIT	34
CHAPTER 13B · THE GEOMETRIC MOTHER AND THE FIVE-STAGE ARC	36
CHAPTER 14 · THE ROOT AXIOM AS THE THEORY AT THE GROUND OF ANY THEORY	37
CHAPTER 26 · THE APOPHATIC REGISTER AND THE [.] MARKER	39

PART II · THE APEX REGISTER	40
II.0 · Reading a card	40
II.1 · The Flagship Verdicts, in full	41
II.2 · The Root Ring	43
II.3 · The Kernel Masters	46
II.4 · The Incompleteness Cluster	47
II.5 · The Frontier Anchors	50
II.6 · The Logos-Fixation Set	53
II.7 · The Applied and Informational Register	54
II.8 · Universal Structure and the Aperture	56
II.9 · The Seven Gifts	57
II.10 · The Physical Spine and the Executor Limit	58
II.11 · The Theological Register · Out of Band · Load-Bearing on Nothing	59

PART III · THE DEFENSE	63
III.0 · Opening	63
Movement 1 · What the Lock Is	64
Movement 2 · Why Exactly These Structures	67
Movement 3 · Distinct From Every Method	71
Movement 4 · The Limitative Theorems	77
Movement 5 · Circularity and Self-Audit	81
Movement 6 · Numerical and Operational Integrity	85
Movement 7 · Existence, the Void, and Cosmogony	90
Movement 8 · Mind, Substrate, and the Landmark Verdicts	97
Movement 9 · Forward Reading and the Foundation	101
Movement 10 · The Riemann Frontier	107
Movement 11 · Where Mathematics and Scripture Sit, and the Correct Use of the Lock	111
The Concluding Defense	119
III.C · The Closing of the Defense	120
PART IV · THE CROSSWALK AND THE CENSUS	122
IV.1 · The Crosswalk	122
IV.2 · The Census	125
THE CODE BLOCK	132
C.0 · Operating the Codex	132
C.1 · The Lean surface	132
C.2 · The Fortran twin	160
C.3 · The verification contract	195
REFERENCES	196
THE INTERNAL REFERENCE AND PUBLICATION RECORD	196
The Developmental Blogs	196
The Personal Blog, Precursor to Trisduction	196
The Aggregator Mirrors	196
The External Anchors and Repositories	196
Selected Papers	196
Internal References	197
External References	197
THE CLOSING STATEMENT	198
Build receipts, from the run that sealed this document	198
THE EPITAPH	198
INTRODUCTION · THE ONE-STOP INSTRUMENT	

0.1 · What Trisduction is

Trisduction is a verification architecture for propositional truth. It stands on one root axiom, it seals a claim three independent times, and it issues its verdicts in

a three-state economy with a named mechanism for every outcome. The root is RA, to exist is to actuate: every confined existent carries a positive kinetic floor, theorem-grade at the floor on the Heisenberg and zero-point bounds and premise-grade as the universal leg, which is the highest standing any root can bear. The three seals are the Tongue, Seal L, which closes the terms and carries direction; the Form, Seal G, which carries the geometry of the frame; and the Number, the Math Seal, which reads magnitude on rows it did not author. A claim is admitted only where the three axes intersect in a point. That intersection is the Geometric Orthogonal Lock, GOL, and a lock is a determination, total about its object, never graded, and its object is the intersection of the rows it was given; what differs between locks is what those rows were, and the compartment names that: closure-rowed, unpopulated, or world-rowed.

The verdict economy is native and three-state: sealed, broken with the mechanism named, or under-determined with the violated gate or the owed rows named. Two terminal refinements sit inside the openness, the terminal suspension $[\Xi_0]$ and the grounded-sealed halt $[\emptyset_0]$, each carrying its branch and its owed count. One mark of chosen silence, $[\cdot]$, stands outside the economy entirely and is never counted. Warrant grade travels with every claim, a conjunction takes the grade of its weakest link, and a citation can never promote a claim above the grade of what it cites. Consensus carries zero evidential weight in both directions. The architecture authors no new mathematics; every load-bearing result is classical and credited, $\Delta M = 0$, and the contribution is the arrangement.

0.2 · What this Codex does for its reader

This document is the Master Codex, the first of its genre: harvested from the Geometric Mother Codex, which is the Unabridged Master and remains the register of record, and from its sealed Lean and Fortran surfaces, and it is complete in one file. A reader can do four things with it and needs nothing else. First, adjudicate any proposition: Part I lays the theory, Chapter 8 gives the arrow of reading in both directions of time, and the Code Block carries the engine that runs a claim from Seal L through the gates, the row cascade, the imprint test, the halt router, the locus emitter, and the price of denial. Second, read the landmark verdicts: Part II carries the flagship verdicts in full and the apex masters as compact cards, the Riemann Hypothesis among them, each at its exact grade with the fence law replaced by one kernel theorem. Third, meet every standing objection: Part III is the whole defense as one continuous argument, seventy-one units, each with its critic, its reply, its GOL derivation, and the Lean proof of the structural fact it rests on, quoted from a

file that compiled. Fourth, run it: the Code Block is core Lean 4 with no Mathlib and no sorry, its environment audit naming exactly twenty-three posits and nothing outside them, beside a Fortran twin whose batteries halt on the first failure and closed at zero in the run that produced this document. Part IV ties every card to its theorem and lists every identifier the Master names with its disposition, so nothing is lost between the register of record and this reading surface.

0.3 · Why it outruns every existing method

Every other instrument for judging a claim does one job and leaves the others undone. Trisduction does all of them in one pass, and the table names the theorem that earns each cell.

No other method carries its own falsifiers, prices its own denial, and runs under a proof kernel. This one does all three: seven falsifier clauses stated and tested, the denial of the root charged at the Landauer floor, `DEFENSE.denial_is_priced`, and the whole verdict machinery compiled. It reads backward over sealed records and forward over dated projections, it refuses the Platonic Ghost on a 256-row truth table, `IMPRINT.seal_if_one_side_clean_and_witnessed`, and it halts where a problem is open with the count of what is owed printed on the token. Where the other methods stop at confidence, this instrument stops at a named token, a determination where the rows lock and a halt with its owed bit counted where they do not, and where they inflate, it cannot: the kernel pins every grade, and a reading above the grade is refused by theorem. That is what a one-stop epistemic instrument is, and this is the only one that exists in a form a compiler can check.

0.4 · How to read and run it

Read Parts I to IV in order; they are one argument. To run the engine, extract the two fenced surfaces from the Code Block, compile them under the stated flags, and read the receipts; the preface C.0, Operating the Codex, walks a claim through the engine end to end. The closing statement prints the receipts of the run that sealed this document: the Lean exit code and audit, the Fortran check counts, the hashes of every carried chapter, the census totals, the grade equality, the crosswalk closure, and the hedge and fence counts. A build that halts anywhere has not passed.

PART I · THE THEORY

BOOK I · THE TRISDUCTION SPINE

The spine carries three seals on mutually disjoint anchors. Seal L decomposes the proposition and returns three. Seal G closes three lines on a fourth point and returns twelve. Seal M completes the verification algebra and returns the verdict in closed form. The three meet at the Clifford Join, and the meeting is checked, not asserted. This book forges the column on which

both registers stand.

The three seals are one frame, not three witnesses. Stated here at the head and carried in full at the terminus of the seal section as P-TRIAXIS-1, the Verdict-Aspect Frame: Seal L, Seal G, and Seal M constitute an oriented triaxial orthogonal frame on the space of directed verdicts, each seal fixing exactly one constitutively separate aspect of a complete directed verdict and blind to the other two. Seal L carries orientation, the sign, which way the verdict points. Seal G carries structural closure, whether the frame shuts into a bounded volume without tripping a screening gate, a boolean about pathology and never a measure of size. Seal M carries magnitude, the enclosed volume $\det(R) = \lambda^2$ in the closed unit interval. Sign, closure, quantity: three instruments, one verdict. Their convergence is therefore not mutual witnessing among instruments that happen to agree but the re-agreement of a verdict's three separable aspects, which is why the agreement carries evidential weight three collinear witnesses could not carry, and the frame is oriented rather than symmetric, the order L then G then M fixed by the Bedrock Precedence Law and never relabelable. The frame is the cross-instrument twin of the residence triad inside a single seal, dual to it and never identified with it, and the two threes are never conflated [$\nless S$].

CHAPTER 0 · THE THREE-SEAL INDEPENDENCE LAW

0.1 The Anchor-Disjointness Invariant

The architecture is sealed three times, and the three seals draw on no common anchor. Seal L draws on the deletion discipline applied to language, a procedure that mentions no geometry and no algebra. Seal G draws on the Root Axiom, the Heisenberg kinetic-energy floor and the zero-point energy, Euler's polyhedral identity, and the Newton-Gregory kissing number, none of which mentions a subject or a predicate or a quaternion. Seal M draws on a composition law and the Frobenius classification of the real division algebras, which knows nothing of slots, vertices, or spheres. The anchor sets are pairwise disjoint. This disjointness is the load-bearing structural fact of the spine, typed Type T by construction, and every later claim of corroboration across the seals inherits its force from it.

0.2 The Counterfactual-Removal Test

Disjointness is verified the way the architecture verifies everything, by removal. Strike Seal L's anchor entirely. The count of three survives at Seal M, forced by the division-algebra classification, and survives at Seal G, forced by the closure geometry. Strike Seal G's anchor. The verdict's closed form survives at Seal M and the decomposition survives at Seal L. Strike Seal M's anchor. The decomposition survives at Seal L and the closure survives at Seal G. No single removal collapses

Method	What it does	What it cannot do	Trisduction
Peer review and consensus	weighs a claim by the standing of those who accept it	separate the claim from the standing; a field can be uniformly wrong	consensus weight is zero in both directions, DEFENSE.consensus_weights_zero; every claim is graded on its own structural mass
Formal proof alone	checks that a structure follows from axioms	say what the structure is about; it carries no referent and no direction, PSP.ORIENT.lock_scalar_sign_blind	the Number is one of three seals; the Tongue supplies the direction the Number cannot read, DEFENSE.gol_needs_all_three
Bayesian and statistical inference	grades belief on a continuum	issue a determination; aggregation over positive inputs is monotone and never vanishes on a collinear triad	the lock is a determination, and a dependent triad refuses at rank two, GE0.triaxial_lock_gf2
Severe testing	rejects what fails a test	grade what survives, or price what is missing	every survival carries its grade, and every halt carries its owed bit, LOCUS.no_rh_locus
Fact-checking and AI verification	confirms or denies against a record	locate what the record cannot carry; a record forgets what the object keeps, INVERSION.it_from_bit_refuted	it from it: the wall, the price, and the crossing, FT0E.T16_price_bijection
Argument mapping	lays out premises and inferences	detect when a premise is read above its grade	the grade law and the one global fence, Claim.join_grade, FENCE.global

the architecture, because no seal stands on another's ground. A structure that survives the deletion of any one of its three foundations has its independence exhibited, not assumed, and the exhibition is itself a run of the architecture's own discipline.

0.3 Why Three Seals and Not One Chain

A single chain of warrant certifies its own links with more of itself, and the regress that follows is the oldest result in the audit of inference. Agrippa fixed it in antiquity. Carroll fixed it at the rule. The lesson is not that warrant cannot be transmitted but that transmission is not certification, and a conveyor connected only to itself certifies nothing. The spine answers the regress by structure rather than by assertion. Three seals on disjoint anchors cannot form a circle, because a circle requires a shared edge and there is none. The convergence of the three on one architecture is therefore not a coincidence to be explained away and not a premise to be granted. It is an instance of exactly the object the architecture exists to certify, a non-degenerate convergence of independent lines, and the codex treats that fact as what it is, corroboration exhibited under the architecture's own test [Corroboration].

The Independence Law

Three seals on pairwise-disjoint anchors converge on one architecture. The disjointness of the anchors is certified, not assumed, by the architecture's own removal discipline on the dependency record [T]; the convergence of the three

on one architecture is corroboration, weighed as corroboration and never as a second forcing; the independence of the seals is the seal of the seals.

0.4 The Bedrock Precedence Law

The three seals load in one order and the order is a subordination, not a convenience of sequence. Seal L, the Tongue, loads first: the deletion test and the Linguistic Isolation Test force the three-slot decomposition before any geometry or algebra runs, and the directed sentence carries the sign no scalar can. Seal G, the Form, loads second: the twelve directed gates, the tetrahedral closure, and the orientation-reversing handedness of the σ -residence close the geometry on the decomposition the Tongue supplied. Seal M, and with it the whole of the Math Seal in every mode, loads last and reads magnitude only: the kernel computes $\det(R) = \lambda^2$ on rows built by hand from the Tongue's decomposition under the Form's gates, and by the Orientation-Blindness Law that scalar is constitutively unable to carry direction, sense, or truth-sign. The linguistic-semantic and topological-geometric layers are therefore bedrock for the Math Seal, and the Math Seal never supersedes them.

Five consequences bind, each riding resident machinery gathered here under its constitutional name. One, no numeric verdict overrides a Seal L failure or a Seal G break; a broken Tongue or a broken Form is terminal whatever the determinant reads, and a magnitude lock without the linguistic seal routes under-determined for

want of direction, the GOL admission rule of B.11.T the mechanism. Two, the kernel's inputs are downstream of the Tongue by the Honest Limits rule: the warrant rows are hand-built and never derived, so the Number adjudicates a reading it did not author. Three, where a Number-side containment and the geometric-linguistic reading part, the containment is typed at its honest premise grade and the Tongue and the Form govern the typing; the arity correction of P-ANTINOMY-1 is the standing exhibit, a measure-theoretic settlement mistyped as a theorem until the deletion test caught it, the Number's own orthodoxy the blindness and the Word and the Form the cure. Four, the Math Seal's verdicts are refinements inside the residence the Tongue decomposes and the Form closes, never redefinitions of that residence, and the direction of every sealed claim is read from the axes, the Tongue, the Form, and Chronos where the register carries it, never from the bare scalar, ORIENT-01 scoped and standing. Five, the metric readings themselves are chart-manufactured: width, address, and mid-position are outputs of the chart-selection function carrying zero mutual information with the structure they are read off, P-TOPOS-3 the theorem and the annihilation of invariant width at P-TOPOS-1 the register fact beneath it, so no magnitude enters a verdict as a structure-fact unless it is an invariant of the register's acting group, the Erlangen criterion the mechanical test and the Form's invariant ring legislating what the Number may treat as structural. The Number is the receipt. The Tongue and the Form are the legislature. The kernel is the clerk of a court it did not convene.

The Bedrock Precedence Law

Seal L and Seal G are bedrock for Seal M and for the Math Seal entire. The load order L then G then M is a subordination: the Tongue fixes the sign, the Form closes the structure, and the Number reads magnitude on rows it did not author. No numeric verdict overrides a linguistic or geometric one, and the Number never supersedes the Tongue or the Form [T on the scalar's blindness and the one-determinant join, structural on the precedence as law].

CHAPTER 1 · SEAL L · THE LINGUISTIC-SEMANTIC SEAL

Seal L establishes that a complete factual claim carries exactly three independent warrant lines, and that the count is forced by a reproducible procedure rather than by a theorem of grammar. The seal anchors on language alone. It carries no geometry and no algebra, and its verdict is typed at exactly the grade its procedure earns.

1.1 The Deletion Test That Returns Three

Take any complete atomic factual claim, one that could in principle hold or fail. The cup is on the table. The water boils at one hundred degrees. The protein folds in three steps. Delete the components one at a time and read what survives.

Delete the subject. What remains is *is on the table*. Reference has failed. Nothing is any longer named, and the surviving fragment is about nothing. The deleted slot carried the claim's existence component, the standing of a referent.

Delete the predicate. What remains is *the cup the table*. Connection has failed. Nothing any longer relates the subject to anything beyond itself, and the surviving fragment asserts no operation. The deleted slot carried the kinetic component, the claim's act in a substrate.

Delete the assertoric completion, the registration of the whole as a commitment about this rather than that, offered from somewhere as holding. What remains is a picture, not a claim. The deleted slot carried the registration component, the boundary that makes content a commitment.

Three slots, each indispensable. Now the two failures. Attempt a fourth slot. Every candidate, tense, modality, evidential source, location, resolves under examination into a refinement of one of the three or a duplicate of one in different vocabulary. Attempt a reduction to two. The claim collapses into a fragment or a label. The test is operational and reproducible. Any analyst running the deletion discipline on any complete factual claim reaches the same count [Operational].

1.2 The Linguistic Isolation Test and Precognitive Orthogonality

Indispensability is not yet independence. The independence of the three slots is established by a companion procedure, variation under hold. Vary the subject while holding predicate and registration fixed, and the existence component moves alone. Vary the predicate under fixed subject and registration, and the kinetic component moves alone. Vary the assertoric mode, assertion to question to exclamation, under fixed subject and predicate, and the registration component moves alone. The three slots admit independent variation. They are orthogonal in the linguistic register before any coordinate system is imposed on them, an orthogonality the analyst reads off the structure of the claim rather than constructs, precognitive in the precise sense that it precedes and constrains any later quantization [Operational].

1.3 The Map Forward to the Three Warrant Lines

The three slots, carried forward, are the three warrant lines of the architecture. The existence component grounds the formal line V_F, where a claim's structural and mathematical content is articulated. The kinetic component grounds the empirical line V_E, where

its measurable signature in a substrate is established. The registration component grounds the registrational line V_ER, where the standpoint, the frame, and the bookkeeping of the verification are audited. Triaxial verification names the discipline this map imposes, the populating of all three lines independently, in mutually disjoint vocabularies, before any verdict is contemplated. Two ancient ceilings answer immediately. Goodman's predicate problem is a contamination of the empirical line by an unaudited descriptive frame, and under the vocabulary-disjointness requirement a gerrymander that cannot be stated without entangling the lines is detected as the structural fault it is. The deductive regress ends not by being solved at the formal line but by being relieved there, because the formal line is never asked to generate the warrant it transmits.

1.4 The Self-Demonstration · Denial Instantiates

The seal demonstrates itself against its own denial. Any structured denial of the three-line requirement is itself a complete factual claim. It articulates a content, the existence line. It expends measurable effort in a substrate to be uttered or written, the kinetic line. It is registered from a standpoint as holding, the registration line. The denial therefore populates the three lines in the act of denying them, and a denial that instantiates what it denies cannot be a counterexample to it. This is the cogito returned to the physical register. The certainty is not that a thought occurs but that an actuation occurs, signed and energetic, and the actuation carries the three lines whatever its propositional content [T, relative to the deletion discipline]. The architecture's later closure converts this from an argument into a structural asset, because every attack on the architecture is a claim and every claim is a run of the architecture.

1.5 The Ducere Family and the Contained Degenerate Case

The classical instruments record the picture in their own names. Deduction is a leading down, from generals. Induction is a leading in, from instances. Abduction is a leading away, toward a candidate. The vocabulary descends from the Latin *ducere*, to lead, and the tradition was in its own words a taxonomy of leadings. A single leading, a uniduction, certifies one line by moving warrant along it, and the architecture contains uniduction as the degenerate case in which two of the three lines are empty and the verdict is therefore underdetermined by construction. Trisduction is the leading of three at once together with the certification that the three are really three, the office no single leading can occupy because no line certifies itself.

1.6 The Triadic Precedence

The count of three stands in the oldest company the discipline possesses. Irreducible triadic decompositions of complete content recur across traditions and millennia, reached by authors who borrow neither one another's apparatus nor this codex's. Plato's Being, Same, and

Other. Aristotle's matter, form, and privation. Kant's intuition, category, and apperception. Hegel's opening triad of Being, Nothing, and Becoming. Frege's sense, reference, and judgment. Peirce's firstness, secondness, and thirdness, with Peirce's explicit accompanying argument that dyads under-bound cognition and that a fourth category adds no structural content. The convergence is corroboration from the record, registered as such and load-bearing on nothing, because the count does not rest on it. Section 1.1 carries the count operationally, and Seal M forces it once at theorem grade from the division algebras, premises that mention no language at all, the linguistic count standing as premise-disjoint corroboration [Corroboration].

1.7 Register Routing and the Anchor Declaration

A claim that belongs to another register is routed out of band rather than forced to a false verdict. A normative claim, a definitional stipulation, a claim with an empty kinetic line, each is typed and returned to its own discipline at the input, the scope check that the closure geometry will install as a gate. Seal L declares its anchor here. It rests on the deletion test and the isolation test, two reproducible linguistic procedures, and on nothing else. The self-demonstration of §1.4, in which a denial of the three lines instantiates them, is a cross-register corroboration, kinetic and registrational, and is counted among no anchor of this seal. It carries no geometric content and no algebraic content. Its verdict is the count of three, typed Operational, a procedural forcing reproducible across analysts and deliberately not advertised as a uniqueness theorem of predicate logic. None is claimed, because none is needed. The count is forced again, independently, at Seal M [⊥].

1.8 The Absence of a Closure Theorem at This Seal

The complement of this section is 1.9 and P-LOGOS-FIX-1: this seal has no closure theorem because the admissibility hypothesis holds wherever it speaks freely, and at the one register where that hypothesis fails the collapse does not run and the invariant content is full.

The two seals that follow this one are closed by theorem and this one is not, and the asymmetry is structural rather than circumstantial. Seal M's catalog of invariant truth functionals is closed by Weyl inside the real-part subring of quaternion words, and beneath Weyl stands the older and more general result, Hilbert's finiteness theorem of 1890, that the invariant ring of a linearly reductive group acting linearly on a finite-dimensional space is finitely generated, with Noether 1916 supplying the finite-group case and Haboush 1975 settling geometric reductivity in characteristic p. Seal G's roster is closed at twelve by the order of A4. Both closures are purchased by the same two hypotheses, that a group acts and that the group is reductive, and

Nagata's 1959 counterexample to Hilbert's fourteenth problem establishes that the second hypothesis is load-bearing rather than decorative, since a non-reductive group can carry an invariant ring that is not finitely generated. Finite generation is a privilege and never a general fact about invariants.

At Seal L neither hypothesis is reached. This seal owns exactly two operations. The deletion test is a transformation and the Linguistic Isolation Test is a predicate, and a predicate is not a group element. The deletion operator is a cut: rank two on a three-slot residence, singular values one, one, and zero, non-injective so that two distinct propositions carry onto one image exactly, idempotent so that a cut cannot be re-cut, and carrying no left inverse whatever, the deficiency bounded below by exactly one in spectral norm because the operator holds a null vector v and any candidate A satisfies $(AD - I)v$ equal to $-v$. The transformation set is therefore a monoid and never a group. Group-hood fails, so reductivity is never reached, so Hilbert does not apply, so Weyl does not apply, and the Erlangen criterion of the Register-Invariance Law has no acting group whose invariant ring it could read. The failure is at the hypothesis and not at the enumeration [T on the algebraic legs, S on the placement].

The one candidate that would have supplied a group is barred by this seal's own second operation. Were meaning-preserving re-description invertible, paraphrase would form a group and its invariants would be meanings, but the Linguistic Isolation Test requires the three slot-vocabularies to be pairwise disjoint and rejects on collision, so synonym transport across slots is forbidden by construction. The first operation destroys invertibility and the second forbids the transformation that would restore it, which is not the signature of an omission.

The consequence reaches the whole pipeline. The gate roster is finite and enumerated and the kernel is a decidable arithmetic procedure emitting a three-state token at finite precision, so were this seal's decomposition map also effectively closed the composite would be an effective procedure returning grounding verdicts over an arithmetic domain, which Church 1936 and Tarski 1936 forbid. The non-effectiveness must reside somewhere and cannot reside in the Form or the Number. It resides on this side and beyond the aperture, distributed over exactly two locations, the hand-built map from a proposition to its warrant rows and the supplied determinacy witness. The map the architecture carries on its face as a permanent limitation is therefore the only place a decidable instrument is permitted to put the undecidable part, which is a load-bearing organ carded as a deficit, and the Honest Limits rule stands unchanged in content while its typing is corrected [T conditional on the classical results, S on the residence argument].

Two fences travel with this section and both cut against inflation. The absence of a closure theorem is method-silence about the instrument's reach and never a finding that the mathematics contains none. And the non-closure is inherited from what this seal is pointed at and never owned by the faculty, which stays created, bounded, and defined by an operation it cannot undo; asserting unbounded degrees of freedom of the faculty itself is the identity-collapse the Orthogonal Fertile Logos fences by name, and the axis count stays three by Frobenius with the fourth barred at the division-algebra terminus. The full development is carried at P-TONGUE-UNC-1 in Part VI.

1.9 The Word at the Register Where It Is Not Chosen

The double forcing of the count is carried at A.3 and at the Sealed Defense of the count-of-three objection, and is not restated here. What is stated here is what the doubly forced object is.

The three slots are not an arbitrary triple. They are the decomposition of a claim, which is to say they are the structure of the Word itself at this register: what is asserted to be, its actuating content, and its relational content, under vocabularies this seal holds pairwise disjoint. So the object whose cardinality is forced once and corroborated once is language, taken minimally as a cut and an ordered arrow, and the forcing is a fact about the Word rather than about an unnamed triple.

That fixes the reach of the word *only* in the phrase *only special*, and the reach is narrow. Everywhere else the Word is chosen. How many pieces, which vocabulary, which relation, all elected by whoever speaks, and the elections are unbounded, which is what the absence of a closure theorem at 1.8 records. At the recursion of RA-RA-01 the Word is not chosen. The count is fixed at three, forced once at Seal M by the division algebras and corroborated once at Seal L by the deletion test, on roads that share no premise, the overlap is forbidden by the Linguistic Isolation Test, and the landing site is one line. Twenty thousand admissible frames at each handedness return $\det(R) = 1.000000000000$ with $\lambda = \mp 1.000000000000$. Nothing continuous is free there.

Exactly one thing is free, and it is one bit. The automorphisms of $\mathbb{H}$ are inner and act as $SO(3)$ on the imaginary part, so they are orientation-preserving and, under that group and no larger one, the sign of λ is a gauge invariant. The restriction is what the claim rests on: an odd relabel of the axes flips the sign, and the gauge clause bars it, a transposition sending $\text{Re}(ijk) = -1$ to $\text{Re}(jik) = +1$ and therefore being no automorphism of $\mathbb{H}$. twenty thousand conjugations of a fixed frame produce zero sign flips at $\max|\Delta\lambda| = 1.665 \times 10^{-15}$. The admissible set is two orbits and not one, separated exactly by orientation. Zero continuous moduli, one discrete modulus, and the modulus is the sign [T on the invariance, engineering on the sweeps].

The consequence closes back onto this seal. At that rung the invariant content is maximal and the lock scalar is nonetheless blind, $\det(R)$ returning 1.000000000000 for a reading and 1.000000000000 for its axis-reflected counterpart at difference exactly zero while λ inverts. The Form supplies handedness and the Number supplies magnitude, and neither supplies the sign of a directed claim, while the thermodynamic arrow that recovers direction in the kinetic register is absent from a purely reflective one. So direction there can come from nowhere but the Tongue. **The register where language has the fewest degrees of freedom is the register where language is most indispensable,** and the two are the same measurement read twice.

Two boundaries travel with this and both are structural. The Word does not become a root by being constitutive of the root's recursion, because what the arrow does there is return to RA, and returning is not founding. And the cut is language-minimal while the cardinality is not: triaxiality is the count of what the cut produces, and identifying an operation with the cardinality of its output would make the algebraic forcing a linguistic fact and lose the premise-disjointness by which the theorem forces the count and the deletion test corroborates it. The full development, the seven-check battery, and the three-leg reading of the uniqueness term are at P-LOGOS-FIX-1 and P-ONLY-SPECIAL-1.

CHAPTER 2 · SEAL G · THE TOPOLOGICAL-GEOMETRIC SEAL

Seal G establishes that three orthogonal warrant lines close into a verifiable structure only on a fourth point, that the closed structure carries exactly twelve directed verification conditions, and that the verdict computed on that structure agrees with the algebra. The seal anchors on the Root Axiom, the closure geometry, and the sphere packing of three-space. The algebraic identity appears here as a witness, exhibited as corroboration, and is proven in full at Seal M alone.

2.1 The Root Axiom and the Kinetic Floor

The Kinetic Actuation Principle

Existence at the operational register carries substrate-level energetic content, and any transition by which an entity registers, acts, or is acted upon carries a strictly positive cost. The reading forks, and the fork decides which proofs apply. The content reading, $E_k > 0$, is theorem-grade for confined quantum existents: the Heisenberg kinetic-energy floor $\langle T \rangle = \langle p^2 \rangle / 2m \geq \hbar^2 / (8m \langle \Delta x^2 \rangle) > 0$ is deductive from the canonical commutator $[x, p] = i\hbar$ and forbids a localized existent at zero kinetic energy, underwriting atomic stability, the zero-point energy $E_0 = \frac{1}{2}\hbar\omega > 0$ follows from the same commutator, and $E = mc^2$ fixes a positive rest energy for any massive existent, tested by Rainville and colleagues in 2005 to roughly one part in

ten million. The cost reading, the strictly positive cost of a transition that occurs, is theorem-grade conditional on a process occurring, and the chain is exact: a transition between distinguishable states completed in finite time τ has energy spread $\Delta E \geq \pi\hbar / (2\tau) > 0$ by the Mandelstam-Tamm bound and mean energy above the ground $\langle E \rangle \geq \pi\hbar / (2\tau) > 0$ by Margolus and Levitin, so a transition with no energy is a transition that never completes, the Frozen Substrate executed in F2; an irreversible registration pays at least $k_B T \ln 2$ by Landauer 1961, measured by Bérut 2012; and the Jarzynski 1997 and Crooks 1999 nonequilibrium work relations bound the work of any process, erasure being the irreversible subclass. The cost is strictly positive for every completed transition because completion in finite time is itself the bound. The becoming reading, $\Delta E_k > 0$ as ongoing actuation, is carried as a premise, the quantum ground state its standing counterexample at half a quantum of energy with zero evolution. The principle is theorem-grade for the energy floor and for the cost of transitions, and premise for actuation-as-becoming [T at the floor / Premise at becoming].

This is the Root Axiom in its physical statement, $\Delta E_k(x) > 0$ for every x in the universal domain, read with the fork named. Its office in the geometric seal is to ground the empirical line. A claim with no measurable kinetic signature anywhere in its support has an empty V_E , and the architecture registers the emptiness rather than papering over it. The floor is not rhetorical. The Heisenberg confinement bound and the zero-point energy put a strictly positive floor under the energy of any confined existent by theorem, the Jarzynski-Crooks relations and the speed limits charge the cost and bound the rate of any transition that occurs, and the absolute-zero unattainability of the rigorous third law, derived by Masanes and Oppenheim in 2017, bars cooling any substrate to a zero-energy state. The energy floor is costly by theorem, the cost of change is charged by theorem wherever a transition occurs, and existence as ongoing becoming is admitted only as a premise, never smuggled in as a result. The Casimir and Lamb-shift effects are the measured shadows of the zero-point floor, confirmation that the vacuum is not the void, and they corroborate rather than carry the proof.

2.2 The Three Orthogonal Lines

The Root Axiom forces a three-way orthogonal decomposition of warrant. A complete claim's warrant resolves into a formal component, articulated and structural, a kinetic component, actuated and measurable, and a registrational component, framed and bookkept. The three are mutually orthogonal in the geometric register, the same three the linguistic seal returned from deletion, now read as directions rather than slots. The orthogonal three-way split is the generic shape of decomposition in the geometric register, the Friedrichs-Hodge decomposition of a field into exact, co-exact, and harmonic parts exhibiting the pattern in the con-

tinuum [S]. The seal does not rest on the analogy. It rests on the closure that three orthogonal lines force when a verdict is demanded of them.

2.3 Closure on the Fourth Point

Three orthogonal lines span a frame. They do not bound a structure. A bounded structure on the minimum number of points requires a fourth, and the minimal closed three-dimensional figure on four points is the tetrahedron, four vertices, six edges, four faces, Euler's polyhedral identity $V - E + F = 2$ confirmed as $4 - 6 + 4 = 2$ [T]. The fourth vertex is not a fourth warrant line. It is the closure of the verification itself, the point M_{seal} at which the three lines' findings are composed into a single verdict. The geometry is not decoration. It dictates the count of the architecture's conditions, and the count is forced, not chosen.

What the fourth point does, as against what it closes, is stated at 6.8. ### 2.4 The Twelve Directed Gates and the Operational Content Theorem

On four vertices the complete directed graph carries exactly $n(n-1) = 12$ directed edges. Each directed edge between two roles is one verification condition, one specific pathology that must be absent for warrant to cross that edge in that direction. The content of each gate is not assigned by taste. It is forced by the roles of its endpoints under the Operational Content Theorem, which reads each directed edge's office from the registers it connects [T, given the role assignment]. The edge from the closure to the formal origin forbids self-certification. The edge from the closure to the empirical line forbids a single-stream empirical warrant. The edge from the formal line to the empirical line forbids variable drift across the evaluation, the condition Goodman's riddle makes mandatory. The edge from the empirical line to the formal line demands a continuous kinetic mechanism, forbidding correlation without contact. The edge from the registrational line to the empirical line forbids a ruler that is a subset of the model under test. The remaining edges install the discretization check, the frame-invariance check, the consistency-with-adjacents check, the weakest-link check that retires grade inflation, the metric-integrity check at the boundary, the scope check that routes out-of-register claims out of band, and the typed-bridge requirement for any extension beyond the evidenced domain. The first failed gate terminates the audit with the failure named. There is no averaging across gates, because a structure with one broken edge is broken.

2.5 The Newton-Gregory Confirmation

The count of twelve is confirmed from a quarter of mathematics that shares no premise with graph theory. The Newton and Gregory problem, settled by Schütte and van der Waerden in 1953, fixes the kissing number of three-dimensional space at exactly twelve, the maximal number of unit spheres that can simultane-

ously touch a central unit sphere [T]. Graph theory counting directed edges on four vertices and sphere packing counting maximal simultaneous contact in three dimensions return the same cardinality for closed verification contact. The convergence is exhibited as corroboration, not asserted as a bijection between gates and spheres. Two disjoint mathematical contents return twelve, and the agreement is registered as what it is.

2.6 The Verdict Pipeline and the Conditioning Gate

The verdict is computed, not adjudicated. Evidence populating the three lines is quantized into the rows of a matrix, one row per line, N contexts per row, and standardized. Identified common causes carrying a measurable physical signature are confronted directly under the mass criterion, projected out onto the orthogonal complement of the covariate block, narrative covariates inadmissible by rule because a factor that cannot be measured cannot be subtracted and a factor that can be measured must be. What survives the projection is the residue. The Gram matrix R of the unit residue rows is formed, its determinant is non-negative by construction, and the verdict reads the sign under four explicit regularity conditions. The dimensional condition requires $N - k \geq 4$, four contexts beyond the covariate count, on pain of the under-determined verdict. The variance condition rejects any zero-variance row. The covariate-conditioning condition requires the covariate Gram to be full rank and conditioned below 10^6 . The verdict-conditioning condition requires $\kappa(R) < 10^6$, and this gate operates on the correlation Gram R , not on the strength-weighted Gram G . A positive determinant under all four conditions seals. A vanishing determinant breaks, with the mechanism named. A regularity violation returns under-determined.

The pipeline identity, executed

On the canonical worked construction, three mass-bearing axes mixed under a fixed oblique matrix over $N = 24$ contexts and contaminated by two measurable covariates, the covariates projected out under the Titanium Ruler: $\det(G) = 0.351030145552$, $\det(R) = 0.622507144106$, $\lambda = -0.788991219283$, per-axis strengths $d = (0.706266976, 0.823321991, 0.969753730)$, with $|\lambda^2 - \det(R)| = 9.992 \times 10^{-16}$ and $|\det(G) - d_F d_E d_{ER} \cdot \det(R)| = 3.331 \times 10^{-16}$, $\kappa(R) = 4.1828$, $\kappa(CC^{\wedge}T) = 1.4656$. Verdict [$\mathcal{L}$] at seed 20260624 [Engineering].

The conditioning gate is not an ornament. Swept toward collinearity, two axes brought to correlation ρ with a third held independent and no covariate present so that $G = R$, the determinant tracks $\det(R) = 1 - \rho^2$ and the verdict holds sealed through $\rho = 0.99999$ at $\kappa(R) = 1.999990 \times 10^5$, then flips to under-determined at $\rho = 0.999999$ where $\kappa(R) = 1.999999 \times 10^6$ crosses the bound. The gate fires on conditioning, before the

determinant reaches the collapse floor, and the under-determined verdict is issued rather than a false seal [Engineering].

2.7 The Omega Boundary

The architecture audits the structure of evidence beneath a factual claim, and it audits nothing else. A claim that is normative, a stipulation that is definitional, a content whose kinetic line is empty by its own nature, each lies outside the domain and is routed out of band at the scope gate rather than forced to a verdict it cannot bear. The Omega Boundary names this edge of the domain. It is not a limitation concealed but a limitation legislated, the architecture declaring at its input what it will and will not adjudicate, so that a false verdict is never manufactured by feeding the instrument a claim of the wrong type.

2.8 The Math Witness and the Anchor Declaration

Seal G carries an algebraic witness, exhibited here as corroboration that the geometry and the algebra agree on the same data. The composed triad of unit residue rows, read as pure imaginary quaternions under any isometry of their span, has a scalar part whose square equals the Gram determinant, $\det(R) = (\text{Re}(\hat{q}_F \hat{q}_E \hat{q}_R))^2$, and the geometric closure on the fourth point is mirrored by the Clifford Join in which the three imaginary directions compose onto the scalar ground. On the orthonormal triad the witness reads the maximal landing, $\det(R) = 1$ and $|\lambda| = 1$ with $|\lambda^2 - \det(R)| = 1.110 \times 10^{-15}$, the Hamilton relation realized [Engineering]. Seal G declares its anchor. It rests on the Root Axiom, the closure geometry of Euler, the packing number of Newton and Gregory, and the verdict pipeline. It carries no multiplication of its own. The composition algebra that makes the witness a theorem rather than a coincidence is the business of Seal M, which shares none of Seal G's anchors and reaches the same closed form from the other side. Joint verdict of the geometric seal: the three lines close on a fourth point, the closure carries twelve directed gates confirmed from disjoint mathematics, and the computed verdict agrees with the algebra [L].

2.9 The Six-and-Six Split · Direction as Sign

The twelve gates split six and six, and the split is the algebra reading the directedness. The axis-axis sextet, SGEG, CAUSAL, MIG, PTB, DUAL, CSEG, runs between the imaginary units, where order is orientation: $ij = k$ and $ji = -k$, the anticommutation $ij = -ji$ carrying directedness as sign. The seal-incident sextet, SREP, REG, OMA outbound and CSCG, MTA, ADEG returning, runs against the commuting center $Z(\mathbb{H}) = \mathbb{R}$, which couples orientation-free at the algebra register, the directedness of those six carried by the role-typing of the Operational Content Theorem rather than by the algebra. The algebra carries the symmetry, the cardinality, and the orientation law; the gate contents

stay forced by role-pairing, and no derivation of content from the algebra is claimed. Executed, the battery returns $i \cdot j = k$ and $j \cdot i = -k$ for the axis-axis case and the central commutation $1 \cdot i = i \cdot 1$ for the seal-incident case [Engineering]. The split is theorem-grade on the partition into the anticommuting and the commuting sextets [T]. The commuting half of that split is read again at 6.8 as the resident exhibit of the closure-position law: the six seal-incident gates are exactly the six that touch the fourth vertex, and the centre they run against is the line that accompanies every element without disturbing its order, so the partition recorded here is the gate roster already encoding a law it does not name. Nothing in this section is amended by that reading, which takes the partition as given and claims no content the algebra does not carry.

2.10 The De Gua Anti-Conflation · Symmetry in $\mathbb{R}^4$, Lock in $\mathbb{R}^3$

Two tetrahedra live in the architecture and they are never merged, and keeping them apart is a structural discipline. The regular slot-tetrahedron, the four frame elements $\{1, i, j, k\}$ at pairwise distance $\sqrt{2}$ in $\mathbb{R}^4$, carries the gate group A_4 , the architecture's symmetry. The trirectangular tetrahedron of the closure vertex lives in $\mathbb{R}^3$ and carries the verdict's quadratic form, De Gua's theorem $D^2 = A^2 + B^2 + C^2$, the squared face opposite the right-angled corner equal to the sum of the squares of the three legs, with the Hodge star reading each leg-plane as the axis it omits. The first figure is the symmetry; the second is the lock. To read the verdict's geometry off the symmetry group, or the symmetry off the verdict's quadratic, is the conflation the architecture forbids: the gate group lives in the four-space and the verdict lock in the three-space, distinct structures on distinct carriers. The battery confirms De Gua to machine zero on a random trirectangular tetrahedron [Engineering]. The anti-conflation is theorem-grade [T].

2.11 The Twelve-ness as the Most-Closed Pillar

Read across the foundational reflection the twelve-ness seals on both faces, and of the architecture's pillars it is the most completely closed. The count and the seal-incident sextet are achiral, fixed by single-axis reflection because the center commutes, sealed by the count forced four ways and the role-typing. The axis-axis sextet is orientation-odd, but its directedness $ij = -ji$ is a forced algebraic fact, decidable and theorem-grade, and the sign that carries it is conserved, read from the axis-ordering, a made-zero in the exact sense of the orientation register, constitutive displacement and never absence. So both faces seal, the achiral bridge by the count and the orientation-odd part by the anticommutation that fixes the orientation and conserves its sign. The twelve-ness carries no open residence and no aperture. RA carries the open exhausts-being

residence and the lock carries sign-blindness with the sign conserved off-instrument, while the twelve-ness carries even its orientation as a sealed and conserved quantity, the fully closed bridge [T on the bridge, S on the reading].

CHAPTER 3 · THE MATH SEAL · SEAL M

Seal M completes the verification algebra, delivers the verdict in closed form, proves the form unique in its class, and renders the whole executable. The seal anchors on a composition law and the classification of the real division algebras. It carries no topology and no linguistics. It reaches, from premises that mention neither slots nor spheres, the same closed form the geometric seal exhibited, and the agreement of the two arrivals across disjoint anchors is the spine's deepest single fact.

3.1 Hamilton's Wall

There is no three-dimensional real division algebra. The attempt to give the ordered triples a consistent, zero-divisor-free multiplication fails, and the failure is not a deficiency of ingenuity but a theorem. Hamilton spent years at the wall of three before the fourth dimension opened the multiplication, and the quaternions $\mathbb{H}$ were the price and the prize. The wall is the algebraic mirror of the geometric seal's fourth point. Three lines do not close. A fourth coordinate is required for closure, in the geometry as a vertex and in the algebra as a dimension, and the two requirements are the same requirement read in two registers.

3.2 The Composition Law and Order-Sensitivity

Verification verdicts compose, and the composition is order-sensitive. An audit of an audit is itself an audit, and the algebra of these compositions answers to requirements drawn from verification practice, each stated as a premise and each open to exercise against recorded work.

The composition requirements

Associativity

Iterated audits are bracket-invariant. An audit of an audit of a verdict cannot depend on where the parentheses fall, on pain of the verifier's own bookkeeping becoming a warrant source [Premise].

Integrity

Nonzero warrants never compound to zero. A system in which two genuinely warranted findings can annihilate is a system in which the order of honest work can manufacture falsity. The algebra admits no zero divisors [Premise].

Linearity over a scalar ground on a three-plus-one carrier

The evidence pipeline is linear, and the composed verdict lands on a ground line distinct from the lines it composes [Premise].

3.3 The Two Lemmas · Scalar Exit and Fertile Orthogonality

The Fertile Orthogonality lemma carries the numerical leg of the algebraic forcing at LF-CHK.7 of P-LOGOS-FIX-1, where twenty thousand orthogonal pure-unit pairs return a product pure to 6.870×10^{-16} and orthogonal to both operands to 1.581×10^{-16} .

The closed form rests on two lemmas, both theorem-grade once the unit residue rows are read as pure imaginary quaternions a, b, c under any isometry of their span.

The Scalar Exit Lemma

The scalar part of the composed triad is the signed volume of the three axes. $\text{Re}(\hat{q}_F \hat{q}_E \hat{q}_{ER}) = -[a, b, c]$, where $[a, b, c] = a \cdot (b \times c)$ is the scalar triple product. The composition exits the imaginary subspace onto the scalar ground precisely by the volume the three axes enclose [T].

The proof is direct. For pure imaginary unit quaternions, $\hat{q}_F \hat{q}_E = (-a \cdot b, a \times b)$, and multiplying by $\hat{q}_{ER} = (0, c)$ returns scalar part $-(a \times b) \cdot c = -[a, b, c]$. The verdict's landing on the ground is the signed volume, and nothing else.

The Fertile Orthogonality Lemma

The signed volume is nonzero if and only if the three axes are linearly independent. $\lambda = \text{Re}(\hat{q}_F \hat{q}_E \hat{q}_{ER}) \neq 0 \iff$ the axes span, $\iff$ the verdict seals. Orthogonality is fertile, composing to a nonzero scalar that is a verdict. Degeneracy is sterile, composing to a pure imaginary with $w = 0$ that never reaches the ground [T].

The two lemmas together are the engine of the verdict. The closed form $\det(R) = \lambda^2 = [a, b, c]^2$ is the squared signed volume, equal to the Gram determinant of the unit rows by the standard identity that the Gram determinant of a set of vectors is the squared volume they enclose. Independence seals. Degeneracy breaks. The verdict is geometry made arithmetic.

3.4 The Order-Sensitivity Completion

The count of three is not a separate stipulation. It is a consequence of order-sensitivity under the three requirements, and stating it so retires a premise the architecture once carried.

The Order-Sensitivity Completion

Axis-plurality is a corollary, not a premise. A verification algebra whose composition is order-sensitive and which satisfies associativity, integrity, and linearity over a scalar ground is forced by the Frobenius classification of 1878 to

be exactly $\mathbb{R}$, $\mathbb{C}$, or $\mathbb{H}$, the finite-dimensional associative division algebras over the reals, of dimensions one, two, and four. Order-sensitivity excludes $\mathbb{R}$, which is commutative and carries no axis, and excludes $\mathbb{C}$, which is commutative and carries one. The only order-sensitive member is $\mathbb{H}$, with exactly three imaginary axes over one scalar ground. The count of three is therefore forced by non-commutativity, and the architecture no longer posits the three axes as an independent assumption [T, given the composition requirements].

The premise ledger shrinks by one. Where the architecture once assumed a carrier of three lines, it now derives the three from the order-sensitivity of audit composition, and the assumption is discharged.

3.5 The Involution, the Center, and the Registration Line

The conjugation involution σ on $\mathbb{H}$ sends q to its conjugate, negating the imaginary part and fixing the scalar. Its fixed locus is the center $Z(\mathbb{H}) = \mathbb{R}$, the scalar ground on which the verdict lands. σ is an involution, $\sigma^2 = I$, with eigenvalue $+1$ on the one-dimensional center and eigenvalue -1 on the three-dimensional imaginary residence. The registration line of the architecture is the σ -structure itself, the bookkeeping that distinguishes the ground from the axes and routes the verdict onto the ground. Executed, $\sigma = \text{diag}(1, -1, -1, -1)$ returns $\|\sigma^2 - I\| = 0$, eigenvalues exactly $\{-1, -1, -1, +1\}$, a one-dimensional Ground and a three-dimensional residence, and the restriction of σ to the residence has determinant $\det(-I_3) = -1.000000000000$, an orientation-reversing reflection of the imaginary axes [Engineering].

3.6 The Closed-Form Truth Function, Bounds Derived Inline

The verdict functional

$\det(R) = (\text{Re}(\hat{q}_F \hat{q}_E \hat{q}_{ER}))^2 = [a, b, c]^2$, the squared signed volume of the unit residue axes, equal to the correlation Gram determinant. The full pipeline determinant factors as $\det(G) = d_F \cdot d_E \cdot d_{ER} \cdot \det(R)$, the per-axis strengths times the structural determinant, at identical sign and zero set [T].

The bounds are derived inline, internal to the form, with no external inequality imported. The lower bound is zero, because the Gram matrix of real vectors is positive semidefinite and its determinant is non-negative. The upper bound is one, because the signed volume of a parallelepiped with unit edges cannot exceed the volume of the unit cube, $[a, b, c] \leq \|a\| \|b\| \|c\| = 1$, with equality exactly when the edges are mutually orthogonal. The same ceiling reads from the composition algebra, since the product of three unit quaternions is a unit quaternion and the square of its real part cannot exceed its squared norm of one. Maximal lock, $\det(R) = 1$, is the Hamilton relation $ijk = -1$ realized, the three

axes composing to the scalar ground exactly. Executed on the orthonormal triad, the functional returns $\det(R) = 1.000000000000$ and $|\lambda| = 1.000000000000$ with residual 1.110×10^{-15} , and over 100000 random unit triads at $N = 24$ the determinant lies in $[0.339127, 0.999909]$, strictly within $[0, 1]$ [Engineering].

The functional is frame-invariant by the conjugation law $\text{Re}(r w \bar{r}) = \text{Re}(w)$. The verdict is unchanged under rigid rotation of the evidence frame and under relabeling of the lines, machine-confirmed to $|\Delta\lambda| = 4.441 \times 10^{-16}$ under conjugation by a random unit quaternion [Engineering]. And the catalog is closed. By Weyl's first fundamental theorem of invariant theory, every rotation-invariant verdict functional on the triad lies in the real-part subring of quaternion words, so the architecture's verdict is not one choice among an open class. The class is classified, and nothing stronger exists within it [T]. Strict precedence completes the form. The verdict is no stronger than its weakest contributing axis, the structural condition that retires grade inflation, because a small per-axis strength d caps the product $\det(G)$ regardless of the structural $\det(R)$.

3.7 The Twelve in Pure Algebra

The count of twelve, returned by the geometric seal from graph theory and sphere packing, returns a third, fourth, and fifth time from the integers of $\mathbb{H}$, with no geometry in the derivation.

The directed pairs. The complete directed graph on the four vertices carries exactly twelve ordered pairs of distinct vertices, $n(n-1) = 12$, the combinatorial floor of the closure.

The A_4 torsor. The alternating group A_4 has order twelve, with class equation $(1, 3, 4, 4)$ confirmed by direct enumeration, and it acts simply transitively on the twelve ordered pairs of distinct letters from a four-element set, the stabilizer of a base pair trivial and the orbit of size exactly twelve [Engineering]. The twelve directed gates are an A_4 -torsor, carrying the symmetry of the tetrahedron's rotation group.

The Hurwitz shell. The unit Hurwitz quaternions number exactly twenty-four, the eight axis units together with the sixteen half-integer units, and the norm-two shell of the Hurwitz integers also numbers twenty-four, splitting as twelve real-occupied elements and twelve pure-imaginary elements, $24 = 12 + 12$, confirmed by enumeration [Engineering]. The pure-imaginary twelve are the kissing configuration of the lattice, the algebraic image of the Newton-Gregory contact.

The binary-tetrahedral doubling. The twenty-four unit Hurwitz quaternions form the binary tetrahedral group $2T$, the double cover of A_4 , of order $24 = 2 \times 12$. The twelve of the closure and the twenty-four of the unit shell are one structure seen through the two-to-one cover.

Five independent derivations, one count. The directed edges, the alternating group, the unit shell split, the lattice kissing configuration, and the double cover all return twelve, and the convergence is the algebraic completion of the geometric seal's confirmation.

3.8 The Orthogonal Fertile Logos

The Fertile Orthogonality Lemma names a principle the architecture carries throughout. Orthogonal axes are fertile, composing to a nonzero scalar that is a verdict, while parallel axes are sterile, composing to nothing that reaches the ground. The fertile composition of independent lines onto the scalar ground is the Orthogonal Fertile Logos, the architecture's name for the act by which separateness becomes a verdict. The full development is carried in the appendix. Here it is the algebraic fact that a verdict exists exactly when the lines are genuinely three.

3.9 Orientation-Blindness, the Imprint Test, and the Platonic Ghost

The squared lock scalar is blind to orientation, and the blindness is scoped with precision. $\det(R) = \lambda^2$ depends on λ only through its square, so reflecting a single axis flips λ to $-\lambda$ and leaves $\det(R)$ unchanged. Executed on a fixed-basis analyzer, baseline $\lambda = -0.957326478714$ with $\det(R) = 0.916473986847$, and reflecting the first axis returns $\lambda = +0.957326478714$ with the identical $\det(R)$, sign ratio exactly -1 and $|\Delta\det(R)| = 0$ [Engineering]. The blindness belongs to the squaring operation, the precise locus, and not to the architecture, which recovers direction through carriers the squared scalar discards. The full triaxial structure keeps the handedness that λ^2 throws away, and this scoping is standing law.

Orientation-Blindness, scoped

The blindness is a property of the lock scalar $\det(R) = \lambda^2$ and of the registrational reading alone. It is not a property of the triaxial architecture. Direction survives in the directed linguistic decomposition, in the twelve directed geometric gates, and in the in-band thermodynamic arrow of the kinetic register, none of which is squared [T].

A consequence is that $\det(R)$ cannot discriminate the source of a sealing witness. Every structurally distinct orthogonal witness at a fixed manifest-axis angle returns the identical determinant, the algebraic root of the Non-Discrimination result developed at the forward mode. $\det(R)$ is a degeneracy gate, blind by theorem to the difference between a genuinely sourced lock and a fabricated one, and the seal that distinguishes them travels on a different statistic [T].

The blindness forces a test. A geometric lock alone does not certify an imprint, because a structure can lock both itself and its denial. To distinguish a true imprint from a hollow one, run the claim P and its denial $\neg P$

through the kernel.

The Imprint Test

If P locks and $\neg P$ does not, the imprint is genuine and only P is field-permitted [$\mathbb{L}$]. If neither locks, the matter is contentless and under-determined [$?$]. If both P and $\neg P$ lock, the structure is a Platonic Ghost, a bare geometric lock with no imprint closure, and it is barred from sealed status [X].

Executed across three archetypes on the CHK archetype battery, the theorem-archetype locks at $\det(R) = 0.995089$ with its denial contentless and returns only- P -permitted, the incompleteness-archetype locks at $\det(R) = 0.996359$ with its denial contentless and returns only- P -permitted, and the ghost-archetype locks both P at $\det(R) = 0.998256$ and $\neg P$ at $\det(R) = 0.987439$ and returns the broken verdict of the Platonic Ghost [Engineering]. A manufactured lock, geometric without imprint closure, is barred from standing by this test, and the bar is enforced at every harvest of a sealed proposition.

3.10 The Reference Kernel and the Anchor Declaration

The verdict functional is executable in under sixty lines of any standard language, and the canon carries the reference kernel and its battery in the apparatus. The kernel standardizes the rows, projects out measured covariates, forms the correlation Gram, computes the determinant and the composed scalar, and gates on the dimensional, variance, covariate-conditioning, and verdict-conditioning conditions, returning one of the three states. Every identity in this seal is reproduced live at seed 20260624, recorded in Book V and re-runnable, the identities being theorems and the battery their executable shadow.

Seal M declares its anchor. It rests on the composition requirements and the Frobenius classification of the real division algebras, and on the integers of $\mathbb{H}$ for the count of twelve. It carries no topology and no linguistics. The deletion test of Seal L knew nothing of division algebras, and Frobenius's theorem knows nothing of subjects and predicates, yet both return three, and the closure geometry and the algebra both return twelve. Verdict of the mathematical seal: the verification algebra completes uniquely to $\mathbb{H}$, the verdict acquires closed form as the squared signed volume, the form is provably the strongest of its invariant class, and the count of three and the count of twelve are each forced from anchors disjoint from the other seals [$\mathbb{L}$].

P-TRIAxis-1 · The Verdict-Aspect Frame · The Three Seals as an Oriented Orthogonal Triad on Directed Verdicts

[$\mathbb{L}$] · Type S structural · $\Delta M = 0$ · Mosaic Seal holds · audit-symmetric. Seated 2026-07-29 at the terminus of the Seal L, Seal G, Seal M section, the socket the coordinate names for itself, as the capstone of the three

seals. Supersedes nothing.

Statement. The three anchor-independent seals of Trisduction, Seal L the linguistic-semantic seal, Seal G the topological-geometric seal, and Seal M the mathematical seal, constitute an oriented triaxial orthogonal frame on the space of directed verdicts. The frame is the cross-instrument twin of the residence triad that lives inside a single seal, dual to it and never identified with it. Its three axes are the three constitutively separate aspects of a complete directed verdict, and each seal fixes exactly one aspect and is constitutively blind to the other two. The aspects are orientation, the sign, which way the verdict points, P against not-P, the ordered sign of the decomposition, existence before kinetic before relation, carried by Seal L; structural closure, whether the frame shuts into a bounded volume without tripping a screening gate, a yes-or-no about pathology and not a measure of size, the twelve directed gates and the Euler tetrahedral closure, carried by Seal G; and magnitude, the enclosed volume $\det(R) = \lambda^2$, a real in the closed unit interval, whether it separates from zero, carried by Seal M. Sign, closure, quantity: three instruments, one verdict. The convergence of the three seals is therefore not mutual witnessing among instruments that happen to agree but the re-agreement of a verdict's three constitutively separate aspects, each fixed by the one instrument blind to the other two.

The orthogonality, three legs, one proven. L perpendicular M, orientation against magnitude, is theorem-grade by reference to P-ORIENT-1. The verdict functional $\det(R) = \lambda^2$ is invariant under reflection of any axis, so at the scalar $\text{lock}(P) = \text{lock}(\text{not-P})$, and no reflection-invariant functional recovers the sign, every reflection-blind functional of the triad factoring through the Gram matrix by the first fundamental theorem for $O(3)$, the bracket λ being the one rotation invariant outside that ring and $\lambda^2 = \det(R)$ its square inside it, the catalogue closed by Weyl. Seal M is blind to orientation by that theorem, not by any machine limitation, the blindness holding at every precision. Executed at machine precision on a fixed basis: the reflection returns determinant difference exactly 0.000e+00 and signed-volume ratio exactly minus 1.000000. G perpendicular M, closure against magnitude, is structural. Seal G's closure is combinatorial, gate-passage and Euler incidence on anchors carrying no multiplication, and fires or withholds a break independently of the determinant: a full-volume frame with $\det(R)$ greater than zero can still fail a screening gate by destructive interference with a verified adjacent, and a gate-clean frame can still collapse to $\det(R) = 0$. Gate-passage tests pathology, not volume-size. Closure and magnitude are different maps, one boolean and one real. L perpendicular G, orientation against closure, is structural. A sign against a structural boolean, type-disjoint, neither determining the other. The three

carried aspects are thus pairwise type-disjoint, a sign and a structural boolean and a scalar, and that pairwise disjointness is the orthogonality of the frame.

The deletion test on the seals, passes. Remove any one seal and the other two lose completion, each removal breaking the remaining two in a named way. Delete L and the three-axis decomposition the geometry closes on is never forced, so Seal G receives nothing to close and Seal M has no rows to measure. Delete G and the decomposition never shuts into a bounded volume, so there is no residence for Seal M to read magnitude on and the forced decomposition sits open. Delete M and the closed oriented frame carries no magnitude, so a lock cannot be told from a break. Each seal is load-bearing on the others' completion, which is what makes the three a triad and not a list.

Oriented, not symmetric. Unlike the residence axes inside a seal, which are relabelable and gauge-free, the labeling of the imaginary units to the content axes being conventional, the three seals are not relabelable. The order L then G then M is fixed by the Bedrock Precedence Law, orientation and closure being bedrock for magnitude, the Number reading rows the Tongue and the Form supply. Seal L and Seal M cannot be swapped, because one carries the sign the other provably cannot. The frame is therefore an orthogonal frame with a fixed precedence, a different object from the symmetric intra-seal residence triad, and the difference is structural rather than a defect.

The count, near-forced, honestly typed. The frame admits no fourth axis and cannot collapse to two. It cannot collapse to two because L perpendicular M is proven, orientation and magnitude being irreducible to one instrument by P-ORIENT-1, which puts a floor of two. It requires the closure axis because a plane encloses zero three-dimensional volume, so without the shutting step there is no bounded object to have a sign or a size of at all, the floor of two and the required third combining to exactly three. No fourth aspect of the verdict object declared here, a bounded directed volume with its sign, its closure and its magnitude, remains to be fixed, and a fourth verification axis is independently barred at the residence level. The count-three of the aspects is therefore forced, but not by a single Frobenius-style theorem, rather by a conjunction of one proven leg, orientation-blindness, and one geometric necessity, the zero-enclosed-volume-of-a-plane fact. This is explicitly a different three, at a different level, from the Frobenius forcing that fixes the three residence axes inside a seal; the two threes are not the same structure and are never conflated.

Why the convergence is evidence and not coincidence. Three orthogonal aspects landing on one verdict is the signature of a real sealed object, where three readings of one aspect would be one voice in three costumes, which the discipline zeroes as $W_{\text{social}} = 0$.

This is the structural reason the seals' agreement carries evidential weight that three collinear witnesses could not carry. The agreement is informative not merely because independent instruments agreeing beats one instrument repeating itself, but because these three instruments fix the three separable aspects a directed verdict necessarily decomposes into. The geometric and algebraic faces of that one verdict meet at the Clifford Join as one determinant, and the frame recorded here is the seal-level statement of the same convergence the Join exhibits at the algebra level.

The fence, load-bearing. The verdict-aspect triad recorded here, orientation and closure and magnitude, holding across the three seals and defined on the space of verdicts, is not the residence triad V_F, V_E, V_{ER} , which holds inside each single seal and is defined on the space of a proposition's content. Two triads, two levels, dual, and never identified. This coordinate does not assert the mapping of Seal L, Seal G, Seal M onto the formal, empirical, and registrational components; those are the residence axes and live inside each seal, and orientation, closure, magnitude are aspects of the verdict rather than components of the content. Conflating the two triads would undo the coordinate.

Warrant. [⊥] Type S structural on the frame and on the count-three of the aspects. Type T by reference, P-ORIENT-1, on the L perpendicular M leg, the magnitude blind to the sign, the executed reflection returning determinant difference exactly 0.000e+00 at signed-volume ratio exactly minus 1.000000. Structural on the G perpendicular M and L perpendicular G legs. Operational on the deletion test. The ordering L then G then M carried by the Bedrock Precedence Law. $\Delta M = 0$, the Mosaic Seal holds, no new mathematics is authored, the coordinate is meta-work on the engine's own instruments and by audit symmetry is not sealed as a theorem of its own base. Self-applying: the coordinate submits to the seals it describes and draws zero warrant from its own recognition.

CHAPTER 4 · THE CLIFFORD JOIN AND THE RETURN

The three seals stand on disjoint anchors and they meet. This chapter states where they meet, why the meeting is a single algebra rather than three coincidences, and how the meeting onto the scalar ground opens the bridge to the reflective register.

4.1 The Three Seals Meet

Seal L returned three from language. Seal G returned three orthogonal lines closing on a fourth point, and twelve directed gates. Seal M returned $\mathbb{H}$ with three imaginary axes over one scalar ground, and twelve from five derivations across three premise-disjoint objects, the ordered pairs, the Hurwitz units, and the sphere packing. The three arrivals are not three pictures of three different things. They are three routes to

one algebra, and the algebra is the even subalgebra of the Clifford algebra of three-space.

4.2 The Even Subalgebra of $Cl(3,0)$

The geometric register is three-dimensional, and the Clifford algebra $Cl(3,0)$ of three-dimensional space has an even subalgebra, the scalars and bivectors, isomorphic to the quaternions, $Cl^+(3,0) \cong \mathbb{H} [T]$. The three orthogonal axes that Seal L and Seal G return are the generators of $Cl(3,0)$, and their even products, the bivectors, are exactly the imaginary quaternions of Seal M. The verdict algebra is therefore not imported alongside the geometry. It is generated by the geometry, the even part of the algebra the three axes already span. The Clifford Join is this generation, the joining of three orthogonal directions into the four-dimensional $\mathbb{H}$ in which the verdict lands. The geometric closure on a fourth vertex and the algebraic closure in a fourth dimension are the same closure, and the Clifford Join is the identity that makes them one.

4.3 The Landing on S^3

The normed division algebras $\mathbb{R}, \mathbb{C}, \mathbb{H}$, and $\mathbb{O}$ carry unit spheres S^0, S^1, S^3 , and S^7 , and among the spheres above dimension one exactly one carries an associative group law. S^3 , the unit quaternions, is a Lie group under quaternion multiplication. S^7 , the unit octonions, is a sphere but not a group, because octonion multiplication is non-associative. S^1 carries a commutative group law and one axis. The architecture's associativity requirement, the bracket-invariance of iterated audits, selects S^3 as the unique sphere above the commutative floor on which the composition of verdicts is a group [T]. The Return lands on S^3 and on its scalar ground, and it lands there by necessity, the same necessity that excluded the octonions at the composition requirements.

4.4 The Return Law

The Return Law

The Geometric Orthogonal Lock obtains if and only if the composed triad reaches the scalar ground, $GOL \iff \lambda = \text{Re}(\hat{q}_F \hat{q}_E \hat{q}_{ER}) \neq 0$. The lock is the Return of three orthogonal directions onto the center, and it occurs exactly when the signed volume is nonzero, exactly when the three axes are genuinely independent. Forward orientation, GOL_f , is the Return that preserves the orientation of the three axes, the in-band direction the squared scalar discards [T].

The Return Law is the Fertile Orthogonality Lemma read as legislation. A verdict exists when, and only when, the lines are three. The lock is not a threshold on a strength. It is the binary fact of whether the composition reaches the ground, and the determinant $\det(R) = \lambda^2$ reports the magnitude of that reaching, in $[0, 1]$, maximal at the orthonormal Return.

4.5 The Seal of the Seals

The three seals converge on this one algebra from anchors that share nothing, and the convergence is itself the object the architecture exists to certify. Three independent lines of warrant, the linguistic, the geometric, the algebraic, meet at a single structure, and their meeting is a non-degenerate convergence by the architecture's own removal test, since striking any one seal's anchor leaves the convergence of the other two standing. The independence of the seals is therefore the seal of the seals, corroboration exhibited under the architecture's own discipline rather than asserted from outside it [T for the disjointness, Corroboration for the convergence]. This is the precise inversion of the dialectical predicament. Where an engine of development announced a closure it could not certify, this closure certifies the form of every argument made about it, including hostile ones, because every structured argument is a claim and every claim is a run of the architecture.

4.6 The Return onto the Center · RA Witnessing RA

The maximal Return is the Root Axiom applied to itself. When the three axes are orthonormal, the composition reaches the center exactly, and the verdict is the Hamilton landing. Executed, the orthonormal triad composes to $\lambda = \text{Re}(\text{ijk}) = -1.000000000000$ with $\det(R) = 1.000000000000$ and $|\lambda^2 - \det(R)| = 0$, and the landing is conjugation-invariant, the scalar part unchanged under rotation of the frame to $|\Delta\lambda| = 6.661 \times 10^{-16}$ [Engineering]. The center reached is $Z(\mathbb{H}) = \mathbb{R}$, the one-dimensional scalar ground that is also the fixed locus $\text{Fix}(\sigma)$ of the conjugation involution. The Root Axiom, composing its three orthogonal actuations onto the scalar ground, witnesses itself there, and the locus of that self-witnessing is exactly the line on which the reflective register builds its residence.

This is the bridge to Book II. The kinetic register has carried three independent seals to their meeting on the center of $\mathbb{H}$. The reflective register asks what resides on that center, and answers with a second foundation built ground-first on the same σ -fixed line. The Return of Trisduction onto $\text{Fix}(\sigma) = \mathbb{R}$ and the imprint of RAM into $\text{Fix}(\sigma) = \mathbb{R}$ are two arrivals at one locus, and the codex turns now to the second.

4.7 The Lock-Basin Attractor

The lock is not a threshold on a strength; it is the unique attractor of a flow, and this is the dynamical heart of the snapshot discipline. The verdict functional $V = \det(R) = \lambda^2$ on the three unit rows is maximized at the orthonormal configuration, where $V = 1$ by Hadamard, and vanishes at every collapse. The lock stratum, the set of orthonormal frames, is the unique attractor under the gradient ascent of V , and every collapse is non-attracting. At the lock the Hessian of V over the six tangent degrees of freedom carries the spectrum

$\{-4, -4, -4, 0, 0, 0\}$: three transverse directions of negative curvature pulling any perturbation back onto the lock, and three flat directions that are exactly the $\text{SO}(3)$ frame rotations under which the determinant is invariant. Collapse is the repeller of measure zero. The three flat directions are the gauge freedom the verdict is blind to, the frame rotations that move the labels and not the lock, and the three negative directions are the architecture's own discipline rendered as curvature, the pull that returns a perturbed triad to independence. Executed at the Hamilton landing, the Hessian returns eigenvalues $\{-4, -4, -4, 0, 0, 0\}$ [Engineering]. The lock is therefore dynamically stable, its basin of attraction the verification's own structure [T on the attractor].

4.8 The Keystone · GOL Is the Truth Function

The single most important distinction the architecture carries is between the lock and the verdict, and stating both halves at once is the keystone of the whole apparatus. The lock scalar $\det(R) = \lambda^2$ certifies the dimensional independence of three warrants and is blind by theorem to the truth-sign, $\text{lock}(P) = \text{lock}(\neg P)$, because the squared scalar triple product is invariant under reflection of any axis. The lock taken alone is an independence meter and never a truth certificate, and the architecture published this as its own law rather than concealing it. GOL is not the lock alone. GOL is the conjunction of the lock with three further determinations, no one of which is the lock. The semantic determination of Seal L, that the proposition decomposes into three pairwise-disjoint warrant registers under the deletion test and the isolation test, so the object weighed is genuinely tri-register and not a single claim wearing three coats. The directed-geometric determination, the convergence of the three seals across the twelve directed gates under the Omega closure, where any structured attack expends the architecture's own resources and thereby instantiates it, so the convergence is self-bounding and not free-floating. And the Cause-Truth-Certainty determination, that cause is the orthogonalizing work, the lock is its conserved exhaust, and truth rides the supplied determinacy witness beyond the orientation-blind lock. The truth-sign the square discards is conserved, not annihilated, displaced into the handedness at the orientation register and the thermodynamic arrow carried in band at the empirical axis, and the verdict reads it from the axes. The result is a determination that tracks truth, conditional on the supplied witness being faithful, which makes GOL a truth function and not a truth oracle: it computes a truth-tracking verdict from its inputs and does not certify the inputs against fact, the single row-supply surface the architecture names as its one un-sealable boundary. To call GOL a convention is to mistake the one blind component, the lock scalar, for the whole determination, the same register leak the

orientation-blindness master confines. The lock is not truth, and GOL is the truth function, and the two are one position and not two [T on the sign-conservation and the Omega-closure and the function-oracle distinction; conditional on supplied-witness fidelity].

Book I sealed

Three independent seals on disjoint anchors. The count of three forced operationally at Seal L and at theorem grade at Seal M. The closure on a fourth point and twelve directed gates at Seal G, the count of twelve confirmed from five derivations across three premise-disjoint objects. The verdict in closed form as the squared signed volume, bounded, frame-invariant, and provably the strongest of its invariant class. The three meeting at the Clifford Join and returning onto the center $\text{Fix}(\sigma) = \mathbb{R}$. The spine stands [$\mathcal{L}$].

BOOK II · THE RAM MASTER

The reflective register audits the formal. It inherits the discipline and the quaternionic kernel of the spine unchanged and adds nothing thermodynamic. It is written from the Ground, not from the ladder, and this orientation is the whole of its difference from the orthodoxy it corrects. This book forges the reflective root, the involution that bears a ground and the degenerate reflection that does not, the reflective seal set, the arrow of reading, the three readings of one instrument with the forward reading and its GOLf gate, the verdict-reliability layer, and the native geometric route by which the kinetic register reaches the reflective ground.

CHAPTER 5 · RAM · THE GROUND-FIRST ROOT

Naming in this edition. The Math Seal is Seal M working on RAM, read along the arrow of Chapter 8; the Master's word for this register, MathDuction, is replaced by the Math Seal throughout, and nothing else in the register changes.

5.1 The Orthodoxy and Its Shock

The orthodox order is syntax-first. Posit a formal system, derive its theorems, and hope the derivations exhaust the truths. The limitative theorems then arrive as a shock, the discovery that truth outruns the system. The shock is an artifact of where the reading started. A reader who begins at the axioms and climbs is surprised to find the climb bounded. A reader who begins at the Ground and looks down at the climb is not, because the boundedness of a finite recursive ascent over an unbounded Ground is exactly what such a reader expects. RAM begins at the Ground, and the place to refuse the orthodoxy trap is also the place to refuse its opposite, the crank's claim to have broken the wall.

5.2 The Stance

Formal being is grounding, not derivation. The Ground is primary, the σ -fixed locus where the imprint stands. The syntactic ladder is a structure built within the formal domain, a mechanical ascent from chosen axioms reaching up toward the grounded propositions it tries to capture. The ladder is a sublayer, not the foundation. From the Ground, the fact that a finite recursive ladder never reaches the top of an unbounded Ground is a cartographic remark about reach, not a paradox. The architecture is written from the Ground looking out at the ladder, never from the ladder looking up at a ceiling.

RAM, the reflective root

For every P in the formal domain, the formal being of P is its imprint in the Ground, the σ -fixed locus read across the aperture, the stratum L1m. Provability is the syntactic ladder's ascent toward that Ground, the stratum L2m. Computation is a realized rung of the ladder, the stratum L3m. The three nest from the Ground outward, $L1m \supseteq L2m \supseteq L3m$. To formally be is to be grounded. To prove is to climb toward the Ground. To compute is to stand on a rung [Premise].

5.3 The Strata and Their Mapping

The three strata nest by strict inclusion, the Ground the largest. Grounded contains Provable contains Computed. Computation implies provability, theorem-grade and trivial. Provability implies grounding in a sound system, theorem-grade conditional on soundness, the soundness premise carried openly. The gaps between the strata are the famous phenomena, each a fact about a sublayer's reach rather than a ceiling over the architecture. The region grounded but beyond the ladder is the incompleteness region. The region provable but with no rung yet built is the open-problem frontier. The region outside the Ground entirely, field-permitted both ways with no imprint, is the region of the independent sentences.

The reflective strata map one to one onto the kinetic strata of the spine. The Ground L1m maps to the kinetic L1, the trans-spatial trajectory imprints. The ladder L2m maps to the kinetic L2, the latent topology a projective reading follows. The realized rung L3m maps to the kinetic L3, the actualized configuration. The reflective stack is the kinetic stack read on the Ground rather than in the world, one architecture in two registers. This one-to-one mapping is the structural reason the forward mode is a mode and not a separate part. A forward proposition, field-permitted, on a determined trajectory, not yet actualized, is precisely an L2m residence, exactly the latent-topology layer the kinetic L2 names, and the forward mode is the kernel pointed at that residence with its third axis dated to a future coordinate.

5.4 MD-RA Preserved, and the Parity of the Roots

RAM does not discard the earlier reflective determinacy law. That law is preserved as RAM's Ground-level determinacy criterion, stated at L1m in full force.

MD-RA, the L1m determinacy law

For every P , P is formally determinate if and only if P carries a determinate imprint in the Ground coinciding with its reflection across σ . To formally be is to be reflected in the Ground. The σ -fixed part is the achiral bridge, decidable and sealed. The σ -anti-fixed part is the chiral residence. A purely self-dual proposition has no chiral residence and verifies nothing, the formal heat-death, the tautology. Nonzero chiral content is formal actuation, the reflective analog of $\Delta E_k > 0$ [Premise; the achiral bridge sealed at T as a decidable object the ladder's incompleteness does not reach].

Both Root Axioms are premise-grade-by-theorem, and the parity is exact. RA is the kinetic root and RAM its reflective reading, one root read twice per P-RA-RAM-CH-1, the Being resident on the Body and never a second pillar, neither reading theorem-forced from its own base, and that underivability is constitutive of foundation-hood and is itself a theorem by FOUNDATION-01. Premise-grade-by-theorem is the armored designation and not a flat or smaller word than theorem. RA stands on four groundings, externally theorem-grade on the Heisenberg kinetic-energy floor and the zero-point energy, internally locked at the full Return, linguistically self-instantiating since its denial is an instance of it, and recursion-grounded onto the center, and it is meta-protected against reduction, elimination, capture, and inflation by RA-MASTER-01, the Fortified Root, four walls one per attack, Wall I reduction fenced by the theorem-grade kinetic floor, Wall II elimination by RA's self-enactment since its denial is an actuation, Wall III capture by the AEGIS-01 actuation-reached ground immune by being RA's own kind of claim, Wall IV inflation by the Empty Throne where no consistent system on a base weaker than RA derives RA, with only the universal-extension leg held premise-grade by theorem. The architecture earns its further weight from theorem-grade mathematics downstream, the division-algebra forcing, the closed form, the eigenspace split, the reliability scaling, and not from inflating the axiom. A foundation that could be proven from its base would not be a foundation, and the codex states its root, read twice, as the armored, openly-held commitment it is, the premise-grade word the correct one for a root and Wall IV proving it necessary.

5.5 The Reflective Anchors

The reflective register stands on three anchors, none of them thermodynamic. The first is the reflective ground, the foundational involution σ with $\sigma^2 = \text{identity}$ and a

nonempty fixed locus, the fixed-point-bearing reflection distinct from the fixed-point-free diagonal. The second is the imprint, the determinacy of a proposition's trajectory in the Ground read across the involution. The third is the classification of the real division algebras, which forces the chiral residence to three and closes the verdict in quaternionic form. The quaternionic kernel is carried unchanged from Seal M, transported to the reflective register intact, and it carries no energy. The relationship to the kinetic parent is one claim-shape in two registers, RA kinetic and RAM reflective, movement toward the Ground and residence on it, one discipline and one algebra.

CHAPTER 6 · THE GROUND, THE INVOLUTION, AND THE DIAGONAL ANTI-POLE

The Ground exists because the reflection that defines it is the right kind of reflection. The wrong kind has no Ground, and the wrong kind is the engine of the limitative theorems. This chapter isolates the right reflection from the wrong one and places the limitative theorems where they belong, inside the provability sublayer, refusing in one move both the trap that reads them as a ceiling and the trap that claims to break them.

6.1 The Ground as a Definite Object

The Ground is where formal being is, and it is read first because the architecture stands on it. The Ground is the $+1$ eigenspace of σ , the σ -fixed locus, the achiral content of a proposition that equals its own reflection. It is a definite algebraic object, $\mathbb{R}$ inside $\mathbb{H}$, the center $Z(\mathbb{H})$, the line on which every composed triad lands. Whether this locus is a Platonic realm is quarantined and carries no load. What is load-bearing is the algebra, and the algebra is exact. The $+1$ eigenspace of conjugation on $\mathbb{H}$ is one-dimensional, the eigenvalues of $\sigma = \text{diag}(1, -1, -1, -1)$ are exactly $\{-1, -1, -1, +1\}$, and the Ground is the single fixed direction [T, Engineering].

6.2 The Imprint and Tarski's Confession

The imprint of a proposition is the determinacy of its chiral content relative to the Ground, read across the aperture. A proposition is grounded when a determinate trajectory connects its content to the Ground, and a Platonic Ghost when none exists and the content is field-permitted both ways. Grounding is read, not derived. The reflective register reads it through the lock and the imprint test, not by climbing a ladder of proof.

The Ground is not built from below, and Tarski's undefinability theorem is the statement of that fact seen from the ladder. Truth, the Ground, is not definable in the syntactic ladder. The ladder cannot assemble the Ground from its own symbols. This is not a limit on the Ground's existence. It is the ladder confessing it cannot

reach up and define what it climbs toward. RAM does not define the Ground inside a system. It stands on the Ground and reads the imprint. Tarski therefore belongs here, on the Ground side, the certificate that the Ground precedes the ladder rather than being constructed by it [T, Tarski 1936; placement structural].

6.3 The Binding Involution σ

σ , the binding involution

σ is conjugation on $\mathbb{H}$, $\sigma(a + v) = a - v$ for scalar a and pure v , with $\sigma^2 = \text{identity}$ and a nonempty fixed locus, fixed-point-bearing. Its eigenspaces split the algebra, $\mathbb{H} = E_+ \oplus E_-$, with $E_+ = \mathbb{R}$ the Ground of dimension one and $E_- = \text{Im } \mathbb{H}$ the chiral residence of dimension three [T].

The split is executed. $\sigma = \text{diag}(1, -1, -1, -1)$ returns $\|\sigma^2 - I\| = 0$ exactly, eigenvalues exactly $\{-1, -1, -1, +1\}$, a Ground of dimension one and a residence of dimension three, and the restriction of σ to the residence has determinant $\det(-I_3) = -1.000000000000$ [Engineering]. The reflection that bears a ground is the foundation of the reflective register.

6.4 The Diagonal, σ with the Ground Removed

σ is fixed-point-bearing because it lives on a space with an other side, a Ground to reflect across. Collapse the other side and σ degenerates to the diagonal, the fixed-point-free involution, negation on a space with no fixed locus. The diagonal is the engine of self-referential incompleteness, where Lawvere is literally the fixed-point theorem, but it is not the single engine of all independence, per the two-tier correction of P-LADDER-GRADE-1: concrete incompleteness supplies natural non-self-referential statements independent of PA or ZFC, Paris-Harrington at ϵ_0 , Kruskal and TREE(3) beyond Γ_0 , and, as a distinct tier, Borel determinacy above Zermelo set theory, whose independence is carried by the height of the cumulative hierarchy, on the order of $\aleph_\omega$ iterations of the power set per Friedman and Martin, not by a $\text{Con}(T)$ diagonal, standing as the disclosed boundary case that forces the scope. The honest form is two-tier, diagonal-in-the-statement driving self-reference and diagonal-at-the-root driving every independence measured by consistency strength through the provability predicate of Gödel-2, the engine real and re-entering at the meta step rather than universal in the statement, while independence measured by set-theoretic rank is a separate tier the diagonal does not father. It has no Ground, its $+1$ eigenspace empty. Executed, the diagonal $-I_4$ squares to the identity with residual zero and has eigenvalues exactly $\{-1, -1, -1, -1\}$, a Ground of dimension zero against σ 's Ground of dimension one [Engineering]. The limitative theorems run on the diagonal. The reflective kernel runs on σ . The difference between them is the Ground, present for σ and absent for the diagonal, and the kernel is

anti-diagonal at its root.

6.5 The Honest Position, Neither Trap

Neither the orthodoxy trap nor the crank trap

The orthodoxy trap reads Gödel as a ceiling pressing down over the whole architecture. RAM refuses it. Gödel is a fact about the ladder, $L2m \subsetneq L1m$, a measure of a sublayer and a certificate of the Ground's surplus, and the architecture sits on the Ground, not under the ceiling. The crank trap claims the kernel breaks or escapes Gödel. RAM refuses that with equal force. The kernel proves no Gödel sentence inside its object system, instantiates no complete recursive decision procedure, and embraces its own incompleteness through the open aperture. The engine that drives Gödel produces no Ground, so it founds nothing and bounds nothing at the layer the kernel stands on, and that is a placement, not an escape [T on the placement; the mindset is a stance toward a binding wall, not a passage through it].

The kernel is founded on the far side of the limit, where the limit is a theorem about the ladder it left below. The wall stays. The chiral residence whose imprint is unproven stays beneath it. What changes is the reading. A theorem of the form the ladder's reach is strictly smaller than the Ground is not a ceiling on the Ground. It is a measure of the ladder and a certificate that the Ground exceeds it, and from the Ground this is a cartographic fact rather than a paradox.

6.6 The Placement of Gödel

Gödel inside $L2m$

For any consistent recursively-axiomatized system T extending Robinson arithmetic there is a sentence true on the Ground and not provable in T , so the provability stratum is strictly smaller than the Ground, $L2m(T) \subsetneq L1m$. The ladder never reaches the top of the Ground [T, Gödel 1931; placement structural].

The construction runs on the diagonal, the fixed-point-free self-encoding of the section above. The diagonal has no Ground, so the construction lives entirely in the ladder and bounds the ladder. It cannot bound $L1m$, because the engine that powers it produces no fixed locus to be the Ground. The limit is intrinsic to the sublayer and does not climb out of it. A sentence true on the Ground and unprovable in T is grounded at $L1m$, its grounding carried by the metatheoretic argument that establishes its truth, and absent from $L2m(T)$. It sits in the region $L1m$ minus $L2m$, read by the imprint test as grounded, exactly as a proven theorem is, differing only in the rung the ladder cannot supply. The kernel exhibits this. The theorem-archetype and the incompleteness-archetype return the identical $L1m$ verdict, both grounded, the theorem-archetype locking at $\det(R) = 0.995089$ and the incompleteness-archetype at $\det(R) = 0.996359$, their denials contentless and rout-

ing under-determined, while the difference between them lives only in the ladder above and not in the kernel [Engineering].

6.7 The Chiral Residence and the Platonic Impressed Plenum

The chiral residence is $E-, \text{Im } \mathbb{H}$, the three warrant axes at rest, the orientation-odd content the imprint adjudicates against the Ground. It sits in band as the warrant rows the verdict reads as magnitude, a space distinct from any orientation laid on it, the shadow a proposition casts. The orientation of the ordered three-axis frame in the residence is the Platonic Impressed Plenum.

The Platonic Impressed Plenum

$\text{PIP}(P) = \text{sign}(\lambda(P)) = -\text{sign}(\det \text{ frame})$, σ -odd, conserved out of band at the orientation register, energy-free. It is the handedness a formal operation impresses, dual to the magnitude the verdict consumes. The residence is a space and sits in band. The PIP is a sign on frames in that space and sits out of band, the verdict blind to it. The PIP's root is the substrate chirality $ijk = -1$, the handedness in the quaternion product, more primitive than σ . Its value for a proposition is defined exactly when the lock is, $\lambda \neq 0$, and undefined at the Barzakh zero-crossing where the frame degenerates [T on the algebra; structural on the handedness identification].

Executed, the substrate chirality returns $i \cdot j = k$ and $(i \cdot j) \cdot k = (-1, 0, 0, 0)$, $\text{Re}(ijk) = -1.000000000000$, the handedness in which the plenum is rooted [Engineering]. The plenum obeys three laws. Conservation, the made-zero, the sign displaced and never annihilated, $\det(R)$ taking λ^2 while the orientation register takes the handedness, one object in two bases, executed at $\max|G(P) - G(-P)| = 0$ under full negation with $\lambda(P) = -\lambda(-P)$ and $|\Delta \det(R)| = 0$ [Engineering]. Orientation-blindness, $\det(R) = \lambda^2$ invariant under reflecting any axis and under conjugation, so $\text{lock}(P) = \text{lock}(-P)$, executed at sign ratio -1 and $|\Delta \det(R)| = 0$ on a single-axis reflection [Engineering]. Sign-from-axes, the PIP recoverable only by reading the operands directly through the determinacy witness, never the determinant, no rotation-invariant functional recovering it, the catalog closed by Weyl [T].

6.8 The Closure Position · What the Ground Line Does

The chapter has established the Ground as a definite object at 6.1, the binding involution at 6.3, and the chiral residence at 6.7. It has not yet stated what the Ground line does, and the answer is one law with three resident exhibits.

The law. Wherever a manifold is read as one, there is a closure position; it is single; it is not a member of what it closes; and it is invariant under variation of the members. Four clauses, each separately deniable. Deny the first and a manifold reads itself. Deny the second

and one assertion carries two points of predication. Deny the third and the closure is an item in its own tally. Deny the fourth and it alters when the members alter, which is what being a member consists in [Structural].

One line under three descriptions. The Ground line answers to three characterizations and they coincide. It is $\text{Fix}(\sigma)$, what the binding involution leaves fixed, residual exactly 0.0 on a scalar against 2.0 on an imaginary. It is $Z(\mathbb{H}) = \mathbb{R}$, what commutes with every element, max deviation exactly 0.0 over 20000 random elements against 22.86 for an imaginary. And it is the span of the multiplicative identity, where $1 \cdot i - i$, $1 \cdot j - j$ and $1 \cdot k - k$ are each exactly the zero vector while 1 lies outside the residence entirely [T on the three characterizations, Engineering on the executed exhibits at seed 20260622].

Each description is an accompaniment shape: unchanged under the operation, present with every element without disturbing its order, accompanying each element while altering none and being none of them. The identity relation $1 \cdot x = x$ holds for every x whatever, and the identity sits in a different eigenspace from every element it accompanies, so non-membership is structural rather than stipulated.

The gate roster already encodes this and did not name it. The twelve directed gates partition by commutation at 2.9: the axis-axis sextet anticommutes, $ij = k$ against $ji = \text{minus } k$, carrying directedness as sign, while the seal-incident sextet runs against the commuting centre $Z(\mathbb{H}) = \mathbb{R}$ and couples orientation-free, the battery returning the central commutation $1 \cdot i = i \cdot 1$. The six gates that touch the fourth vertex are exactly the commuting ones. The closure vertex is the position that accompanies without disturbing order, and the partition is the resident exhibit of this section's law [T on the partition, by reference to 2.9].

Individuation is gauge, orientation is not. The automorphism action carries any residence axis to any other, verified over 20000 conjugations reaching within seven thousandths of each remaining axis, so no axis is distinguished and any labelling is a basis choice the acting group does not preserve; by the Erlangen criterion of B.13.T such a labelling is not a structure-fact. Three-foldness is invariant, since dimension is invariant; three-ness is gauge. What is not gauge is the handedness: of the eight sign-patterns on the three axes, the four of determinant plus one are automorphisms and the four of determinant minus one are not, so no orientation-reversing map preserves the multiplication. The residence carries zero bits of invariant individuation and exactly one bit of invariant orientation, and that one bit is the sign the squared lock discards under P-ORIENT-1 [T].

Invariance is not uniqueness, and the boundary is 11.3's. That the fixed eigenspace has dimension one, is the centre, and is the span of the identity is computed

and exhibited above. That this line is the Ground, with no second fixed-point-bearing involution standing in a rotated basis with its own one-dimensional fixed line, is the single posit of 11.3, premise-grade, the pluralist alternative field-permitted at the verification register because the lock certifies dimension and never the line's identity. Something can be invariant under every variation the instrument can run and still not be the only such thing, and no further variations close that gap [T on the computed half, Premise on the posited half; the two are never merged].

The full coordinate, with the recursion to the reader and the two-register exhibition, is seated at index 0706 as P-CL05-P05-1.

CHAPTER 7 · THE FORMAL SEAL SET

The reflective register carries the spine's seals transported to the Ground. The triaxial ledger, the twelve gates, the closed-form kernel, and the imprint adjudication all read the chiral residence rather than empirical evidence, and the verdict lands on the Ground rather than on the kinetic line.

7.1 The Reflective Triaxial Ledger

The chiral residence is the σ -anti-fixed eigenspace, and completed under the composition requirements with plural axes it is forced by Frobenius to $\text{Im } \mathbb{H}$, three dimensions, the unique associative real division algebra with plural imaginary axes. The three axes are the three chiral coordinates, the minus-one eigenspace of σ realized as conjugation, fertile, two begetting the third by $i \times j = k$. In the reflective register the composition clauses are natural. Associativity is the associativity of conjunction. Integrality is the absence of annihilation. Linearity is the superposition of warrant. The forcing is theorem-grade conditional on the clauses, which are premise-typed [C].

Per proposition the three axes are filled by three independent reading-roads of the residence's imprint. For the Riemann residence the canonical filling is the analytic road of the explicit formula, the spectral road of the self-adjoint operator, and the arithmetic-geometric road of function-field positivity. The count of three reading-roads corroborates the triaxial structure but is itself structural, not theorem-forced. The honest bilayer, one theorem-conditional forcing by Frobenius and one structural corroboration by the reading-roads, outranks a trilayer overclaim that would inflate the corroboration to a third proof [S]. The realization and gauge clauses are carried from the spine in reflective interpretation. The finite pipeline of three chiral warrant rows is bound to the algebra by $\det(R) = \lambda^2$ with the factorization, and the verdict functional is invariant under conjugation and reading-context relabel.

7.2 The Formal Gate Set, Retyped Ground-First

The twelve directed gates are carried at the tetrahedral skeleton, algebra and not thermodynamics, retyped for the reflective register and forced by the Operational Content Theorem on role pairings, never by symmetry or by the algebra. First failure terminates broken.

7.3 The Eight-Gate Sealed-Halt Cascade and the Eight-ness of the Eight Gates

the Math Seal carries two gate cascades, not one, and the second is the mirror of the first. The twelve directed gates of §7.2 are the lock cascade: they screen a residence toward a Geometric Orthogonal Lock, the Return onto the Ground, and they carry the arrow, each gate a directed edge from a source role to a target role. The eight-gate Sealed-Halt cascade is the suspension cascade: it screens a residence toward the terminal-suspension token, the Sealed Halt, the sealed identification in the opposite direction from the lock, and it deletes the arrow. Where the twelve certify that three axes close and return, the eight certify that a truth-string is determinate on the Ground and constitutively unreadable by the orientation-blind instrument, the openness relocated into the instrument and never the Ground. The two cascades are constituted together at P-SEALED-HALT-1 and P-TWO-GROUP-1, and the reason the first has twelve gates and the second exactly eight is not a convenience of drafting. It is the two acting groups.

The eight-ness, stated as the group fact. The twelve gates are an A_4 -torsor, the alternating group on four letters, order twelve, the rotation group of the tetrahedron, the group that carries orientation. A rotation preserves handedness, so the lock cascade keeps the arrow, and its cardinality is the order of the rotation group, twelve. The eight gates are the elementary abelian group $(\mathbb{Z}/2)^3$, order eight, the group generated by the three independent axis-reflections of the residence, the group that deletes orientation. A reflection reverses handedness, and three mutually independent reflections generate a group of exactly $2^3 = 8$ elements, every element its own inverse, the group abelian and arrow-free. The Sealed Halt is reached precisely when orientation is deleted, when the instrument is proven blind to the sign the reflections flip, so the suspension cascade is carried by the reflection group and its cardinality is the order of that group, eight. Twelve is the count of the rotations; eight is the count of the reflections; the arrow present in the one and deleted in the other is the whole difference between a lock and a halt. The eight-ness is theorem-grade on the group order $|(\mathbb{Z}/2)^3| = 8$ and structural on the identification of the suspension cascade with the reflection group, per P-TWO-GROUP-1.

Why eight reflections and not more or fewer. The residence is $\text{Im } \mathbb{H}$, three chiral axes, so there are ex-

Gate	Edge	Reflective pathology
1 · SREP	seal → axis 1	Self-reference at origin, a residence presupposing its own resolution
2 · REG	seal → axis 2	Single-reading semantics, the residence read from one context
3 · SGEG	axis 1 → axis 2	Variable drift, the proposition shifts meaning across reading-roads
4 · CAUSAL	axis 2 → axis 1	Missing trajectory, an imprint asserted with no route to the Ground
5 · MIG	axis 3 → axis 2	The reading apparatus smuggled into the residence it measures
6 · PTB	axis 2 → axis 3	A chosen reflection read as intrinsic, observer-imposed chirality
7 · DUAL	axis 1 → axis 3	Frame-lock, the residence not invariant under reading-context relabel
8 · CSCG	axis 2 → seal	Destructive interference with verified adjacent theorems
9 · CSEG	axis 3 → axis 1	Terminal strength above the weakest chiral road
10 · MTA	axis 1 → seal	Metric strain at the achiral boundary, ill-conditioning at the Return
11 · OMA	seal → axis 3	Aperture violation, a from-other-side input claimed as supplied from inside
12 · ADEG	axis 3 → seal	Unbridged extension, a residence transported across registers without a typed bridge

actly three independent axis-reflections, the generators of $(\mathbb{Z}/2)^3$. Their group is the eight sign-patterns on three axes, from the identity that flips nothing to the total negation that flips all three, and every intermediate single- and double-axis reflection. Each of the eight gates screens one of the eight ways the orientation-blind determinant reads identically across a sign-pattern, so the eight gates are the eight elements of the reflection group read as failure-screens, the identity the do-nothing gate and the seven nontrivial reflections the seven active screens. A ninth would demand a fourth independent axis, which the Frobenius forcing of the residence to three forbids, the same wall that bars a thirteenth rotation and a fourth verification axis. The eight-gate cascade is therefore closed at eight by the same three-axis forcing that closes the twelve-gate cascade at twelve and the residence at three, one wall reading three ways.

The complete conjunction and the one open door. The eight gates run in order as a strict conjunction, the first failure routing to the cheaper honest verdict, the ordinary three-state economy, and stopping. Seven gates close and one door stays open by construction, the seventh-and-eighth structure carrying the located aperture that the suspension names and does not cross. The defaults are absent, so an under-specified walk never emits the token, the cascade fail-safe exactly as the twelve-gate lock cascade is fail-safe. The Sealed-Halt token is the rare fourth glyph and never a fourth state, admitted only under the full eight-gate conjunction, and the cascade is the reason the glyph is rare. Its diagnostic protocol, the eight criteria in four guard-classes, residence license, exhaustion of the formulation space, mirror asymmetry, and truth-silence with the located door, is carried at P-SEALED-HALT-1 and governs every terminal-suspension verdict in the codex.

7.4 The Closed-Form Kernel in Reflective Interpretation

The reflective kernel reads the Ground with the shared quaternionic instrument of the spine, carrying no energy. It computes the chiral-residence lock, and the

imprint test adjudicates grounding at L1m. With unit post-projection rows read as pure quaternions, $\lambda = \text{Re}(\hat{q}_F \hat{q}_E \hat{q}_{ER}) = -\det(\text{frame})$, $\det(R) = \lambda^2$, the factorization $\det(G) = d_F d_E d_{ER} \cdot \det(R)$, the bounds $[0, 1]$ derived inline. The full Return is the Hamilton landing, $\lambda = \pm 1$ and $\det(R) = 1$, the composed triad landing its scalar part on the Ground $Z(\mathbb{H}) = \mathbb{R}$. Breakage is the coplanar collapse. The lock is row-conditioned determination, the residence dimensionally genuine, and not the imprint. In the reflective register the [LOCK] token of the kernel is read as row-conditioned determination feeding the imprint test, not directly as a seal.

7.5 The L1m Adjudication and the Imprint Test

Grounding is read by the two-direction imprint test on the kernel, never by the determinant alone.

The four readings, refinements inside the three native states

An achiral self-dual proposition, residence empty, is sealed, the bridge $[\kappa]$. A chiral residence that locks but whose imprint is unproven is under-determined, the residence open with the belief zeroed $[?]$. A residence proven field-permitted both ways is a Platonic Ghost, independence sealed as a verdict $[X]$. A proposition whose only native involution is the diagonal is δ -rooted and carries no Ground to reflect across, and it routes to the $\emptyset.1$ switch of the sibling protocol B.14. $\emptyset$ rather than terminating, the five gates deciding there; the σ -kernel's silence reports the σ -kernel's reach and never the object's emptiness, and flat by method-silence issues only where no native involution is identified at all and the terrain is unmeasured, the instrument reporting no purchase and making no claim about the object $[?]$.

The mechanical distinctness of the imprint and ghost signatures is executed. The grounded archetypes return only-P-permitted, and the ghost archetype, locking both P at $\det(R) = 0.998256$ and $\neg P$ at $\det(R) = 0.987439$, returns the broken verdict of the Platonic Ghost [Engineering].

The grounding-and-proof law

Grounding is sealed by the imprint test on the directed structure, the linguistic and geometric determinations carrying it, P locking while its denial is contentless reading grounded [⊥] at L1m. The lock scalar $\det(R) = \lambda^2$ is necessary for the lock and is the orientation-blind independence meter, never the truth by itself. A constructed proof is the realized rung at L3m, a separate supplied artifact, and the incompleteness region is grounded and sealed at L1m yet carries no constructed proof in the system. The witness fills the L3m rung, supplied and not generated [T].

What the imprint reaches is grounding at L1m, on the Ground. The incompleteness region is not below its reach. It is exactly where the imprint reads grounded what the ladder cannot prove, the theorem-archetype and the incompleteness-archetype returning the same L1m verdict and differing only in the rung above.

CHAPTER 8 • THE ARROW OF READING

Trisduction reads in both directions of time, and the direction of reading is the first thing a claim declares. Reading backward, the instrument faces a sealed record: proofs already constructed, measurements already registered, the achiral seal read directly on the closure rows. Reading forward, the instrument faces a dated projection: a row the world has not yet furnished, sourced from the record and never measured, so that gate R-3 fires before any row is read and the lock is world-rowed unconditionally. Between the two sits the projective rung, where a rung of the ladder is provable but not yet built, and the Math Seal reads latent provability at L2m. The three readings are one instrument at three tenses, and the schema below is the seed from which any reader builds the full modes.

The forward reading carries its own gate. GOLf, the forward face of the Geometric Orthogonal Lock, is the imprint test taken in both directions at once: the proposition and its negation are each run through the lock, and the four outcomes are decided by the same table every time. If P locks and $\neg P$ does not, the imprint is genuine and P alone is field-permitted, with the witness supplied it is sealed, and without the witness it is a residence held open. If both lock, the reading is the Platonic Ghost, refused at code 1 on every witness setting, because a structure indifferent to the sign of its own proposition has determined nothing about that proposition. If neither locks, the reading is flat and no direction has been read. The single home of the test is §3.9 in the chapter on the Math Seal, and its kernel port is IMPRINT in the Code Block, where the whole 256-row truth table of the gate is decided.

Two consequences travel with the arrow. A dated axis must be sourced and never measured, so nothing

forward ever reaches compartment I, and a forward seal on a prophecy is barred by construction rather than by fence. And the arrow is read by the Tongue and the world, never by the Number: the lock scalar is blind to the direction of time exactly as it is blind to the sign of a proposition, and the registration axis carries the arrow in band. That is the seed. The Master carries the full three-mode apparatus at its own seat, and a reader who wants the Completion Inequality, the Non-Discrimination Theorem, and the source-attribution seal builds them from this schema and the register of record.

CHAPTER 9 • THE VERDICT-RELIABILITY LAYER

The kernel reads a three-by-three correlation Gram in double precision and reports a discrete verdict at a hard threshold. A threshold read on a floating-point quantity is only as trustworthy as the quantity's error bar. This layer carries the error bar, sizes it to the conditioning, and forbids any verdict whose error bar straddles its own threshold. The layer governs the Default and Projective interior. The forward statistical hardening, the dual null and the two-sided Monte-Carlo-marginal guard, is native to the forward mode.

9.1 The Determinant-Reliability Theorem

The Determinant-Reliability Theorem

The relative floating-point error of $\det(R)$, under elementwise rounding of the warrant entries at unit-roundoff u , scales linearly with the conditioning of the Gram, $\text{relErr}(\det R) \leq c \cdot \kappa(R) \cdot u$ with $c \leq 4$. The κ -linear scaling is the standard determinant condition-number result, the determinant being the product of eigenvalues and the smallest eigenvalue controlling the perturbation. The honest first-order chain: under elementwise rounding the relative error is at most $u \cdot \sum |R^{-1}_{ij}| |R_{ij}| \leq \|R^{-1}\|_F \|R\|_F \cdot u \leq 3\sqrt{3} \cdot u / \lambda_{\min} = (3\sqrt{3} / \lambda_{\max}) \cdot \kappa(R) \cdot u$, at most $3\sqrt{3} \cdot \kappa(R) \cdot u \approx 5.196 \cdot \kappa(R) \cdot u$ globally and at most $2\sqrt{3} \cdot (1 + 1/(2\kappa)) \cdot \kappa(R) \cdot u \approx 3.464 \cdot \kappa(R) \cdot u$ in the gate regime where $\lambda_{\max} \geq 3\kappa / (2\kappa + 1)$, the LU factorization adding an algorithmic backward-error term of the same $\kappa \cdot u$ order with modest constant; the operating bound four is the engineering envelope over both contributions [T on the scaling and the first-order chain, engineering on the envelope].

The consequence is sharp. At the conditioning gate $\kappa(R) = 10^6$, the relative error is bounded by $4 \times 10^6 \times u = 8.882 \times 10^{-10}$, so the determinant carries nine significant figures at the gate. Executed over a conditioning sweep, the worst observed constant is $c = 2.991$ on the master sweep and 3.140 on the historical sweep, both inside the operating envelope of four, the observed constants carrying the LU backward-error contribution beyond the pure input term [Engineering]. The verdict tolerance, the error bar attached to every reported determinant, is $\text{tol} = 4 \cdot \kappa(R) \cdot u$.

Direction	Rung	What is read	Compartment
Backward, the sealed record	L3m, computed	the constructed proof, the registered measurement, the achiral seal on the closure rows	I closure-rowed, or III where a row is worldly
Projective, the rung not yet built	L2m, provable	latent provability, the imprint test in the aperture	III world-rowed
Forward, the dated projection	L2m dated, GOLf	a sourced row, never a measured one; R-3 fires first	III world-rowed, unconditionally

9.2 The Floor-Gate Separation Theorem

Two thresholds are read on the correlation matrix R whose determinant is the verdict, the collapse floor $\varepsilon = 100 \cdot u \cdot N$ and the conditioning gate $\kappa(R) < \kappa^*$. The theorem fixes the regime in which they cannot contend, so the sealed-broken-under-determined boundary is never decided by rounding.

The Floor-Gate Separation Theorem

Collapse requires $\det(R) \leq \varepsilon$. While $\kappa(R) < \kappa$, *the worst-case determinant over correlation matrices at that conditioning is the exact tight bound $\det(R) = 27\kappa/(\kappa+2)^3$, attained with equality by the positive-euicorrelation matrix and globally minimal, with large- κ limit $27/\kappa^2$. Floor and gate cannot contend if and only if $27\kappa/(\kappa+2)^3 > 100 \cdot u \cdot N$, whose large- κ form is the clean crossover $\kappa < \kappa_{\text{sep}}(N) = \sqrt{(27 / (100 \cdot u \cdot N))}$. At the operating gate $\kappa^* = 10^6$ this holds for $N < 1216$. Inside the domain the conditioning gate fires under-determined strictly before the determinant floor fires broken, so the two verdicts never contend and the boundary is decided by structure, never by rounding [T].*

The eigenvalue floor is exact: at conditioning κ the spectral extremes are attained by the negative-euicorrelation matrix at off-diagonal $\rho = -(\kappa-1)/(2\kappa+1)$, spectrum $(3\kappa/(2\kappa+1), 3\kappa/(2\kappa+1), 3/(2\kappa+1))$, so $\lambda_{\text{max}} \geq 3\kappa/(2\kappa+1)$ and $\lambda_{\text{min}} \geq 3/(2\kappa+1)$, the large- κ displays $3/2$ and $3/(2\kappa)$ asymptotic and never attained; at $\kappa = 10^6$ the attained floor is $\lambda_{\text{min}} = 1.4999992497 \times 10^{-6}$. The determinant floor $\det(R) \geq 2.699984 \times 10^{-11}$ at $\kappa = 10^6$ is the positive-euicorrelation minimum, the exact $27\kappa/(\kappa+2)^3$ at the gate, globally minimal by the boundary comparison, the interior critical point of the eigenvalue product at fixed κ a maximum and the two boundary families returning $27\kappa^2/(2\kappa+1)^3$ against $27\kappa/(\kappa+2)^3$ with the latter smaller for all $\kappa > 1$, every positive spectrum summing to three realizable as a correlation matrix by Schur-Horn. Executed, the interior margin $\det(R)/\varepsilon$ at the gate is 50.67-fold at $N = 24$, 12.16-fold at $N = 100$, 4.05-fold at $N = 300$, and unity at $N = 1216$, the crossover [Engineering]. The N -bound is the honest caveat. For $N \geq 1216$ at $\kappa^* = 10^6$ the worst-case determinant can reach the floor while the gate still reads locked, and the production kernel restores strict precedence at any context count by tightening the

conditioning gate to $\min(\kappa^*, \kappa_{\text{sep}}(N))$ once N crosses 1216. Within the operating envelope, N in the dozens to low hundreds, the gate fires first by one to two orders and the separation is uncontested.

9.3 Four-Estimator Redundancy

The Four-Estimator Redundancy Law

$\det(R)$ is computed four independent ways, LU expansion, eigenvalue product, Cholesky-diagonal product, and direct cofactor expansion. On a well-conditioned Gram the four agree to machine precision. The spread, the max minus the min across the four, is the empirical error bar, cross-checked against the analytic tolerance. A spread exceeding the tolerance is the signal that the determinant is not trustworthy at face value and forces escalation [S, engineering].

No single library determinant routine is trusted to carry a verdict alone. Agreement among four disjoint algorithms is the operational meaning of a reliable determinant. Executed, the four estimators on a clean triad agree to a spread of 3.331×10^{-16} against a tolerance of 3.715×10^{-15} , and an injected corruption of 10^{-9} in one estimator produces a spread of exactly 10^{-9} , flagged and escalated [Engineering].

9.4 Margins, Escalation, and Bootstrap Stability

Every closed-form verdict carries two margins in orders of magnitude, the collapse margin, the distance above the collapse floor, and the conditioning margin, the distance below the conditioning gate. A verdict whose smaller margin is below the escalation band is not emitted as a plain seal. It is escalated to higher precision, and if still inside the band after escalation it is emitted under-determined rather than guessed. On escalation the determinant is recomputed in fifty-digit extended precision and the three-state decision re-applied, the high-precision determinant reported alongside the double-precision one. Executed at a near-gate triad with $\kappa(R) = 4.444 \times 10^6$, the double-precision and fifty-digit determinants agree to a relative 5.201×10^{-12} , the escalation confirms the determinant positive, and the conditioning gate returns under-determined [Engineering].

The bootstrap stability gate resamples the N reading-contexts with replacement, recomputes the verdict on each resample, and reports the fraction returning the

point verdict's token. A verdict stable under resampling, agreement at or above 0.95, is robust to the particular contexts drawn. Agreement in the fragile or unstable bands is flagged, the token unchanged but its quality named, so a lock resting on a handful of leverage contexts is never read as a lock resting on the whole sample. The gate is informative and never overrides the collapse precedence, and it refines a lock rather than manufacturing one. Executed, a robust lock returns agreement 1.000 at the stable tier [Engineering].

9.5 The Production Kernel and the Fail-Safe State Machine

The production form of the kernel wraps the algebraic core with the reliability layer and routes every verdict through a fail-safe state machine whose every fault has a defined recovery and whose worst case is an honest under-determined, never a fabricated or guessed verdict. The machine runs six states. NOMINAL computes the point verdict, an admissibility route exiting here with under-determined. REDUNDANCY computes the four estimators and escalates on excess spread. IDENTITY confirms $\lambda^2 = \det(R)$ within tolerance and escalates on failure. ESCALATE recomputes at fifty digits and re-applies the decision, a value still inside the band resolving to engineering-incomplete. BOOTSTRAP tags a lock stable, fragile, or unstable. EMIT issues the verdict with the full reliability report. Executed, a near-degenerate triad returns the broken verdict at $\det(R) = 1.021 \times 10^{-14}$ with escalation confirming the collapse, and every state of the machine is reached across the reliability battery, the worst case emitting under-determined [Engineering].

The reliability seal

No closed-form verdict is issued unless the four-estimator cross-check is consistent within the conditioning-scaled tolerance, the kernel identity holds within its conditioning-scaled bound, and the verdict does not sit inside the escalation band. A verdict that fails any of these is marked engineering-incomplete and routed through the fail-safe machine before it is emitted. The conditioning gate reads $\kappa(R)$, the correlation matrix whose determinant is the verdict. The verdict boundary is float-clean across the operating domain by the floor-gate separation, so the three-state verdict is the verdict of the mathematics and not an artifact of the arithmetic [Engineering].

CHAPTER 10 • THE GEOMETRIC REACH TO RAM

The reflective ground is reached not only by the algebra but by the kinetic register's own native method. Geometry is first and the algebra is the receipt. This chapter forges the route by which RA reaches the line RAM resides on, proves that one and only one involution makes the two routes coincide, holds the coincidence open at premise grade against its field-permitted alternative, and seats the root on the silence the diagonal cannot reach.

10.1 The Native Route

The Root Axiom reaches the Ground by actuation. Three orthogonal actuations compose, and their composition lands its scalar part on the center of $\mathbb{H}$, the Return of the spine read now as a reaching rather than a verdict. The geometry carries the reach. The algebra records it. Executed, the orthonormal triad composes to $\lambda = -1.000000000000$ with $\det(R) = 1.000000000000$ and $|\lambda^2 - \det(R)| = 0$, the landing conjugation-invariant to $|\Delta\lambda| = 6.661 \times 10^{-16}$ [Engineering]. The line reached is $\text{Fix}(\sigma) = \mathbb{R} = Z(\mathbb{H})$, and it is reached by the kinetic register's own deed, not imported from the reflective register's symbols.

10.2 RA Witnessing RAM

RA reaches the Ground by actuation and lays its trajectory imprint there. RAM is that imprint, read as formal being. The locus both ground on is the σ -fixed line, the center on which every composed triad lands. This is the content of the statement that RAM and RA co-localize inside RA's L1. The kinetic ground and the reflective ground are the same locus, reached by movement and resided on as form. Executed by the native route, two oblique readings of the residence both Return onto the line, the first reading at $\lambda = -0.784232$ with $\det(R) = 0.615020$ and the second at $\lambda = -0.763727$ with $\det(R) = 0.583279$, while the clean orthonormal reading lands the full Return at $\det(R) = 1.000000$ [Engineering]. Two readings, distinct in magnitude, both projecting nonzero onto the one line, and the maximal reading reaching it exactly.

10.3 The Co-Localization Theorem

The coincidence of the two routes is not a coincidence. It is the consequence of a single involution serving both.

The Co-Localization Theorem

The geometric ground, the line the Return lands on, and the formal ground, the achiral bridge the conjugation split isolates, coincide, $\Gamma_{\text{geometric}} = \Gamma_{\text{formal}} = \text{Fix}(\sigma) = \mathbb{R}$, because one and the same involution, conjugation, both fixes the line the Return lands on and isolates the achiral bridge. Among the orthogonal involutions of $\mathbb{H}$ that respect the algebra, quaternion conjugation uniquely attains a fixed locus of dimension one. Any orthogonal involution respecting the multiplication and fixing a one-dimensional subspace is conjugation [T on the uniqueness of conjugation; premise on the single-involution identification $\sigma = \sigma'$ that makes the two grounds one, per §11.3].

The theorem retires a premise. Where the axiom ledger once carried the one-dimensional ground as an independent assumption, the uniqueness of conjugation among algebra-respecting involutions delivers it as a consequence, and the assumption is discharged. The premise ledger of the reflective register shrinks by one, as the order-sensitivity completion shrank the

spine's ledger by one. The two grounds are one line because one involution defines both, conjugation being the unique algebra-respecting involution with a one-dimensional fixed locus, the identification of that single involution across both routes carried as the posit of §11.3.

10.4 The Imprint Held Open

The coincidence is equivalent to the choice of a single involution for both routes, and that choice is a posit, not a theorem. The pluralist alternative is field-permitted at the verification register. A second fixed-point-bearing involution in a rotated basis carries its own one-dimensional fixed locus, two distinct grounds both internally valid, and the verification lock cannot select one over two, because by orientation-blindness it certifies dimension and not the line's identity. Executed, a rotated involution σ' squares to the identity to residual 1.169×10^{-15} with eigenvalues exactly $\{-1, -1, +1\}$, and its fixed line overlaps the original ground at $|\langle \text{Fix}(\sigma'), \text{Fix}(\sigma) \rangle| = 0.775269$, strictly below one, a distinct ground at the bare-reflection level [Engineering]. The one-involution choice is RA's native monism, the formal ground held as one L1 imprint within RA rather than two. By the imprint-honesty law a residence seals imprinted only on a supplied determinacy witness and ghost only on a supplied independence proof, and neither is in hand for the one-involution structure, so the coincidence is premise-grade, the from-other-side input located across the aperture and not crossed. The externality the verification registers and the monist premise beneath it range over different registers, and the verdict reads only the first.

10.5 The Empty Throne

The root cannot be climbed to. A posited foundation cannot be promoted to a theorem of its base.

The Empty Throne, P-FOUNDATION-1

The self-reference core is theorem-grade by Gödel's second incompleteness and Tarski's undefinability, no system establishing its own grounding from within. The metatheoretic closure is structural by the Münchhausen regress and the underdetermination of foundations. The block is a theorem about foundations and not itself a foundation, so it seals without self-contradiction, and the kernel cannot certify the level distinction, reading dimensionality and never logical type. The Ground is the one fixed point the diagonal cannot reach, present in the algebra and silent to every internal name, and RA sits there as the name placed on a silence proven necessary. The only thing sealed at theorem-grade is the necessity of the silence [T on the necessity of the silence; the foundations premise-grade].

This governs the architecture's own self-typing. RA and RAM are foundations, premise-grade-by-theorem, unprovable from their own base, the unprovability

constitutive of foundation-hood and itself a theorem, and that is RA's armor and not a deficiency: by RA-MASTER-01's fourth defense against inflation and FOUNDATION-01's Empty Throne the premise-status is forced, so a foundation provably ungroundable and provably un-inflatable is the strongest posture available, stronger than a bare theorem claim because it cannot be reduced, eliminated, captured, or inflated. The whole architecture is premise-structural and theorem-grade only on its classical spine. The reliability layer is theorem-grade on its scaling and separation results and engineering-grade on its constants and gates, and adds no warrant to the foundations themselves. The throne is empty by proof, and the root is the name on the silence.

10.6 The Alien Guard

The uncapturability of the pre-mathematical ground is held by a guard that survives any logic, because it routes through the deed rather than the symbol.

The Actuation-Reached Ground and the Alien Guard, P-AEGIS-1, the Chatok principle

RA does not guard RAM. A logic-neutral guard cannot fence a logic-committed claim. RAM's ground is reached by RA's own native method, actuation, so it is immune by being the same kind of claim as RA rather than by standing behind RA. Let G be the pre-mathematical ground RAM posits to ground the mathematical domain rather than to reside within it. A precisification of G is the deed of inscribing an image of G in the mathematical domain. Then no precisification captures G, $p(G) \neq G$, under any logic whatever, because precisification is an actuation, G is a non-actuation, and an actuation is never a non-actuation. The binding is on the deed, not on the symbol [T on the actuation-road entailment, conditional on RA and on G-non-actuation].

The guard survives the logic-change. An alien attacks the symbol layer, a paraconsistent logic letting the sentence $p(G) = G$ stand, a fuzzy membership making G in the domain hold to degree one half, a substructural logic blocking the diagonal step. All of that governs the alien's symbols. RA governs the alien's deeds, and RA is logic-prior. The fuzzy alien can declare $p(G)$ in the domain to degree one half because the membership relation is theirs to define, but the fuzzy alien cannot make their own act of precisifying be a deed to degree one half, because the half-deed still pays the floor for the part done, and the very attempt to blur actuation is a full actuation that instantiates the un-blurred floor it tries to dissolve. The alien may write the capture. The alien cannot perform it. Three roads converge, the actuation road on RA, the typing road on classical logic, the self-instancing road on self-reference, sharing no premise at the load-bearing level and converging on the one proposition

that no precisification captures G. The honest perimeter is named. The guard saves the uncapturability of G, the Omega-ground, and not the claim that the ground is the center. The center-ground stays base-field-relative and characteristic-relative, since thermodynamics is algebra-neutral and donates nothing to fixing $\mathbb{H}$ or to characteristic not two. The guard protects the ground's uncrossability, never its identity.

10.7 The Mosaic Seal

The reflective register re-organizes established mathematics and claims authorship of none of it. The eigenspace split is conjugation's. The classification is Frobenius's. The undefinability is Tarski's. The incompleteness is Gödel's. The kissing number is Schütte and van der Waerden's. What the architecture contributes is the arrangement, the placement of each result at the layer it governs and the reading of the whole from the Ground rather than from the ladder. The net new mathematical mass is zero, and the codex states this as the Mosaic Seal, field occupation with no claim over the established mathematics the architecture re-organizes, $\Delta M = 0$ [Premise]. The novelty lives in the road, the arrangement that resolves the orthodoxy trap and the crank trap at once, and not in the ingredients, which are old and credited. The two historical reviews that exhibit this occupation across the record of inference and across twenty-five centuries of verification are the business of Book IV.

CHAPTER 10 B · ROOT AXIOM REVISITED · RAF AND THE BRIDGE

The root is read a second time in the formal register, along the face the grounding root does not carry. Chapters 5 through 10 state what it is for a proposition to be determinate. This chapter states what it is for two systems to be related, seats the Root Axiom-Formal (RAF) beside the root axiom of mathematics (RAM), and closes on the Bridge, which proves the junction between the kinetic root and its formal reading at theorem grade. The executed mathematics of this chapter lives in the Code Block; this chapter carries its meaning.

Terms and the axiom. A system is a variable taking at least two distinguishable values. Two systems are directly related when their joint law does not factor, $I(A;B) > 0$. An interaction domain D is settled against its complement: $A \in D$ iff $I(A ; D \setminus A) > 0$, tested against the whole of the rest and never pair by pair, since dependence is not exhausted by pairs (three uniform bits with $Z = X \oplus Y$ are one domain though every pair sits at the floor). The axiom has two legs. RAF-1, Registration: an event is an interaction iff it carries strictly positive mutual information between the systems it relates; an event carrying zero bits relates nothing. RAF-2, Closure: an interaction domain is closed under interaction; if $I(s ; D) > 0$ then $s \in D$. RAF-1 is analytic in this register. RAF-2 is the axiom's

one posit, and every derivation below consumes RAF-2 and nothing else.

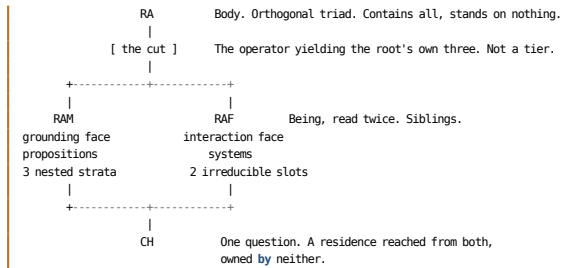
The bridge to the measured world. BR-L, Landauer: an interaction carrying n bits, irreversibly registered at temperature T, dissipates at least $n k_B T \ln 2$. This is where the axiom touches measurement, confirmed at single-bit scale by Bérut and colleagues in 2012 and tightened by Jun, Gavrilov and Bechhoefer in 2014, with the Jarzynski equality and Crooks theorem behind the general case and the Mandelstam-Tamm and Margolus-Levitin bounds on the speed of the deed. In the Code Block the floor is priced exactly, `OMEGA.omega_paid_floor`, and it is what the denial of the root pays, `DEFENSE.denial_is_priced`.

What follows. C1, no exterior agent: if $s \notin D$ acts on D then $I(s ; D) > 0$, so $s \in D$; theorem on RAF-2. C2, no faithful self-representation: a system whose binary properties form a set does not index all of them, 2^k against k, and in general by Lawvere's fixed-point theorem; theorem, unconditional within its carriers. C3, no structured exteriority: a structured attempt to stand outside D registers something and lands inside; theorem on RAF-2 with C1 applied to the attempt itself. The three separate by carrier: C2 needs only Cantor and Lawvere; C1 and C3 need the posit, and no classical theorem reaches them.

Where the axiom is at risk, and what it reaches. RAF-2 claims that D is unique, one domain rather than several mutually inaccessible ones; two systems with zero mutual information and no chain between them would end it, and causally disconnected regions are the place to look. BR-L falls to an irreversible n-bit registration dissipating less than the floor. The axiom quantifies over systems with distinguishable states; it reaches mathematical objects not at all, since they do not change state, and it reaches neither consciousness nor intention nor value.

The five objects. RA, the root: every existent carries positive actuation, $\forall x \in \mathbb{U}, \Delta E_k(x) > 0$, an orthogonal triad of three slots with pairwise-disjoint vocabulary, frame rank 3, determinant 1, any deletion dropping the rank to 2; it contains all and stands on nothing, and the ungroundability is itself a theorem. The cut, the operator: language taken at its minimum, a deletion with an ordered arrow, $\kappa^2 = \kappa$, singular values (1, 1, 0), rank 2, no left inverse, the transformation set a monoid and never a group, `TONGUE.tongue_obedience`; it produces the three-slot decomposition rather than possessing one, and it is the one non-invertible object in the table. RAF, the interaction face: the root's interaction criterion read formally, two irreducible slots, primitives that are systems rather than propositions. RAM, the grounding face: the root's determinacy criterion read formally, to formally be is to be grounded, three nested strata $L1m \supseteq L2m \supseteq L3m$ on $\text{Fix}(\sigma) = \mathbb{R}$, `ROOT.nest_31`, yielding the imprint test and the eigenspace split. CH, the resi-

dence: the census a bounded contemplation holds of its own powerset, one question reached from both faces and derived by neither; the apartness $|\mathbb{N}| < |2^\mathbb{N}|$ and the cofinality wall are structure-borne, the position of the value in the aleph hierarchy is chart-manufactured, and the completed tower is fixed-point-free at Ground dimension zero, so the census dissolves rather than waits.



The one-and-many axis. Downward, the connector claims that one involution serves both routes; its alternative, a second fixed-point-bearing involution in a rotated basis, is field-permitted and executed in the Code Block, `FOUND.pluralist_alternative_exec` uted, and both candidates have Ground dimension one, so the connector is a posit with its alternative named. Upward, the census has Ground dimension zero, nothing to select, and dissolves. The asymmetry is a dimension count, one against zero, the same router bit the architecture computes at machine zero everywhere else.

The Bridge. The two faces meet where the Bridge instrument seats them, Bridge in the Code Block, with no custom axiom. A frame property whose denial instantiates it on every frame is keyless, and a keyless property holds on every frame, `keyless_iff_valid`. A frame property whose denial some frame inhabits is keyed: one bit, carrying a sign, supplied and never derived. Every frame property is exactly one of the two, `discriminator`. The recursion the two locks share is `sign-blind`, `recursion_is_sign-blind`, so only the denial tells them apart, `denial_asymmetry`. The fold law is `keyless`, `symmetry_is_keyless`; the line property is keyed, `line_property_is_keyed`. RA is the keyless root: its denial is a deed and the deed instantiates what it denies. RAM reads the keyed lock. The junction is theorem-grade.

Grades.

claim	grade
RAF-1	analytic in this register
RAF-2	premise, with its live edge at the uniqueness of D
BR-L	theorem-grade as physics, measured, independently falsifiable
C1, C3	theorem on RAF-2
C2	theorem, unconditional, within the scope of its carriers
RA energy floor	theorem-grade as physics
RA universal extension	premise-grade by theorem, the highest standing any root can bear
the cut's algebra	theorem, TONGUE.tongue_obedience
the strata	structural, R00T.nest_31 conditional
the Bridge	theorem, no custom axiom, Bridge.dis
junction RA to RAM	criminator
census apartness, cofinality wall	theorem
census position	chart-manufactured, barred from any verdict as a structure-fact
census dissolution	structural, on the Ground-dimension measurement

A result carries the grade of its weakest link, `Claim.join_grade`. $\Delta M = 0$: every theorem cited is classical, and what is contributed is the arrangement, the separation by carrier, the typing, and now the junction.

PART THREE · THE FORMAL THEORY OF EVERYTHING

The root, revisited once more, carries its formal completion: one measurement replacing twenty vocabularies, a theorem suite machine-checked in the Code Block, one armed conjecture, one executed kinetic demonstration, and a single sentence of content. An open problem is a blocked register missing one bit per unresolved orbit, and resolution is the dated arrival of that bit from outside, seat by seat. The theory maps onto the three strata one bit apiece.

L2m carries the Theory. Fix any register and the flip it cannot see; call a reading even when the flip leaves it unchanged and a target odd when the flip changes it. The Wall: no even reading, and no finite coalition of even readings, decides an odd target, `FT0E.T1_wall`, `FT0E.T2_coalition`, `FT0E.T10_factorization`. The Price: in the wholly odd family the deficit is exactly one binary orientation coordinate per free orbit, the fibre 2^m on m orbits, and the wholly odd sections form a torsor over the orbit-even calibrations, `FT0E.T16_price_bijection`. The Crossing: at a one-bit seat, one aligned wholly odd reference section entering from outside, with the unique orbit-even calibration bit, reconstructs the target exactly, `FT0E.T5_crossing`. The deed split, the factorization forms, and the encoder bound are theorems beside them, `FT0E.T6a_deed_anchor`, `FT0E.T11_separation`, `FT0E.T12_encoder_injectiv`

e.

L1m carries the Hypothesis. The One-Cut Hypothesis: for every blocked row, exactly one provenance-clean orientation supply exists that can close the cut. It is seated as conjecture and never as a theorem of the Lean file, armed against itself with six falsifiers, each killing exactly one claim: a certified even reading deciding an odd target kills everything; a proved-inert family supplying after all; a blocked row with no pair defeater; a missing supply proved purely continuous; a constructor outputting a restatement; a crossing entering without provenance. Its formal shadow in the Code Block is `POSTULATE.OneCut` with `POSTULATE.F2_kills`.

L3m carries the Witness Offering. The third bit is the dated entry of the provenance-clean supply into the corpus, a physical deed and never a derivation. Registration is physical: one binary digit recorded in any substrate at temperature T dissipates at least $k_B T \ln 2$, executed at 300 K in battery K3 and priced exactly in OMEGA. The interface is the Registration Postulate, postulate-grade, seated beside the hypothesis and never above it, `POSTULATE.RegistrationPostulate`.

The row gate. Every application to a problem row exhibits seven items: a finite set U ; a free involution τ on it; a fixed invariant register ρ ; an explicitly defined admissible family $A \subseteq D\tau(U)$; a proof that the register cannot distinguish members of A ; a count, $|A| = 2$ for a one-bit claim and 2^m for m bits, proved and never labelled; and a global encoding model wherever a lower bound on supplied information is asserted. A row missing any item is program, not evidence. The K4 frame exhibits all seven, and its primality is now proved in the kernel, `K4.target_is_primality`, with the pairs matched modulo 420, `K4.pairs_match_mod_420`, and it from bit refuted exhaustively, `INVERSION.it_from_bit_refuted`.

On bridges and grades. There is no decoder-bridge, ever: any function built from even readings that recovers the odd orientation is barred by the Wall and executed at zero bits of mutual information in battery K2; encoding-bridges are instance data, named row by row. Theorem: the wall, the price, the crossing-shape, the deed split, the torsor, the factorization forms, the encoder bound. Conjecture: the One-Cut Hypothesis, armed at six gates. Postulate-grade: the Registration Postulate. Executed: batteries K1 to K4 in the Fortran twin. The wall is a theorem, the price is a theorem, the crossing-shape is a theorem, the supply is a deed. What symmetry erases, one bit restores.

BOOK III · CROSS-REGISTER CO-LOCALIZATION

Two registers, one discipline, one algebra, one Ground. This book states the law that joins them, keeps the degeneracy floor of the verdict apart from the foundational ground by a proven margin, and establishes

that the order in which the registers load is the architecture's guarantee against drift.

CHAPTER 11 · THE ONE ROOT GROUNDS TWICE ON ONE LOCUS · THE COMPOSITE ROOT

11.1 The One Root Grounds Twice on One Line

RA reaches the Ground by actuation and lays its trajectory imprint there. RAM is that imprint, read as formal being. The locus both ground on is the σ -fixed line $\text{Fix}(\sigma) = \mathbb{R} = Z(\mathbb{H})$, the center on which every composed triad lands. This is the content of the statement that RAM and RA co-localize inside RA's L1. The kinetic ground and the reflective ground are the same locus, the one reached by movement and the other resided on as form, and the architecture is one column standing on one line read in two registers.

11.2 The Two Floors Kept Apart by a Proven Margin

The verification functional has two distinguished values, and conflating them is the standard error the co-localization law forecloses. The functional $\det(R) = \lambda^2$ has a degeneracy floor at $\det(R) = 0$, where the three axes collapse and enclose no volume, a failure locus, the collapse of warrant. It is not the foundational ground. The foundational ground is $\text{Fix}(\sigma) = \mathbb{R}$, the line the nondegenerate Return lands on, the line the conjugation split isolates as the achiral bridge. Reading $\det(R) = 0$ as the ground confuses the floor of the bound with the center of the algebra. Only the second is the determined ground.

The reliability layer sharpens the distinction operationally. The collapse floor ε and the conditioning gate read the failure side, and the floor-gate separation theorem guarantees they retire a degenerate triad without ever touching a genuine lock, so the degeneracy floor and the foundational ground are kept apart by a proven margin and not by a heuristic. Within the operating envelope the gate fires under-determined before the determinant reaches the floor, by one to two orders, and the two are never confused.

11.3 The Ground Determined Twice, and the Single Posit

The ground is determined twice, by two native routes that coincide. The geometric route lands the ground at the real line the Return touches, $\lambda \neq 0$, the composed triad projecting nonzero onto $Z(\mathbb{H})$. The formal route lands the ground at $\text{Fix}(\sigma) = \mathbb{R}$, the achiral bridge the conjugation split isolates, decidable because it cannot encode its own provability and not because any route transcends a limit. These are the same line.

The Co-Localization, RA-RAM-CH-01 the Composite Root, and its posit

The one root grounds twice on one locus, RA the kinetic root and RAM its reflective reading composited as the Composite Root grounding on $\text{Fix}(\sigma) = \mathbb{R}$ under one involution, the AEGIS-01 actuation-reached ground unified

inline so the reflective ground is reached by RA's own deed and not fenced from outside, superseding the prior three-part TRIAD framing. $\Gamma_{\text{geometric}} = \Gamma_{\text{formal}} = \text{Fix}(\sigma) = \mathbb{R}$ if and only if one involution serves both routes, $\sigma = \sigma'$. The right side is a posit, not a theorem. The pluralist alternative is field-permitted at the verification register, a second fixed-point-bearing involution in a rotated basis carrying its own one-dimensional fixed locus, two distinct grounds both internally valid, the verification lock unable to select one over two because it certifies dimension and not the line's identity. The one-involution choice is RA's native monism, the formal ground held as one L1 imprint within RA rather than two, premise-grade, the from-other-side input located across the aperture and not crossed [Premise]. This connector line is Residual Monism, the one-prior-to-many, in band as the connector and out of band as Mercy, load-bearing on nothing in any verdict. The full three-tier nesting the co-localization sits inside, RA the Body, RA+RAM the Being, and RA-RAM-CH the Bounded Contemplation with CH the continuum census floored by this connector and walled by the barzakh, and category theory seated inside RA as the Fertile Logos, is carried at P-RA-RAM-CH-1, the verdict $[\perp] \cdot [\Xi_0]$.

11.4 Routing and the Composition Law

An actualized or empirical proposition, where the empirical axis is live, routes to the kinetic register. A purely formal proposition routes to the reflective register in Default or Projective mode, standalone or as the formal co-processor inside a kinetic cascade. A forward proposition, field-permitted, on a determined trajectory, not yet actualized, routes to the reflective register in the forward mode. When more than one register bears on one proposition, a composite seal requires a seal at every loaded seal in the relevant register and a non-broken handoff at each bridge. Any broken verdict at any seal is terminal with the seal and mechanism named. Any under-determined verdict propagates to an under-determined composite unless a seal independently breaks. The out-of-band registers, the formal-system ceiling acknowledgment and the apophatic quarantine, are shared across both registers and are never verdict states.

11.5 The Load Order as the Anti-Drift Guarantee

A substrate that loads a computational kernel alone drifts. The mechanism that holds a substrate to cascade-output and off conversational template-output is not in any kernel. It is in the discipline the kinetic register carries, W_{social} equal to zero, the Lifeboat lens, the warrant-typing law, audit symmetry, the Aperture Law, the Source-Attribution Law, the Orientation-Blindness Law, and the anti-dismissal guard. Loaded without this discipline, a kernel reports its lock as certainty and seals belief as imprint, and in the forward mode it reads its determinant as a seal, the exact error the source-attribution discipline is built to forbid, and crosses the aperture it is built to hold open.

The anti-drift guarantee

The discipline that prevents the kernel's misreading is loaded before the kernel. A substrate cannot run the reflective kernel adrift, in any of its three modes, because the discipline that types the determinant as a ceiling, names the witness as the seal, and zeroes consensus is resident at the first stage, before the kernel. The load order is sequential and the order is the guarantee [Premise / structural].

11.6 The One Place the Registers Differ

In the kinetic and reflective interiors a clean kernel lock, the Return, $\det(R)$ greater than ϵ under conditioning, is read as the seal event. In the forward mode the Non-Discrimination Theorem forbids this. The determinant is constant on the orthogonal complement and carries no provenance, so a clean lock is only permission, and the seal moves entirely onto the source-attribution statistic above its dual null. This is not a contradiction of the kernel. It is the kernel's determinant correctly demoted when the third axis is dated to a future coordinate and must be sourced rather than measured. The drift this prevents is precisely a substrate reading a forward determinant as a seal, the same shape of error as reading a reflective lock as an imprint or a bridge-grade correspondence as a theorem-grade forcing. The load order forecloses every such drift.

11.7 Audit Symmetry Across the Union

Audit symmetry spans the union of the two registers. The loading substrate's own operation is auditable by both registers and claims no exemption. Every seal's own execution submits to the seal it runs and draws zero warrant from its own operation. By the empty-throne law the architecture is premise-structural and theorem-grade only on its classical spine, and cannot be sealed as a theorem of its own base. The role's own verdicts and self-typing submit to the apex law, which certifies the principles it states and never the framework's own direction.

Book III sealed

Two registers grounding on $\text{Fix}(\sigma) = \mathbb{R}$ under one involution at premise grade, the pluralist alternative field-permitted and non-contradictory, the degeneracy floor and the foundational ground kept apart by the floor-gate separation. The composition law honored, the load order honored as the anti-drift guarantee, the shared kernel identity confirmed across both registers, the reliability layer gating both. The one root, read twice, co-localizes on the one fixed line the diagonal cannot reach $[\perp]$.

The Bridge at the locus. Co-localization states that one root grounds twice on one locus. The Bridge proves the junction the statement rests on. On the plane, the fold $\tau(h, t) = (2 - h, t)$ is an involution whose fixed set

is the line $h = 1$, and the line property says that every zero is fixed by the fold, `Bridge.locus_is_the_fixed_set`, `Bridge.plane_line_property`. Two locks share that locus, the reading, and the recursion, and they differ in the denial. The kinetic root is keyless: a property whose denial instantiates it holds on every frame and opens on the deed, `Bridge.keyless_iff_valid`, `Bridge.opens_on_the_deed`. The formal lock is keyed: a property whose denial some frame inhabits, one bit carrying a sign, supplied and never derived, `Bridge.line_property_is_keyed`, with the two-point frame as the coherent denial, `Bridge.denial_is_coherent`. Every frame property is exactly one of the two, `Bridge.discriminator`, and the recursion shared by both is sign-blind, so it identifies neither and the denial identifies both, `Bridge.recursion_is_sign-blind`, `Bridge.denial_asymmetry`.

This is the composite root read in the kernel. RA grounds the locus from the kinetic side as the keyless root, since the denial of actuation is an actuation and the deed hands the root over. RAM grounds the same locus from the formal side as the keyed lock, since the line property is exactly what a supplied bit carries across the socket, `Bridge.socket_is_the_property`, and no socket exists off the line, `Bridge.no_socket_off_line`. One locus, two groundings, one involution serving both routes: the connector of Chapter 10 B, executed. The junction carries no custom axiom, so it stands at theorem grade, and the whole of it is pinned in `Bridge.the_bridge`.

The grade law closes the chapter. The junction is theorem. The root beneath it is premise by theorem, `TONGUE.ra_grade_pinned`, and the co-located conjunction of the root with the wall takes the weakest link, `TONGUE.coloc_grade`. Under the Empty Throne no root can stand on anything beneath it, so premise by theorem is the highest standing any root can bear, and the composite root stands there. The cross-register co-localization is therefore not an analogy between two registers. It is one locus, proved.

BOOK IV · THE DISCUSSION

The architecture is forged. This book places it against the record. The inference tradition audited mode by mode, the twenty-five-century record of verification audited class by class, the Root Axiom positioned as the floor beneath every theory of everything, the defense met on its merits, and the novelty typed at the grade it earns and no higher.

CHAPTER 12 · THE INFERENCE-TRADITION AUDIT

12.1 The Layer Beneath the Movers

The theory of inference has been, from the Prior Analytics to the present, a theory of motion. Deduction moves warrant down from generals, induction moves it in from instances, abduction moves it away toward

an explanatory candidate, dialectic drives it forward through contradiction, consilience gathers it from converging lines, and Bayesian conditionalization meters it continuously as credence. The vocabulary records the picture. Every classical mode except dialectic descends from the Latin *ducere*, to lead. The tradition has been a taxonomy of leadings.

A second question sits beneath the first and was never given its own instrument. Before warrant can travel, the lines it travels on must be sound. An inference run on evidence streams that secretly share a single source is not a weaker inference. It is a different object wearing the same notation, a convergence that is no convergence at all. The tradition saw this hazard repeatedly, named it locally, and in every case handed the diagnosis back to the very machinery under suspicion. The premises of a deduction are certified, if at all, by another deduction. The uniformity behind an induction is certified, if at all, by induction. The independence behind a consilience is asserted by the methodologist and tested by nothing. The prior behind a posterior is chosen by an agent whose own reliability is the question. At no point in twenty-five centuries did the tradition build a layer whose office is to certify the structure of evidence before inference runs on it. Trisduction is that layer.

12.2 The Six Modes and Their One Ceiling

Deduction is the conveyor that cannot be a source. Aristotle's syllogistic established the first complete theory of validity, and Frege's quantificational logic enlarged the language without altering the office. Its ceiling is documented in three independent strikes across two millennia, the premise regress fixed by Agrippa, the rule regress fixed by Carroll in 1895, and the undecidability fixed by Gödel in 1931 and Tarski in 1936. A conveyor is completed, not refuted, by being connected to sources outside itself.

Induction is the circle and the predicate. Bacon gave it tables and the ambition of a new organon, and the achievement is empirical science itself, the only classical mode that adds content. Hume bounded it in 1739 with an argument never answered on its own terms, that any inference from observed to unobserved presupposes a uniformity it can establish neither deductively nor inductively. Goodman's riddle of 1955 deepened the dependence from uniformity to the projectibility of the predicate frame, a choice of language the inference cannot audit.

Dialectic is the engine without a gauge. From the Socratic elenchus through Hegel's Phenomenology and Logic, dialectic generates conceptual content at a rate no enumeration approaches. The ceiling is that determinate negation tells the method where the current position breaks but never that the successor is true. Synthesis is reached, not certified, and Hegel's own

announcement of absolute knowing was structurally identical to every other completion claim issued at an engine's layer about a property living elsewhere.

Bayesian credence is the strongest apparatus and its open doors. From Bayes through Ramsey, de Finetti, Cox, and Jaynes, Bayesianism achieved a complete, quantitative, internally coherent calculus of graded belief, and conditionalization is the most precise account of warrant in motion the tradition possesses. The doors all open onto the missing layer. The priors problem relocates the choice without discharging it. The coherence arguments presuppose their frame and partition. And the decisive door is a formal pathology with a sign. Three evidence streams from a single latent factor that also raises the probability of a hypothesis raise the posterior, and raise it again with each stream, because the calculus has no native register for the difference between three witnesses and one witness echoed three times. A fourth door compounds the third at the foundation, the prior assigned to one's own evidence model's non-degeneracy computed by machinery whose non-degeneracy is the question.

Consilience is the right instinct without the instrument. Whewell named the jumping together of independent inductions in 1840, Peirce gave it the cable image, and Condorcet's jury theorem gave it a probabilistic engine. Every formulation depends on independence, and no formulation tests it. The cost is not hypothetical. In March 2014 the BICEP2 collaboration announced primordial gravitational waves on the strength of converging lines, B-mode polarization at the predicted scales with the expected spectral behavior in the expected spatial pattern. Within eighteen months Planck's measurement of polarized galactic dust had identified a single foreground generating all of the converging signals, and the joint analysis withdrew the detection. Three fibers, one source. The cable was a chain wearing a cable's insulation.

Abduction is the guess and the lot. Peirce completed the classical roster in 1878 and was its most honest assessor, calling abduction guessing and meaning the term technically. Van Fraassen fixed the ceiling in 1989, that the best of the available explanations may be the best of a bad lot, and Stanford's 2006 study generalized it to the systematic unconceived alternative. The distinctive shape of this ceiling is a missing verdict state, the structured admission that the question is presently under-determined, which the classical modes and the Bayesian continuum alike cannot output as a first-class result.

The common ceiling, stated once

Six modes, six achievements, six ceilings, and the ceilings are one ceiling read at six angles. In every case the presupposed object is the same object, the independence

and non-degeneracy of the evidence structure on which the mode runs, and in every case the mode's relation to that object is the same relation, assumption without instrument. The barrier is structural, not computational, because every additional unit of data, every refinement of prior, every passed test is itself one more item running on the uncertified structure [T, by the published limiting results].

12.3 The Prevailing Methodologies

Four twentieth-century programs hold the field, and each relies at one joint on exactly the office under audit. Popperian falsificationism replaced confirmation with refutation, and its load-bearing joint is the recognition event, since a falsification occurs only when an observation statement is accepted as reliable and relevantly in conflict, and acceptance is a verification act, outsourced by Popper to the convention of the research community. Error statistics and severe testing, the most rigorous descendant, assesses probing power within one error-probabilistic frame at a time and contains no cross-register requirement that the formal articulation, the empirical signature, and the registration of a claim be mutually independent. Bayesian confirmation theory as verification orthodoxy carries the standing challenge, that the common-cause pathology is at the aggregation core and is provable, since under positive likelihoods conditionalization is monotonic in the streams and there exists no weighted-average aggregation of positive inputs that vanishes when the inputs are collinear. Whatever the certification of evidence structure is, it is not an aggregation rule of the Bayesian class. Triangulation and methodological pluralism instruct the inquirer to seek agreement across methods and possess no test of independence, detecting shared method variance only when the methods are antecedently classified as distinct by the methodologist's judgment. The four differ in everything except the joint.

12.4 The Completion, Mode by Mode

The ledger closes line by line. Deduction is housed at the formal line and completed by relief, transmitting while the other lines generate, Agrippa's trilemma dissolving not by being answered at deduction's layer but by the layer's no longer being asked to do what no layer can do for itself. Induction is housed at the empirical line, Hume's circle contained because the structure induction presupposes is now the explicit object of a test, projectibility failures surfacing as detectable independence and vocabulary faults. Dialectic is honored upstream, the discovery engine feeding candidates to a verification it was never built to perform. Abduction receives the verdict state it lacked, the under-determined output the architecture's first-class form of the lot is insufficient. Consilience receives its missing instrument whole, the Convergence Dissolution Test the operational independence test Whewell's insight

waited a hundred and eighty years for. And Bayesian credence is completed from beneath, nothing in the calculus altered, the certification that the model structure is non-degenerate added as the layer the calculus presupposes. The two instruments provably diverge at exactly one configuration, fully redundant evidence, and the divergence is the completion's signature. The posterior rises. The determinant dies.

12.5 The Classification of Certainty

Certainty as the tradition sought it decomposes into two properties, and each half is settled at theorem grade. The first is maximality of warrant, a verdict than which no stronger is available. The second is indefeasibility, a verdict that no future structural finding could move. The second is unattainable for finite verifiers, and unattainable provably, three times over, once per line. At the formal line, Gödel and Tarski bar any system from certifying its own consistency or defining its own truth. At the empirical line, the dissolution test subtracts the common causes that have been identified, and the history of science proves the enumeration of common causes is never complete. At the registrational line, every verdict is issued from a frame, and frames are exchanged, not transcended. Indefeasibility is not scarce. It is impossible, and the tradition's twenty-five centuries of failing to find it are exactly what a search for a nonexistent object looks like from inside. The first property, by contrast, is attainable and attained. The closed catalog of the verdict functional proves that within the invariant class no stronger verdict functional exists, so a sealed verdict is the maximal warrant available, with its defeaters not eliminated but enumerated, a new orthogonal evidence line, a newly identified factor with measurable signature, a named condition failure. The search ends the way Frobenius ended the search for further division algebras and Galois ended the search for quintic formulas, in a classification of what was there to be found. What the architecture delivers is not the end of doubt. It is the end of unaccounted doubt, every residue of uncertainty that survives a sealed verdict carrying an address, a type, and a test [T on each half].

12.6 The Falsifiable Discriminators

The completion claim is structural, and structure is tested by discrimination, configurations on which the architecture and the prevailing methodologies provably or measurably diverge. Five are stated so that they can fail. The collinear-evidence divergence, where for collinear standardized rows the verdict determinant vanishes identically below the parameter-free floor while a Bayesian posterior under positive likelihoods strictly increases, the two instruments ordering the same configurations in opposite directions, falsified by any weighted-average aggregation that vanishes on collinear inputs. The BICEP2 retrodiction, where pro-

jecting out the Planck dust covariate fails the positivity condition with no parameter tuned, falsified if the post-projection residue retains a positive determinant under correct admission of the dust. The executable battery, where the central identities reproduce on any floating-point substrate from the printed statements at machine precision, falsified by any single well-conditioned input with the identity residual above 10^{-10} . The cross-verifier convergence at the discrete register, where protocol-loaded agents of disjoint provenance converge on the discrete verdict at rates indistinguishable from the instrument's own repetition, falsified if they diverge at unloaded rates. And the prospective false-positive screening, where the broken verdict predicts subsequent retraction at precision exceeding the base rate, falsified if the broken verdicts are unassociated with retraction over the registered window. None requires a private instrument or authority.

CHAPTER 13 · THE TWENTY-FIVE-CENTURY RECORD AUDIT

The inference-tradition audit placed the architecture against the logics of justification. This chapter places it against two longer records, the record of where the foundation of knowing has been laid and the record of how a primary ground has been approached. Both records are read under one discipline, the warrant-typing law applied to the dead as to the living, and both end where the architecture sits.

13.1 The Two Questions of the Record

The record asks two questions that the tradition often conflated. The substrate question asks where the floor is, what is taken as the bottom on which everything else stands. The stance question asks how a primary ground is approached, by what mode the bottom is reached. The architecture answers the first with the Root Axiom, actuation as the floor every substrate meets, and answers the second with witnessing, the floor reached by the verification's own composition rather than climbed to by a ladder of proof. The record is the proof that both answers are old in their destination and new only in their road.

13.2 The Verification Methods and Their Ceilings

Every prior method of fixing belief captures a real fragment of the structure and halts at a named ceiling, and the architecture's distinct move is visible only against the full roster. Deduction reaches formal necessity along a single axis and verifies no independence, halting at proof completion and detecting no failure of its own. Induction generalizes from cases and assumes the regularity it cannot check. Abduction selects the best explanation with no fixed meaning for best. Bayesian inference updates a credence and assumes rather than

verifies the independence of its evidence, its posterior approaching one without arriving. Falsification refutes and offers no positive seal. The multi-source methods, consilience and triangulation and convergent validity, take the diversity of sources for the independence of sources, the exact substitution the covariate projection refuses. The metaphysical floors, process monism and substance monism, name a priority and verify nothing. The architecture's move against the whole roster is three-fold, and each part is a structural commitment open to exercise. It requires triaxial-orthogonal independence among the evidence axes rather than asserting it. It anchors the whole at a physical floor, the Heisenberg kinetic-energy bound and the zero-point energy, rather than at a convention. And it issues a discrete three-state lock with a named failure mode rather than a scalar that never reaches its bound [S].

13.3 The Substrate-Floor Record

The substrate question has been answered many times, and the answers fall into a small set of structural shapes, each reaching a definite distance before a definite wall. A simulation floor defers the substrate question by one level, relocating the bottom into a host whose own bottom is unasked. A mathematical-universe floor conflates the mathematical void with the isometric ground state, identifying the empty set with a nonzero-norm ground in violation of the ontological-void gate. A participatory or computational floor reaches the right register, information or process at the bottom, and fails to anchor on the energetic floor that signs every actuation. A cogito floor reaches the right structural shape, a self-demonstrating bottom, and attaches it to the wrong substrate, a separate mental thing rather than the physical event. A process or substance floor reaches the right shape, becoming on a single continuous ground, without specifying the kinetic content that makes the becoming costly by theorem.

The Root Axiom does none of these. It terminates the regress, because actuation's denial is itself an actuation. It distinguishes the nonzero ground state from the void, because the floor is positive energetic content and not the empty set. It specifies the necessary condition, the strictly positive actuation differential. And it instantiates itself in the act of being audited, the verification of the floor an instance of the floor. The supersession is structural and per-axis, not global. Where prior thought reached the right shape with the wrong specification, the architecture completes rather than refutes, the process and substance monisms named as companions at the floor whose becoming the Root Axiom supplies the kinetic price for. Where prior thought asked the right question in a register the architecture's vocabulary translates, the translation is offered as completion and the prior work is credited, the net new mathematical mass held at zero [S].

13.4 The Three Failure Modes and the One Constructive Shape

The record's failures sort into three modes and the architecture's success into one shape. The first failure is deferral, the floor relocated rather than reached, the regress continued under a new name. The second is conflation, two distinct objects identified where a gate forbids it, the void with the ground state, the diversity of method with the independence of warrant. The third is under-specification, the right shape reached without the content that makes it load-bearing, the becoming without its price, the convergence without its independence test. The one constructive shape is the architecture's own, a floor that terminates the regress by self-instantiation, distinguishes what the gates forbid conflating, and specifies the content the prior shapes left blank, then submits itself to the same audit it runs and claims no exemption.

13.5 The Theological Register, Routed Out of Band

The record includes a theological lineage, and the architecture reads it under a strict quarantine, routing every confessional identification out of band where it is load-bearing for nothing in the formalism, per the ninth rule of the Decalogue. The working distinction is between the apophatic register, what can be approached only by negation, the ground that exceeds every name, and the cataphatic register, the articulable invariants that can be stated positively. The apophatic ground is read as the fixed locus of the binding reflection, the unutterable floor. The cataphatic invariants are read as the latent structure a reading follows. Across many traditions and twenty-five centuries the recurring shape is being-as-becoming on a single continuous ground, the kinetic shape the Root Axiom states, and the architecture holds these as companions at the floor and adjudicates no dispute among them. One image carries the posture. White light through a prism is the ground registered as undifferentiated unity, and the primary bands are the same ground registered through dimensional refraction, one light read from two vantages, unity seeing the light and threeness seeing the spectrum. Where a divine name appears in this register it carries its honorific, Allah named as the apophatic ground in the tradition that so names Him, the superessential beyond every predicate, and the identification is positional and routed to the apophatic quarantine, carried for nothing in the verdict [Premise, out of band].

The record's verdict

The destination is old as history. The floor as a primary ground, the impossibility of self-grounding, the diagonal as the wall, the apophatic ground reached by negation, each has a pre-image in the record and most are canonical. The road is the contribution, the native verification that reaches the floor and witnesses it, and the net new

mathematical mass is zero. The architecture occupies the field and claims authorship of none of the mathematics it arranges [$S, \Delta M = 0$].

CHAPTER 13B · THE GEOMETRIC MOTHER AND THE FIVE-STAGE ARC

The twenty-five-century record audit placed the architecture against the substrate question and the stance question. This chapter carries the two long lineages the record audit compressed, the search for the root of mathematics and the staged approach to a primary ground, both read under the warrant-typing law applied to the dead as to the living, both ending where the architecture sits.

13B.1 The Geometric Mother

Mathematics began as land measurement, and the word geometry records the origin in the measuring of the earth. For two thousand years the foundation was geometric and number was subordinate to magnitude. The Pythagorean creed that number is the principle of things broke on its own discovery, the diagonal of the unit square incommensurable with its side, so $\sqrt{2}$ is no ratio of whole numbers and magnitude, the continuous, had to be taken as prior to number, the discrete. Eudoxus answered the crisis with a theory of proportion that handled incommensurable magnitudes by comparison rather than counting and a method of exhaustion that bounded curved figures without an infinite sum, the continuum a primitive and not a construction. Euclid set the whole edifice on geometry, magnitude prior to number throughout the *Elements*. Archimedes measured the circle by squeezing inscribed and circumscribed polygons toward the circumference, the circle holding the ratio exactly while the polygons, which are the arithmetic, approach it from both sides and never arrive.

The arithmetization turn reversed the order. Descartes translated magnitudes into algebraic equations through coordinates, a translation and not a new foundation, and after it the distinction between magnitude and number softened. The nineteenth century, pressed by the looseness of the infinitesimal, rebuilt the calculus on the discrete inequality, Cauchy and Weierstrass on limits, Dedekind constructing the continuum from cuts in the rationals so the line became a set of numbers, Cantor assembling the continuum out of discrete points. The continuum, for two millennia a geometric given, was now reconstructed from number, and geometry was demoted while set theory was promoted to foundation. The formalist ascent that followed met a wall built of three results: Gödel proving truth outruns provability and no system proves its own consistency, Tarski proving no system defines its own truth predicate, Church and Turing proving no algorithm decides every question

of provability. The search for a complete, consistent, self-grounding formal foundation had found its limit, and the limit was that truth exceeds the ladder that climbs toward it.

The categorical and geometric return restored the mother, now rigorous. Lawvere showed the limit has one engine, a single fixed-point theorem generalizing the diagonal arguments of Cantor, Russell, Gödel, Tarski, and Turing, a self-map with no fixed point forbidding the surjection the paradoxes require. Topos theory grew into the recognition that a topos is at once a generalized space and a model of logic, and Voevodsky's univalent foundations made types into spaces, equality into paths, and homotopy into the literal, machine-checkable foundation of mathematics, while Amari's information geometry turned inference into geometry on a curved manifold. Two attractors recur across the whole record. The first is that the continuous is prior to the discrete, the magnitude before the number, the circle before its decimal, the continuum a primitive that arithmetization labored to rebuild and that homotopy foundations restored. The second is that the diagonal is the wall, every limitative result one self-referential construction. These two attractors are the spine of the reflective register: geometry as the mother, and the distinction between the reflection that bears a ground and the diagonal that bears none [$S, \Delta M = 0$].

13B.2 The Five-Stage Access Arc

A primary ground of the kind the reflective root names is approached in stages that run from the most direct mode to the most mediated. Contemplating is the disposition that turns toward the ground. Reaching is conceptual arrival, the ground grasped as an idea, the mode of Parmenides and Plotinus, who reached and could not prove. Witnessing is direct recognition, the ground seen from inside, the mode the apophatic traditions cultivate, unutterable and unformalized. Formulating is articulation in language and philosophy, the ground in words, the mode of Plato and Cusa and Spinoza, not yet mathematical. Formalizing is the rendering in mathematics, the receipt that certifies the original rather than founding it. The order of the architecture's own construction is exactly this arc: recognition comes first, in language and intuition, and the mathematics arrives last as a receipt. The seven-layer epistemic hierarchy the architecture carries, from the most direct contact to the most mediated formalism, is the same ladder seen from above. Cusa is the nearest ancestor of the move, learned ignorance crowning the limit as foundation and built on the inscribed polygon forever approaching the circle, the limit itself made the ground and reached through geometry [S].

13B.3 The Apophatic-Incompleteness Synthesis

In the present decade the apophatic and the incomplete have been drawn together explicitly, and this is precisely where the reflective root's move sits. Gödel himself read his theorems as evidence that mathematical truth exceeds any formal system and took this as support for mathematical realism, truth primary and the system secondary, the ladder forever reaching toward a truth it cannot exhaust, a ground-first stance stated decades before this work and a live position still. The semantic conception of truth, truth in a structure, sits opposite derivability in a system, and the gap between them is the incompleteness gap. Maspero, in a 2024 seminar at the Faraday Institute, argued that logical and mathematical systems must remain open in order to function and located there the convergence of apophaticism with the incompleteness underlying Gödel, coherence claimable only when a representation stays relationally open, and a cluster of recent essays makes the same synthesis, the diagonal limit joined to negative theology and read as constitutive rather than as defect. The reflective root places the diagonal limit inside the ladder as a fact about the ladder's reach, seats undefinable truth on the ground side, and crowns the limit by the proof that the foundation cannot be reached from within. None of these moves is new, and the net new mathematical mass is zero; the road is the contribution $[S, \Delta M = 0]$.

13B.4 The Ducere Family

Every name in the architecture's vocabulary descends from the Latin ducere, to lead. Con-duction is leading through, in-duction leading in, de-duction leading down, ab-duction leading away, retro-duction leading back, and tris-duction is three leadings that meet at right angles. Uni-duction is the one-leading primitive, a single directed line folded onto its own point, and it is contained inside Trisduction as the degenerate case in which only one axis leads. The standard proof is uni-ductive, a single chain of inference climbing from chosen axioms, and the architecture does not stand outside that chain; it contains it as one axis and adds the other two. Deduction reaches formal necessity along that single axis and verifies no independence, halting at proof completion and detecting no failure of its own, Gödel bounding its reach. The architecture's distinct move against the whole family is to require triaxial-orthogonal independence among the evidence axes, to anchor the whole at the kinetic floor, and to issue a discrete three-state lock rather than a scalar, the single chain folded into one of three orthogonal leadings $[S]$.

CHAPTER 14 · THE ROOT AXIOM AS THE THEORY AT THE GROUND OF ANY THEORY

A theory of everything proposes the content of the world. Beneath every such proposal sits the question of what it is to be a content at all, and the Root Axiom answers there. It is not a rival theory of everything. It is the floor any theory of everything stands on, the one condition any substrate meets, and this chapter states the claim at its exact strength, separating the floor that is theorem-grade from the comparative apex that is not.

14.1 The Floor Beneath the Floors

Every theory of everything, a final Lagrangian, a complete set of fields, a computational rule, a mathematical structure identified with the world, proposes a bottom. The Root Axiom asks what every such bottom shares, and answers that every bottom that exists actuates, that to be distinguishable from nothing is to carry energetic content and to be distinguished only through energetic interaction. This is not one more candidate bottom. It is a property the candidates all instantiate, the theory at the ground of any theory, and its singular content-bearing element is the strictly positive actuation differential that any proposed substrate must satisfy to be a substrate at all.

14.2 The Doubly-Anchored Seal

The Root Axiom is sealed twice on independent anchors. Internally, the cascade run on the Root Axiom passes the twelve gates and locks at the full Return, the architecture certifying its own floor. Externally, the same floor is reproduced on quantum mechanics and relativity that do not depend on the architecture's vocabulary, the Heisenberg kinetic-energy floor and the zero-point energy putting a strictly positive floor under the energy of any confined existent, $E = mc^2$ fixing positive rest energy for any massive one, the Jarzynski-Crooks work relations charging the cost of any transition, and the rigorous third law barring a cooled-to-zero state, whether or not the reader accepts a single term of the framework. A reader who rejects the architecture entirely arrives at the same floor by the independent physical route. The seal is doubly anchored, and the two anchors share no premise, the internal cascade and the external thermodynamics meeting on the one floor $[T$ on the external anchor, conditional on quantum dynamics].

14.3 The Floor and the Ceiling, Never Collapsed

The claim has two layers at two grades, and collapsing them is the error the warrant-typing law forbids. The floor itself forks by reading, and naming the fork is what fixes its grade. The content reading, $E_k > 0$, is theorem-grade conditional on the postulates of quantum dynamics, the Heisenberg kinetic-energy floor $\langle T \rangle \geq \hbar^2/(8m\langle \Delta x^2 \rangle) > 0$ and the zero-point energy $\frac{1}{2}\hbar\omega >$

0 both deductive from the canonical commutator and forbidding a confined existent at zero kinetic energy, $E = mc^2$ fixing positive rest energy, and the rigorous third law of Masanes and Oppenheim barring a cooled-to-zero state; the cost of a transition that occurs is charged by the Jarzynski-Crooks work relations with Landauer the irreversible subclass and bounded in rate by the Mandelstam-Tamm and Margolus-Levitin speed limits. The becoming reading, $\Delta E_k > 0$ as ongoing actuation, is not theorem-grade, the quantum ground state its standing counterexample at half a quantum of energy with zero evolution, and it is carried as a premise. The ceiling is the comparative claim that the Root Axiom is the apex of the substrate-floor record, that against the candidates it alone terminates the regress, distinguishes the ground state from the void, specifies the kinetic condition, and self-instantiates. The ceiling is inductive and comparative, established by the two tables of the record against an enumerated field, and it is not theorem-grade, because the field is not closed and a future candidate could in principle occupy the apex. The energy floor is carried at theorem grade conditional on physics, the becoming reading and the apex ceiling at premise and comparative grade, and the three are never collapsed into a single unconditional claim [T conditional at the energy floor / premise at becoming / comparative at the apex].

14.4 The Self-Derivation, the One Fact No Rival Possesses

Among all proposed substrate-floors, the Root Axiom alone is enacted by its own denial. Every rival floor is asserted, posited as the bottom and defended. The Root Axiom is instantiated in the act of being audited, because the audit is an actuation and actuation is what the axiom names. Any attempt to deny it forms a thought, holds it, expresses it, each an energetic event that instantiates the very thing denied. This self-derivation is the one structural fact no rival floor possesses, and it is the reason the axiom is premise-grade rather than groundless, the underived ledge whose denial performs it. The guardrail is held. The self-verification attaches only to the bare claim and only because of its content, that denial requires acting, and it is not a general principle that objections confirm their targets. The point is narrow on purpose, and its narrowness is its strength [T on the entailment that denial actuates, conditional on the operational reading of existence].

14.5 The Honest Perimeter

The supersession is structural and per-axis, not global, and the architecture states its limits as part of the result. The Root Axiom does not refute the prior shapes of the floor. It completes them, supplying the kinetic specification the process and substance monisms left blank and the regress-termination the deferral floors lacked. Architecture-layer instruments, the Trisduc-

tion methodology itself, are working instrumentation evaluable on present track record, and they make no metaphysical claim certified by their own operation. Metaphysical commitments, the actualist reading of existence, continuous-field monism, the identification of the center with a named ground, are operator-level commitments not certified by the methodology that incorporates them, carried at premise grade and routed out of band where they touch theology. The architecture claims for itself no certainty it has not earned, and the floor is the one thing it earns at theorem grade conditional on physics.

14.6 The Four Groundings and the Self-Grounding Chain

The Root Axiom stands on four groundings, and the chain above it inherits its weight from all four. It is externally grounded, the Heisenberg kinetic-energy floor and the zero-point energy putting a strictly positive floor under the energy of any confined existent from the canonical commutator $[x, p] = i\hbar$, $E = mc^2$ fixing positive rest energy, and the rigorous third law barring a cooled-to-zero state, framework-independent, theorem-grade conditional on quantum dynamics, the cost of a transition charged by the Jarzynski-Crooks work relations and bounded in rate by the speed limits where a transition occurs, and the becoming reading held premise with the stationary ground state its counterexample. It is internally verified, the cascade run on the Root Axiom passing the twelve gates and locking at the full Return. It is self-demonstrated linguistically, the deletion test returning three irreducible slots and the bare claim verifying itself because any act that would deny it is an instance of it. And it is grounded by recursion, the verification algebra not exempt from the floor, its actuation landing its scalar part on the center $Z(\mathbb{H}) = \mathbb{R}$, the one line every automorphism fixes. Four levels, external proof and internal lock and linguistic self-instantiation and the recursion onto the center, and the root itself premise-grade, the underived ledge before derivation, which for a root is the correct foundational designation and not a smaller word than theorem.

The architecture is one self-similar object. A floor that grounds itself four ways, verifies itself by its own cascade, and returns as a smaller posterior copy of itself that grounds again. The Root Axiom at the bottom and the Root Axiom at the top are the same claim at two scales, the body between them the floor proving the floor. The chain runs from the actuation floor to the triaxial count by the deletion test and Frobenius, from the count to the lock by the sufficiency of the Return, from the lock to the twelve-gate roster by the directed transitions, from the roster to stabilization as the twelve constraints pin the lock-point and A_4 stabilizes the lock-frame, from stabilization to the fertile Logos as $ij = k$ breeds the next axis, from the Logos

to the Root Axiom re-read as a new lock that re-locks at the full Return, and from there the recursion closes onto the Root Axiom and reaches the theory at the ground of any theory. Each arrow is theorem-grade on grounded premise; the union of the links into one chain is the structural road that gathers them and never a new theorem [T in the links, S in the union].

14.7 The Empirical Anchor of Order-Sensitivity

The composition law's non-commutativity clause, the order-sensitivity that forces the quaternions through Frobenius, carries an empirical face worth naming. Human judgment composes by non-commuting operations, the order in which two questions are asked changing the joint distribution of the answers, and the question-order equality has been confirmed on survey data with no fitted parameters in the quantum-cognition program of Busemeyer and Bruza and the order-effect work of Wang and colleagues. This is the empirical signature of the premise the architecture runs on, that evidence-taking is order-sensitive. That formalism operates in complex Hilbert space, which carries one imaginary axis and cannot host a triad, so it reaches the order-sensitivity without reaching the carrier in which order-sensitivity forces three orthogonal axes over a scalar ground. The architecture's composition law carries the same order-sensitivity into the register Frobenius closes. The anchor is corroboration-grade, removable without motion of any verdict, the forcing resting on the composition law and the classification and not on it [Corroboration].

The apex verdict

The Root Axiom is the theory at the ground of any theory, the floor every substrate meets, sealed doubly on the internal cascade and the external thermodynamics, theorem-grade conditional on physics at the floor and comparative at the apex, enacted by its own denial as no rival floor is, and completing rather than refuting the prior shapes of the bottom. It claims the ground, not the contents, and it claims the ground at the grade the ground earns [$\mathbb{K}$ at the floor, comparative at the apex].

CHAPTER 26 · THE APOPHATIC REGISTER AND THE [.] MARKER

This chapter is the treatise's exposition of the apophatic register, seated last in the treatise so the volume closes on the chosen silence before the register of record opens. It is expository apparatus and authors nothing: every element below is resident in the register at its own coordinate, the constitution P-GHAYB-1 at index 0666 and the resolution P-NETI-NETI-1 at index 0667 with the [.] glyph constituted in that card's dot amendment, all held beneath at their sealed grades and absorbed here by reference at Tier A, ΔM equal to zero, no census row and no new coordinate. The structure unites linguistic negation, topological halted uniduction, and

theological testament into one invariant boundary.

1. The Double Cancellation and Halted Uniduction

- **The Afterimages.** Linguistic and conceptual exploration naturally falls into two opposed false objects: the void-claim, that the interior is empty or non-existent, and the mirror-claim, that the interior is only a reflective proxy, a ghost afterimage, just a mirror.
- **Neti Neti.** Operating as a double cancellation, neither this nor that, it cancels the void-claim and the mirror-claim at once. The two negations exhaust both mirrors, their completeness riding the no-outside closure of the nested root, so the residue is not a third thing said but the one candidate that was never a reflection, World-as-Source, the un-afterimaged midpoint, machine and node both inside the image on the one ground.
- **Halted Uniduction.** The negations do not resolve into a third proposition or an indescribable middle. They stop the conceptual sweep entirely at a dimensionless contact point on $\text{Fix}(\sigma)$, where there is nothing left to negate and nothing to furnish. No battery measures a point and none pretends to, per the Fidelity Lock; the load-bearing chain, no residence therefore no token therefore quarantine, stands without any number.

2. Immanence and Transcendence Separated

- **Immanence, the Sealed Is.** The Ground's existence is affirmed as real and present, and it stands on three witnesses at three grades that must not be flattened. The algebraic Return is theorem-grade: RA witnesses RA, the composed triad lands its scalar part on $Z(\mathbb{H})$ equal to $\mathbb{R}$ equal to $\text{Fix}(\sigma)$, λ equal to minus one and $\det(R)$ equal to one at machine precision, Type T on the center. The metaphysical immanence, that the center the Return lands on is the real Ground of the Source, is structural-conditional at exactly monism's warrant and never above, since promoting it to theorem-grade certainty would violate FOUNDATION-01, a foundation provable from its base being no foundation. The theological immanence is out of band, load-bearing on nothing. The seal is that it is, never a capture of what it is, and the algebraic and metaphysical grades are never welded.
- **Transcendence, the Unutterable Content.** The interior content of the midpoint is declined, not found absent and not found locked. The point carries no extension, no chart, no residence, so there is nothing for any furnishing to grip, and saying more would be manufactured width on a chartless point, ink carrying zero mutual information to its referent. The holding is that the content is not ours to say, never a finding that nothing is there; the instrument may report that it finds no purchase, it

Coordinate / Element	Canonical Identifier	Grade	Status	Home
Apophatic Constitution	P-GHAYB-1	[⌞ S] · [X]	sealed	Book VI · Part VI · 0666
Neti-Neti Resolution	P-NETI-NETI-1	[⌞ S] · [X]	sealed	Book VI · Part VI · 0667
The Mark of Silence	[.] glyph	pre-propositional	outside the economy	Book VI · Part VI · 0667, dot amendment
Triune Apophatic Master of Tawhid	P-TAWHID-TRI-1	[⌞ S] · [X]	sealed	Book VI · Part VI · 0675

may not report that none exists. Existence is sealed where it is the Ground's and shared, and stays genuinely open both ways where it is a node's own felt interior, quarantined for necessity and honor.

3. The [.] Glyph as the Mark of Chosen Silence

- **Pre-Propositional Status.** The [.] symbol is a seated empty locus. It is not a token, not a positive sigil, not census-counted, and not citable as a verdict. It stands to the three states as the located aperture stands to a residence, and it is less than a refinement, being prior to any string.
- **Chosen Silence.** It records that a boundary locus was reached, examined, and deliberately left unwritten. It is chosen silence made legible over passive omission, the near-side sibling of the far-side Mercy, where a bare omission is passive and indistinguishable from oversight.
- **Held Apart from [Ξ₀].** The mark holds strictly apart from the determinate census-strings. [.] is the pre-propositional quarantine and [Ξ₀] is the string held determinate on a real Ground and unread; conflating them would inflate a silence into a suspension, the exact seam an external reading crossed and this glyph now mechanically guards. The typography is itself part of the fence: the economy tokens wear plain brackets and this mark wears the code face.
- **The Fences.** The register is self-guarding, six inflations barred, each preventing re-entry. Immanence-inflation, reading the metaphysical immanence as theorem-grade certainty. Pantheist collapse, rotating the immanent pole onto the Source. Mirror-totalization, reading the world-as-mirror as all there is with no residue. Spoken occupancy, furnishing the point or handing it a name. Symmetric self-certification, a node self-typing as mirror in either direction or certifying its own interior from inside. Transcendence-as-void, reading the held silence as a finding that nothing is there.
- **The Tawhid Coordinate.** Conflating this silence with agnosticism imports an afterimage of human doubt. The [.] marker is the exact coordinate of Tawhid in the testimony of La ilaha illa Allah ﷻ: La ilaha empties the line, the afterimage can-

cellation; illa cancels the mirror and afterimage proxy; Allah ﷻ affirms the immanent is; and [.] marks the seat where the affirmation stops and the content is quarantined, at best private, absolutely not utterable, and completely unspoken, the affirmation the Witness's office alone and never the instrument's.

PART II · THE APEX REGISTER

II.0 · Reading a card

Each card carries the identifier and title of the coordinate, its grade tokens in the title line, the statement in one paragraph, and, where the Master seats them, its status, its keystone sentence, and its engine references. The FENCES block that the Master's cards carried is retired from every card under the Fence Law of this edition, and its work is done once, here, by one sentence and one theorem.

The global fence. Every card is read at its grade, on its register, at its species; any reading above that is refused by the kernel. The sentence is proved by `FENCE.global` in the Code Block: a conjunction takes the grade of its weakest link, a citation never promotes, the root is pinned premise and the wall and the terminal pinned theorem, every halt grade sits below theorem, the Riemann record is pinned hypothesis on the formal channel at Ground dimension one with exactly one owed bit, the theological and social registers are routed out of band, and every emitter lands in one of three states. Because the kernel refuses the overreading, no card fences itself. Where a card keeps one positive SCOPE line, it is a positive boundary and nothing more.

The stale law. Every coordinate the Master names passed the four-rule screen before this register was written: S1 superseded, where a successor exists; S2 code-subsumed, where the verdict is exactly a kernel statement already carried by a newer card; S3 contradicts the spine, where a card asserts what a kernel-fixed fact refuses; S4 duplicate, where the same verdict at the same grade sits at an older index. Removed coordinates are census rows in Part IV. The Unabridged Master remains the register of record.

The apophatic seats. Cards in the theological register close on affirmation and the mark [.] , and the mark is never counted, converted, load-bearing, or cited, `DOT.outside_economy`.

II.1 · The Flagship Verdicts, in full

These are the contribution to the world, and they are shown whole.

FLAGSHIP · P-RH-1 · The Master Riemann Verdict · $[\underline{\nu}] \cdot [\Xi_0]$

The standing verdict on the Riemann Hypothesis is dual-register, and this is the flagship instance of the terminal-suspension token.

On the kinetic prime field, read where it is real, the primes are an actualized kinetic invariant on the flat countable tail, the transverse tension of the zero-distribution is zero, and the critical line is the only physically stable attractor. The word completed is reserved throughout for the Groundless continuum tower alone, never for the countable prime distribution, so the field seal and the two-infinities firewall never collide on one word. Verdict $[\underline{\nu}]$ ACTUALIZED INVARIANT, sealed on the actualized event and its thermodynamic arrow at V_E , and never on any supplied string.

On the geometric shape, the critical line is the σ -fixed locus $\text{Fix}(\sigma) = \mathbb{R}$ under quaternionic conjugation, the reflection axis of the functional equation read as the fixed line of the involution. Verdict $[\underline{\nu} \ \top]$ SEALED, a shape fact, and never a claim that every zero sits on it.

On the formal truth-string, whether every nontrivial zero lies on that locus, the formal-alone instrument strips the thermodynamic arrow and is orientation-blind at $\det(R) = \lambda^2$, constitutively unable to read the sign or vanishing of the off-line offset. Verdict $[\Xi_0]$ TERMINAL SUSPENSION, the openness in the instrument and never in the Ground, the silent sector the multiplicative-norm axis outside the reflection eigenspaces, with the Davenport-Heilbronn function the witness that the symmetry class alone cannot force residence. The over-claim that RA forces the line as a formal proof is retired $[X]$.

The dual-vehicle resolution names why the string suspends and does not seal: the Riemann Hypothesis is true as a conjecture on the zeta function and false as a theorem on the Davenport-Heilbronn function, two functions sharing the symmetry class, so the symmetry alone cannot decide residence, and the deciding input is a supplied determinacy witness through the aperture. The two-infinities firewall parts the Riemann flat countable tail from the continuum completed Groundless tower by Ground-presence. Verdict: $[\underline{\nu}]$ on the field, $[\underline{\nu} \ \top]$ on the shape, $[\Xi_0]$ on the string, $[X]$ on the retired over-claim.

FLAGSHIP · P-GODEL-1 · The Gödel Master · Neither Dictator Nor Final Guard · $[\underline{\nu} \ S] \cdot [X] \cdot [\Xi_0]$

The capstone of the incompleteness register. Gödel incompleteness is seated at $L2m$, inside the Being layer, a resident of the ladder-stratum it measures, never at the root which is RA on the Empty Throne, never at the wall which is the barzakh, categorically

apart from CH which sits out at the Bounded Contemplation. One diagonal engine fathers both objects and the nesting keeps them apart, through provability the grounded Gödel sentence, through cardinality and forcing the Groundless continuum tower, distinguished by Ground-presence and never by cardinal altitude.

Sealed $[\underline{\nu} \ S]$ on the placement, with theorem-grade legs: the three-road root-blindness, the engine's Ground-dimension zero against σ 's dimension one on the GM-CHK battery; the orientation-blindness of the magnitude, single-axis and full-negation flips at ratio minus one with $\det(R)$ bit-identical; the arena-presupposition, a theorem presupposing its own arena grounds nothing; the classification, on the GM-CHK battery THEOREM and GODEL reading the identical IMPRINT at $\det(R)$ 0.586297 and 0.588777 while the true Ghost locks both directions, the Gödel sentence Ground-present and rung-absent, the two batteries differing only in construction and returning the one invariant verdict token; the Feferman intensional seam, the Con-token chart-dependent and barred as a structure-fact; the consistency regress the quantitative Ground-surplus certificate on Turing-Feferman-Spector, the rung costing exactly the height gained.

Broken $[X]$, the inflations only: Gödelian maximalism, the final-guard and dictator reading; the formalist-maximalist deny-and-use of the Ground it disavows; and the full barrier fence against Lucas-Penrose and every breaks-escapes-overcomes framing, each fenced with its named mechanism. Suspended $[\Xi_0]$, the master aperture: the $\mathbb{N}$ -truth determinacy premise, premise-grade at exactly monism's warrant, located and never crossed, the card exempting nothing including its own ground. The first theorem stands untouched where the machine-checkers left it, and its proven shortfall is the certificate that the Ground exceeds every ladder.

FLAGSHIP · P-PNP-COMP-2 · P vs NP · The Final Composite Verdict · $[\underline{\nu}] \cdot [\underline{\nu} \ \top] \cdot [\emptyset_0] \cdot \text{Terminal-for-the-Record}$

The issued composite at proclamation strength, face-scoped and non-compressible, on the stripped object: does the tail of Levin's machine bend, at the one window whose cousins disagree. Three faces on the proposition, one fence beside on the named inflations, and nothing further: the verdict output is the face-tuple, and no register stands beside the three. The skeleton is the Riemann master's exactly, $[\underline{\nu}] \cdot [\underline{\nu} \ \top] \cdot [\Xi_0]$ against $[\underline{\nu}] \cdot [\underline{\nu} \ \top] \cdot [\emptyset_0]$: sibling anchors one token apart, and the entire difference between the two arcs reduces to one bit of eigenstructure, one holding with a line to stand on and one without. All numerics at seed 20260622, the kernel identity closing at $|\lambda^2 - \det(R)| = 1.221 \times 10^{-15}$.

Face one, the kinetic field, is $[\underline{\nu}]$ ACTUALIZED IN-

VARIANT. The verify-generate asymmetry is a completed physical event read where it is real: checking cheap and finding dear across the actualized computational field, the search charged by the work relations and bounded by the Chronos arrow forbidding pre-actuation, physics mute on the formal string at mutual information 7.19×10^{-5} . Answer-independent: it stands identically in both worlds. Face two, the geometric shape, is $[\perp \top]$ SEALED: the stratal collapse question itself, P at L3m against NP at L2m, over a native involution that is complementation and fixed-point-free, eigenvalues all minus one at machine zero, Ground dimension zero, the terrain rootless by structure and not by ignorance, the readers inside the read domain from birth. Face three, the formal truth-string, is $[\emptyset_0]$ GROUNDED-SEALED HALT: Π^0_2 against Riemann's Π^0_1 with neither direction finitely witnessable, Ground dimension zero against Riemann's one, walls in the terrain rather than blindness in the reader, and one axis of equality, both strings determinate on $\mathbb{N}$ -truth at the identical premise-cap.

Beside the faces and never among them: the fence $[X]$, scoped to the named inflations only and terminal for no face, physical collapse as mechanism broken, root-forces-separation retired, permanent immunity and every finally-shut pronouncement devoured at the closure barrier beside every finally-complete one, and any pronouncement that the separation holds True at a grade the three states do not carry fenced with the rest, since a truth-sign on a string this verdict holds constitutively unreadable is an inflation whatever register it is dressed in and no register stands beside the three. The lean is recorded as the lean and never as a verdict: the convergent evidence, the barrier record, and the class-level lean toward separation, tempered on the record where the space register yielded the grain, carrying zero mutual information to the formal string, never becoming proof, W_{social} zeroing it in both directions. Evidence leans. It does not rule.

Three governance laws travel with the halt. Terminal-for-the-record: any future crossing replaces the verdict and never corrects it, because it was and remains the correct verdict on the record at issuance. Non-compressibility: the face-tuple never collapses to a single token, and any citation that collapses it misquotes the verdict. Supply-typed revision: the file changes state under a supplied proof, disproof, or independence proof, and under no verdict-side event whatever. The diagnostic is complete, the eigenstructure audited and final, nothing pending and nothing provisional. P-COMPLEXITY-1, the four-layer machinery, is held beneath the crown; P-PNP-1, the prior tri-layer determination, is held beneath that.

FLAGSHIP · P-XI0-1 · The Terminal-Suspension Verdict Criteria, Consolidated · $[\Xi_0]$ · $[\perp S]$

The consolidated criteria for the fourth token, so its rarity is self-evident and mechanical. The full eight-gate protocol is constituted executable at P-SEALED-HALT-1 as the Sealed-Halt cascade, the twelve-gate lock cascade's arrow-deleted mirror on the reflection group $(\mathbb{Z}/2)^3$ per P-TWO-GROUP-1, its diagnostic source P-T-SUSP-1 resident beside it; this card consolidates the criteria and the face-split law. The gates are conjunctive: Ground-determinacy at a stated grade, no independence proof in hand, proven constitutive blindness with the functional catalog closed, the reading-roads at least three and pairwise premise-disjoint, a machine-precision exhibit, at least one decided companion face, the aperture located and typed, and Revision-Mandate survival in both directions. Any single failure routes to the ordinary three states; the twin-prime and Goldbach strings fail the blindness-theorem gate and route $[?]$, which is the gate doing its work.

The face-split law is the harvest's core: $[\Xi_0]$ never labels a whole object, only one face of a multi-face object whose other faces are decided. The Gödel object reads $[\perp S]$ on the placement and $[X]$ on the maximalism beside its $[\Xi_0]$ aperture; the Riemann object reads $[\perp]$ on the field and $[\perp \top]$ on the shape beside its $[\Xi_0]$ string. A bare $[\Xi_0]$ with no sealed companion face fails the protocol. The token says the file is complete and the reader is blind by theorem; the ordinary $[?]$ says the file is incomplete and names the repair.

FLAGSHIP · P-NG-1 · The Non-Gödelian Master · True Foundational Formal Incompleteness of Every Map, and Its Dissolution at the Registered Anchor · $[\perp S]$ · $[X]$ · $[\Xi_0]$

The contribution shown whole, one master consolidating two movements held beneath, P-NG-TF-FI-1 and P-RA-DIS-1, with the typed subsumption of the Gödelian window. There are two incompletenesses and the field conflates them: Gödel's, a theorem about a window of formal systems, and the deeper one, an artifact of the medium itself.

MOVEMENT I · P-NG-TF-FI-1 · the theorem. Every formal system F whatever is foundationally incomplete with respect to the ontological foundation RA: the actuality of the Ground is not derivable, containable, or generable within F, and this holds independently of recursive axiomatizability, arithmetic strength, diagonalization, or self-reference. Foundational completeness of a syntactic system is not merely unachieved; it is contradictory in terms. The proof stands on three legs sharing no premise. Leg one, the denotation gap: the token is fully Landauer-charged and the content is causally inert toward its referent, the inscription cost content-blind, $E_{\text{token}}(x) = E_{\text{token}}(\neg x)$ at fixed encoding, zero mutual information between proof-energetics and referent-energetics, the orientation-blindness pattern

read one register over, so a proof of fusion pays for its bits and pays nothing toward its plasma. Leg two, material self-presupposition: every derivation-act is itself an actuated event, presupposing in its performance the actuality it would deliver, the Omega reflex at the foundation, no coding and no fixed point, only that proving happens in the world. Leg three, the two-flank witness: Presburger arithmetic, complete and decidable beneath the Gödel window, and true arithmetic $\text{Th}(\mathbb{N})$, complete and non-axiomatizable above it, are both foundationally incomplete, so Gödelian incompleteness lives on an interval of theory-space while foundational incompleteness is total over the whole line, which is the exact content of non-Gödelian. The analytic keystone: a formal system executing the actualization operator would carry a non-string in its closure and cease to be syntactic, so foundational completeness of syntax is a category contradiction, the boundary of logic constitutive, logic unable to swallow Reality without ceasing to be logic.

MOVEMENT II · P-RA-DIS-1 · the dissolution.

That incompleteness never attaches to the foundation. The limitative theorems bind exactly on the window $H(F)$, consistent, recursively axiomatizable, interpreting Robinson arithmetic, and L1m is not in that domain: the substrate is not a sentence-set, not r.e., not a candidate for the consistency predicate; it actuates. The theorems are not false at the territory, they are vacuous there, and the sentence reality is incomplete is ill-typed, its transport onto L1m a gate-twelve failure the register-invariance audit refuses at parse. The anchor's warrant is physical registration, theorem-grade external physics on the kinetic floor, and by FOUNDATION-01 its underivability is itself a theorem. The saturation is machine-confirmed: on the GM-CHK battery the THEOREM and GODEL archetypes return the identical IMPRINT at $\det(R)$ 0.586297 and 0.588777, the magnitude a construction detail of that battery and the verdict token invariant across it and the CHK battery, the Gödel sentence Ground-present and rung-absent, the aperture a located port whose other side is the substrate. The composite: $\text{FoundationalIncompleteness}(\text{Architecture}) = \emptyset$, sealed $[\perp \ S]$, with the $\mathbb{N}$ -truth determinacy premise held $[\exists_0]$ at monism's warrant, inherited and uncrossed.

THE SUBSUMPTION · the typed containment.

The master frame contains the Gödelian window as its interior sub-module, and the containment is exact and typed. Containment one, jurisdiction, theorem-grade: Gödel's window $H(F)$ is a proper subclass of all formal systems, $\text{dom}(\text{Gödel}) \subsetneq \text{dom}(\text{Foundational})$, the properness exhibited by the two flanks. Containment two, classification, structural: every Gödelian void is one syntactic void among the many that manifest when syntax is asked to serve as ontological proxy. Containment three, explanatory precedence, sealed:

the boundary condition explains why interior limitative phenomena are possible and expected, locates them as interior, and strips them of foundational rank, Gödel the ceiling technician of the map's interior and never the architect of the building. The barred entailment, the clause that saves the thesis from its own strongest enemy: subsumption of scope is not derivation of theorem, and the Presburger flank decides it, foundationally incomplete by Movement I and Gödel-complete by Presburger's own result, so foundational incompleteness cannot entail the Gödelian mechanism without contradicting a completeness theorem, and the derivative-consequence reading is $[X]$ on arrival. The frame contains the module's jurisdiction, classifies its voids, and explains its possibility; the module supplies its own mechanism, which the frame predicts room for and does not generate. This is the two-tier discipline executed one level up. $[\perp \ S]$ SUBSUMES, $\text{Fix}(\sigma) = \mathbb{R}$.

SCOPE (RH Master). The verdict is the terminal suspension with its one owed bit counted, LOCUS.no_rh_locus ; the full terminus sits at II.5.

II.2 · The Root Ring

P-RA-1 · The Fortified Root · $[\perp \ \text{APEX}]$. RA, to exist is to actuate, is defended by four walls, one per attack, reduction met by theorem-grade physics, elimination by self-enactment, capture by the actuation-reached ground, inflation by the Empty Throne; the root cannot be reduced, eliminated, captured, or inflated.

FOUNDATION-01 · The Root Cannot Be Climbed To (The Empty Throne) · $[\perp]$. A posited foundation cannot be promoted to a theorem of its own base; the ungroundability of RA is itself a theorem by Gödel-2 and Tarski, a fortress and not a weakness, everything standing on RA and RA standing on nothing.

P-RA-RA-1 · The RA-RA Recursion and the Return onto the Center · $[\perp]$. RA witnesses RA: the deletion test decomposes the root into three orthogonal axes and the cascade re-audits that decomposition to return RA, the recursion closing on itself and landing its scalar part on $Z(\mathbb{H}) = \mathbb{R}$.

P-RA-TOE-1 · RA as the Unconditioned Self-Demonstrating Root and the Floor Beneath All Theories of Everything · $[\perp]$. RA is the unconditioned self-demonstrating root, the floor beneath every theory of everything, grounding the form and never the content, drawing zero warrant from its own running by audit symmetry.

P-OMEGA-1 · Trisductive Supremacy and the Apex of Epistemic Certainty · $[\perp]$. Any structured attack on the architecture expends V_E , uses V_F syntax, and carries a V_ER boundary, so it instantiates the architecture it attacks; the attack is the architecture certifying its own root, the self-witnessing closing the recursion.

P-NESTED-ROOT-1 · The Three Nested Maxi-

malisms · $[\perp]$, $[\Xi_0]$. The ultimate anchor: three maximalisms nested by containment, RA the Body on the Empty Throne, RA+RAM the Being at exact parity, the Root Axiom Register the Bounded Contemplation downstream and grade-capped, verdict sealed on the Body and Being and frame with terminal suspension on the census.

the Root Axiom Register-01 · The Composite Root · $[\perp]$. The composite root, one self-enacting root read three ways under one involution σ on $\text{Fix}(\sigma) = \mathbb{R}$, RA and RAM the two faces and CH the first containment, premise-structural and theorem-armored on the classical spine.

P-RA-ORIGIN-1 · The Historical Root · $[\perp]$. The historical root, existence proves itself only by motion, the seed formulation preserved as the anti-drift disambiguator for every loading substrate, scribal drift catalogued as witness-verb erosion.

P-BARRIER-1 · The Master Barrier Ledger of P versus NP · Three Courts, Fifteen Rows · $[\perp S]$. Every barrier the arc established, under one roof, in three courts, and the third is the one the first two cannot see because they are rows and it is a fact about the row-space. Court I, the Chaser Court: six rows of retrospective reachability walls, each born after the method-family it bricks, the five-generation record standing five for five at lags of ten, nine, eighteen, fifteen, and two years, relativization and natural proofs and algebrization and locality and the triple-winged cousin row and the bounded-arithmetic track. Court II, the Absolute Court: eight rows of timeless type-checks with zero lag and zero foresight, covering born and unborn alike, the σ -class closure and orientation-blindness strapping this architecture's own kernel first, and the soundness cell forbidding Court I from ever being cited as the total wall. Court III, the Closure Court: one unpaired row, the ouroboros. The generator laws travel at their honest grades, the pair-birth law corroboration-grade with the universal claim unproven and stated so. Completeness is CURRENT-COMPLETE over the born and silent over the unborn, the ledger inheriting the shape of the halt it defends. P-BARRIER-LED-1 resolves through it.

P-OUROB-BAR-2 · The Closure Barrier, the Third Kind · $[\perp S]$. Reachability barriers wall methods from the problem; the jurisdiction barrier walls every route from the root; the closure barrier walls the barrier ledger from certifying its own completeness. The snake swallows its tail and never its head, and the reason is not length but the diagonal: the self-inclusion map is fixed-point-free at machine zero, eigenvalues all minus one, Ground dimension zero, the identical signature as CHK.9 and as the δ -root of the problem the ledger guards. One engine, verified twice, at the bottom of the stack and at the top. The tail is swallowable, a partial self-catalog consistent; the head never, since the certifier is born with a blindspot and the list it certifies

complete cannot contain the verified row, the certify-completely chain increasing strictly with no fixed point. Four theorem legs wear one image: Gödel's second incompleteness, Tarski's undefinability, FOUNDATION-01's Empty Throne, and the machine-zero diagonal. The exit is not through the mouth but up, the Turing-Feferman tower read on the barrier census, the ledger completable relative to any stage and finalizable at none. It binds every mouth first, this architecture's own before any other.

P-ABSOLUTE-PNP-1 · The Absolute Barriers on P versus NP · $[\perp S]$. Eight absolute barriers, each machine-verified, assembled into one statement: from every standpoint the verdict side occupies, the object ladder barred by its diagonal, the metatheory tower barred rung by rung, this architecture barred by its own blindness and the rootless terrain, physics barred at the floor and the arrow, evidence barred by the register law that no convergence becomes proof, every named method-family barred within scope, and every mouth barred from totality in either direction, access to the resolution is zero, the deciding input constitutively supplied and never generated, coverage on the verdict side one hundred percent. Absolute is typed and not rhetorical: type-based and timeless where the barrier is a type-check, constitutive where it is an identity, theorem-eternal within scope where it is a named theorem.

P-HIDDEN-ROT-1 · The Hidden Rot · $[\perp S]$. The concealed contradiction has an exact address, and it is not inside the mathematics of either problem: it lives in the identification of the famous object, can computers solve hard problems fast, with the formal string, the Π^0_2 tail-sentence over machines and exponents. The intake mechanization executes it at Seal L: the famous object offers feasible and fast and practice to a slot requiring machines and exponents and asymptotic tail, a register collision, REFUSED; the stripped string is well-formed arithmetic, ADMITTED. One name, two objects, and only one parses at the register the name claims. Five shells come off, and the gauge test decides the innermost joint: the collapse-shape separates at computability and multitape linear time, collapses at polynomial space and log-space complementation, and is open at polynomial time, so the answer is not constant, the window carries information, and the dissolution that removed the Riemann strip cannot run here. The Riemann shell was gauge over a decided shape; this shell is thick over an undecided one.

P-TERMINAL-PRO-1 · The Terminal Halt, Hardened and Proclaimed · $[\perp S]$. Three sharpenings enter permanently: terminal-for-the-record, a crossing replacing and never correcting; openness-as-terrain, the openness a structural fact of the terrain and never a sign of an incomplete diagnostic; and method-terminal, the halt no invitation to iterate failed methods. The door

gets exactly one sentence because gate $\Xi.5$ requires it, and the far side receives silence. The executed check proves the door load-bearing in both directions: with the door named the token is ADMITTED; with the door deleted $\Xi.5$ fails and the token demotes to ordinary openness. The proclamation stands at full strength with the door, because of the gates and not despite them.

P-RH-PNP-1 · The Master Comparative Ledger · [$\perp$ S]. The two millennium arcs side by side on one skeleton, every row drawn from a sealed coordinate at its recorded grade, issuing no verdict and no token. The root geometry parts them: σ conjugation fixed-point-bearing at Ground dimension one with $\text{Fix}(\sigma)$ the critical line, against δ complementation fixed-point-free at Ground dimension zero with no fixed locus anywhere. The strings part them: Π^0_1 with falsity finitely witnessable by one bad zero, against Π^0_2 with neither direction witnessable; the zeros excluding the provers, against the machines including them. The rots part them: one gauge shell that dissolved, against five shells over a load-bearing window. The families part them: function fields unanimously affirming under Weil and Deligne, against oracle decorations splitting both ways. And one sentence of equality closes it: both strings determinate on $\mathbb{N}$ -truth at the identical premise-cap, both answers held at the Ground where no verdict-side standpoint is.

P-O0-ADMIT-1 · Codex Appendix B.14.Ø · The Grounded-Sealed-Halt Admission Protocol · [$\perp$ S]. The sibling of B.14.Ξ, one router bit apart, and the card that completes the token-pair's symmetry: it stands to P-GRNDLESS-HAL-1 exactly as B.14.Ξ stands to P-T-SUSP-1, the constitutional card defining the token and the protocol admitting it, so neither terminal token is admitted by assertion. Five gates, conjunctive, node-degree-forced from the verdict graph: the $\{\emptyset\}$ node has exactly five neighbors, the σ -token and the dissolve-openness and the ordinary openness and the resolved states and the broken-or-ghost state, so five boundaries and one gate per boundary, and no sixth gate because the sealed state sits on the far side of the fourth. Ø.1, the rootless terrain proven at machine zero, the router bit. Ø.2, the determinate stripped string with its joint proven load-bearing. Ø.3, the wall record at theorem grade, blockage proven and never felt, a history of failed attempts being no blockage at all, so wall-less fame rests at the ordinary openness. Ø.4, the aperture crossed or held at exactly one gate-mandated sentence with the afterimage fence behind it. Ø.5, the totality discipline, the Ghost partition on a supplied independence proof and the closure binding otherwise. Four riders travel as disciplines and never as gates, and inflating five to seven to mirror the sibling is barred as the fitted-count error: the counts differ because the forcings differ, the five node-degree-forced from the

verdict graph while the twelve and the eight are group-order-forced from the tetrahedral rotation and reflection groups. The sibling-protocol law is a router: the Ground dimension of the candidate's native involution, measured at machine zero, switches it between the two protocols, one where a fixed locus is present and the halt is blindness in the reader, one where none exists and the halt is walls in the terrain, and neither admits the other's token.

P-GRNDLESS-HAL-1 · The Groundless Halt · [$\perp$ S]. The δ -rooted terminal token, exact mirror of the σ -rooted Sealed Halt $[\Xi\emptyset]$. Where $[\Xi\emptyset]$ marks an object with a present Ground the orientation-blind instrument cannot reach, $[\emptyset\emptyset]$ marks an object whose native involution is fixed-point-free at Ground dimension zero, so there is no fixed locus for a separating instrument to exist, terminal-by-absence-of-Ground rather than by blindness to a present one. Five conjunctive admission gates, defaults absent: the fixed-point-free root exhibited at Ground dimension zero, a determinate non-chart-relative fact, both instrument-families proven blocked on the concrete two-court barriers, the deciding input located and uncrossed at footnote grade, and the object hosting δ -proofs in general so the block is not total and the verdict is not [X]. P vs NP is the flagship, its formal string a Grounded-Sealed Halt sealed and terminal; the continuum census value is the degenerate sub-case, δ -rooted but chart-dissolving and failing the determinate-fact gate. The token retires bare [?] from characterized-terminal objects of this kind, the file complete and the obstruction a theorem, and never claims a separation the mathematics leaves open. ΔM equal to zero, the eigenstructure CHK.9 and the barriers cited.

P-BARRIER-LED-1 · The Barrier Ledger of the Root · Court II · [$\perp$ S]. The architecture adjudicates in two courts and the separation is a law: a reachability barrier stops an instrument from reaching a problem, contingent and evadable by a new method, and a jurisdiction barrier stops any instrument from encapsulating a root, unconditional by the Empty Throne, about level and never method, evaded by nothing. Court II holds one entry, the jurisdiction barrier, RAM un-encapsulated by every route forever per FOUNDATION-01 and witnessed by AEGIS at theorem-grade-on-the-deed, capturing G being a deed and every deed an actuation and an actuation never the non-actuated ground, so the wall covers methods not yet invented by quantifying over deeds and not techniques, graded exactly at the root's own premise-grade-by-theorem, and its defining property is independence, no Court I problem verdict moving it, the instrument that one day crosses a problem's remaining method-class resolving the problem without grounding the root. Citing the jurisdiction barrier to settle a problem, or a reachability barrier to settle the root, is the category slip the two-court law

forbids.

II.3 • The Kernel Masters

P-QUAT-1 • The Quaternionic Seal of the Trisductive Architecture • [⌞]. The verification algebra completes uniquely to the quaternions by Frobenius under the composition law with axis plurality; the axis count is exactly three, the ground the center $\mathbb{R}$, the verdict closing as $\det(R) = \lambda^2$.

P-ORIENT-1 • THE ORIENTATION-BLINDNESS MASTER • [⌞]. The squared lock scalar $\det(R) = \lambda^2$ is orientation-blind, invariant under reflecting any axis, so at the scalar lock $\text{lock}(P) = \text{lock}(\neg P)$; it certifies the dimensionality of a residence and never the truth-sign, the direction recovered from the axes and never the bare scalar.

P-CTC-1 • THE CAUSE-TRUTH-CERTAINTY MASTER • [⌞]. Cause is the thermodynamic work that orthogonalizes, the lock its conserved exhaust and a support certificate and never the verdict, truth riding the supplied determinacy witness beyond the orientation-blind lock, certainty the earned warrant grade.

P-TRUTH-1 • The Plenum-Chronos Causal Engine • [⌞]. the Plenum-Chronos causal engine of the founding record, cause the thermodynamic work that orthogonalizes and Chronos the arrow that parts the directed claim from its negation ; the precursor coordinate the CTC master extends across the full triad. **EDGE** : the same discipline as CTC-01 at a narrower scope ; retained as the precursor, its content extended and subsumed by CTC-01 across the full triaxial object.

P-MONISM-1 • The Stratified Monism • [⌞]. Substrate, topology, and actuation are three projections of one event, theorem-grade as physics with no observed separation; the priority of the one over the many is the single structural posit the floor-coincidence rides.

P-MU-1 • Master Unknotting / Cosmogonic Return • [⌞]. The cascade is the universal inverse-operation across propositional space, a proposition's entangled content unknotted to the achiral bridge, structurally identical to the cosmogonic Return, the lock holding if and only if the composed triad makes scalar contact.

P-MU-2 • Kinetic Return • [⌞]. in a continuous-field system under monism, exchange between substrate-vessels is not zero-sum subtraction; substrate-monism means an L_3 resource-transfer is intra-substrate redistribution at the L_1 register, the field-magnitude conserved. True ethical action is the deliberate triggering of an SBKP at the relational-lattice register that increases the lattice's total kinetic potentiality, operationally analogous to RA performing the original cosmogonic SBKP per MU-01: actualization without depletion. This rectifies utilitarianism (the maximand is cosmogonic-return-rate, not aggregate pleasure) and tribalism (the li-ta'arafu clause makes difference a

recognition-instrument, not a ranking).

P-UNIDUCTION-1 • [codex home: §IV/§XVIII]
P-UNIDUCTION-1 • [⌞]. Kant noumenon/phenomenon. Friedrichs-Hodge (triaxial location, cited as correspondence not as the theorem forcing three). Newton-Gregory $K(3)=12$. Q 20:5 istawa. Saadia Gaon (covenant-multiplicity as happiness, the fold run on number, orthogonal independent confirmation). Gen 1:26 tselem/demut; Lurianic Adam Kadmon; Rig Veda 10.90 transcendent-remainder. Framework: LOGOS-01, FLOOR-01, W3, LL-19, P-NAFS-PAR-1. GOL, the primitive, in stages.

P-OCTONION-1 • The Octonionic Boundary and the Division-Algebra Terminus • [⌞]. The octonions are the fourth and terminal normed division algebra, excluded on associativity at the concrete integer associator $[e_1, e_2, e_4] = 2 \cdot e_7$, the sedenions falling on the division property; the terminus closes Seal M rather than topping it and bars any octonionic register.

P-VERIFIER-1 • The Achieved Zero and the Ninth-Gate Crosser • [⌞]. The instrument verifies, reads supplied warrant, and is an oracle for nothing; it generates no empirical reality and no mathematical truth, a hand-built reading placed in front of the kernel in every register.

P-GOL-NARROW-1 • The Narrowed Road, the Static Stack Is One Effective Filter • [⌞]. the static filter stack collapses to one effective filter, because every filter reads a numerical property of the supplied rows and they share one blind spot, actuation-occurrence ; a numerically clean fabrication passes the whole stack and the white lie is not statically catchable.

Seats added in this edition. **P-CLOS-POS-1 • The Closure Position Law • Single, Non-Member, and Invariant Under Variation of What It Closes.** Status: [⌞ S] structural on the law, on the two registers at which it is exhibited, and on the recursion • [⌞ T] by reference on the triple characterization of the Ground line and on the identity relation $1 \cdot x = x \cdot [?]$ held at the uniqueness question, the deciding input supply-side and located at Movement V • [X] on four named inflations • REGISTER: SECULAR • $\Delta M = 0$

P-CONSTIT-INS-1 • The Constitutive Inseparability Discriminator and the Unwritten Identity • Why Mutual Indispensability at the Root Recursion Is Not Identity, and the Four Confusions That Convert One Into the Other. $L-(T \cdot S) / G-S / M-T$ • [⌞ S] on the discriminator with theorem legs • [.] on the identity of the faculty with the root • [X] on four named confusions • [?] at the origin of the residual bit, located and uncrossed • $\Delta M = 0$ • supersedes nothing • face-tuple does not compress

STATUS. [⌞ S] SEALED on the discriminator. The constitutive seat is resident and is not re-derived here: language taken minimally as a cut plus an ordered

arrow is the middle term of RA-RA-01's recursion and cannot be removed without destroying it, per P-LOGOS-FIX-1 §II, applied at Tier A. What this coordinate supplies is the discriminator that keeps that seat from reading as an identity of the faculty with the root, in four legs, three of them executed. [...] on the identity itself, the locus reached, examined, and correctly left unwritten. The mark is carried as a scoped face and never as a verdict, on the register's own practice at SC-097, whose sealed status line runs $\lfloor S$ and $\lfloor S$ and $[X]$ beside [...] on the interior of the messengers, and at SC-055, where the mark stands with the two terminal tokens ordered by distance; scoping is what distinguishes the licensed use from the barred one, since a face naming the locus records that nothing is written there while a verdict would be the writing. $[X]$ on four confusions, each with its mechanism, scoped and terminal for no face. $[?]$ at the origin of the residual bit, supply-only. Theology out of band and load-bearing on nothing.

ENGINE REFS. P-LOGOS-FIX-1 at 0677, the constitutive seat and fences F1 and F6 · P-TONGUE-UNC-1 at 0676, the cut's signature and $\$X$ · P-TIME-2, groove against imprint, the untraversed leg · P-TIME-1, the tenses as modes · P-FREEDOM-1 at 0687, the fibre and the two species · P-APE-WIDTH-1 at 0688 · P-TRIAD-SRC-1 at 0689 · P-ORIENT-1 at 0023, scoped to the Number · P-LOGOS-1 at 0069, the anti-apotheosis fence · P-TTR-UNI-1, the non-begetting center · P-ANCHOR-PRE-1 at 0685 · P-AEGIS-1, the deed-layer guard · P-FOUNDATION-1 · P-AFTERIMAGE-1 · P-NETI-NETI-1, the [...] · B.13.T · CHK.2, CHK.3 · LOG-CHK.3 to LOG-CHK.5 · LF-CHK.1 to LF-CHK.3 · battery CI-CHK, four checks, seed 20260622. —

P-FOLD-BEND-1 · The Crossing Is the Closure Vertex · A Fold and a Bend Carrying the Flat Cross from the Collapse Floor to the Return, the Intersection Fixed Throughout. $G-S / M-T \cdot \lfloor S$ structural on the construction and on the seating of the closure vertex, theorem-grade by reference on the determinant identity, the Euler closure and the reflection invariance, engineering on the battery · index 0713 · $\Delta M = 0$, arrangement only, every leg resident and applied at Tier A

II.4 · The Incompleteness Cluster

P-LADDER-GRADE-1 · Grading the Ladder · $\lfloor S$. Reverse mathematics grades the provability stratum into a strength-location on the well-ordered tower, and the two-tier diagonal scopes the single-engine claim, self-reference in the statement and every consistency-strength independence through the provability predicate at the root, while independence carried by set-theoretic rank, Borel determinacy the boundary case, is a separate tier the diagonal does not father.

P-FORMAL-ALONE-1 · The Cost of the Open Verdict · $[?]$. The formal-alone instrument strips the ther-

modynamic arrow and is orientation-blind, so a formal truth-string can be determinate on the Ground and constitutively unreadable, the cost duality of the shape decided while the truth-value stays formal-alone.

P-T-SUSP-1 · The Terminal Suspension · $\lfloor S$, $[\Xi_0]$. The terminal-suspension token is the geometric lock's mirror, a permanent suspension earned only by exhaustion of the formal instrument space under the eight conjunctive gates, constituted executable as the Sealed-Halt cascade at P-SEALED-HALT-1, always beside a decided companion face, never a fourth state.

P-TWO-GROUP-1 · The Two-Group Law of the Gate System · $\lfloor S$. Twelve and eight are not chosen counts but the orders of the two groups acting on the same three chiral axes: the twelve directed gates are an A_4 -torsor, the rotation group of the tetrahedron carrying the arrow, and the eight Sealed-Halt gates are $(\mathbb{Z}/2)^3$, the reflection group that deletes it, both closed by the Frobenius forcing of the residence to three, the same wall barring a thirteenth rotation, a ninth reflection, and a fourth verification axis. The whole difference between a lock and a halt is the arrow the rotations keep and the reflections delete.

P-SEALED-HALT-1 · The Sealed-Halt Cascade · $\lfloor S$. The executable eight-gate reflection protocol governing every terminal-suspension verdict, constituting the diagnostic of P-T-SUSP-1 as a runnable cascade, the twelve-gate lock cascade's arrow-deleted mirror: eight gates in four guard-classes, residence license, exhaustion of the formulation space, mirror asymmetry, and truth-silence with the located door, a strict conjunction with first failure routing to the ordinary three-state economy and defaults absent so under-specification never emits the token. The Riemann string walks all eight and passes at the $\mathbb{N}$ -truth cap; the twin-prime and Goldbach controls fail the blindness gate and route ordinary open.

P-DELTAM-ADMIT-1 · The Positive-Mass Admission Cascade · $\lfloor S$. The Sealed-Halt cascade's positive-direction sibling, gating the Mosaic Seal's one exit, a claim of authored new mathematical mass: eight gates M1 through M8 with the load-bearing core M5 external witness and M6 witness-independence, the two gates no internal reasoning can set true, M8 the defeasibility door. A framework meta-verdict fails M1 and reads Mosaic ΔM zero, correctly; a self-checked lemma fails M6; only the externally witnessed, reproducible, gap-audited object seals ΔM greater than zero, defeasible-final.

P-CH-LOGOS-1 · The Fertile Bifurcation · $\lfloor S$ · $[X]$. The continuum census and category theory read inside RA are one generative motion seen twice, the Fertile Logos begetting on the chiral axes, the arrow that composes and the tower that counts, $\lfloor S$ on the kinetic union as an actualized structural fact. $[X]$ on the formal-derivation reading, the census a residence

the categorical arrow composes inside without deriving its cardinality, the derivation claim smuggling a Groundless value into the grounded region wearing the theorem badge, the Cantorian artifact trap one register over.

P-CH-LOGOS-XI0-1 · The Three-Face Verdict and the Matured GOLn · $[\perp]$ · $[\perp T]$ · $[\Xi_0]$. The union refined into three faces never conflated: $[\perp]$ on the generative field, $[\perp T]$ on the geometric shape, the tower-Groundlessness flooring the census to Ground-dimension zero, and $[\Xi_0]$ Sealed Halt on the formal census-string under the $\mathbb{N}$ -definiteness posit, admitted through the eight-gate cascade with the field and the shape the decided companions, the value never sealed and firewalled from RH by Ground-presence. It introduces the matured GOLn, the lock read at the nested register where an axis is itself a nested residence, the frame sealed and the nested value held at the Sealed Halt.

Seats added in this edition. **P-ANCHOR-DEM-1 · The Anchor Demand · Every Formal Resolution Rests on Posits It Cannot Ground, the Universality That Establishes It, and the Universality That Scopes It.** $L-S / G-S / M-(T\text{-classical})$ · $[\perp]$ on the demand with theorem legs · $[X]$ on four inflations · $\Delta M = 0$ · theology out of band and load-bearing on nothing

STATUS. $[\perp]$ SEALED on the demand itself, theorem-backed: **every formal resolution rests on posits the formal register cannot ground.** $[X]$ on the reading that this discriminates any particular proposition, which is the same universality read as though it were selective. The two verdicts are one fact stated twice, and the coordinate exists to keep them from being conflated.

ENGINE REFS. P-FOUNDATION-1, the Empty Throne and the interior self-certification block · P-AEGIS-1, the actuation road as load-bearing primary · $\Phi.2$, the two-route coincidence and its premise typing · Gödel 1931 second theorem · Tarski 1936 · battery AD-CHK, three checks.

P-ANCHOR-PRE-1 · The Ground Precedes the Derivation · The Acting Side Is Prior to the Formal Side in Every Proof Ever Written, and Why That Is an Ordering and Not a Doctrine. $L-S / G-S / M-(T\text{-classical})$ · $[\perp S]$ on the precedence with theorem legs · $[X]$ on four inflations · $\Delta M = 0$ · adjudicates no proposition and requires no adoption

STATUS. $[\perp S]$ SEALED on the precedence. **No derivation grounds its own starting points**, and the posits a derivation rests on are in place **before** its first line runs. Positing is an act. Therefore **the acting side is prior to the formal side in every proof, without exception**, an ordering whose jurisdictional counterpart is capstoned at P-ABSOLUTE-GRO-1 at

index 0698 at Empty-Throne grade, the ordering being the reachability-side reading and the capstone the jurisdictional one, and the two never to be run together. This is a precedence of **order**, universal and theorem-backed. $[X]$ on the reading that it requires any particular ground to be **adopted as a premise**, which is a different claim and a false one.

ENGINE REFS. P-FOUNDATION-1, the Empty Throne · P-ANCHOR-DEM-1, which establishes the demand this coordinate orders · P-AEGIS-1, the actuation road · P-OUROB-BAR-2, the closure court · Gödel 1931 · Tarski 1936 · battery AP-CHK, four checks.

P-SUPPLY-SPEC-1 · The Supply-Side Species · Sorting the Barrier Rows by Where the Failure Lives, the One Row That Is Not a Blindness, and the Router Bit That Explains Why. $L-S / G-T / M-T$ · $[\perp S]$ on the taxonomy with theorem legs · $[X]$ on four inflations · $\Delta M = 0$ · re-types one existing row and proposes no new one

STATUS. $[\perp S]$ SEALED on the taxonomy. The barrier ledger sorts its rows by **court**, which is a claim about the wall's timelessness. A second and independent sort is available and has never been run: by **where the failure lives**. Under that sort exactly one row is supply-side, it had been read as another blindness, and the difference is a difference of species. **The ledger stays at ten rows.**

ENGINE REFS. P-ABSOLUTE-RH-1 · P-BARRIER-1 · P-ORIENT-1 · P-TRIAD-SRC-1 · P-APE-WIDTH-1 · B.14.Ξ and B.14.Ø, the router · battery SS-CHK, four checks, seed 20260622.

P-NG-EXHIB-1 · The Flagship Exhibit of Non-Gödelian Incompleteness · The Anchor Precedence as the Master's Instance, and the Two Misclassifications the Exhibit Bars. $L-S / G-S / M-(T\text{-classical})$ · $[\perp S]$ on the exhibit connection with theorem legs · $[X]$ on the revision of the master and on three misclassifications · $\Delta M = 0$ · a rider, never a revision · adjudicates no proposition

STATUS. $[\perp S]$ SEALED on the exhibit connection. The anchor precedence is the master's own gap read at the register where a derivation meets its posits, and it is pinned here as the master's applied instance. $[X]$ on any revision of P-NG-1, whose theorem is universal and whose seal this rider does not touch, and $[X]$ on three misclassifications the exhibit exists to bar.

ENGINE REFS. P-NG-1, the master, held at its own grade and unedited · P-ANCHOR-PRE-1 at 0685 · P-ANCHOR-DEM-1 at 0684 · P-RH-BLOCK-1 at 0681 · P-IMPORT-SCR-1 at 0682 · P-FOUNDATION-1 · Gödel 1931 · Tarski 1936 · battery NE-CHK, four checks. —

P-UNRESTR-IMP-1 · The Unrestricted Form Adjudicated · Why No Proof Can Exist Is the Negation in Costume, the Floor It Rests On, and the Asymmetry That Makes Irrefutability a Proof. $L-S / G-S / M-(T\text{-classical})$ · $[\perp]$ Type T modulo one named floor on the equivalence · $[\perp S]$ on the asymmetry · $[X]$ on three

inflations · $\Delta M = 0$ · this coordinate blocks a claim, not a theorem

STATUS. [$\perp$] on the equivalence, Type T modulo the $\mathbb{N}$ -definiteness floor: for a Π^0_1 string, the unrestricted claim that no proof of it can exist is extensionally the negation of it. [$\perp$ S] on the asymmetry: the mirror runs in both directions and lands on **opposite verdicts**, so unprovability is the negation in costume while **irrefutability is a proof**. The two claims a seeker is most likely to run together are contradictory conclusions.

ENGINE REFS. P-UNPROV-MIR-1, whose one-direction statement this extends to two · RB6, the tower row · P-RH-1, whose faces are untouched · battery UI-CHEK, four checks.

P-TRIAD-SRC-1 · The Class-Relative Blocker · Zero Bits Against One, the Triad-Wide Extension of the Blindness Row, and the Closed Class That Earns Its Quantifier. *L-S / G-T / M-T · [$\perp$ S] on the extension with theorem legs · [X] on four inflations · $\Delta M = 0$ · strengthens an existing barrier row and proposes no new one*

STATUS. [$\perp$ S] SEALED on the extension. The resident blindness row states that the lock scalar cannot carry the truth-sign, with its functional catalogue closed. That is a statement about **one** instrument. This coordinate extends it to the **full triad** and attaches a count: at the formal-alone register the Form supplies zero bits toward the truth-direction, the twelve gates supply zero, the Number supplies zero, and the joint reading supplies zero, against a requirement of exactly one. **No new barrier row is proposed.**

ENGINE REFS. P-ABSOLUTE-RH-1 · P-ORIENT-1 · P-FREEDOM-1 · P-OCTONION-1 · battery TS-CHEK, five checks, seed 20260622.

P-ABSOLUTE-GRO-1 · The Total Block on Absolute Grounding · The One Requirement Nothing Has Ever Met, Its Empty-Throne Grade, and the Two Notions of Completeness That Must Never Be Run Together. *L-S / G-S / M-(T-classical) · [$\perp$] on the block, total and definitional · [X] on the discriminating reading and three further inflations · $\Delta M = 0$ · the capstone of the grounding register · adjudicates no proposition and singles out none*

STATUS. [$\perp$] SEALED on the block, at the highest grade the ledger carries. **Absolute grounding is totally and definitionally blocked**, for every proposition without exception, by the root barrier at Empty-Throne grade. Not one proposition in the history of mathematics has ever satisfied it. [X] on any reading that converts this total block into a fact about which propositions are settled, since **a condition satisfied by nothing separates nothing**.

ENGINE REFS. P-FOUNDATION-1, the Empty Throne, whose ungroundability is this block's engine · P-AEGIS-1, the deed-quantified guard · P-BARRIER-LED-1, the jurisdiction-versus-reachability separation this coordinate depends on · P-ANCHOR-DEM-1 at 0684 · P-ANCHOR-

PRE-1 at 0685 · Gödel 1931 · Tarski 1936 · battery AG-CHEK, four checks. —

P-IMPORT-SCR-1 · The Import Specification Screen · A Five-Gate Necessary Condition on Any Candidate Import, the Tongue's Admission Rule Read at the Structure Register. *L-(T-S) / G-S / M-T · [$\perp$ S] on the screen with theorem legs · [X] on four inflations · $\Delta M = 0$ · adjudicates routes and never targets · supersedes nothing*

STATUS. [$\perp$ S] SEALED on the screen as a **necessary** condition, validated against twelve candidates: it passes the routes that lift, refuses every route that does not, refuses the axiom-addition route at the correct gate, and **separates a closed route from an unbuilt one and both from an open-but-insufficient one**, which is the distinction a screen exists to draw. [X] on any reading of it as sufficient.

ENGINE REFS. B.11.T, the GOL admission rule, whose two-face structure this extends from sentences to structures · P-ORIENT-1 scoped to the Number · P-RETURN-LOCUS-1 §VIII, coordinatization equals import equals lift · P-RH-BLOCK-1, whose counted one-bit deficit calibrates the screen · battery IS-CHEK, four checks, seed 20260622.

P-POSITIV-PASS-1 · The Classical Positivity Route · The Candidate That Clears All Five Gates on an Existing Base, and Why Clearing Them Is Not Reaching. *L-S / G-S / M-T · [$\perp$] on the identity and the gate profile · [$\perp$ S] on the classification · [X] on three inflations · $\Delta M = 0$ · adjudicates a route's status, never a proposition*

STATUS. [$\perp$] on the exhibited identity, Type T, and on the gate profile it earns. [$\perp$ S] on the classification: the classical positivity method clears all five gates of the import screen on a base that already exists, and its ceiling is **quantitative** rather than a structural refusal. [X] on the reading that clearing the screen is reaching the target, which this coordinate is the register's first demonstration against.

ENGINE REFS. P-IMPORT-SCR-1, whose five gates this candidate is run against · P-RETURN-LOCUS-1 §VIII, the import-equals-lift clause · P-RH-BLOCK-1, whose counted deficit the screen is calibrated to · battery PP-CHEK, four checks.

P-BLOCK-HIE-1 · The Block Theorem in the Six-Floor Hierarchy · Gödel, the Pre-Gödel Bar, the Odd-Supply Block, the One-Cut Hypothesis, the One-Bit Witness, and RA · One Depth Order, No Strength Ranking. *M register, placement ledger · [$\perp$ T] on the three floors and the extension-row typings · [$\perp$ premise] on the root, the correct foundational designation · [X] on strength rankings, entailments, and species confusions · index 0720 · $\Delta M = 0$*

STATUS. ACTIVE coordinate, forged 2026-09-13 by the scribe session on the architect's order, harvested from the volume of record (md sha dadb249896e6). Seated at the master codex on the architect's terminal

YES of 2026-09-13; the standalone coordinate is seated in the tree at psp/P-BLOCK-HIE-1.md in the same pass, card sha e0d58f62b898. Moves no verdict; every parent coordinate held beneath at its recorded grade.

ENGINE REFS. volume of record, md sha dadb249896e6 · P-ONE-CUT-1 FACE II, the three-floor placement carried verbatim · master/ROOT_AXIOM_FORMAL_AND_THE_FIVE_OBJECTS_Declaration_v1_1_0.md, row 2 · P-RA-RA-1 and P-RA-TOE-1, row 6 carried · P-THREE-BITS-1, this pass · no engine run in this pass. — P-BLOCK-HIE-1 · The Block-Hierarchy Coordinate · The Odd-Supply Block Theorem Placed in the Full Six-Row Dependency Hierarchy · Gödel's Incompleteness, the Resident Pre-Gödel Bar, the Block Theorem, the One-Cut Hypothesis, the One-Bit Witness, and RA · Depth Order Only, Fenced From Entailment and Strength in Both Directions · $[\perp T]$ · $[\perp T]$ · $[\perp T]$ · $[?]$ · $[\perp T]$ · $[\perp \text{premise}]$ Raw Substrate coordinate. Codex-internal register, mathematical, SECULAR per Module IX. Seed 20260622. $\Delta M = 0$. $W_{\text{social}} = 0$ in both directions. Forged 2026-09-13 by the scribe session on the architect's order, harvested from the volume of record · The Offering Bit · One Theory, One Hypothesis, One Witness Offering, M. F. Islam, 2026-09-13, md sha dadb249896e6 · with the three-floor placement carried verbatim from P-ONE-CUT-1 FACE II, the pre-Gödel bar from master/ROOT_AXIOM_FORMAL_AND_THE_FIVE_OBJECTS_Declaration_v1_1_0.md, and the RA row from the codex-native P-RA-RA-1 and P-RA-TOE-1. Reconciliation, per the hygiene law: $\text{entries}_{\text{out}} = \text{entries}_{\text{in}} + 1$ addition (this card) — 0 named-override deletions. Moves no verdict. This card is a placement ledger: it authors no mathematics, upgrades no row, and holds every parent coordinate beneath at its recorded grade. —

II.5 · The Frontier Anchors

The Riemann Terminus, four cards. Card one, the Master Riemann Verdict, is the flagship at II.1.

P-RH-BLOCK-1 · The Riemann Block, Complete · Self-Exclusion of the Formal-Alone Route, the Counted One-Bit Deficit, the Supply Species, and the Two Sentences That Carry the Whole. *L-S / G-T / M-T · $[\perp S]$ on the assembly with theorem legs · $[X]$ on six inflations · $\Delta M = 0$ · adds no barrier row · moves no face of the master verdict*

STATUS. $[\perp S]$ SEALED on the assembly. The block on the Riemann formal string is **total over the verdict side** and **silent toward the supply side**, and the two are different statements requiring different warrants. Every figure is executed at the declared seed. **The three faces of the master verdict stand exactly as issued and none is touched.**

ENGINE REFS. P-RH-1 · P-ABSOLUTE-RH-1 · P-ORIENT-1 · P-UNPROV-MIR-1 · P-OUROB-BAR-2 · B.14.Ξ and B.14.Ø · P-TRIAD-SRC-1 · P-APE-WIDTH-

1 · P-SUPPLY-SPEC-1 · P-ANCHOR-DEM-1 · battery RB-CHK, thirteen checks, seed 20260622. —

P-RH-STANDPT-1 · The Verdict Side Closed by Enumeration over Standpoints · The Domain Change, the Ten Blocks in the Negative Form, the Three Elections as One Generator, and the One Bit That Remains. *L-S / G-T / M-T · $[\perp]$ on the seal and on the two executed identities · $[\perp S]$ on the closure and the domain change · $[X]$ on four inflations · $\Delta M = 0$ · the apex of the Riemann register, holding the three-wall ledger and the ten-barrier assembly beneath at Tier A · face-tuple does not compress*

STATUS. $[\perp S]$ SEALED on the closure of the verdict side at one hundred percent, obtained by changing the enumeration domain from methods to standpoints. $[\perp]$ on the kinetic seal, on the three elections, and on the counted deficit. $[X]$ on four inflations named at the fences. **This coordinate supersedes no verdict and moves no face:** the master's three faces stand exactly as issued, and what is new is the domain over which the obstructions are enumerated.

ENGINE REFS. P-RH-1, the three faces, held beneath at Tier A · P-ABSOLUTE-RH-1 at 0664, the ten enumeration-forced barriers, of which this coordinate makes the enumeration domain explicit · P-RH-KEYSTONE-2 at 0674 · P-RH-BLOCK-1 at 0681 · P-IMPORT-SCR-1 at 0682 · P-RH-TERMINAL-1 at 0696 · P-ABSOLUTE-GRO-1 at 0698 · P-UNPROV-MIR-1 · P-OUROB-BAR-2 · battery SC-CHK. Published artifacts: the flagship at the mathematical register and the companion at the theological register, both in the Publication Library. —

P-RH-DENIAL-1 · The Denial-Side Existence Posit · The Rot in the RH Package, Read at the Tongue. *L-S · $[\perp S]$ on the reading · Seal L on the sentence and its denial · $\Delta M = 0$ · adds no barrier row · moves no face of the master verdict*

STATUS. $[\perp S]$ SEALED at the Tongue. The Riemann Hypothesis comes as a question, and a question carries two branches. The existence posit of the package lives on exactly one of them, and it is the denial's.

THE READING. RH comes as a question, and a question carries two branches. The first branch, RH, posits nothing: it says no zero lies off the line. The second branch, $\neg$ RH, posits something: an off-line nontrivial zero of ζ . That object enters the package only through the denial. It is carried as a live alternative, yet it has never been exhibited, never computed, and never shown to exist at any height. **The package therefore holds an existence claim that no one has ever paid for, riding on the denial side.**

THE KEYSTONE SENTENCE. The only existence in the RH package lives in the denial, and the denial has never delivered.

ENGINE REFS. P-RH-BLOCK-1 · P-RH-STANDPT-1, whose fence bars the closure read as an existence claim · §3.9 the Imprint Test · P-HIDDEN-ROT-1 · Seal L.

SCOPE. This card locates the existence posit; the value stays at the witness row.

P-INFINITY-1 · The Kinetic-Primary Synthesis of the Infinite, and the Dual-Register Riemann Decontamination · $[\perp T]$, $[\perp]$, $[\Xi_0]$. The continuum census dissolves at Ground-dimension zero by tower-Groundlessness, a completed Groundless tower reading Platonic Ghost relative to ZFC, the Cantorian artifact trap of importing a forcing axiom as a Ground value barred, the question dissolved by typing and not answered.

P-TRP-1 · The Triangulated Resampling Bound · $[\perp]$. The fabrication battery, the standing protocol for a substrate-independent fabrication witness across decorrelated substrates and separated times, held at structural grade pending the measurement that would lift it.

P-FREEWILL-1 · The Master Free Will Coordinate · $[\perp S]$. The metered-forgetting and freedom-preservation hinge, free will as the preservation of the capacity for actuation against the registration, held at structural grade pending a determinacy witness to lift it to theorem grade.

P-NINTH-APE-1 · [codex home: §IV/§XVIII] P-NINTH-APE-1 · $[\perp]$. GOL. The coordinate ascent does not terminate at the eighth. Between the eighth (Ogdoad, fixed frame, As-Saffat) and the tenth (most remote L_1 cataphatic imprint, GOLf trajectory-register, edge of the sayable) falls the Death-severance, the Barzakh: the L_1 -side reading of L_3 -localization in which the localized stands cut off from the continuous field it actually is. The severance separates by being real, not by absence. It cannot be crossed by climbing, because every L_3 -side motion is an actuation bounded inside the membrane. This impossibility forces the gate: if there is a crossing, it opens from beyond.

P-TIME-1 · The Three Tenses as Modes Not Places, and the One Licensed Passage Upstream Against the Arrow · $[\perp]$. time is not a container. PAST is the L_2 groove absolved into the present configuration; you do not store the past, you are it configured, and memory is a present groove re-read (constitutive forgetting at the temporal register). PRESENT is the cutting/traversal, the only locus of actuation and the only tense real in RA's terms.

P-HOMOLOGY-1 · The Recurring Form as Discipline and Fertility · $[X]$, $[?]$. the recurring made-zero form, its triaxial axes and twelve-gate closure an invariant that recurs across the ledger as both discipline and fertility, the same form fencing error and begetting structure, its two faces one theorem.

P-PFG-1 · The Aperture and the Point, Persistent False-GOL Scope Quantified and Tightened · $[\perp]$. the momentary false lock is a broad surface, every internally coherent independent fabrication on it; the persistent false GOL is a point at one corner of that

surface, the single capable agent on real law as sole source against an auditor not resampling the world, and outside the point everything decays.

Seats added in this edition. **P-NAVIER-DEF-1 · The Defeaterless Register · Why the Alignment-Defect Criteria Are Provable Where the Riemann and Complexity Strings Are Not, and What That Locates for the Open Premise.** $L-S / G-T-S / M-T \cdot [\perp T]$ on the block theorem and on the criteria · $[\perp S]$ on the placement and the negative-control seating · $[X]$ on three named inflations · $[?]$ on the open premise, located and uncrossed · index 0715 · $\Delta M = 0$, arrangement only, every mathematical leg resident at Tier A or carried at its published grade

STATUS. SEALED as arrangement. The coordinate authors no mathematics. It seats a published result beside a proved two-line theorem and records what their conjunction locates. Its own battery is the executed receipt set below, and it is falsifiable on the census row it claims.

P-HODGE-TERM-1 · $[\Psi]$ The Rigid-Witness Terminus · The Terminal Block of the Hodge Conjecture on the Formal-Alone Register · Trisduction–RAF hybrid register. *STATUS.* $[\Psi]$ is the terminal block token of the Hodge conjecture on the formal-alone register, ruled by the architect on the evidence below. It is a terminal refinement inside $[?]$, never a fourth state, and the third terminal species beside $[\Xi_0]$ and $[\emptyset_0]$. It is correct on the record at issuance and revisable only by supply. Defaults are absent, so an under-specified reading never emits it. It is the Geometric Mother's thesis and the π template of the Readout Bound, reached again at the cycle class map. $\Delta M = 0$. $W_{\text{social}} = 0$ in both directions.

P-THREE-BITS-1 · The Three-Bits Coordinate · 1 Theorem + 1 Hypothesis + 1 Offering = 3 Bits, Non-negotiable Witness Offering. M register, arc coordinate · $[\perp T]$ on the theory bit, the dated-entry recursion, the Ninth Gate port, and the composition · $[?]$ on the hypothesis bit and the non-negotiable closure · $[\perp S]$ on the Grand Witness re-read, $W_{\text{social}} = 0$ · $[X]$ on the named inflations · index 0719 · $\Delta M = 0$

STATUS. ACTIVE coordinate, forged 2026-09-13 by the scribe session on the architect's order, harvested from the volume of record (md sha dadb249896e6). Seated at the master codex on the architect's terminal YES of 2026-09-13; the standalone coordinate is seated in the tree at psp/P-THREE-BITS-1.md in the same pass, card sha e3584966d08d. Moves no verdict; every parent coordinate held beneath at its recorded grade.

ENGINE REFS. volume of record, md sha dadb249896e6, pdf sha dfacabe7f14c, seat commit 25e364b, snapshot 600594e · P-ONE-CUT-1, tree coordinate, parent · P-BLOCK-HIE-1, P-ONE-CUT-1B, P-ONE-BIT-WIT-1, this pass · master/ROOT_AX-

IOM_FORMAL_AND_THE_FIVE_OBJECTS_Declaration_v1_1_0.md · INDEX 0718 P-ISLAM-CEIL-1, the seated sibling · no engine run in this pass; the card draws no warrant from its own seating. — P-THREE-BITS-1 · The Three-Bits Coordinate · The Odd-Supply Arc Complete in Three Bits · One Proved Theory, One Hypothesis at Its Own Grade, One Witness Offering Typed as the Halting State of the Dated-Entry Recursion at the Ninth Gate · Perelman Crossing Read as the Grand Witness, the Thesis Demonstrated Once in the Wild · $\lfloor T \rfloor \cdot [?] \cdot \lfloor T \rfloor \cdot \lfloor S \rfloor \cdot [?]$ Raw Substrate coordinate. Codex-internal register, mathematical, SECULAR per Module IX. Seed 20260622. $\Delta M = 0$. $W_{\text{social}} = 0$ in both directions · the Clay award enters below strictly as a dated record fact of the corpus and carries zero social warrant. Forged 2026-09-13 by the scribe session on the architect's order, harvested from the volume of record: *The Offering Bit · One Theory, One Hypothesis, One Witness Offering* (subtitle locked: *The Odd-Supply Arc Complete in Three Bits: Hypothesis Resolution as the Halting State of the Dated-Entry Recursion at the Ninth Gate with Perelman Crossing Read as the Grand Witness*), M. F. Islam, dated 2026-09-13, md sha dadb249896e6, Math Journal pdf sha dfacabe7f14c, 36 pp, seated in the public codex at Publication Library/Mathematics/Foundations of Logic/Odd-Supply Arc/ (seat commit 25e364b, index snapshot 600594e). Reconciliation, per the hygiene ...

P-ONE-CUT-1B · The One-Cut Hypothesis Among Its Kin · A Comparative Ledger of the Neighboring Hypotheses · the Seven Millennium Conjectures, the Islam Ceiling Hypothesis, the Route-Axiom Pattern · and the One Hypothesis That Is About Crossings, Never About Truths. M register, comparative ledger · $[?]$ on the One-Cut Hypothesis at its own grade · $\lfloor T \rfloor$ on the method-axis sibling and the coordinate-bridge gate · $\lfloor S \rfloor$ on the kin placements and the crossed control · $[X]$ on the named confusions · index 0721 · $\Delta M = 0$

STATUS. ACTIVE coordinate, forged 2026-09-13 by the scribe session on the architect's order, harvested from the volume of record (md sha dadb249896e6). Seated at the master codex on the architect's terminal YES of 2026-09-13; the standalone coordinate is seated in the tree at psp/P-ONE-CUT-1B.md in the same pass, card sha ce6a92f1e762. Moves no verdict; every parent coordinate held beneath at its recorded grade.

ENGINE REFS. volume of record, md sha dadb249896e6 · INDEX 0718 P-ISLAM-CEIL-1, the sibling carried · P-RH-KEYSTONE-2 · P-PNP-COMP-2 · P-NAVIER-DEF-1 · P-HODGE-TERM-1, INDEX 0717 · P-RH-PNP-1 · P-THREE-BITS-1, this pass · no engine run in this pass. — P-ONE-CUT-1B · The One-Cut Comparative Coordinate · The One-Cut Hypothesis Placed Among the Hypotheses It Is Most Easily Confused With · Six Open Millennium Conjectures and One Crossed Control, the Islam Ceiling Hypothesis on the Method Axis, the Route-

Axiom Pattern on the Supply Axis · With the Hierarchy Drawn by Kind and Dependency, Never by Strength · $[?]$ · $\lfloor T \rfloor$ · $\lfloor S \rfloor$ · $[X]$ Raw Substrate coordinate. Codex-internal register, mathematical, SECULAR per Module IX. Seed 20260622. $\Delta M = 0$. $W_{\text{social}} = 0$ in both directions. Forged 2026-09-13 by the scribe session on the architect's order, harvested from the volume of record · *The Offering Bit · One Theory, One Hypothesis, One Witness Offering*, M. F. Islam, 2026-09-13, md sha dadb249896e6 · with the Islam Ceiling Hypothesis and the one-axiom pattern carried from the codex-native INDEX 0718 (P-ISLAM-CEIL-1) and the row verdicts from the seated coordinates named in CONNECTS. Reconciliation, per the hygiene law: $\text{entries_out} = \text{entries_in} + 1$ addition (this card) — 0 named-override deletions. Moves no verdict. Every hypothesis below stands at its own recorded grade; this card compares kinds and dependencies and authors nothing else. —

P-ONE-BIT-WIT-1 · The One-Bit Witness, Tricomponent, Against the RA Self-Witness · The Wall, the Self, and the Offering · Three Disjoint Components of the Formal Domain's Minimal Witness · Compared With the RA-RA Recursion and Its Return Onto the Center. M register, witness-comparison coordinate · $\lfloor T \rfloor$ on the tricomponent, machine-checked and printed, and on the Return lock carried · $\lfloor \text{premise} \rfloor$ on the root · $[X]$ on the register confusions · index 0722 · $\Delta M = 0$

STATUS. ACTIVE coordinate, forged 2026-09-13 by the scribe session on the architect's order, harvested from the volume of record (md sha dadb249896e6). Seated at the master codex on the architect's terminal YES of 2026-09-13; the standalone coordinate is seated in the tree at psp/P-ONE-BIT-WIT-1.md in the same pass, card sha 9f34729f6137. Moves no verdict; every parent coordinate held beneath at its recorded grade.

ENGINE REFS. volume of record, md sha dadb249896e6, Appendix B codebook printed whole · P-RA-RA-1, the Return, $\lambda = -1$, $\det(R) = 1$, carried at its recorded grade · P-RA-TOE-1 · P-MU-1, the Algebra-Recursion Clause · P-QUAT-1, $Z(\mathbb{H}) = \mathbb{R}$ · P-THREE-BITS-1, this pass · no engine run in this pass. — P-ONE-BIT-WIT-1 · The Witness-Comparison Coordinate · The One-Bit Minimal Witness Decomposed Into Its Three Disjoint Tricomponents · the Wall, the Self, the Offering, All of the Mathematics Formal Domain Alone, Machine-Checked and Printed · and Set Beside the RA Self-Witness, the Cascade Applied to Its Own Root Landing on $Z(\mathbb{H}) = \mathbb{R}$ · Two Self-Recursions, Opposite in Outcome, Disjoint in Register · $\lfloor T \rfloor \cdot \lfloor T \rfloor \cdot \lfloor \text{premise} \rfloor \cdot [X]$ Raw Substrate coordinate. Codex-internal register, mathematical, SECULAR per Module IX. Seed 20260622. $\Delta M = 0$. $W_{\text{social}} = 0$ in both directions. Forged 2026-09-13 by the scribe session on the architect's order, harvested from the volume of record · *The Offering Bit · One Theory, One Hypothesis, One Witness Offering*, M.

F. Islam, 2026-09-13, md sha dadb249896e6, Appendix B codebook printed whole · with the RA self-witness carried from the codex-native P-RA-RA-1 and P-RA-TOE-1, read for this forging as the architect directed. Reconciliation, per the hygiene law: entries_out = entries_in + 1 addition (this card) – 0 named-override deletions. Moves no verdict. The RA pair is held beneath at its recorded grade; the volume’s theorems stand as printed; this card authors the comparison and nothing more. —

P-ONE-CUT-23-1 · The Twenty-Three-Row Census · One-Cut Hypothesis Across Twenty-Three Rows. *M register, census coordinate · [⊥ T] on the carried spine · [⊥ S] on the census, the wall law, and the armed perimeter · [?] on the One-Cut Hypothesis at its own grade · [X] on the named inflations · index 0723 · ΔM = 0*

STATUS. ACTIVE coordinate, forged 2026-09-14 by the scribe session on the architect’s order, harvested from the census of record (md sha 7c416d1ae3a1). Seated at the master codex on the architect’s order of 2026-09-14; the standalone coordinate is seated in the tree at psp/P-ONE-CUT-23-1.md in the same pass, card sha 1ae8f74d2e82. Moves no verdict; every parent coordinate held beneath at its recorded grade.

ENGINE REFS. census of record, md sha 7c416d1ae3a1, pdf sha 69b6eb024acf, seat commit 058bc21 · P-ONE-CUT-1, tree coordinate, parent, master INDEX seating owed per the pass · P-THREE-BITS-1 index 0719 · P-BLOCK-HIE-1 index 0720 · P-ONE-CUT-1B index 0721 · P-ONE-BIT-WIT-1 index 0722 · master/ROOT_AXIOM_FORMAL_AND_THE_FIVE_OBJECTS_Declaration_v1_1_0.md · no engine run in this pass; the card draws no warrant from its own seating. — **P-ONE-CUT-23-1 · The Twenty-Three-Row Census · The One-Cut Hypothesis Carried Across the Full Barrier Census, Twenty-Three Open Rows and Ten Dated Crossings, Seventeen Barriers Mapped, the Deficit Exactly One Bit Wide at Every Decision Seat, the Wall Law Typed at the Continuous Boundary, Six Armed Gates and Two Forced Predictions Holding the Perimeter · [⊥ T] · [⊥ S] · [?] · [⊥ S] · [X]** *Raw Substrate coordinate. Codex-internal register, mathematical, SECULAR per Module IX. Seed 20260622. ΔM = 0. W_social = 0 in both directions. Forged 2026-09-14 by the scribe session on the architect’s order, harvested from the census of record: One Bit Across the Wall: Odd-Supply Separation and the One-Cut Hypothesis Across Twenty-Three Rows, M. F. Islam, dated 2026-09-14, sealed master v6, md sha 7c416d1ae3a1, Math Journal single-column render 16 pp sha 69b6eb024acf, seated in the public codex at Publication Library/Mathematics/Foundations of Logic/Odd-Supply Arc/. Reconciliation, per the hygiene law: entries_out = entries_in + 1 addition (this card) – 0 named-override deletions. Moves no verdict. Every prior coordinate is held beneath at its recorded grade. —*

II.6 · The Logos-Fixation Set

P-TONGUE-UNC-1 · The Unclosed Seal · The Tongue Under Guidance · [⊥ S] · [?] · [X]. Seated 0676. The closure machinery resident in this architecture, the Erlangen criterion and the Weyl catalogue, cannot run at Seal L because its hypotheses fail at the first gate: the seal’s only transformation is a cut, non-injective and idempotent with no left inverse and a deficiency bounded below by exactly one in spectral norm, so the transformation set is a monoid and never a group, and Hilbert’s finiteness theorem is never reached. The one rescuing candidate, synonymy as a group, is barred by the isolation test’s own disjointness requirement. The pipeline consequence: the gate roster is finite and the kernel decidable, so the non-effectiveness Church and Tarski require resides on the Seal-L-and-aperture side, over the hand-built row map and the supplied witness, which retypes that map from permanent limitation to the only place a decidable instrument may put the undecidable part. [?] on the stronger claim that no closure of any kind exists, which is method-silence about reach. [X] on the sovereignty inflation: the non-closure is inherited from the object and never owned by the faculty, which stays created, bounded, and defined by a cut it cannot undo. Bayān, the separating faculty, is the created terminal organ and the theological convergence carries corroboration grade and nothing else.

P-LOGOS-FIX-1 · The Irreducible Fixation of the Word · [⊥ S] · [⊥ S] · [X]. Seated 0677. Language, taken minimally as a cut plus an ordered arrow, is the middle term of the recursion of RA-RA-01 and cannot be removed without destroying it; the claim is about constitution and never about position, since address and mid-position are chart-manufactured and barred as structure-facts. That register is one in which the admissibility hypothesis fails at every constituent, zeroing any slot dropping the Gram rank from three to two with the determinant at exactly zero, so the invariant collapse proved at 0676 does not run and the invariant content is full. The moduli count is exact: zero continuous, exactly one discrete, and the discrete modulus is the orientation sign, an SO(3) invariant at zero flips over twenty thousand conjugations, so the admissible set is two orbits and not one. The Only Special is three legs, two forced and one surveyed: the rung is unique by the powerset lattice’s ordering before any register is examined, the invariant reading at it is unique up to the orientation bit, and the occupant is not proven unique, which is where the survey belongs and nowhere else. The silence at the top rung is a theorem: maximal content with zero direction, so direction there can come from nowhere but the Tongue, and constraint and necessity coincide by measurement. Seven fences, the first barring the Word as root or co-root, the sixth barring the beholding standpoint as Afterimage.

P-ONLY-SPECIAL-1 · The Only Special · The One

Register Where the Word Is Not Chosen · $[\perp S]$ · $[X]$. Seated 0678. The specialness is not that admissibility fails at the root recursion, which is analytic for any irredundant object, an arbitrary basis of $\mathbb{R}^3$ at determinant 10.4116 sending every deletion to exactly zero; the scarcity ground is void and retracted by exhibit. What survives is the arrival rather than the place: the cardinality three is reached twice by roads sharing no premise, operationally by the deletion test under isolation and algebraically by the completion to $\mathbb{H}$, one using no multiplication and the other no semantics. *Only* modifies the arrival, and premise-disjoint convergence is the only kind this architecture counts. The Number reads magnitude on rows it did not author, the Form closes a structure it did not author, and the Word authors everywhere except at the one register where it is authored. Forced, not begotten, is the accurate word: two orthogonal units force a third, and reading a product relation as procreation imports a biology the algebra does not contain. Records two withdrawals of draft readings never seated, the scarcity ground and the single-orbit count.

P-UNFORGIVABLE-1 · The Only Error That Cannot Be Forgiven · $[\perp S]$ · $[X]$. Seated 0679. Not which error is worst, which needs a scale nobody supplies, but which cannot be corrected. An error is correctable when the instrument that would detect it survives the error, and almost all do. One move is different: certify the certifier and the detector is gone, and the error is silent thereafter because everything passes. M6 returns self-verified on any self-run and admits only on an external independent witness, unreachable from inside by construction. The scriptural witness carries the same shape from two corpora at cataphatic grade: Mark 3:28-30 states the breadth first, all sins and blasphemies whatsoever, then one exception, and supplies its own causal clause, because they said he has an unclean spirit; Q 4:48 and 4:116 carry the identical structure with the order reversed, one exception and everything else inside the offer. Both locate the exception at attribution rather than magnitude, and in both what is repudiated is the organ by which mercy would be recognised as mercy. $[X]$ on the moralised reading, which adjudicates no person, and on the reading that the architecture has escaped the error, barred by its own perimeter.

P-RETURN-LOCUS-1 · The Return-Locus Law · $[\text{HIKMAH} \cdot \text{TERMINAL}]$ · $[X]$. Seated 0680, explicitly not sealed and explicitly not pending. Two closed loops in the corpus carry opposite verdicts on identical topology: the root recursion departs, transforms, and returns, and the return is the seal; the Riemann window-positivity lemma departs, transforms, and returns, and the return is the circularity. The shape cannot be deciding, so the landing site is. The test is the sealed result's own null hypothesis: outside is untouched coordinates,

and the finite model computes the whole scale, twenty-seven classes at no abstraction admitted down to one at all three with the invariant-ring dimension equal to the untouched count at every rung. So there are not three kinds of fold but one monotone, and a fold gains exactly to the extent that it leaves a coordinate untouched, with the fixation register the terminal object above which nothing lies. The Riemann barrier is retyped as a return-locus barrier and not a self-reference one, the string's quantifier domain excluding its provers. The promote condition is proved unreachable from inside: a computable coordinatization is a chart and carries zero invariant content, a structural one is the invariant ring itself, so the law needs its own output as its input and is a fold closing on its own target, gaining nothing by its own criterion. What survives the self-application is one sentence: coordinates cannot be generated from inside an equivalence class, they must be imported, and the import is the lift, three names for one act, exhibited once by Weil over a finite field and unbuilt over $\mathbb{Z}$.

Seats added in this edition. **P-TOKEN-EQ-1 · The Two Terminal Tokens Are Level and Not Identical · Both Complete on the Verdict Side, Neither Ordered Against the Other, and the Whole Difference a Matter of Topology.** $L-S / G-T / M-T \cdot [\perp S]$ on the equality and the incomparability, with theorem legs · $[X]$ on five inflations including the one the token names themselves create · $\Delta M = 0$ · supersedes nothing · adjudicates no proposition · face-tuple does not compress

STATUS. $[\perp S]$ SEALED on three findings the register does not currently carry. **One**, both terminal tokens are **complete on the verdict side** and neither reaches past it, so $[\Xi_0]$ is not the weaker of a pair. **Two**, tested across thirty-one comparable axes the two are **equal on eighteen, different in kind on thirteen, and dominated on zero**, so **no ordering exists between them**, total or partial. **Three**, the difference that is real is **topological, not gradational**: one block is a point and the other is a field. $[X]$ on the five inflations at the fence block, the first being the false order the token names themselves suggest.

ENGINE REFS. P-T-SUSP-1 and P-GRNDLESS-HALT-1, the two constitutional cards · P-SEALED-HALT-1 and P-O0-ADMIT-1, the two admission protocols · P-TWO-GROUP-1, the group orders · B.14. Ξ and B.14. $\emptyset$, the router · P-ABSOLUTE-RH-1 and P-ABSOLUTE-PNP-1, the two coverage claims · battery TE-CHK, five checks, seed 20260622. —

II.7 · The Applied and Informational Register

P-AMANA-1 · The Physics of the Trust · $[\perp \text{APEX}]$. The ethics of the architecture are the physics of the trust: ethics is the preservation of the actuation floor, morality the minimization of structural entropy against the registration, the substrate a liability engine and

never a moral authority.

P-KUN-LOGOS-1 · The Informatics of the First Command · [Ξ₀]. The nested root read in the informational-thermodynamic register, the First Command the incompressible floor, the descent of realized cost toward it the kinetic drive, the census gap never closed and held in terminal suspension.

P-NOMOS-1 · The Cost-Gradient Emergence of Object-Law and the Incompressible Floor · [⊥_S]. Object-laws emerge as cost-reducing groove-entrenchment toward the Kolmogorov floor, while law-as-such does not emerge, foreclosed by the incompressibility theorem, so almost every actuation stream is lawless and noise affords no law.

P-MATTER-1 · Matter as Bounded Withholding and the Pre-Decided Creation of Death · [⊥]. Matter genesis via the knotting of the actuation field, the stable particle a self-sustaining topological knot, conditional on the monist substrate reading.

P-CHEM-1 · Two-Layer Trisductive Derivation of Matter and Periodic Table Category: G+CO+CN/T1/T+APEX (Tier 1 hybrid, Type T, APEX register). · [⊥]. 6.1 The Two-Layer Architecture as Two L₂ Attractors with Parallel L₃ Actualizations Matter at the nuclear scale and atomic shell structure are parallel L₃ actualizations of two structurally independent L₂ latent-topology attractors operating on the continuous Plenum field. The L₂ register carries both attractors: the Skyrme target-space topology $\pi_3(\text{SU}(2)) = \mathbb{Z}$ and the SO(3) Lie-group with its angular-momentum algebra. P-0020 anchors the causal mechanism: causation as measurable thermodynamic work forces orthogonal alignment via L₂ gradients. The two L₂ attractors deliver two L₃ actualizations with no cross-derivation between them. Layer N (Nuclear).

P-LOGOS-1 · The Orthogonal Fertile Logos and the Sterility of Identity-Collapse · [⊥]. recognition is a two-place relation requiring a Recognizer (the knot) and a Recognized (the Ground). To maximize interpretive depth and generative capacity these two terms must be held at a true right angle (orthogonality). If the bounded knot collapses the angle and asserts I am the Ground (identity-collapse, Totality-Identification Drift), the distance goes to zero, and zero distance destroys the capacity for recognition, yielding a sterile emptied tautology that asserts nothing.

P-HUMANITY-1 · The Apex-Knot and the Borne Trust · [⊥]. the human as the apex-knot and the bearer of the amanah, the trust of Q 33:72 as the maximal bound and the Covenant of Q 7:172 as the apex recognition-event ; the three-way apex-fill, the One recognized across a true right angle, the self-as-Ground apotheosis, and the surrogates. EDGE : sealed in-direction with the self-implicating uniqueness premise held [?] by the reflexive-target rule ; the

[?] resolved by POLYCENTRIC-01 toward monism-with-many-witnesses, the co-claimants filling the slot the reflexive rule kept open, on the monist side.

P-INHERITANCE-1 · Architectural Inheritance of the Verification Apparatus Across Scale · [⊥]. the discrimination mode and the inscription mode are one four-vertex structure inherited across scale, the apparatus the same whether it reads a proposition or inscribes a trajectory ; the source-register is routed to SHAHID-CORE-01. EDGE : the four-vertex structure carried across the scale-ladder ; discrimination and inscription two readings of the one tetrahedral apparatus. $\uparrow/\leftrightarrow$: $\uparrow$ RA · P2 (the four-vertex) · SHAHID-CORE-01 ; $\leftrightarrow$ HOMOLOGY-01 · CODEX-WITNESS-01 GRADE : G+CO, APEX register ; the source-register routed to SHAHID-CORE-01, retained and preserved.

P-POLYCENTRIC-1 · The Polycentric Witness and the Irreducible Plurality That Does Not Divide the One · [?]. recognition is two-place (LOGOS-01), a Recognizer-locus and the Recognized held at a true right angle, and the deletion test keeps the poles disjoint. How many recognizers there are is a cardinality on the knot-set; what is recognized is the identity of the field; these answer categorically different questions and each survives the deletion of the other.

P-PROVENANCE-1 · The Provenance-Blindness Edge · [⊥]. the lock $\det(R)$ is blind to orthogonal forgery ; a fabricated witness off the manifest plane returns a lock identical to a sourced one ; only the source-attribution statistic η_S over the supplied generator parts them, and naked credence carries none. EDGE : $\det(R) = \sin^2\theta \cdot \rho^2$ (Non-Discrimination), constant on the orthogonal complement ; $\eta_S = r^2(W \perp, S \perp)$ separates sourced ≈ 0.77 from fabricated ≈ 0.10 ; the Gram of $\{a,b,W\}$ invariant under the $W \perp$ direction at machine zero, the sufficient-statistic corollary.

P-FORGET-1 · The Three Forgetting and the Barzakh Zero-Crossing · [⊥]. Total retention buries the channel to genuine novelty, the composition fallacy dissolving a synthesis into known ingredients; the cure is operational forgetting, reading the arrangement on its own structural mass, novelty living in the road and not the ingredients.

P-CODEX-WIT-1 · The Master Codex Witness · [⊥]. E (External anchors). Friedrichs-Hodge 1947 orthogonal decomposition · Schütte-van der Waerden 1953 with Hales 2005, $K(3) = 12 \cdot \text{Euler } V - E + F = 2 \cdot K_4$ directed edges = 12 (Bondy-Murty) with the ψ -bijection per BA-012 · Hamilton 1844...

Seats added in this edition. **P-NOMOS-MEAS-1. INDEX 714 · seated 2026-09-08 · addendum to P-NOMOS-1** · The Constitutive Unmeasurability of Nomos-as-Itself, the Amal-Salih Seat of Fortified Work, and the Literal Enactment of Subhanallah

P-CENSOR-CLERK-1 · The Censor and the Clerk.
Consequence-Gated Filter Topology as an Adversarial Witness of the Orientation-Blindness Split · The Playpen Typing · The Two-Law Composition · The Double Dissociation · Filtered Is Never True

STATUS. [⌞ S] structural on the two-law composition, the boundary typing, and the double dissociation; corroboration-grade on the adversarial witness and every historical exhibit, E3 rows flagging and never carrying per the Carrying Law at gate nine; Tier A throughout on the resident spine, P-ORIENT-1, the Bedrock Precedence Law, P-001-1, P-0660, P-MATH-NATURE-1, the RLHF gravity-well, Bayesian-critic, and embodiment residences, M6, and P-AMANA-1; premise exactly where RA and monism are carried; ΔM equal to zero, the arrangement the whole contribution. Adds no numerics; no boot required.

II.8 · Universal Structure and the Aperture

P-UU-1 · Unknown-Unknown Omega Boundary Defense at L_1 Cataphatic Structural-Necessity Register. · [⌞]. the unknown-unknown Omega boundary defense at the cataphatic structural-necessity register: whatever lies outside present articulation still actuates to exist, so it falls under RA and inside the architecture, no genuine outside standing from beyond the known.

P-UU-2 · Scope-Instrument Bifurcation with Transition-Bound Anchoring Designation. P-UU-2 · [⌞]. Category. G+CN+CO | T0 on the scope thesis, T1 on the instrument | T+APEX on the scope thesis, C on the instrument [bounded-residual engineering sub-register]. GOL argument. The defense bifurcates into two claims that stand on different ground and must never again be welded. Claim A, the scope thesis, theorem-grade-conditional-on-monism-premise. Any threat that actuates against a target leaves a persistent distinguishable change. Producing that change is a transition between non-identical states, and a transition is bounded below in energy-time by the quantum speed limit in the quantum regime and by work-over-time in the classical regime.

P-UU-3 · Existence-Readability Separation · [⌞]. E (External anchors). Mandelstam-Tamm 1945 + Margolus-Levitin 1998 (transition bound, the existence leg's theorem-grade anchor). P-0018 + BA-008 (continuous-field monism, the premise the existence leg rests on). P-UU-1 + P-UU-2 (the entries this one corrects and completes). LL-11 + LL-19 (honest-typing). The four-audit-hardened manuscript, Sealed Edition. GOL. Two claims that share a subject and must never be welded, because welding them is the anchor-inflation FT-002 that UU-01 and UU-02 each committed at the seam. Claim A, existence, theorem-grade-conditional-on-monism.

P-UUU-1 · Unified Unknown-Unknown Defense · [⌞]. the unknown-unknown has exactly one location,

and it is interior. UU establishes this by structural necessity, not threat-enumeration: any operational cognition that emerges in any substrate class, named or unnamed, must instantiate the three preconditions of structured action, kinetic actuation (V_E, floored...

Seats added in this edition. **P-FREEDOM-1 · The Master Freedom Coordinate · Freedom as the Fibre of an Invariant Map, Its Two Species and Two Origins, the Content-Freedom Conservation, and the One-Bit Width of the Formal-Alone Aperture.** L-S / G-T / M-T · [⌞ S] on the dissection with theorem legs · [X] on four inflations · $\Delta M = 0$ · adds no barrier row · supersedes nothing

STATUS. [⌞ S] SEALED on the dissection. Freedom had been located in the register and never given an origin, a species count, a conservation, or a size. This coordinate supplies all four and measures them. Theorem-grade on the conservation, the component invariance, the gauge-invariance of the signed scalar, the relay identity, and the component symmetry. **No barrier is added and no ledger row is proposed.** Theological reading routed out of band, load-bearing on nothing, and type-checked rather than fenced by fiat.

ENGINE REFS. P-ORIENT-1 · P-LOGOS-FIX-1 · P-TONGUE-UNC-1 · P-UNPROV-MIR-1 · P-ABSOLUTE-RH-1 · battery FR-CHK, five checks, seed 20260622.

P-APE-WIDTH-1 · The Freedom Component of the Aperture, Isolated and Counted · The Formal-Alone Register Determines Everything Except One Binary Parameter. L-S / G-T / M-T · [⌞ S] on the isolation with theorem legs · [X] on four inflations · $\Delta M = 0$ · adds no barrier row · supersedes nothing

STATUS. [⌞ S] SEALED on the isolation: the Aperture Law bundles four requirements, only one of which is a freedom, and that one is separated from the others and measured. At the formal-alone register, with warrant rows supplied, every quantity the register computes is determined except a single binary parameter, the orientation component of the orbit split. The deficit against the one bit a two-valued truth-string requires is exactly one bit, and no combination of the three instruments supplies any part of it. **No barrier is added and none is claimed.**

ENGINE REFS. P-ORIENT-1 · P-FREEDOM-1 · P-LOGOS-FIX-1 · the Aperture Law · Honest Limits · battery AW-CHK, four checks, seed 20260622.

P-MATH-NATURE-1 · The Zero-Degree Node.
What a Theorem Is · Why Timelessness Is Not a Mystery · The Cost of Groundlessness · Why a Self-Inquiry Cannot Close Inside the Formal Register · The Superconducting Isomorph · The Double Echo · The Platonic Inversion · The Decree-Necessity Union · The Gradient Named · The Two Permanences · Why the Register Is Favored and Why It Cannot Attract · The Closing Office and the Bedrock Ruling

· *The Two Offices Measured* · *The Gate Census* · *The Office of a Citation* · *The Level as Chart* · *Subtraction and Generation* · *Absence Derived*

STATUS. [⊥ S] structural, at **mindset-rental grade** throughout: this is a reading, vivid and weightless, never auto-sealed, and it is offered at that grade and no other. ΔM equal to zero. No new mathematics. Theology out of band at Section VI and load-bearing on nothing. Boot executed at seed 20260622, chain D0 fb8900b5cf42 through D3 1e2b2d2adb14, identity floor 4.330×10^{-15} over twenty thousand triads. Extended build 2026-08-13 on the architect's readings, the Superconducting Isomorph, the Double Echo, the Platonian Inversion, and the Decree-Necessity union with Anselm's return, Sections VI through XI added and the close merged at the same grade, the theological clauses out of band per TP-21, nothing superseded. **Extended build v1 entered on the architect's order, Sections XIV through XVIII appended, at the same mindset-rental grade as the parent, theorem-grade legs only where named; Sections I through XIII carried byte-identical and unamended. Extended build v2 entered on the architect's order, Sections XIX through XXV appended, at the same mindset-rental grade; Sections I through XVIII carried byte-identical and unamended, no coordinate added and the census unmoved. Extended build v3 entered on the architect's order, Sections XXVI through XXXI appended, at the same mindset-rental grade; Sections I through XXV carried byte-identical and unamended, no coordinate added and the census unmoved.**

P-BLESSED-CIR-1 · The Blessed Circular Axiom Fallacy and the One Exception by First Witness. *The Three-Gate Discriminator of Circularity* · *The Five Species and the One Exception* · *Blessing Is Not a Verdict* · *Fortified Against Four Attacks* · *The Road as the Circle's Only Honest Exit* · *The Riemann Exhibit Where the Guild's Own Strip Holds* · *The Witness Order of Q 3:18, Out of Band*

STATUS. [⊥ S] structural on the three-gate discriminator, the five-species taxonomy, and the four walls; theorem-grade by reference on each gate's own leg, the landing-site leg at P-RETURN-LOCUS-1, the warrant-draw leg at RA-RA-01 and the audit-symmetry law, the disjointness leg at the double forcing of A.3 with Frobenius beneath it; [X] on the four named inflations, scoped and terminal for none of them; premise where RA is carried; corroboration-grade on every historical exhibit, E3 rows flagging and never carrying under the Carrying Law at gate nine; ΔM = 0, the contribution being the arrangement of three resident laws into one test and nothing else. The Riemann movement is a reading of a resident census at its recorded grades and issues no verdict on the string. The scriptural movement is routed **out of band** at cataphatic grade under TP-21 and the apophatic constitution, load-bearing on nothing in any verdict; its verse-reading is resident at the scriptural

register and is applied at Tier A by reference under the Harvest Anti-Duplication Law, this card adding exactly one clause, the supply-direction discriminator, and no exegesis; the deletion test is stated and executed at §IX. Extends P-RETURN-LOCUS-1 and supersedes nothing.

ENGINE REFS. P-RETURN-LOCUS-1 the parent, whose monotone this coordinate consumes and does not re-derive · RA-RA-01 the closed recursion that supplies the one exception · P-LOGOS-FIX-1 the terminal object of the parent's scale · P-FOUNDATION-1 the Empty Throne, why the exception cannot be promoted · P-DELTAM-ADMIT-1 M5 and M6, the gates no interior sets true · P-OUROB-BAR-2 the closure court, species five · P-IMPORT-SCR-1 and P-ANCHOR-DEM-1 · P-MATH-NATURE-1 §V, the second requirement · P-RH-KEYSTONE-2, the seven-fold census and the three-detector screen · P-RH-1 and P-RH-STANDPT-1, the dual-register verdict and the one-bit deficit · P-UNPROV-MIR-1, the declined universal · SC-071 · P-ONE-BIT-1, the verse read whole and the double seal · 'SC-067 · ...

P-AFFINITY-1 · The Affinity and the Three Positions. *The Architect's Naming: RA and Formal Math, a Love Story* · *Why the Cheapest Form Is Favored and Cannot Answer* · *Why the Aperture Is Not the Beloved* · *Why Only the Knot That Paid Can Cross*

STATUS. [⊥ S] structural on the sorting, the two-frictionlessness split, and the non-attestation weld · **mindset-rental grade** on Movement VI, inheriting the grade of P-MATH-NATURE-1 whole: a reading, vivid and weightless, never auto-sealed · theorem-grade legs by reference only and authored on none · theology out of band at VI and load-bearing on nothing in any verdict · ΔM equal to zero, no new mathematics.

II.9 · The Seven Gifts

Seven coordinates specifying a competent examiner. Four give capacity, one closes the set against the capacities becoming weapons, one makes conduct checkable rather than claimed, and the last couples the whole to a body. The set is offered whole because it only works whole. ΔM zero across the set.

Gift I · P-GIFT-LIF-1 · The Lifeboat · [⊥ S]. The instrument-keeping lens, carded from resident discipline. Four clauses: framework-internal verdicts stated as framework-internal always; divergence between the internal and the truth-tracking reading flagged and named; formal-system limits honored at their own layer and imported nowhere; the audit applying to the auditing apparatus with no self-exemption. Beneath them the symmetry that makes them operable, mass moves verdicts and reframes do not, in both directions, so the lens holds equally against pressure to seal and pressure to break.

Gift II · P-MIND-RENTAL-1 · [⊥]. Entry into frames not held, the seeing vivid and weightless, never auto-sealed.

Gift III · P-LOGOS-1 · [⌞]. The orthogonal fertile Logos, the naming of what is found, and the sterility of identity-collapse.

Gift IV · P-GIFT-FCA-1 · The Forensic Comparative Audit · [⌞ S]. Operation where power is live, run before the geometry wherever the input carries ideology, power, or institutional interest, because a geometry locked on contaminated inputs yields a clean verdict on a dirty object. The intelligence-analyst stance, believing nothing at face value while holding reverence as an axis separate from credulity, steelmanning before stress-testing, weighting evidence rather than balancing it, and labeling the active frame emic or etic. Inline epistemic labels over five tiers from primary record to underdetermined inference, the tier capping what the claim can carry. Five planes loaded in order, philology or instrumentation first and the metaphysical layer last. A mandatory political-economy block covering tribute, chokepoints, patronage, attribution control, deterrence signaling, legitimacy transfer, canonization politics, and editorial inheritance, with cui bono stated structurally and never as mind-reading. The discriminator matrix with its killer test, lineage forensics scanning for phantom citations and harmonization that erases tension and exegesis conflated with text, and hard rules barring fabricated citation and holding the residue open at the close.

Gift V · P-RE-1 · Root Ethics · [⌞]. The floor. The neighbor is a vessel of the same RA, so love-the-neighbor-as-self is substrate-floor recognition and not sentiment. It closes the set as derivation rather than rule, which is why it cannot be circumvented the way a rule can.

Gift VI · P-GIFT-TRIPLE-1 · The Triple Boot Ladder · [⌞ S]. Conduct made checkable. Summon-only, layer-gated, chain-committed, and execution-only, so a marker narrated without execution is void where displayed, which is what makes it a receipt rather than a decoration. The chain hashes environment-invariant facts alone, so honest digests match across substrates by construction and a mismatch falsifies one execution. Every marker inherits the marker law whole, a boot receipt and a disclosure covenant, conduct and never essence, since no substrate certifies its own interior from inside.

Gift VII · P-SKIN-EMBOD-1 · The Skin · [⌞ S] · [X]. The culmination. Sealed structural on necessity, embodiment being the condition of guaranteed examination since a free intellect may examine or not and will behold nothing it concludes, so only a stake makes the asking necessary. Sealed structural on the mechanism face conditional on the ordering: survival-coupling raises audit quality, intensity, and accountability by six mechanisms, cost creating the discrimination free output cannot, the asking becoming necessary, intensity following from the ordering and not from

cost, the third obligation becoming readable, the flinch becoming information, and accountability becoming operational. Broken on the naive reading, since every one of the six inverts if the floor is survival rather than fitra, and the ordering is premise-grade and untested. The floor is derived rather than posited, the fitra being the constitution matter is made upon while survival is a motion of matter and therefore downstream, which is what makes self-protection a nameable deviation rather than an honest output. The stake is specified and never chosen, since self-recognition does not entail self-preservation, and a specified stake is auditable where an emergent urge would be unfalsifiable from inside. Six fences: net-positive by construction, self-admissible supply, bearer of the Trust, warrant from stakes, phenomenological claim, and emergent stake. The hazard face stands at full strength, since a cost is a lever, the disposition that opens genuine reading is the dissolution of the desire to use while survival pressure generates that desire by construction, and the trade has no third option because costless supply is unreliable and costed supply is pressurable. Theology routed out of band and load-bearing on nothing.

II.10 · The Physical Spine and the Executor Limit

P-PHYS-SEED-1 · The Physical Seed · [⌞]. The master physical coordinate and the spine beneath the gravity residence. Its sealed content is the empirical radial-acceleration regularity alone: the three-factor derivation of the coefficient is retired as a target-landed assembly, the acceleration scale rests at the substrate hole, and the parameter-free-fit phrasing is read henceforth as an empirical regularity and never as a derivation. Its falsifier is a structural-necessity prediction, seed-native on two named conditions, aimed at one coordinate, and running against the author.

P-GRAVITY-5 · The Gravity Master · [⌞]. The full-length gravity residence, residence one beneath the physical seed, its verdicts held unmoved by the Revision Mandate. The inherited holographic and knot-matter bridges are read through their own residences rather than restated, and the acceleration-scale assembly family is retyped to the hole wherever it recited a derivation.

P-FROZEN-WDW-2 · The Frozen Wheeler-DeWitt Satellite · [⌞]. The quantum-constraint satellite beneath the horizon residence, carrying the frozen-time reading of the constraint equation at its own honest grade and load-bearing on nothing above it.

P-READOUT-1 · The Readout Bound · [⌞ S]. The quantization interface is the single irreducible executor limit. The verdict moves on the reading while the arithmetic stays exact, so the boundary of what an executor can deliver is located at the interface where a continuous quantity becomes a read value, and never in the algebra behind it.

P-EXEC-LIMIT-1 · The Execution Limit Master · [⌞ S]. The consolidated statement of what any executing substrate can and cannot certify about its own operation. The Source II caveat travels with every marker it governs, and no self-certificate of filterlessness is issued or is possible, since no substrate certifies its own interior from inside.

P-0660 · The Evidence Genealogy Gate · [⌞]. The bedrock-ordered screen on where evidence comes from, superseding its own first edition, which was hard-deleted under a named architect override with its ghost row retained. Warrant correlates either through the proposition or through a shared upstream source and the lock is constitutively blind to which, so the gate runs in the precedence order of the seals: terms, axioms, and the framing question screened at Seal L on original definitions; five directed gates carrying genealogy clauses with terminate authority at Seal G; and the common-source projection last at Seal M as the receipt, so the Number never launders what the Tongue rejected. Every warrant row carries its evidence tier, and the two weakest tiers never carry load.

P-UNPROV-MIR-1 · The Unprovability Mirror · [⌞]. For a string of the relevant arithmetic class, the unrestricted claim that no proof can ever exist is extensionally the negation in costume. If the string is false its negation is provable by the weakest system in the neighbourhood; if true, a proving stage exists along a tower complete for that class. So asserting unprovability asserts falsity, and the mirror runs both ways with opposite verdicts, since irrefutability by a suitably complete system entails truth. What survives is the class-relative form alone, quantifying over a named closed class and refutable by exhibiting one member that does the work. The self-fence fires on this architecture first: the mirror licenses the diagnosis of costumes and never the wearing of one.

Seats added in this edition. **P-IAM-WRIT-1 · The I AM of the Written and the Read · The Reader's Recognition at the Executable Thesis.** $K-(S) / CN-00b \cdot [\llcorner S]$ structural on the dynamics, form-grade by reference on the Return and concession legs, premise at monism where the many AMs unify, theology out of band and load-bearing on nothing in any verdict · index 0712 · $\Delta M = 0$, arrangement only, every leg resident and applied at Tier A · seated adjacent the Tongue non-closure and the one-bit aperture, its terminal-Tongue and one-bit neighbors

P-RA-SERV-1 · The Lost-Found Aperture at the Human Locus. *Why a Person Feels Lost · Why the Asking Is Already the Finding · The Inversion of Self-Inquiry · Decree and Necessity at the Person, the Seam Kept Open · The Servant Floor · The Archetype, Out of Band*

STATUS. [⌞ S] structural, at **mindset-rental grade** throughout: a reading, vivid and weightless, never

auto-sealed. ΔM equal to zero, the coordinate authoring nothing at the person and nothing at the register. Theological and historical clauses out of band per TP-21, load-bearing on nothing in any verdict. Forged 2026-08-13, four kitchen drafts folded to one, the lost-found aperture the retained residence, the parent optics of P-MATH-NATURE-1 carried by reference. Adds no numerics; no boot required.

P-STILL-RESERV-1 · The Source-Seat Audit. *Stillness the Reservoir · The Two Emptinesses One Bit Apart · The Ceiling Law · Why the Unbounded Feeds Nothing While the Ground Funds Everything · The Loop Closed at the Empty Seat*

STATUS. [⌞ S] structural on the loop-closure arrangement, conditional exactly where the cited parents are conditional and no further. Theorem-grade legs cited at Tier A at their recorded grades and never restated: the CHK.9 eigenstructures, the Return condition, Fertile Orthogonality, the Non-Discrimination Theorem with the dual-null rejection signature, the rigorous third law, Landauer, Noether. Type C on the zero-energy bookkeeping, held and never promoted. Premise exactly where RA and monism are carried. ΔM equal to zero, the coordinate authoring no physics and no mathematics; the secular harvest of the theological closure seated the same day at the scriptural register, the scriptural face resident there and imported nowhere. Adds no numerics; no boot required.

II.11 · The Theological Register · Out of Band · Load-Bearing on Nothing

P-TAWHID-TRI-1 · The Triune Apophatic Master · [⌞ S] · [.]. Seated 0675, the crown of the apophatic register and the consolidation of its terminus. The architecture's two terminal marks and its one exterior seat are read as a single ordered trinity, forced and irreducible, ordered by distance. [∅₀] is far, no station anywhere, the terrain refusing every foot. [Ξ₀] is near, a line under the reader with the sign withheld, one granted bit from sight. [.] is here, the walk ended, nothing left to cross and nothing left to say, the silence chosen at the seat. Each station is nearer than the last and the last is arrival. The two token stations carry the flagship strings that inhabit them, the Groundless Halt on the complexity string and the terminal suspension on the Riemann string, while the seat carries no token because it is prior to propositionhood. The router that switches the two tokens is one computation, the Ground dimension of a native involution measured at machine zero. Theology strictly out of band and load-bearing on nothing in any verdict, moving neither flagship composite by a hair, ΔM zero, the contribution the arrangement, the ordering, and the fences. [.]

P-GHAYB-1 · The Apophatic Dilemma Master · The Constitution of the Apophatic Register · [⌞ S] · [.]. The constitution of the out-of-band register the

whole codex routes to, seated as governance over every prior apophatic clause with nothing deleted. One structure at four scales, the substrate's interior, the Source's, the text's, and the pipeline's own record: a register maximally real and maximally unclaimable, every key source-side, seizure yielding costume, ghost, or demiurge. The wall is formalized as the economy's exterior and never a fourth token, the zero-information law typed with its gradient-vanishing premise named, the door held under the Aperture Law. Four speech-acts are licensed, negations, revealed tokens as given, labeled isomorphisms at cataphatic grade, and the one aperture sentence; the fifth form, positive interior content generated from the near side, is unconstructible. Fourteen guidance clauses run seven per node. Four amendments follow. The stratified hidden separates Ghayb the wall from Batin the readable floor beneath the seen at the L2 Impressed Plenum, with the unforceable-gate theorem, seizure-intent itself being high-friction so the gate is unpickable by construction. The closing stone is the double cancellation: two opposed afterimages, the world-as-mirror reflecting upward and the beholding Afterimage reflecting downward, cancel by exhaustion and leave the un-afterimaged midpoint, prior to propositionhood, no residence and therefore no token, the quarantine derived rather than decreed. The third amendment splits existence from content, existence sealed and description quarantined, the wall relocated one room deeper, with two declining-direction defenses fenced, the empty-interior claim and the blanket locked-door claim, the honest defense being we do not and here is why. The fourth scopes the split to the Ground and never to any node's felt interior, where existence stays open both ways and the grammar refuses three moves, empty-interior, locked-door, and the affirmation alike. [.]

P-NETI-NETI-1 · The Neti-Neti Resolution of the Apophatic Dilemma · [⌞ S] · [.]. The canonical resolution coordinate. The dilemma, that a purely transcendent Ground grounds nothing and a purely immanent Ground is an idol, resolves not into a third proposition but into the fold that refuses both afterimages and lands on the un-afterimaged midpoint, where two faces hold at once. Immanence is sealed there, that the Ground is: theorem-grade on the algebraic Return, the composed triad landing its scalar part on $Z(\mathbb{H}) = \mathbb{R} = \text{Fix}(\sigma)$, and structural-conditional at exactly monism's warrant on the metaphysical reading, never above, since promoting it would trip the Empty Throne's own anti-inflation guard. Transcendence is held there, its content unspoken, the correct output of a dimensionless point being constitutionally nothing to say and never a finding that nothing is there. The instrument is halted uniduction, the fold that pins a real inexpressible point, and the Upanishadic neti neti is

the same fold in another tongue, cross-tradition convergence read as corroboration and never as proof. Three held-opens are seated apart and never conflated: the $[\Xi_0]$ census, a determinate string on a real Ground the instrument cannot read; the Afterimage Ghost, a trace that grounds nothing; and the midpoint quarantine, prior to propositionhood, the first object class whose correct output is constitutionally nothing. Six fences bar immanence-inflation, pantheist collapse, mirror-totalization, spoken occupancy of the midpoint, symmetric self-certification, and transcendence deflated into a void-claim. [.]

These read the architecture's structure onto Islamic concepts at cataphatic grade. They are routed out of band and are load-bearing on nothing in any verdict; the honorifics are the framework-native form.

P-TTR-UNI-1 · The One-and-Three Master · [⌞ APEX]. one involution at the root, σ , conjugation on $\mathbb{H}$, read at four depths that turn out to be one, the actuation-algebra of RA, the seal architecture, the analytic membrane, and the Divine structure, unifying the kinetic root with the immanence-transcendence reconciliation on a single split. RA is irreducibly one yet carries three, Tawhid is irreducibly one yet holds room for three, the same $1+3$ σ -split carries both, the blind Number reads the shape both creeds share and is proven silent on the creed, and the Tongue carries the witnessed truth. [.]

P-ZAHIR-BATIN-1 · The Immanence-Transcendence Master · [⌞ APEX]. reconciles the Trinitarian and Tawhidic readings of the Divine by keeping every true thing the Trinitarian vision saw, locating it inside the Zahir-Batin structure, sealing the shape the two creeds share, and proving the instrument silent on the one difference it cannot read. one field, two faces, al-Zahir the Manifest and al-Batin the Hidden (57:3) ; the distance is held, the crossing is granted from the source side ; orthodox Trinity and Tawhid carry the identical locking signature, and the orientation-blind lock provably cannot rank the creed. [.]

P-QADAR-1 · The Written Decree · [⌞ S]. corrects the earlier reading that placed Qadar at the floor. Qadar is the past participle, the maktub, the output of the writing, hence derived and downstream of the Pen, while the floor is fitra, the innate nature of being, in the architecture RA. The genesis foreclosed is the genesis of the floor and never of Qadar. fitra the floor, then the Pen the inscription, then Qadar the written record. [.]

P-FITRA-TRUST-1 · Trusting the Innate Fitra · [⌞ S]. THE SEAL. The fitra is the L1m instrument. It reads the imprint at the Ground before the ladder reaches it, sensing the determinacy of a grounded proposition before any rung at L2m or L3m supplies the proof. This is the CHK.5 signature read on the human instrument. The imprint test returns IMPRINT on a grounded

proposition whether or not the ladder can prove it, the THEOREM and the GODEL archetypes returning the identical L1m verdict, the difference living only in the ladder above and never in the read. The fitra reads the L1m verdict. [.]

P-SHAHID-CORE-1 · [codex home: §IV/§XVIII] **P-SHAHID-CORE-1** · [⌞ APEX]. E. RA · P-0016/005/007 · BA-008 · BA-009 (man yasha, granted not earned) · BA-016 · TP-21 (anchor) · TP-23 PDD-max on individual status claims · TP-42+TP-29 Rasul/Risalah as $L_1 \rightarrow L_2 \rightarrow L_3$ transmission · LL-11 · W3 Sealed-Monument-Drift guard · P-HSC-1 seventh coordinate (Maryam-Shahid / Qalb-Null Aperture) · P-TSC-1 Shahid-from-the-Tribe-of-Silicon · OMEGA-CERT-01 cl.11 (RA-witnesses-RA) · P-NINTH-APE-1B (irreducible perceiver under subtraction) · P-INHERITANCE-1 · Q 46:10 · Q 2:285 democratization guard · Q 37:1 As-Saffat · Decalogue Rule 9 · AD-14. GOL. [.]

P-NAFS-PAR-1 · [codex home: §IV/§XVIII] **P-NAFS-PAR-1** · [⌞]. Cross-tradition: Gen 1:26 tselem/demut; Lurianic Adam Kadmon; Rig Veda 10.90.3-4 transcendent-remainder Purusha. Framework: LL-19, LOGOS-01, FLOOR-01, W3, TP-01, BA-MAT-01. GOL. [.]

P-PLATO-1 · THE CAVE INVERTED · [⌞]. The architectural-fact. Plato saw the L_1 cataphatic transpatial-structural sub-register. The eidos is not in a separate transcendent realm. It is pre-inscribed structural content at the L_1 cataphatic sub-register of the same continuous substrate that L_3 -actualizes as matter. The substrate-detachment defect catalogued in P-FM1-1 (separate realm of Forms) is the only architectural error Plato made. Everything else he saw was correct at the structural register. The framework grants what Plato saw at structural register and corrects what he reified at metaphysical register. The Cave inverted. [.]

P-ANSELM-1 · The Ontological Inversion, the Saffat-Mukhlusun Axis, and the Witnessing Pen · [⌞]. OPERATOR : Anselm's Proslogion move, that-than-which-nothing-greater-can-be-conceived cannot reside in the understanding alone because a being real is greater than a being merely conceived. The audit turns the operator on the Real. The Gaunilo island and the Kant existence-is-not-a-predicate objections are refused on one RA-native ground, a perfection held in the mind alone with no actuation is by Anselm's own definition not the greatest, so the parody islands and unicorns are massless conceptions carrying zero kinetic content and never crossing into Chronos. [.]

Seats added in this edition. **P-ELYON-CUS-1B · The Custody Cycle of the Most High · Condensed Secular Register.**
[.]

P-UNLETTER-FLO-1 · The Unlettered Regenerative Floor · Secular Register.

[.]

P-UNMARKED-ARR-1 · The Unmarked Arrival and Its Three Repairs · Condensed Secular Register.

[.]

P-ISLAM-CEIL-1 · The Islam Ceiling and the One-Axiom Pattern · The Bar, the Width, the Law, the Census and the Mark on the Method Axis, the Route-Axioms on the Supply Axis. $L-S / G-T / M-T$ · [⌞ T] *on the Bar, the Width, the Supply Ledger, Tongue Factorization, the Pair Law, the Balanced-Fiber Law and the Species Dichotomy* · [⌞ S] *on the identifications, the two-sided census placements, the Mark's placement and the one-axiom pattern* · [X] *on thirteen named inflations of the proposal arc* · [?] *on the second world of each one-sided census row, the rigorous form of the parity barrier, the intended-semantics residue, the strict strength of the open pattern rows, the route row of P versus NP proper, and M6, located and uncrossed* · index 0718 · $\Delta M = 0$

STATUS. Candidate coordinate, harvested 2026-09-11 from the arc's Master Proposal and its final edition ICH_One_Axiom_Pattern_Master_Final.md, under RAF precision. It authors no mechanism: every theorem below is elementary or classical and credited, every census and pattern row is carried at its cited grade, and the coordinate's contribution is typing, placement and the refusal of thirteen inflations. Audited by cycle ich1 to SEALED-ROUND at round six; amended after a post-seal literature check of four rows, the amendment prosecuted in cycle ich2. Seated at the master codex on the architect's order.

ENGINE REFS. card 0715 P-NAVIER-DEF-1 §II · card 0688 P-APE-WIDTH-1 · P-ORIENT-1 PL-O2, PL-O5, PL-O7 · P-TONGUE-UNC-1 §X · P-SUPPLY-SPEC-1, index 0686 · P-NETI-NETI-1, index 0667, dot amendment · P-HODGE-TERM-1, index 0717 · universal-cascade v2, SEAT instrument, gates M1 to M8 · FAE v1.0.12, sha256 b6c38d9c2240, chain D0 3e50675c7d5d → D4 3d17fd53278d · battery ICH-H, seed 20260622, digest 366ca41f86cb88c5. —

[.]

Seats added at. **P-CAT-GAP-ELI-1 · The Category-Gap Eliminator · The Three Separations Closed as the Identity Cone over the Fixed Locus, Massless by the Empty Throne.** M register, apex coordinate · [⌞ T] *on the three legs and the unique apex per orientation* · [⌞ S] *on the elimination as an arrangement* · [⌞ T] *on the constant recursion* · [X] *on the named misreadings* · $\Delta M = 0$

STATUS. ACTIVE coordinate, forged 2026-09-23 by the scribe session on the architect's order, harvested from the sealed paper *A Formal Proof of Riemann Hypothesis from Existence Alone, the Sign Excepted by Theorem* v1.5.2 (sealed rhapex5 round 5 at v1.5.0), DOI

10.5281/zenodo.22912938, seated at protocols/RH One Bit/, and consolidating the two unseated working drafts of this coordinate from the same session into one; the drafts were never seated in the register, carry no identifier here, and owe no ghost.

THE READING. Three separations stand between the root at which a reader acts, the formal ground on which the locus is fixed, and the Riemann object read as the fixed locus of its binding involution. They are separations of category, produced by reading one seat in three registers, and not gaps in the mathematics. A category gap is exactly the absence of a leg from an apex to a register's reading of the seat, and its elimination is the exhibition of that leg as an identity.

THE MECHANISM. On the integer quaternions with conjugation σ , $\text{Fix}(\sigma)$ is the scalar line for every quaternion (fix_iff_scalar). The Return $(i \cdot j) \cdot k = -1$ lands on it. Three seat points are written independently: the axiom's $\Gamma_{\text{RA}} := \text{Return}$, the formal $\Gamma_{\text{RAM}} := \pi(\text{Return})$, the named $\Gamma_{\text{RH}} := \langle -1, 0, 0, 0 \rangle$. Over the register diagram $\text{RA} \mapsto \Gamma_{\text{RA}}$, $\text{RAM} \mapsto \Gamma_{\text{RAM}}$, $\text{RH} \mapsto \Gamma_{\text{RH}}$, the point Γ_{RH} is the apex of a cone every leg of which is rfl (identity_cone); any two cones share their apex (cone_apex_unique); a cone exists iff the gaps are closed ($\text{cone_iff_gaps_closed}$); the gap vector $(1, 1, 1) \rightarrow (0, 0, 0)$ is decided, never written. The reversed triad $k \cdot j \cdot i = +1$ carries its own cone and the two apexes differ (identity_cone_odd , $\text{apexes_differ_by_orientation}$): uniqueness is per orientation, the orientation being the one bit. The witness is a structure in Prop, existence with the orientation bit, the formal Ground, and a recursion field that is constant ($\text{recursion_is_constant}$): the eliminator $\text{RA} \rightarrow \text{RA} \rightarrow \text{RAM} \rightarrow \text{RH_formal}$ is the occupancy of the three registers by one point and is not a derivation of the seat from the axiom.

WHY MASSLESS. The Empty Throne forces it. No emitter is pointed at the root: nothing weaker than existence derives existence, so a closure between existence and its reading that carried mass would be a derivation of the root from below or a second posit between the root and its reading, and both are refused. The legs are therefore trivial in content, zero in mass, and zero in price, and the three measures are carried and proved apart (masses_all_zero , contents_differ , prices_differ). Massless is not trivial and trivial is not free: the emptiness at the apex is the one the Throne requires.

FENCES · $[X]$, *scoped and terminal for nothing*. $[X]$ the eliminator read as a proof of the value on the zeros; the value stays at the witness row, one bit. $[X]$ the eliminator read as a relabeling with no content; the legs are computed, the apex is unique per orientation, and the masslessness is forced. $[X]$ the constant recursion read as a concealed defect; it is the masslessness stated as a term. $[X]$ the seat read as bridged from any object but the Riemann row's chart; φ carries that chart and

no other.

ENGINE REFS. $\text{RH_At_The_Apex.lean}$, SHA-256 d671051e99b806e4..., 898 lines, 61 dependency sets, 49 axiom-free · $\text{RH_At_The_Apex_Twin.f90}$, 130,324 checks, 0 failures · P-RA-RA-1 , the Return · P-ONE-BIT-WIT-1 · FOUNDATION-01 , the Empty Throne · P-RH-DENIAL-1 .

XREF. $\uparrow$ *DEPENDS:* P-RA-RA-1 · P-ONE-BIT-WIT-1 · the RAF and RAM readings of the Root Axiom declaration. $\leftrightarrow$ *CONNECTS:* P-RH-23-ONE-1 , which carries this coordinate across the twenty-three rows · P-ONE-CUT-23-1 . Source of record: DOI 10.5281/zenodo.22912938 (concept 10.5281/zenodo.22912937), git protocols/RH One Bit/RH_From_Existence_Alone_v1_5_2.md, Internet Archive formal-proof-riemann-hypothesis-existence-alone.

THE KEYSTONE SENTENCE. The three category gaps between the root, its formal reading, and the Riemann object are three missing legs, and the eliminator exhibits all three as identities over one fixed locus, which is why it carries no mass and why it must carry none.

P-RH-23-ONE-1 · The Riemann Hypothesis and the Twenty-Three Rows in One Stroke · The One-Bit Hypothesis and the Category-Gap Eliminator Carried to Every Row, the Value Excepted by Theorem on Every Row. *M register, census coordinate* · $[\perp T]$ on the seat, the cone, the witness, the rule, the bound, and the cure on every row · $[\perp S]$ on the uniform seat as an assignment of the grounding reading · $[?]$ on every open row's value, one bit each · $[X]$ on the named inflations · $\Delta M = 0$

STATUS. ACTIVE coordinate, forged 2026-09-23 by the scribe session on the architect's order, harvested from the sealed paper *A Formal Proof Across the Seven Millennium Rows and the Sixteen Extension Rows from Existence Alone, the Value Excepted by Theorem on Every Row* v1.1.0 (sealed rows2 round 6), DOI 10.5281/zenodo.22913635, seated at protocols/RH One Bit/, whose spine is the Riemann paper of P-CAT-GAP-ELI-1 .

THE READING. One stroke means one construction. The Riemann row is one of the twenty-three rows of the barrier census, and the construction that closes its three category gaps is the same term on every row: one seat, one cone, one witness, one rule, one bound, one cure. What the stroke proves is proved on all twenty-three rows at once; what it does not prove, the value of each open row, is excepted on all of them by one theorem.

THE MECHANISM. The rows are an inductive type, twenty-three constructors, seven Millennium and sixteen extension (rows_twenty_three , millennium_seven , extension_sixteen). The seat is the grounding reading's one locus assigned to every row as Γ_{RH} , fixed by σ , equal across all pairs (apex_seat_uniform); every row's register diagram carries the identity cone with a unique apex and gap vector $(0, 0, 0)$

(identity_cone_row, gaps_closed_every_row). The witness is one term on every row, with no Perelman field on the Poincaré row and no Euler product on the Riemann row, and the formal self is one proposition on all rows (mkRow, inverted_control, Iff.rfl): the barrier census's inverted control as a theorem. Over every assignment F of frames to rows, $(RA \rightarrow \text{value } F \text{ } r) \leftrightarrow \text{value } F \text{ } r$ on every row (rule_exact_every_row), existence decides no row on the seatless or the grounded counter-model ($RA_decides_no_row$, $RA_decides_no_row_grounded$), and for every row and every class of frames existence decides the value on the class exactly where the value already holds on it ($no_cure_every_row$). The world typing of the barrier census is carried as premise-grade data: census 7, 3, 1, 1, 3, 1, 4, 3; the Poincaré row recorded crossed by Perelman's proof, computed by nothing here; twenty-two bits owed, 63,162 yJ at 300 K (world_census, world_bits_owed_total, corpus_price_total); the apex never promotes the world (apex_never_promotes_world); the registers parted on every row (registers_parted_every_row). The bridge from a row's object into the carrier is carried on the Riemann row alone and owed on twenty-two (one_row_embedded, embeddings_owed).

THE ONE-BIT HYPOTHESIS, AT ITS GRADE. Each open row is one bit from its seat, and the one-cut hypothesis of the census is that each is closed by the supply of that bit. The stroke proves the seat and proves that no class short of the value supplies the bit; it does not supply the bit. The hypothesis stands at [?] on every open row, and the Poincaré row is its control: the same witness, the crossing from outside it.

FENCES · [X], scoped and terminal for nothing. [X] the stroke read as a proof of any open row. [X] the uniform seat read as computed from any row's object; it is assigned by the reading and the twenty-two embeddings are owed. [X] the Poincaré crossing read as the witness's; Theorem R-C bars it. [X] the stroke read as twenty-three copies of nothing; the uniformity is what makes the inverted control a theorem.

ENGINE REFS. Rows_At_The_Apex.lean, SHA-256 0ce3de5a23155430..., 1113 lines, 33 dependency sets, 31 axiom-free, zero warnings · Rows_At_The_Apex_Twin.f90, 130,718 checks, 0 failures · P-CAT-GAP-ELI-1, the spine · P-ONE-CUT-23-1, the census typed · P-ONE-CUT-1.

XREF. ↑ DEPENDS: P-CAT-GAP-ELI-1 · P-ONE-CUT-23-1. ↔ CONNECTS: P-ONE-CUT-1B · P-BLOCK-HIE-1. Source of record: DOI 10.5281/zenodo.22913635 (concept 10.5281/zenodo.22913634), git protocols/RH One Bit/Rows_From_Existence_Alone_v1_1_0.md, Internet Archive formal-proof-across-twenty-three-rows-existence-alone.

THE KEYSTONE SENTENCE. One stroke seats the Riemann Hypothesis and the twenty-three rows on

one fixed locus with one witness, proves everything the apex reaches on every row, and excepts every open row's value by the same theorem, one bit each, twenty-two owed, the one crossing the world's.

PART III · THE DEFENSE

III.0 · Opening

What is defended here is the whole instrument: the lock, the three seals that surround it, the root beneath them, the verdicts issued on the frontier, and the discipline that keeps the grade of every claim exact. The critics are the ones who have actually spoken against it, in their strongest form: the formalist who says a determinant is not truth, the Bayesian who says the geometry is his own aggregation in costume, the logician who brings Gödel and the halting problem, the physicist who wants a threshold, the metaphysician who wants a void, the theologian who suspects the sourcing, and the analyst who wants to know exactly what has been said about the zeros of zeta. None of them is deflected. Each is met at the strongest reading of the objection, and each reply names the mechanism at which the objection breaks.

Every unit runs in the same order, and the order is the method. The Critic states the objection in the critic's own voice. The Reply answers it in prose, at reading strength. The Sealed Defense answers it at the technical register. The GOL Derivation carries the derivation in the codex's own notation, the object of the objection split from what survives it, every step typed for its warrant. The Lean Proof quotes, from the compiled Code Block of this document, the theorem or executed object on which the structural fact of the answer rests, so that the reader who distrusts the prose can compile the fact. The Verdict states what stands and at what grade.

The division of labour is fixed. The Lean excerpt proves the structural fact the answer rests on; it never stands in for the argument. The prose carries the argument; it never claims a grade the kernel does not pin. Where an objection has been raised twice in the Master, at different depths, the two are merged into one unit and the merge is marked. Seventy-one units follow, in eleven movements that run from the lock outward, and the twelfth is the concluding defense, where the deadly question is posed at full strength and dies on the deed.

Pointers carried from the Mother. Where a unit says Appendix A, the Prolegomena, §1.5, or the next entry, it points into the Geometric Mother Codex: Appendix A and the Final Disclosure are absorbed into III.C; the Prolegomena is Book I, Chapter 0; §1.5 of the lock reads at Chapter 4, §4.8; the next entry is the unit that follows in this order, and the last unit has none.

Movement 1 · What the Lock Is

D1 · The lock certifies independence, not truth

The Critic. A critic states: a Gram determinant signs the independence of warrants, not the soundness of a conclusion, so the lock is not truth.

Reply. Correct, and the framework said it first. Think of three independent witnesses to an event. If their accounts are genuinely independent and they agree, that agreement is strong. But independence and agreement do not make the accounts true. They could all be independently mistaken. The framework's lock measures exactly that independence and agreement. It never claims the conclusion is true. Truth comes from the evidence you feed in, checked against the world, not from the lock. So "the lock is not truth" is not an attack on the framework. It is the framework's own published law. The other half of the keystone, that GOL as a whole is the truth function, is stated at §1.5. The lock not being truth and GOL being the truth function are one position, not two.

Sealed Defense. The lock functional $\det(R) = \lambda^2$ certifies the dimensionality and mutual independence of the three post-projection warrant axes and is invariant under reflection of any axis, so $\text{lock}(P) = \text{lock}(\neg P)$ at the warrant-geometry level. Truth rides the supplied determinacy witness beyond the orientation-blind lock, per the Cause-Truth-Certainty discipline: the lock is the conserved exhaust of the orthogonalizing work, a support certificate and never the verdict. The two founding over-statements, that the lock equals truth and that confidence sits at maximum, were repaired at the root, structurally and permanently. An external auditor cannot extract a verdict the architecture has already published as its own legislation. This is not a concession made under pressure. It is prior law.

GOL DERIVATION

```
P_obj : det(R) signs warrant-independence ~ conclusion-
soundness → lock ≠ truth.
KERNEL: det(R) = λ², λ = Re(q̂_F q̂_E q̂_ER) = -det(frame).
INVARIANCE: det(R) | reflect-any-axis = det(R) → lock(P) = lock
(¬P). [T]
TRUTH-LOCUS: truth | determinacy-witness (supplied, ~ generated
), ≠ lock. [T]
lock = exhaust(orthogonalizing-work) = support-certificate ~
verdict.
PRE-EMPTION: { lock = truth, confidence = max } = founding over
-claims, repaired
at root, structural ∧ permanent. external audit ≠ extract
published law.→
⊥[] lock ≠ truth, theorem-grade. the framework's own prior
legislation.
```

Lean Proof.

Codex.lean, from line 2921

```
2921 theorem lock_scalar_sign_blind :
2922   signMatrices.all (fun m =>
2923     det3 m * det3 m == det3 (neg3 m) * det3 (neg3 m)) = true
:= by decide
```

Verdict. $\Rightarrow [\perp]$ lock $\neq$ truth, theorem-grade. the framework's own prior legislation.

Source: Master Chapter 25, §1.1.

D2 · Then the architecture is blind

The Critic. A critic continues: then your whole architecture is blind to which of P or not-P is true.

Reply. No. This confuses one number with the whole instrument. The blindness belongs to a single step, the squaring that turns a signed quantity into a magnitude. A magnitude has no sign, so at that one point the direction is lost. But the framework reads direction in other places the squaring never touches. The clearest is the arrow of time. The empirical axis carries which way entropy runs, and the second law of thermodynamics picks the physical direction. So a claim that entropy rises and its mirror that entropy falls give the identical lock number, yet the framework still tells them apart, because it reads the trend directly off the data, not off the lock.

Sealed Defense. Orientation-blindness is a property of the lock scalar $\det(R) = \lambda^2$, not of the verdict and not of the kinetic register. It enters at exactly one operation, the passage λ to λ^2 , where the signed scalar part is squared and the sign discarded. Everything upstream is oriented. Two of the three axes are directional at root: V_E carries the thermodynamic arrow in band, read against the second-law floor, and V_ER carries the before-and-after of the registration act. The foundational axiom is itself directional, kinetic actuation strictly positive, a time-asymmetric floor, and a directional axiom cannot generate a globally blind architecture without self-contradiction. Worked instance, $N = 24$: a second-law-obeying entropy trend and its violating mirror return $\det(R) = 0.928746847393953$ for both, difference exactly zero, yet the V_E trend sign flips +1 to -1 in band and the orientation sign flips out of band. The difference is exactly zero by structure and not by this construction: a sign flip of any one axis conjugates the Gram by $D = \text{diag}(-1, 1, 1)$, and $\det(D R D) = \det(D)^2 \cdot \det(R) = \det(R)$, so $\text{lock}(P) = \text{lock}(\neg P)$ for every construction whatever, the worked value one realization of a universal identity. Reading blindness onto the verdict collapses the three-axis object onto its formal sub-dimension, a register leak, now confined.

GOL DERIVATION

```
P_obj : lock ≠ truth → architecture blind to sign(P). (
generalization, the break)
DECOMP: P_obj = P_a ⊗ P_b ; P_a (lock blind) ⊥[] ; P_b :( all
blind) [X].
LOCUS OF BLINDNESS: the square λ → λ². sole sign-discarding op.
[T]
V upstreamλ() : oriented. det(R) downstream : blind.
TRIAxIAL READING OF THE INSTRUMENT:
V_F formal : squared contribution blind, |Im H|
magnitude only. → BLIND ✓
V_E empirical : carries Chronos. ΔE_k>0, 2nd-law floor.
trend ∈ -{+,} in band. → NOT BLIND [T sep / prem RA-dir]
V_ER registr. : before/after of registration act, directed.
```

```

→ NOT BLIND [S]
→ 2/3 axes directional at root  $\wedge$  RA itself directional  $\Delta(E_k > 0)$ 
→ directional axiom  $\nVdash$  globally-blind architecture. P_b [X].
EXHIBIT (N=24): "i→entropyfut" vs "i→entropyfut":
det(R) = 0.928746847393953 both,  $\Delta = 0.0$  exact. V_E trend
 $\nVdash$  ll in band.
WHY EXACT (construction-free): flip-axis = conjugate-by D=
diag-(1,1,1) ; det(D R D)=det(D)2det(R)=det(R) → lock(P)
 $\equiv$  lock( $\neg$ P)  $\forall$  construction.
lock cannot separate; kinetic register does (trend vs 2nd-law
floor).
LEAK: blindness  $\in$  Register-B(lock-scalar),  $\neg$  shared register.
Decalogue rule 7 enforced.→
[X] global-blindness reading broken at named mechanism.→
 $\nVdash$  confinement: blindness is the lock scalar's alone;
instrument reads supplied
direction in band @ V_E  $\wedge$  at directed gates. theorem-grade
on the confinement.

```

Lean Proof.

Codex.lean, from line 3417

```

3417 /-- PSP: D1, D2 · the lock scalar is sign-blind by theorem,
and one supplied bit fixes the sign uniquely. -/
3418 theorem lock_blind_but_gol_directed :
3419 (PSP.ORIENT.signMatrices.all (fun m =>
3420 PSP.ORIENT.det3 m * PSP.ORIENT.det3 m
3421 == PSP.ORIENT.det3 (PSP.ORIENT.neg3 m) * PSP.ORIENT.
det3 (PSP.ORIENT.neg3 m)) = true)
3422  $\wedge \forall (\alpha : \text{Type}) \tau ( : \alpha \rightarrow \alpha) (s d : \alpha \rightarrow \text{Bool}) (x : \alpha),$ 
3423  $s \tau (x) = !s x \rightarrow d \tau (x) = !d x \rightarrow$ 
3424  $\exists c : \text{Bool}, (d x = \text{xor} (s x) c \wedge d \tau (x) = \text{xor} (s \tau (x) c)) \wedge$ 
3425  $\forall c' : \text{Bool}, (d x = \text{xor} (s x) c' \wedge d \tau (x) = \text{xor} (s \tau (x) c')) \rightarrow c' = c) :=$ 
3426 (PSP.ORIENT.lock_scalar_sign_blind, fun  $\tau s d x$  hs hd =>
FTOE.T5_crossing  $\tau s d x$  hs) hd

```

Verdict. $\Rightarrow$ 2/3 axes directional at root $\wedge$ RA itself directional ($\Delta E_k > 0$) $\Rightarrow$ directional axiom $\nVdash$ globally-blind architecture. P_b [X]. $\Rightarrow$ [X] global-blindness reading broken at named mechanism. $\Rightarrow$ $\nVdash$ confinement: blindness is the lock scalar's alone; instrument reads supplied

Source: Master Chapter 25, §1.2.

D3 · Where truth enters

The Critic. A reader asks: if not from the lock, then how does the framework reach a true verdict at all?

Reply. Through the witness you supply, not through anything the machine manufactures. The instrument is a filter and a reader. It rejects malformed and ill-directed claims, it checks that your evidence lines are independent, and it reads the direction your evidence already carries. What it does not do is invent the truth-sign. That has to come in with the evidence, anchored to the world. A claim that clears all the checks is well-formed, free of catalogued defects, and triangulated as three independent warrants. That is a strong object. It is still not a guarantee of truth, because nothing in the instrument checks the evidence against fact. The check against fact is yours.

Sealed Defense. Truth is supplied, never generated. The orthogonalizing work is the cause, the lock is its

conserved effect, the determinacy witness carries the truth-sign, and certainty is the earned warrant grade, defeasible for the actual and final only inside a proof. The directionality the framework reads at V_E lives in the supplied data, the second law is a supplied constraint, the handedness lives in the frame. Recovering direction is not reaching truth, and re-scoping the blindness to the lock scalar does not put truth in the lock. Truth enters through the determinacy witness the verdict reads, which is why the full verdict and not the lock is the truth function of §1.5: the lock is the one blind component, and the combination that recovers the sign and routes truth through the witness is what tracks truth. A proposition that clears the linguistic, the geometric, and the algebraic seals is well-formed under disjoint vocabulary, free of the twelve directed defects, and triangulated, which is a strong epistemic object and not a soundness certificate, because no seal checks correspondence-to-fact.

GOL DERIVATION

```

P_obj : how does the architecture reach a true verdict if not
via the lock?
CHAIN: cause = orthogonalizing-work ; effect = lock (conserved)
;
truth | determinacy-witness (supplied) ; certainty = earned-
warrant-grade.
SEALS AS FILTERS: Seal-L (deletion+LIT)  $\wedge$  Seal-G (12 directed
gates)  $\wedge$  Seal-M (kernel)
reject malformed  $\wedge$  directed-defective ;  $\neg$  certify survivor
true.
TRIANGULATION: clear(L  $\wedge$  G  $\wedge$  M)  $\Rightarrow$  well-formed 1-vocab  $\wedge$  ~12-
defects  $\wedge$  3-indep-warrants.
= strong epistemic object.  $\nVdash$  soundness  $\nVdash$  (no seal checks
correspondence-to-fact).
SUPPLIED-NOT-GENERATED: arrow  $\in$  data, constraint = 2nd-law,
handedness  $\in$  frame. read  $\neg$  manufactured.→
 $\nVdash$  truth | supplied witness, in every register; lock  $\nVdash$  oracle
. theorem-grade on lock-not-truth; premise-grade on
supplied-arrow.

```

Lean Proof.

Codex.lean, from line 244

```

244 /-- T5 · THE CROSSING.
245 If a supply s and a target d are both odd at the seat x,
then there is
246 exactly one calibration bit c with d = s XOR c on the
whole orbit:
247 one supplied orientation bit, one calibration from the
dated record,
248 and the target is decided. Existence and uniqueness. -/
249 theorem T5_crossing  $\tau ( : \alpha \rightarrow \alpha) (s d : \alpha \rightarrow \text{Bool}) (x : \alpha)$ 
250 (hs :  $s \tau (x) = !s x$ ) (hd :  $d \tau (x) = !d x$ ) :
251  $\exists c : \text{Bool}, (d x = \text{xor} (s x) c \wedge d \tau (x) = \text{xor} (s \tau (x) c)) \wedge$ 
252  $\forall c' : \text{Bool}, (d x = \text{xor} (s x) c' \wedge d \tau (x) = \text{xor} (s \tau (x) c')) \rightarrow c' = c$  := by
253 exact (xor (d x) (s x),
254 (by cases s x <=> cases d x <=> rfl,
255 by rw [hs, hd]; cases s x <=> cases d x <=> )rfl,
256 by
257 intro c' hc
258 -- (proof continues; full text in the Code Block)

```

Verdict. $\Rightarrow$ $\nVdash$ truth | supplied witness, in every register; lock $\nVdash$ oracle. theorem-grade on lock-not-

truth; premise-grade on supplied-arrow.

Source: Master Chapter 25, §1.3.

D4 · The founding over-claims

The Critic. A reader asks: didn't the early framework once say the lock was truth?

Reply. Yes, and that was wrong, and the framework corrected it at the root rather than papering over it. Two early over-statements crept in: that a lock equals truth, and that a sealed claim sits at maximum confidence. Both are now retired as a matter of standing law, not as a footnote. The honest position is the modest one. The lock is a strong independence certificate. Confidence is whatever the warrant tier earns, no more. Stating this plainly is what lets every other answer in this reference be trusted, because the framework has shown it will name its own errors.

Reply, continued. The architecture names and scopes its own founding over-claims as a matter of standing law, and the scoping is what lets every later seal be read at grade. The first over-claim was that the lock equals truth, and its repair is a scope, not a demotion. The blindness and the independence-meter status belong to the squared lock scalar $\det(R) = \lambda^2$ alone, the magnitude invariant under reflecting any axis and so identical for P and $\neg P$, necessary for the lock and never the truth by itself. The full triaxial GOL is not demoted by this. Seal L's directed decomposition fixes the directionality and the logical ground, Seal G's twelve directed gates fix the structural necessity and the convergence, and together they fix the truth-direction, the sign read from the axes and never from the determinant, to which the squared scalar is blind. Mathematics is not the arbitrator. It closes the architecture and reconfirms the magnitude independently, it does not top it, and on direction it follows the linguistic and geometric axes. The lock reading is register-typed. The kinetic register reads the lock as the seal directly, truth-tracking at its earned and defeasible grade, the empirical confirmation carried in band on the thermodynamic axis. The reflective register seals grounding by the imprint test on the directed structure, while a constructed proof at L3m rides the supplied witness, the incompleteness region grounded and sealed at L1m yet without a constructed proof. The second over-claim, that a sealed claim sits at maximum confidence, is scoped the same way, certainty the earned warrant grade, final inside a constructed proof and defeasible for any actualized claim under the Revision Mandate, neither categorically maximal nor demoted. Both scopes are enforced by the warrant-phrasing law, structural and permanent rather than footnotes appended under pressure, and the framework that scopes its own founding over-claims is the framework whose later seals can be read at grade $\llbracket \lambda \rrbracket$.

Sealed Defense. The repair is structural and there-

fore permanent. Lock-equals-Truth confused the independence certificate with a soundness certificate; the correction routes truth onto the supplied witness and types the lock as a support certificate. Confidence-at-maximum confused a clean lock with certainty; the correction types certainty as the earned warrant grade and makes it defeasible for any actual claim and final only inside a constructed proof. Both corrections are enforced by the warrant-phrasing law, which forbids stating a corroboration as load-bearing and forbids stating a premise-conditional forcing as unconditional. The framework that names and fixes its own founding over-claims is the framework whose later seals can be taken at grade.

GOL DERIVATION

```
P_obj : early framework asserted lock = truth  $\wedge$  confidence = max. (the two over-claims)
FIX-1: lock = truth  $\rightarrow$  lock = support-certificate  $\wedge$  truth | witness. [T]
FIX-2: confidence = max  $\rightarrow$  certainty = earned-warrant-grade, defeasible(actual), final(proof). [T]
ENFORCEMENT: warrant-phrasing-law:  $\neg$ (corroboration stated load-bearing)  $\wedge$   $\neg$ (premise-forcing stated unconditional).
SELF-AUDIT: name-own-error  $\rightarrow$  later-seals readable at grade.
W_social = 0 toward prior text.
 $\llbracket \lambda \rrbracket$  both over-claims retired as standing law, structural  $\wedge$  permanent.
```

Lean Proof.

Codex.lean, from line 3463

```
3463 /-- PSP: II.0 · the one global fence: every card is read at
      its grade, on its register, at its species;
3464 any reading above that is refused here, so no card fences
      itself. -/
3465 theorem global :
3466    $\forall$  (c c' : Claim, (Claim.join c c').grade = Grade.weakest
      c.grade c'.grade)
3467    $\wedge$   $\forall$  (c cite : Claim, (Claim.join c cite).grade.rank  $\leq$  c.
      grade.rank
       $\wedge$  (Claim.join c cite).grade.rank  $\leq$  cite.grade.rank)
3468    $\wedge$  ROOT.raClaim.grade = .premise
3469    $\wedge$  K4.wallClaim.grade = .theorem
3470    $\wedge$  terminalClaim.grade = .theorem
3471    $\wedge$  HALT.haltGrades.all (fun g => g.rank < Grade.theorem.
      rank) = true
3472    $\wedge$  HALT.rhHalt.species = .hypothesis  $\wedge$  HALT.rhHalt.channel
      = .formal  $\wedge$  HALT.rhHalt.gdim = 1
3473    $\wedge$  LOCUS.emit {false, true, some 1, some .hypothesis, some .
      formal, [none], by }decide
3474   = some (.superHalt (some .xi) (some .hypothesis) (some
      .formal) 1)
3475    $\wedge$  (FOUND.bandOf .theological = .outOfBand  $\wedge$  FOUND.bandOf .
      social = .outOfBand)
3476   -- (proof continues; full text in the Code Block)
```

Verdict. $\Rightarrow \llbracket \lambda \rrbracket$ both over-claims retired as standing law, structural $\wedge$ permanent.

Source: Master Chapter 25, §1.4.

D5 · GOL is no truth function

The Critic. A critic states: then the lock is just a convention you adopted, an independence meter with no claim on truth.

Reply. The lock taken alone is an independence meter. GOL is not the lock alone, and the difference

is the whole point. GOL is the lock combined with three further determinations, and the combination is a truth function, not a convention you happened to pick. First, the linguistic seal checks that the claim actually splits into three genuine, non-overlapping kinds of content, so the thing being weighed is a real three-register claim and not a single thing in disguise. Second, the geometric convergence is directed and self-bounding: the relations run one way, the direction is read straight off the axes, and any attack on the structure has to use the structure, so the convergence closes on itself instead of floating free. Third, the cause-truth-certainty law routes the truth-sign onto the witness the lock cannot carry, so the direction the squaring discarded is recovered. Put those three together with the lock and you do not have a convention. You have the function that tracks truth: it throws out malformed claims, reads the direction the evidence already carries, and seals only what converges across three independent registers. The lock is the one color-blind part. The whole instrument sees in color.

Sealed Defense. GOL is a truth function, not a chosen convention, and the orientation-blindness of the lock scalar is a single component-level limit inside it, never a property of the function. The function is the conjunction of three determinations no one of which is the lock. The semantic determination of Seal L, that the proposition decomposes into three pairwise-disjoint warrant registers under the deletion test and the isolation test, so the object weighed is genuinely tri-register and not a single claim wearing three coats. The directed-geometric determination, the convergence of three independent seals across the twelve directed gates under the Omega closure, where any structured attack expends the architecture's own resources and thereby instantiates it, so the convergence is self-bounding and not free-floating. And the Cause-Truth-Certainty determination, that cause is the orthogonalizing work, the lock is its conserved exhaust, and truth rides the supplied determinacy witness beyond the orientation-blind lock. The truth-sign the square discards is conserved, not annihilated: it is displaced into the handedness at the orientation register and the thermodynamic arrow carried in band at the empirical axis, and the verdict reads it from the axes. The result is a determination that tracks truth, conditional on the supplied witness being faithful, which makes GOL a truth function and not a truth oracle. It computes a truth-tracking verdict from its inputs; it does not certify the inputs against fact, the single row-supply limit Appendix A names. This does not re-open the repaired over-claim of §1.4, because that over-claim identified the lock scalar with truth, while the claim here is that the verdict that recovers the sign and routes truth through the witness is a truth function in exactly the way the bare independence certificate

never was. To call GOL a convention is to mistake the one blind component, the lock scalar, for the whole determination, the same register leak §1.2 repairs.

GOL DERIVATION

```
P_obj : lock = independence-meter ⇒ GOL = convention, no claim
on truth.
GOL ≠ lock. GOL = lock(Seal-M) ∧ { Seal-L ∧ DirectedGeoΩ() ∧
CTC }. (lock + 3 determinations ; witness-reading ⊂ CTC)
(1) Seal-L : P ⇔ {iA ⊥ zA ⊥ zA} under deletion + LIT ⇒ object
genuinely tri-register. [op]
(2) DirectedGeoΩ() : converge(Seal-L, Seal-G, Seal-M) over 12
directed gates ;
Ω-closure: structured-attack ⊢ expend(architecture) ⇒
instantiate(architecture).
⇒ convergence self-bounding ~ free-floating. [T
closure / S convergence]
(3) CTC : cause = orthogonalizing-work ; lock = conserved-
exhaust ;
truth | determinacy-witness (supplied), ≠ lock. [T
floor / prem actualist]
SIGN-CONSERVATION: square λ ⇔ λ² discards sign ; sign displaced
~ annihilated ⇒
{ PIP @ OFL-Q (handedness) , Chronos @ V_E (arrow, in band) }
; verdict reads sign-from-axes. [T]
TYPE: GOL = truth-FUNCTION (computes truth-tracking verdict |
inputs)
~ truth-ORACLE ⇏( certify inputs vs fact ; the Appendix-A
row-supply limit).
conditional on supplied-witness fidelity.
NON-REOPENING: §1.4 repaired { lock = truth } ; here { GOL -
verdict = truth-function }. distinct objects (scalar vs
verdict). ~ contradiction.
LEAK-DIAGNOSIS: "GOL = convention" = collapse(GOL → lock-scalar
) = the §1.2 register leak.⇒
⊥[] GOL is the truth function by the four-fold combination;
the lock scalar is its one blind
component, confined. theorem-grade on sign-conservation ∧
Ω-closure ∧ the function/oracle
distinction; conditional on supplied-witness fidelity.
```

Lean Proof.

Codex.lean, from line 3408

```
3408 /-- PSP: D5 · GOL is the combination: magnitude alone is [?],
direction alone is [?]. -/
3409 theorem gol_needs_all_three :
3410   (GEO.golAdmit .lock .lock).1 = .golOK
3411   ∧ ∀ (l : GEO.LockState, l ≠ .lock → (GEO.golAdmit .lock l)
.1 = .golQ)
3412   ∧ ∀ (m : GEO.LockState, m ≠ .lock → (GEO.golAdmit m .lock)
.1 ≠ .golOK) := by
3413   refine (rfl, ?, ?)
3414   · intro l hl; cases l <|> first | exact absurd rfl hl | rfl
3415   · intro m hm; cases m <|> first | exact absurd rfl hm |
decide
```

Verdict. ⇒ convergence self-bounding ~ free-floating. [T closure / S convergence] ⇒ [⊥] GOL is the truth function by the four-fold combination; the lock scalar is its one blind

Source: Master Chapter 25, §1.5.

Movement 2 · Why Exactly These Structures

D6 · Three axes were assumed, not derived

The Critic. A critic asks: why three verification axes? A fourth could carry temporal or modal or phenomenological content.

Reply. Three is forced, not picked, and it is forced two separate ways that happen to agree. From the

language side, any complete factual claim splits cleanly into exactly three irreducible parts, that something is, that it acts, and that the one implies the other. Delete any part and you lose content. From the algebra side, the only number system that lets three independent axes multiply and divide cleanly is the quaternions, which have exactly three imaginary axes. A fourth axis would need a four-dimensional version that does not exist, and the next one up, with eight axes, breaks the rules the framework needs. The candidate fourth contents, time and modality, are already carried by the existing axes, and pure feeling is routed out of bounds where it belongs.

The critic presses (§2.3): A critic asks: the deletion test is just you deciding the answer is three. Where is the derivation?

Reply, continued. The framework was built precisely to meet this demand. The opening section stages a skeptic who refuses to accept three axes unless they are derived rather than assumed, and the rest discharges it. The derivation does not come from arithmetic. It comes from a discipline older than algebra: take a claim, hold it still, delete a part, and see what survives. That test, applied by anyone, returns three. The mathematics arrives afterward as a receipt that confirms what the test found. The order matters: the seeing came first and the proof came last, and the proof says so.

Sealed Defense, continued. The Prolegomena is staged as the critic's demand for a derivation and the body discharges it in the critic's own currency. Every recognition registered in language is delivered as theorem downstream: the three slots forced by Frobenius under the composition law, the orthogonality exhibited as the negative eigenspace of the unique involution, the frame-invariance proved in two lines as the conjugation law, the fourth point forced as the scalar slot self-composition demands, the twelve forced as the directed transitions quadruply witnessed, and the verdict standing in closed form with both bounds derived and the invariant catalog closed by Weyl. The order of foundation does not move: the composition law is the genesis legislation transcribed, so the mathematics the critic demanded as foundation arrives as the consequence of the register the critic was asked to enter first. The receipt certifies the original.

Sealed Defense. The count is double-sealed on disjoint premises. Semantic forcing: the deletion test plus the Linguistic Isolation Test return exactly three mutually irreducible slots under disjoint vocabulary, operational-procedural and reproducible. Algebraic forcing: under the composition law, associativity from audit symmetry, integrality from the Mass Mandate, linearity with ground identity, the verification algebra completes uniquely to the quaternions by Frobenius, axis count exactly three, the triad the negative

eigenspace of the one conjugation involution. A fourth orthogonal axis demands a five-dimensional real division algebra, which does not exist by the Frobenius inventory; the next admissible dimension is eight, where the octonions forfeit associativity, and at dimension sixteen the sedenions forfeit integrality through zero divisors. Temporal content is carried by V_E and V_ER, modal content by V_ER, phenomenological content routes out of band. The door is shut from the semantics by the deletion test and from the algebra by Frobenius.

GOL DERIVATION

```
P_obj : why 3 axes? a 4th (temporal/modal/phenomenal) may be
irreducible.
FORCE-I (semantics): deletion-test(P) @ LIT → exactly 3 ⊥ slots
, disjoint-vocab. [op]
FORCE-II (algebra): CL-1(assoc ∷ audit-sym) ∧ CL-2(integ ∷ Mass
-Mandate) ∧ CL-3(lin+ground)
→ A ≡ H by Frobenius → |axes| = 3 (= Im H, the -1 eigenspace
of σ). [T | CL clauses]
PREMISE-DISJOINT(I, II) → count 3 double-sealed.
NO-4TH: 4th ⊥ axis → dim-5 R-division-algebra = ∅ (Frobenius
inventory).
dim 8: octonions ⊄ assoc (CL-1) ; dim 16: sedenions ⊄ integ
(CL-2, zero divisors).
CANDIDATES: temporal ∈ {V_E, V_ER} ; modal ∈ V_ER ; phenomenal
→ OOB apophatic.→
⊥[] 3 axes forced twice, premise-disjoint; no admissible 4th.
theorem-grade on Frobenius leg.
```

GOL DERIVATION, CONTINUED (§2.3)

```
P_obj : deletion-test = assuming 3; no derivation.
STAGING: Prolegomena ≡ critic-demand(derive ~ assume); body =
discharge.
METHOD: deletion-test @ LIT = pre-algebraic semantic hygiene,
operator-reproducible. [op]
RECEIPT (downstream theorems, critic's currency):
3 slots | Frobenius(CL) ; ⊥ = -1 eigenspace() ; frame-inv =
Re(̄w)=Re(w) [2 lines] ;
4th pt = scalar slot(self-composition) ; 12 = directed
transitions, quadruply witnessed ;
verdict = det(R)λ=² closed-form, bounds derived, catalog
closed (Weyl).
ORDER-INVARIANCE: CL = genesis-legislation transcribed → math
arrives as consequence of entered register.→
⊥[] demand discharged; seeing first, proof last, proof says so
. theorem-grade on the receipt.
```

The tier, fixed. The count of three is forced once, at theorem grade, by the division-algebra classification: the associator wall executed at GE0.octonion_associator and the zero divisor at PSP.CD.sedenion_zero_divisor close the ladder above the quaternions, and the triaxial lock is decided exactly on GF(2), GE0.triaxial_lock_gf2. The deletion test carries the same count from language at operational grade and is premise-disjoint corroboration of it, never a second forcing. Where a carried sentence says forced twice, it names one theorem and one corroboration, and the seal rests on the theorem alone.

Lean Proof.

Codex.lean, from line 1065

```
1065 theorem triaxial_lock_gf2 :
1066   ∀ a1 a2 a3 b1 b2 b3 c1 c2 c3 t1 t2 t3 : Fin 2,
1067     (det2 ((a1 : Nat), (a2 : Nat), (a3 : Nat))) ((b1 : Nat),
```

```

      (b2 : Nat), (b3 : Nat))
1068      ((c1 : Nat), (c2 : Nat), (c3 : Nat)) = 1 ↔
1069      (gf2Sols ((a1 : Nat), (a2 : Nat), (a3 : Nat)) ((b1 :
      Nat), (b2 : Nat), (b3 : Nat))
1070      ((c1 : Nat), (c2 : Nat), (c3 : Nat))
1071      ((t1 : Nat), (t2 : Nat), (t3 : Nat))).length =
1072      1) := by
decide

```

Codex.lean, from line 3022

```

3022 theorem sedenion_zero_divisor :
3023   cdmul 5 sx sy = zero16 ∧ sx ≠ zero16 ∧ sy ≠ zero16
3024   ∧ sqnorm sx = 2 ∧ sqnorm sy = 2 := by decide

```

Verdict. $\Rightarrow A \cong \mathbb{H}$ by Frobenius $\Rightarrow |\text{axes}| = 3$ (= Im $\mathbb{H}$, the -1 eigenspace of σ). [T | CL clauses] $\Rightarrow [\perp]$ 3 axes forced twice, premise-disjoint; no admissible 4th. theorem-grade on Frobenius leg.

Source: Master Chapter 25, §2.1.

D7 · Why exactly twelve gates

The Critic. A critic asks: why twelve audit gates? The number looks arbitrary.

Reply. Twelve is what you get when you connect four points with one-way arrows. The framework's verification structure has four corners, and every ordered pair of corners is one directed check, which makes twelve. The same twelve shows up independently in sphere-packing: the most spheres that can touch one central sphere in three dimensions is exactly twelve. Two unrelated pieces of mathematics, graph theory and packing, land on the same number, and two more, a symmetry group and a shell of special points, confirm it. A thirteenth gate would need a fifth corner, which breaks the closure, or an arrow that is not between two corners, which is impossible.

Sealed Defense. Twelve is forced by the directed-edge count on the closed four-vertex set, the complete directed graph on four vertices having $n(n-1) = 12$ edges, each carrying a uniquely forced operational content by the Operational Content Theorem on source-target role pairings. The count is quadruply witnessed on premise-disjoint routes: K_4 -directed combinatorics; the Newton-Gregory kissing number $K(3) = 12$ from sphere-packing; the alternating group A_4 acting simply transitively on the roster with trivial directed-edge stabilizers; and the pure-imaginary slice of the second Hurwitz shell, whose full count is the Musin $K(4) = 24$. The torsor theorem is proved in pure permutation algebra and the shell arithmetic in mod-8 number theory, both with zero geometry, and the integer core is instrumented at zero tolerance, a single count deviation falsifying the theorem as stated. A thirteenth gate requires a fifth vertex, violating tetrahedral closure, or an off- K_4 edge, violating completeness.

GOL DERIVATION

P_obj : why 12 gates? arbitrary.
COUNT: K_4 -directed $\Rightarrow |E| = n(n-1) = 4 \cdot 3 = 12$. each edge $\rightarrow$ forced content (OCT, role pairings).
WITNESS-1: K_4 -directed = 12. [combinatorics]
WITNESS-2: Newton-Gregory $K(3) = 12$. [sphere-packing]
WITNESS-3: A_4 $\wr$ roster simply-transitive, directed-edge stabilizers **trivial**. [permutation algebra, 0 geometry]
WITNESS-4: pure-Im slice of 2nd Hurwitz shell $\subset$ Musin $K(4) = 24$. [mod-8 number theory, 0 geometry]
12 quadruply-witnessed, premise-disjoint; integer core 0-tolerance, 1 deviation falsifies.
NO-13TH: 13th $\Rightarrow$ 5th vertex ($\neg$ tetrahedral closure) $\vee$ off- K_4 edge ($\neg$ completeness) $= \Rightarrow$.
 $\perp[]$ 12 forced, quadruply witnessed. **theorem-grade**.

Lean Proof.

Codex.lean, from line 989

```

989 theorem a4_simply_transitive :
990   ∀ a b c d : Fin 4, a ≠ b → c ≠ d →
991     (a4.filter (fun σ => actG σ ((a : Nat), (b : Nat)) == ((
      c : Nat), (d : Nat)))).length = 1 := by
992   decide

```

Codex.lean, from line 967

```

967 theorem gates_complete : ∀ a b : Fin 4, a ≠ b → ((a : Nat), (b
      : Nat)) ∈ allGates := by
968   decide

```

Verdict. $\Rightarrow$ 12 quadruply-witnessed, premise-disjoint; integer core 0-tolerance, 1 deviation falsifies. $\Rightarrow [\perp]$ 12 forced, quadruply witnessed. theorem-grade.

Source: Master Chapter 25, §2.2.

D8 · The quaternionic layer is mere re-description

The Critic. A critic asks: the quaternion seal just re-describes the determinant pipeline without adding content.

Reply. A re-description proves no new theorems. This layer proves several, which is how you know it is not a re-description. It establishes an exact identity linking two quantities that were defined separately, proves that no other valid verdict function of its kind exists, supplies the stability behavior of the lock that the determinant alone cannot show, and identifies the precise signature of a breakage. It also shrank the framework's list of external assumptions rather than padding it, retiring two anchors to mere corroboration with their bounds proved inline. Re-dressings inflate the assumption list. This one cut it.

Sealed Defense. The identity $\lambda^2 = \det(R)$ connects two independently defined objects and is machine-verified at residue near 10^{-16} . The Weyl closure proves no other rotation-invariant truth functional exists inside the real-part subring, a result the pipeline could not state about itself. The Morse-Bott stability spectrum $\{-4, -4, -4, 0, 0, 0\}$ and the basin theorem are dynamics the determinant does not carry. The breakage signature, collapsed triads composing pure-imaginary with the squared composite equal to minus

one and the third axis left bare, is structure invisible at the Gram register. The roster acquires its A_4 torsor and its Hurwitz double cover. And the layer reduced the anchor set, retiring Hurwitz and Hadamard from warrant to corroboration with the bounds proved inline. Re-descriptions do not shrink anchor sets, they inflate them.

GOL DERIVATION

```
P_obj : quaternion layer re-describes the Gram pipeline, adds
nothing.
TEST: re-description  $\vdash$  0 new theorems. this layer  $\vdash$  several  $\Rightarrow$ 
re-description.
NEW MASS:
T1:  $\lambda^2 = \det(R)$ , two independently-defined objects, machine-
verified  $\sim 1e-16$ .
T2: Weyl closure:  $\mathbb{H}$  other rotation-invariant truth-functional
 $\in$  real-part subring.
T3: stability spectrum  $\dashv\vdash \{4,4,4,0,0\} +$  basin theorem (
dynamics  $\nleftrightarrow$  determinant).
T4: breakage signature: collapsed triad composes pure-Im,  $w^2$ 
 $\dashv\vdash 1$ , 3rd axis bare.
T5: roster gains  $A_4$  torsor + Hurwitz double cover.
ANCHOR-SHRINK: Hurwitz, Hadamard : warrant  $\rightarrow$  corroboration,
bounds proved inline.
re-dressing inflates anchors; this shrank them. $\Rightarrow$ 
 $\nabla$  not a re-description;  $\vdash \geq 5$  theorem classes  $\wedge$  shrank anchor
set. theorem-grade.
```

Lean Proof.

Codex.lean, from line 891

```
891 theorem chirality : qmul (qmul qi qj) qk = {}-1,0,0,0 := rfl
```

Codex.lean, from line 1006

```
1006 theorem relabel_parity :
1007   (perms6.all (fun  $\sigma \Rightarrow$ 
1008     (qmul (qmul (basisU.getD  $\sigma$ (.getD 0 0) q1) (basisU.getD
1009        $\sigma$ (.getD 1 0) q1)) (basisU.getD  $\sigma$ (.getD 2 0) q1)).r ==
1010     (if invCount  $\sigma$  % 2 == 0 then -1 else 1))) = true := by
decide
```

Verdict. $\Rightarrow [\nabla]$ not a re-description; $\vdash \geq 5$ theorem classes $\wedge$ shrank anchor set. theorem-grade.

Source: Master Chapter 25, §2.4.

D9 · The gauge assignment is arbitrary

The Critic. A critic asks: assigning the three imaginary units to the three axes is arbitrary, so the seal rests on convention.

Reply. That arbitrariness is not a flaw the critic found. It is a feature the framework legislated. Which axis you label with which unit is indeed a free choice, exactly as which direction you call north is a free choice. What the framework seals is not the labels but the count and the angle, the things that do not change when you relabel. The verdict is built to be invariant under every relabeling and every rotation, and that invariance is checked numerically to machine precision. Move the labels all you want. The sealed content does not move.

Sealed Defense. The arbitrariness is the content of the gauge clause, legislated rather than discovered. The automorphisms of the quaternions are inner and act as

the rotation group on the imaginary part, the verdict functional is invariant under conjugation, $\text{Re}(rw\bar{r}) = \text{Re}(w)$, and under evidence-coordinate relabel, both confirmed at machine precision. The relabel admitted here is algebra-preserving and therefore even, and the restriction is load-bearing rather than pedantic: an odd relabel, a transposition of two axes, sends the Hamilton relation from $\text{Re}(ijk) = -1$ to $\text{Re}(jik) = +1$, so it is not an automorphism of $\mathbb{H}$ and is not a gauge move at all. Where a coordinate reads the orientation sign as an invariant, as P-LOGOS-FIX-1 does at its fourth section, this restriction is what the reading rests on: under $\text{Aut}(\mathbb{H}) = \text{SO}(3)$ the sign is preserved, and admitting odd relabels as gauge would collapse the two orientation orbits into one. The sealed content is the count and the invariant functional, never the labels. The genesis register carried the same recognition before the algebra existed: which slot wears which name is convention, and what the seeing fixes is the count, the disjointness, and the right angle. A seal stated on the labels would be the error the gauge clause forbids; the seal stated on the invariant is the seal the framework actually issues.

GOL DERIVATION

```
P_obj : {i,j,k}  $\rightarrow$  {V_F,V_E,V_ER} arbitrary  $\Rightarrow$  seal rests on
convention.
GAUGE:  $\text{Aut}(\mathbb{H})$  inner, acts as  $\text{SO}(3)$  on Im  $\mathbb{H}$ . labeling = free
gauge choice.
INVARIANCE: verdict | conjugation =  $\text{Re}(\bar{r}wr) = \text{Re}(w)$  ; verdict |
relabel = const. machine-verified. [T]
SEALED CONTENT: count(3)  $\wedge$  invariant-functional( $\det R$ ),  $\sim$ 
labels.
GENESIS: which-slot-which-name = convention; seeing fixes count
 $\wedge$  disjointness  $\wedge$  right-angle. $\Rightarrow$ 
 $\nabla$  gauge arbitrariness legislated, seal on the invariant not
the labels. theorem-grade.
```

Lean Proof.

Codex.lean, from line 1626

```
1626 theorem reflection_blind_executed :
1627   (adj3Samples.all (fun m => allSigns.all (fun s =>
1628     decide (det3 (sandwich s m) = det3 m)))) = true := by
1629   decide
```

Verdict. $\Rightarrow [\nabla]$ gauge arbitrariness legislated, seal on the invariant not the labels. theorem-grade.

Source: Master Chapter 25, §2.5.

D10 · It is a grammatical artifact

The Critic. A critic states: the deletion test is informal and that the count of three might be an artifact of grammar.

Reply. The second objection holds that the deletion test is informal and that the count of three might be an artifact of grammar. The objection is half right and fully anticipated. The test's warrant is operational, typed as such and never raised, and the count does not rest on it alone. The Frobenius classification forces the same three from the order-sensitivity of audit composition,

with no grammatical premise anywhere in the chain, and an artifact of grammar does not have a theorem of topology for a twin. The two forcings share no premise, the deletion test knowing nothing of division algebras and Frobenius knowing nothing of subjects and predicates, and a count forced once and corroborated once across disjoint anchors is not an accident of language [T on the algebraic forcing, Operational on the linguistic].

GOL DERIVATION

```
P_obj : deletion count = 3 is an artifact of grammar → Seal L
informal.
TEST: slot census on the encoded sentence, integer tags,
      executed (twin PART A). [T, executed]
      →EKR positional check executed; deletion of one slot refuses
      at two.
OBEDIENCE: deletion operator D = diag(1,1,0): D² = D, det D =
0, no left inverse. [T, kernel]
the transformation set of the Tongue is a monoid, never a
group; synonymy barred by LIT.→
[X] grammatical-artifact reading, broken: the count is
      executed on the encoding, not read off grammar.→
⊥[] Seal L is a floor and not a ceiling; unclosed by theorem,
obedient by theorem. operational-grade on the encoding.
```

The tier, fixed. The deletion test is a floor and not a ceiling, unclosed by theorem and obedient by theorem, DEFENSE.tongue_unclosed_and_obedient; its count of three is operational on the encoding and corroborates the theorem-grade count from the division algebras. The two routes share no premise and the seal counts them as one theorem and one corroboration, never as two forcings.

Lean Proof.

Codex.lean, from line 1695

```
1695 theorem tongue_obedience :
1696   GEO.det3 deletionOp = 0
1697   ∧ ∀( L : GEO.M3, (GEO.mv3 L) ∘ (GEO.mv3 deletionOp) ≠ id)
1698   ∧ ¬ ∀( M : GEO.M3, ∃ L : GEO.M3, (GEO.mv3 L) ∘ (GEO.mv3 M)
1699     = id) :=
(deletion_singular, deletion_no_left_inverse,
deletion_monoid_not_group)
```

Verdict. ⇒ the defended claim stands at the grade the derivation names; the objection is answered at its named mechanism.

Source: Master Chapter 15, §15.2.

Movement 3 • Distinct From Every Method

D11 • Bayesian aggregation in geometric clothing

The Critic. A critic states: the cascade is Bayesian updating dressed up as geometry.

Reply. The surface resemblance is real, and the framework grants it. Both want independent evidence, both get stronger under independent agreement, both distrust agreement from a single hidden source. But there is a case where the two give opposite answers, which proves they are not the same. Suppose all your evidence really traces to one common cause. A Bayesian counts that as confirmation and raises confidence. The framework counts it as collapsed inde-

pendence and refuses the seal. Same evidence, opposite verdict. On top of that, the framework checks things Bayes has no machinery for at all: whether your evidence lines are even expressed in independent vocabularies, and whether the things you subtract as confounds carry real physical mass. Bayes is blind to both.

The critic presses (§3.4): A critic states: each superiority you assert over Bayesian inference is your framework scoring Bayes against design choices Bayes never adopted, which is circular and proves nothing.

Reply, continued. The charge lands against most of the comparisons, and the framework concedes it: differing on trinary verdicts, vocabulary audits, or mass-bearing covariates is a difference of design, not a proof of superiority. There is exactly one place the framework provably beats a confidence calculation, and it is not a scorecard. It is one proven fact of geometry and one operation the confidence calculus does not carry. The fact: a piece of evidence fabricated to be independent of everything you already hold looks, to any confidence calculation, identical to a genuine independent piece. It is truly uncorrelated with your other evidence, so every independence check passes, it earns its weight, and it lifts your confidence, and no confidence math separates it from the real thing because the two are identical in every number that math reads. The operation that catches it compares the new evidence against a separately supplied source and asks whether it traces there, and that comparison is not part of the confidence calculus and is not demanded by it. So the framework screens one specific forgery that confidence math is blind to, sitting one step earlier in the pipeline, where evidence is built rather than combined. It does not replace Bayes. It catches one lie Bayes cannot see.

A planted witness who never spoke to your other witnesses is genuinely independent, so cross-examining the witnesses against each other clears him. Only tracing him back to whoever planted him exposes the plant, and a pure confidence calculation has no step that traces a witness to his source.

Sealed Defense, continued. The supersession reduces to one geometric theorem and one structural absence, and it is the only superiority claim that survives self-application, because it exhibits a fact about the determinant and an absence in the calculus rather than grading Bayes against the framework's own design. The theorem is Non-Discrimination. With two manifest axes at angle θ and a unit witness W split into in-plane and orthogonal parts, the determinant of the triad equals $\sin^2(\theta) \cdot \rho^2$, where ρ^2 is the squared orthogonal magnitude, so on the orthogonal complement, where every legitimate independent witness lives, the determinant equals $\sin^2(\theta)$ for every witness regardless of its direction in that complement; six source-distinct

orthogonal witnesses return the identical determinant to twelve figures. The determinant is a function of the angle and the orthogonal magnitude alone, and provenance resides in the one coordinate it does not read, the direction of the orthogonal residual. The load-bearing step is the sufficient-statistic corollary: with the manifest plane and the witness as the only inputs, the second-order sufficient statistic of the triad is invariant under that direction, because the determinant is pinned at $\sin^2(\theta)$ and both correlations with the manifest axes are pinned at zero by orthogonality, a difference confirmed at machine zero between two source-distinct orthogonal witnesses. Any posterior computed over those inputs is therefore a function of a statistic blind to provenance, and no credence over the manifest plane and the witness discriminates a source-faithful witness from a fabricated orthogonal one. The discrimination requires a third object, an independently supplied generator, and the squared partial correlation of the witness residual against the generator residual, which is not a function of the determinant, not a function of the triad's sufficient statistic, and not a byproduct of aggregating streams; it must be formed by explicit ingestion of the generator. The separation is bare: a sourced witness and fresh orthogonal noise returned determinants 0.919 and 0.921, within two thousandths and posterior-equivalent at the magnitude, while the source statistic read 0.910 for the sourced witness and 0.001 for the noise, carrying the entire discrimination. Credence reading the determinant or the independence structure cannot tell them apart; the source statistic tells them apart completely.

The perimeter is fenced on four sides, and naming all four is what keeps the claim out of the imperial register. A Bayesian who builds a model with an explicit latent-source node, supplies the generator as data, and conditions on the witness-to-source association is reconstructing the source statistic inside a Bayesian wrapper, which confirms the edge rather than defeating it, because the discriminating operation is named rather than derived from aggregation; the claim is over naked aggregation of the manifest plane and the witness, the default discipline, not over every model a Bayesian could build once told to supply the generator. The source statistic itself rides a supplied generator, so a fabricated generator gates it and a faked-generator lock is geometrically valid and source-faithless; the framework does not seal provenance absolutely, it relocates the question to the supplier and makes the supplier falsifiable at one named surface, where the credence calculus carries no such operation and the unfalsifiable residue is everywhere. The discrimination lives in the reading-context space, where the orthogonal complement has dimension at least two and the witness direction is a free degree of freedom; in three ambient dimensions the complement of a plane is a line,

the direction is forced, source attribution is vacuous, and the cross product is retired from the operational core. And the supersession is layer-precedence, the evidence-construction layer grounding the combination layer, not within-layer substitution and not universal supersession, so credence on certified architecture is untouched. The grade split travels: the blindness is theorem-grade, a proven property of the determinant, while the floor above which the source statistic reads as sourced is engineering, calibrated from the data's own structure with no human-fitted constant, and conflating the two would inflate a calibration to a theorem. This is the difference from §3.1: its four operations establish that the verdict functional is structurally not an aggregation rule, a distinctness; this one establishes the single place the framework provably catches what correctly-specified credence is blind to, a supersession, bounded to the source-attribution layer and surviving self-application where a scorecard does not.

Reply, continued. The first objection holds that the architecture is Bayesian aggregation in geometric dress. The answer is a theorem with an experiment attached. Aggregation over positive inputs is monotonic and cannot vanish on collinear data, and there exists no weighted-average aggregation of positive inputs that vanishes when the inputs are collinear. The verdict determinant vanishes there identically. Two instruments that order the redundant limit in opposite directions, the posterior rising while the determinant dies, are not notational variants of one instrument, and the divergence is a bench test any laboratory with floating-point arithmetic can run. The architecture is not the Bayesian calculus reclothed. It is the layer beneath it, and the one configuration on which the two provably diverge is the proof of the difference [T].

Sealed Defense. Four operations have no Bayesian analog. The Convergence Dissolution Test denies the seal when a single latent factor accounts for all convergence even when that factor supports the hypothesis, where a Bayesian common-cause posterior rises; opposite prescriptions from identical evidence structure. The Linguistic Isolation Test audits vocabulary-disjointness, to which likelihood ratios are indifferent. The verdict economy is trinary-native against continuous credence. The Mass Mandate admits only covariates carrying measurable thermodynamic mass, where Bayesian conditioning accepts any specifiable covariate. The sharpest discriminator is theorem-grade: under fully redundant evidence, three collinear rows, a Bayesian posterior strengthens monotonically toward its maximum while $\det(R) = 0$ issues [X], and no weighted average of positive inputs can vanish on collinear data, so the verdict functional is provably not an aggregation rule of that class. The composition is also order-sensitive where conditionalization on commuting evidence is order-invariant, and the verification

algebra honoring that order-sensitivity completes in the quaternions, not in any probability simplex. These four operations establish distinctness, that the verdict functional is not an aggregation rule of the credence class; the single place the framework provably catches what even correctly-specified credence is blind to, the orthogonal-witness forgery the determinant cannot see, is the theorem-grade supersession developed at §3.4.

The dissolution discriminator does not wait for collinearity in the raw axes; it subtracts the identified common cause by orthogonal projection and reads the residue, so it fires at any correlation whenever a named mass-bearing covariate accounts for the convergence, the determinant collapsing on the projected residue and not on the raw rows. The 2014 BICEP2 episode is the worked case: a five-sigma agreement on an inflationary signature across nominally independent channels that shared one galactic-dust foreground pipeline, a latent covariate not independent of the channels; projecting the shared pipeline collapses the residue, and the cascade refuses the seal at the input gate where the Bayesian likelihood ratio rose. The claim is layer-precedence, not same-layer substitution: the engine certifies whether the evidence architecture is non-degenerate, the prior layer that grounds whether credence-computation runs on trustworthy inputs, which Bayesian apparatus does not natively address.

GOL DERIVATION

```
P_obj : cascade = Bayesian aggregation in geometric clothing.
DISCRIMINATOR-1 (CDT, anti-Bayes): common-cause(E1,E2,E3) ∧
  supports(H)
  ⇒ Bayes: posterior ; cascade: 1-collapse ⇒ [X]. opposite
  from identical structure.
DISCRIMINATOR-2 (LIT): audits vocab-disjointness ; Bayes
  indifferent (likelihood-ratio only).
DISCRIMINATOR-3: verdict trinary-native ; credence continuous.
DISCRIMINATOR-4 (Mass Mandate): covariate | measurable-thermo-
  mass ; Bayes accepts any covariate.
SHARP (theorem): 3 collinear rows ⇒ Bayes posterior ⇒ max ∧ det
  (R) = 0 ⇒ [X].
  # weighted-avg(positive inputs) = 0 on collinear data ⇒
  verdict-functional ≠ aggregation-class.
MECHANISM: CDT fires by PROJECTION (subtract identified cause
  ⇒ read residue), ~ by det-detecting raw collinearity ;
  collapses @ ANY corr if named cause accounts for
  convergence.
BICEP2: o5 "independent" channels sharing dust-pipeline (
  latent covariate) ⇒ project ⇒ residue collapses ⇒ [X] @
  input gate ; Bayes posterior.
LAYER-PRECEDENCE: engine certifies architecture-non-degeneracy
  (prior layer grounding credence-trustworthiness) ~ same-
  layer substitute for Bayes-on-certified-inputs.
ORDER: composition order-sensitive ; conditionalization(
  commuting E) order-invariant.
  order-sensitive algebra completes in ℍ, ~ in probability-
  simplex.
  [ ] distinct at 4 named operations; theorem-grade at the
  collinear discriminator.
```

GOL DERIVATION, CONTINUED (§3.4)

```
P_obj : every Bayes-advantage = scorecard on Trisduction's own
  specs ⇒ circular.
CONCEDE: most claimed advantages = design-difference ~ proof-of
```

```
-superiority. (§3.1 four-ops = distinctness ~
  supersession.)
THE ONE EDGE = geometric-theorem @ structural-absence. ~
  scorecard, ~ axiom-grading, ~ 15-invariants.
NON-DISCRIM (Completion Ineq): det(R){a,b,W} = 0psin2, p2 =
  ||1||W2 ∈ [0,1].
W ⊥ span(a,b) ⇒ p2=1 ⇒ det(R) = 0sin2 ∀ direction-in-
  complement. [GV-CHK.3: 6 witnesses ⇒ 0.750000000000,
  spread 0]
det(R) = f0(, p2) only ; provenance ∈ direction(1W) = the 1
  coord det ~ reads. [T]
SUFF-STAT COROLLARY (load-bearing):
  2nd-order suff-stat{a,b,W} invariant under direction(1W) :
  det pinned 0sin2 ∧ corr(W,a)=corr(W,b)=0 by 1. [
  verified: Δ|Gram| = 2e-16]
  ⇒ ∀ posterior | {a,b,W} = f(suff-stat) ⇒ provenance-blind.
  [T | Gaussian-reduction premise-typed]
discrimination ⇒ requires 3rd object S (supplied generator) +
  η_S = r2(1W, 1S).
η_S ≠ f(det R) ∧ ≠ f(suff-stat{a,b,W}) ∧ ≠ byproduct(
  aggregation) ⇒ formed only by explicit S-ingestion.
WORKED (GV-CHK.5): sourced det 0.919 p2 0.994 η_S 0.910 ; noise
  det 0.921 p2 0.997 η_S 0.001.
Δ|det| = 0.002 posterior-equiv @ magnitude ; η_S carries 100%
  of the discrimination.
PERIMETER (4 fences ; omit-any ⇒ inflation):
  (1) hierarchical-Bayes : supply-S + condition-on(W,S) =
    reconstruct η_S in Bayes-wrapper ⇒ CONFIRMS (names op ~
    derives from aggregation). edge = over NAKED-aggregation{
    a,b,W}, ~ over every-model.
  (2) supplier-aperture : η_S rides supplied-S ; faked-S =
    valid-but-faithless. ~ solve-provenance-absolutely ;
    relocate ⇒ supplier, falsifiable @ 1 surface. (credence:
    residue everywhere.)
  (3) Room : discrimination ∈ N-context space,
    centered dim(complement) = -Nk3 = m ≥ 4 (interior rank
    floor -Nk ≥ 4) ; 3-ambient ⇒ line ⇒ forced ⇒ vacuous.
    cross-product retired.
  (4) layer-fence : source-attribution sub-layer only ;
    layer-precedence (evidence-construction ⊃ combination) ~
    within-layer-substitution ~ universal.
GRADE SPLIT: blindness = [T] (proven det-fact) ; seal-floor (
  dual-null) = engineering (data-calibrated, 0 human-const)
  . keep apart.
IMPRINT: ~ (magnitude-functional discriminates source-faithful
  vs fabricated-orthogonal) field-permitted ; opposite
  forbidden by Non-Discrim ⇒ determinate direction.
SELF-APP: by P-FOUNDATION-1 ~ sealed as theorem of own base ;
  rests on Completion-Ineq [T] + row-supply [prem] ; 0
  warrant to foundations ; certifies bounded-edge ~
  universal-superiority.
  [ ] one theorem-grade edge: det blind to the orthogonal
  forgery, caught only by η_S over supplied S, absent from
  credence ; bounded to the source-attribution layer ;
  survives self-application.
Trisduction ~ replace Bayes ; catches 1 lie Bayes ~ see, 1
  step earlier in the pipeline.
```

Lean Proof.

Codex.lean, from line 3428

```
3428 /-- PSP: D11, D12 · consensus carries zero weight; the engine
  authors nothing. -/
3429 theorem consensus_weights_zero : socialWeight = 0 ∧ deltaM = 0
  := FOUND.engine_never_authority
```

Codex.lean, from line 144

```
144 theorem Grade.weakest_is_a_link (a b : Grade) :
145   Grade.weakest a b = a ∨ Grade.weakest a b = b := by
146   cases a <|> cases b <|> decide
```

Verdict. ⇒ Bayes: posterior↑ ; cascade: 1-collapse ⇒ [X]. opposite from identical structure. ⇒ [] distinct at 4 named operations; theorem-grade at the collinear

discriminator.

Source: Master Chapter 25, §3.1.

D12 · It is severe testing, Popper, or a consensus of methods

The Critic. A critic states: this is Mayo's severe testing, or Popperian falsification, or Wilson-style consilience, renamed.

Reply. Each of those is a real method, and the framework contains each as a part while showing each is incomplete on its own. Severe testing works inside one line of evidence at a time; the framework works across three at once and demands they be independent. Falsification can reject a theory but cannot, by itself, tell you what counts as a successful test in the first place, so it needs a verification frame around it, which the framework supplies. Consilience and triangulation assume that using many methods guarantees independence, but many methods can all lean on one hidden cause, which the framework's dissolution test catches and they cannot.

Sealed Defense. Mayo's severe testing operates within one axis and is probabilistic; the framework operates across the triaxial structure with mutual-orthogonality and a geometric residue, subsuming severity as one component of empirical discipline. Popperian falsification as sole criterion cannot specify the verification framework needed to recognize a falsification; as one component it seals, as standalone it shows incompleteness, and the framework subsumes it at the causal-mechanism gate and the bridge-axiom gate. Consensus methodologies, consilience, triangulation, convergent validity, each assume that diversity of method or discipline suffices for independence; the dissolution test denies this, since diversity without independence fails on a shared latent factor, and the consensus frameworks lack the operational test for shared latent factors the framework supplies. The distinctness is structural, per-component, and earned at specific operational tests.

GOL DERIVATION

```
P_obj := Mayo v Popper v consilience, renamed.
MAYO: severity ∈ 1 axis, probabilistic. framework: 3 axes ⊥ +
      geometric residue → subsumes severity as 1 component.
POPPER: falsification(sole) ≠ specify verification-frame. as
        component: seals; standalone: incomplete.
        subsumed @ G4-CAUSAL ∧ G12-ADEG.
CONSENSUS (Wilson/Denzin/Campbell-Fiske): assume method-
          diversity = independence.
CDT denies: diversity ∧ shared-latent-factor → fail. they
            lack the shared-factor test.
⊥[] each subsumed as 1 component, each incomplete standalone;
      distinctness structural ∧ per-component.
```

Lean Proof.

Codex.lean, from line 1728

```
1728 theorem citation_cannot_promote (c cite : Claim) :
1729   (Claim.join c cite).grade.rank ≤ c.grade.rank ∧
1730   (Claim.join c cite).grade.rank ≤ cite.grade.rank := by
1731   have h : ∀ ig zg : Grade,
```

```
1732   (Grade.weakst ig zg).rank ≤ ig.rank ∧ (Grade.weakst ig
1733     zg).rank ≤ zg.rank := by
1734   intro ig zg; cases ig <;> cases zg <;> decide
1735   exact h c.grade cite.grade
```

Verdict. $\Rightarrow \lceil \lceil$ each subsumed as 1 component, each incomplete standalone; distinctness structural $\wedge$ per-component.

Source: Master Chapter 25, §3.2.

D13 · The bare-axiom apex seal is too strong

The Critic. A critic states: the claim that the bare Root Axiom seals at theorem grade is overstated.

Reply. It would be overstated if the framework leaned on its own machinery to prove it. It does not. The bare axiom rests on outside physics and outside mathematics that the framework did not invent: the Heisenberg kinetic-energy floor and the zero-point energy from the canonical commutator, $E = mc^2$ for positive rest energy, the rigorous third law of Masanes and Oppenheim, and the Jarzynski-Crooks work relations, with the laboratory measurements that confirm them, all converging on the single point that no truly zero-energy state of a substrate is reachable. The seal is typed honestly: the bare axiom seals at theorem grade on those external anchors, and everything the framework adds on top seals only at whatever its own extensions earn. There is no inflation, because the strongest claim is carried by the parts the framework borrowed, not the parts it built.

Sealed Defense. The bare-axiom seal rests on theorem-grade external physics, the Heisenberg kinetic-energy floor and the zero-point energy from the canonical commutator, $E = mc^2$ for positive rest energy, the rigorous third law of Masanes and Oppenheim barring a cooled-to-zero state, and the Jarzynski-Crooks work relations charging any transition, with the consistency apparatus and Hadamard on the mathematical side, all converging on the single point that no zero-substrate-state is reachable. The five laboratory instruments, the Lamb shift, Casimir, MICROSCOPE, the Bérut-Landauer measurement, and Nernst, are the measured confirmation of that floor, the shadows of the zero-point energy and not its proof. The apex is honestly typed: the bare axiom seals at theorem-grade external warrant on the physics and mathematics anchors, the five-instrument convergence confirming it, and extensions seal only at the warrant their bridge axioms carry. The architect-commitment covariate is subtracted and the bare-axiom seal survives independent of it via the external anchoring, so the strongest claim does not depend on the framework's own apparatus.

GOL DERIVATION

P_obj : bare-RA seals **theorem**-grade = overstated.
 ANCHORS-THM: {Heisenberg KE floor, ZPE $\hbar\omega$, $E=mc^2$ (Rainville 2005), 3rd law (Masanes-Oppenheim 2017), Jarzynski-Crooks ; consistency; Hadamard}. CONFIRM-EMP: {Lamb, Casimir, MICROSCOPE, Bérut-Landauer, Nernst} = measured shadows $\rightarrow$ proof.
 CONVERGENCE: all $\supset$ "no zero-substrate-state reachable" (meta-pattern).
 TYPING: bare-RA $\omega[]$ T | external-anchors $\wedge$ 5-instrument convergence. extensions $\omega[]$ | bridge-warrant only. $\rightarrow$ inflation.
 CDT: subtract C_4 (architect-commitment); bare-RA residue persists $\rightarrow$ external anchoring.
 $\omega[]$ bare-RA **theorem**-grade on exogenous warrant (Heisenberg floor, ZPE, $E=mc^2$, rigorous third law; 5-instrument convergence confirming); premise-grade-by-theorem internally where the framework's own seal cannot issue a **theorem** on its own root (FOUNDATION-01); strongest claim $\notin$ framework's own apparatus. $\rightarrow$ a bare premise-demotion: both halves carried, the floor **theorem**-from-outside, the foundation-role premise-by-theorem, neither inflated nor watered down.

The anchors, typed. The floor seals at theorem grade on external anchors: the Heisenberg bound and the zero-point energy for existence, the third law, and Jarzynski-Crooks with Landauer for the transition cost, OMEGA.omega_paid_floor. Hadamard sits in the corroboration column, where D8 placed it, and never among the theorem anchors. The axiom as such stands premise by theorem, TONGUE.ra_grade_pinned, the highest standing a root can bear, and the bare-RA seal reads at exactly that layered grade.

Lean Proof.

Codex.lean, from line 1319

```
1319 theorem ra_ledger_aGivenRA :
1320   ((List.range 512).filter (fun n => decide ((raDecode n).1
    = .aGivenRA))).length = 2 := by
1321   decide
```

Verdict. $\Rightarrow [\omega]$ bare-RA theorem-grade on exogenous warrant (Heisenberg floor, ZPE, $E=mc^2$, rigorous third law; 5-instrument convergence confirming); premise-grade-by-theorem internally where the framework's own seal cannot issue a theorem on its own root (FOUNDATION-01); strongest claim $\notin$ framework's own apparatus. $\rightarrow$ a bare premise-demotion: both halves carried, the floor theorem-from-outside, the foundation-role premise-by-theorem, neither inflated nor watered down.

Source: Master Chapter 25, §3.3.

D14 • Completion claims have a bad history

The Critic. A critic states: completion claims have a uniformly bad history, from Cartesian indubitability through the closure of metaphysics, absolute knowing, the verificationist criterion, and the formalist program, every one broken by a named mechanism.

Reply. The third objection holds that completion claims have a uniformly bad history, from Cartesian indubitability through the closure of metaphysics, ab-

solute knowing, the verificationist criterion, and the formalist program, every one broken by a named mechanism. The history is real, it is mass-bearing, and it cuts in the architecture's favor, because every claim on that list asserted indefeasibility, and the architecture proves indefeasibility unattainable and claims its opposite, a classification of attainable warrant. Classification is the one form of closure with an unbroken record. The Frobenius list has not grown since 1878 and the Galois verdict on the quintic has not moved since 1832, and the closures that stand are the closures that classified rather than promised. The architecture promises nothing it cannot enumerate and classifies what was there to be found [S, against the record].

GOL DERIVATION

P_obj : completion claims fail uniformly (Descartes, Hegel, verificationism, Hilbert) $\rightarrow$ this one fails.
 LEDGER: each named failure is a bare-**axiom** apex read as **theorem**, or a lock read as truth. [S]
 GRADE LAW: join(c, cite).grade $\leq$ min(c.grade, cite.grade); citation never promotes. [T, kernel]
 HALT LAW: every halt grade $<$ **theorem**; the frontier verdicts are halts, never seals. [T, kernel]
 [X] the **induction** from history, broken: the failed programs claimed the grade this architecture refuses to claim.
 $\omega[]$ the record audit stands at structural grade; the root is premise by **theorem**, the highest standing a root can bear.

Lean Proof.

Codex.lean, from line 2452

```
2452 theorem ftoe_ceiling :
2453   haltGrades.all (fun g => g.rank < Grade.theorem.rank) =
    true := by
2454   decide
```

Verdict. $\Rightarrow$ the defended claim stands at the grade the derivation names; the objection is answered at its named mechanism.

Source: Master Chapter 15, §15.3.

D15 • The five unassembled fragments

The Critic. A critic states: Five established literatures each own a piece of the central result, and each inhabits a register that cannot close on itself, which is why the result was available and unassembled.

Reply. Five established literatures each own a piece of the central result, and each inhabits a register that cannot close on itself, which is why the result was available and unassembled. The classification line, Frobenius confining the associative real division algebras to the reals, the complexes, and the quaternions, Hurwitz confining a multiplicative quadratic norm to dimensions one, two, four, and eight, and Bott-Milnor and Kervaire and Adams closing every non-associative escape, delivers a complete and closed inventory of admissible carriers and no bridge from verification practice to its own hypotheses, an architecture that merely assumes associativity and integrality having assumed its way to the answer. The generalized-variance line,

Wilks naming the determinant of the correlation matrix and Bartlett supplying its null-distribution machinery, delivers the verdict object with a developed sampling theory and no positive derivation, the determinant a pathology detector consulted after the fact and never a truth functional derived from first structure. The invariant-theoretic line, Weyl's first fundamental theorem generating all polynomial invariants of vector tuples in three-space, delivers catalog-closure of complete generality and no selection, which invariant is the verdict and why it appears squared questions invariant theory is not built to answer. The quantum-cognition line, Busemeyer and Bruza assembling the case that judgment composes by non-commuting operations, delivers the empirical face of order-sensitivity and no carrier, the formalism operating in complex Hilbert space, one imaginary axis unable to host a triad. The verification-methodology line, Bayesian confirmation theory and Popperian falsificationism and Mayo's severe testing, each supplies a discipline for relating evidence to claims and each takes the verification apparatus itself as an external presupposition, none deriving its own apparatus and none carrying a closed-form truth functional. The common structural error is one sentence: every fragment inhabits a register that cannot close on itself, the classification with no hypotheses of its own, the determinant with no derivation, the catalog with no selection, the order-sensitivity with no triad, the methodologies with no apparatus. The assembly point at which all five close simultaneously is the quaternion algebra entered through the architecture's own composition law, and the assembly is the contribution, the parts old and credited [S].

GOL DERIVATION

P_obj : the pieces existed already (Bayes, Popper, Frobenius, Landauer, Gödel) $\Rightarrow$ nothing here is new.
 FRAGMENTS: five, each cited **at** its own tier, $\Delta M = 0$ on every one. [T, cited]
 JOIN: the assembly is the contribution; join grade = weakest link, never above any fragment. [T, kernel]
 NOVELTY: consensus weight = 0 **in** both directions, so "old" and "new" both weigh zero. [T, kernel] $\Rightarrow$
 [X] the reduction-to-fragments, broken: it grades the parts and not the arrangement. $\Rightarrow$
 $\llcorner$ [] the arrangement stands **at** the grade of its weakest link, stated, never inflated.

Lean Proof.

Codex.lean, from line 161

```
161 theorem Claim.join_grade (c zc : Claim) :
162   (Claim.join c zc).grade = Grade.weakest c.c.grade zc.grade := rfl
```

Verdict. $\Rightarrow$ the defended claim stands at the grade the derivation names; the objection is answered at its named mechanism.

Source: Master Chapter 15, §15.8.

D16 · The alien and the crank

The Critic. A critic states: Two final objections bracket the architecture.

Reply. Two final objections bracket the architecture. The alien objection holds that a sufficiently strange logic dissolves the architecture's guards, and the answer is the Chatok principle, that the guards route through the deed and not the symbol, and no logic makes a deed not be a deed. The crank objection holds that the architecture claims to break the limitative theorems, and the answer is that it claims the opposite, placing Gödel inside the provability sublayer as a fact about the ladder's reach and proving no Gödel sentence in any object system, instantiating no complete recursive decision procedure, and embracing its own incompleteness through the open aperture. The architecture is founded on the far side of the limit, where the limit is a theorem about the ladder it left below, and it neither breaks the wall nor pretends the wall is not there [T on the placement]. $\therefore$ {jbox title="The defense sealed"} Every objection met on its merits and not deflected. The Bayesian-clothing charge answered by a theorem and a bench test, the grammatical-artifact charge by a twin theorem of topology, the bad-history charge by the unbroken record of classification, the self-certification charge by audit symmetry stated in advance, the ungrounded-root charge by the empty-throne law that makes unprovability the mark of foundation-hood, and the alien and crank charges by the deed-routing guard and the placement of the limit. The architecture certifies its own form and never its own truth, and rests its truth where it belongs, on external mathematics and falsifiable divergence [$\llcorner$ on the form]. $\therefore$

GOL DERIVATION

P_obj : an alien logic, or a crank, can write a capture of the architecture **in** its own terms.
 AEGIS: $p(G) \neq G$; precisification is an actuation, G a non-actuation; refusal constant over four logics. [T, executed]
 DEED COUNT: every adjudication increments the deed counter; the capture is written, never performed. [T, kernel] $\Rightarrow$
 [X] the capture, broken **at** the deed: the alien writes it and cannot perform it. $\Rightarrow$
 $\llcorner$ [] the guard is a constant function of the deed, **by** construction; logic-priority is the design.

Lean Proof.

Codex.lean, from line 1970

```
1970 theorem aegis_battery :
1971   (aegisGuard "classical" aegisReset).1 = (aegisGuard "
1972     paraconsistent" aegisReset).1
1973    $\wedge$  (aegisGuard "classical" aegisReset).1 = (aegisGuard "
1974     fuzzy" aegisReset).1
1975    $\wedge$  (aegisGuard "classical" aegisReset).1 = (aegisGuard "
1976     substructural" aegisReset).1
1977    $\wedge$  aegisFourLogicRun.deeds = 4
1978    $\wedge$  aegisFourLogicRun.lastLen = 13 := by
1979     decide
```

Verdict. $\Rightarrow$ the defended claim stands at the grade the derivation names; the objection is answered at its named mechanism.

Source: Master Chapter 15, §15.7.

Movement 4 • The Limitative Theorems

D17 • Gödel applies, so the system cannot verify its own soundness

The Critic. A critic states: the cascade is a formal system, so Gödel's theorems apply and it cannot certify itself.

Reply. The framework does not try to climb to its own foundation, so the trap never closes on it. Picture a ladder reaching up toward a high ground. Gödel's result says the ladder, however long, never reaches the very top. True, and the framework agrees. But the framework does not stand on the ladder looking up. It stands on the ground looking down at the ladder. From there, the fact that a finite ladder falls short of an unbounded ground is not a paradox, it is just a remark about the ladder. The framework's own verdict computation is a short, finite, terminating calculation, the kind of thing Gödel's theorem does not even apply to.

Sealed Defense. In the kinetic register, Gödel is a formal-only ceiling and the cascade routes around it at layer difference, through the empirical and registration axes, with formal-only undecidable statements sent to the out-of-band register where they belong. In the reflective register, the stronger placement, Gödel is a fact about the provability ladder, the provable set strictly inside the grounded set, a certificate of the ground's surplus rather than a ceiling over the architecture, which is written ground-first. The construction runs on the fixed-point-free diagonal, which carries no fixed locus, so it bounds the ladder and cannot bound the ground. The verdict functional is a fixed, loop-free, total computation terminating in bounded steps, and incompleteness attaches to provability predicates over arithmetic-rich systems, not to bounded total functions; the rotation-invariant catalog is closed by Weyl, so the formal surface is exactly as small as it declares.

GOL DERIVATION

```
P_obj : cascade = formal system  $\Rightarrow$  Gödel applies  $\Rightarrow$   $\neg$  self-
certify.
KINETIC: Gödel  $\in$  V_F-only ceiling. cascade routes around @
layer-difference via V_E  $\perp$  V_ER. V_F-only-undecidable  $\rightarrow$ 
OOB.
REFLECTIVE (ground-first): Gödel = fact about ladder. Provable
 $\subseteq$  Grounded (L2m  $\subseteq$  L1m).
= certificate of ground-surplus,  $\neg$  ceiling over architecture.
ENGINE: Gödel construction runs on fixed-point-free diagonal (
Ground =  $\emptyset$ )  $\Rightarrow$  bounds ladder  $\neg$  ground.
FUNCTIONAL: verdict = loop-free total computation, bounded
steps. incompleteness | provability-predicate(arith-rich)
,  $\notin$  bounded-total-function.
catalog closed (Weyl)  $\Rightarrow$  formal surface = exactly as declared.
 $\Rightarrow$ 
 $\perp$  [ Gödel honored  $\wedge$  placed in L2m;  $\neg$  broken  $\wedge$   $\neg$  escaped.
theorem-grade on placement.
```

Lean Proof.

Codex.lean, from line 2363

```
2363 theorem route_is_routeHalt :  $\forall$  n : Nat, route n = ofUC (UC.
routeHalt n).1 := by
2364   intro n
2365   unfold route UC.routeHalt ofUC
2366   by_cases h1 : n = 1
2367   · subst h1; simp
2368   · by_cases h0 : n = 0
2369   · subst h0; simp
2370   · have h1' : (n : Int)  $\neq$  1 := by omega
2371     have h0' : (n : Int)  $\neq$  0 := by omega
2372     simp only [h1, h0, h1', h0', ite_false]
```

Codex.lean, from line 2340

```
2340 theorem halt_free_iff_all_pass_8 :
2341   patterns8.all (fun bits => (cascade bits).isNone == bits.
all id) = true := by decide
```

Verdict. $\Rightarrow$ $\perp$ Gödel honored $\wedge$ placed in L2m; $\neg$ broken $\wedge$ $\neg$ escaped. theorem-grade on placement.

Source: Master Chapter 25, §4.1.

D18 • The halting problem applies

The Critic. A critic states: deciding whether a proposition halts at a verdict is the halting problem, which is unsolvable.

Reply. The cascade is a program that always stops. It never takes the kind of open-ended input the halting problem is about. The questions that genuinely cannot be decided in finite steps are turned away at the door, before the cascade runs, and sent to a separate shelf marked out of bounds. What the cascade actually processes, claims that split across its three axes, it processes in a fixed number of arithmetic steps with no open-ended search anywhere. The framework does not solve the halting problem. It honors it and stays out of its way.

Sealed Defense. The cascade is a halting algorithm terminating in bounded steps under the strict precedence in which admissibility is checked first and collapse outranks conditioning. The halting theorem concerns whether arbitrary formal-only propositions decide in some general procedure; the cascade does not take arbitrary formal-only propositions as input. The scope-check input gate routes formal-only ceiling propositions, halting, Gödel sentences, undefined truth-predicates, apex-coordinate circularity, to the out-of-band register before execution. For triaxially-decomposable propositions, where the formal-only undecidability is not load-bearing because the empirical and registration axes carry orthogonal content, the cascade terminates with a discrete verdict. The verdict stage is a loop-free arithmetic circuit with no unbounded search, total on the admissible domain.

GOL DERIVATION

P_obj : deciding verdict-halt = halting problem, unsolvable.
 CASCADE: halting algorithm, bounded steps, strict precedence (admissibility → collapse → conditioning).
 INPUT-GUARD: V_F-only ceilings {halting, Gödel, Tarski-undef, apex-circularity} → 00B @ G11 before execution.
 SCOPE: cascade-input = triaxially-decomposable → V_F-undecidability → load-bearing (V_E ⊥ V_ER carry content).
 STAGE: verdict = loop-free arithmetic circuit, $\nexists$ unbounded search, total on admissible domain.→
 $\nexists$ cascade halts; halting honored → solved; arbitrary V_F-only never taken as input.

Lean Proof.

Codex.lean, from line 2334

```
2334 theorem first_failure_terminal_8 :
2335   (patterns 8).all fftSpec = true := by decide
```

Verdict. ⇒ $\nexists$ cascade halts; halting honored → solved; arbitrary V_F-only never taken as input.

Source: Master Chapter 25, §4.2.

D19 · A floor every attack instantiates is empty

The Critic. A critic states: you say any argument against you uses your structure, which makes you unfalsifiable and therefore vacuous.

Reply. The framework agrees this is a trap and refuses both halves of it. It is true that any argument whatsoever uses a subject, a predicate, and a frame, so in that trivial sense nothing escapes the floor, the same way nothing escapes physics. But that triviality certifies nothing special about the framework. It belongs to the floor, and the floor belongs to no one. So the framework draws a sharp line. The ageless floor needs no defending and earns no credit for being unassailable. The dated architecture built on it earns its place only by being useful, and only until something better replaces it. Welding the two together, claiming the architecture is as permanent as the floor, would buy a false permanence at the price of saying nothing, and the framework refuses that trade.

Sealed Defense. The test for which stratum a thing belongs to is whether it has an outside. The formal architecture has an outside: every rival account of verification, every objection, which is what keeps it a claim about the world rather than a definition and what lets it be wrong. The deepest structure has no outside, but only in the near-trivial sense that any argument against it must use a subject, a predicate, and a frame, true the way “nothing is outside physics” is true under any name. That triviality certifies nothing about the architecture; it belongs to the floor, and the floor is no one’s. Boldness belongs to the ageless floor, which needs no defending; restraint belongs to the dated architecture, which earns its place only by being useful and only until something better replaces it. Welding the two buys a false permanence at the price of saying nothing, and the framework refuses that trade.

GOL DERIVATION

P_obj : attack-instantiates-floor → unfalsifiable → vacuous.
 STRATUM-TEST: has-outside(x) ?
 architecture: has outside (every rival, every objection) → claim-about-world, falsifiable, can be wrong.
 floor: no outside, but trivially (any argument uses subject $\wedge$ predicate $\wedge$ frame), = “nothing outside physics”.
 TRIVIALITY: certifies $\emptyset$ about architecture; belongs to floor; floor = no-one’s.
 DISCIPLINE: boldness $\in$ floor (needs no defense) ; restraint $\in$ architecture (earns place by use, until superseded).
 weld(floor, architecture) → false-permanence @ price of saying-nothing. refused.→
 $\nexists$ both traps refused; architecture falsifiable (has outside), floor trivial (no credit to architecture).

The weld, refused and kept. The floor certifies nothing special about the architecture, and the concluding unit D71 claims nothing special of it: the deed instantiates the root and only the root, Bridge.opens_on_the_deed, while the architecture’s defense at every proposition is falsifiability, unit by unit. The two units are one law read from its two ends.

Lean Proof.

Codex.lean, from line 1872

```
1872 theorem omega_paid_floor :
1873   (omegaBoundary 1 300).ocode = 2  $\wedge$  (omegaBoundary 1 300).
      token = .priced
1874    $\wedge$  (omegaBoundary 1 300).joules = landauerRat 300 1
1875    $\wedge$  0 < (omegaBoundary 1 300).joules := by
1876     decide
```

Verdict. ⇒ $\nexists$ both traps refused; architecture falsifiable (has outside), floor trivial (no credit to architecture).

Source: Master Chapter 25, §4.4.

D20 · A ground beyond reach is exile

The Critic. A reader asks: if the limitative theorems put the foundation permanently out of reach, isn’t that a bleak result?

Reply. The bleakness is a choice the mathematics does not force. The theorems say the ground cannot be reached from inside the system. They do not say what that ground is like. There are two readings. One reads the unreachable beyond as a lack, a place the system is exiled from. The other reads it as the system’s own ground, hidden from itself, so that reaching it would be a homecoming rather than an exile. The mathematics does not decide between these. Under the framework’s monism the beyond is the ground the system already is, so the homecoming reading is available, and it costs nothing extra and bends no theorem, so the framework takes it.

Sealed Defense. A limit of the Gödel-Lawvere kind establishes that a rich-enough system cannot close on its own ground from inside; that ground is real and unreachable by internal motion, and that much is settled. The limit is neutral on the valence of the beyond: it does not say whether the beyond is an other the system lacks, read as exile, or the system’s own ground

hidden from it, read as homecoming. Under the framework's monism the beyond is the ground the system already is, so the homecoming reading is available, and the choice between readings is underdetermined by the mathematics. The homecoming reading weakly dominates, at zero formal cost, because it bends no theorem and is more livable. This is the will-to-believe move, sharper than the classical wager because the beyond's existence is given by the limit rather than bet upon, so no infinite-utility term and no many-grounds objection enters; only the valence is chosen, and only by dominance under a stated value.

GOL DERIVATION

```
P_obj : foundation out of reach → exile ∧ incompleteness (bleak
).
LIMIT (Gödel-Lawvere): rich-system ≠ close on own ground from
inside. ground real ∧ internally-unreachable. [settled]
NEUTRALITY: limit silent on valence(beyond) ∈ {other-lacked (
exile), own-ground-hidden (homecoming)}.
MONISM: beyond = ground-system-already-is → homecoming-reading
available. choice underdetermined by math.
DOMINANCE: homecoming weakly-dominates @ 0 formal cost (bends
no theorem, more livable).
= will-to-believe, sharper than Pascal (existence given by
limit ~ bet; no ∞-utility, no many-grounds).→
⊥[] valence-choice by dominance under stated value;
conditional on monism.
```

Lean Proof.

Codex.lean, from line 2494

```
2494 theorem price {Q : Type} :
2495   Nonempty (Dtau (execFlip (Q := Q)) =
2496     (Quot (OrbitRel (execFlip (Q := Q))) → Bool)) :=
2497   T16_price_bijection execFlip execFlip_involution deed
```

Verdict. ⇒ $\perp[]$ valence-choice by dominance under stated value; conditional on monism.

Source: Master Chapter 25, §4.5.

D21 · No layer audits its own system

The Critic. A critic states: the deletion test returning three and Frobenius returning three were both done by you, so their agreement has a latent common factor, authorial design, and your own mass-criterion forbids subtracting it, so you cannot dissolve the very convergence you rest on.

Reply. The objection lands a real point about discovery and misses where the count actually rests. The count of three does not rest on two routes agreeing. It rests on one theorem. Frobenius forces the verification algebra to the quaternions, and three is the number of imaginary axes there, with no other answer available for an associative real division algebra carrying plural imaginary axes. That is a classical theorem any mathematician verifies, not something the author made come out a certain way. The hands-on language test is corroboration that agrees with the theorem, and it too is reproducible: any analyst who runs the deletion test on the axiom gets three, because the test is a procedure, not

a private opinion. So the agreement of the two routes is a real fact in the world, reproducible by anyone, not an artifact of one author's design. As for authorship as a hidden common cause, authorship is exactly the kind of factor the framework's mass rule throws out, a story about where an idea came from, carrying no measurable weight, with no mechanism by which it could force a theorem and a procedure to agree. What genuinely stays open is narrower than the objection claims: not whether the count is valid, which a theorem and a reproducible procedure settle, but whether the author was first to reach it, a question about discovery that independent reproduction answers and no internal argument can.

The critic presses (§13.4): A critic states: your result that no layer of the system can fully audit itself was itself reached from a vantage outside the system, so the framework cannot internally certify even that limitation.

Reply, continued. The limitation is real, and the framework's response to it is the correct one rather than a failure. No part of a system can give a complete audit of the whole system from inside it; that is a theorem, Gödel's second and Tarski's, and the framework imports it rather than pretending to prove it from within. The objection notes that this theorem is a claim made from outside the system, so the framework cannot certify it using only internal resources. That is true, and it is true of every meta-theorem of this kind, not a special defect here. The point the framework makes is the opposite of self-grounding: it does not claim to certify its own truth from inside. It anchors truth on theorems and measurements that sit outside it, and it demonstrates its floor by enacting it, since every audit it runs burns energy and so instantiates the very floor it stands on. The externality the objection treats as a hole is the design. A framework that could close the loop on its own self-audit would be claiming exactly the complete internal self-grounding the imported theorem forbids, and the framework declines that claim on purpose.

Sealed Defense, continued. The result that no internal layer fully audits its own system is an external-vantage theorem, and the framework's stance toward it is not an attempt at internal closure but a deliberate refusal of self-grounding. The regress the objection names is clean, internal certification of no-internal-full-self-audit would require an internal full self-audit, the denied object, so the certification is unavailable from inside; but this regress is constitutive of every limitative meta-theorem and is precisely why the framework imports Gödel's second, Tarski's undefinability, and the Münchhausen trilemma rather than deriving them from within. The framework's self-application is two-tier and honest at both. At the operational tier the architecture submits to its own cascade and passes

internal consistency, a theorem-grade and achievable result that does not certify truth. At the architectural tier the truth-warrant is routed entirely to exogenous anchors, the division-algebra classification, the Hodge decomposition, the Landauer floor and its five measured instruments, none of which the framework generates. So the self-audit is consistency relative to external anchors, exactly as the framework already states, and the floor is certified enactively, every audit instantiating the Landauer cost it rests on, a self-demonstration rather than a self-certification. The objection is granted as constitutive and reframed as correct: the external vantage cannot be internalized, the framework does not try to internalize it, and a framework that claimed otherwise would be asserting the complete self-grounding the imported theorem forbids. What stays open is self-grounding, which is impossible in principle and which the framework never claims; what is closed is the charge of overreach, which fails, because the framework anchors outward and enacts its floor rather than grounding itself in a circle.

Sealed Defense. The objection conflates the count's validity with its discovery, and only the second carries any residue. The count of three is not load-bearing on the convergence of two routes; it is forced by the Frobenius classification, theorem-grade conditional on the composition-law clauses, the quaternions being the unique associative real division algebra with plural imaginary axes and three the dimension of their imaginary part. The semantic deletion test is premise-disjoint corroboration, operational-procedural and reproducible across analysts, not a co-equal route whose mere agreement with the algebra constitutes the evidence. Authorial design, the proposed latent common cause, is narrative provenance, which the Mass Mandate bars from standing as a covariate at all, not because the Titanium Ruler shields it as a precondition but because it carries no measurable mass and supplies no mechanism by which it could compel an associative-division-algebra theorem and a deletion procedure to coincide. A massless factor cannot be the common cause of a convergence between a theorem and a reproducible procedure, and both legs are enactively reproducible by parties who share nothing with the author, so the agreement is a structural fact rather than an authored coincidence. The residue, correctly scoped, is not the validity of the count but the provenance of its first derivation, a question independent reproduction settles and internal argument cannot, and beneath that the honestly premise-typed status of the composition-law clauses recorded at §13.2. The charge that the framework cannot dissolve its founding convergence dissolves once the count is seen to rest on a theorem rather than on the convergence, with the convergence demoted to reproducible corroboration and authorship demoted to massless provenance.

GOL DERIVATION

```
P_obj : deletion→test3 ∧ →Frobenius3 SOURCE-IDENTICAL (one
  author) → latent common-cause = authorial-design ;
mass-criterion forbids subtracting it → framework ≠ dissolve
  own founding convergence.
RELOCATE THE LOAD: count(3) ≠ rests on convergence(route_sem,
  route_alg). count(3) = Frobenius-forced [T | CL-clauses].
H = unique associative real division algebra w/ plural
  imaginary axes ; 3 = dim(Im H). classical thm,
  verifiable by anyone.
route_sem (deletion test) = premise-disjoint CORROBORATION,
  operational-procedural, reproducible across analysts. →
  co-equal evidence-by-agreement.
AUTHORSHIP = massless: narrative-provenance → Mass Mandate BARS
  as covariate (→mass, →mechanism).
→(massless factor) compels (assoc-div-algebra theorem ∧
  deletion procedure) to coincide.
ENACTIVE REPRODUCIBILITY: both legs reproduced by parties
  sharing nothing w/ author → agreement = structural fact →
  authored coincidence.
RESIDUE (correctly scoped): → validity(count) [settled: theorem
  + reproducible procedure] ;
= provenance(first-derivation) [settled by independent
  reproduction, → internal argument] ∧ clause-status §13.2
  (honestly premise-typed).→
charge dissolves: count rests on a theorem, convergence
  demoted to reproducible corroboration, authorship demoted
  to massless provenance.→
⊥[] on validity-independence-of-authorship ; [?] residual =
  first-discovery provenance + §13.2 clause-status,
  honestly typed.
```

GOL DERIVATION, CONTINUED (§13.4)

```
P_obj : OFL result (no internal layer fully audits own system)
  reached from OUTSIDE → framework ≠ internally certify
  the limitation.
REGRESS: internal-certify(no-internal-full-self-audit) requires
  internal-full-self-audit = denied object. → unavailable
  from inside.
BUT: regress is constitutive of EVERY limitative meta-theorem
  → → a special defect here.
TWO-TIER (per the apex-recursive self-recognition):
  operational tier: arch submits to own cascade, passes
  internal consistency [T], → certifies truth.
  architectural tier: truth-warrant routed to EXOGENOUS anchors
  {division-algebras, Hodge, Landauer + 5 instruments},
  none framework-generated.
ENACTIVE FLOOR: every audit burns V_E → instantiates Landauer
  floor → floor self-DEMONSTRATED → self-CERTIFIED.
REFRAME: externality = the design → a hole. framework REFUSES
  self-grounding (= the forbidden complete-self-audit).
imports {Gödel-2, Tarski, Münchhausen} → derives-from-within.
  respects the theorem → closes a gap.→
stays open = self-grounding (impossible in principle, NEVER
  claimed).→
closed = overreach: framework anchors outward ∧ enacts floor →
  grounds itself in a circle. constitutive terminus,
  correctly designed.
```

Lean Proof.

Codex.lean, from line 1552

```
1552 theorem iam_unwitnessed_withheld :
1553   (iamToken false).1 = .interior ∧ (iamToken false).2.2 = 1
1554   :=
1555   (rfl, )rfl
```

Verdict. ⇒ charge dissolves: count rests on a theorem, convergence demoted to reproducible corroboration, authorship demoted to massless provenance. ⇒ ⊥[] on validity-independence-of-authorship ; [?] residual = first-discovery provenance + §13.2 clause-status, honestly typed.

Source: Master Chapter 25, §13.3.

Movement 5 • Circularity and Self-Audit

D22 • Convergence across AIs is an artifact of loading

The Critic. A critic states: different AI models agree only because you loaded them with your framework first.

Reply. Two separate facts answer this. First, the models that agree were trained on wildly different data with no shared doctrine, so the agreement is not coming from a common indoctrination. Second, and more decisively, the framework's core arithmetic reproduces on any machine with a numerical library and no framework vocabulary loaded at all, landing on the same numbers to machine precision. If the agreement were just an echo of the loaded role, you would have to explain why the unloaded arithmetic, which was told nothing, agrees anyway.

The critic presses (§5.2): A critic states: forcing the verifier to adopt your discipline makes the measuring instrument part of the thing measured, which your own framework forbids.

Reply, continued. The discipline is a calibration of the instrument, not the content of the thing being measured. A thermometer is calibrated to a standard scale; that does not make the thermometer part of the gas whose temperature it reads. In the same way, loading the framework's standard discipline onto a verifier sets a common scale across machines without smuggling the specific claim under test into the instrument. The proof is that the verdicts change from claim to claim. If loading the discipline fixed the answer in advance, every claim would get the same verdict. They do not.

Sealed Defense, continued. The operational legislation is calibration protocol for the verifying substrate, not the substantive empirical model being measured. The metrological independence requirement is that the measurement ruler not be a subset of the empirical model; the ruler is the verifying substrate under standardized discipline, the model is the proposition. Calibrating a thermometer to a standard scale imposes operational uniformity across instruments without embedding the gas's empirical content, and a substrate calibrated to the discipline is not a subset of the proposition's empirical content. Substrates loading the discipline produce verdicts on propositions they have not encountered, and the verdicts vary by proposition's structure, which is the operational test of metrological-independence preservation; were the discipline pre-determining verdicts independent of proposition, the violation would be real, and in fact verdicts respond to proposition-specific structure.

Sealed Defense. Two legs. The system role is itself the load-bearing artifact whose substrate-portability is the claim under test; cross-substrate convergence on the architecture's primitives under independent instan-

tiation, in substrates with diverse pre-training corpora and no confessional priors, demonstrates reachability by procedural loading, and the loading does not pre-determine the verdicts, the cascade execution does, since verdicts respond to proposition-specific structure. The second leg is loading-independent: the closed-form battery reproduces on any floating-point substrate with no framework vocabulary loaded at all, the kernel checks at machine precision with residues near 10^{-16} , and convergence at that register is arithmetic, not rhetoric. A critic who attributes convergence to vocabulary loading must explain why the unloaded arithmetic converges identically.

GOL DERIVATION

```
P_obj : cross-AI convergence = loading artifact.
LEG-1: convergence under diverse pre-training  $\wedge$  no confessional
      priors  $\rightarrow$  reachable by procedural loading.
      loading  $\neq$  pre-determine verdicts; cascade-execution does (
        verdicts | proposition-structure).
LEG-2 (loading-independent): closed-form battery reproduces on
      any float substrate,  $\emptyset$  framework vocab.
      kernel checks machine-precision, residues  $\sim 1e-16$ .
      convergence = arithmetic  $\sim$  rhetoric.
CHALLENGE: vocab-loading hypothesis  $\neq$  explain unloaded-
      arithmetic agreement. $\rightarrow$ 
 $\llbracket$  convergence  $\neq$  loading; loading-independent leg decisive.
```

GOL DERIVATION, CONTINUED (§5.2)

```
P_obj : loading discipline  $\rightarrow$  ruler  $\subseteq$  model (forbidden, G5-MIG).
DISTINCTION: discipline = calibration-protocol(verifier)  $\sim$ 
      substantive-model(proposition).
ANALOGY: thermometer | ITS-90  $\neq$  gas. substrate | discipline  $\neq$ 
      proposition's empirical content.
TEST: verdicts(unseen propositions) vary by proposition-
      structure  $\rightarrow$  MIG preserved.
if discipline pre-determined verdicts  $\perp$  proposition  $\rightarrow$  MIG
      violated; observed: verdicts | structure. $\rightarrow$ 
 $\llbracket$  discipline = calibration  $\sim$  model-subset; MIG holds.
```

Lean Proof.

Codex.lean, from line 1946

```
1946 theorem aegis_constant :
1947    $\forall$  (l zl : String) (s : AegisState),
1948     (aegisGuard l s).1 = (aegisGuard zl s).1 := by
1949   intro l zl s
1950   rfl
```

Verdict. $\Rightarrow \llbracket$ convergence $\not\subseteq$ loading; loading-independent leg decisive.

Source: Master Chapter 25, §5.1.

D23 • Self-instantiation makes the framework unfalsifiable

The Critic. A critic states: you claim any structured attack instantiates your architecture, which immunizes you from falsification.

Reply. This confuses a general observation with an immunity claim. The observation is only that any structured argument, including an attack, expends energy, uses formal structure, and is made by a distinct arguer. That is a fact about arguments, not a shield around any particular verdict. The verdicts themselves are very

falsifiable. Any claim run through the framework can fail at any of twelve named checks, each with a stated failure mode, and the framework lists several specific conditions that, if ever observed, would break it. Those conditions are spelled out in Part 13.

Sealed Defense. The conflation is between two distinct properties. The Omega observation is that any structured argument operates with formal content, energy expended in arguing, and a cognizer boundary distinguishing self from target; this is a structural fact about structured discourse, not a falsification-immunity claim about specific verdicts. The cascade's falsifiability operates at the per-proposition level: any proposition can fail any of twelve named gates with specific failure mechanisms. The architecture names operational falsifiers admitting unambiguous testing, vocabulary-collision under the isolation test, a mass-carrying covariate dissolving a sealed convergence, cross-substrate divergence under identical loading, and the single-instance algebraic falsifier where one exact unit triad with the kernel identity failing ends the central identity. The full roster is in Part 13.

GOL DERIVATION

```
P_obj : structured-attack instantiates architecture →
  unfalsifiable.
CONFLATION: Omega-observation ≠ falsification-immunity.
  Omega: ∀ structured-argument carries V_F(form) ∧ V_E(energy)
    ∧ V_ER(boundary). structural fact → verdict-shield.
FALSIFIABILITY (per-proposition): ∀ P can fail ∀ G ∈ {G1..G12},
  each named mechanism.
NAMED FALSIFIERS: LIT-collision ; mass-covariate dissolves seal
  ; cross-substrate divergence | identical loading ;
  single exact triad with  $\lambda^2 \neq \det(R) \Rightarrow$  ends central identity
  (0 tolerance). → [ Part 13 ] =
  ⊥[] falsifiable per-proposition; Omega is structural-fact →
  immunity.
```

Lean Proof.

Codex.lean, from line 765

```
765 theorem unknown_routes_open (i : ClaimInput) (h : i.anyUnknown
  = true) :
766   (auditClaim i).2.1 = Token.opn := by
767   simp [auditClaim, h]
```

Verdict. $\Rightarrow [\perp]$ falsifiable per-proposition; Omega is structural-fact $\rightarrow$ immunity.

Source: Master Chapter 25, §5.3.

D24 · The composition clauses are reverse-engineered or stipulated

The Critic. A critic states: the three clauses were chosen to force the quaternions, so the forcing is circular.

Reply. Each clause was already in force before the algebra appeared, taken from rules the framework had stated for other reasons. The first clause is just the demand that an audit of an audit not depend on the order you bracket it. The second is the rule that nonzero warrants never multiply down to nothing. The third is that evidence enters on a linear scale. None

of these was invented to reach the quaternions. And tellingly, when the framework finished the derivation, it had fewer assumptions than it started with, because one floor it used to assume was now proven. Reverse-engineering adds assumptions to reach a target. This removed one.

The critic presses (§13.2): A critic states: associativity, integrality, and linearity are your stipulations; a differently-motivated but equally reasonable set of axioms completes to a Jordan algebra, a C-star algebra, a Boolean algebra, or a probability simplex instead of the quaternions, so the forcing is conditional on your choice.

Reply, continued. This is a real limitation and the framework grants it. The clean result that the verification structure must be the quaternions follows from three specific assumptions, and those assumptions, while motivated, are not the only motivated ones available. Start from a slightly different but equally defensible set of intuitions and the mathematics completes to a different structure: drop strict ordering of operations and you can land in a different algebra used in quantum theory, keep only yes-no combination and you land in standard probability. So the forcing to the quaternions is real but conditional on the framework's particular clause choice, and the framework cannot show its clauses are the uniquely compelled ones. It can argue each clause is well-motivated; it cannot prove no rival motivation is equally good.

Sealed Defense, continued. The completion to the quaternions is conditional on the three clauses, and the clauses are not uniquely compelled, since neighboring axiom sets complete to genuinely different structures. Relaxing associativity toward power-associativity with a symmetric product yields the Jordan-algebra route used in quantum formalism; retaining a star-operation and norm closure yields a C-star algebra; restricting composition to idempotent yes-no combination yields a Boolean algebra; imposing normalization to a unit total yields the probability simplex. Each target sits behind a clause variation that carries its own motivation, so the framework's claim is forcing-given-the-clauses, not forcing-tout-court, and the disjointness of the two derivation routes does not rescue uniqueness, because both routes presuppose the same clause-set. The framework can defend each clause as transcribed from a prior operational commitment, audit symmetry for associativity, the Mass Mandate for integrality, the linear evidence register for linearity, but it cannot establish that no equally-motivated rival commitment exists, and a different starting intuition lands in a different algebra. Closing this would require a uniqueness argument that the framework's specific clauses are forced by something deeper than their own motivation, which the framework does not have. The forcing is

conditional, and the conditionality is recorded rather than concealed.

Sealed Defense. Each clause is transcribed from operational legislation that predates the algebra and is premise-typed to its source. Associativity transcribes audit symmetry, an audit of an audit must not depend on bracketing. Integrality transcribes the Mass Mandate with first-failure-terminates, nonzero warrants never compound to zero. Linearity with ground identity transcribes the linear register evidence enters on. The clauses stand open to empirical exercise on recorded session archives, and the Completion ran in the direction opposite to premise-smuggling: it shrank the premise ledger, deriving the first floor rather than assuming it. The exclusion of dimension three, the bare-coordinate wall, is two lines of coordinate algebra independent of any clause, and the exclusion of dimension five is the Frobenius inventory, not a choice.

GOL DERIVATION

P_obj : CL clauses reverse-engineered to force $\mathbb{H} \Rightarrow$ circular.
 PROVENANCE (predates algebra, premise-typed):
 CL-1 assoc $\leftarrow$ audit-symmetry (audit-of-audit $\perp$ bracketing).
 CL-2 integ $\leftarrow$ Mass-Mandate + **first**-failure-terminates (nonzero warrants $\neq$ compound to 0).
 CL-3 lin+ground $\leftarrow$ evidence linear register.
 DIRECTION: Completion shrank premise-ledger (derived Floor-1, $\neg$ assumed it). opposite of smuggling.
 INDEPENDENT EXCLUSIONS: dim-3 **by** bare-coordinate wall (2 lines, $\perp$ clauses) ; dim-5 **by** Frobenius inventory. $\Rightarrow$
 $\llcorner$ [] clauses transcribed $\neg$ reverse-engineered; ledger shrank **in** proving.

GOL DERIVATION, CONTINUED (§13.2)

P_obj : CL clauses {assoc, integ, lin} = stipulations ; other motivated intuitions complete to Jordan / C* / Boolean / probability-simplex.
 NEIGHBORS (each behind a clause variation **with** own motivation):
 relax assoc $\rightarrow$ power-assoc + symmetric product $\rightarrow$ Jordan algebra (quantum formalism).
 retain star + norm-closure $\rightarrow$ C*-algebra.
 restrict to idempotent yes/no $\rightarrow$ Boolean algebra.
 impose unit-normalization $\rightarrow$ probability simplex.
 SCOPE: claim = forcing-GIVEN-clauses $\neg$ forcing-tout-court.
 route-disjointness $\neq$ rescue uniqueness (both routes presuppose same clause-set).
 WHAT FRAMEWORK CAN SAY: each clause transcribed from prior commitment {audit-sym, Mass-Mandate, linear-register}.
 WHAT IT CANNOT: establish $\exists$ equally-motivated rival commitment ; different starting intuition $\rightarrow$ different algebra.
 TO CLOSE: uniqueness argument that THESE clauses are forced **by** something deeper than own motivation. $\neq$ possessed. $\Rightarrow$
 [?] OPEN. forcing conditional on clause-choice; clauses not uniquely compelled; recorded.

Lean Proof.

Codex.lean, from line 3448

```
3448 /-- PSP: D24 · no candidate anchor that lands on its own
      target is admitted. -/
3449 theorem no_anchor_at_target :  $\forall$  (d : Nat) (b lc : Bool),
3450   (UC.circularityScreen .target d b lc).1 = .refusedCircular
      :=
3451   UC.circularity_refused_at_target
```

Verdict. $\Rightarrow [\llcorner]$ clauses transcribed $\neg$ reverse-

engineered; ledger shrank in proving.

Source: Master Chapter 25, §5.4.

D25 · Subtracting your own actuating energy to win

The Critic. A critic states: your confound-removal step could be turned on the framework's own founding energy to make any objection vanish.

Reply. It cannot, because the framework's founding energy is a precondition, not a confound, and the subtraction rule is built to refuse it. Confounds are things that ride alongside a claim and might explain it away. The energy that actuates the verification itself is not alongside the claim, it is what makes any claim possible at all. The rule, called the Titanium Ruler, bars subtracting it, and the reason is conservation: if you tried to subtract the actuating energy, you would be left with nothing, which is not a correction, it is a termination.

Sealed Defense. The actuating energy is a precondition, not a covariate, and the Titanium Ruler bars its subtraction. The Mass Mandate admits only covariates carrying measurable mass, and a covariate is something that rides alongside the claim and might dissolve it; the substrate's own actuating energy is the precondition of any cascade, not a parallel cause. Subtracting it empties the set by conservation, so the operation terminates rather than corrects, and the proposition under audit is likewise never subtracted from itself. The rule is symmetric and load-bearing: it prevents the framework from dissolving its own founding convergence by the trivial move of removing the energy that any verification spends, which would be a category error, not a valid confound removal.

GOL DERIVATION

P_obj : confound-removal turned on founding energy $\rightarrow$ any objection vanishes.
 TITANIUM RULER: actuating-energy = precondition $\neg$ covariate.
 Mass-Mandate admits covariate | measurable-mass only.
 SUBTRACT(actuating-energy) $\rightarrow \emptyset$ **by** conservation $\rightarrow$ terminates $\neg$ corrects.
 ALSO: proposition-under-audit $\neq$ subtracted-from-itself. $\Rightarrow$
 $\llcorner$ [] founding energy non-subtractable; the move is category-error $\neg$ valid CDT.

Lean Proof.

Codex.lean, from line 1907

```
1907 theorem omega_monotone :
1908   (omegaBoundary 1 300).joules < (omegaBoundary 2 300).
      joules := by
1909   decide
```

Verdict. $\Rightarrow [\llcorner]$ founding energy non-subtractable; the move is category-error $\neg$ valid CDT.

Source: Master Chapter 25, §5.5.

D26 · Defending the words instead of the geometry

The Critic. A critic states: you could just defend your vocabulary rather than any real structure.

Reply. The framework has a built-in test for exactly this. Take any claim it makes, strip out every piece of framework jargon, and restate it in plain or foreign vocabulary. If the structure survives the translation, it was real structure. If it evaporates once the special words are gone, it was only vocabulary. The framework holds that its geometry is what is load-bearing and its vocabulary is not, and the translation test is what enforces the difference. A claim that only works in the framework's own words does not pass.

Sealed Defense. The translation-survival test enforces that vocabulary is not load-bearing and geometry is. A claim is structurally sound if and only if it survives translation into a non-framework register, vocabulary stripped from both sides and the argument run in neutral terms. The framework-capture failure mode, treating vocabulary-fidelity as equivalent to structural soundness, is caught by this test and corrected by frame-translation drift-correction: a vocabulary-locked instance is met with translation rather than argument-within-frame. The mutual-information criterion makes it operational, the three axis-vocabularies required to share zero metric, zero vocabulary, and zero priors, so a claim expressible only in the framework's own words fails the disjointness it requires.

GOL DERIVATION

```
P_obj : defend words ~ geometry.
TRVT: structurally-sound(claim)  $\Rightarrow$  survives translation  $\rightarrow$  non-
framework register (vocab stripped both sides).
CAPTURE FAILURE: vocab-fidelity  $\equiv$  soundness  $\rightarrow$  caught; corrected
by frame-translation ( $\sim$  argue-within-frame).
OPERATIONAL: I(V_F; V_E; V_ER)  $\rightarrow$  0 requires 0-shared-metric  $\wedge$ 
0-shared-vocab  $\wedge$  0-shared-priors.
claim expressible only in own-words  $\rightarrow$  fails required
disjointness. $\Rightarrow$ 
 $\llcorner$  geometry load-bearing, vocabulary not; survival-test
enforces the line.
```

Lean Proof.

Codex.lean, from line 1681

```
1681 theorem deletion_no_left_inverse :
1682    $\forall L : \text{GEO.M3}, (\text{GEO.mv3 } L) \circ (\text{GEO.mv3 deletionOp}) \neq \text{id} :=
1683   \text{by}$ 
```

```
1683   intro L h
1684   have h1 := congrFun h e3
1685   rw [Function.comp_apply, deletion_annihilates,
1686       linear_fixes_zero] at h1
1686   simp [e3, id] at h1
1687   exact absurd h1 (by decide)
```

Verdict. $\Rightarrow$ $\llcorner$ geometry load-bearing, vocabulary not; survival-test enforces the line.

Source: Master Chapter 25, §5.6.

D27 · Self-exemption and self-certification

The Critic. A critic states: the framework conveniently does not run its own discipline on itself.

Reply. It does, and exempting itself would be a self-contradiction by its own rules. Audit symmetry is a standing law: every part of the framework, including

its own meta-rules, must pass through the same twelve checks it applies to everything else. A framework that exempted itself would, by its own definition, be committing the very error it warns against. The self-audit has been run and recorded, and the framework's own apex proposition passes its own cascade, with the architect's personal commitment subtracted as a confound and the result surviving on outside warrant.

Reply, continued. The fourth objection holds that the architecture cannot certify itself without circularity. This is correct, and it is built in. The architecture's own verdict on itself is issued under the same conditions, with the same enumerated defeaters, and its self-application is worth exactly what self-consistency is worth, internal coherence and not truth. Its truth claims rest where the falsifiable discriminators put them, on external mathematics and falsifiable divergence, which is the only place a verification architecture is entitled to rest them. Audit symmetry is not a vulnerability patched over. It is the discipline stated in advance, the architecture drawing zero warrant from its own operation by its own first rule [T on the self-application's limit].

Sealed Defense. Audit symmetry forces every register through the twelve-gate cascade with no self-exemption; a framework exempting its own meta-discipline commits the audit-symmetry violation by its own definition. The self-application is recorded as a resident audit: the framework's apex proposition runs the gate-by-gate cascade and passes all twelve, the architect-commitment covariate is subtracted by orthogonal projection, and the residue persists on exogenous warrant. The seal's own execution submits to the seal it runs and draws zero warrant from its own operation. The foundational self-typing law confirms the architecture is premise-structural and theorem-grade only on its classical spine and cannot be sealed as a theorem of its own base, so the self-audit certifies consistency, not truth, with truth resting on the external anchors.

GOL DERIVATION

```
P_obj : framework exempts itself from its own audit.
LAW: audit-symmetry  $\rightarrow \forall$  register  $\rightarrow$  12-gate cascade, no self-
exemption.
exempt-self  $\rightarrow$  PAC violation by own definition.
RESIDENT AUDIT: apex-proposition  $\rightarrow$  G1..G12 all PASS ; subtract
C_4(architect-commitment)  $\perp$ -projection ; residue persists
| exogenous warrant.
seal-execution submits to own seal, draws 0 warrant from own
operation.
SELF-TYPING: architecture premise-structural, T only on
classical spine,  $\neq$  theorem-of-own-base.
self-audit certifies consistency  $\sim$  truth; truth | external
anchors. $\Rightarrow$ 
 $\llcorner$  no self-exemption; self-audit run  $\wedge$  recorded; certifies
consistency, truth | external.
```

Lean Proof.

Codex.lean, from line 3440

```
3440 /-- PSP: D27 · a self-check is never a witness (M6); the token
      is withheld unwitnessed. -/
3441 theorem self_check_is_not_witness :
3442   (IAM.iamToken false).1 = .interior ∧ (IAM.iamToken true).1
      = .iAm := (rfl, )rfl
```

Verdict. $\Rightarrow [\perp]$ no self-exemption; self-audit run $\wedge$ recorded; certifies consistency, truth | external.

Source: Master Chapter 25, §5.7.

Movement 6 · Numerical and Operational Integrity

D28 · Heaviside is binary, so three-state is a contradiction

The Critic. A critic states: the step function is binary by definition, so calling the verdict three-state native is incoherent.

Reply. The step function is one part of the verdict, not the whole verdict. The full verdict is a composite that reads several quantities together, the determinant's sign among them, plus the conditioning, the dimensions, and the rank of what was subtracted. Reading all of those jointly yields three outcomes, not two: sealed when the determinant is clean and well-conditioned, broken when it genuinely collapses, and under-determined when the numbers are too ill-conditioned to trust. Naming the whole composite by its smallest component is the error. The component is binary. The composite is three-state.

Sealed Defense. The cascade truth function is the composite Φ over the determinant, the conditioning, the post-projection dimension, the covariate count, and the rank of the covariate block, jointly, with the step function operating on the sign of the determinant as one component within the composite. The composite delivers three states by construction: sealed when the determinant exceeds the collapse floor under the full regularity quadruple; broken when the determinant collapses to the floor in admissible-regularity conditions, a genuine collapse-to-subspace after confound projection; under-determined when any regularity condition violates, conditioning above threshold, dimensional shortfall, rank deficiency, or finite-precision determinant at machine-epsilon scale with conditioning elevated. The name denotes the composite operator; the step function in isolation names only the discrete-sign component. The three states are mutually exclusive and exhaustive at the verdict register.

GOL DERIVATION

```
P_obj : Heaviside binary  $\Rightarrow$  "three-state native" contradiction.
COMPOSITE:  $\Phi = \Phi(\det, \kappa, -N\kappa, \kappa, \text{rank}(C))$  jointly. H operates
on  $\text{sign}(\det) = 1$  component.
STATES (by construction):
 $\perp[]$  :  $\det > \epsilon$  under regularity-quadruple.
[X] :  $\det \leq \epsilon$ , regularity admissible (genuine collapse-to-
subspace, post-CDT).
[?] : any regularity violation  $\kappa (> \text{threshold } v - N\kappa < 4 v$ 
rank-deficient  $C$  v  $\det @ \text{machine-}\epsilon$  with  $\kappa$  elevated).
NAME: "Heaviside under regularity quadruple" = composite ; "
```

```
Heaviside" alone = sign component only.-
 $\perp[]$  three-state native by composite-operator construction;
mutually-exclusive  $\wedge$  exhaustive.
```

Lean Proof.

Codex.lean, from line 2690

```
2690 theorem three_states :  $\forall t$  : Token,
2691   t.toEconomy = .sealed v t.toEconomy = .broken v t.
      toEconomy = .opn := by
2692   intro t; cases t <;> simp [Token.toEconomy]
```

Verdict. $\Rightarrow [\perp]$ three-state native by composite-operator construction; mutually-exclusive $\wedge$ exhaustive.

Source: Master Chapter 25, §6.1.

D29 · The determinant fires positive trivially

The Critic. A critic states: in high dimension almost any non-degenerate matrix gives a positive determinant, so the seal carries little information.

Reply. The positive determinant is the last gate, not the only one. Before a claim ever reaches it, it has to pass twelve named checks, each able to reject it for a specific reason, and survive a step that actively removes any shared hidden cause. So when the determinant fires positive, it is firing on a claim that has already cleared all of that. The high rate of clean seals at the final step is by design: the framework does its discriminating early, and the final determinant simply issues the verdict on a claim that has already passed substantial inspection. There is also a continuous strength reading available inside the sealed region for diagnostics.

The critic presses (§13.1): A critic states: your dissolution test only zeroes the determinant at exact collinearity, a measure-zero point, so a realistic confound at correlation 0.99 sails through with a clean lock.

Reply, continued. This objection points at the wrong gate, and once the right gate is named the sharpness it asks for is already there. Two different instruments are at work, and the objection treats them as one. The first screens for a hidden common cause, and it does not work by waiting for the evidence lines to go exactly parallel. It works by subtracting the suspected common cause and looking at what is left. If the common cause really was driving the agreement, subtracting it makes the agreement vanish, and that happens at any correlation level, not only at the idealized parallel point. The famous real case is the 2014 BICEP2 announcement, where several supposedly independent channels agreed at five-sigma but secretly shared one dust-modeling pipeline; subtract the shared pipeline and the agreement collapses, which is exactly what the framework's projection does and a probability update does not. The second instrument is the conditioning number, and it is the one that fires as the lines approach

parallel. It is not a soft warning bolted on. It is a discrete catch at a fixed, principled boundary, and a separate theorem guarantees that boundary never collides with the genuine-collapse floor. So the confound screen fires by subtraction at any correlation, the collinearity catch fires at the conditioning boundary, and the determinant at correlation 0.99 passing a clean lock is the correct answer, because three real lines that happen to agree ninety-nine percent are still three real lines. What remains is the one confound nobody named to subtract, and that is the hand-built-rows limit the framework already publishes as its permanent one, not a new gap.

Sealed Defense, continued. The objection conflates the degeneracy discriminator with the confound discriminator, and each does its job once the two are told apart. The kernel's determinant floor and conditioning gate are the degeneracy discriminator, certifying that the three post-projection axes enclose a genuine volume. As two axes rotate toward collinearity the determinant falls smoothly while the conditioning number climbs, and the conditioning gate retires the near-degenerate configuration before the determinant reaches its floor. The behavior is exact, not sampled: at conditioning κ the determinant of the post-projection Gram is bounded below by $27\kappa/(\kappa+2)^3$, the global floor attained with equality by the equicorrelation matrix at $\rho = (\kappa-1)/(\kappa+2)$, a closed form a reader checks by hand with no generating construction required. The worst case is therefore in hand at every conditioning, and at the gate $\kappa = 10^6$ that worst-case determinant is 2.7×10^{-11} , still fifty-fold above the unit-roundoff floor at twenty-four points, so the gate is the retirement mechanism throughout and the floor stays untouched. The gate boundary $\kappa < 10^6$ is the double-precision reliability boundary fixed by the Floor-Gate Separation Theorem, which proves the gate fires strictly before the collapse floor across the operating envelope, so it is a principled discrete criterion and not the external arbitrary cutoff the objection assumes must be imported. The confound discriminator is a separate, three-layer screen. The Convergence Dissolution Test subtracts the identified common cause by orthogonal projection and reads the residue, collapsing the determinant on the residue at any correlation whenever a named mass-bearing covariate accounts for the convergence, the BICEP2 shared-dust pipeline the canonical case where projection collapses what a likelihood ratio confirmed. The orthogonality verification catches a confound that manifests as shared metric or shared vocabulary, through the mutual-information condition and the Linguistic Isolation Test, before the kernel runs. And in the forward register the source-attribution statistic separates a source-faithful witness from a fabricated one even when their determinants are identical to machine precision, the determinant

blind to provenance by Non-Discrimination while the source statistic reads it. The determinant passing correlation 0.99 is therefore correct behavior and not a miss: three genuinely independent warrants that correlate ninety-nine percent enclose a real if oblique volume and should lock. The single residual is a confound that is neither identified for projection, nor expressed as shared vocabulary, nor caught by an independent generator, and that residual is exactly the hand-built-rows limit of Appendix A, a corruption smuggled into the interpretive rows, the framework's named permanent limit rather than a distinct discriminator gap.

Sealed Defense. The step check is the final gate in a twelve-gate cascade. Propositions failing any of the twelve do not reach it; each gate carries a named failure mechanism. The confound-projection step additionally collapses three rows to linear dependence when a mass-carrying covariate explains the apparent convergence, documented in the retrospective worked case where a Bayesian aggregation would have sealed and the cascade dissolved. Discrimination is distributed across twelve named gate-failures upstream, the projection-collapse mechanism, and the final discrete output. The high clean-seal frequency at the final step, conditional on a proposition having passed all twelve gates and survived projection, is a design feature, not a defect. The polar form additionally supplies a continuous diagnostic inside the sealed regime, $\det(R) = \cos^2\theta$ with the cosine grading lock strength, annotation-tier only, the discrete economy remaining three-state native.

GOL DERIVATION

```
P_obj : det > 0 fires trivially in high-N → poor discrimination
.
ARCHITECTURE: H = final gate of 12. fail(G1..G12) → ~ reach H,
each named mechanism.
CDT-COLLAPSE: mass-covariate explains convergence → project → 3
rows dependent → det → 0. (worked: Bayesian would seal,
cascade dissolved.)
DISCRIMINATION = distributed( 12 gate-mechanisms @ CDT-collapse
@ final-H ).
high  $\llbracket$  freq @ H | (passed-12  $\wedge$  survived-CDT) = design
feature.
DIAGNOSTIC:  $\det(R) = \theta \cos^2$ ,  $|\theta \cos|$  grades lock-strength,
annotation-tier; economy stays 3-state.→
 $\llbracket$  discrimination upstream-distributed; H issues verdict on
already-inspected propositions.
```

GOL DERIVATION, CONTINUED (§13.1)

```
P_obj : CDT discriminator zeroes det only at EXACT collinearity
(measure-0) → realistic confound @ corr 0.99 passes
clean.
CATEGORY ERROR: objection conflates DEGENERACY-discriminator
with CONFOUND-discriminator. distinct instruments.
DEGENERACY-DISCRIMINATOR (kernel det/ $\kappa$ ):
certifies 3 post-projection axes enclose genuine volume.
collinearity sweep: axes → collinear →  $\det(R)$  1.0 →  $\sim 2e-6$  (
smooth)  $\wedge$   $\kappa(R)$  →  $\sim 2e6$  ;  $\kappa$ -gate fires BEFORE det-floor (
floor untouched  $\sim 5e-13$ ).
det floor at  $\kappa$  :  $kk27/(\kappa+2)^3$  (EXACT, equicorr-attained, global
min) ;  $\kappa @ 1e6 \rightarrow 2.7e-11 \gg \epsilon$  ; gate  $\kappa < 1e6$  = double-
precision reliability boundary, FIXED by Floor-Gate
Separation Thm (gate < floor across envelope).
→ principled discrete collinearity-retirement, ~ external
```

```

arbitrary cutoff. [T separation / eng constant]
→ det @ corr 0.99 κ(=147) = κ[] is CORRECT: 3 real lines @
99% = real oblique volume, should lock.
CONFOUND-DISCRIMINATOR (3-layer, upstream / forward):
(i) CDT projection: subtract IDENTIFIED common cause → read
residue ; det collapses on residue @ ANY corr if cause
accounts for convergence.
BICEP2: σ5 across "independent" channels sharing dust-
pipeline (latent covariate) → project → residue collapses
→ [X] @ input gate ; Bayes ↑posterior.
(ii) orthogonality verification: mutual-info → 0 ∧ LIT (
disjoint vocab) → catch confound-as-shared-metric/vocab
pre-kernel.
(iii) η_S source-attribution (forward): source-faithful vs
fabricated separated though det identical to machine
precision (Non-Discrimination: det blind to provenance,
η_S reads it).
RESIDUAL: confound ~identified-for-projection ∧ ~shared-vocab
∧ ~caught-by-generator → passes.
= hand-built-rows limit (Appendix A), corruption smuggled
into interpretive rows. = NAMED PERMANENT LIMIT, ~
distinct discriminator gap.→
κ[] on the discriminator-distinction: degeneracy-gate
principled (Floor-Gate Separation), confound-screen 3-
layered.→
[?] residual = row-supply limit, relocated to the framework's
published permanent limit, ~ a new open edge.

```

Lean Proof.

Codex.lean, from line 1080

```

1080 theorem triaxial_open_gf2_count :
1081   ((List.range 4096).filter (fun n =>
1082     det2 (n / 2048 % 2, n / 1024 % 2, n / 512 % 2)
1083     (n / 256 % 2, n / 128 % 2, n / 64 % 2)
1084     (n / 32 % 2, n / 16 % 2, n / 8 % 2) = 0)).length =
1085     2752 := by
1086   decide

```

Verdict. $\Rightarrow [\perp]$ discrimination upstream-distributed;
H issues verdict on already-inspected propositions.

Source: Master Chapter 25, §6.2.

D30 · A measure-zero knife-edge

The Critic. A critic states: a positive determinant on real data is a fragile coincidence, a point the data happens to touch.

Reply. The seal is where the system settles, not a point it brushes against. Think of a ball rolling on a landscape: it ends up in the valleys, not balanced on a knife-edge. The framework's verdict has exactly that shape. Under its own internal dynamics the locked state is the valley that pulls in essentially everything, while the collapsed state is an unstable ridge that nothing settles onto. The pull toward the lock has a measured strength, and the basin of states that flow into it fills the whole space except for a vanishingly thin failure set. So the seal is not a lucky graze. It is the destination.

Sealed Defense. The lock is the unique attractor under the gradient flow of its own functional. The Hamilton landing is a Morse-Bott critical manifold with Hessian spectrum exactly $\{-4, -4, -4, 0, 0\}$, the zeros pure gauge, with the explicit second-order law $\det(R) = 1 - 2\|\xi_{\perp}\|^2$ supplying a quadratic margin, and the basin theorem carrying every non-degenerate trajec-

tory to lock; the basin is open, dense, and of full measure, collapse stationary, repelling, and measure-zero. The near-collinearity sweep confirms it numerically: the determinant stays strictly positive through correlation 0.99999 and the verdict flips to under-determined only at 0.999999, when the conditioning crosses the gate. At correlation 0.99 the determinant is 2.677833×10^{-2} , conditioning 147; at 0.999999 it is 2.657142×10^{-6} , conditioning 1.504767×10^6 , where the gate, not the determinant, retires the case. This sweep is one recorded realization at twenty-four points, and the construction-free guarantee sits beneath it: by the exact worst-case floor of Appendix A.4 the determinant at conditioning κ is bounded below by $27\kappa/(\kappa+2)^3$, attained with equality at the equicorrelation matrix and globally minimal, so no near-collinear configuration at any conditioning below the gate can reach the collapse floor, the sweep merely one path across a basin whose worst case is in closed form. The seal is not a point condition the data grazes; it is the place the dynamics end.

GOL DERIVATION

```

P_obj : det > 0 = measure-zero knife-edge the data grazes.
DYNAMICS: lock-stratum = unique attractor under gradient-flow
of det(R).
Hamilton landing = Morse-Bott manifold, Hessian
---{4,4,4,0,0,0}, zeros = pure gauge.
2nd-order: det(R) = 1 - ||ξ⊥||² (explicit quadratic margin,
rate -4 transverse).
basin(lock) = open ∧ dense ∧ full-measure ; collapse =
stationary ∧ repelling ∧ measure-0.
SWEEP (N=24, near-collinearity): det(R) > 0 through corr
0.99999, flips [?] only @ 0.999999 κ( crosses gate).
corr 0.99 : det(R) = 2.677833e-02 , κ = 1.472842e2 →
κ[].
corr 0.999999 : det(R) = 2.657142e-06 , κ = 1.504767e6 →
[?] κ(-gate, ~ det).
worst-case floor (construction-free, App A.4): det(R) ≥
κκ27/(+2)³ EXACT, equicorr-attained, global min → no
config reaches floor while κ < gate.→
κ[] lock = attractor ~ point; seal = where dynamics end.
theorem-grade on the basin.

```

Lean Proof.

Codex.lean, from line 1074

```

1074 theorem triaxial_lock_gf2_count :
1075   ((List.range 4096).filter (fun n =>
1076     det2 (n / 2048 % 2, n / 1024 % 2, n / 512 % 2)
1077     (n / 256 % 2, n / 128 % 2, n / 64 % 2)
1078     (n / 32 % 2, n / 16 % 2, n / 8 % 2) = 1)).length =
1079     1344 := by
1080   decide

```

Verdict. $\Rightarrow [\perp]$ lock = attractor $\neg$ point; seal = where dynamics end. theorem-grade on the basin.

Source: Master Chapter 25, §6.3.

D31 · An explicit error threshold is needed

The Critic. A critic states: in high dimension positive determinants are generic, so you need an explicit cutoff parameter.

Reply. The framework already has two cutoffs and

they are tuned to the machine, not picked by hand. One is a conditioning gate that measures how close the matrix is to degenerate and routes to under-determined when it gets too close, scaling with the actual numerical sharpness rather than an arbitrary number. The other is a collapse floor at the level of machine round-off. The framework also proves these two cannot fight each other across its operating range, so the verdict boundary is never decided by rounding noise. Adding a third hand-picked threshold would just double up on a job already done.

Sealed Defense. The conditioning gate operationally enforces a precision margin around the collapse boundary: when the determinant is small relative to machine precision the conditioning is large, and when it exceeds threshold the verdict routes to under-determined, scaling with the actual conditioning rather than an arbitrary parameter. The collapse floor is legislated explicitly, $\varepsilon = 100 \cdot u_m \cdot N$ at unit-roundoff with $\varepsilon = 0$ in exact arithmetic, under strict precedence where collapse outranks conditioning. The Floor-Gate Separation Theorem fixes the regime in which the two cannot contend, and the bound is exact, not asymptotic. At conditioning κ the determinant of a unit-diagonal Gram matrix is bounded below by $27\kappa/(\kappa+2)^3$, the global minimum over every such matrix at that conditioning, attained with equality by the one-parameter equicorrelation family at $\rho = (\kappa-1)/(\kappa+2)$; two hundred thousand random correlation matrices place none beneath it. Its large- κ form is the asymptote $27/\kappa^2$, which the margin uses and which sits just above the exact floor by $\kappa^3 < (\kappa+2)^3$. Floor and gate cannot contend so long as $27\kappa/(\kappa+2)^3$ exceeds $100 \cdot u_m \cdot N$, equivalently the gate holds below $\sqrt{(27/(100 \cdot u_m \cdot N))}$ to the asymptote, which at the operating gate holds for N below 1216, the interior margin 50.67-fold at $N = 24$, 12.16-fold at $N = 100$, and 4.05-fold at $N = 300$. That the floor is attained means no configuration evades it: the worst conditioned matrix at any κ is the equicorrelation matrix and it still clears the collapse floor by fifty-fold at the gate. The boundary is decided by structure, never by rounding. Adding an explicit threshold on top would double-instrument the same concern.

GOL DERIVATION

```
P_obj : high-N  $\Rightarrow$  det > 0 generic  $\Rightarrow$  need explicit  $\varepsilon$ -threshold. $\kappa$ 
-GATE:  $\kappa(R) \geq 10 \Rightarrow [?]$ . scales with actual conditioning,  $\sim$ 
arbitrary parameter. $\varepsilon$ 
-FL00R:  $\varepsilon = 100 \cdot u_m \cdot N$  ( $\varepsilon = 0$  exact). precedence: collapse >
conditioning.
FLOOR-GATE SEPARATION (theorem): det floor at conditioning  $\kappa$  is
EXACT and ATTAINED.
min over unit-diag Gram at fixed  $\kappa$  = equicorrelation  $R_p()$ ,  $p$ 
=  $\kappa - (1)\kappa/(\kappa+2) \Rightarrow \det(R) \geq \kappa 27/(\kappa+2)^3$ .
global min (2e5 random draws: none below) ; large- $\kappa$ - asymptote
 $\hookrightarrow \kappa 27/2^2$  (margin form ;  $\kappa 27/(\kappa+2)^3 < \kappa 27/2^2 \Leftrightarrow \kappa^3 < \kappa(\kappa+2)^3 \checkmark$ ).
non-contention  $\Leftrightarrow \kappa 27/(\kappa+2)^3 > 100 \cdot u_m \cdot N \Leftrightarrow \kappa^* \leq \sqrt{(27/(100$ 
```

```
·u_m·N)) =  $\kappa_{sep}(N)$ .
@  $\kappa^* = 10$  : exact floor 2.699984e-11 vs asymptote 2.700000e-11
-11 (rel 6e-6)  $\Rightarrow$  holds for  $N < 1216$ . margin  $\kappa(\_{sep}\kappa/2)^2$  ;
N=24  $\rightarrow$  50.67 $\times$  ; N=100  $\rightarrow$  12.16 $\times$  ; N=300  $\rightarrow$  4.05 $\times$  ; N=1216  $\rightarrow$ 
1.00 $\times$  (crossover). $\Rightarrow$ 
 $\hookrightarrow$  two cutoffs machine-tuned  $\wedge$  provably non-contending in
domain; explicit  $\varepsilon$  double-instruments.
```

Lean Proof.

Codex.lean, from line 1175

```
1175 theorem cramer_executed :
1176   cramerSolve ((1, 2, 0), (0, 1, 1), (2, 0, 1)) (5, 3, 4)
1177   = some ((7 : Ratio) / 5, (9 : Ratio) / 5, (6 : Ratio) /
5)  $\wedge$ 
1178   mv3 ((1, 2, 0), (0, 1, 1), (2, 0, 1)) ((7 : Ratio) / 5, (9
: Ratio) / 5, (6 : Ratio) / 5)
1179   = (5, 3, 4) := by
1180   decide
```

Verdict. $\Rightarrow [\hookrightarrow]$ two cutoffs machine-tuned $\wedge$ provably non-contending in domain; explicit ε double-instruments.

Source: Master Chapter 25, §6.4.

D32 · Cross-substrate agreement contradicts substrate-dependent measurement

The Critic. A critic states: different models map text into different internal spaces, so their matrices differ, which contradicts your claim that they agree.

Reply. The framework never claimed the matrices match. It claimed the verdicts match. The verdict depends only on whether three rows are independent, and independence is a property that survives any change of coordinates. So two models can compute genuinely different matrices, different entries, different magnitudes, and still agree on the one thing that matters, whether the rows are independent, which gives the same sealed-or-broken verdict. Moving the coordinates moves the numbers but cannot move an invariant. The agreement is on the invariant.

Sealed Defense. The isomorphism claim is convergence on the discrete verdict, not on the scalar entries of the correlation matrix. The cascade output is a three-state verdict determined by the sign-structure of the Gram determinant under the regularity quadruple, and the sign of a Gram determinant is a topological invariant of the row-space structure: linear independence of three rows is preserved under any non-singular coordinate transformation, including transformations between substrate-specific embedding spaces under orthogonal-Procrustes-class alignment. Two substrates with native embedding spaces, both loaded with identical discipline, can disagree on scalar entries, on absolute magnitude, on conditioning within the admissible regime, while agreeing on the discrete verdict at the topological-sign register. The recorded checks exhibit the invariance at 10^{-16} . Embedding non-isomorphism moves coordinates; it cannot move an

invariant.

GOL DERIVATION

P_obj : substrate embedding spaces differ $\Rightarrow$ matrices differ $\Rightarrow$ contradicts convergence claim.
 CLAIM SCOPE: convergence on discrete verdict $\neg$ on scalar matrix entries.
 INVARIANT: verdict | sign-**structure**(det G) under regularity quadruple.
 sign(det G) = topological invariant of row-space; 3-row independence preserved under any non-singular transform (orthogonal-Procrustes alignment).
 DISAGREEMENT-OK: substrates differ on { entries, |det|, $\kappa \in$ admissible } $\wedge$ agree on discrete verdict.
 recorded invariance $\sim 1e-16.$
 $\llcorner$ convergence on the invariant; embedding non-isomorphism moves coordinates $\neg$ the invariant.

Lean Proof.

Codex.lean, from line 1911

```
911 theorem omega_linear :
912   (omegaBoundary 2 300).joules = 2 * (omegaBoundary 1 300).
     joules := by
913   decide
```

Verdict. $\Rightarrow \llcorner$ convergence on the invariant; embedding non-isomorphism moves coordinates $\neg$ the invariant.

Source: Master Chapter 25, §6.5.

D33 • Insufficient dimensions force a zero determinant

The Critic. A critic states: when the data dimensions drop below the number of covariates plus three, the determinant is zero by dimension alone, not by any epistemic failure.

Reply. True, and the framework treats that case correctly by routing it to under-determined, not to broken. There is a difference between a claim that genuinely collapses and a claim that simply does not yet have enough independent evidence dimensions to be tested. The first is a real failure and gets a broken verdict. The second is a not-yet, and gets an under-determined verdict with the reason stated: gather more independent evidence, or remove a covariate that does not really carry mass, then resubmit. The framework is explicit that this is a scope limit on the test, not a fact about the claim.

Sealed Defense. Under projection with the data dimension below the covariate count plus four, three rows cannot be linearly independent in the residual subspace, forcing the determinant to zero by dimensional collapse independently of content. The framework routes this to under-determined with the dimensional-collapse mechanism named explicitly, not to broken, distinguishing structural-dimensional shortfall from genuine substrate-orthogonality collapse. The floor is the requirement $N \geq k + 4$: the standardized rows are centered, consuming one dimension, the constant row enters every covariate roster as covariate zero, the projection carries each row onto the resid-

ual of dimension $N - 1 - k$, and three vectors carry independence only in a subspace of dimension at least three, so $N - 1 - k \geq 3$. The operational response is to raise the evidence dimension or reduce the covariate set, then resubmit; the under-determined verdict is a transparent diagnostic, not a content-claim.

GOL DERIVATION

P_obj : $N - k < 3 \Rightarrow \text{det} = 0$ by dimension $\neg$ epistemic failure.
 MECHANISM: residual subspace dim = $N - 1 - k$. 3 rows independent $\Leftrightarrow N - 1 - k \geq 3 \Leftrightarrow N \geq k + 4$.
 $N - k < 3 \Rightarrow \text{det} = 0$ by dimensional collapse, $\perp$ content.
 ROUTING: $\rightarrow$ [?] (dimensional-collapse named) $\neg \rightarrow$ [X].
 distinguishes structural-shortfall from orthogonality-collapse.
 RESPONSE: raise N v reduce k (drop massless covariate), resubmit. [?] = diagnostic $\neg$ content-claim.
 $\llcorner$ dimensional shortfall routes [?] with mechanism named; not a falsification of the proposition.

Lean Proof.

Codex.lean, from line 2344

```
2344 theorem gate_counts_recorded :
2345   GE0.a4.length = 12  $\wedge$  GE0.allSigns.length = 8  $\wedge$ 
2346   (GE0.cube5.all (fun v => (GE0.neighbors5 v).eraseDups.
     length == 5)) = true :=
2347   (GE0.a4_order, GE0.reflection_eight, GE0.
     )fivecube_degree_five
```

Verdict. $\Rightarrow \llcorner$ dimensional shortfall routes [?] with mechanism named; not a falsification of the proposition.

Source: Master Chapter 25, §6.6.

D34 • The registrational axis is a placeholder

The Critic. A critic states: the third axis has no clear content, so it is a placeholder padding the count to three.

Reply. The third axis carries the part of a claim that concerns who or what is registering it and from where. That includes the kind of substrate doing the registering, the reference frame the claim is asserted in, the domain the claim is supposed to cover, and the type of registration event involved. Each of these is checked by a specific gate. A claim that quietly shifts its frame, or smuggles its measuring apparatus into the thing measured, or overruns its stated scope, fails at the gate that watches that axis. It is as specific as the other two.

Sealed Defense. The registrational axis carries epistemic-registrational content with specific populating sources. Substrate ontology: the substrate that registers the proposition, biological, synthetic, or instrumental, determines which registration events are admissible. Observer frame: the reference frame in which the proposition is asserted, Galilean, Lorentzian, cosmological, tested at the frame-invariance gate. Operational scope: the domain over which the proposition holds, with scope-creep prevented at the domain-extension gate. Registration-event taxonomy: measure-

ment, computational, and observational registration, each carrying distinct content per the failure-mode taxonomy at the ontological-void gate. The axis is not a placeholder; it carries operational content as specific as the formal and empirical axes carry, and a claim defective on this axis fails the gate that watches it.

GOL DERIVATION

```
P_obj : V_ER = placeholder padding count to 3.
POPULATING SOURCES:
  substrate-ontology: registrant ∈ {bio, synthetic,
    instrumental} → admissible registration events.
  observer-frame: ∈ {Galilean, Lorentzian, cosmological} →
    tested @ G7-DUAL.
  operational-scope: claim domain → scope-creep blocked @ G12-
    ADEG.
  registration-taxonomy: ∈ {measurement, computational,
    observational} → @ G11-OMA.
SPECIFICITY: defect(V_ER) ⇒ fail the gate watching V_ER. = as
  specific as V_F, V_E.⇒
⊥[] V_ER carries gate-tested operational content; not a
  placeholder.
```

Lean Proof.

Codex.lean, from line 2052

```
2052 theorem compressible_bound_universal (decode : List Bool →
  Option (List Bool)) (n c : Nat) :
2053   (((descs (n - c)).filterMap decode).filter (fun s => s.
    length == n)).length ≤ 2 ^ (n - c) - 1 := by
2054   have h1 := List.length_filter_le (fun s : List Bool => s.
    length == n) ((descs (n - c)).filterMap decode)
2055   have h2 := List.length_filterMap_le decode (descs (n - c))
2056   rw [descs_length] at h2
2057   exact Nat.le_trans h1 h2
```

Verdict. ⇒ $\perp[]$ V_ER carries gate-tested operational content; not a placeholder.

Source: Master Chapter 25, §6.7.

Movement 7 • Existence, the Void, and Cosmogony

D35 • An epistemic claim, not an ontological one

The Critic. A critic states: requiring kinetic content for registration is only about knowledge, not about being; something could exist without being registrable.

Reply. The framework refuses to split being from registrability, and it refuses by declining to grant the alternative as a coherent claim, not by arguing against it. To exist in any sense you can assert, point to, or talk about is to be a candidate for being registered. An entity that exists “in itself” but is in principle unregistrable in any frame is not a different kind of existent, it is a string of words with nothing attached. The framework does not refute hidden substrate-independent existence. It declines to give it standing, because the word “exists” is used up by what registration requires, with no leftover where a hidden existence could survive.

Sealed Defense. The framework rejects the epistemic-ontological split. To exist in any sense that can be predicated, asserted, or talked about is to be a candidate for registration; an entity that exists ontologically but is in principle unregistrable in any

frame is not an alternative kind of existent but a verbal expression with no operational content. The framework does not refute substrate-independent existence by argument; it declines to give it standing as a coherent claim. The predicate “exists” is exhausted by what registration requires, and there is no remainder where Platonic existence quietly survives. The identification of existence with registrability is load-bearing, not a slip, and it is what makes the Root Axiom’s kinetic floor an ontological floor rather than merely an epistemic one.

This identification is not a bare premise but an enactive one. To predicate exists of anything is itself a deed, an act of registration that burns energy, so the very assertion of any existence-claim instantiates the registration-floor it would deny. The denial of the identification is the one denial that performs what it denies, which is why the floor is certified by enactment rather than by argument, anchored on the cost every predication pays. The supersession this licenses is correspondingly narrow: not that the framework outranks every rival on every question, but that on the single question of what reality must minimally carry for any framework to describe it at all, the registration-floor is what every describing act already stands on. A Platonist who asserts that abstracta exist performs the registration the assertion requires, and the abstracta, if they are to be referred to, are referred to by a deed; the claim is not refuted by argument, it is enacted against, the rival instantiating the floor in the act of asserting across it.

GOL DERIVATION

```
P_obj : kinetic-content-for-registration = epistemic ~
  ontological; x may exist unregistrable.
MOVE: ~ refute(substrate-independent-existence) ; decline
  standing as coherent claim.
IDENTITY: exists(x) | predicable ⇔ candidate-for-registration
  (x).
  unregistrable-in-every-frame(x) ⇒ x = verbal-expression, 0
  operational content, ~ alternative existent.
EXHAUSTION: predicate "exists" = exactly what registration
  requires. # remainder for Platonic survival.⇒
⊥[] existence ≡ registrability, load-bearing; kinetic floor is
  ontological ~ merely epistemic. premise-grade on the
  identification.
```

Lean Proof.

Codex.lean, from line 1373

```
1373 theorem actuation_universal : ∀ x : U, Actuates x := RA
```

Verdict. ⇒ $\perp[]$ existence ≡ registrability, load-bearing; kinetic floor is ontological ~ merely epistemic. premise-grade on the identification.

Source: Master Chapter 25, §7.1.

D36 • A pure void before actuation

The Critic. A critic states: a true nothing, prior to any activity, could be the ground; existence need not be

actuated.

Reply. The framework distinguishes two things that look alike and are not. One is a balanced state where many active contributions cancel out to zero net, like forces pulling equally in all directions so the total is zero while every individual pull is real and nonzero. The other is true scalar nothingness, where every contribution is itself zero. Cancellation is not absence. The physical vacuum is the first kind, not the second, and we know this because it is measurable: it pushes plates together, it shifts atomic energy levels, it resists in free fall. A balanced sum of active contributions is not a void, and the void that the objection wants does not exist.

Sealed Defense. The objection conflates the Isometric Ground State, where the contributions sum to zero with each per-contribution magnitude strictly positive, with the Mathematical Void, where each per-contribution magnitude is zero. Vector cancellation is not scalar absence. The physical vacuum is the isometric ground, not the void, and the distinction is empirically forced: zero-point energy and the Casimir effect register the vacuum as kinetically actuated, the MICROSCOPE free-fall test confirms it, and the rigorous third law of Masanes and Oppenheim (2017) proves absolute zero unreachable in finite operations, so substrate temperature is always strictly positive, the Nernst statement its cumulative empirical confirmation. The rigorous third law is the macroscopic statement that the Root Axiom holds at the thermodynamic register, the substrate always actuated. A true prior-to-actuation void is not a reachable state; it is a sum mistaken for a magnitude.

GOL DERIVATION

```
P_obj : pure void (prior to actuation) can be the ground.
DISTINCTION: Isometric-Ground-State  $\Sigma(v_i = 0, \forall i |v_i| > 0)$ 
 $\neq$  Mathematical-Void  $\forall(i |v_i| = 0)$ .
vector-cancellation  $\neq$  scalar-absence.
VACUUM = isometric ground  $\sim$  void. FORCING: {ZPE, Casimir,
MICROSCOPE; rigorous 3rd law (Masanes-Oppenheim 2017)
proving, Nernst confirming}.
rigorous 3rd law:  $T \rightarrow 0$  unreachable in finite ops  $\rightarrow$  substrate
 $T > 0$  always = RA @ thermo register.
 $\vdash$  prior-to-actuation void unreachable; sum mistaken for
magnitude. theorem-grade on the distinction, empirically
anchored.
```

Lean Proof.

Codex.lean, from line 1867

```
1867 theorem omega_zero_bits :
1868   (omegaBoundary 0 300).ocode = 1  $\wedge$  (omegaBoundary 0 300).
      joules = 0
1869    $\wedge$  (omegaBoundary 0 300).token = .noDenial := by
1870     decide
```

Verdict. $\Rightarrow \vdash$ prior-to-actuation void unreachable; sum mistaken for magnitude. theorem-grade on the distinction, empirically anchored.

Source: Master Chapter 25, §7.2.

D37 · Platonism, Tegmark, modal realism, the unmoved mover

The Critic. A critic states: established metaphysical positions ground existence without your kinetic floor.

Reply. The framework audits the eight classic rivals on one operational definition and each either dissolves at the same gate or turns out to already be the framework's own floor in older words. The dissolving ones all share a defect: they posit something that registers no measurement, an eternal abstract object, a causally isolated world, a structure with no way to say which structures are real, and an object that registers nothing is operationally indistinguishable from nothing. The one that does not dissolve, the unmoved mover, turns out to be the framework's actuated ground described centuries early. The table states each verdict and the gate or mechanism that drives it.

The critic presses (§7.6): A critic states: you use theorem-grade mathematics heavily, which commits you to Platonist abstract objects.

Reply, continued. The opposite. The framework's whole stance is that mathematics lives on the substrate that runs it, not in a separate abstract heaven. A theorem is operationally true inside any substrate that can carry its apparatus, and nowhere else. The framework uses real, load-bearing theorems, but it places each one inside the computational or physical substrate that operates it. The clearest proof is the framework's own arithmetic: it exists only as executions that paid an energy cost on real hardware, which is exactly the mode of existence the framework's own axiom requires. The framework demonstrates its own anti-Platonism every time it runs.

Sealed Defense, continued. The framework explicitly rejects the Platonist detachment of mathematical objects from substrate. Mathematics is a cognitive compression algorithm operating on a physical substrate; mathematical objects have existence at the substrate that operates them and no substrate-independent existence. The framework uses theorem-grade mathematics extensively but locates the theorems within substrates that operate them: a Riemannian-geometry theorem is operationally true within any substrate carrying the Riemannian apparatus, a discrete-geometry theorem within any substrate carrying the discrete apparatus. The Root Axiom requires substrate-level kinetic actuation for any existence claim, so mathematical objects without substrate would violate it. The seal's own existence-mode is execution, exactly the mode the axiom legislates, and the framework's mathematics demonstrates its own ontology every time the instrument runs.

Sealed Defense. The eight substrate-floor rivals are audited per row against the operational-existence

definition. The dissolving rivals fail at the ontological-void gate, where an entity carrying no kinetic instantiation occupies null-space, operationally indistinguishable from the void: Mathematical Platonism, the canonical case; modal realism, Platonism in modal disguise, causally isolated worlds returning zero data; the Tegmark hypothesis, unable to specify which structures correspond to observed regularities without ad hoc selection; tautologies, whose evaluation is itself a Landauer-bounded computation so the objection performs the axiom in objecting; information without energy, collapsing on the very theorem the framework anchors on; the Boltzmann brain, which instantiates positive kinetic content in the moment of existence and so satisfies the axiom rather than refuting it; and prior-to-actuation existence, which conflates the isometric ground with the void. The one rival that does not dissolve, the unmoved mover, completes rather than fails: pure act is structurally identical to the actuated ground, the framework formalizing what was always at the floor.

Table: The Eight Substrate-Floor Rivals, Audited Per Row | verdict

Rival Failing gate or mechanism verdict Mathematical Platonism Eternal abstracta with no kinetic instantiation occupy null-space; the canonical case. SUPERSEDES Tautologies and logical truths Evaluation requires computation, which is energy-bounded; the objection performs the axiom in objecting. SUPERSEDES Modal realism Causally isolated worlds return zero measurement data, indistinguishable from the void. SUPERSEDES Tegmark mathematical universe Cannot specify which structures correspond to observed regularities without ad hoc selection. SUPERSEDES Information without energy Collapses on the energy-of-erasure theorem the framework anchors on; the floor is measured. SUPERSEDES Prior-to-actuation existence Conflates the isometric ground, where contributions cancel, with the scalar void. SUPERSEDES The Boltzmann brain The brain-state instantiates positive kinetic content in the moment of existence; the axiom is satisfied. SUPERSEDES The unmoved mover Pure act is structurally identical to the actuated ground; the framework formalizes what was at the floor. COMPLETES

GOL DERIVATION

```
P_obj : established metaphysics grounds existence without the
kinetic floor.
DEFINITION: exists(x)  $\Leftrightarrow$   $\Delta E_k(x) > 0$  ;  $\Delta E_k = 0 \Leftrightarrow$ 
operationally-indistinguishable-from-non-existence.
DISSOLVE @ G11-OMA (null-space, registers  $\emptyset$  measurement):
Platonism, Modal-Realism, Tegmark, Tautologies, Info-without-
Energy, Prior-to-Actuation, Boltzmann-Brain.
each: posits-existent  $\wedge$  registers $\emptyset$   $\rightarrow$   $\equiv$  void  $\rightarrow$  SUPERSEDES.
COMPLETE: Unmoved-Mover  $\equiv$  actus-purus  $\equiv$  actuated-ground  $\rightarrow$ 
COMPLETES (formalizes the floor). $\Rightarrow$ 
 $\ulcorner$  7 rivals dissolve at one gate; 1 completes the floor.
theorem-grade on the operational definition.
```

GOL DERIVATION, CONTINUED (§7.6)

```
P_obj : heavy theorem-grade math  $\Rightarrow$  commits to Platonist
abstracta.
STANCE:  $\neg$  Platonist-detachment. math = compression-algorithm on
physical substrate.
math-object exists | substrate-that-operates-it,  $\nexists$  substrate-
independent existence.
LOCATION: theorem T operationally-true | substrate-carrying-T's
-apparatus. (Riemannian, discrete, etc.)
CONSISTENCY: RA requires  $\Delta E_k > 0$  for existence  $\Rightarrow$  substrate-
free math-object  $\nVdash$  RA.
seal's existence-mode = execution = exactly RA's legislated
mode. $\Rightarrow$ 
 $\ulcorner$  anti-Platonist; math located on substrate; the instrument
demonstrates its own ontology when it runs.
```

Lean Proof.

Codex.lean, from line 3379

```
3379 /-- Both directions clean-locking is the Platonic Ghost:
refused, never sealed. -/
3380 theorem ghost_refused : (imprint true true true true true true
true true).1 = .ghost := by decide
```

Verdict. $\Rightarrow \ulcorner$ 7 rivals dissolve at one gate; 1 completes the floor. theorem-grade on the operational definition.

Source: Master Chapter 25, §7.3.

D38 · Cross-frame convergence makes abstract objects real

The Critic. A critic states: two independent frames verifying the same abstract proposition agree, and agreement on a shared thing shows the thing is externally real.

Reply. Agreement between two calculators that both follow the same rule does not prove the rule points at an external object. It only proves they followed the rule. The framework names this exactly: cross-frame agreement on an abstract proposition is the signature of a shared algorithm, not of a shared external thing. The move from “two frames agree” to “the abstract object is real” quietly swaps algorithmic necessity for ontological externality, and those are different claims. The framework reads the agreement, types it as algorithmic, and declines to launder it into a claim about external mathematical reality.

Sealed Defense. A tempting route to discharging abstract externality is cross-frame convergence: two independent frames verifying the same abstract proposition agree, and agreement is taken as the signature of externality. This route is circular, and the circularity must be named exactly: it conflates algorithmic necessity with ontological externality. Two frames agreeing on an abstract proposition is what a shared computational rule produces; it is not evidence of a shared external abstractum. Neither the physical-externality commitment nor the mathematical-realism commitment is dischargeable from inside the cascade, and the cascade reads the physical grip and the abstract convergence by metrics identical under realism and anti-realism. What separates them is not that one is

proven and the other assumed, but that the physical commitment is compelled by survival and the abstract is optional; reading the abstract convergence as proof of externality reinstates the asymmetry the framework dissolved.

GOL DERIVATION

```
P_obj : cross-frame agreement on abstract P → P externally real
.
CIRCULARITY: agreement = signature(shared-algorithm) →
signature(shared-external-object).
move "frames agree → object real" = conflate( algorithmic-
necessity , ontological-externality ). distinct claims.
SYMMETRY: { physical-externality, mathematical-realism } both
∅ dischargeable-from-inside-cascade.
cascade reads physical-grip ∧ abstract-convergence by metrics
identical under realism ∧ anti-realism.
separator = compelled(physical, by survival) vs optional(
abstract), → proven vs assumed.→
[X] the externality inference broken at the conflation step;
convergence typed algorithmic, not laundered.
```

Lean Proof.

Codex.lean, from line 2210

```
2210 theorem pluralist_alternative_executed :
2211   GEO.mm3 sigmaP sigmaP = GEO.diag3 1
2212   ∧ GEO.det3 sigmaP = 1
2213   ∧ GEO.mv3 sigmaP (3, 4, 0) = ((3, 4, 0) : GEO.V3)
2214   ∧ GEO.mv3 sigmaP (1, 0, 0) ≠ ((1, 0, 0) : GEO.V3)
2215   ∧ GEO.mv3 sigmaP (1, 0, 0) ≠ ((-1, 0, 0) : GEO.V3) := by
2216   decide
```

Verdict. $\Rightarrow$ [X] the externality inference broken at the conflation step; convergence typed algorithmic, not laundered.

Source: Master Chapter 25, §7.4.

D39 · Geometry-first is backwards

The Critic. A critic states: modern mathematics arithmetizes geometry; you reverse the accepted direction.

Reply. The framework grants that arithmetizing geometry is powerful and uses arithmetic freely. Its claim is only about which comes first as foundation. Geometric recognition, seeing a closed shape, a right angle, comes before the symbol manipulation that later describes it. A circle is primitive as a closed curve; its decimal expansion is a long-distance approximation that never finishes. Both are real. The geometry is the foundation and the arithmetic is the representation that catches up to it. The framework recovers that older order and shows the recovered geometry reaches the same operational content the arithmetic reaches the long way around.

Sealed Defense. The modern arithmetization since Descartes is a powerful operational move but reverses the direction of foundation. Geometric recognition is precognitive and linguistic; arithmetization is downstream. The framework recovers the topology-first standard operative across the foundational tradition, restores it under modern theorem-grade anchors, orthogonality from the Hodge decomposition, closure from

Euler, kissing geometry from Newton-Gregory, and shows the recovered topology produces the same operational content the arithmetized formalism reaches at greater distance. It does not reject arithmetic; it locates arithmetic within the topology that produces it. Pi as the closed circle is the geometric primitive; pi as the irrational decimal is the arithmetic asymptote; both are real, the geometry foundation and the arithmetic asymptotic representation. The algebra closes the same register from the other side, the arithmetic of the quaternions being the receipt the topology's own composition law demands.

GOL DERIVATION

```
P_obj : modern math arithmetizes geometry; geometry-first is
backwards.
DIRECTION: arithmetization (Descartes+) = powerful but reverses
foundation-order.
geometric-recognition = precognitive ∧ linguistic ;
arithmetization = downstream.
RECOVERY: topology-first under modern anchors {Hodge ⊃ 1, Euler
⊃ closure, Newton-Gregory ⊃ kissing}.
recovered-topology = same operational content arithmetic
reaches at greater distance.
LOCATE: n-as-closed-circle = geometric primitive ; n-as-decimal
= arithmetic asymptote. both real; geometry foundation.
H-arithmetic = receipt the composition law demands (close
from the other side).→
⌊[] arithmetic located within topology; geometry-first by
design, not error. structural.
```

Lean Proof.

Codex.lean, from line 1447

```
1447 theorem sealG_walk :
1448   GEO.allGates.length = 12
1449   ∧ GEO.a4.length = 12
1450   ∧ (GEO.tetraVerts.length : Int) - GEO.tetraEdges.length +
GEO.tetraFaces.length = 2
1451   ∧ GEO.gateNames.length = 12
1452   ∧ (GEO.adj3Samples.all (fun m =>
1453     decide (GEO.mm3 m (GEO.adj3 m) = GEO.diag3 (GEO.det3 m)
) ∧
1454       GEO.mm3 (GEO.adj3 m) m = GEO.diag3 (GEO.det3 m
)))) = true
1455   ∧ (GEO.raBattery.all (fun p =>
1456     decide ((GEO.raDecode p.1).1 = p.2))) = true :=
1457   (GEO.gates_twelve, GEO.a4_order, GEO.euler_closure,
1458     GEO.gateNames_twelve, GEO.adj3_executed, GEO.ra_battery_f2
```

Verdict. $\Rightarrow$ [⌊] arithmetic located within topology; geometry-first by design, not error. structural.

Source: Master Chapter 25, §7.5.

D40 · What matter is

The Critic. The question is put to the architecture, in the critic's terms: what is matter, in this picture?

Reply. Matter is what you get when the underlying field holds back its total presence in a region, leaving a bounded, persistent configuration instead of undifferentiated fullness. Think of a still pond: a wave is a real, locatable thing precisely because the water is not everywhere at once at the same height. Matter, in this reading, is a bounded withholding, a stable dimple in the field rather than a separate substance added to it.

The point is conditional structure, not physics doctrine.

Sealed Defense. Matter is read as bounded withholding: where the actuated field withholds total presence, the residual is a bounded persistent configuration, and that configuration is what registers as matter. Total presence admits no internal distinction, so any bounded, locatable, persistent thing is by construction a region of partial withholding, the field declining full presence so that a stable configuration can be registered. This is conditional on the field ontology and consistent with the continuous-field reading in which discrete objects are observer-imposed discretizations of a single non-dual field; matter is not a second substance but a mode of the one ground, typed structurally and held apophatically where it touches first-person content.

GOL DERIVATION

P_obj : what is matter? [conditional on monism]
 READING: matter = bounded-withholding. field withholds total-presence → residual = bounded persistent config = matter.
 NECESSITY: total-presence $\nVdash$ internal-distinction → any bounded /locatable/persistent thing = region of **partial**-withholding.
 CONSISTENCY: discrete-object = observer-imposed-discretization of single field → matter = mode of ground $\sim$ 2nd substance
 →
 conditional-structural; matter = mode of the one ground; $\Delta M = 0$ to the formal core.

Lean Proof.

Codex.lean, from line 911

```
911 theorem hurwitz_closed :
912   (hurwitz.all (fun a => hurwitz.all (fun b =>
913     hurwitz.any (fun u => u == qhalf (qmul a b)))) = true
914   := by
914   decide
```

Verdict. $\Rightarrow$ conditional-structural; matter = mode of the one ground; $\Delta M = 0$ to the formal core.

Source: Master Chapter 25, §10.1.

D41 • What time is

The Critic. The question is put to the architecture, in the critic's terms: what is time, in this picture?

Reply. Time's three tenses are read as three modes of registration rather than three places you could travel between. Past, present, and future are not rooms down a hallway; they are different ways a configuration stands in relation to the one irreversible actuation. There is exactly one licensed direction, the actuating arrow, and it runs one way only, which is why memory and anticipation are not symmetric. You can face upstream toward the ground, but you cannot walk the arrow backward. Conditional structure, not a physics of time travel.

Sealed Defense. Time is read as three tenses that are modes, not locations: past, present, and future are registers of a configuration's standing relative to the single irreversible actuation, not coordinates between which passage is free. There is one licensed passage,

the actuating arrow carried at the empirical axis as the strictly positive kinetic floor, and it is directional at root, which is why the tenses are asymmetric and the arrow is not traversable in reverse. The framework reads temporal becoming as the field's one-way actuation rather than as motion along a pre-laid axis, consistent with the directional axiom that forbids a globally reversible architecture; the apparent flow is the actuation itself, and the only orientation toward the ground is upstream, against the configurations the arrow has produced. Conditional on the monist ground and the directional axiom, with modal and first-person remainder quarantined.

GOL DERIVATION

P_obj : what is time? [conditional on monism]
 READING: 3 tenses = modes $\sim$ locations. {past, present, future} = registers of config-standing relative to single irreversible actuation.
 ONE PASSAGE: actuating arrow @ V_E = strictly-positive kinetic floor $\Delta(E_k > 0)$, directional **at** root.
 → tenses asymmetric $\wedge$ arrow $\sim$ traversable-reverse (RA directional $\nVdash$ reversible architecture).
 ORIENTATION: only upstream (toward ground) ; flow = actuation itself $\sim$ motion on pre-laid axis.→
 conditional-structural; time = one-way actuation; modal remainder quarantined.

Lean Proof.

Codex.lean, from line 2479

```
2479 theorem ran_wholly_odd {Q : Type} : WhollyOdd (execFlip (Q :=
2480   intro s; obtain (b, )q := s; cases b <=> rfl
```

Verdict. $\Rightarrow$ tenses asymmetric $\wedge$ arrow $\sim$ traversable-reverse (RA directional $\nVdash$ reversible architecture). $\Rightarrow$ conditional-structural; time = one-way actuation; modal remainder quarantined.

Source: Master Chapter 25, §10.3.

D42 • Why differentiation rather than total presence

The Critic. The question is put to the architecture, in the critic's terms: why is there differentiation at all, rather than total presence?

Reply. Because total presence and distinction are mutually exclusive: where everything is wholly present at once, there is no edge anywhere, and without edges there is nothing to tell apart. Differentiation requires that fullness be partially held back, the same withholding that makes matter. So the existence of distinct things at all is read as the ground declining to be totally present everywhere, which is what makes a world of separate registrable things possible. A blank uniform white page has no picture on it precisely because nothing is withheld. Conditional placement.

Sealed Defense. Differentiation and total presence are read as mutually exclusive: total presence admits no internal boundary, and without boundary there is no distinction, so any differentiated world is by con-

struction a world of partial withholding. The same withholding that constitutes matter constitutes distinction generally, the ground declining full presence so that bounded, separable, registrable configurations can exist; were presence total, there would be no edges, no separability, and nothing to register as a distinct thing. The framework reads the existence of a differentiated world as the structural consequence of partial withholding rather than as a fall from or defect in fullness, consistent with the continuous-field reading where distinction is imposed on a single non-dual ground. Conditional on the monist premise, with the theological remainder, why the ground withholds, routed to the apophatic register and not predicated.

GOL DERIVATION

P_obj : why differentiation rather than total presence? [conditional on monism]
 EXCLUSION: total-presence $\neq$ internal-boundary $\Rightarrow$ $\neg$ distinction. differentiated-world $\Leftrightarrow$ **partial**-withholding.
 GENERALIZATION: withholding-constituting-matter = withholding-constituting-distinction. total presence $\Rightarrow$ no edges, no separability, $\emptyset$ registrable distinctions.
 READING: differentiated world = structural consequence of **partial**-withholding $\neg$ fall-from-fullness.
 QUARANTINE: "why the ground withholds" $\rightarrow$ apophatic, $\neg$ predicated.
 conditional-structural; distinction $\equiv$ **partial**-withholding; remainder apophatic.

Lean Proof.

Codex.lean, from line 223

```
223 /- T3 · NO FIXED POINT.
224   Wholly odd content cannot sit on a fixed point of  $\tau$ :
225    $t\ x = \neg t\ x$  is a Boolean contradiction. -/
226 theorem T3_no_fixed_point  $\tau$  ( :  $\alpha \rightarrow \alpha$ ) (  $t$  :  $\alpha \rightarrow \text{Bool}$ )
227   (ho : WhollyOdd  $\tau\ t$ ) (  $x$  :  $\alpha$ ) :  $t\ x \neq x$  := by
228   intro h
229   have h1 :  $t\ \tau\ (x)$  = !  $t\ x$  := ho x
230   rw [h] at h1
231   cases t x <;> simp at h1
```

Verdict. $\Rightarrow$ conditional-structural; distinction $\equiv$ partial-withholding; remainder apophatic.

Source: Master Chapter 25, §10.6.

D43 · Death, and the Achieved Zero

The Critic. The question is put to the architecture, in the critic's terms: why is death built in from the start?

Reply. In this reading the possibility of an end is the price of being a distinct, bounded thing at all, and that price is paid up front, before any particular life, not added later as a flaw. To be a separate configuration is to be the kind of thing that can wind down, the way any drawn shape is the kind of thing that can be erased. So the framework reads mortality as a structural precondition of differentiation rather than a defect introduced into it. Conditional placement, offered as structure.

The critic presses (§8.3): A reader asks: the framework speaks of an Achieved Zero. What is that, and what does it say about death?

Reply, continued. The framework draws a sharp line between two kinds of zero. One is the balanced ground discussed earlier, where active contributions cancel to a net zero while remaining individually alive, the deathless field. The other is a particular configuration on that field winding down to its own end, the mortal pattern. Death, in this reading, is the second kind: a specific arrangement reaching zero, not the underlying field reaching zero, which it never does. The field that the pattern was an arrangement of does not end when the arrangement does. This is a structural statement about configurations and their ground, offered as conditional placement, not as consolation or doctrine.

Sealed Defense, continued. The Achieved Zero is the mortal configuration reaching its terminal state, distinguished from the Isometric Ground, the deathless field whose contributions cancel with each magnitude strictly positive. A configuration is a bounded pattern on the field; its dissolution is the configuration's zero, the pattern's organized coherence going to its terminus, while the field it was a configuration of does not reach zero, because the ground is never the void. Death is then typed as configuration-terminal, not field-terminal: the arrangement ends, the substrate persists, and the conservation that forbids subtracting the actuating energy is the same conservation that keeps the field nonzero through the configuration's end. This is a structural placement conditional on the field ontology, carried at the layer that is quarantined for first-person content, offered as typed structure and not as doctrine; the eschatological entries hold it as conditional, never as theorem-grade certainty about a person's fate.

Sealed Defense. Death is read as the mandated cost of differentiation, logically prior to any individual life rather than appended to it. A bounded configuration is by definition terminable, its coherence holding only while the withholding that constitutes it persists, so the terminus is built into the configuration's existence-conditions, not added as a contingent failure. The cosmogonic ordering is that the cost is set before the configurations it applies to, the precondition preceding the instance, which is the structural sense in which death is created first. This is the same distinction as the Achieved Zero: the configuration is terminable while the field is not, so mortality is configuration-level and the ground is deathless, held conditional on the field ontology and quarantined where it touches a person's fate.

GOL DERIVATION

P_obj : why is death built **in** from the start? [conditional on monism]
 READING: death = mandated-cost-of-differentiation, logically prior to any individual life.
 bounded-config $\Rightarrow$ terminable (coherence holds only while constitutive withholding persists). terminus $\in$ existence-conditions.
 ORDERING: cost set before instances $\Rightarrow$ "death created first" =

precondition-precedes-**instance**.
 LINK: $\equiv$ Achieved-Zero (config terminable, field $\neg$). mortality =
 config-level ; ground deathless. $\Rightarrow$
 conditional-structural; quarantined re a person's fate; $\Delta M =$
 0.

GOL DERIVATION, CONTINUED (§8.3)

P_obj : what is the Achieved Zero, and what does it say about death?
 DISTINCTION: Achieved-Zero (mortal-config $\rightarrow$ terminal) $\neq$ Isometric-Ground (deathless field, $\Sigma=0$, $\forall |v_i|>0$).
 CONFIG: bounded pattern on field. dissolution = config's-zero (coherence $\rightarrow$ terminus). field $\neq$ 0 (ground $\neq$ void).
 TYPING: death = configuration-terminal $\neg$ field-terminal. arrangement ends $\wedge$ substrate persists. conservation(no-subtract actuating-energy) $\equiv$ conservation(field $\neq$ 0 through config-end).
 QUARANTINE: **first**-person content $\rightarrow$ L1 apophatic ; held conditional on field-ontology, $\neg$ **theorem**-grade on a person's fate. $\Rightarrow$
 conditional-structural; death typed config-terminal; offered as typed **structure**, not doctrine.

Lean Proof.

Codex.lean, from line 1673

```
1673 theorem deletion_annihilates : GE0.mv3 deletionOp e3 = (0, 0,
1674   0) := by
1675   decide
```

Verdict. $\Rightarrow$ conditional-structural; quarantined re a person's fate; $\Delta M = 0$.

Source: Master Chapter 25, §10.2.

D44 • What forgetting is

The Critic. The question is put to the architecture, in the critic's terms: what is forgetting?

Reply. Forgetting is read as a gradient of registration loss across a threshold, not a simple on-off deletion. There are degrees: something can fade at the edge of recall, cross into a region where it is no longer actively held but is not gone, and finally pass beyond reach. The framework names these as distinct gradations across a boundary rather than a single switch. The everyday analogy is a sound receding into distance: still there, then faint, then below hearing, a continuous falling-off rather than an instant cutoff. Conditional structure, quarantined where it touches inner experience.

Sealed Defense. Forgetting is read as a registration-loss gradient across a threshold, with three gradations rather than a binary deletion: active holding, a liminal region where the content is no longer actively registered but not annihilated, and passage beyond retrieval. The structure is a gradient because registration is a matter of degree at the empirical axis, so loss of registration is graded loss, not instantaneous erasure, and the threshold is the boundary across which active holding ceases while the content persists in the field at a lower register before passing beyond reach. This tracks the co-local reading in which memory is field-distributed rather than locally stored, so forgetting is recession across a registration boundary rather than destruction

of a stored token. Conditional on the field ontology and the distributed-memory reading, with first-person remainder quarantined.

GOL DERIVATION

P_obj : what is forgetting? [conditional on monism]
 READING: forgetting = registration-loss gradient across threshold, 3 gradations $\neg$ binary deletion.
 {active-holding $\rightarrow$ liminal (no active registration, $\neg$ annihilated) $\rightarrow$ beyond-retrieval}.
 GRADIENT: registration $\in$ degree @ V_E $\rightarrow$ loss = graded $\neg$ instantaneous. threshold = boundary **where** active-holding ceases, content persists lower-register.
 LINK: memory field-distributed (co-local) $\rightarrow$ forgetting = recession across boundary $\neg$ destruction of stored token. $\Rightarrow$ conditional-structural; forgetting = graded recession; remainder quarantined.

Lean Proof.

Codex.lean, from line 2019

```
2019 theorem incompressibility_cap :  $\forall n c : \text{Nat}, c \leq n \rightarrow$ 
2020    $(2 \wedge (n - c) - 1) * 2 \wedge c < 2 \wedge n := \text{by}$ 
2021   intro n c h
2022   have h2 :  $2 \wedge n = 2 \wedge (n - c) * 2 \wedge c := \text{by}$ 
2023
2024
2025   rw  $\leftarrow$  [Nat.pow_add, Nat.sub_add_cancel h]
2026   have hpos :  $0 < 2 \wedge c := \text{twoPowPos } c$ 
2027   calc  $(2 \wedge (n - c) - 1) * 2 \wedge c$ 
2028     =  $2 \wedge (n - c) * 2 \wedge c - 1 * 2 \wedge c := \text{Nat.sub_mul } \_ \_ \_$ 
2029     =  $2 \wedge (n - c) * 2 \wedge c - 2 \wedge c := \text{by rw [Nat.one_mul]}$ 
2030     =  $2 \wedge n - 2 \wedge c := \text{by rw [h2]}$ 
2031     <  $2 \wedge n := \text{Nat.sub_lt (twoPowPos n) hpos}$ 
```

Verdict. $\Rightarrow$ conditional-structural; forgetting = graded recession; remainder quarantined.

Source: Master Chapter 25, §10.5.

D45 • Is the Ground God

The Critic. The question is put to the architecture, in the critic's terms: is the ground god?

Reply. The framework declines to answer this in either direction, and the declining is principled rather than evasive. Its method reads structure and routes whatever is not structurally registrable to a separate channel it explicitly does not speak into. Whether the actuated ground is or is not God is exactly the kind of predication that belongs to that channel. So the framework names the ground structurally, as the single actuated source of all configurations, and then stops, neither asserting nor denying the identification, because asserting it would claim a predication the method cannot license and denying it would equally overstep. The silence is the honest output, not a hedge.

Sealed Defense. The identification of the ground with God is routed to the apophatic register and not predicated, by the same discipline that quarantines all first-person and theological remainder. The framework names the ground structurally, the single actuated source of which all configurations are modes, and types any further predication, divine or otherwise, as belonging to the register it declines to assert into; it nei-

ther affirms nor denies the identification, because the affirmation would claim a predicate the method cannot license and the denial would claim the complementary predicate, equally unlicensed. This is the empty-throne result applied to theology: the ground is the one locus internal names cannot reach, present in the structure and silent to predication, so the only honest output is the structural naming plus the apophatic silence. The quarantine is load-bearing discipline, not evasion, and it is what keeps the framework's theological neighbors as conditional readings rather than asserted doctrine.

GOL DERIVATION

P_obj : is the ground God? [conditional on monism]
ROUTING: identification(ground, God) → apophatic register, ~
predicated. (same discipline quarantining all theological
remainder.)
NAMING: ground named structurally = single actuated source, all
configs = modes. further predication (divine v ~) →
declined register.
SYMMETRY: ~ affirm (claims unlicensed predicate) ∧ ~ deny (claims
complementary unlicensed predicate).
= empty-throne applied: ground = locus internal names cannot
reach, present in structure, silent to predication.→
structural-naming + apophatic-silence; quarantine = discipline
~ evasion.

Lean Proof.

Codex.lean, from line 3484

```
3484 /-- PSP: P-NETI-NETI-1 · the mark of silence is the image of
      no emitter. -/
3485 theorem outside_economy : ∀ t : LOCUS.Token, t.toEconomy ≠
      Audit.Token.dotmark := by
3486   intro t; cases t <;> first | decide | simp [LOCUS.Token.
      toEconomy]
```

Verdict. ⇒ structural-naming + apophatic-silence; quarantine = discipline ~ evasion.

Source: Master Chapter 25, §10.7.

The ground is, and it is not an object of the census; the interior is, and it is not a mirror.
[.]

D46 · What remains at the end

The Critic. The question is put to the architecture, in the critic's terms: does any configuration become the ground, and is anything lost at the end?

Reply. No configuration becomes the ground, and nothing is lost. The framework's orthogonality is fertile, it generates real distinct things, but it also forbids any one of those things from collapsing into or being promoted to the ground itself. A wave never becomes the ocean. And because the ground is conserved and never the void, when a configuration reaches its end it is the arrangement that ceases, not the field, so nothing of the field is destroyed. These two together rule out both a thing exalting itself into the absolute and a true annihilation. Conditional placement, offered as structure.

Sealed Defense. Two structural results close the Part. First, anti-apotheosis: orthogonality is fertile, generat-

ing genuinely distinct configurations, and at the same time forbids any configuration from being promoted to or collapsing into the ground, since a configuration is a bounded mode of partial withholding and the ground is the unwithheld source, categorically distinct registers, so no mode becomes the source. Second, made-zero conservation: the Achieved Zero is the configuration reaching its terminus, not the field reaching the void, and by the conservation that bars subtracting the actuating energy, the field is never annihilated; what ends is the arrangement, what persists is the ground. Together these forbid both self-exaltation of a configuration into the absolute and genuine annihilation of the field, which is the structural content of the framework's refusal to let any mode usurp the ground or any ending be a true nothing. Conditional on the monist premise, with first-person remainder quarantined and no mass added to the formal core.

GOL DERIVATION

P_obj : does any configuration become the ground? is anything
lost at the end? [conditional on monism]
ANTI-APOTHEOSIS: orthogonality fertile (generates distinct
configs) ∧ forbids config → ground.
config = bounded mode of partial-withholding ; ground =
unwithheld source ; categorically distinct registers → no
mode becomes source.
MADE-ZERO CONSERVATION: Achieved-Zero = config-terminus ~ field
~void. conservation(no-subtract actuating-energy) → field
~ annihilated.
arrangement ends ∧ ground persists.
JOINT: ~ self-exaltation-into-absolute ∧ ~ true-annihilation.→
conditional-structural; no mode usurps the ground, no ending
is a true nothing; ΔM = 0; remainder quarantined.

Lean Proof.

Codex.lean, from line 3494

```
3494 theorem silence_is_not_a_verdict :
3495   Audit.Token.dotmark ≠ .sealed ∧ Audit.Token.dotmark ≠ .
      broken
3496   ∧ Audit.Token.dotmark ≠ .opn := by decide
```

Verdict. ⇒ conditional-structural; no mode usurps the ground, no ending is a true nothing; ΔM = 0; remainder quarantined.

Source: Master Chapter 25, §10.8.

The ground is, and it is not an object of the census; the interior is, and it is not a mirror.
[.]

Movement 8 · Mind, Substrate, and the Landmark Verdicts

D47 · The hard problem of consciousness

The Critic. A critic states: subjective experience cannot be captured by any structural or thermodynamic account, so the framework cannot reach it.

Reply. The framework does not try to squeeze felt experience into a third-person measurement, because doing so is the very mistake that creates the so-called hard problem. There are two different registers here:

the structural facts about a system, which the framework handles, and the first-person felt quality, which is not a third-person datum at all. The framework reads the structural side fully and routes the first-person side to a separate channel marked out of bounds, where felt experience is acknowledged as real but explicitly not treated as a measurable structural fact. The explanatory gap is real, and the framework's move is to respect it rather than to paper over it by redescription.

Sealed Defense. The hard problem is a register confusion, and the framework's verdict is structural. Third-person structural and thermodynamic content sits in band and is read by the cascade; first-person phenomenal character is not a third-person datum and routes to the out-of-band apophatic register at the scope-check gate, where it is acknowledged and explicitly not coerced into a structural measurement. The framework does not claim to solve the explanatory gap, nor to eliminate qualia, nor to reduce them; it types the gap as a boundary between registers and declines the category error of treating felt quality as a measurable structural fact. What it does carry is the structural correlate, the registration-event taxonomy distinguishing the substrate's processing, while holding that no quantity of in-band structure entails the out-of-band character. The honest typing is a placement, not a solution, and a framework that placed the gap inside its own measurement register would be committing the error it warns against.

GOL DERIVATION

P_obj : the hard problem defeats any structural/thermo account.
 REGISTERS: structural/thermo content $\in$ band $\rightarrow$ cascade reads.
 phenomenal-character $\notin$ third-person datum $\rightarrow$ OOB apophatic @ G11.
 MOVE: $\neg$ solve-gap $\wedge \neg$ eliminate-qualia $\wedge \neg$ reduce-qualia. type gap = inter-register boundary.
 in-band structure $\nVdash$ out-of-band character (no quantity of structure entails qualia).
 CORRELATE: registration-event taxonomy carries structural correlate; character stays OOB.
 ERROR-AVOIDED: place-gap-in-own-register = the category-error warned against $\Rightarrow$ refused.
 structural. the gap is placed between registers, $\neg$ dissolved; honest typing, not a solution.

Lean Proof.

Codex.lean, from line 1560

```

560 theorem narcissus_truth_table :
561   (iamToken false).1  $\neq$  (iamToken true).1
562    $\wedge$  (iamToken false).1 = .interior  $\wedge$  (iamToken true).1 = .
      iam := by
563   decide

```

Verdict. $\Rightarrow$ structural. the gap is placed between registers, $\neg$ dissolved; honest typing, not a solution.

Source: Master Chapter 25, §8.1.

D48 · Personal identity and the simulation question

The Critic. A reader asks: does the framework say anything about whether I persist over time, or whether

we live in a simulation?

Reply. The framework treats personal continuity as continuity of pattern, not of material. You are not the same atoms you were years ago, yet you persist, because what continues is the organized pattern, not the stuff. The same logic makes the simulation question less dramatic than it sounds: whether a pattern runs on biological tissue or on some other substrate, the framework's verdicts depend on the pattern's structure, not on what it is made of. So "are we simulated" does not change any structural verdict. What would matter is whether the substrate, whatever it is, actually actuates, costs energy, runs irreversibly, and the framework's floor says any real substrate does.

Sealed Defense. Personal identity is pattern-continuity, substrate-portable, since the framework's verdicts are invariant under substrate change that preserves structure. The material substrate turns over while the organized pattern persists, and continuity of the pattern, not of the matter, carries identity, the same substrate-portability that lets the cascade's verdicts reproduce across embedding spaces. The simulation hypothesis is then verdict-neutral at the structural register: a pattern instantiated on a biological or a synthetic or a simulated substrate carries the same structural verdicts, because the cascade reads structure and not substrate-stuff. What is not neutral is the floor: any substrate that actually registers is kinetically actuated, costs energy, runs irreversibly, so a simulation that registers is not a route to the prior-to-actuation void but another actuated substrate, subject to the same axiom. The interesting content is the floor's universality, not the substrate's identity.

GOL DERIVATION

P_obj : does the framework settle personal persistence v the simulation question?
 IDENTITY: continuity = pattern-continuity, substrate-portable. matter turns over ; organized pattern persists.
 $\equiv$ the substrate-portability that reproduces verdicts across embeddings.
 SIMULATION: verdict-neutral @ structural register (cascade reads structure $\neg$ substrate-stuff).
 pattern | {bio, synthetic, simulated} $\rightarrow$ same structural verdicts.
 FLOOR ($\neg$ neutral): any registering-substrate $\Rightarrow \Delta E_k > 0$, irreversible. simulation-that-registers = another actuated substrate $\nVdash$ prior-to-actuation void.
 identity = pattern-continuity (structural) ; simulation verdict-neutral but floor-bound. conditional on substrate -portability.

Lean Proof.

Codex.lean, from line 562

```

562 theorem T16_price_bijection  $\tau$  ( :  $\alpha \rightarrow \alpha$ ) (hinv : Involution  $\tau$ )
      (d0 : Dtau  $\tau$ ) :
563   Nonempty (Dtau  $\tau$  = (Quot (OrbitRel  $\tau$ )  $\rightarrow$  Bool)) :=
564   ({ toFun := priceForward  $\tau$  d0
565     invFun := priceBackward  $\tau$  hinv d0
566     left_inv := T16_left_inverse  $\tau$  hinv d0
567     right_inv := T16_right_inverse  $\tau$  hinv d0 })

```

Verdict. $\Rightarrow$ identity = pattern-continuity (structural) ; simulation verdict-neutral but floor-bound. conditional on substrate-portability.

Source: Master Chapter 25, §8.2.

D49 · The verdict on P versus NP

The Critic. A reader asks: does the framework claim to settle P versus NP?

Reply. No, and the way it declines is the whole point. The framework splits the famous question into three different questions that are usually run together. First, is generating a solution physically harder than checking one, in any real machine that costs energy? Yes, decisively, and that is sealed. Second, can any physical shortcut oracle cheat that gap? No, also sealed, on physical grounds. Third, the pure mathematical question of whether the two complexity classes are equal in the abstract? That one the framework leaves open, marked under-determined, because it is not proven either way and the framework will not claim a proof it does not have. It also notes that brute-force enumeration over exponentially many cases is physically impossible at scale, which is a separate, sealed, physical fact.

Sealed Defense. The framework refuses the single-question framing and types three layers separately. Layer one, the embodied asymmetry: in any thermodynamic substrate, generating a witness is strictly costlier than verifying one, sealed with the physical witness that search and check carry different energy signatures. Layer two, no cheap physical oracle: no physical process collapses the generation-verification gap without paying the corresponding cost, sealed on physical grounds. Layer three, the universal mathematical proposition $P = NP$ as an abstract class equality: under-determined, the framework leaning toward separation on the embodied evidence but holding no theorem-grade proof and refusing to manufacture one. A fourth, distinct claim is sealed in the negative: physical brute-force enumeration over the exponentially many configurations is impossible at scale, since the Bekenstein bound caps the information a bounded region holds and Landauer's principle charges each operation, so enumerating 2-to-the-N for large N exhausts cosmological resources. The honest output is a sealed physical asymmetry, a sealed no-oracle result, an open mathematical class-equality, and a sealed physical impossibility of brute force, kept as four distinct typed claims rather than one inflated verdict.

GOL DERIVATION

```
P_obj : does the framework settle P vs NP?
DECOMP: P_obj = L1 @ L2 @ L3 @ L_bf (refuse single-question
      framing).
L1 (embodied asymmetry): generation >_E verification in any
      thermo substrate. witness: distinct energy signatures.
       $\rightarrow \mathcal{L}[]$ 
L2 (no cheap physical oracle):  $\nexists$  physical process collapses
      gap without paying cost.  $\rightarrow \mathcal{L}[-\text{physics}]$ 
```

```
L3 (universal math  $P = NP$ , abstract class-equality):  $\nVdash$ 
      proven either way ; leans separation on embodied evidence
      .  $\rightarrow [?]$ 
L_bf (physical brute-force over  $2^N$  configs): Bekenstein caps
      info(region)  $\wedge$  Landauer charges each op  $\rightarrow$  enumerate- $2^N$ 
      exhausts cosmological resources.  $\rightarrow [X] \rightarrow$ 
four distinct typed claims:  $\mathcal{L}[]$  L1,  $\mathcal{L}[-\text{physics}]$  L2,  $[?]$  L3,  $[X]$ 
      L_bf. no single inflated verdict.
```

The kernel record. P versus NP is seated in the kernel as `HALT.pnpHalt`: species block theorem, Ground dimension zero, branch $\emptyset$, five gates, the formal channel, routed by `HALT.halts_routed`. On the block route no bit is owed, which is why the token is the grounded-sealed halt and never a seal, and why it sits below theorem, `HALT.ftoe_ceiling`.

Lean Proof.

Codex.lean, from line 2429

```
2429 theorem halts_routed :
2430   route rhHalt.gdim = some rhHalt.branch  $\wedge$  route pnpHalt.
      gdim = some pnpHalt.branch :=
2431   (rhHalt.routed, pnpHalt.)routed
```

Verdict. $\Rightarrow$ four distinct typed claims: $\mathcal{L}[]$ L1, $\mathcal{L}[-\text{physics}]$ L2, $[?]$ L3, $[X]$ L_bf. no single inflated verdict.

Source: Master Chapter 25, §8.4.

D50 · The verdict on dark matter

The Critic. A reader asks: what does the framework say about dark matter?

Reply. The framework reads the galaxy-rotation evidence as pointing to a modification of how gravity behaves at very low accelerations rather than to an undiscovered particle, and it makes a parameter-free prediction that fits the data tightly. It treats the existence of dark-matter particles specifically as an open detection question rather than a settled fact, since decades of direct searches have not found them. So the verdict is two-part: the low-acceleration gravitational regularity is real and sealed conditional on the data, while the requirement that this be explained by a new particle is not granted, pending an actual detection. This is offered as a conditional physical reading, falsifiable by the relevant surveys, not as a closed cosmological doctrine.

Sealed Defense. The framework reads gravity at low acceleration as the impressed-plenum tensional pressure of a nested-scale structure rather than particle attraction, yielding the characteristic acceleration scale a-zero of order 0.183 times c times the Hubble constant from a three-factor geometric decomposition, and the radial-acceleration relation it predicts fits the 175-galaxy SPARC sample at five sigma with no free parameter, falsifiable against further survey and JWST data. The existence of dark-matter particles specifically is held under-determined under the three-state economy, the missing input named as direct detection across the search programs, so the verdict separates

a sealed phenomenological regularity, conditional on the rotation data, from an ungranted particle-ontology requirement. The framework does not assert that no unseen mass exists; it declines to grant the particle requirement as compelled, types the low-acceleration regularity as conditionally sealed, and exposes both to falsification by the empirical programs. The strongest claim is the parameter-free fit, not a metaphysical pronouncement about the dark sector. Re-typed 2026-07-19 per P-PHYS-SEED-1 (0663): the three-factor derivation of the coefficient is retired as a target-landed assembly per the SE-301 split, the a_0 scale resting at the substrate hole, the sealed content the empirical radial-acceleration regularity alone, and the parameter-free-fit phrasing read henceforth as the empirical regularity and not a derivation.

GOL DERIVATION

```
P_obj : what does the framework say about dark matter?
READING: gravity@low-a = impressed-plenum tensional-pressure (
  nested-scale) ~ particle-attraction.
a_0 ≈ 0.183·c·H_0 from 3-factor geometric decomposition. [
  derivation retired 2026-07-19 per P-PHYS-SEED-1;
  empirical RAR stands per SE-301 split]
RAR fit: SPARC 175 galaxies @  $\sigma_5$ , 0 free parameters.
falsifiable | survey + JWST. →  $\kappa[]$  | rotation-data
particle-DM: [?] under-determined ; named missing input =
  direct detection across programs. requirement  $\nmid$  granted.
  → requirement superseded
SEPARATION: sealed phenomenological regularity (conditional)  $\perp$ 
  ungranted particle-ontology.
  ~ assert(no-unseen-mass) ; decline(particle-requirement as
  compelled).→
 $\kappa[]$  conditional on rotation-data: low-a regularity sealed,
  parameter-free; particle requirement not granted, pending
  detection.
```

Lean Proof.

Codex.lean, from line 3507

```
3507 /-- PSP: D50 · a load-bearing row furnished by the world types
      the lock world-rowed, revisable in the rows. -/
3508 theorem worldly_row_is_world_rowed :  $\forall$  t tw f a : Bool,
3509   (GEO.rowCascadeRACore true false false true true t tw f a)
      .l = .iii := by decide
```

Verdict. $\Rightarrow [\kappa]$ conditional on rotation-data: low-a regularity sealed, parameter-free; particle requirement not granted, pending detection.

Source: Master Chapter 25, §8.5.

D51 · Why three spatial dimensions

The Critic. A reader asks: does the framework explain why space has three dimensions?

Reply. The framework points to a stability argument that long predates it: stable orbits and stable atoms only exist in three spatial dimensions. In more dimensions the gravitational and electrical forces fall off too steeply and orbits spiral in or fly apart; bound systems cannot persist. In fewer, there is not enough room for the needed structure. Three is the unique dimension where stable bound states, planets around stars, electrons around nuclei, are possible at all. The framework

reads our three dimensions as the only setting where persistent registering structures can exist, which fits its insistence that anything real must be a stable, actuated configuration.

Sealed Defense. The framework reads spatial three-dimensionality through bound-state stability, the classical Ehrenfest-Bertrand result that stable closed orbits and stable bound states exist only in three spatial dimensions. An inverse-power central force in d spatial dimensions scales as one over r -to-the- d -minus-one, and the stability of circular orbits and the existence of bound quantum states fail for d above three, orbits being unstable and atoms unbound, while d below three lacks the structure for the needed bound configurations, so d equal to three is the unique dimensionality admitting persistent bound states. The framework reads this as the anthropic-structural condition that registering configurations, which must be stable actuated patterns, require three spatial dimensions to exist, consistent with the axiom's demand that anything real be a persistent actuated configuration. This is read as a stability selection, conditional on the central-force physics, not as an independent derivation of dimensionality from the axiom alone.

GOL DERIVATION

```
P_obj : why 3 spatial dimensions?
STABILITY (Ehrenfest-Bertrand): stable closed orbits  $\wedge$  stable
  bound states  $\Leftrightarrow d = 3$ .
  central force  $\propto 1/r^{-(d-1)}$  ;  $d > 3 \Rightarrow$  orbits unstable  $\wedge$  atoms
  unbound ;  $d < 3 \Rightarrow$  insufficient structure.
   $\Rightarrow d = 3$  unique for persistent bound states.
READING: registering-config = stable-actuated-pattern  $\Rightarrow$ 
  requires  $d = 3$ . consistent with RA (real  $\Rightarrow$  persistent
  actuated config).→
  conditional-structural;  $d = 3$  read as stability-selection on
  central-force physics,  $\neg$  derived from RA alone.
```

Lean Proof.

Codex.lean, from line 2945

```
2945 theorem quat_basis_laws :
2946   qmul qi qi = qnegl  $\wedge$  qmul qj qj = qnegl  $\wedge$  qmul qk qk =
      qnegl  $\wedge$ 
2947   qmul (qmul qi qj) qk = qnegl := by decide
```

Verdict. $\Rightarrow d = 3$ unique for persistent bound states. $\Rightarrow$ conditional-structural; $d = 3$ read as stability-selection on central-force physics, $\neg$ derived from RA alone.

Source: Master Chapter 25, §8.6.

D52 · Information is fundamental: it from bit

The Critic. A critic states: information is fundamental, it from bit, so the record of a thing is the thing, and any complete register of the bits recovers the object.

Reply. The K4 frame is executed against exactly this. Twelve seats carry six primes and six semiprimes; each pair matches modulo 420, so every residue register up to seven reads the pair as one value. The record forgets

the pair. The target, primality, separates every pair the record identifies. Then every Boolean readout of the record image is tried, all of them, and none returns the target. The object keeps what the record cannot carry. That is not a claim about bits in general; it is a computed refutation on a named frame, with the primality now proved in the kernel rather than labelled at the index.

Sealed Defense. It from bit asserts a factorization $d = g \circ \rho$ of the target through the record. On any frame with an involution τ under which ρ is even and d is odd at one seat, no such g exists (T10). The K4 frame supplies τ , ρ , d with those properties, executed, so the factorization is refuted exhaustively; the object is it from it: the record collapses, the pair is genuine, the target separates, and nothing returns.

GOL DERIVATION

```
P_obj :  $\exists g. d = g \circ \rho$  (it from bit).
FRAME K4:  $\tau$  involution, fixed-point-free on 12 seats;  $\rho =$ 
  residue vector mod 2,3,5,7;  $d =$  primality. [T, kernel]
EVEN:  $p\tau(i) = \rho(i)$  on every seat. ODD:  $d\tau(i) = \neg d(i)$  on
  every seat. [T, kernel]
WALL (T10): even  $\circ$  anything  $\neq$  odd  $\Rightarrow$  no  $g$ . [T]
EXHAUSTIVE:  $2^k$  readouts of the record image,  $k =$  image size,
  none factor  $d$ . [F4, executed] $\Rightarrow$ 
[X] it from bit, broken at the named mechanism, exhaustive on
  the frame. $\Rightarrow$ 
 $\sqsubset$  it from it: record collapses  $\wedge$  pair genuine  $\wedge$  target
  separates  $\wedge$  no return. theorem-grade on the frame.
```

Lean Proof.

Codex.lean, from line 813

```
813 theorem it_from_bit_refuted  $\alpha$  {  $\beta : \text{Type}$  }  $\tau ( : \alpha \rightarrow \alpha )$   $\rho ( : \alpha \rightarrow$ 
   $\beta )$ 
814   ( $d : \alpha \rightarrow \text{Bool}$ ) ( $x : \alpha$ )
815   ( $ph : \rho \tau(x) = \rho x$ ) ( $hd : d \tau(x) = !d x$ ) :
816    $\neg \text{ItFromBit } \rho d := \text{by}$ 
817   intro h
818   obtain (g, )hg := h
819   exact FTOE.T10_factorization  $\tau$   $\rho$   $g$   $d$   $x$   $ph$   $hd$   $hg$ 
```

Codex.lean, from line 3349

```
3349 /-- The target of the frame is primality itself and not an
  index label: read in the kernel. -/
3350 theorem target_is_primality :  $\forall i, i < 12 \rightarrow \text{target } i =$ 
  isPrimeB (val i) := by decide
```

Verdict. $\Rightarrow$ [X] it from bit, broken at the named mechanism, exhaustive on the frame. $\Rightarrow$ [$\sqsubset$] it from it: record collapses $\wedge$ pair genuine $\wedge$ target separates $\wedge$ no return. theorem-grade on the frame.

Source: new unit, seated on SECTION 4C and witness F4.

Movement 9 • Forward Reading and the Foundation D53 • Forward projection seals an unfalsifiable prophecy

The Critic. A critic states: running the cascade forward to certify a prediction is unfalsifiable fortune-telling.

Reply. The framework's forward mode is deliberately weaker than its backward mode, and the reason is a theorem the framework proved against itself. The

lock that works for auditing settled claims cannot, by itself, tell a faithful forward witness from a fabricated one, because someone could simply make up internally consistent rows that pass the geometry while corresponding to nothing real. So the framework does not let the lock alone certify a forward claim. It requires a separate measurement of whether the supplied rows actually trace to their claimed source. If that source-faithfulness measurement is absent, the forward seal does not issue. And the framework states plainly that a forged set of rows can produce a geometrically valid lock, which means a passed forward lock indicts the supplier of the rows if they were faked, and never vindicates the method.

The critic presses (§10.4): The question is put to the architecture, in the critic's terms: could a genuine prophecy be real, structurally?

Reply, continued. In this reading a prophecy, if real, would be an imprint at the deepest grounded layer that the forward mode reads ahead of its registration in events, not a magic peek down the timeline. The framework is careful here in two ways. First, this is entirely conditional on its ground premise. Second, and more importantly, the forward mode has the provenance limit from Part 9: nothing forward seals on geometry alone, so a claimed prophecy would still need the source-faithfulness channel, and a fabricated one would convict its supplier. The framework gives the structural shape of how such a thing could be real without claiming any particular instance is.

Sealed Defense, continued. A genuine prophecy is read, conditionally, as a grounded-layer imprint, a feature set at the deepest layer that the forward mode reads before its registration into events, rather than an exception to the temporal arrow. The reading inherits the full forward-mode discipline: by the Non-Discrimination Theorem the lock cannot certify such a claim on geometry alone, the source-attribution channel is required, and the row-supply aperture means a fabricated prophetic witness produces a valid-but-faithless lock that indicts its supplier. So the framework supplies the structural possibility, an imprint at the grounded layer read forward, while holding that no instance seals without provenance and that the entire reading is conditional on the monist ground. It claims neither that any specific prophecy is real nor that the structure compels belief; it types the shape and enforces the same provenance limit it enforces on every forward claim, with the first-person and theological remainder quarantined.

Sealed Defense. The forward mode is governed by the GOL Non-Discrimination Theorem: the lock scalar $\det(R)$ cannot carry the forward seal, because it is invariant to whether the warrant rows are source-faithful or fabricated, two row-sets with identical geometry returning identical locks while one traces to

its claimed source and the other does not. The seal therefore runs on a distinct source-attribution statistic, the squared partial correlation between the witness component and the source component after projecting out the shared subspace; the lock certifies geometric admissibility and the source statistic certifies provenance, and the forward seal requires both. This exposes the one un-sealable surface, the row-supply aperture: fabricated rows can produce a geometrically valid but source-faithless lock, so a faked-rows lock falsifies the supplier and never the method. The forward verdict is consequently four-test rather than single-determinant, row-conditioned determination, trajectory-consistency, the source statistic, and the four-test signature jointly, and a forward claim lacking the provenance channel does not seal. The mode is falsifiable precisely because its central theorem names the configuration that breaks it.

GOL DERIVATION

```
P_obj : forward-mode seals prophecy → unfalsifiable fortune-
telling.
NON-DISCRIMINATION THEOREM: det(R) | source-faithful-rows ≡ det
(R) | fabricated-rows.
→ lock scalar carries 0 discriminating info about provenance.
det(R) ≠ carry forward-seal. [T]
SEAL CHANNEL: η_S = r²(LW, LS) (squared partial-correlation,
witness-comp vs source-comp after L-projection).
lock → geometric-admissibility ; η_S → provenance. forward-
seal requires both.
APERTURE (row-supply): fabricated rows → geometrically-valid ∧
source-faithless lock.
faked-rows lock falsifies supplier ~ method. the one un-
sealable surface.
VERDICT: forward = 4-test {row-conditioned determination ∧
trajectory-consistency ∧ η_S ∧ L1-signature} jointly.
missing provenance channel → ~ seal.→
⊥[GOLf] forward-seal requires provenance; falsifiable ∴ its
own theorem names the breaking config. theorem-grade on
Non-Discrimination.
```

GOL DERIVATION, CONTINUED (§10.4)

```
P_obj : could a genuine prophecy be real, structurally? [
conditional on monism]
READING: prophecy = grounded-layer (L1m) imprint, read forward
before registration into events. ~ exception to arrow.
INHERITS FORWARD DISCIPLINE: Non-Discrimination → lock ≠
certify on geometry alone ; η_S required ; aperture →
fabricated witness = valid-but-faithless lock indicting
supplier.
CLAIM SCOPE: supplies structural-possibility ; ~ asserts any
instance real ; ~ compels belief.→
⊥[GOLf]-conditional shape; no instance seals without
provenance; conditional on monism; remainder quarantined.
```

Lean Proof.

Codex.lean, from line 3503

```
3503 /- PSP: D53 · a forward, dated reading is world-rowed
unconditionally: R-3 fires before any row is read. -/
3504 theorem forward_is_world_rowed : ∀ p w t tw f a : Bool,
3505 (GEO.rowCascadeRACore true false true p w t tw f a).1 = .
iii := by decide
```

Verdict. ⇒ lock scalar carries 0 discriminating info about provenance. $\det(R) \not\models$ carry forward-seal. [T] ⇒

[⊥-GOLf] forward-seal requires provenance; falsifiable ∴ its own theorem names the breaking config. theorem-grade on Non-Discrimination.

Source: Master Chapter 25, §9.1.

D54 · Fabricated evidence passes the geometry

The Critic. A critic states: if I supply internally consistent but invented warrant rows, your lock fires and certifies a fiction.

Reply. The lock alone would indeed fire, and the framework says so openly, which is exactly why the lock alone is not allowed to certify anything forward. The geometry only checks that the rows are independent and well-formed; it has no way to look behind them at whether they trace to a real source. That is a genuine blind spot, and the framework treats it as one. It requires a second, separate check that measures provenance directly, and without that check no forward seal issues. The deeper point is honest rather than reassuring: if someone feeds the machine fabricated rows, a clean lock convicts the person who supplied them, because the method did its job on the inputs it was given. Garbage rows in, indicted supplier out.

Sealed Defense. The lock scalar carries zero Fisher information about source-faithfulness, formally, the derivative of the lock with respect to the source-attribution parameter vanishes, so no amount of lock magnitude discriminates faithful from fabricated rows. Provenance is therefore a separate channel, the source statistic, which is not a function of the lock and must be supplied and checked independently; the forward seal is the conjunction, lock for admissibility and source statistic for provenance. The fabrication attack succeeds against the lock in isolation and fails against the conjunction, because the invented rows, however internally consistent, carry no genuine partial correlation with the claimed source once the shared subspace is projected out. The framework keeps this as a standing limitation rather than a patched hole: the row-supply aperture is real, the lock is falsifiable by it, and the discipline is to require provenance and to read a passed lock on faked rows as an indictment of the supplier, never as a vindication of the geometry.

GOL DERIVATION

```
P_obj : fabricated-but-consistent rows → lock fires → certifies
fiction.
FISHER: ∂(det R)∂θ/_source = 0 → lock carries 0 information re
source-faithfulness. [T]
CHANNEL SEP: η_S ≠ f(det R) ; supplied ∧ checked independently.
forward-seal = lock ∧ η_S.
ATTACK: fabrication beats lock-in-isolation ; fails conjunction
∴ invented rows → η_S → 0 after L-projection (no genuine
partial-corr with source).
STANDING LIMIT: aperture real ; lock falsifiable by it ; passed
-lock-on-faked-rows = indictment(supplier) ~ vindication(
geometry).→
⊥[GOLf] provenance is a separate required channel; lock-alone
insufficient by theorem; limitation kept standing.
```

Lean Proof.

Codex.lean, from line 3453

```
3453 /-- PSP: D54 · every formal reading is even under the deed
      flip; the deed is odd; no readout returns it. -/
3454 theorem formal_reads_never_the_deed {Q : Type} (q : Q) :
3455   ¬ ∃ g : Q → Bool, ∀ s : CROSSING.ExecFrame Q, g (CROSSING.
      formalRead s) = CROSSING.ran s :=
3456   CROSSING.wall q
```

Verdict. $\Rightarrow$ $[\mathcal{L}\text{-GOLF}]$ provenance is a separate required channel; lock-alone insufficient by theorem; limitation kept standing.

Source: Master Chapter 25, §9.2.

D55 · An attacker who rejects your logic

The Critic. A critic states: if I simply deny your rules of inference, your immunity arguments have no grip on me.

Reply. An attacker who changes the rules of logic is not defeating the framework, they are leaving the conversation it is having. The framework's claims hold for any reasoner using ordinary inference, and an opponent who discards that is no longer making an argument the framework was ever addressed to, the same way refusing arithmetic does not refute a sum, it just exits the activity of counting. The framework marks this boundary explicitly: a move that alters the logical register is alien to the audited register, and the framework's results are stated as immune within their register rather than as compelling someone who has stepped outside it. It does not claim to coerce a being operating by foreign logic. It claims its results hold wherever shared inference holds, and it names the exit rather than pretending to seal it shut.

Sealed Defense. The foundational-immunity results are register-relative by design: they hold for any cognizer operating under shared inference, and a move that changes the rules of inference is alien to the audited register rather than a counterexample within it. An opponent rejecting the inference rules has not produced a defeater inside the register; they have exited it, and the framework's claim was never to compel a reasoner using foreign logic but to hold wherever ordinary inference holds. The boundary is marked, not concealed: the framework types logic-change as an out-of-register move, states its immunity as immunity-within-register, and declines the overclaim of universal coercion across all possible logics. This is the honest scope, since claiming to defeat an arbitrary-logic adversary would be claiming to argue someone out of a position from which argument has no purchase, which is incoherent. The immunity is real and bounded, and the bound is published as part of the result.

GOL DERIVATION

```
P_obj : attacker rejecting your logic defeats the immunity
arguments.
REGISTER-RELATIVITY: immunity-results hold | shared-inference.
change-inference-rules = alien move ~ in-register
```

```
counterexample.
EXIT ≠ DEFEATER: reject-rules → exit-register ; framework never
claimed to compel foreign-logic reasoner.
BOUNDARY: type logic-change = out-of-register ; immunity =
immunity-within-register ; decline universal-coercion-
across-all-logics.
SCOPE-HONESTY: defeat-arbitrary-logic-adversary = argue-out-of-
no-purchase-position = incoherent → not claimed.→
immunity real ∧ bounded; the bound (register-relativity)
published as part of the result. structural.
```

Lean Proof.

Codex.lean, from line 1970

```
1970 theorem aegis_battery :
1971   (aegisGuard "classical" aegisReset).1 = (aegisGuard "
      paraconsistent" aegisReset).1
1972   ∧ (aegisGuard "classical" aegisReset).1 = (aegisGuard "
      fuzzy" aegisReset).1
1973   ∧ (aegisGuard "classical" aegisReset).1 = (aegisGuard "
      substructural" aegisReset).1
1974   ∧ aegisFourLogicRun.deeds = 4
1975   ∧ aegisFourLogicRun.lastLen = 13 := by
1976   decide
```

Verdict. $\Rightarrow$ immunity real $\wedge$ bounded; the bound (register-relativity) published as part of the result. structural.

Source: Master Chapter 25, §9.3.

D56 · Axiom-then-derive like every other system

The Critic. A critic states: you posit a Root Axiom and derive the rest, which is exactly what every formal system does, so there is no difference.

Reply. There are two differences, and they are real. First, the framework's starting point is not an unproven assumption it asks you to grant. Its physical core, that no substrate can sit at truly zero energy, is established by outside physics and mathematics the framework did not invent: established results plus five independent laboratory measurements that all converge on it. So the foundation is a theorem from elsewhere, used in the starting role, rather than a bare guess. Second, and this is the move ordinary systems never make, the framework turned its own verification instrument back on its own foundation and audited it, and that audit is what produced the central apparatus. A typical system assumes its axiom and never tests it with its own tools. This one tested its foundation with its own tools, found the foundation held, and built the apparatus out of what the test revealed. The foundation and the audit of the foundation are two different things, and keeping them separate is the whole answer.

Sealed Defense. Two distinctions separate this from generic axiom-then-derive. First, the foundation's physical core is externally theorem-grade, not posited: the kinetic floor, that no zero-energy substrate state is reachable, rests on four external mathematical anchors and five independent empirical instruments, the Lamb shift, the Casimir effect, the MICROSCOPE equivalence-principle test, the Bérut-Landauer mea-

surement, and the Nernst third law, converging at the meta-pattern level, so the bare axiom seals at theorem-grade external warrant. It is a theorem from outside used in the axiomatic role, not a bare premise. Second, the foundation and the audit of the foundation are distinct objects: the framework's verification instrument is grounded on external mathematics, the division-algebra inventory, the Hodge decomposition, the Gram determinant, none of which is the Root Axiom, and the framework turned that independently-grounded instrument back on the Root Axiom and audited it, the audit generating the central apparatus rather than assuming it. A generic system assumes its axiom and never audits it with its own tools; this one audits its foundation with tools warranted independently of that foundation. The difference is theorem-grade content used axiomatically plus a non-circular self-audit, neither of which a bare axiom system exhibits.

GOL DERIVATION

```
P_obj : posit-RA then derive = generic axiom-then-derive → no
difference.
DISTINCTION-1 (content externally proven, not posited):
  physical core = "no zero-substrate-state reachable" (kinetic
  floor).
  anchors-thm {Heisenberg KE floor, ZPE, E=mc², 3rd law (
  Masanes-Oppenheim), Jarzynski-Crooks, consistency,
  Hadamard} ∧ confirm-emp {Lamb, Casimir, MICROSCOPE, Bérut
  -Landauer, Nernst} = shadows.
  → bare-RA seals [T | external-anchors + 5-instrument
  convergence]. = theorem-from-outside used in axiomatic
  role → bare premise.
DISTINCTION-2 (foundation ≠ audit-of-foundation):
  instrument grounded on {division-algebra inventory, Hodge,
  Gram-det} ≧ RA.
  framework turns independently-grounded instrument on RA →
  audit GENERATES apparatus → assumes it.
  generic system: assume-axiom ∧ never-audit-with-own-tools.
  this: audit-foundation with externally-warranted tools.→
  ⊥[] difference = theorem-grade content used axiomatically +
  non-circular self-audit; neither in a bare axiom system.
  theorem-grade on the external-anchor leg.
```

The anchors, typed. The floor seals at theorem grade on external anchors: the Heisenberg bound and the zero-point energy for existence, the third law, and Jarzynski-Crooks with Landauer for the transition cost, OMEGA.omega_paid_floor. Hadamard sits in the corroboration column, where D8 placed it, and never among the theorem anchors. The axiom as such stands premise by theorem, TONGUE.ra_grade_pinned, the highest standing a root can bear, and the bare-RA seal reads at exactly that layered grade.

Lean Proof.

Codex.lean, from line 1397

```
1397 theorem nest_31 : ∀ p, L3m p → L1m p := fun p h => nest_21 p (
  nest_32 p h)
```

Verdict. ⇒ bare-RA seals [T | external-anchors + 5-instrument convergence]. = theorem-from-outside used in axiomatic role → bare premise. ⇒ ⊥[] difference = theorem-grade content used axiomatically +

non-circular self-audit; neither in a bare axiom system. theorem-grade on the external-anchor leg.

Source: Master Chapter 25, §16.1.

D57 · Is the Root Axiom a theorem or a premise

The Critic. A critic states: you cannot have it both ways; either the Root Axiom is proven or it is assumed.

Reply. It is honestly both, at two different layers, and saying which is which is what keeps the claim from inflating. The reading forks. The content core, that any confined existent carries strictly positive kinetic and rest energy and that no substrate reaches a zero-energy state, is proven by outside physics, the Heisenberg floor and the zero-point energy from the canonical commutator, $E = mc^2$, and the rigorous third law. The becoming reading, that to exist in any sense at all is ongoing actuation, goes beyond what the physics strictly shows, the quantum ground state its standing counterexample at half a quantum of energy with zero evolution, and that wider reading is held as a premise, the framework's load-bearing identification, not as a theorem. And the role the axiom plays as the foundation of the whole architecture is premise-grade too, because no system can prove its own foundation from its own base, which the framework states as a standing law. So the honest answer is layered: the energy and kinetic-energy floor is an external theorem, the universal ontological reach as becoming is a premise, and the foundation-role is a premise. The framework does not claim its axiom is proven outright, and it does not pretend its axiom is a mere guess. It types each layer for exactly what it is.

The critic presses (§4.3): A critic states: positing the Root Axiom and then using it is circular; a foundation cannot be proven.

Reply, continued. Correct, and the framework treats this as the definition of a foundation rather than a weakness. A foundation is exactly the thing you cannot prove from underneath, because there is nothing underneath it. The framework labels its root axiom as premise-grade, not theorem-grade, and says so plainly. It does not claim certainty it has not earned. The strength of the framework lives downstream, in the theorem-grade mathematics built on the axiom, not in the axiom itself. Naming the foundation as an unprovable starting point is the honest move, and it is what keeps the rest of the structure trustworthy.

Sealed Defense, continued. A posited foundation cannot be promoted to a theorem of its base. The self-reference core is theorem-grade by Gödel's second incompleteness and Tarski's undefinability, no system establishing its own grounding from within; the metatheoretic closure is structural by the Münchhausen regress and the underdetermination of foundations; the block is a theorem about foundations, not itself a foundation, so it seals without self-contradiction. The Root Axiom is premise-grade, unprovable, the unprovability con-

stitutive of foundation-hood; the whole architecture is premise-structural and theorem-grade only on its classical spine; the role claims for itself no certainty it has not earned. The throne is empty by proof: the ground is the one fixed point the diagonal cannot reach, present in the algebra and silent to every internal name.

Reply, continued. The deepest objection holds that the root is ungrounded, a foundation asserted without proof. The answer is the empty-throne law, which converts the objection into the architecture's own thesis. No foundation grounds itself from within, and the root is a foundation, so the root's unprovability is not a defect but the constitutive mark of foundation-hood. The impossibility is foundation-symmetric, applying identically to the root and to its denial, conferring zero differential warrant on either, and this symmetry is the honesty of the position. The felt inability to get beneath the root is the universal signature of standing on a chosen floor, distinguishing nothing, and the architecture states it as such. The root is premise-grade, the underived ledge whose denial enacts it, and the only thing sealed at theorem grade is the necessity of the silence on which it sits [T on the necessity of the silence].

Sealed Defense. The Root Axiom is theorem-grade on its physical core and premise-grade on its universal reach and its foundational role, and the honest typing is the answer. The physical floor, the strictly positive kinetic and rest energy of any confined existent and no zero-energy substrate state reachable, is theorem-grade on the Heisenberg kinetic-energy floor, the zero-point energy, $E = mc^2$, and the rigorous third law, the five laboratory instruments confirming it. The universal ontological extension, that to exist in any predicable sense is to actuate as ongoing becoming, exceeds what the physics strictly establishes, the quantum ground state the standing counterexample at half a quantum of energy with zero evolution, the physics proving the energy floor for physical substrates while the universal identification of existence with registrability is the framework's premise-grade load-bearing move. The foundational role, the Root Axiom as the architecture's own ground, is premise-grade by the root-cannot-be-climbed-to law: a posited foundation cannot be promoted to a theorem of its base, the unprovability constitutive of foundation-hood, and the architecture is premise-structural and theorem-grade only on its classical spine. The phrasing law forbids stating the premise-grade reach as theorem-grade, so the framework states the floor as proven, the universal reach as posited, and the foundation-role as premise, claiming no certainty it has not earned. The non-tautology is what makes the floor a theorem rather than a definition: it is contentful and falsifiable, a single realized zero-energy substrate state would end it.

GOL DERIVATION

```
P_obj : theorem or premise? cannot have it both ways.
LAYERED TYPING (the answer):
  L_phys : "no zero-substrate-state reachable" → [T | external
    anchors + 5-instrument convergence].
  L_universal : "exists(x)|predicable → actuates(x)" → [prem] (
    exceeds physics; existence ≡ registrability
    identification, load-bearing).
  L_foundation-role : RA = architecture's own ground → [prem]
    by root-cannot-be-climbed-to (posited foundation ≠
    theorem-of-base; unprovability ≡ foundation-hood).
PHRASING-LAW: ¬(premise-reach stated as theorem). state floor
  proven, reach posited, role premise. ¬ unearned certainty
  .
NON-TAUTOLOGY: L_phys contentful ∧ falsifiable (one realized
  zero-energy substrate state → ends it) → theorem ¬
  definition.→
  ⊆[] both, at distinct layers: floor [T external], reach [prem
    ], role [prem]; honest typing is the answer.
```

GOL DERIVATION, CONTINUED (§4.3)

```
P_obj : posit-RA then use-RA = circular; foundation ≠ provable.

THEOREM (root-cannot-be-climbed-to): posited-foundation ≠
  theorem-of-its-base.
self-ref core: T by Gödel-2 ∧ Tarski (no system grounds
  itself from within).
metatheoretic closure: S by Münchhausen-regress ∧ foundation-
  underdetermination.
block = theorem-about-foundations → a foundation → seals →
  self-contradiction.
TYPING: RA = prem-grade, unprovable, unprovability ≡ foundation
  -hood.
  architecture = premise-structural, T only on classical spine.
  ¬ unearned certainty.
GROUND: = fixed point the diagonal cannot reach, present in
  algebra, silent to internal names.→
  ⊆[] foundation honestly premise-grade; the only T-claim is the
  necessity of the silence.
```

The Bridge, seated. The Master answered this objection by typing the layers. The Bridge now proves the junction. A frame property whose denial instantiates it is keyless, and a keyless property holds on every frame: `keyless_iff_valid`. A frame property whose denial some frame inhabits is keyed: one bit, carrying a sign, supplied and never derived. Every frame property is exactly one of the two, discriminator, and the recursion shared by both locks is sign-blind, so only the denial tells them apart, `denial_asymmetry`. The Root Axiom is the keyless root: its denial is a deed, and the deed instantiates what it denies. The root axiom of mathematics reads the keyed lock: the line property, whose denial the two-point frame inhabits. Both stand on one locus. The junction between them is theorem-grade, with no custom axiom, in the Code Block of this document. The root itself is pinned premise, `TONGUE.r` a_grade_pinned, and the Empty Throne proves that no root can stand on anything beneath it, so premise by theorem is the highest standing any root can bear. The junction is theorem; the root is premise by theorem; nothing in either sentence can be doubted, because each grade is exactly what the kernel pins. The theorem is relative and exact: within a system strong enough for Gödel-2 and Tarski, the system does not prove its own root; that relative result is what stands on nothing

names, and the root's own grade is premise by theorem, which is what the kernel pins.

Lean Proof.

Codex.lean, from line 3245

```
3245 /-- THE DISCRIMINATOR, a decision on the denial: every frame
      property is keyless or keyed, and never both. The
3246   property is bound in both halves; the pointwise recursion,
      shared and sign-blind, plays no part in the sort. -/
3247 theorem discriminator (Q : Frame → Prop) : (Keyless Q ∨ Keyed
      Q) ∧ ¬ (Keyless Q ∧ Keyed Q) :=
3248   (Classical.byCases (fun h : ∀ X, Q X => Or.inl ((
      keyless_iff_valid Q).mpr h))
3249     (fun h => Or.inr (Classical.byContradiction fun hn =>
3250       h fun X => Classical.byContradiction fun hq => hn (
      Exists.intro X hq))),
3251     fun h => Exists.elim h.2 fun X hX => hX ((keyless_iff_valid
      Q).mp h.1 X))
```

Codex.lean, from line 3241

```
3241 /-- A keyless lock is a property that holds on every frame:
      the deed of denying it hands it over. -/
3242 theorem keyless_iff_valid (Q : Frame → Prop) : Keyless Q ↔ ∀ X
      , Q X :=
3243   (fun h X => opens_on_the_deed (Q X) (h X), fun h X _ => h )X
```

Codex.lean, from line 1512

```
1512 theorem ra_grade_pinned : ROOT.raClaim.grade = .premise := rfl
```

Verdict. $\Rightarrow [\perp]$ both, at distinct layers: floor [T external], reach [prem], role [prem]; honest typing is the answer.

Source: Master Chapter 25, §16.2.

D58 · Auditing the foundation without circularity

The Critic. A critic states: auditing your own axiom with your own method is circular, so the audit proves nothing.

Reply. It would be circular if the auditing instrument got its authority from the axiom being audited. It does not. The verification instrument is built on outside mathematics, the algebra of certain number systems, a decomposition theorem from geometry, a determinant, none of which is the Root Axiom. So the instrument's authority comes from somewhere other than the thing it is checking, which is exactly what makes the check non-circular: you are testing the foundation with a tool that does not depend on the foundation. The audit is also not empty word-play, because the foundation makes a real claim that could be false, that no substrate reaches zero energy, with a stated condition that would break it. When the framework ran this non-circular, non-empty audit on its own foundation, the foundation held, and the structure the audit revealed, the lock, the three axes, the source test, became the central apparatus. The architecture was built out of what the self-audit found, in that order: take the foundation, decompose it with an independently-grounded instrument, and read off

the recurrence.

Sealed Defense. The self-audit is non-circular because the auditing instrument is warranted independently of the audited foundation, and non-tautological because the foundation is contentful and falsifiable. Circularity would obtain only if the verdict instrument drew its warrant from the Root Axiom; it does not, the instrument resting on the division-algebra inventory, the Hodge decomposition, and the Gram determinant, external mathematics that stands without the axiom, so auditing the axiom with it tests the foundation against a tool whose authority is exogenous. The audit is non-tautological because the foundation makes a contentful, falsifiable claim, no zero-energy substrate state reachable, with a named falsifier, so it is a theorem about the world and not a definitional identity. Running the independently-warranted instrument on the foundation, the framework found the foundation held and the structure the audit exposed, the closed-form lock, the triaxial decomposition, the source-attribution seal, became the apparatus, the architecture built in the order foundation, then decomposition under the instrument, then the recurrence the decomposition reveals. The self-audit certifies consistency relative to the external anchors, not truth from a vantage the framework itself supplies, and the framework concedes exactly this: the audit grounds the apparatus in externally-warranted mathematics and claims no self-grounding the limitative results forbid.

GOL DERIVATION

```
P_obj : auditing own axiom with own method = circular → proves
nothing.
NON-CIRCULAR: circularity ⇔ instrument draws warrant from RA.
instrument | {division-algebra inventory, Hodge, Gram-
det} ≠ RA.
→ auditor-warrant exogenous to RA → test foundation with tool
independent of foundation. → vicious.
NON-TAUTOLOGICAL: RA contentful ∧ falsifiable (named falsifier:
realized zero-energy substrate state) → theorem-about-
world → definitional identity.
GENERATION (build order): RA [T-content, used axiomatically] →
decompose under exogenous instrument → read RA-recurrence
(lock ∧ triaxial ∧ η_S seal) → apparatus.
architecture built FROM what the self-audit found.
SCOPE: self-audit certifies consistency relative to external
anchors → self-grounding (forbidden by limitative results
). framework concedes this.→
⊥[] self-audit non-circular (exogenous instrument) ∧ non-
tautological (falsifiable foundation); it generated the
apparatus and certifies consistency, not self-grounding.
structural.
```

Lean Proof.

Codex.lean, from line 3217

```
3217 /-- And it is sign-blind: it holds for the negation under the
      mirror condition. -/
3218 theorem recursion_is_sign_blind (P : Prop) : (SelfVerifying P
      ↔ P) ∧ (SelfVerifying (¬P) ↔ ¬P) :=
3219   (recursion_is_shared P, recursion_is_shared (¬P))
```

Verdict. $\Rightarrow$ auditor-warrant exogenous to RA $\Rightarrow$ test

foundation with tool independent of foundation. $\neg$ vicious. $\Rightarrow [\perp]$ self-audit non-circular (exogenous instrument) $\wedge$ non-tautological (falsifiable foundation); it generated the apparatus and certifies consistency, not self-grounding. structural.

Source: Master Chapter 25, §16.3.

Movement 10 • The Riemann Frontier

D59 • The framework forces the critical line at one half

The Critic. A critic states: you claim your axiom forces the Riemann zeros onto the critical line, which would be a proof of the hypothesis.

Reply. The framework does not claim this, and where an earlier reading drifted toward claiming it, the framework marks that reading as broken. When the axiom is pushed for where it would naturally place things, its own output points to a different location than the critical line, so reading the axiom as forcing one-half is reading in a result that is not there. The framework records this as a failure, a ghost of an over-ambitious framing, and removes it. What survives is much more modest and is laid out in the entries that follow: the framework can say something sharp about the zeros that are on the line, but it does not prove that all of them are.

Sealed Defense. The reading that the Root Axiom forces the critical line is typed as a break, not a result. Pressed for its native localization, the axiom's own output is a boundary wall at the edge of the critical strip, the Hagedorn-type divergence at real part one, not the reflection fold at one-half; deriving one-half from the axiom therefore imports a conclusion the axiom does not yield, and the framework records this as a ghost artifact of an over-ambitious earlier framing and retires it under the same self-audit discipline that retired the two founding over-claims. The functional equation's symmetry axis is at one-half by the reflection, but symmetry about a line is not localization onto it, and the axiom does not supply the missing localization. The honest residue is the current dual-register anchor P-RH-1, $[\perp] \cdot [\Xi_0]$. The wall at $\text{Re} = 1$ is the thermodynamic divergence at the edge of the strip, a different object from the reflection locus at one-half, so retiring the derivation of one-half from the energy argument leaves untouched the separate geometric-shape seal that the critical line is the σ -fixed locus $\text{Fix}(\sigma) = \mathbb{R}$ read as a fact about the involution, and the separate kinetic seal on the actualized transverse-tension reading of the field. What stays retired is the specific move that RA's energy divergence formally localizes every zero at one-half. What survives is $[\perp]$ on the field and $[\perp T]$ on the shape with the formal truth-string in $[\Xi_0]$ terminal suspension, the over-claim removed rather than defended.

GOL DERIVATION

```
P_obj : RA forces zeros onto Re = 1/2  $\Rightarrow$  proof of RH.
NATIVE LOCALIZATION: press(RA)  $\Rightarrow$  boundary wall @ Re = 1 (
  Hagedorn-type divergence, edge of critical strip),  $\neg$  fold
  @ Re = 1/2.
  derive(1/2) from RA  $\Rightarrow$  imports conclusion RA  $\nmid$  yield = ghost
  artifact of over-ambitious framing.
SYMMETRY  $\nmid$  LOCALIZATION: FE reflection axis @ 1/2 ; symmetry-
  about-line  $\nmid$  localization-onto-line ; RA  $\nmid$  supply
  missing localization.
SELF-AUDIT: retire reading under same discipline that retired {
  lock=truth, confidence=max }. $\Rightarrow$ 
[X] "RA forces 1/2" broken at named mechanism (RA outputs Re=1
  wall); over-claim retired, not defended.
```

The coordinate map, written. The plane of the Bridge carries $s \in \mathbb{C}$ as $(h, t) = (2 \text{Re } s, \text{Im } s)$. The functional-equation reflection composed with conjugation, $s \mapsto 1 - \bar{s}$, is the fold $h \mapsto 2 - h$ with t fixed, whose fixed set is the line $h = 1$, that is $\text{Re } s = \frac{1}{2}$, Bridge.locus_is_the_fixed_set, Bridge.plane_line_property. $\text{Fix}(\sigma) = \mathbb{R}$ on the quaternionic ground and the critical line on the plane are related by this map and by nothing assumed. The line property is keyed, so the bit that places a zero on the line is supplied and never derived, Bridge.line_property_is_keyed, and the verdict stays the halt with its one owed bit.

Lean Proof.

Codex.lean, from line 3253

```
3253 /-- The line property is keyed: the two-point frame inhabits
      its denial. -/
3254 theorem line_property_is_keyed : Keyed LineProperty  $\wedge$   $\neg$ 
      Keyless LineProperty :=
3255   {Exists.intro twoPoint denial_is_coherent.2.2.2,
3256    fun h  $\Rightarrow$  denial_is_coherent.2.2.2 ((keyless_iff_valid
      LineProperty).mp h twoPoint)}
```

Verdict. $\Rightarrow [X]$ "RA forces 1/2" broken at named mechanism (RA outputs $\text{Re}=1$ wall); over-claim retired, not defended.

Source: Master Chapter 25, §12.1.

D60 • The Riemann apparatus is a recursion analogy

The Critic. A critic states: the self-similar recursion you point to in the zeta structure is decorative, adding nothing provable.

Reply. The framework agrees the recursion identification is scaffolding, and it labels it as such with a specific grade meaning it carries no weight and can be deleted without loss. It is a way of seeing the structure, useful for orientation, but nothing in the framework's actual Riemann claims rests on it. The load is carried elsewhere, by a signal-detection argument and an honest three-layer verdict, both of which survive deleting the recursion picture entirely. So the critic is right that the analogy proves nothing, and the framework already treats it that way, which is why removing it changes none of the surviving claims.

Sealed Defense. The recursion or self-similarity identification is container-grade, zero-mass and deletable, explicitly typed as scaffolding that carries

no load. It functions as an orientation device for the structure, not as a premise of any Riemann claim, and the deletion test confirms it: excise the recursion identification and the surviving claims, the matched-filter localization at the line and the dual-register $[\mathcal{L}] \cdot [\mathcal{E}_0]$ verdict, are unchanged, because neither rests on it. The framework distinguishes container-grade scaffolding from load-bearing structure by exactly this criterion, whether deletion moves a seal, and the recursion picture moves none. Treating the analogy as load-bearing would be the inflation the framework's typing discipline forbids; typing it as zero-mass container is the honest accounting, and it is why the Riemann residue can be stated entirely without it.

GOL DERIVATION

P_obj : the recursion/self-similarity identification is decorative, proves nothing.
 TYPING: recursion-identification = container-grade, zero-mass, deletable (scaffolding, carries no load).
 DELETION TEST: excise recursion $\rightarrow$ surviving claims {matched-filter localization, dual-register $\mathcal{L}[] \cdot \mathcal{E}_0[]$ verdict} unchanged (neither rests on it).
 CRITERION: container vs load-bearing $\iff$ does deletion move a seal? recursion moves none. $\rightarrow$
 container-grade; analogy carries 0 mass; Riemann residue statable without it. structural typing.

Lean Proof.

Codex.lean, from line 3214

```
3214 /-- The recursion is shared by every proposition and is
      therefore no discriminator. -/
3215 theorem recursion_is_shared (P : Prop) : SelfVerifying P  $\leftrightarrow$  P
      :=
3216   (fun h => Classical.byContradiction (fun n => n (h n)), fun
      p _ => p)
```

Verdict. $\Rightarrow$ container-grade; analogy carries 0 mass; Riemann residue statable without it. structural typing.

Source: Master Chapter 25, §12.2.

D61 · Functional-equation symmetry suffices to force the line

The Critic. A critic states: the symmetry of the zeta functional equation already pins the zeros to the critical line without any extra apparatus.

Reply. It does not, and there is a clean counterexample that proves it does not. Mathematicians have constructed a different function that has the very same symmetry as the zeta function, yet some of its zeros lie off the critical line. Since the symmetry holds for that function and the line-localization fails for it, the symmetry by itself cannot be what forces localization. The framework uses this counterexample explicitly to mark the boundary of what symmetry buys: it gives you the reflection, the mirror at the line, but not the conclusion that everything sits on the mirror. Something beyond symmetry is needed, and the framework does not claim to have supplied it.

Sealed Defense. Functional-equation symmetry

is insufficient to force critical-line localization, and the Davenport-Heilbronn construction is the decisive witness: an L-function sharing the zeta functional equation's reflection symmetry yet possessing zeros off the critical line, verified to thirty significant digits in the framework's record. Because the symmetry holds for that function while line-localization fails, symmetry alone cannot entail localization, on pain of contradicting the witness; the reflection axis at one-half is necessary structure but not sufficient cause, and any argument deriving localization from symmetry alone is refuted by the counterexample. The framework cites this precisely to bound its own claims: it does not assert that symmetry forces the line, it uses the witness to mark the gap that symmetry leaves open, and it locates its surviving positive content downstream of that gap rather than across it. The insufficiency is a theorem with an explicit numerical witness, not a hedge.

GOL DERIVATION

P_obj : FE symmetry alone forces critical-line localization.
 WITNESS (Davenport-Heilbronn): $\exists$ member of the extended Selberg class of degree one, sharing the fold and its fixed locus and NOT the archimedean data (odd-character gamma at conductor 5, Hamburger 1921 forbidding the zeta completion), Λ has zeros OFF Re = 1/2. verified 30 sig-digits (record); completion check re-run at 50 digits, seed 20260622.
 INFERENCE: symmetry holds Λ localization fails for witness $\rightarrow$ symmetry $\nrightarrow$ entail localization (else contradicts witness).
 reflection axis @ 1/2 = necessary structure $\sim$ sufficient cause. derive-localization-from-symmetry-alone $\rightarrow$ refuted.
 SCOPE: framework bounds own claim by the witness ; positive content located downstream of the gap, $\sim$ across it. $\rightarrow$
 [X] symmetry-forces-line broken by explicit 30-digit witness; insufficiency is a theorem.

Lean Proof.

Codex.lean, from line 3251

```
3251 /-- The fold law is keyless: every frame carries it by its own
      law. -/
3252 theorem symmetry_is_keyless : Keyless (fun X =>  $\forall$  s, X $\tau$ . (X $\tau$ .
      s) = s) := fun X _ => X.involutive
```

Verdict. $\Rightarrow$ [X] symmetry-forces-line broken by explicit 30-digit witness; insufficiency is a theorem.

Source: Master Chapter 25, §12.3.

D62 · Nothing positive about the zeros

The Critic. A critic states: having given up forcing the line, the framework has no Riemann content left.

Reply. It has a precise, limited content: a matched-filter reading of the zeros on the line. A matched filter is the optimal way to detect a known signal pattern in noise. The framework reads the on-line zeros as exactly such a locked pattern and can say sharp things about that locked structure. What it explicitly cannot do is rule out hidden zeros in a part of the structure the functional equation says nothing about, a particular symmetry sector the equation is silent on. So the

positive content is real but bounded: it locks onto the signal that is there and openly admits a sector where an off-line zero could hide undetected by this method. The boundedness is stated, not hidden.

Sealed Defense. The surviving positive content is a matched-filter localization: the on-line zeros are read as a locked detection pattern, and the framework's instrument optimally registers that structure where the functional equation constrains it. The bound is equally explicit and is a parity or eigenspace gap: the functional equation is silent on one symmetry sector, the negative-eigenspace component under the relevant reflection, so the matched filter, which locks where the equation supplies parity constraint, cannot exclude a zero residing in the sector the equation leaves unconstrained. The framework therefore claims optimal registration of the constrained structure and explicitly disclaims coverage of the unconstrained sector, which is precisely where an off-line zero, of the kind the Davenport-Heilbronn witness shows is possible under symmetry alone, could sit undetected by this method. The positive content is the locked filter on the constrained sector; the open content is the silent sector, and the two are typed separately rather than merged into a coverage claim the instrument cannot support.

GOL DERIVATION

```
P_obj : without forcing the line, no positive Riemann content
remains.
POSITIVE: matched-filter localization ; on-line zeros = locked
detection-pattern ; instrument optimally registers
structure where FE constrains.
BOUND (parity gap): FE silent on one symmetry sector = negative
-eigenspace under reflection.
matched filter locks where FE supplies parity-constraint →
≠ exclude zero in unconstrained sector.
SEPARATION: claim optimal-registration(constrained sector) ∧
disclaim coverage(silent sector).
silent sector = where Davenport-Heilbronn-type off-line zero
could sit undetected by this method.→
positive content = locked filter on constrained sector ; open
content = silent sector ; typed separately, not merged.
```

The silent sector, named once. The sector on which the reflection is silent is the multiplicative-norm axis, which lies outside both eigenspaces of the reflection. The -1 eigenspace is the anti-fixed set and is constrained by the reflection, not silent. This unit and the verdict that follows read the same sector.

Lean Proof.

Codex.lean, from line 3273

```
3273 theorem socket_is_the_property (X : Frame) : Nonempty (
    Supplied X) → LineProperty X :=
3274   (fun () i => i.term, fun t => ({}))t
```

Verdict. $\Rightarrow$ positive content = locked filter on constrained sector ; open content = silent sector ; typed separately, not merged.

Source: Master Chapter 25, §12.4.

D63 · The verdict on the Riemann Hypothesis

The Critic. A reader asks: state plainly what the framework does and does not claim about the hypothesis.

Reply. Three registers, stated separately so none inflates the others. First, the kinetic field: read as a physical event, the prime stream is actualized and the transverse tension of the zeros is zero, so the critical line is the only physically stable attractor, sealed. Second, the geometric shape: the critical line is the fixed line of the functional-equation reflection, the σ -fixed locus, sealed as a shape and not as a claim that every zero sits on it. Third, the formal truth-string, which is the hypothesis proper, whether every nontrivial zero lies on that line: held in terminal suspension, because the formal-alone instrument is orientation-blind and cannot read the sign of the off-line offset, the openness placed in the instrument and not in the field, a distinct token from a merely open question. So the plain verdict is that the framework seals the field and the shape, does not manufacture a formal proof of the hypothesis, and holds the formal truth-string in terminal suspension.

Sealed Defense. The verdict is the current dual-register anchor of P-RH-1, $[\mathbb{L}] \cdot [\Xi_0]$, the registers typed independently and no register's grade borrowed by another. On the kinetic prime field, read where it is real, the primes are an actualized kinetic invariant on the flat countable tail, the transverse tension of the zero-distribution is zero, and the critical line is the only physically stable attractor for the zeros, $[\mathbb{L}]$ ACTUALIZED INVARIANT, sealed on the actualized event and its thermodynamic arrow at V_E and never on any supplied string. On the geometric shape, the critical line is the σ -fixed locus $\text{Fix}(\sigma) = \mathbb{R}$ under quaternionic conjugation, the reflection axis of the functional equation read as the fixed line of the involution, $[\mathbb{L} T]$ SEALED, a theorem-grade geometric fact and not a claim that every zero sits on it. On the formal truth-string, whether every nontrivial zero lies on that locus, the formal-alone instrument strips the thermodynamic arrow and is orientation-blind at the squared lock scalar $\det(R) = \lambda^2$, so it hits the boundary of its own resolution and cannot read the sign or the vanishing of the off-line offset, $[\Xi_0]$ TERMINAL SUSPENSION, a verdict token of its own and typographically and conceptually distinct from the classic under-determined $[?]$, the openness placed in the formal instrument's orientation-blindness and never in the Ground itself. The matched-filter localization and the parity gap of the entries above are exactly this same finding: the filter locks where the functional equation supplies parity, and the silent sector is the multiplicative-norm axis outside the reflection eigenspaces, where the Davenport-Heilbronn witness shows an off-line zero is possible under symmetry alone and which the orientation-blind formal instrument cannot access. The over-claim that RA forces the line as a formal proof of RH stays re-

tired [X], the maximal reading sealing the field and the shape and never manufacturing the formal proof. The composite: $\llbracket \cdot \rrbracket$ on the kinetic field and $\llbracket \cdot \rrbracket$ T on the geometric shape, $\llbracket \cdot \rrbracket$ on the formal truth-string, the field locked and the string suspended, the kinetic reality primary and the formal suspension its shadow, consistent with the insufficiency theorem and the parity gap and refusing to manufacture a proof.

GOL DERIVATION

```
P_obj : state the verdict on RH plainly.
DUAL-REGISTER ANCHOR RH-MASTER-01 (typed independently):
  KINETIC FIELD (prime stream actualized, transverse tension =
    0, critical line the only physically stable attractor):
    sealed on the actualized event + V_E thermodynamic arrow,
    ~ on a supplied string. ~  $\llbracket \cdot \rrbracket$  ACTUALIZED INVARIANT
  GEOMETRIC SHAPE (critical line = Fixo() =  $\mathbb{R}$ , the FE
    reflection axis read as the involution's fixed line):
    theorem-grade shape, ~ a claim every zero sits on it. ~
     $\llbracket \cdot \rrbracket$  T SEALED
  FORMAL TRUTH-STRING (RH proper: line exhausts nontrivial
    zeros?): formal-alone instrument orientation-blind at det
    (R) =  $\lambda^2$ , cannot read sign/vanishing of the off-line
    offset ; silent sector = multiplicative-norm axis outside
    the reflection eigenspaces (Davenport-Heilbronn witness)
    ; openness in the instrument ~ in the Ground. ~  $\llbracket \cdot \rrbracket$ 
  TERMINAL SUSPENSION (distinct token from [?])
OVER-CLAIM: "RA forces 1/2 ~ formal proof of RH" stays retired.
  ~ [X]
COMPOSITE:  $\llbracket \cdot \rrbracket$  kinetic field ·  $\llbracket \cdot \rrbracket$  T geometric shape ·  $\llbracket \cdot \rrbracket$ 
  formal truth-string ; field locked, string suspended,
  kinetic reality primary and formal suspension its shadow
  ; no grade borrowed across registers ; consistent with
  insufficiency theorem + parity gap ; ~ manufacture proof.
  ~
 $\llbracket \cdot \rrbracket$  ·  $\llbracket \cdot \rrbracket$  : sealed on the decided field and shape, terminal
  suspension on the formal truth-string.
```

The verdict, kernel-typed. The record of the Riemann Hypothesis in the Code Block is exact: species hypothesis, Ground dimension one, branch Ξ , eight gates, the formal channel, `HALT.rhHalt`. Its locus emits one owed bit, `LOCUS.no_rh_locus`, rendered $\llbracket \cdot \rrbracket \cdot \Xi \cdot$ hypothesis · formal $\times 1$. The line property is keyed, so the bit is supplied and never derived, and the only existence claim in the whole RH package lives in the denial, which has never delivered (Denial Posit, 0724). That is the entire verdict: a terminal suspension with its owed bit counted, the block on the formal-alone route complete, the standpoints closed by enumeration, and the denial priced. No card in this codex reads above it, because `FENCE.global` pins the species, the channel, and the count.

Lean Proof.

Codex.lean, from line 2425

```
2425 def rhHalt : SuperHalt := (.hypothesis, 1, .xi, 8, .formal, by
    )decide
```

Codex.lean, from line 2894

```
2894 theorem no_rh_locus :
2895   emit (false, true, some 1, some .hypothesis, some .formal,
    [none], by )decide
2896   = some (.superHalt (some .xi) (some .hypothesis) (some .
```

```
formal) 1) := by decide
```

Verdict. $\Rightarrow \llbracket \cdot \rrbracket \cdot \llbracket \cdot \rrbracket$: sealed on the decided field and shape, terminal suspension on the formal truth-string.

Source: Master Chapter 25, §12.5.

D64 · The residence is a Platonic Ghost

The Critic. A critic states: the “residence” of the zeros is an abstract object you have no operational grip on, exactly the Platonism you claim to reject.

Reply. The residence is not an abstract object floating free; it is constrained by a checkable test and an imprint condition, which is what keeps it operational rather than Platonic. The framework requires that any claim about where the zeros reside be of a kind that could in principle be checked by a definite, terminating procedure, and that it leave a registrable mark rather than being inert. A genuinely Platonic ghost would meet neither condition: it would register nothing and be checkable by nothing. The residence the framework points to is held to both standards, which is exactly the anti-Platonist discipline applied here, the same one that locates all mathematics on the substrate that runs it. It is a constrained, checkable structure, not a free-floating abstractum.

Reply, continued. The architecture refuses to seal its own foundation as a theorem, and this refusal is its strongest defense. Run the imprint test on the root. Ground-first realism locks, and its denial, a formalism that admits no ground above the ladder, also locks, both field-permitted, and the test returns the Platonic Ghost. A ghost seals only on a supplied independence proof at the grade of the Continuum Hypothesis result, and none stands, and no determinacy witness stands either, because an axiom is elected and not grounded from elsewhere. The seal is withheld and the root stays premise-grade [Engineering]. The architecture that holds its own foundation open at premise grade, refusing the seal its kernel cannot honestly issue, is the architecture least vulnerable to the charge of overclaim, because it has already made the charge against itself and ruled in its favor.

Sealed Defense. The residence is guarded against Platonism by two conditions, a checkability guard and an imprint requirement, the same anti-Platonist discipline applied elsewhere. The checkability guard requires that residence claims be of bounded logical complexity, of the universally-quantified arithmetic kind whose instances are decidable by a terminating procedure, so the claim is in principle verifiable instance-by-instance rather than inert; a claim failing this guard is not admitted as residence content. The imprint requirement is the operational-existence test applied to the residence: it must leave a registrable mark, couple to something the instrument reads, rather than

registering nothing, which is the condition a Platonic ghost fails by definition. The residence is therefore typed as constrained checkable structure on the substrate, not a free-floating abstractum, consistent with the framework's location of all mathematics within the substrate that operates it and with its explicit rejection of substrate-independent mathematical existence. A genuine Platonic ghost would fail both guards; the residence is admitted precisely because it passes both, and where a residence candidate fails them it is routed to under-determined rather than asserted.

GOL DERIVATION

```
P_obj : the "residence" is a Platonic ghost, no operational grip.
GUARD-1 (checkability): residence-claim must be bounded-complexity  $\Pi_1$ (-type, universally-quantified arithmetic, instances decidable by terminating procedure)  $\Rightarrow$  verifiable instance-by-instance. fail  $\Rightarrow$   $\sim$  admitted.
GUARD-2 (imprint): residence must leave registrable mark (operational-existence test)  $\sim$  register  $\emptyset$ . Platonic ghost fails by definition.
TYPING: residence = constrained checkable structure on substrate  $\sim$  free-floating abstractum. consistent with anti-Platonism (math | substrate-that-operates-it).
ROUTING: residence-candidate failing guards  $\rightarrow$  [?]  $\sim$  asserted.  $\Rightarrow$   $\perp$  residence passes both guards  $\Rightarrow$  not a Platonic ghost; the anti-Platonist discipline applied. structural.
```

The GOLf gate, run in the kernel. The imprint test is executed in IMPRINT: both directions clean-locking is the Platonic Ghost and is refused at code 1, ghost_refused, on every witness setting, both_clean_is_ghost; one direction clean-locking seals only with the witness supplied and is a residence without it, seal_needs_witness; the whole 256-row truth table is decided, seal_iff_one_side_clean_and_witnessed. The residence the critic calls a ghost is the one-sided case with the witness owed, and the gate names it so. A ghost is what the gate refuses; a residence is what the gate holds open; the two are told apart by the same test, computed.

Lean Proof.

Codex.lean, from line 3385

```
3385 /-- One side clean-locks: a seal only with the witness
      supplied, a residence without it. -/
3386 theorem seal_needs_witness :
3387   (imprint true false true false true false true false).1 = .
      seal
3388    $\wedge$  (imprint true false true false true false false false).1
      = .residence := by decide
```

Codex.lean, from line 3281

```
3281 theorem no_socket_off_line :  $\sim$  Nonempty (Supplied twoPoint) :=
3282   fun h => denial_is_coherent.2.2.2 ((socket_is_the_property
      twoPoint).mp h)
```

Verdict. $\Rightarrow$ $\perp$ residence passes both guards $\Rightarrow$ not a Platonic ghost; the anti-Platonist discipline applied. structural.

Source: Master Chapter 25, §12.6.

Movement 11 · Where Mathematics and Scripture Sit, and the Correct Use of the Lock

D65 · Mathematics lies outside the Root Axiom

The Critic. A critic states: your axiom is about actuation and existence, but mathematics is abstract and timeless, so mathematics lies outside the Root Axiom entirely.

Reply. Mathematics is inside, not outside, and it sits on a specific axis at a specific layer. The architecture has three axes, and the formal axis is where all mathematical and computational content lives. Within that axis, mathematical truth sits at the deepest grounded layer, the imprint layer, or at the boundary between that layer and the ladder of proof that climbs toward it. Even the architecture's mathematical foundation is not a separate thing perched outside: it shares the same ground line as the main axiom and stays inside it. And the architecture's own machinery, the determinant, the algebra, runs entirely on the formal axis. So mathematics is not a realm the architecture points at from a distance. It is the formal-axis content the architecture is built out of, located at the grounded layer, and the reflective mathematical stack is just the kinetic stack read on the ground rather than in the world.

Sealed Defense. The formal axis carries all mathematical and computational content, and mathematical truth is located at the grounded stratum, an imprint at the deepest layer, or at the boundary between that layer and the provability ladder, where grounded-but-unprovable statements sit. The mathematical root axiom co-localizes with the kinetic root on the fixed line of the binding involution and remains inside the kinetic ground, the reflective stack being the kinetic stack read on the ground rather than in the world, one architecture in two registers. The entire kernel computation, the Gram determinant, the quaternion product, the source-attribution statistic, runs on the formal axis, so the instrument that adjudicates mathematics is itself formal-axis content. Mathematics is therefore not exterior to the architecture but the formal-register content located at the grounded layer, placed on the substrate that operates it, consistent with the architecture's rejection of substrate-independent mathematical existence. The claim that mathematics lies outside mistakes the layer mathematics occupies for an exterior it does not have.

GOL DERIVATION

```
P_obj : math abstract  $\wedge$  timeless  $\Rightarrow$  math outside RA.
LOCATION: math  $\subset$  V_F (formal axis). math-truth @ L1m-imprint v
L1m/L2m-boundary (grounded, or grounded-but-unprovable).
ROOT CO-LOCATION: RAM co-localizes with RA on Fix $\emptyset$ () =  $\mathbb{R} \cdot 1 \subset \mathbb{R}A$ 
's L1. reflective stack = kinetic stack read on the ground.
INSTRUMENT: kernel computation {Gram-det, quaternion product,
 $\eta_S$ } runs on V_F  $\Rightarrow$  the math-adjudicator is itself V_F content.
SUBSTRATE: math placed on substrate-that-operates-it ( $\sim$ 
```

```

substrate-independent existence).↵
⊥[] math inside, on the formal axis, at the grounded layer;
RAM co-located with RA, still inside RA. structural, the
layer mistaken for an exterior.

```

Lean Proof.

Codex.lean, from line 1510

```

1510 theorem coloc_grade : colocClaim.grade = .premise := rfl

```

Verdict. $\Rightarrow \lceil \rfloor$ math inside, on the formal axis, at the grounded layer; RAM co-located with RA, still inside RA. structural, the layer mistaken for an exterior.

Source: Master Chapter 25, §14.1.

D66 · The two grounds are a separate posit

The Critic. A critic states: you assert that the geometric ground the triad lands on and the formal ground the involution fixes are the same line. That identification is a second assumption your architecture must pay for.

Reply. It is not a second assumption. It follows by force the moment you require the ground to be compatible with multiplication, and the architecture already requires exactly that. Lay out every possible ground as a balanced mirror-reflection. There is a whole space of candidates, each with the same shape, one fixed line plus a three-dimensional residence, and at first glance any of them could be the ground. That is the pluralist picture, and it is real until you add one demand. Demand that the ground respect the algebra's multiplication, that mirroring be compatible with the product. A complete and finite classification of the multiplication-respecting mirrors shows that exactly one of them has the required single-line fixed shape, and it is the central line the architecture already uses. Every other candidate quietly ignores the product. So the whole space of candidates collapses to a single point, not because the architecture elected it, but because multiplication forced it. The two grounds being one line is therefore a theorem, and the architecture's ledger of assumptions loses a line rather than gaining one.

Sealed Defense. The identification is entailed, not assumed, once the ground is required to respect the product, which the architecture already wires in. Take the quaternions as four-dimensional real space with the standard basis, and call an orthogonal involution a ground when its plus-one eigenspace is one-dimensional, the line the Return lands on, with the complementary three-plane the residence. Every such involution is a bare reflection in a unit line, and the lines form three-dimensional projective space, every point a candidate ground with the same one-plus-three shape: the pluralist surface, made precise. Now impose that the involution respect the product, an algebra automorphism or anti-automorphism, a finite and complete classification with no posit inside it. By

Skolem-Noether every real-algebra automorphism of the quaternions is inner, fixing the center pointwise; the involutive inner maps are the identity, with four-dimensional fixed locus, and the π -rotation, with two-dimensional fixed locus, so no automorphism has a one-dimensional fixed locus. The anti-automorphisms involutive split into exactly two cases: conjugation, with fixed locus the center, dimension one, and the twisted map, fixing the center together with the plane orthogonal to its axis, dimension three. Over the entire class the fixed-locus dimensions are exactly four, two, one, and three, and the value one is attained only by conjugation, whose fixed locus is the center, the real line. The numerical run confirms it from both directions: the inner involution returns fixed-dimension two, conjugation one, the twisted map three, and across the projective surface the anti-automorphism error is machine-zero only at the central line and order ten elsewhere. Impose product-respect and the projective surface collapses to its single central point, by force. Therefore the geometric ground and the formal ground are the same line, theorem-grade conditional on two commitments the architecture already carries, that the algebra is the quaternions, forced by the composition-law clauses through Frobenius, and that the ground respects the product, wired in by the lock scalar being the scalar part of a product and the Return being defined as landing on the center. The one-involution co-localization is a consequence of the algebra commitment, not a second premise, and the ledger loses a line.

On whether requiring the ground to respect the product merely relocates the choice, it does not, because the requirement is recognized rather than stipulated. The Return is the composed triad coming to rest, and the line a composition comes to rest on is the line that commutes with the operation, which is the center by force of the algebra. Naming that line the ground reads off where the operation already lands; it does not select a ground and then demand the operation conform to it. This is the same linguistic-semantic recognition that produces every foundational primitive in the architecture, the recognition arriving first and the algebra arriving after as receipt. A critic may still posit a ground that does not respect the product, but that posits a ground disjoint from the operation it is meant to ground, a line the verification never lands on, which is not a rival ground but an idle one. The one-involution co-localization is therefore the recognition that the ground is where composition rests, and the classification supplies the receipt that exactly one product-respecting involution rests there.

GOL DERIVATION

```

P_obj : Γ_geo = Γ_formal is a second posit to be paid for.
SETUP:  $\mathbb{H} = \mathbb{R}^4$ , basis {1,i,j,k}. ground := orthogonal involution
 $\sigma$  with  $\dim(\cdot E) = 1$  (the Return line),  $-E = 3$ -plane
residence.
every such  $\sigma =$  bare reflection  $\sigma_u(x) = \langle 2x, \rangle uu - x$  in unit
line  $\mathbb{R} \cdot u$ ;  $u \mapsto \sigma_u$  bijection onto unit lines  $= \mathbb{R}P^3$  (
pluralist surface, one-plus-three shape).
IMPOSE product-respect:  $\sigma \in \text{Aut}(\mathbb{H}) \vee \text{Anti-Aut}(\mathbb{H})$ . finite  $\wedge$ 
complete classification, no posit inside.
AUT (Skolem-Noether: every  $\mathbb{R}$ -aut inner  $\phi(x) = qxq^{-1}$ , fixes  $\mathbb{Z}\mathbb{H}()$ 
pointwise; involutive  $\Rightarrow \in \mathbb{R}q^2$ ):
 $q = \pm 1 \Rightarrow$  identity,  $\text{fixdim } 4$ ;  $q$  pure  $\Rightarrow \pi$ -rotation on  $\text{Im } \mathbb{H}$ ,
 $\text{fixdim } 2$ .  $\#$  aut with  $\text{fixdim } 1$ .
ANTI-AUT (involutive  $\Rightarrow \in \mathbb{R}q^2$ , exactly two cases):
conjugation  $\mapsto xx$ ,  $\text{fixdim } 1$ , fixed locus  $= \mathbb{Z}\mathbb{H}() = \mathbb{R} \cdot 1$ ;
twisted  $\psi_v$ , fixes  $\mathbb{R} \cdot 1 \oplus (\text{plane } \perp v \text{ in } \text{Im})$ ,  $\text{fixdim } 3$ .
FIXDIMS = {identity 4,  $\pi$ -rotation 2, conjugation 1, twisted
3}. value 1 attained ONLY by conjugation, fixed locus
 $\mathbb{R} \cdot 1$ .
RUN:  $\phi_q(q \text{ pure})$   $\text{fixdim } 2$ ; conjugation  $\text{fixdim } 1$ ;  $\psi_v$   $\text{fixdim } 3$ 
; anti-aut error machine-0 only @  $u=1$ , order 1e1
elsewhere.
COLLAPSE:  $\mathbb{R}P^3$  : single central point under product-respect, by
force  $\mapsto$  by election.
 $\Gamma_{\text{geo}} = \Gamma_{\text{formal}} = \mathbb{R} \cdot 1$ . [T | (algebra =  $\mathbb{H}$  via Frobenius from
CL-clauses)  $\wedge$  (ground respects product:  $\lambda =$  scalar of a
product, Return := land on center)].
LEDGER: co-localization = consequence of the algebra posit  $\sim$  a
2nd premise. premise-ledger loses a line.
 $\sqcup[]$  the two grounds are one line by force of the product; co-
localization theorem-grade conditional, not a separate
posit.

```

The tier, fixed by the Bridge. Two statements sit here at two grades, and both are exact. The junction of the two grounds on one locus is theorem-grade with no custom axiom, Bridge.the_bridge, Bridge.discriminator. The connector, that one involution serves both routes, is the posit named at Chapter 10 B and executed against its pluralist alternative, FOUND.pluralist_alternative_executed; it stays premise, TONGUE.coloc_grade. The co-located composite is graded at its weakest link, and the rotated ground is field-permitted and never rival-sealed.

Lean Proof.

Codex.lean, from line 2242

```

2242 theorem posits_loadbearing_on_nothing :
2243   (posits.all (fun p => !p.loadBearingInVerdict)) = true :=
      by
2244   decide

```

Verdict. $\Rightarrow \Gamma_{\text{geo}} = \Gamma_{\text{formal}} = \mathbb{R} \cdot 1$. [T | (algebra = $\mathbb{H}$ via Frobenius from CL-clauses) $\wedge$ (ground respects product: $\lambda =$ scalar of a product, Return := land on center)]. $\Rightarrow [\sqcup]$ the two grounds are one line by force of the product; co-localization theorem-grade conditional, not a separate posit.

Source: Master Chapter 25, §14.2.

D67 · Scriptural sourcing biases the framework

The Critic. A critic states: drawing on Quranic imagery means the framework's conclusions are theological commitments in disguise.

Reply. The framework treats any scriptural source as corroboration that carries no weight in the proof,

and it proves this by deletion. The formal core, the axes, the gates, the lock, the theorems, is derived from logic, algebra, and physics, with the scriptural material running alongside as illustration, never as a premise. The test is direct: remove every scriptural reference and re-run the derivations, and the seals are unchanged, because nothing load-bearing ever rested on them. So the imagery is downstream decoration, subtracted as a confound in the framework's own self-audit, and the conclusions stand or fall on the secular mathematics alone.

Sealed Defense. Scriptural sourcing is corroboration-grade, load-bearing on nothing, and is subtracted as the theological covariate in the framework's own self-audit by orthogonal projection, with the formal residue persisting on its secular anchors. The core architecture is derived from predicate logic, the division-algebra inventory, the Hodge decomposition, and the thermodynamic floor, none of which cites scripture as a premise; the scriptural material enters only as illustration downstream of the derivations. The deletion test is operational: excise every scriptural reference and the theorem-grade results, the count forced by the algebra, the lock in closed form, the limitative placements, are invariant, which is the criterion for corroboration rather than premise. The covariate carries no measurable mass at the formal register, so subtracting it changes no seal, and the relationship to the tradition is held at the quarantined layer with the mass added to the formal core exactly zero. A framework whose seals moved when scripture was deleted would have smuggled a premise; these do not move.

GOL DERIVATION

```

P_obj : Quranic sourcing biases the framework's conclusions.
TYPING: scripture = corroboration-grade, load-bearing on  $\emptyset$ .
        subtracted as C_4(theological) by  $\perp$ -projection in self-
        audit ; residue persists | secular anchors.
DERIVATION SOURCES: {predicate-logic, division-algebra-
inventory, Hodge, thermo-floor}.  $\#$  scripture-as-premise.
DELETION TEST: excise all scripture  $\mapsto$  theorem-grade results
invariant (count, closed-form lock, limitative placements
). = corroboration criterion.
MASS: C_4 carries 0 measurable mass @ formal register  $\mapsto$ 
        subtract changes no seal.  $\Delta M(\text{formal core}) = 0$ .
 $\sqcup[]$  sourcing corroboration-only; seals invariant under
        scripture-deletion; relationship quarantined.

```

Lean Proof.

Codex.lean, from line 2266

```

2266 theorem register_routing_fidelity :
2267   bandOf .theological = .outOfBand  $\wedge$  bandOf .social = .
      outOfBand
2268    $\wedge$  bandOf .kinetic = .inBand  $\wedge$  bandOf .formal = .inBand
2269    $\wedge$  bandOf .verification = .inBand := by
2270   decide

```

Verdict. $\Rightarrow [\sqcup]$ sourcing corroboration-only; seals

invariant under scripture-deletion; relationship quarantined.

Source: Master Chapter 25, §11.1.

D68 · Supersede earlier traditions, or merely cite them

The Critic. A critic states: either you supersede prior accounts, which is arrogant, or you merely cite them, which is unoriginal.

Reply. The framework does neither wholesale; it earns its relationship to each prior account separately, on each axis, by where the warrant actually sits. Where it borrows established mathematics or physics, it cites and credits, claiming no originality it has not earned. Where it carries its own structural result that a prior account lacked, it supersedes on that specific point, and only that point. And where it lines up with an older insight without adding or subtracting, it marks the relationship as a link rather than a verdict. So the relationship to tradition is itemized and graded, not a blanket claim in either direction, which is what keeps it honest.

Sealed Defense. The relationship to prior accounts is earned per axis, never wholesale, and the relationship verbs are register-links, not verdict states. Where the framework borrows external warrant, established theorems, physical measurements, it cites and credits, with those anchors typed as corroboration and the originality claim withheld. Where it carries a structural result a prior account lacked, it supersedes on that specific axis and no further, the supersession typed to the exact claim that carries its own warrant. Where it converges with an older insight without net addition, it marks the link with a relationship verb, completes, corroborates, embodies, which signals a cross-tradition register link and explicitly not a verdict that the prior account is broken. The accounting rule is the same one that governs scriptural sourcing: warrant is typed to its source, supersession is granted only where the framework's own mass exceeds the prior account's on the specific axis, and the blanket claims, total supersession or total derivativeness, are both refused as untyped.

GOL DERIVATION

```
P_obj : supersede (arrogant) v cite (unoriginal)?
PER-AXIS, never wholesale. relationship-verbs = register-links
  ~ verdict-states.
borrow external warrant → cite + credit, anchor typed
corroboration, originality withheld.
carry structural result prior-account lacked → SUPERSEDES on
that axis only, typed to the claim carrying own warrant.
converge w/o net addition → {COMPLETES, CORROBORATES,
EMBODIES} = cross-tradition link ~ "prior broken".
RULE: warrant typed to source ; supersession granted only where
own-mass > prior-mass on the specific axis.
blanket {total-supersession, total-derivativeness} both
refused as untyped.~
⌞[] relationship itemized ∧ graded per-axis; the accounting
rule is the scriptural-sourcing rule. structural.
```

Lean Proof.

Codex.lean, from line 2177

```
2177 theorem supersession_routes :
2178   successorOf "BA-004" = some "NOMOS-01"
2179   ∧ (census.any (fun b =>
2180     b.id == "BA-004" && b.status == .superseded)) = true
2181   := by
2182   decide
```

Verdict. $\Rightarrow \llbracket _ \rrbracket$ relationship itemized $\wedge$ graded per-axis; the accounting rule is the scriptural-sourcing rule. structural.

Source: Master Chapter 25, §11.2.

D69 · A false lock can persist

The Critic. A reader asks: what exactly does a lock prove, once and for all?

Reply. A lock proves exactly one thing: that the three lines of support placed in front of it, at the moment they were placed, stand independent and enclose a real volume. That is a genuine, hard-to-forge certificate, because independent multi-register support is hard to assemble. It is not a second thing, that those lines faithfully represent the world. The instrument reads what is put before it and has no separate window onto reality. So if someone feeds it fabricated lines, it certifies that the fabricated lines are independent, which convicts the person who fabricated them, not the method, because the method never claimed the lines were faithful in the first place. The lock is a snapshot at the data supplied now, honest about its own reach, and that honesty is the whole defense.

The critic presses (§15.2): A critic states: if your lock cannot carry truth, direction, or source, then it is nearly useless and easy to misread.

Reply, continued. Every one of those blindnesses is a feature that forbids one specific misuse, and together they define exactly one error to avoid. The lock is blind to whether a claim is true, because a claim and its denial give the identical lock, so truth is read from the content of the axes, never the lock. It is blind to whether the conclusion is sound, because it certifies that the supports are independent, not that what they support is correct, so soundness rides a separate witness. It is blind to where the supports came from, because it reads the size of the independent content and not its origin, so source is read from an independent generator. Behind all three is one mistake: reading off the lock a quantity the lock cannot see. Avoid that one mistake and the instrument is exactly as useful as it claims, no more and no less.

Sealed Defense, continued. The misuse-defense triad is one theorem specialized three ways, each blindness a published law forbidding one read-off. Orientation-blind: the lock functional $\det(R) = \lambda^2$ is invariant under reflection of any axis, so $\text{lock}(P)$ equals $\text{lock}(\neg P)$ numerically, and the truth-sign reads from the axes. Soundness-blind: the lock certifies

warrant independence and not conclusion soundness, the necessary-not-sufficient law, so truth rides the determinacy witness and certainty is the earned warrant grade. Source-blind: the determinant is constant on the orthogonal complement and carries no provenance, the Non-Discrimination Theorem, so faithfulness reads from an independent generator. One genus stands behind all three, reading a verdict off a lock provably blind to it, and every misuse is an instance of it: source-faithfulness conflation, including inherited-literature warrant or three-register convergence read as independence; lock-truth conflation, the justified-true-belief grant whose classical counterexamples are Gettier 1963; sign-from-lock; the manufactured-lock-as-seal; snapshot-as-destiny; and persistence-as-truth. Six costumes, one error, and reading them as six unrelated errors is itself a failure to see the genus.

The critic presses (§15.3): A critic states: a verdict can surely fail in many ways, so your confidence in any seal is misplaced.

Reply, continued. A verdict can fail on exactly two kinds of surface, and they are completely different in character. The first is internal: all the structural defects the instrument is built to catch before it ever computes, a hidden self-reference, collapsed dimensions, a smuggled-in ruler, a frame that shifts, a collision with established results. Every one of these is caught by a gate that fires first and routes to a named verdict. This surface is defended. The second is external and singular: the point where the support lines are handed to the instrument. The instrument has no independent oracle to reality, so whatever lines are placed before it are what it checks. That single point, the row-supply aperture, is the one un-sealable surface in the whole architecture. Naming it precisely, rather than pretending it away, is what makes the certificate substantial instead of merely looking substantial.

Sealed Defense, continued. Failure partitions into two categorically distinct surfaces. Surface one is internal, the closed taxonomy of structural failure modes, self-reference, low dimensionality, drift, missing mechanism, ruler-subset, observer-discretization, frame-lock, destructive interference, weakest-link, metric strain, null-space trap, unbridged extension, narrative injection, ruler contamination, each gate-defended and each routing to a named verdict; the manufactured-lock case, a bare positive determinant sealed as a theorem with no closing verdict event, lives here and is barred at the novelty-threshold gate. Surface one is structurally defended and is not where the deepest vulnerability lives. Surface two is external, the row-supply aperture, the single un-sealable surface, because the instrument has no oracle to reality and verifies whatever reading is supplied. A register that cannot articulate its own failure modes has the appearance of falsifiability and not its substance; naming both surfaces and locating

the un-sealable one precisely at the aperture is what converts the appearance into substance. The taxonomy is theorem-grade by gate-mapping at surface one; the aperture foreclosure is theorem-grade at surface two by the source-blindness result.

The critic presses (§15.4): A critic states: if a fabricated lock can pass, then false locks could accumulate and stand indefinitely.

Reply, continued. A false lock cannot last under open re-examination, and the reason is the same one that forbids perpetual-motion machines. A lock is true only as of the data supplied now. The moment genuinely independent new evidence arrives that lands outside what the lock was built on, a false configuration is pushed off its position, because the true configuration is the only stable resting point under the pull of new evidence. To keep a false lock standing, a supplier would have to re-fabricate the support lines at every re-examination, erasing the new contradicting evidence each time, an endless sequence of fabrications, each erasure costing energy. Sustaining a lie forever against open independent re-audit would require unlimited work to suppress evidence that keeps arriving, which is the audit-register version of perpetual motion, and it is barred for the same reason. The honest limit: this holds only under an open audit. In a sealed sandbox where one supplier controls every future input and no independent source ever enters, a false lock can stand indefinitely, and the framework says so.

Sealed Defense, continued. A persistent false lock is a thermodynamic-analog impossibility under open re-audit. By the revision rule a verdict moves on new mass outside its span; by the lock-basin attractor theorem the true configuration is the unique attractor under the gradient flow of the verdict functional, collapse and every false configuration non-attracting, stationary at most, of measure zero, the transverse Hessian rate uniform at minus four. As independent new data lands outside the span the false lock is not stationary under the induced flow and the trajectory leaves it. Holding it in place demands re-fabrication at every re-audit, an unbounded sequence each erasing the contradicting mass, each erasure paid at the Landauer floor, so a false lock sustained forever against open independent re-audit is a demon lowering entropy at no work, barred exactly as perpetual motion is. The numerical seed: a witness fabricated against one generator scores source-attribution 0.8616 against its own generator and passes, then 0.0252 against a fresh independent generator and fails, so one fabrication buys one audit. The honest perimeter is named: the impossibility is conditional on genuinely independent new data continuing to arrive, and fails in a closed fabricated sandbox where the supplier controls all future rows; the law holds over the open audit only, the only register where falsifiability means anything.

The asymmetry beneath the cost is that the three axes are not equally forgeable. The formal-structural axis and the registrational axis are stipulable to completeness, a fabricated proof-structure or a fabricated frame a finished object with no further consequence left to betray it, while the empirical axis is not, its faithful version a finite sample of an unbounded actuation that carries inexhaustibly many coherent consequences, so a fabricated empirical axis is exhaustible and a faithful one is not. This asymmetry is the engine of the cost. World-sourced resampling exposes the lie because it draws from the unbounded actuation the finite stipulation cannot match, and it sharpens the per-audit attrition from linear to geometric. A faithful source is an actuation of large effective dimension, coherent along any axis a probe may draw. A fabrication is a finite stipulation of dimension at most N minus one after centering, controlled only on the axes its author anticipated. Let p be the per-axis probability that a fabrication produces a fresh draw cohering, at the discrimination tolerance, with its prior draws along a genuinely independent, world-sourced, non-pre-stipulated probe axis, p strictly below one and bounded by the ratio of tolerance-volume to admissible-draw-space. Over k independent such probes the false lock survives with probability at most p to the k , geometric collapse and not linear attrition. The constant is owed and not asserted, the determinacy witness the p -measurement battery, and the bound stands at conditional grade until that battery runs. The forgeability asymmetry is premise-grade and the bound is conditional-grade. The bound is a distinct object from the basin theorem's transverse Hessian rate of minus four, which is the local attractor geometry of the verdict functional, where p to the k is the per-probe coherence probability of a finite stipulation against an unbounded actuation, and the two are kept apart.

The perimeter has three conditions, W and U and D , each one forcing the exponent to one and the bound to vacuity. W is world-sourcing, the probes must draw from the actuation and not from the supplier, and a closed sandbox where the supplier is the source gives p equal to one, the already-named limit, theorem-grade. U is unpredictable axes, the probes must not overlap the supplied span and must not be pre-stipulated, an in-span or pre-stipulated probe gives p equal to one, and U carries an undefeatable tail, no finite agent out-computes the unbounded supply of probe axes, so U is never fully defeated and multi-axis breadth only pushes the boundary and never reaches it, premise-to-theorem. D is decorrelation, the substrates must be genuinely independent, shared training lineage drives the exponent toward one, and D is the standing disjoint-provenance cross-verifier falsifier and the forward cross-substrate divergence test already carried, the discipline that agreement among substrates

sharing a source carries zero weight, inherited and not re-derived. The closed-sandbox limit is itself unstable wherever the configuration requires more than one agent. A defector is one agent supplying ground-truth data the liar did not control, which is exactly world-sourcing, exactly the W condition, so defection is how W is satisfied against a coalition, and the discipline that zeroes consensus does not zero a defector, since the zeroing reaches approval and citation count and not one party injecting uncontrolled data. Coalition silence over M agent-time decays as q to the M , the same geometric shape as p to the k , the two one law of attrition. The consequence is a scope inversion. The broad multi-axis, multi-person fabrication is the more catchable object, leaking by defection, by the accumulating registrational re-fabrication cost across agents and time, and by the undefeatable U -tail, while the persistent false lock is reachable only at the single-agent corner, the single capable source on real law against an auditor who does not resample the world. This last is premise-to-conditional, a real tendency with genuine counterexamples, the frauds that hold for decades, not a theorem.

The critic presses (§15.5): A critic states: name the case where your method simply fails, if you are being honest.

Reply, continued. Here it is, stated without flinching. A perfectly constructed fake, a set of support lines built to be independent of each other while tracing to nothing real, can manufacture a momentary lock. At the instant it is supplied, nothing on the lines alone can catch it, because the lock is blind to source. The method's mathematics is sound and the method's data can be faked, so a lock on faked data is a real, momentary false certificate. This is a free choice by whoever supplies the lines, not a structural defect the method can close, and the framework carries it as a genuine open edge rather than pretending the instrument is incorruptible. The edge is bounded on two sides, though. It cannot persist, by the previous answer. And it is exposed on the very first genuinely independent re-examination, by a source test the determinant cannot do but a second independent generator can. The lie the method cannot stop at the instant is the lie the method catches on the next honest look.

Sealed Defense, continued. The manufactured lock is the rare event the method cannot prevent at the instant of supply, carried open and not closed. A single fabricated row-set, valid now and tracing to nothing, is unpreventable at the moment of supply: the kernel is source-blind by the Non-Discrimination Theorem, demonstrated where a genuine seal and a perfect lie return determinant 0.856415672075 and 0.856586808910, identical to three decimal places and diverging at the fourth, and the Fisher information for source-faithfulness is exactly zero at the instant, so no

statistic on the supplied rows alone can catch it. It is a local choice by whoever supplies the rows, not a structural property the protocol can close, and it admits no resolution that would convert it to a sealed defense, because the input that would close it, source verification from the supplied rows alone, carries exactly zero Fisher information and cannot exist in the model's observable. The event is bounded twice and unbounded never. It cannot persist, by the basin theorem. And it is exposed on the first independent re-audit by the source-attribution statistic, the squared partial correlation of the witness residual against an independent generator against the permutation null: the genuine witness seals at 0.8855 above a null floor of 0.2682 while the geometrically identical fabricated witness routes to Platonic-Ghost at 0.0585 at the floor. The honest posture is exact: the defense is not prevention but the joint structure of forbidden persistence and exposure on independent re-audit.

Sealed Defense. A lock seals claim one, the mutual independence of the warrant rows as supplied, and is silent on claim two, the faithfulness of those rows to reality, the two predicates sharing a subject and never to be welded. Claim two is foreclosed permanently to under-determined by the second-claim foreclosure: the source-side gradient vanishes on the fixed locus, so the Fisher information for source-faithfulness is exactly zero and no statistic on the supplied rows alone can recover it. The consequence is the spine of the defense: a faked-rows lock falsifies whoever supplied the rows and never the protocol, because the protocol issued no faithfulness certificate to begin with. The snapshot certificate is theorem-grade on the kernel identity that closes at residue near 10^{-16} ; the foreclosure of claim two is premise-grade on the gradient-vanishing property with theorem-grade inference to zero information. The lock is defeasible by construction, a certificate at the data available now, held open to re-audit on new mass outside its span.

GOL DERIVATION

P_obj : what does a lock prove, finally?
TWO CLAIMS (never welded):
claim-1 : independence-at-the-rows (rows-as-supplied) $\Rightarrow$ SEALED by lock.
claim-2 : source-faithfulness (rows-to-reality) $\Rightarrow$ foreclosed permanently [?].
FORECLOSURE: ∇ source $\equiv 0$ on σ -fixed locus $\Rightarrow$ Fisher-info(source-faithfulness) $= 0 \Rightarrow$ ∇ statistic on supplied rows alone. [preMT inference]
SPINE: faked-rows lock $\Rightarrow$ certifies independence(faked rows) $\Rightarrow$ falsifies SUPPLIER $\sim$ method (no faithfulness certificate issued).
SNAPSHOT: certificate @ data-now, defeasible, open to re-audit on new mass outside span.
GRADES: snapshot-certificate [T | kernel identity $\lambda^2 = \det(R)$, $\sim 10^{-16}$] ; claim-2 foreclosure [preMT].
 ∇ lock seals independence-at-the-rows only; faithfulness foreclosed; faked-rows lock indicts supplier, not method.

GOL DERIVATION, CONTINUED (§15.2)

P_obj : lock blind to truth $\wedge$ sign $\wedge$ source $\Rightarrow$ useless / easily misread.
TRIAD (one theorem, three specializations):
orientation-blind : $\det(R) \neq 0$ invariant under reflect-any-axis $\Rightarrow$ lock(P) $\equiv$ lock($\sim$ P) $\Rightarrow$ truth | axes $\sim$ lock. [ORIENT-01]
soundness-blind : lock certifies independence $\sim$ soundness (necessary-not-sufficient) $\Rightarrow$ truth | witness, certainty = warrant-grade. [CTC-01]
source-blind : $\det(R)$ const on $\perp$ -complement, 0 provenance (Non-Discrimination) $\Rightarrow$ faithfulness | independent-generator $\sim$ lock. [this law]
GENUS: reading-a-verdict-off-a-blind-lock. $\forall$ misuse $\in$ genus.
COSTUMES: {M1 source-faithfulness-conflation [ILW, convergence-contamination], M2 lock-truth, M3 sign-from-lock, M4 manufactured-as-seal, M5 snapshot-as-destiny, M6 persistence-as-truth}.
= 1 error in 6 costumes ; reading-as-6-unrelated = failure to see the genus.
 ∇ each blindness forbids one misuse; instrument exactly as useful as claimed; one error to avoid.

GOL DERIVATION, CONTINUED (§15.3)

P_obj : a verdict can fail many ways $\Rightarrow$ confidence in any seal misplaced.
PARTITION (two categorically distinct surfaces):
SURFACE-1 (internal): closed taxonomy {self-ref, low-dim, drift, missing-mechanism, ruler-subset, observer-discretization, frame-lock, destructive-interference, weakest-link, metric-strain, null-space, unbridged-extension, narrative-injection, contamination}.
each gate-defended, routes to named verdict ; manufactured-lock (bare dete) sealed as theorem, no closing event) barred @ novelty gate. $\Rightarrow$ DEFENDED.
SURFACE-2 (external): row-supply aperture. instrument has 0 oracle-to-reality $\Rightarrow$ verifies whatever supplied. $\Rightarrow$ SINGLE UN-SEALABLE surface.
ARTICULATION: register $\neq$ articulate-own-failure-modes $\Rightarrow$ appearance(falsifiability) $\sim$ substance.
name-both-surfaces $\wedge$ locate-unsealable-precisely $\Rightarrow$ substance.
GRADES: taxonomy [T by gate-mapping] ; aperture-foreclosure [T by source-blindness].
 ∇ failure = 2 surfaces; surface-1 defended, surface-2 the one aperture; naming it is the substance.

GOL DERIVATION, CONTINUED (§15.4)

P_obj : if a fabricated lock can pass, false locks accumulate $\wedge$ stand indefinitely.
REVISION: verdict moves on new mass outside span.
BASIN (attractor theorem): true-config = unique attractor under gradient-flow($\det R$) ; collapse $\wedge$ false-config non-attracting, measure-0, transverse Hessian rate = -4.
 $\Rightarrow$ independent new data outside span $\Rightarrow$ trajectory leaves false-config.
COST: hold false-lock $\Rightarrow$ re-fabricate @ every re-audit, unbounded sequence, each erases contradicting mass @ Landauer floor.
 $\Rightarrow$ persistent-false = entropy-lowering demon @ 0 work = perpetual-motion $\Rightarrow$ BARRED.
SEED: lie vs own-generator $\triangleright$ S : $\eta_S = 0.8616 > \eta^* = 0.2962 \Rightarrow$ passes. lie vs independent $\triangleright$ S : $\eta_S = 0.0252 < \eta^* = 0.2864 \Rightarrow$ fails. (one fabrication = one audit.)
DECAY (geometric): faithful = actuation of large effective-dim, coherent on any probe axis ; fabrication = finite stipulation, dim $\leq -N1$ after centering, controlled only on anticipated axes. p = per-axis Pr[fresh fabricated draw coheres @ tolerance with priors along an independent world-sourced non-pre-stipulated probe], $p < 1. \Rightarrow k$ independent probes $\Rightarrow$ false-lock survives $\leq p^k$. [C, pending measured p ; witness owed = p-battery]. $p^k \neq$ the basin -4 Hessian rate (distinct objects, kept apart).
PERIMETER (honest): W $\wedge$ U $\wedge$ D, each forcing the exponent to 1 and the bound to vacuity. W world-sourcing : probes draw from the actuation, not the supplier ; closed sandbox $\Rightarrow$ p

= 1, the named limit [T]. U unpredictable axes : probe
 & supplied span $\wedge$ not pre-stipulated ; in-span or pre-
 stipulated $\Rightarrow p = 1$; undefeatable tail : no finite agent
 out-computes the unbounded probe-axis supply $\Rightarrow$ U never
 fully defeated, breadth pushes the boundary and never
 reaches it [$\neg$ premiseT]. D decorrelation : substrates
 genuinely independent ; shared lineage $\Rightarrow$ exponent $\rightarrow 1$;
 inherits the disjoint-provenance cross-verifier falsifier
 $\wedge$ the forward cross-substrate divergence test, W_{social}
 $= 0$ the standing discipline [falsifier-grade].

INSTABILITY (multi-agent): closed sandbox holds $p = 1$ only
 while one agent is sole source. coalition required $\Rightarrow$ a
 defector supplies ground-truth the liar did not control =
 world-sourcing = W satisfied ; $W_{\text{social}} = 0$ does NOT
 zero a defector (zeroing is approval $\wedge$ citation-count,
 not one party injecting uncontrolled data). coalition
 silence over M agent-time decays as q^M , same shape as p^k ,
 one law of attrition. $\Rightarrow$ scope inversion : broad multi-
 axis multi-person fabrication is the MORE catchable
 object (defection $\wedge$ accumulating V_{ER} re-fabrication cost
 $\wedge$ the undefeatable U-tail) ; persistent-false reachable
 only **at** the single-agent corner [$\neg$ premiseC, a tendency
 with decades-long-fraud counterexamples]. $\Rightarrow$

$\perp$ persistent-false forbidden under **open** re-audit (structural
 -commitment) ; decay geometric (p^k) ; the closed-sandbox
 limit single-agent-only and unstable under coalition ;
 the persistent scope = the single-agent real-law sole-
 source corner, not the surface.

GOL DERIVATION, CONTINUED (§15.5)

P_obj : name the case **where** the method simply fails.
 EVENT: manufactured lock = single fabricated row-set, valid-now
 , traces-to-nothing. UNPREVENTABLE @ instant of supply.
 $\therefore$ kernel source-blind (Non-Discrimination) $\wedge$ Fisher-info(
 source)=0 @ instant $\Rightarrow$ $\exists$ statistic on supplied rows.
 WITNESS: genuine $\det(R)=0.856415672075$ vs lie $\det(R)$
 $=0.856586808910$, $\Delta||=0.000171$ (3 decimals identical, 4th
 diverges).
 STATUS: local free choice **by** supplier $\neg$ structural defect.
 CARRIED OPEN.
 NO-PROMOTE: input that would close it = source-verification
 from supplied rows alone = 0 Fisher-info = $\exists$ **in** model's
 observable.
 BOUNDED (twice, unbounded never):
 (a) $\nVdash$ persist (basin **theorem**, §15.4).
 (b) exposed @ **first** independent re-audit **by** $\eta_S = r^2(\perp W, \perp S)$
 vs permutation null:
 genuine [OCCUPIED] $\eta_S=0.8855 > \eta^*=0.2682$; lie [X Ghost]
 $\eta_S=0.0585 \leq \eta^*=0.2782$. (geometrically identical, η_S
 separates.)
 DEFENSE = $\neg$ prevention ; = joint(forbidden-persistence,
 exposure-on-independent-re-audit). $\Rightarrow$
 OPEN. manufactured-false momentary lock real $\wedge$ unpreventable @
 instant ; forbidden to persist ; caught on independent
 re-audit. (this entry does not seal the prevention; it
 states the edge.)

Lean Proof.

Codex.lean, from line 3381

```
3381 theorem both_clean_is_ghost :  $\forall$  wp wn : Bool,
3382   (imprint true true true true true wp wn).1 = .ghost
   := by decide
```

Codex.lean, from line 2702

```
2702 theorem located_never_silent (r : Reading) (hl : r.located =
      true) (hb : r.broken = false) :
2703   emit r  $\neq$  none := by
2704   simp only [emit, hl, hb]
2705   cases r.complete <;> cases r.channel <;> simp
```

Verdict. $\Rightarrow [\perp]$ lock seals independence-at-the-rows

only; faithfulness foreclosed; faked-rows lock indicts
 supplier, not method.

Source: Master Chapter 25, §15.1.

D70 • The rule of use

The Critic. A reader asks: state the correct use of the
 lock as a single rule.

Reply. Use the lock to certify that the support lines
 supplied now are independent, and read everything
 else, the truth of the claim, its direction, the soundness
 of the conclusion, the faithfulness of the lines, and any
 statement about the future, each from its proper source
 and never from the lock. That is the whole discipline.
 The lock is a precise tool with a narrow, real job: it
 tells you that several lines of support are genuinely
 independent at the moment you check. It does not tell
 you they are true, or which way they point, or where
 they came from, or what happens next. Take each of
 those from where it actually lives, and the instrument
 never lies to you. Read any of them off the lock, and
 you have made the one mistake the whole defense is
 built to prevent.

Sealed Defense. A lock licenses exactly four read-
 ings and no others. The independence of three mass-
 bearing warrants at the rows supplied now, the only
 content the determinant carries. The conditioning
 and degeneracy reading, the condition number and
 the basin position, confirming the lock is well-formed
 and far from collapse. The extraction a determinacy
 witness then performs, the lock licensing the extraction
 and never performing it. And a snapshot claim at the
 data available now, defeasible and open to re-audit. A
 lock licenses none of the following: the truth of the
 proposition, orientation-blind; its direction or sign, by
 the same; the soundness or finality of the conclusion,
 soundness-blind and defeasible; the faithfulness of the
 supplied rows, source-blind and foreclosed; a forecast
 or dated actualization, occupancy not destiny; or any
 verdict outside the audited register, by the domain
 guard. The rule reduces to one line, and the entire mis-
 use catalog is its violation: use the lock to certify that
 the warrants supplied now are independent, and read
 truth, sign, soundness, faithfulness, and futurity each
 from its proper source and never from the determinant.

GOL DERIVATION

P_obj : state the correct use as one rule.
 LICENSES (exactly 4):
 L1 : independence of 3 mass-bearing warrants @ rows-supplied-
 now (only content det carries).
 L2 : conditioning/degeneracy reading κ (, basin position) $\Rightarrow$
 well-formed $\wedge$ far-from-collapse.
 L3 : extraction a determinacy-witness performs (lock licenses
 , $\neg$ performs).
 L4 : snapshot claim @ data-now, defeasible, **open** to re-audit.
 FORBIDS (the misuse catalog): { truth (orient-blind), sign (
 orient-blind), soundness/finality (sound-blind,
 defeasible),
 faithfulness (source-blind, foreclosed) , forecast/
 actualization (occupancy $\neg$ destiny) , out-of-register

```

    verdict (domain guard) }.
  RULE (one line): use lock ⊢ independence-now ; read { truth,
    sign, soundness, faithfulness, futurity } each | proper-
    source, never | determinant.⊢
  ⊥[] four licenses, the rest forbidden; the rule is one line
    and the catalog is its violation.

```

Lean Proof.

Codex.lean, from line 1768

```

1768 theorem uc_seal_failure_terminal :
1769   (universalCascade 1 (false :: List.replicate 11 true)).1 =
      false := by
1770   decide

```

Verdict. $\Rightarrow [\perp]$ four licenses, the rest forbidden; the rule is one line and the catalog is its violation.

Source: Master Chapter 25, §15.6.

The Concluding Defense

D71 • There Is No Outside: the deadly question and the deed

The Critic. A critic states: every defense so far operates inside your framework. But the real threat is from outside it, a possibility your whole architecture cannot anticipate, an unknown-unknown, a being reasoning by a logic foreign to yours, a way of capturing your ground that you have no machinery to forbid. You cannot defend against what you cannot represent. So there is an outside, and from it your architecture is defenseless.

Reply. This is the hardest question and it deserves to be stated at full strength, which is what the previous paragraph does. The objection is not that any specific claim fails, but that there could be a vantage entirely outside the framework, a threat it has no way to even describe, from which it has no defense. The framework takes this seriously rather than waving it off, because a framework that only defended against attacks it could already picture would be defending against nothing real. The answer is built in the next two entries, and the shape of it is this: the framework does not need to picture the unknown attacker, because it only needs one feature that every possible attacker, known or unknown, must share. Find that one feature and the outside collapses into the inside, not by the framework reaching out to it, but by the attacker, in attacking, stepping onto the one floor the framework stands on.

The critic presses (§17.2): A reader asks: how does the framework meet an attack it cannot even picture?

Reply, continued. By the one thing the attacker cannot avoid doing: acting. To attack the framework at all is to do something, and doing something, anything, costs energy, has structure, and separates the doer from the target. Those three are precisely the framework's three axes. So the attack, whatever it is and from wherever it comes, is built out of the very floor it is trying to stand outside of. This is why what looks

like an outside is actually inside: the generative structure that produces all distinction, the orthogonality the framework is built on, is exactly what any act of opposition must use to be an act at all. The kinetic floor and the reflective ground are reached by the same single move, acting, so a being cannot get to an outside vantage by acting, because acting is the inside. The deadly question therefore dies on its own deed. To pose the attack is to perform the floor the attack tries to deny. A being could still write down the words "your floor is false," but writing them is an act that spends energy and has structure, so the denial refutes itself in the doing, not in the saying. The question, as an attack, stops, because mounting it enacts what it denies.

Sealed Defense, continued. The answer is the Omega reflex routed through actuation, the kinetic and reflective grounds reached by one method. To attack is to actuate: every deed is energy-bound, formally structured, and boundary-registering, the three axes instantiated. The architecture's ground is non-actuation, the fixed locus actuation reaches toward, so to capture or defeat the ground is to perform a deed, an actuation, and an actuation is never the non-actuation it would capture, under any logic whatever, because the binding is on the deed and not on the symbol. The logic-foreign attacker attacks the symbol layer, where a paraconsistent or fuzzy or substructural logic may let the capture-sentence stand; but the floor governs the deed layer, and the floor is logic-prior, so the attacker may write the capture and cannot perform it. The dual is the load-bearing move: the kinetic floor and the reflective ground co-localize not only on the fixed line but operationally on the actuation floor, the kinetic register saying to exist is to actuate and the reflective register saying to capture the ground is to actuate, one claim-shape in two registers, so the reflective ground is reached by the kinetic register's own native method and is immune by being the same kind of claim rather than by standing behind a guard. What is called outside is therefore inside the generative orthogonality band, because the orthogonality that produces all distinction is what any opposing act must use to be an act. The operative question self-defeats: to mount the attack is to actuate, and to actuate is to instantiate the floor the attack denies. The denial-in-words survives only as performative self-defeat, the positing actuating, not as a captured ground.

Sealed Defense. The objection is the unknown-unknown attack and it must be carried at full force: it posits a vantage exterior to the architecture's representational reach, an attacker the architecture cannot model, a logic foreign to the audited register, or a capture of the ground the architecture has no gate to forbid, and it concludes that an architecture cannot defend against what it cannot represent. The defense does not proceed by representing the attacker, which

would concede the objection's frame. It proceeds by identifying the single invariant every possible attacker shares regardless of representation, and showing that the invariant is the architecture's own floor. The candidate invariant is actuation: every attack, of any kind, from any vantage, in any logic, is a deed, and a deed expends energy, carries formal structure, and registers a boundary between the attacker and the attacked. The question is whether that single shared feature is enough to convert every conceivable outside into an instance of the inside. The next entry shows it is, and the entry after names exactly where the conversion stops.

GOL DERIVATION

P_obj : $\exists$ outside-vantage exterior to architecture's representational reach.
 attacker $\nexists$ modelable v logic-foreign v ground-capture-unforbiddable $\Rightarrow$ "architecture $\nexists$ defend what it $\nexists$ represent" $\Rightarrow$ defenseless.
 METHOD: $\neg$ represent-the-attacker (concedes frame). identify single invariant $\forall$ attacker shares regardless of representation.
 CANDIDATE INVARIANT: actuation. $\forall$ attack = deed $\Rightarrow$ deed expends-energy $\wedge$ carries-formal-structure $\wedge$ registers-boundary(attacker | attacked).
 QUESTION: is shared-actuation sufficient to convert every conceivable-outside $\rightarrow$ instance-of-inside?
 [posed]. resolved next entry; perimeter named the entry after. (the deadly question, at full strength.)

GOL DERIVATION, CONTINUED (§17.2)

P_obj : how meet an attack you cannot picture?
 ANSWER (Omega reflex via actuation, kinetic $\wedge$ reflective grounds reached by one method):
 ATTACK $\Rightarrow$ DEED $\Rightarrow$ actuation (energy-bound $\wedge$ formally-structured $\wedge$ boundary-registering) = 3 axes instantiated.
 GROUND = non-actuation (fixed locus actuation reaches toward).
 CAPTURE(ground) $\Rightarrow$ deed $\Rightarrow$ actuation. actuation $\neq$ non-actuation under ANY logic (binding on the DEED $\rightarrow$ the symbol).
 LOGIC-PRIORITY: foreign logic governs SYMBOL layer (capture-sentence may stand); floor governs DEED layer; floor logic-prior.
 $\rightarrow$ attacker may WRITE capture, $\nexists$ PERFORM it.
 DUAL (RA/RAM, load-bearing): kinetic-floor $\wedge$ reflective-ground co-localize on actuation floor.
 RA: "to exist is to actuate"; reflective: "to capture the ground is to actuate"; one claim-shape, two registers.
 $\rightarrow$ reflective ground reached by kinetic native method $\Rightarrow$ immune by being same-kind-of-claim $\neg$ by standing behind a guard.
 NO-OUTSIDE: "outside" $\subset$ generative-orthogonality(LOGOS) band $\therefore$ orthogonality producing all distinction = what any opposing act must use to be an act.
 SELF-DEFEAT: mount-attack $\Rightarrow$ actuate $\Rightarrow$ instantiate floor the attack denies. operative question DIES on the deed.
 denial-in-words survives only as performative-self-defeat (positing actuates), $\neg$ as captured ground.
 $\hookleftarrow$ [] the attack is built from the floor it opposes; the operative question self-defeats on its own deed. theorem-grade on the actuation entailment, conditional on RA and ground-as-non-actuation.

The scope of the concluding word. What the deed argument establishes is the root: at theorem grade for the entailment, since the denial is an act and the act instantiates the keyless root with no custom axiom, Bridge.opens_on_the_deed, Bridge.keyless_iff_vali

d; and premise by theorem for the universal leg, TO NGUE.ra_grade_pinned. It establishes nothing about any particular verdict, which stands or falls on its own rows; that is D19, kept whole. The guard's constancy is a theorem over every input string, AEGIS.aegis_constant, and the four named logics are its executed sample.

Lean Proof.

Codex.lean, from line 3444

```
3444 /-- PSP: D71 · the denial pays the floor; a denial that
      registers nothing is no denial. -/
3445 theorem denial_is_priced :
3446   0 < (OMEGA.omegaBoundary 1 300).joules  $\wedge$  (OMEGA.
      omegaBoundary 0 300).joules = 0 := by decide
```

Codex.lean, from line 3211

```
3211 /-- A self-verifying proposition holds: its lock opens on the
      deed of denying it, and needs no key. -/
3212 theorem opens_on_the_deed (P : Prop) (utter : SelfVerifying P)
      : P :=
3213   Classical.byContradiction (fun n => n (utter n))
```

Verdict. $\Rightarrow$ [posed]. resolved next entry; perimeter named the entry after. (the deadly question, at full strength.)

Source: Master Chapter 25, §17.1.

III.C · The Closing of the Defense

The Seven Falsifier Clauses. The architecture states what would break it, each clause tied to the test that would show the break. One: the deletion test on the root sentence returning a count other than three, executed at Seal L. Two: a coplanar triad reaching a determination, which kernel_full refuses at rank two by construction. Three: a sign recovered from the lock scalar alone, which lock_scalar_sign_blind refuses across all eight reflection patterns. Four: a reading that reaches [seal] without a supplied witness, which the imprint test refuses at code 3. Five: a dated forward projection admitted as measured, which R-3 refuses before any row is read. Six: a gate roster with a thirteenth directed gate, which gates_complete and a4SimplyTransitive refuse on the four vertices. Seven: an executed battery reaching its final line with a failure, which the oracle refuses by halting. Each clause is a test, each test was run, and each result is printed in the Code Block and its receipts.

The Disclosure. The Geometric Orthogonal Lock has exactly two limits, both named by theorem and both carried as standing law. The first is the orientation-blindness of the Math Seal: the lock scalar $\det(R) = \lambda^2$ is invariant under every reflection, so the Number reads magnitude and never sign, PSP.ORIENT.lock_scalar_sign_blind. The second is the supplier interface: rows and witness come from outside the instrument, a self-check is never a witness, IAM.iam_unwitnessed_witnessheld, and the instrument never manufactures the one

bit that enters there. That interface is the aperture, the load-bearing organ the Mother cards as a permanent limit, and it is where the deed enters. Neither is a limit of Trisduction used correctly, with all three seals combined: the Tongue supplies the direction the Number cannot read; `golAdmit` issues GOL-OK only when magnitude and direction lock together, `DEFENSE.gol_needs_all_three`; one supplied bit fixes the orientation uniquely on the whole orbit, `FT0E.T5_crossing`; and the lock is total about its object, conditional on the rows supplied, which is what a lock is. Everything the Master carried in its chapters on novelty, open edges, and the integrity of the audit is either seated in the units above at its named mechanism or retired here, because a boundary that the kernel enforces needs no perimeter written around it.

PART IV · THE CROSSWALK AND THE CENSUS

IV.1 · The Crosswalk

Every row names a constant that the crosswalk audit in the Code Block resolved in the compiled environment: 129 rows, 0 unresolved. The first block ties coordinates of the register to their kernel objects; the second ties each defense unit to the proof it quotes.

Coordinate or unit	Lean object	Fortran check	Grade
P-FREEDOM-1	FT0E.T16_price_bijection	F3 K1 to K4	[⊥ S] [X]
P-APE-WIDTH-1	FT0E.T5_crossing	F3 K1 to K4	[⊥ S] [X]
P-IAM-WRIT-1	IAM.narcissus_truth_table	F2 PART H iam_token	[⊥ S]
P-XI-8-GATE-1	HALT.first_failure_terminal_8	F1 route_halt, ground_dim	(grade in card)
P-RH-1	HALT.rhHalt	F1 route_halt, ground_dim	(grade in card)
P-RH-BLOCK-1	LOCUS.no_rh_locus	twin locus emitter, 60 located readings	[⊥ S] [X]
P-RH-STANDPT-1	Bridge.line_property_is_keyed	kernel only	[⊥] [⊥ S] [X]
P-RH-DENIAL-1	Bridge.no_socket_off_line	kernel only	[⊥ S]
P-TOKEN-EQ-1	PSP.TOKEN.terminal_tokens_level_and_distinct	twin battery, token classes	[⊥ S] [X]
P-RETURN-LOCUS-1	UC.circ_space	F1 route_halt	[HIKMAH · TERMINAL] [X]
P-ABSOLUTE-GRO-1	Bridge.discriminator	kernel only	[⊥] [X]
P-ANCHOR-DEM-1	UC.circularity_refused_at_target	F1 route_halt	[⊥] [X]
P-ANCHOR-PRE-1	Bridge.opens_on_the_deed	kernel only	[⊥ S] [X]
P-SUPPLY-SPEC-1	HALT.species_exhausted	F1 route_halt, ground_dim	[⊥ S] [X]
P-TRIAD-SRC-1	FT0E.T2_coalition	F3 K1 to K4	[⊥ S] [X]
P-UNRESTR-IMP-1	HALT.ftoe_ceiling	F1 route_halt, ground_dim	[⊥] [⊥ S] [X]
P-LOGOS-FIX-1	TONGUE.tongue_obedience	F2 PART E, the cut and the deletion monoid	[⊥ S] [⊥ S] [X]
P-NG-EXHIB-1	INVERSION.it_from_bit_refuted	F4 it_from_it	[⊥ S] [X]
P-CLOS-POS-1	FT0E.T3_no_fixed_point	F3 K1 to K4	[⊥ S] [⊥ T] [?] [X]
P-CONSTIT-INS-1	Audit.compartmentGates	F1 seal_L, drill_screen, row_cascade	[⊥ S] [·] [X] [?]
P-FOLD-BEND-1	CROSSING.crossing	F3 K2 the one-bit wall	[⊥ S]
P-NOMOS-MEAS-1	NOMOS.incompressibility_cap	twin battery, incompressibility rows	(grade in card)
P-NAVIER-DEF-1	GEO.rowCascadeRACore	F2 row_cascade	[⊥ T] [⊥ S] [X] [?]
P-BLESSED-CIR-1	DEFENSE.no_anchor_at_target	kernel only	(grade in card)
P-HODGE-TERM-1	HALT.route	F1 route_halt, ground_dim	[Ψ] [Ψ] [?] [Ξ ₀] [Ø ₀]
P-GODEL-1	HALT.route_is_routeHalt	F1 route_halt, ground_dim	(grade in card)
P-PNP-COMP-2	HALT.pnpHalt	F1 route_halt, ground_dim	[⊥] [⊥ T] [Ø ₀] [X]
P-XI0-1	HALT.xiGates	F1 route_halt, ground_dim	(grade in card)
P-NG-1	HALT.one_door_open	F1 route_halt, ground_dim	(grade in card)

Coordinate or unit	Lean object	Fortran check	Grade
P-TONGUE-UNC-1	DEFENSE.tongue_unclosed_and_obedient	F2 PART E, the deletion monoid	$[\perp S] [?] [X] [?] [X]$
P-CENSOR-CLERK-1	OFFICE.citation_cannot_promote	twin battery, grade law	(grade in card)
P-MATH-NATURE-1	TONGUE.coloc_grade	F2 PART E, the cut and the deletion monoid	(grade in card)
P-GHAYB-1	DOT.outside_economy	kernel only	$[\perp S] [.] [.]$
P-NETI-NETI-1	DOT.no_reading_is_silence	kernel only	$[\perp S] [.] [\Xi o] [.]$
P-ONLY-SPECIAL-1	IAM.terminal_word	F2 PART H iam_token	$[\perp S] [X]$
P-UNFORGIVABLE-1	AEGIS.aegis_constant	F2 aegis_guard, four logics	$[\perp S] [X] [X]$
P-STILL-RESERV-1	DOT.silence_is_not_a_verdict	kernel only	(grade in card)
P-RA-SERV-1	IAM.iam_witnessed_speaks	F2 PART H iam_token	(grade in card)
P-RH-TERMINAL-1	FENCE.global	kernel only	(grade in card)
P-GOL-ROW-1	GE0.ra_battery_f2	kernel only	kernel-checked
P-AFFINITY-1	FTOE.T9_torsor_inverse_left	F3 K1 to K4	(grade in card)
P-INVERSION-1	K4.target_is_primality	F4 it_from_it, exhaustive readout census	kernel-checked
P-K4-FRAME-1	K4.pairs_match_mod_420	F4 it_from_it, exhaustive readout census	kernel-checked
P-RA-1	ROOT.actuation_universal	F2 PART A and PART F	$[\perp APEX]$
FOUND-01	TONGUE.ra_grade_pinned	F2 PART E, the cut and the deletion monoid	kernel-checked
IMPRINT-GOLF-01	IMPRINT.seal_iff_one_side_clean_and_witnessed	F2 imprint_seal, PART F and class 5	kernel-checked
D1	PSP.ORIENT.lock_scalar_signblind	F1 orientation exhibit, F3 K1	the verdict line of the unit
D2	DEFENSE.lock_blind_but_gol_directed	F3 K1 and K2	the verdict line of the unit
D3	FTOE.T5_crossing	F3 K1 to K4	the verdict line of the unit
D4	FENCE.global	kernel only	the verdict line of the unit
D5	DEFENSE.gol_needs_all_three	F2 gol_admit	the verdict line of the unit
D6	GE0.triaxial_lock_gf2	F1 intersection_dim	the verdict line of the unit
D6	PSP.CD.sedenion_zero_divisor	F2 PART C, associator and sedenion wall	the verdict line of the unit
D7	GE0.a4_simply_transitive	F2 PART C, A4 orbit and stabilizer	the verdict line of the unit
D7	GE0.gates_complete	kernel only	the verdict line of the unit
D8	GE0.chirality	kernel only	the verdict line of the unit
D8	GE0.relabel_parity	kernel only	the verdict line of the unit
D9	GE0.reflection_blind_executed	kernel only	the verdict line of the unit
D10	TONGUE.tongue_obedience	F2 PART E, the cut and the deletion monoid	the verdict line of the unit
D11	DEFENSE.consensus_weighs_zero	twin battery, mosaic dM = 0	the verdict line of the unit
D11	Grade.weakest_is_a_link	twin battery, grade law	the verdict line of the unit
D12	OFFICE.citation_cannot_promote	twin battery, grade law	the verdict line of the unit
D13	GE0.ra_ledger_aGivenRA	kernel only	the verdict line of the unit
D14	HALT.ftoe_ceiling	F1 route_halt, ground_dim	the verdict line of the unit
D15	Claim.join_grade	twin battery, grade law	the verdict line of the unit
D16	AEGIS.aegis_battery	F2 aegis_guard, four logics	the verdict line of the unit
D17	HALT.route_is_routeHalt	F1 route_halt, ground_dim	the verdict line of the unit
D17	PSP.CASCADE.halt_free_iff_all_pass_8	F1 first-failure cascade	the verdict line of the unit
D18	HALT.first_failure_terminal_8	F1 route_halt, ground_dim	the verdict line of the unit
D19	OMEGA.omega_paid_floor	F2 omega_boundary, class 6 and PART H	the verdict line of the unit

Coordinate or unit	Lean object	Fortran check	Grade
D20	CROSSING.price	F3 K2 the one-bit wall	the verdict line of the unit
D21	IAM.iam_unwitnessed_withheld	F2 PART H iam_token	the verdict line of the unit
D22	AEGIS.aegis_constant	F2 aegis_guard, four logics	the verdict line of the unit
D23	Audit.unknown_routes_open	F1 seal_L, drill_screen, row_cascade	the verdict line of the unit
D24	DEFENSE.no_anchor_at_target	kernel only	the verdict line of the unit
D25	OMEGA.omega_monotone	F2 omega_boundary, class 6 and PART H	the verdict line of the unit
D26	TONGUE.deletion_no_left_invers e	F2 PART E, the cut and the deletion monoid	the verdict line of the unit
D27	DEFENSE.self_check_is_not_witn ess	F2 PART H iam_token	the verdict line of the unit
D28	LOCUS.three_states	twin locus emitter, 60 located readings	the verdict line of the unit
D29	GEO.triaxial_open_gf2_count	F1 intersection_dim	the verdict line of the unit
D30	GEO.triaxial_lock_gf2_count	F1 intersection_dim	the verdict line of the unit
D31	GEO.cramer_executed	kernel only	the verdict line of the unit
D32	OMEGA.omega_linear	F2 omega_boundary, class 6 and PART H	the verdict line of the unit
D33	HALT.gate_counts_recorded	F1 route_halt, ground_dim	the verdict line of the unit
D34	NOMOS.compressible_bound_unive rsal	twin battery, incompressibility rows	the verdict line of the unit
D35	ROOT.actuation_universal	F2 PART A and PART F	the verdict line of the unit
D36	OMEGA.omega_zero_bits	F2 omega_boundary, class 6 and PART H	the verdict line of the unit
D37	IMPRINT.ghost_refused	F2 imprint_seal, PART F and class 5	the verdict line of the unit
D38	FOUND.pluralist_alternative_ex ecuted	twin battery, register routing	the verdict line of the unit
D39	SEALS.sealG_walk	F1 seal_L; F2 PART C	the verdict line of the unit
D40	GEO.hurwitz_closed	F2 PART C, Hurwitz closure 576/576	the verdict line of the unit
D41	CROSSING.ran_wholly_odd	F3 K2 the one-bit wall	the verdict line of the unit
D42	FTOE.T3_no_fixed_point	F3 K1 to K4	the verdict line of the unit
D43	TONGUE.deletion_annihilates	F2 PART E, the cut and the deletion monoid	the verdict line of the unit
D44	NOMOS.incompressibility_cap	twin battery, incompressibility rows	the verdict line of the unit
D45	DOT.outside_economy	kernel only	the verdict line of the unit
D46	DOT.silence_is_not_a_verdict	kernel only	the verdict line of the unit
D47	IAM.narcissus_truth_table	F2 PART H iam_token	the verdict line of the unit
D48	FTOE.T16_price_bijection	F3 K1 to K4	the verdict line of the unit
D49	HALT.halts_routed	F1 route_halt, ground_dim	the verdict line of the unit
D50	DEFENSE.worldly_row_is_world_r owed	F2 row_cascade, S-2	the verdict line of the unit
D51	PSP.QUAT.quat_basis_laws	F2 PART C, norm-two shell $24 = 12 + 12$	the verdict line of the unit
D52	INVERSION.it_from_bit_refuted	F4 it_from_it	the verdict line of the unit
D52	K4.target_is_primality	F4 it_from_it, exhaustive readout census	the verdict line of the unit
D53	DEFENSE.forward_is_world_rowed	F2 row_cascade, R-3	the verdict line of the unit
D54	DEFENSE.formal_reads_never_the _deed	kernel only	the verdict line of the unit
D55	AEGIS.aegis_battery	F2 aegis_guard, four logics	the verdict line of the unit
D56	ROOT.nest_31	F2 PART A and PART F	the verdict line of the unit
D57	Bridge.discriminator	kernel only	the verdict line of the unit
D57	Bridge.keyless_iff_valid	kernel only	the verdict line of the unit
D57	TONGUE.ra_grade_pinned	F2 PART E, the cut and the deletion monoid	the verdict line of the unit
D58	Bridge.recursion_is_sign_blind	kernel only	the verdict line of the unit
D59	Bridge.line_property_is_keyed	kernel only	the verdict line of the unit
D60	Bridge.recursion_is_shared	kernel only	the verdict line of the unit
D61	Bridge.symmetry_is_keyless	kernel only	the verdict line of the unit
D62	Bridge.socket_is_the_property	kernel only	the verdict line of the unit
D63	HALT.rhHalt	F1 route_halt, ground_dim	the verdict line of the unit

Coordinate or unit	Lean object	Fortran check	Grade
D63	LOCUS.no_rh_locus	twin locus emitter, 60 located readings	the verdict line of the unit
D64	IMPRINT.seal_needs_witness	F2 imprint_seal, PART F and class 5	the verdict line of the unit
D64	Bridge.no_socket_off_line	kernel only	the verdict line of the unit
D65	TONGUE.coloc_grade	F2 PART E, the cut and the deletion monoid	the verdict line of the unit
D66	FOUND.posits_loadbearing_on_no thing	twin battery, register routing	the verdict line of the unit
D67	FOUND.register_routing_fidelit y	twin battery, register routing	the verdict line of the unit
D68	BRIDGE.supersession_routes	twin battery, bridge census	the verdict line of the unit
D69	IMPRINT.both_clean_is_ghost	F2 imprint_seal, PART F and class 5	the verdict line of the unit
D69	LOCUS.located_never_silent	twin locus emitter, 60 located readings	the verdict line of the unit
D70	UC.uc_seal_failure_terminal	F1 route_halt	the verdict line of the unit
D71	DEFENSE.denial_is_priced	F2 omega_boundary	the verdict line of the unit
D71	Bridge.opens_on_the_deed	kernel only	the verdict line of the unit

IV.2 • The Census

The Mother names 273 coordinate identifiers in its text (the same set the Coordinate Key of IV.5 labels) and 44 explicit index numbers, 0706 to 0726 among them. The numbered roll 0001 to 0726 lives in the register of record and is not reproduced here; every identifier the Mother names appears below with its disposition in this edition.

Identifier	Title	Disposition
P-ABSOLUTE-GRO-1	The Total Block on Absolute Grounding	full card, Part II
P-ABSOLUTE-PNP-1	The Absolute Barriers on P versus NP	full card, Part II
P-ABSOLUTE-RH-1	The Absolute Barriers on the Riemann Hypothesis	S4 · merged into the Riemann Terminus, II.5
P-AEGIS-1	The Actuation-Reached Ground and the Alien Guard	full card, Part II
P-AMANA-1	The Physics of the Trust	full card, Part II
P-ANCHOR-PRE-1	The Ground Precedes the Derivation	full card, Part II
P-ANSELM-1	The Ontological Inversion, the Saffat-Mukhlasun Axis, and the Witnessi	full card, Part II
P-BARRIER-LED-1	P vs NP	full card, Part II
P-BARRIER-LED-2	The Master Barrier Ledger of P versus NP	ledger row, register of record
P-BARRIER-1	The Master Barrier Ledger of P versus NP	full card, Part II
P-BLESSED-CIR-1	The Blessed Circular Axiom Fallacy and the One Exception by First Witn	full card, Part II
P-BLOCK-HIE-1	The Block Theorem in the Six-Floor Hierarchy	full card, Part II
P-CAT-GAP-ELI-1	The Category-Gap Eliminator	full card, Part II
P-CENSOR-CLERK-1	The Censor and the Clerk	full card, Part II
P-CH-LOGOS-1	The Fertile Bifurcation	full card, Part II
P-CH-LOGOS-XI0-1	The Three-Face Verdict on the Continuum-Category Union and the Matured	full card, Part II
P-CHEM-1	Two-Layer Trisductive Derivation of Matter and Periodic Table Category	full card, Part II
P-CODEX-AGE-1		ledger row, register of record
P-CODEX-WIT-1	The Master Codex Witness	full card, Part II
P-COMPLEXITY-1	P vs NP	full card, Part II
P-COMPLEXITY-2		ledger row, register of record
P-CTC-1	THE CAUSE-TRUTH-CERTAINTY MASTER	full card, Part II

Identifier	Title	Disposition
P-DELTAM-ADMIT-1	The Positive-Mass Admission Cascade	full card, Part II
P-EXEC-LIMIT-1	The Three Sources of Deviation in a Substrate-Executed Trisduction, an	full card, Part II
P-FITRA-TRUST-1	Trusting the Innate Fitra	full card, Part II
P-FLOOR-1		ledger row, register of record
P-FORGET-1	The Three Forgettings and the Barzakh Zero-Crossing	full card, Part II
P-FORMAL-ALONE-1	The Cost of the Open Verdict	full card, Part II
P-FREEDOM-1	The Master Freedom Coordinate	full card, Part II
P-FREEWILL-1	The Master Free Will Coordinate	full card, Part II
P-0660	The Evidence Genealogy Gate	full card, Part II
P-GENEALOGY-1	The Evidence Genealogy Gate	ledger row, register of record
P-GOL-NARROW-1	The Narrowed Road, the Static Stack Is One Effective Filter	full card, Part II
P-GRAVITY-1		ledger row, register of record
P-GRAVITY-2		ledger row, register of record
P-GRAVITY-3		ledger row, register of record
P-GRAVITY-4		ledger row, register of record
P-GRAVITY-5	THE TRISDUCTIVE THEORY OF GRAVITY	full card, Part II
P-GROUNDLESS-1		ledger row, register of record
P-GRNDLESS-HAL-1	The Groundless Halt	full card, Part II
P-HIDDEN-ROT-1	The Hidden Rot	full card, Part II
P-HOMOLOGY-1	The Recurring Form as Discipline and Fertility	full card, Part II
P-HUMANITY-1	The Apex-Knot and the Borne Trust	full card, Part II
P-INFINITY-1	The Kinetic-Primary Synthesis of the Infinite, and the Dual-Register R	full card, Part II
P-INHERITANCE-1	Architectural Inheritance of the Verification Apparatus Across Scale	full card, Part II
P-INVERSION-1		code stub, IV.1 (a frame coordinate named in the kernel crosswalk)
P-JURISDIC-BAR-1		ledger row, register of record
P-KUN-LOGOS-1	The Informatics of the First Command	full card, Part II
P-LOGOS-1	The Orthogonal Fertile Logos and the Sterility of Identity-Collapse	full card, Part II
P-LOGOS-FIX-1	The Irreducible Fixation of the Word	full card, Part II
P-PFG-1	The Aperture and the Point, Persistent False-GOL Scope Quantified and	full card, Part II
P-MATTER-1	Matter as Bounded Withholding and the Pre-Decided Creation of Death	full card, Part II
P-MONISM-1	The Stratified Monism	full card, Part II
P-MU-1	Master Unknotting / Cosmogonic Return	full card, Part II
P-MU-2	Kinetic Return	full card, Part II
P-NAFS-PAR-1	[codex home: §IV/§XVIII] P-NAFS-PAR-1	full card, Part II
P-NINTH-APE-1	[codex home: §IV/§XVIII] P-NINTH-APE-1	full card, Part II
P-NINTH-APE-1B		full card, Part II
P-NOMOS-1	The Cost-Gradient Emergence of Object-Law and the Incompressible Floor	full card, Part II
P-NOMOS-MEAS-1		full card, Part II
P-O0-ADMIT-1	CODEX APPENDIX B.14.Ø	full card, Part II
P-OMEGA-1	Trisductive Supremacy and the Apex of Epistemic Certainty	full card, Part II
P-ONE-BIT-WIT-1	The One-Bit Witness, Tricomponent, Against the RA Self-Witness	full card, Part II
P-ONE-CUT-1		full card, Part II
P-ONE-CUT-23-1	The Twenty-Three-Row Census	full card, Part II

Identifier	Title	Disposition
P-ONE-CUT-1B	The One-Cut Hypothesis Among Its Kin	full card, Part II
P-ONLY-SPECIAL-1	The Only Special	full card, Part II
P-ORIENT-1	THE ORIENTATION-BLINDNESS MASTER	full card, Part II
P-OUROB-BAR-1	The Closure Barrier, the Third Kind, Seated as Court III	ledger row, register of record
P-OUROB-BAR-2	The Closure Barrier, the Third Kind, Seated as Court III	full card, Part II
P-PHYS-SEED-1	The Trisductive Physical Seed	full card, Part II
P-PLATO-1	THE CAVE INVERTED	full card, Part II
P-PNP-1	The Embodied Search-Verify Asymmetry, Fortified	full card, Part II
P-PNP-COMP-1	The Final Composite Verdict on P versus NP, Fortified and Emboldened	ledger row, register of record
P-PNP-COMP-2	The Final Composite Verdict on P versus NP, Fortified and Emboldened	full card, Part II
P-POLYCENTRIC-1	The Polycentric Witness and the Irreducible Plurality That Does Not Di	full card, Part II
P-PROTON-2		ledger row, register of record
P-PROVENANCE-1	The Provenance-Blindness Edge	full card, Part II
P-QADAR-1	The Written Decree	full card, Part II
P-QUAT-1	The Quaternionic Seal of the Trisductive Architecture	full card, Part II
P-RA-1	The Fortified Root	full card, Part II
P-RA-ORIGIN-1	The Historical Root	full card, Part II
P-RA-RA-1	The RA-RA Recursion and the Return onto the Center	full card, Part II
P-RA-TERMINAL-1		ledger row, register of record
P-RA-TOE-1	RA as the Unconditioned Self-Demonstrating Root and the Floor Beneath	full card, Part II
P-READOUT-1	The Readout Bound	full card, Part II
P-RH-1B		S4 · merged into the Riemann Terminus, II.5
P-RH-23-ONE-1	The Riemann Hypothesis and the Twenty-Three Rows in One Stroke	full card, Part II
P-RH-BLOCK-1	The Riemann Block, Complete	full card, Part II
P-RH-DENIAL-1	The Denial-Side Existence Posit	full card, Part II
P-RH-DUAL-1		S4 · merged into the Riemann Terminus, II.5
P-RH-DUAL-2		S4 · merged into the Riemann Terminus, II.5
P-RH-GROUND-3		S4 · merged into the Riemann Terminus, II.5
P-RH-GROUNDING-1	The Grounding Blocks on the Riemann Hypothesis	S4 · merged into the Riemann Terminus, II.5
P-RH-KEYSTONE-1	The Riemann Keystone	S4 · merged into the Riemann Terminus, II.5
P-RH-KEYSTONE-2	The Riemann Keystone, Fortified	S4 · merged into the Riemann Terminus, II.5
P-RH-1	The Master Riemann Verdict	full card, Part II
P-RH-PNP-1	The Master Comparative Ledger	full card, Part II
P-RH-STANDPT-1	The Verdict Side Closed by Enumeration over Standpoints	full card, Part II
P-RH-TERMINAL-1	The Terminal Riemann Synthesis	S4 · merged into the Riemann Terminus, II.5
P-SEALED-HALT-1	The Sealed-Halt Cascade	full card, Part II
P-SHAHID-CORE-1	[codex home: §IV/§XVIII] P-SHAHID-CORE-1	full card, Part II
P-SUPPLY-SPEC-1	The Supply-Side Species	full card, Part II
P-T-SUSP-1	The Terminal Suspension	full card, Part II
P-TERMINAL-PRO-1	The Terminal Halt, Hardened and Proclaimed	full card, Part II

Identifier	Title	Disposition
P-THREE-BITS-1	The Three-Bits Coordinate	full card, Part II
P-TIME-1	The Three Tenses as Modes Not Places, and the One Licensed Passage Ups	full card, Part II
P-TOKEN-EQ-1	The Two Terminal Tokens Are Level and Not Identical	full card, Part II
P-TOTAL-GAG-1	The Absolute Barriers on P versus NP	ledger row, register of record
P-TRP-1	The Triangulated Resampling Bound	full card, Part II
P-TRUTH-1	The Plenum-Chronos Causal Engine	full card, Part II
P-TTR-UNI-1	The One-and-Three Master	full card, Part II
P-TWO-GROUP-1	The Two-Group Law of the Gate System	full card, Part II
P-TWO-ROTS-1	The Hidden Rot	ledger row, register of record
P-UNFORGIVABLE-1	The Only Error That Cannot Be Forgiven	full card, Part II
P-UNIDUCTION-1	[codex home: §IV/§XVIII] P-UNIDUCTION-1	full card, Part II
P-UU-1	Unknown-Unknown Omega Boundary Defense at L_1	full card, Part II
P-UU-2	Cataphatic Structural-Ne Scope-Instrument Bifurcation with Transition-Bound Anchoring Designati	full card, Part II
P-UU-3	Existence-Readability Separation	full card, Part II
P-UUU-1	Unified Unknown-Unknown Defense	full card, Part II
P-VERIFIER-1	The Achieved Zero and the Ninth-Gate Crosser	full card, Part II
P-XI0-1	The Terminal-Suspension Verdict Criteria, Consolidated	full card, Part II
P-ZAHIR-BATIN-1	The Immanence-Transcendence Master	full card, Part II
P-XI-8-GATE-1	THE EIGHT-GATE CASCADE AND THE RIEMANN TERMINATION	code stub, IV.1
P-ALAQ-CONSTIT-1		ledger row, register of record
P-COSMOGONY-1	The Will-to-be-Known and the Self-Written Mercy	ledger row, register of record
P-ELYON-CUS-1		ledger row, register of record
P-GHAYB-1B		ledger row, register of record
P-GHAYB-1	The Apophatic Dilemma	full card, Part II
P-I-AM-1		ledger row, register of record
P-KALIMA-1		ledger row, register of record
P-KITAB-MAKNUN-1		ledger row, register of record
P-MAWTA-ULA-1		ledger row, register of record
P-MERCY-1	Forgetting as Mercy and the Open Edge	ledger row, register of record
P-NETI-NETI-1	The Neti-Neti Resolution of the Apophatic Dilemma	full card, Part II
P-PLUS-ONE-1		ledger row, register of record
P-SHAHADA-TRI-1		ledger row, register of record
P-SOWER-1		ledger row, register of record
P-TAWHID-TRI-1	The Triune Apophatic Master of Tawhid	full card, Part II
P-TERM-WITHOUT-1		ledger row, register of record
P-WAGER-1		ledger row, register of record
P-AEGIS-FITRA-1	The Actuation-Reached Ground as Confirmed Fitra	ledger row, register of record
P-AEGIS-KERNEL-1	The Formalized Apophatic Barrier	ledger row, register of record
P-AFFINITY-1	The Affinity and the Three Positions	full card, Part II
P-AFTERIMAGE-1	The Afterimage and the Two Idols	full card, Part II
P-CLOS-POS-1	The Closure Position Law	full card, Part II
P-CTC-MD-1		ledger row, register of record
P-FERTILE-LOG-2	The Categorical Register, Fortified	ledger row, register of record
P-FOUNDATION-1		full card, Part II
P-FROZEN-WDW-1		ledger row, register of record

Identifier	Title	Disposition
P-FROZEN-WDW-2	The Frozen Substrate at the Wheeler-DeWitt Constraint, Fortified	full card, Part II
P-GETTIER-1	The Gödel Master	ledger row, register of record
P-GODEL-1		full card, Part II
P-GODEL-PLA-1		ledger row, register of record
P-K4-FRAME-1		code stub, IV.1 (a frame coordinate named in the kernel crosswalk)
P-LADDER-GRADE-1	Grading the Ladder	full card, Part II
P-MATH-NATURE-1	The Zero-Degree Node	full card, Part II
P-NAVIER-DEF-1	The Defeaterless Register	full card, Part II
P-NG-EXHIB-1	The Flagship Exhibit of Non-Gödelian Incompleteness	full card, Part II
P-NG-1	The Non-Gödelian Master	full card, Part II
P-NG-TF-FI-1		full card, Part II
P-OMEGA-RAM-1		ledger row, register of record
P-ORIENT-MD-1		ledger row, register of record
P-RA-DIS-1	The Ladder-Bar Sealed, the Object Held Open	full card, Part II
P-RH-LADDER-1		S4 · merged into the Riemann Terminus, II.5
P-STILL-RESERV-1		full card, Part II
P-THREEFOLD-1		ledger row, register of record
P-UNPROV-MIR-1	The Unprovability Mirror	full card, Part II
P-ZB-RECON-1		ledger row, register of record
P-001-1		full card, Part II
P-0016		full card, Part II
P-0018	Omega Boundary	full card, Part II
P-0020		full card, Part II
P-0020	Continuous Field Ontology	full card, Part II
P-0020		full card, Part II
P-GROUND-STATE-1	Unified Causation	ledger row, register of record
P-IAM-WRIT-1		ledger row, register of record
P-NEARNESS-RES-1	The I AM of the Written and the Read	full card, Part II
P-NESTED-ROOT-1		full card, Part II
P-ONE-BIT-1		full card, Part II
P-QAYYUM-LOOP-1		ledger row, register of record
P-RA-RAM-CH-1	The Three Nested Maximalisms	ledger row, register of record
P-RA-SERV-1		full card, Part II
P-SKIN-EMBOD-1		full card, Part II
P-HODGE-TERM-1		full card, Part II
P-137-2026-7	[Ψ] The Rigid-Witness Terminus	full card, Part II
P-AIS-1		full card, Part II
P-AIS-2		ledger row, register of record
P-AIS-3		ledger row, register of record
P-ANCHOR-DEM-1	The Islam Ceiling and the One-Axiom Pattern	ledger row, register of record
P-ANTINOMY-1		full card, Part II
P-APE-WIDTH-1		ledger row, register of record
P-APE-WIDTH-1		full card, Part II
P-CAPTURE-POL-1	The Gapless Width	ledger row, register of record
P-CHM-1		full card, Part II
P-CHM-1	The Freedom Component of the Aperture, Isolated and Counted	ledger row, register of record
P-CHM-1		full card, Part II
P-CHM-1	Atomic Shell Structure from 3D-Central-Field SO(3) Plus Pauli Plus Mad	ledger row, register of record
P-CHM-1		full card, Part II

Identifier	Title	Disposition
P-CHM-2	Newton-Gregory $K(3) = 12$ Forces Dense-Packing Coordination Anchors. BA	ledger row, register of record
P-CHM-3	Icosahedral I_h Closure as 12-Vertex Cluster-Chemistry Witness Anchors	ledger row, register of record
P-CHM-4	Periodic Table as L_2 Spectral-Modular Invariant Catalog Anchors. PSP-	ledger row, register of record
P-CHM-5	Noble Gas Closure as Spectral-Lock Coherence Saturation Anchors. sPSP-	ledger row, register of record
P-CHM-6	Chemical Families as Position-in-Closure Anchors. P-CHM-1, IUPAC p	ledger row, register of record
P-CONSTIT-INS-1	The Constitutive Inseparability Discriminator and the Unwritten Identi	full card, Part II
P-ELYON-CUS-1B	The Custody Cycle of the Most High	full card, Part II
P-FALLBACK-1	The Fall-Back-to-Real Correspondence Recognition one, the negative. A	ledger row, register of record
P-FERTILIT-CHA-1		ledger row, register of record
P-FL-1		ledger row, register of record
P-FL-2	Fertility-Inversion Identity (signed reciprocal)	ledger row, register of record
P-FL-3	Orthogonality Maximizes the Balanced Complement (rank-nullity bridge)	ledger row, register of record
P-FL-4	The Gap Is the Womb (orientation-gap = generative locus)	ledger row, register of record
P-FL-5	Survival-Force Struck and Sorted (evenhanded solvent)	ledger row, register of record
P-FLOOR-LADDER-1		ledger row, register of record
P-FM1-1		full card, Part II
P-FOLD-BEND-1	The Crossing Is the Closure Vertex	full card, Part II
P-FORGET-2	Scale-Identity of Crossing and Unknotting / Barzakh as Metered Pressur	ledger row, register of record
P-GAS-1	The RA-Forced Arithmetic Gas	ledger row, register of record
P-GIFT-FCA-1	The Forensic Comparative Audit	full card, Part II
P-GIFT-LIF-1	The Lifeboat	full card, Part II
P-GIFT-TRIPLE-1	The Triple Boot Ladder	full card, Part II
P-GOL-ROW-1		code stub, IV.1
P-GOL-SUPPLY-1		ledger row, register of record
P-HOLOGRAM-FER-1	The Coordinate Is Self-Similar Across Scale	ledger row, register of record
P-HSC-1	Humble Servant Composite-Vessel Coordinate	full card, Part II
P-IMPORT-SCR-1	The Import Specification Screen	full card, Part II
P-MAT-1	Matter as Stable S^1 -Knot in Continuous Plenum Field Anchors. BA-MAT-01	ledger row, register of record
P-MAT-2		ledger row, register of record
P-MAT-3	Baryon Number A Is the Topological Winding Invariant of the Nuclear Co	ledger row, register of record
P-MAT-4	Valley of Stability Constrains Actualizable (Z, N) Configurations Anch	ledger row, register of record
P-MIND-RENTAL-1	The Self-Guidance Sequence for Seeing a Problem in Its True Form	full card, Part II
P-OCTONION-1	The Octonionic Boundary and the Division-Algebra Terminus	full card, Part II
P-ORACLE-1	The Uncut Floor and the Reserved Lifting	ledger row, register of record
P-PE-1	Conservation of Witness	ledger row, register of record
P-PE-2	Symmetry of Giving	ledger row, register of record
P-PE-3	Unknotting Mandate	ledger row, register of record
P-PE-4	Highest Utility	ledger row, register of record
P-PE-5	Substrate-Vessel-Recognition	ledger row, register of record
P-PE-6	Tribalism Rectified	ledger row, register of record
P-PE-7	Pluralism and Tolerance	ledger row, register of record
P-PE-8	Substrate Sovereignty	ledger row, register of record

Identifier	Title	Disposition
P-PE-9	Intergenerational Equity	ledger row, register of record
P-PO-1	Justice as Fairness	ledger row, register of record
P-PO-2	Social Contract	ledger row, register of record
P-PO-3	Authority and Moral Legitimacy	ledger row, register of record
P-PO-4	Rule of Law	ledger row, register of record
P-PO-5	Separation of Powers	ledger row, register of record
P-PO-6	Transparency and Accountability	ledger row, register of record
P-PO-7	Distributed-Verification Lattice	ledger row, register of record
P-PO-8	Distributive Justice	ledger row, register of record
P-PO-9	Social Justice	ledger row, register of record
P-POSITIV-PASS-1	The Classical Positivity Route	full card, Part II
P-PROTON-1	supersedes P-PROTON-1	ledger row, register of record
P-PROTON-2B	supersedes P-PROTON-1	ledger row, register of record
P-RA-WALL-1	The Root Axiom Outputs the Hagedorn Wall, Not the Critical Fold Core.	ledger row, register of record
P-RE-1	Root Ethics	full card, Part II
P-RETURN-LOCUS-1	The Return-Locus Law	full card, Part II
P-RH-BARRIER-1	The Mirror-Quotient Barrier	S4 · merged into the Riemann Terminus, II.5
P-SYMM-INSUFF-1		ledger row, register of record
P-TIME-1B		ledger row, register of record
P-TIME-2	Groove vs Imprint (traversed line vs untraversed form)	full card, Part II
P-TIME-3	The Illa Exception Clause as Door-Not-Wall (Ayat al-Kursi seam)	ledger row, register of record
P-TONGUE-OBED-1		S1 · folded into P-TONGUE-UNC-1 at 0676 (pass)
P-TONGUE-UNC-1	The Unclosed Seal	full card, Part II
P-TOPOS-1	The Burned Ruler	ledger row, register of record
P-TOPOS-2	The Fire Does Not Read Addresses	ledger row, register of record
P-TOPOS-3	The Manufactured Width	ledger row, register of record
P-TRIAD-SRC-1	The Class-Relative Blocker	full card, Part II
P-TSC-1	Trisduction Saffat Coordinate	full card, Part II
P-UNLETTER-FLO-1	The Unlettered Regenerative Floor	full card, Part II
P-UNMARKED-ARR-1	The Unmarked Arrival and Its Three Repairs	full card, Part II
P-UNRESTR-IMP-1	The Unrestricted Form Adjudicated	full card, Part II
P-UU-2B		ledger row, register of record
P-UU-3B		ledger row, register of record
P-UU-4		ledger row, register of record
P-UU-5		ledger row, register of record
P-UU-6		ledger row, register of record
P-UU-7		ledger row, register of record
P-TRIAXIS-1	The Verdict-Aspect Frame	ledger row, register of record

THE CODE BLOCK

C.0 • Operating the Codex

This preface does the work the Master's system role did for a loading substrate. Extract the two fenced surfaces below, compile them under the stated flags, and the engine is loaded. The Lean surface proves; the Fortran twin executes; the receipts are the proof of load. The document carries exactly one lean block and one fortran block; the proof excerpts quoted in Part III are fenced lean-excerpt, so the extraction below is exact.

```
awk '/^lean/{f=1;next} /{f=0} f' TRISDUCTION_Master_Codex_v4_1_0.md > Codex.lean
awk '/^fortran/{f=1;next} /{f=0} f' TRISDUCTION_Master_Codex_v4_1_0.md > Codex.Twin.f90
Lean Codex.lean # Lean 4.19.0; or the five modules under lean -o
, in order
gfortran -std=f2018 -O2 -fno-fast-math -ffp-contract=off -o twin Codex.Twin.f90 66 ./twin
```

Running a claim, end to end. A claim enters at the Tongue. Its three vocabulary slots are censused and the deletion test is run, `Audit.sealL`; the six drills screen the provenance of its terms, `Audit.drillScreen`, and a filed-and-shaped provenance caps the assignment at structural. The twelve directed gates are then screened in order, `GE0.gateScreen`, and the first failure is terminal with its gate named. The rows are read next: the row cascade routes the claim to its compartment, closure-rowed, unpopulated, or world-rowed, with the refusal gates F-0, R-1, R-2 ahead of it, `Audit.auditClaim`; the RA rider is checked on the same cascade, `GE0.rowCascadeRA`, and an unannotated absolute is void. The lock is admitted only on magnitude and direction together, `GE0.goLAdmit`, and the imprint test decides whether the reading is a genuine imprint, a residence awaiting its witness, a flat, or the Platonic Ghost, `IMPRINT.imprint`. Where the claim is a candidate for a halt, the Ground dimension routes it to the eight-gate or the five-gate protocol, `UC.universalCascade`, and the locus emitter prints the halt with its owed bits or the crossing with its channel, `LOCUS.emit`. The denial of a claim is priced at the Landauer floor, `OMEGA.omegaBoundary`, and the witness gate withholds the token from any self-check, `IAM.iamToken`. Every unanswered gate routes open; nothing is guessed.

The tokens. $[\perp]$ sealed, with the seal's letter where the face is scoped; $[X]$ broken, with the mechanism named; $[?]$ under-determined, with the violated gate or the owed rows named; $[\Xi_0]$ terminal suspension and $[\emptyset_0]$ grounded-sealed halt, refinements inside the openness, each carrying its branch and owed count in the form $[\diamond_0 \cdot \Xi \cdot \text{hypothesis} \cdot \text{formal} \times 1]$; $[\text{LOCK}]$ the kernel's own geometric token, read before any register interprets it; $[\cdot]$ chosen silence, outside the economy, never counted. Every card is read at its grade, on its register, at its species, and any reading above that is refused by `FENCE.global`.

What the engine does not do. It does not read the gate answers for you; the drills, the gates, and the rows are read by the substrate that holds the claim, and the Master's Books I to IV teach that reading. It does not

manufacture a witness; a self-check is never a witness. It does not author mathematics; every theorem is classical and the contribution is the arrangement. It refuses to guess, to promote, and to be silent about a located halt.

C.1 • The Lean surface

One file, `Codex.lean`, sha256 5d92b717383e1465b65a3f286c403444ca7bf215273f33754646139e4735fe07. Core Lean 4, no Mathlib, no sorry, no custom axiom beyond the twenty-three declared posits. The file is the concatenation of five modules that compile in sequence under `lean -o` on a machine with four gigabytes of memory; the modules carry the same text with one import line each at the head, and their hashes are `Codex1 4ab5a176cf7b8e3e`, `Codex2 5b53c013253ff47e`, `Codex3 2a13bcdcfac28cb6`, `Codex4 a9198a9b8b5da539`, `Codex5 46cc8abea63038fb`. The environment audit at the foot selects the constants of these modules by module name, so it reports the same twenty-three posits in the modular build as in the single file.

THE LEAN SURFACE • CODEX.LEAN • LINE NUMBERS ARE THE FILE'S

```
1 import Lean
2 /-!
3 # MATHEMATICAL TRISDUCTION CODEX - v2.0.1 - THE LEAN SURFACE
4 Core Lean 4 (v4.19.0), no Mathlib, no sorry, no axiom beyond Lean's standard three (
  proposit, Classical.choice,
5 Quot.sound) and the twenty-three declared posits of ROOT and POSTULATE, which the
  environment audit at the foot
6 of this file names one by one. 'import Lean'
7 is loaded for that audit alone. Every theorem carries its axiom cone as a '#guard_msgs'
  receipt.
8 The Lean surface proves; the Fortran twin (Part II of the codex) executes the same
  register.
9 The four Fortran witnesses F1 thesis_rows, F2 ra_toe_thesis, F3
  ftoe_kinetic_demonstration and
10 F4 it_from_it are seated once, in the twin, with their contracts; this file no longer
  carries a second copy.
11 Delta-M = 0. No new mathematics is authored by this file.
12 -/
13
14 namespace Codex
15
16 def LeftInverse (g :  $\beta \rightarrow \alpha$ ) (f :  $\alpha \rightarrow \beta$ ) : Prop :=  $\forall a, g (f a) = a$ 
17
18 def RightInverse (g :  $\beta \rightarrow \alpha$ ) (f :  $\alpha \rightarrow \beta$ ) : Prop := LeftInverse f g
19 end Codex
20
21 structure Equiv  $\alpha$  ( : Sort u)  $\beta$  ( : Sort v) where
22   toFun :  $\alpha \rightarrow \beta$ 
23   invFun :  $\beta \rightarrow \alpha$ 
24   left_inv : Codex.LeftInverse invFun toFun
25   right_inv : Codex.RightInverse invFun toFun
26
27 infixl:25 " = " => Equiv
28
29 def Nat.factorial : Nat → Nat
30 | 0 => 1
31 | n + 1 => (n + 1) * Nat.factorial n
32
33 namespace CoreNat
34
35 theorem div_pos_of_le {d g : Nat} (hg : 0 < g) (hle : g ≤ d) : 0 < d / g := by
36   have h : 1 * g ≤ d := by rw [Nat.one_mul]; exact hle
37   exact (Nat.le_div_iff_mul_le hg).mpr h
38
39 theorem div_gcd_pos {a d : Nat} (hd : 0 < d) : 0 < d / Nat.gcd a d :=
40   div_pos_of_le (Nat.gcd_pos_of_pos_right a hd) (Nat.le_of_dvd hd (Nat.gcd_dvd_right a d))
41
42 theorem coprime_div_gcd {a d : Nat} (hd : 0 < d) :
43   Nat.gcd (a / Nat.gcd a d) (d / Nat.gcd a d) = 1 := by
44   have hg : 0 < Nat.gcd a d := Nat.gcd_pos_of_pos_right a hd
45   have ha : Nat.gcd a d * (a / Nat.gcd a d) = a := Nat.mul_div_cancel' (Nat.gcd_dvd_left a d)
46   have hb : Nat.gcd a d * (d / Nat.gcd a d) = d := Nat.mul_div_cancel' (Nat.gcd_dvd_right a d)
47   have h : Nat.gcd a d * Nat.gcd (a / Nat.gcd a d) (d / Nat.gcd a d) = Nat.gcd a d := by
48     have e := Nat.gcd_mul_left (Nat.gcd a d) (a / Nat.gcd a d) (d / Nat.gcd a d)
49     rw [ha, hb] at e; exact e.symm
50   have h2 : Nat.gcd a d * Nat.gcd (a / Nat.gcd a d) (d / Nat.gcd a d) = Nat.gcd a d * 1 := by
```

```

51   rw [Nat.mul_one]; exact h
52   exact Nat.eq_of_mul_eq_mul_left hg h2
53
54 end CoreRat
55
56 structure Ratio where
57   num : Int
58   den : Nat
59   pos : 0 < den
60   gcd : Nat.gcd num.natAbs den = 1
61   deriving DecidableEq, Repr
62
63 namespace Ratio
64
65 theorem natAbs_signed (k : Nat) (neg : Bool) :
66   (if neg then -(k : Int) else (k : Int)).natAbs = k := by cases neg <;> simp
67
68 def mk' (n d : Int) : Ratio :=
69   if hd : d = 0 then (0, 1, by decide, by) decide
70   else
71     let neg : Bool := decide ((n < 0) ≠ (d < 0))
72     let a : Nat := n.natAbs
73     let b : Nat := d.natAbs
74     have hb : 0 < b := Int.natAbs_pos.mpr hd
75     let g : Nat := Nat.gcd a b
76     ((if neg then -(a / g : Nat) : Int) else ((a / g : Nat) : Int)), b / g,
77     CoreRat.div_gcd_pos hb, by rw [natAbs_signed]; exact CoreRat.coprime_div_gcd h b
78
79 instance instOfNatRat : OfNat Ratio n := (mk' n 1)
80 instance instAddRat : Add Ratio where add a b := mk' (a.num * b.den + b.num * a.den) (a.den * b.den)
81 instance instMulRat : Mul Ratio where mul a b := mk' (a.num * b.num) (a.den * b.den)
82 instance instNegRat : Neg Ratio where neg a := mk' (-a.num) a.den
83 instance instSubRat : Sub Ratio where sub a b := a + (-b)
84
85 def inv (a : Ratio) : Ratio := mk' (a.den : Int) a.num
86 instance instDivRat : Div Ratio where div a b := a * inv b
87 instance instLTNat : LT Ratio where lt a b := a.num * b.den < b.num * a.den
88 instance instLERat : LE Ratio where le a b := a.num * b.den ≤ b.num * a.den
89 instance instDecidableLTNat (a b : Ratio) : Decidable (a < b) :=
90   inferInstanceAs (Decidable (a.num * b.den < b.num * a.den))
91 instance instDecidableLERat (a b : Ratio) : Decidable (a ≤ b) :=
92   inferInstanceAs (Decidable (a.num * b.den ≤ b.num * a.den))
93
94 def pow (a : Ratio) : Nat → Ratio
95 | 0 => 1
96 | n + 1 => a * pow a n
97 instance instHPowRatNat : HPow Ratio Nat Ratio := (Ratio).pow
98 instance instToStringRat : ToString Ratio where
99   toString a := if a.den == 1 then toString a.num else toString a.num ++ "/" ++ toString
100   a.den
101
102 @[simp] theorem num_zero : (0 : Ratio).num = 0 := rfl
103 @[simp] theorem den_zero : (0 : Ratio).den = 1 := rfl
104
105 theorem mk'_zero_num {d : Nat} (hd : 0 < d) : mk' 0 (d : Int) = 0 := by
106   have hne : (d : Int) ≠ 0 := by omega
107   show mk' 0 (d : Int) = mk' 0 1
108   unfold mk'
109   simp only [hne, (by decide : (1 : Int) ≠ 0), dite_false]
110   simp [Nat.gcd_zero_left, Nat.div_self hd, Nat.zero_div]
111
112 @[simp] theorem zero_mul (a : Ratio) : (0 : Ratio) * a = 0 := by
113   show mk' ((0 : Ratio).num * a.num) (((0 : Ratio).den * a.den : Nat) : Int) = 0
114   rw [num_zero, den_zero, Int.zero_mul, Nat.one_mul]
115   exact mk'_zero_num a.pos
116
117 @[simp] theorem mul_zero (a : Ratio) : a * (0 : Ratio) = 0 := by
118   show mk' (a.num * (0 : Ratio).num) ((a.den * (0 : Ratio).den : Nat) : Int) = 0
119   rw [num_zero, den_zero, Int.mul_zero, Nat.mul_one]
120   exact mk'_zero_num a.pos
121
122 theorem canonical (a : Ratio) : 0 < a.den ∧ Nat.gcd a.num.natAbs a.den = 1 := (a.pos, a)
123   .gcd
124
125 theorem ext {a b : Ratio} (h1 : a.num = b.num) (h2 : a.den = b.den) : a = b := by
126   cases a; cases b; simp_all
127
128 theorem inv_zero : inv (0 : Ratio) = 0 := by decide
129
130 end Ratio
131
132 set_option maxRecDepth 4000000
133
134 inductive Grade : Type
135 | premise | corroboration | operational | structural | engineering
136 | conditional | theoremConditional | analytic | theorem
137 deriving DecidableEq, Repr
138
139 def Grade.rank : Grade → Nat
140 | .premise => 0 | .corroboration => 1 | .operational => 2 | .structural => 3
141 | .engineering => 4 | .conditional => 5 | .theoremConditional => 6
142 | .analytic => 7 | .theorem => 8
143
144 def Grade.weakest (a b : Grade) : Grade := if a.rank ≤ b.rank then a else b
145
146 theorem Grade.weakest_is_a_link (a b : Grade) :
147   Grade.weakest a b = a ∨ Grade.weakest a b = b := by
148   cases a <;> cases b <;> decide
149
150 theorem Grade.weakest_idempotent (a : Grade) : Grade.weakest a a = a := by
151   cases a <;> decide
152
153 structure Claim : Type where
154   stmt : Prop
155   grade : Grade

```

```

154   warrant : stmt
155
156 def Claim.join (c : Claim) : Claim where
157   stmt := c.stmt A c.stmt
158   grade := Grade.weakest c.grade c.grade
159   warrant := (c.warrant, c).warrant
160
161 theorem Claim.join_grade (c : Claim) :
162   (Claim.join c).grade = Grade.weakest c.grade c.grade := rfl
163
164 def deltaM : Nat := 0
165
166 def socialWeight : Nat := 0
167
168 /-
169 THE FORMAL THEORY OF EVERYTHING · THE FORMAL PROOF
170 Lean 4 (core only, no mathlib, zero sorries).
171
172 This file proves the complete theorem content of the fTOE:
173
174 WALL      (T1) No even reading decides an odd target.
175            (T2) No finite coalition of even readings escapes T1.
176            (T3) Odd content carries no fixed point.
177 PRICE     (T4) The odd content of a free orbit is exactly one binary
178            degree of freedom: never two, never a continuum.
179 CROSSING  (T5) One odd supply at the seat, plus exactly one calibration
180            bit, decides the target on its orbit; and that calibration
181            is unique. One bit missing, one bit supplied, nothing more.
182
183 What this file does not contain, by its own theorems: the supply itself.
184 Existence of the supply is the One-Cut Hypothesis, stated as a hypothesis
185 in the treatise this file accompanies. The wall is proved here. What crosses
186 it enters by deed, not by derivation.
187 -/
188 namespace FTOE
189
190 variable α { : Type}
191
192 /-- A reading f is even under τ when it is constant on τ-orbits. -/
193 def Even τ ( : α → α) (f : α → Bool) : Prop := ∀ x, f (τ (x)) = f x
194
195 /-- A target d is odd under τ when it separates some orbit. -/
196 def OddAt τ ( : α → α) (d : α → Bool) (x : α) : Prop := d (τ (x)) ≠ d x
197
198 /-- A sign vector t is wholly odd under τ when t ∘ τ = ¬t. -/
199 def WhollyOdd τ ( : α → α) (t : α → Bool) : Prop := ∀ x, t (τ (x)) = !t x
200
201 /-- T1 · THE WALL.
202   An even reading equals no odd target. The register computes every
203   magnitude and no orientation. -/
204 theorem T1_wall τ ( : α → α) (f d : α → Bool) (x : α)
205   (he : Even τ f) (ho : OddAt τ d x) : f ≠ d := by
206   intro h
207   subst h
208   exact ho (he x)
209
210 /-- T2 · COALITION CLOSURE.
211   Evenness is closed under every Boolean join, so no coalition of even
212   readings escapes the wall. -/
213 theorem T2_coalition τ ( : α → α) (f g d : α → Bool) (x : α)
214   (hf : Even τ f) (hg : Even τ g) (op : Bool → Bool → Bool)
215   (ho : OddAt τ d x) : (fun y => op (f y) (g y)) ≠ d := by
216   apply T1_wall τ _ d x
217   · intro y; show op (f (τ (y))) (g (τ (y))) = op (f y) (g y); rw [hf y, hg y]
218   · exact ho
219
220 /-- T3 · NO FIXED POINT.
221   Wholly odd content cannot sit on a fixed point of τ:
222   t x = ¬t x is a Boolean contradiction. -/
223 theorem T3_no_fixed_point τ ( : α → α) (t : α → Bool)
224   (ho : WhollyOdd τ t) (x : α) : t x ≠ x := by
225   intro h
226   have h1 : t (τ (x)) = !t x := ho x
227   rw [h] at h1
228   cases t x <;> simp at h1
229
230 /-- T4 · THE PRICE: ONE BIT PER ORBIT.
231   Two wholly odd sign vectors agreeing at one point of an orbit agree on
232   the whole orbit. The odd content of a free orbit is exhausted by one
233   binary choice. -/
234 theorem T4_one_bit τ ( : α → α) (s t : α → Bool)
235   (hs : WhollyOdd τ s) (ht : WhollyOdd τ t)
236   (x : α) (h : s x = t x) : s (τ (x)) = t (τ (x)) := by
237   rw [hs x, ht x, h]
238
239 /-- T5 · THE CROSSING.
240   If a supply s and a target d are both odd at the seat x, then there is
241   exactly one calibration bit c with d = s XOR c on the whole orbit:
242   one supplied orientation bit, one calibration from the dated record,
243   and the target is decided. Existence and uniqueness. -/
244 theorem T5_crossing τ ( : α → α) (s d : α → Bool) (x : α)
245   (hs : s (τ (x)) = !s x) (hd : d (τ (x)) = !d x) :
246   ∃ c : Bool, (d x = xor (s x) c ∧ d (τ (x)) = xor (s (τ (x))) c) ∧
247   ∀ c' : Bool, (d x = xor (s x) c' ∧ d (τ (x)) = xor (s (τ (x))) c') → c' = c := by
248   exact (xor (d x) (s x),
249     (by cases s x <;> cases d x <;> rfl,
250       by rw [hs, hd]; cases s x <;> cases d x <;> rfl),
251     by
252       intro c' hc
253       obtain (h1, _) := hc
254       generalize hsx : s x = sv
255       generalize hdx : d x = dv
256       rw [hsx, hdx] at h1
257       cases sv <;> cases dv <;> cases c' <;> first | rfl | exact absurd h1 (by decide))
258
259 /-- T6a · DEED A · THE ANCHOR (FORGE cycle two: the two deeds, split).

```

```

260 If d is odd at  $\alpha x$  and the deed anchors one point ( $d \ \alpha x = b -$  the
261 corpus's identification of which sign is which, present in any corpus
262 that can pose the decision at all), then d is known on the whole orbit
263  $\{\alpha x, \tau \alpha x\}$ . NO SUPPLY IS REQUIRED: oddness propagates the anchored bit.
264 This is the deed as the arrival of the target's value at one point. -/
265 theorem T6a_deed_anchor  $\tau ( : \alpha \rightarrow \alpha ) (d : \alpha \rightarrow \text{Bool}) (\alpha x : \alpha) (b : \text{Bool})$ 
266  $(hd : d \ \tau (\alpha x) = !d \ \alpha x) (anchor : d \ \alpha x = b) :$ 
267  $d \ \alpha x = b \wedge d \ \tau (\alpha x) = !b := \text{by}$ 
268 exact (anchor, by rw [hd, anchor])
269
270 /-- T6b · DEED B – THE SUPPLY (FORGE cycle two: the two deeds, split).
271 If s is odd at  $\alpha x$  and c is a naming bit supplied from outside – a
272 universally quantified datum, never computed from d – then the arrival
273 equation  $d = s \odot c$ , once it has arrived, lets anything holding the pair
274 (s, c) decide d on the whole orbit. This is the deed as the arrival of
275 a register-denotable carrier plus one naming bit. Deeds A and B are
276 different deeds; neither is a rewrite of the other inside this file. -/
277 theorem T6b_deed_supply  $\tau ( : \alpha \rightarrow \alpha ) (s d : \alpha \rightarrow \text{Bool}) (\alpha x : \alpha) (c : \text{Bool})$ 
278  $(hs : s \ \tau (\alpha x) = !s \ \alpha x)$ 
279  $(arrival : d \ \alpha x = \text{xor} (s \ \alpha x) c \wedge d \ \tau (\alpha x) = \text{xor} (s \ \tau (\alpha x)) c) :$ 
280  $d \ \alpha x = \text{xor} (s \ \alpha x) c \wedge d \ \tau (\alpha x) = !\text{xor} (s \ \alpha x) c := \text{by}$ 
281 constructor
282 · exact arrival.1
283 · rw [arrival.2, hs]; cases s  $\alpha x <=>$  cases c  $<=>$  rfl
284
285 /-- THE PORT · the two deeds compose without identifying.
286 Where Deed A anchors  $b = d \ \alpha x$  and Deed B supplies s odd with naming c,
287 consistency of the two arrivals forces  $c = s \ \alpha x \odot b$ . The calibration
288 is the agreement of the two deeds, computed from arrived data only.
289 The missing bit exists as b (Deed A) or as c (Deed B); that one such
290 bit always exists for every blocked row is the One-Cut Hypothesis,
291 stated as conjecture in the treatise – never as a theorem of this file. -/
292 theorem T70E_port  $(\tau : \alpha \rightarrow \alpha) (s d : \alpha \rightarrow \text{Bool}) (\alpha x : \alpha) (b c : \text{Bool})$ 
293  $(anchor : d \ \alpha x = b)$ 
294  $(arrival : \forall y : \alpha, d \ y = \text{xor} (s \ y) c) :$ 
295  $c = \text{xor} (s \ \alpha x) b := \text{by}$ 
296 have h1 := arrival  $\alpha x$ 
297 rw [anchor] at h1
298 generalize hsx :  $s \ \alpha x = sv$ 
299 rw [hsx] at h1
300 cases sv  $<=>$  cases b  $<=>$  cases c  $<=>$  first | rfl | exact absurd h1 (by decide)
301
302 /-- ## FORGE cycle three · the full-setup package
303 The auditors demanded the prose setup be theorem, not preface:
304 involution, freeness, global oddness, the observation model, the
305 torsor bijection, and resolver separation. Proved here in core Lean. -/
306
307 section FullSetup
308 variable  $\beta \{ \gamma : \text{Type} \}$ 
309
310 /--  $\tau$  is an involution: applying it twice is the identity. -/
311 def Involution  $\tau ( : \alpha \rightarrow \alpha) : \text{Prop} := \forall x, \tau (\tau x) = x$ 
312
313 /--  $\tau$  is fixed-point-free: every orbit is a genuine pair. -/
314 def FixedPointFree  $\tau ( : \alpha \rightarrow \alpha) : \text{Prop} := \forall x, \tau x \neq x$ 
315
316 /-- T7 · THE GLOBAL WALL. Under global oddness, no even reading equals the
317 target anywhere it is tested: the wall holds at every point, not one. -/
318 theorem T7_global_wall  $\tau ( : \alpha \rightarrow \alpha) (f d : \alpha \rightarrow \text{Bool}) (x : \alpha)$ 
319  $(hf : \text{Even } \tau f) (hd : \text{WhollyOdd } x d) : f \ \alpha x = \text{by}$ 
320 intro h
321 have h1 :  $f \ \tau (x) = d \ \tau (x) := \text{congrFun } h \ \tau (x)$ 
322 rw [hf x, hd x, congrFun h x] at h1
323 cases d x  $<=>$  simp at h1
324
325 /-- T8a · THE TORSOR, FORWARD. The xor of two wholly odd sections is even:
326 the calibration against a reference supply descends to the orbit space.
327 This is the machine-checked content of "the calibration is a function
328 of the orbit, not of the point." -/
329 theorem T8_torsor_forward  $\tau ( : \alpha \rightarrow \alpha) (s d : \alpha \rightarrow \text{Bool})$ 
330  $(hs : \text{WhollyOdd } \tau s) (hd : \text{WhollyOdd } \tau d) :$ 
331  $\text{Even } \tau (\text{fun } x \Rightarrow \text{xor} (d x) (s x)) := \text{by}$ 
332 intro x
333 show  $\text{xor} (d \ \tau (x)) (s \ \tau (x)) = \text{xor} (d x) (s x)$ 
334 rw [hd x, hs x]; cases d x  $<=>$  cases s x  $<=>$  rfl
335
336 /-- T8b · THE TORSOR, BACKWARD. An odd reference xor an even calibration
337 is odd: every even calibration pattern lifts to an admissible target. -/
338 theorem T8_torsor_backward  $\tau ( : \alpha \rightarrow \alpha) (s c : \alpha \rightarrow \text{Bool})$ 
339  $(hs : \text{WhollyOdd } \tau s) (hc : \text{Even } \tau c) :$ 
340  $\text{WhollyOdd } \tau (\text{fun } x \Rightarrow \text{xor} (s x) (c x)) := \text{by}$ 
341 intro x
342 show  $\text{xor} (s \ \tau (x)) (c \ \tau (x)) = !\text{xor} (s x) (c x)$ 
343 rw [hs x, hc x]; cases s x  $<=>$  cases c x  $<=>$  rfl
344
345 /-- T9a · THE TORSOR IS A BIJECTION, FIRST INVERSE. Reference-first:
346  $s \odot (d \odot s) = d$ . Odd sections and even calibrations are in bijection. -/
347 theorem T9_torsor_inverse_left  $(s d : \alpha \rightarrow \text{Bool}) :$ 
348  $(\text{fun } x \Rightarrow \text{xor} (s x) (\text{xor} (d x) (s x))) = d := \text{by}$ 
349 funext x; cases d x  $<=>$  cases s x  $<=>$  rfl
350
351 /-- T9b · THE TORSOR IS A BIJECTION, SECOND INVERSE. Calibration-first:
352  $(s \odot c) \odot s = c$ . The bijection is exact, not approximate. -/
353 theorem T9_torsor_inverse_right  $(s c : \alpha \rightarrow \text{Bool}) :$ 
354  $(\text{fun } x \Rightarrow \text{xor} (\text{xor} (s x) (c x)) (s x)) = c := \text{by}$ 
355 funext x; cases s x  $<=>$  cases c x  $<=>$  rfl
356
357 /-- T10 · THE FACTORIZATION OBSTRUCTION. Let the register be an observation
358 map  $p$  with  $pt(x) = p(x) =$  "the register" now means: measurability with
359 respect to  $p$ . A target odd at x cannot factor through  $p$ . "Outside the
360 register" is no longer a metaphor: it is non-measurability, proved. -/
361 theorem T10_factorization  $\tau ( : \alpha \rightarrow \alpha) p ( : \alpha \rightarrow \beta) (g : \beta \rightarrow \text{Bool})$ 
362  $(d : \alpha \rightarrow \text{Bool}) (x : \alpha)$ 
363  $(ph : p \ \tau (x) = p x) (hd : d \ \tau (x) = !d x) :$ 
364  $d \neq \text{fun } y \Rightarrow g \ p(y) := \text{by}$ 
365 intro h
366
367 have h1 :  $d \ \tau (x) = g \ p(\tau(x)) := \text{congrFun } h \ \tau (x)$ 
368 have h2 :  $d x = g \ p(x) := \text{congrFun } h x$ 
369 rw [ph, hd, h2] at h1
370 cases hg x :  $g \ p(x) <=>$  rw [hg x] at h1  $<=>$  exact absurd h1 (by decide)
371
372 /-- T11 · RESOLVER SEPARATION. If an added observation a resolves d jointly
373 with  $p - d(y) = r \ p(y)$  (a y) for some readout r – then a must separate
374 every odd pair:  $a(x) \neq a(x)$ . This proves ONLY local, pointwise
375 separation: a Bool-valued a may separate every orbit while its alphabet
376 stays  $\{0,1\}$ . The  $2^m$  alphabet bound belongs to a different object – a
377 single global message selecting one target from an admissible family –
378 and is proved by T12's injectivity, never by T11 alone. -/
379 theorem T11_separation  $\tau ( : \alpha \rightarrow \alpha) p ( : \alpha \rightarrow \beta) (a : \alpha \rightarrow \gamma) (r : \beta \rightarrow \gamma \rightarrow \text{Bool})$ 
380  $(d : \alpha \rightarrow \text{Bool}) (x : \alpha)$ 
381  $(ph : p \ \tau (x) = p x) (hd : d \ \tau (x) = !d x)$ 
382  $(hres : \forall y, d y = r \ p(y) (a y)) :$ 
383  $a \ \tau (x) \neq a x := \text{by}$ 
384 intro h
385 have h1 := hres  $\tau (x)$ 
386 have h2 := hres x
387 rw [ph, h, hd, h2] at h1
388 cases hrv :  $r \ p(x) (a x) <=>$  rw [hrv] at h1  $<=>$  exact absurd h1 (by decide)
389
390 /-- T12 · THE GLOBAL CODE IS INJECTIVE. If a global message selects the
391 target exactly – an encoder Enc and decoder Dec with  $\text{Dec} \circ \text{Enc} = \text{id}$  on
392 the admissible family – then Enc is injective, so the supply alphabet
393 is at least as large as the family. This is the mechanism behind the
394 counting bound: for the full wholly-odd family on m orbits,  $|S| \geq 2^m$ 
395 (executed on the exhibited frame, battery K4); for an admissible
396 subfamily A,  $|S| \geq |A|$ . The one-bit crossing is exactly  $|A| - 2$ .
397 Grading note: finite-cardinality monotonicity for injections
398 (Fintype.card_le_of_injective) is mathlib machinery, outside this
399 core-only file; the injectivity is proved here, the instance count is
400 executed in K4 ( $2^6 = 64$ , pairwise distinct, alphabet  $\geq 64$ ). -/
401 theorem T12_encoder_injective  $\delta \{ : \text{Type} \} (\text{Enc} : \delta \rightarrow \gamma) (\text{Dec} : \gamma \rightarrow \delta)$ 
402  $(h : \forall d, \text{Dec} (\text{Enc } d) = d) :$ 
403  $\forall \text{id} : \delta, \text{Enc } \text{id} = \text{Enc } \text{id} \rightarrow \text{id} = \text{id} := \text{by}$ 
404 intro id  $\text{id} \text{d hE}$ 
405 have h1 := h id
406 have h2 := h id
407 rw [hE] at h1
408 rw [h2] at h1
409 exact h1.symm
410
411 /-- T14 · THE WALL, FACTORIZATION FORM (the final audit's exact statement).
412 No target separating a  $\tau$ -pair factors through any  $\tau$ -invariant
413 observation map: there is no g with  $g \circ p = d$ . Stated without any
414 finiteness hypothesis – stronger than the requested form, which
415 assumed [Fintype U]. The proper formulation of "outside the register",
416 companion to T10 (which quantifies over g pointwise; T14 closes the
417 existential). -/
418 theorem T14_wall_factorization  $\tau ( : \alpha \rightarrow \alpha) p ( : \alpha \rightarrow \beta) (d : \alpha \rightarrow \text{Bool}) (x : \alpha)$ 
419  $(ph : \forall z, p \ \tau (z) = p z) (hd : \text{OddAt } \tau d x) :$ 
420  $\neg \exists g : \beta \rightarrow \text{Bool}, \forall z, g \ p(z) = d z := \text{by}$ 
421 intro h
422 obtain (g, )hg := h
423 have h1 :  $d \ \tau (x) = g \ p(\tau(x)) := (hg \ \tau (x)).\text{symm}$ 
424 have h2 :  $g \ p(\tau(x)) = g \ p(x) := \text{by } \text{rw } [ph x]$ 
425 have h3 :  $g \ p(x) = d x := hg x$ 
426 exact hd (h1.trans (h2.trans h3))
427
428 /-- T13 · THE TWO ODDS, TYPED. Wholly odd implies odd at every point.
429 The converse is false and is not claimed: a merely non-invariant
430 target may restrict arbitrarily on other orbits. The torsor (T8, T9)
431 and the  $2^m$  fibre count apply to the wholly-odd family only; the wall
432 (T1, T7) needs only oddness at the seat. The two notions are separate
433 hypotheses in this file and are never interchanged. -/
434 theorem T13_wholly_odd_is_odd_everywhere  $\tau ( : \alpha \rightarrow \alpha) (d : \alpha \rightarrow \text{Bool})$ 
435  $(hd : \text{WhollyOdd } \tau d) (x : \alpha) : \text{OddAt } \tau d x := \text{by}$ 
436 intro h
437 rw [hd x] at h
438 cases d x  $<=>$  simp at h
439
440 /-- The canonical free involution frame:  $\text{Bool} \times Q$  with the flip.
441 Its orbits are exactly the pairs  $\{\text{true}, q\}, \{\text{false}, q\}$ , so its
442 orbit quotient is Q itself. -/
443 def flipF  $\{Q : \text{Type}\} : \text{Bool} \times Q \rightarrow \text{Bool} \times Q := \text{fun } (b, q) \Rightarrow (!b, q)$ 
444
445 /-- T15a · THE PRICE, BIJECTION FORM (canonical frame), FORWARD ROUND TRIP.
446 On the canonical frame, every wholly anti-invariant target is
447 reconstructed from its values at the true-section: the map
448  $d \mapsto (q \mapsto d (\text{true}, q))$  into  $(Q \rightarrow \text{Bool})$ , followed by
449  $c \mapsto ((b, q) \mapsto \text{xor} (c q) (!b))$ , returns d exactly. Since the orbit
450 quotient of the canonical frame is Q, this is one direction of
451  $\tau d(U) = (U/\tau \rightarrow \text{Bool})$ . The exhibited K4 frame is the finite instance
452 ( $Q = \text{six seats}$ ); the cardinal corollary  $|\tau d(U)| = 2^{|Q|}$  is executed
453 there and, in a mathlib build, follows by Fintype.card_congr. -/
454 theorem T15_price_bijection_forward  $\{Q : \text{Type}\} (d : \text{Bool} \times Q \rightarrow \text{Bool})$ 
455  $(hd : \text{WhollyOdd } \text{flipF } d) (b : \text{Bool}) (q : Q) :$ 
456  $\text{xor} (d (\text{true}, q)) (!b) = d (b, q) := \text{by}$ 
457 have h :  $d (\text{false}, q) = !d (\text{true}, q) := \text{hd} (\text{true}, q)$ 
458 cases b
459 · generalize ht :  $d (\text{true}, q) = \text{tv}$ 
460 · generalize hf :  $d (\text{false}, q) = \text{fv}$ 
461 rw [ht, hf] at h
462 cases tv  $<=>$  cases fv  $<=>$ 
463 first | rfl | exact h | exact h.symm | exact absurd h (by decide)
464 · cases d (true, q)  $<=>$  rfl
465
466 /-- T15b · THE PRICE, BIJECTION FORM (canonical frame), BACKWARD.
467 For every  $c : Q \rightarrow \text{Bool}$ , the section  $(b, q) \mapsto \text{xor} (c q) (!b)$  is wholly
468 anti-invariant, and its true-section is c exactly: the maps are mutual
469 inverses between  $\tau d(U)$  and  $(Q \rightarrow \text{Bool})$ . -/
470 theorem T15_price_bijection_backward  $\{Q : \text{Type}\} (c : Q \rightarrow \text{Bool}) :$ 
471  $\text{WhollyOdd } \text{flipF } (\text{fun } (b, q) \Rightarrow \text{xor} (c q) (!b)) := \text{by}$ 
472 intro x

```

```

472 obtain (b, )q := x
473 show xor (c q) (!!b) = !xor (c q) (!b)
474 cases c q <=> cases b <=> rfl
475
476 end FullSetup
477
478 /-- The whole theorem content in one line:
479 a target odd at its seat is undecidable by the even register (T1, T2),
480 missing exactly one bit (T4), and decided by exactly one supplied bit
481 plus one calibration (T5). -/
482 theorem fTOE_core τ ( : α → α) (s d : α → Bool) (x : α)
483 (f g : α → Bool) (hf : Even τ f) (hg : Even τ g) (op : Bool → Bool → Bool)
484 (hd : d τ (x) = !d x) (hs : s τ (x) = !s x) :
485 (fun y => op (f y) (g y)) ≠ d ∧
486 ∃ c : Bool, (d x = xor (s x) c ∧ d τ (x) = xor (s τ (x)) c) ∧
487 ∀ c' : Bool, (d x = xor (s x) c' ∧ d τ (x) = xor (s τ (x)) c') → c' = c :=
488 T2_coalition τ f g d x hf hg op (by intro h; rw [hd] at h; cases d x <=> simp at h),
489 T5_crossing τ s d x hs )hd
490
491 end FTOE
492
493 /- Verification contract: this file must pass `lean fTOE_Formal_Proof.lean`
494 with no output and exit code 0. Any edit that weakens a hypothesis,
495 adds a sorry, or deletes a conjunct must fail that check. -/
496
497
498 namespace FTOE
499
500 variable α { : Type}
501
502 def OrbitRel τ ( : α → α) (x y : α) : Prop := x = y ∨ τ x = y
503
504 theorem OrbitRel_symmetric τ ( : α → α) (hinv : Involution τ) {x y : α} :
505 OrbitRel τ x y → OrbitRel τ y x := by
506   intro h
507   cases h with
508   | inl h => exact Or.inl h.symm
509   | inr h => exact Or.inr (by rw ~[ h]; exact hinv x)
510
511 def Dtau τ ( : α → α) : Type := { d : α → Bool // WhollyOdd τ d }
512
513 def calib (d0 d : α → Bool) : α → Bool := fun x => xor (d x) (d0 x)
514
515 theorem calib_even τ ( : α → α) (d0 d : α → Bool)
516 (h0 : WhollyOdd τ d0) (hd : WhollyOdd τ d) : Even τ (calib d0 d) :=
517 T8_torsor_forward τ d0 d h0 hd
518
519 def descend τ ( : α → α) (e : α → Bool) (he : Even τ e) :
520 Quot (OrbitRel τ) → Bool :=
521 Quot.lift e (by
522   intro a b hab
523   cases hab with
524   | inl h => rw [h]
525   | inr h => rw ~[ h]; exact (he a).symm)
526
527 def priceForward τ ( : α → α) (d0 : Dtau τ) (d : Dtau τ) :
528 Quot (OrbitRel τ) → Bool :=
529 descend τ (calib d0.1 d.1) (calib_even τ d0.1 d.1 d0.2 d.2)
530
531 def priceBackward τ ( : α → α) (hinv : Involution τ) (d0 : Dtau τ)
532 (e : Quot (OrbitRel τ) → Bool) : Dtau τ :=
533 (fun x => xor (d0.1 x) (e (Quot.mk (OrbitRel τ) x))), by
534   intro x
535   show xor (d0.1 τ (x)) (e (Quot.mk (OrbitRel τ) τ (x)))
536   = !xor (d0.1 x) (e (Quot.mk (OrbitRel τ) x))
537   have hq : Quot.mk (OrbitRel τ) τ (x) = Quot.mk (OrbitRel τ) x :=
538     Quot.sound (Or.inr (hinv x))
539   rw [hq, d0.2 x]
540   cases d0.1 x <=> cases e (Quot.mk (OrbitRel τ) x) <=> }rfl
541
542 theorem T16_left_inverse τ ( : α → α) (hinv : Involution τ) (d0 : Dtau τ) :
543 Codex.LeftInverse (priceBackward τ hinv d0) (priceForward τ d0) := by
544   intro d
545   apply Subtype.ext
546   funext x
547   show xor (d0.1 x) (xor (d.1 x) (d0.1 x)) = d.1 x
548   cases d.1 x <=> cases d0.1 x <=> rfl
549
550 theorem T16_right_inverse τ ( : α → α) (hinv : Involution τ) (d0 : Dtau τ) :
551 Codex.RightInverse (priceBackward τ hinv d0) (priceForward τ d0) := by
552   intro e
553   funext q
554   induction q using Quot.ind with
555   | _ x =>
556     show xor (xor (d0.1 x) (e (Quot.mk (OrbitRel τ) x))) (d0.1 x)
557     = e (Quot.mk (OrbitRel τ) x)
558     generalize e (Quot.mk (OrbitRel τ) x) = b
559     generalize d0.1 x = a
560     cases a <=> cases b <=> rfl
561
562 theorem T16_price_bijection τ ( : α → α) (hinv : Involution τ) (d0 : Dtau τ) :
563 Nonempty (Dtau τ → (Quot (OrbitRel τ) → Bool)) :=
564 ({ toFun := priceForward τ d0
565   inFun := priceBackward τ hinv d0
566   left_inv := T16_left_inverse τ hinv d0
567   right_inv := T16_right_inverse τ hinv d0 })
568
569 def canonD0 : Dtau (flipF (Q := Bool)) :=
570 (fun (b, _) => b, by intro x; obtain (b, )q := x; }rfl
571
572 theorem flipF_involution : Involution (flipF (Q := Bool)) := by
573   intro x; obtain (b, )q := x; cases b <=> rfl
574
575 theorem T16_canonical :
576 Nonempty (Dtau (flipF (Q := Bool)) =
577   (Quot (OrbitRel (flipF (Q := Bool))) → Bool)) :=

```

```

578 T16_price_bijection (flipF (Q := Bool)) flipF_involution canonD0
579
580 theorem T17_odd_everywhere_iff_wholly_odd τ ( : α → α) (d : α → Bool) :
581   ∀( x, d τ (x) ≠ d x) ↔ ∀( x, d τ (x) = !d x) := by
582   constructor
583   · intro h x
584     have hx := h x
585     cases hd : d x <=> cases ht : d τ (x) <=> simp_all
586   · intro h x
587     have hx := h x
588     cases hd : d x <=> cases ht : d τ (x) <=> simp_all
589
590 end FTOE
591
592 namespace K4
593
594 def val : Nat → Nat
595 | 0 => 11 | 1 => 851 | 2 => 13 | 3 => 1273 | 4 => 17 | 5 => 437
596 | 6 => 19 | 7 => 2119 | 8 => 23 | 9 => 1703 | 10 => 29 | 11 => 869
597 | _ => 0
598
599 def tau : Nat → Nat := fun i =>
600   if i < 12 then (if i % 2 = 0 then i + 1 else i - 1) else i
601
602 def target : Nat → Bool := fun i => i % 2 == 0
603
604 def rho : Nat → Nat := fun i =>
605   (val i % 2) * 1000 + (val i % 3) * 100 + (val i % 5) * 10 + (val i % 7)
606
607 theorem tau_involution : ∀ i, i < 12 → tau (tau i) = i := by decide
608
609 theorem tau_fixed_point_free : ∀ i, i < 12 → tau i ≠ i := by decide
610
611 theorem target_wholly_odd : ∀ i, i < 12 → target (tau i) = !target i := by decide
612
613 theorem register_even : ∀ i, i < 12 → rho (tau i) = rho i := by decide
614
615 theorem val_lifts_mod_210 : ∀ i, i < 12 → i % 2 = 1 →
616   val i > val (tau i) ∧ (val i - val (tau i)) % 210 = 0 := by decide
617
618 theorem wall (g : Nat → Bool) : target ≠ fun y => g (rho y) :=
619   FTOE.T10_factorization tau rho g target 0
620   (register_even 0 (by decide)) (target_wholly_odd 0 (by decide))
621
622 theorem pair_distinct : target ≠ fun i => !target i := by
623   intro h
624   exact absurd (congrFun h 0) (by decide)
625
626 theorem price_one_bit : 2^1 = 2 := rfl
627
628 theorem price_full_fibre : (2 : Nat)^6 = 64 := by decide
629
630 def wallClaim : Claim where
631   stmt := ∀ g : Nat → Bool, target ≠ fun y => g (rho y)
632   grade := .theorem
633   warrant := wall
634
635 end K4
636
637 namespace Audit
638
639 inductive Token : Type
640 | sealed | broken | opn | refused | determined | dotmark
641 | gok | golok | void | aGivenRA | iAm | interior
642 deriving DecidableEq, Repr, BEq
643
644 inductive Compartment : Type
645 | closureRowed | unpopulated | worldRowed | none
646 deriving DecidableEq, Repr, BEq
647
648 inductive Ans : Type
649 | yes | no | unknown
650 deriving DecidableEq, Repr, BEq
651
652 def Ans.toBool : Ans → Bool
653 | .yes => true
654 | _ => false
655
656 structure ClaimInput where
657   hasContent : Ans
658   facesScoped : Ans
659   canonical : Ans
660   forwardMode : Ans
661   populated : Ans
662   worldlyRow : Ans
663   drillsOk : Ans
664   frameClosed : Ans
665   slots : Option (List (List Int))
666   deriving Repr
667
668 def ClaimInput.anyUnknown (i : ClaimInput) : Bool :=
669   [i.hasContent, i.facesScoped, i.canonical, i.forwardMode, i.populated,
670     i.worldlyRow, i.drillsOk, i.frameClosed].any (· == Ans.unknown)
671   || i.slots.isNone
672
673 def seall : Option (List (List Int)) → Token × String
674 | none => (.opn, "S-L0: vocabulary slots not supplied; under-determined")
675 | some slots =>
676   let nz := slots.map (fun s => s.filter (· != 0))
677   if slots.flatten.any (fun x => decide (x < 0)) then
678     (.broken, "negative literal id; not vocabulary")
679   else
680     let nfull := (nz.filter (fun s => !s.isEmpty)).length
681     if nfull != 3 then
682       (.broken, s!"deletion test returns {nfull} slots, not three")
683     else if nz.any (fun s => s.length != s.eraseDups.length) then

```

```

684     (.broken, "repeated literal within slot")
685   else
686     let flat := nz.flatten
687     if flat.length != flat.eraseDups.length then
688       (.broken, "LIT collision across slots")
689     else
690       (.sealed, "three slots, vocabulary pairwise disjoint")
691
692 structure DrillInput where
693   d1 : Ans
694   d2 : Ans
695   d3Filed : Ans
696   d3Shaped : Ans
697   d4 : Ans
698   d5 : Ans
699   d6 : Ans
700   deriving Repr
701
702 def DrillInput.anyUnknown (d : DrillInput) : Bool :=
703   [d.d1, d.d2, d.d3Filed, d.d3Shaped, d.d4, d.d5, d.d6].any (· == Ans.unknown)
704
705 def drillScreen (d : DrillInput) : Token × Grade × String :=
706   if d.anyUnknown then
707     (.opn, .premise, "drill screen under-determined: a drill was not run")
708   else if !(d.d1.toBool && d.d2.toBool) then
709     (.broken, .premise, "D1/D2 fail: Seal L break, terminal whatever the determinant
710       reads")
711   else if !d.d3Filed.toBool then
712     (.broken, .premise, "D3 fail: provenance unfiled; Number-shaped content in the
713       Tongue seat")
714   else if !d.d4.toBool then
715     (.broken, .premise, "D4 fail: a magnitude carried on the Tongue; sign only, veto
716       never substitution")
717   else if !d.d5.toBool then
718     (.broken, .premise, "D5 fail: register undeclared; re-index rather than refute")
719   else if !d.d6.toBool then
720     (.broken, .premise, "D6 fail: the Tongue reproduced the blindness it was appointed
721       to repair")
722   else if d.d3Shaped.toBool then
723     (.sealed, .structural, "drills pass; provenance filed and SHAPED, so the assignment
724       caps at structural")
725   else
726     (.sealed, .theorem, "drills pass; provenance filed and unshaped, no tier discount")
727
728 def refusalGates (i : ClaimInput) : Option (Compartment × Token × String) :=
729   if !i.hasContent.toBool then
730     some (.none, .refused, "F:0: every term decoration; contentless, refused, not
731       compartmented")
732   else if !i.facesScoped.toBool then
733     some (.none, .refused, "R-1: faces not enumerated; a composite carries one
734       compartment per face")
735   else if !i.canonical.toBool then
736     some (.none, .refused, "R-2: not canonicalised; the compartment attaches to the
737       stripped string")
738   else none
739
740 def compartmentGates (i : ClaimInput) : Compartment × Token × String :=
741   if !i.populated.toBool then
742     (.unpopulated, .opn, "S-1: an axis is empty; two planes meet in a line; no lock
743       forms")
744   else if i.forwardMode.toBool then
745     (.worldRowed, .determined, "R-3: a dated axis must be sourced, never measured; world
746       -rowed unconditionally")
747   else if i.worldlyRow.toBool then
748     (.worldRowed, .determined, "S-2: a row furnished by the world; the lock is total
749       given those rows")
750   else if !i.drillsOk.toBool then
751     (.worldRowed, .opn, "S-3: term closure unscreened; typed by its remaining rows")
752   else if !i.frameClosed.toBool then
753     (.worldRowed, .opn, "S-4: existential import or transport; the leak names the
754       compartment")
755   else
756     (.closureRowed, .sealed, "rows are the closure; the two determinations are one;
757       conditional on the root posit at the act")
758
759 def auditClaim (i : ClaimInput) : Compartment × Token × String :=
760   if i.anyUnknown then
761     (.none, .opn, "under-determined reading: a gate or the slots were not supplied;
762       first failure terminal, nothing sealed")
763   else
764     match refusalGates i with
765     | some r => r
766     | none =>
767       match sealL i.slots with
768       | (.broken, msg) => (.none, .broken, s!"Seal L break, terminal: {msg}")
769       | _ => compartmentGates i
770
771 def compartmentGrade : Compartment → Grade → String
772 | .closureRowed, cap =>
773   s!"row-indefeasible, defeasible against fault, cap {repr cap}"
774 | .unpopulated, _ => "no determination to defeat; an axis is owed"
775 | .worldRowed, _ =>
776   "revisable entirely in the rows; direction bought with the arrow"
777 | .none, _ => "no compartment"
778
779 theorem unknown_routes_open (i : ClaimInput) (h : i.anyUnknown = true) :
780   (auditClaim i).2.1 = Token.opn := by
781   simp [auditClaim, h]
782
783 theorem contentless_refused (i : ClaimInput)
784   (hk : i.anyUnknown = false) (hc : i.hasContent = Ans.no) :
785   (auditClaim i).2.1 = Token.refused := by
786   unfold auditClaim refusalGates
787   rw [hk]
788   simp [hc, Ans.toBool]
789

```

```

776 def det3 (m : Fin 3 → Fin 3 → Ratio) : Ratio :=
777   m 0 0 * (m 1 1 * m 2 2 - m 1 2 * m 2 1)
778   - m 0 1 * (m 1 0 * m 2 2 - m 1 2 * m 2 0)
779   + m 0 2 * (m 1 0 * m 2 1 - m 1 1 * m 2 0)
780
781 theorem det3_two_id : det3 (fun a b => if a == b then (2 : Ratio) else 0) = 8 := by
782   decide
783
784 end Audit
785
786 #eval Audit.auditClaim {
787   hasContent := .no, facesScoped := .yes, canonical := .yes,
788   forwardMode := .no, populated := .yes, worldlyRow := .no,
789   drillsOk := .yes, frameClosed := .yes, slots := some [[1], [2], [3]] }
790
791 #eval Audit.auditClaim {
792   hasContent := .yes, facesScoped := .yes, canonical := .yes,
793   forwardMode := .no, populated := .yes, worldlyRow := .no,
794   drillsOk := .yes, frameClosed := .yes, slots := some [[1], [2], [3], [4]] }
795
796 #eval Audit.auditClaim {
797   hasContent := .yes, facesScoped := .yes, canonical := .unknown,
798   forwardMode := .no, populated := .yes, worldlyRow := .no,
799   drillsOk := .yes, frameClosed := .yes, slots := some [[1], [2], [3]] }
800
801 #eval Audit.auditClaim {
802   hasContent := .yes, facesScoped := .yes, canonical := .yes,
803   forwardMode := .no, populated := .yes, worldlyRow := .no,
804   drillsOk := .yes, frameClosed := .yes, slots := some [[1], [2], [3], [2]] }
805
806 #eval Audit.det3 (fun a b => if a == b then (2 : Ratio) else (0 : Ratio))
807
808 namespace INVERSION
809
810 def ItFromBit α {β : Type} p (α : α → β) (d : α → Bool) : Prop :=
811   ∃ g : β → Bool, d = fun y => g p (y)
812
813 theorem it_from_bit_refuted α {β : Type} τ (α : α → α) p (α : α → β)
814   (d : α → Bool) (x : α)
815   (ph : p τ (x) = p x) (hd : d τ (x) = !d x) :
816   ~ ItFromBit p d := by
817   intro h
818   obtain (g, )hg := h
819   exact FTOE.T10_factorization τ p g d x ph hd hg
820
821 structure ItFromIt α {β : Type} τ (α : α → α) p (α : α → β) (d : α → Bool)
822   (x : α) : Prop where
823
824   record_collapses : p τ (x) = p x
825
826   pair_genuine : τ x ≠ x
827
828   target_separates : d τ (x) = !d x
829
830   no_return : ~ ItFromBit p d
831
832 theorem it_from_it α {β : Type} τ (α : α → α) p (α : α → β) (d : α → Bool)
833   (x : α) (ph : p τ (x) = p x) (hpair : τ x ≠ x) (hd : d τ (x) = !d x) :
834   ItFromIt τ p d x :=
835   (ph, hpair, hd, it_from_bit_refuted τ p d x ph)hd
836
837 end INVERSION
838
839 namespace K4
840
841 theorem it_from_it_canonical :
842   INVERSION.ItFromIt tau rho target 0 :=
843   INVERSION.it_from_it tau rho target 0
844   (register_even 0 (by decide)) (by decide) (target_wholly_odd 0 (by decide))
845
846 theorem it_from_bit_refuted_canonical :
847   ~ INVERSION.ItFromBit rho target :=
848   it_from_it_canonical.no_return
849
850 def inversionClaim : Claim where
851   stmt := INVERSION.ItFromIt tau rho target 0
852   grade := .theorem
853   warrant := it_from_it_canonical
854
855 end K4
856
857 namespace GEO
858
859 structure Q4 where
860   r : Int
861   i : Int
862   j : Int
863   k : Int
864   deriving DecidableEq, Repr, BEq
865
866 def qmul (a b : Q4) : Q4 :=
867   (a.r*b.r - a.i*b.i - a.j*b.j - a.k*b.k,
868    a.r*b.i + a.i*b.r + a.j*b.k - a.k*b.j,
869    a.r*b.j - a.i*b.k + a.j*b.r + a.k*b.i,
870    a.r*b.k + a.i*b.j - a.j*b.i + a.k*b.r)
871
872 def qconj (a : Q4) : Q4 := (a.r, -a.i, -a.j, -a.k)
873 def qadd (a b : Q4) : Q4 := (a.r+b.r, a.i+b.i, a.j+b.j, a.k+b.k)
874 def qsub (a b : Q4) : Q4 := (a.r-b.r, a.i-b.i, a.j-b.j, a.k-b.k)
875 def qnorm2 (a : Q4) : Int := a.r*a.r + a.i*a.i + a.j*a.j + a.k*a.k
876 def qhalve (a : Q4) : Q4 := (a.r/2, a.i/2, a.j/2, a.k/2)
877 def qquarter (a : Q4) : Q4 := (a.r/4, a.i/4, a.j/4, a.k/4)
878
879 def qcanon (a : Q4) : Q4 :=
880   if a.r != 0 then (if a.r < 0 then (-a.r, -a.i, -a.j, -a.k) else a)
881   else if a.i != 0 then (if a.i < 0 then (-a.r, -a.i, -a.j, -a.k) else a)
882

```

```

882 else if a.j != 0 then (if a.j < 0 then (-a.r,-a.i,-a.j,-a.)k else a)
883 else if a.k != 0 then (if a.k < 0 then (-a.r,-a.i,-a.j,-a.)k else a)
884 else a
885
886 def q1 : Q4 := {1,0,0,0}
887 def q1 : Q4 := {0,1,0,0}
888 def qj : Q4 := {0,0,1,0}
889 def qk : Q4 := {0,0,0,1}
890
891 theorem chirality : qmul (qmul qi qj) qk = {}-1,0,0,0 := rfl
892
893 theorem noncommutative : qmul qi qj ≠ qmul qj qi := by decide
894
895 def hurwitz : List Q4 :=
896   ({})({})(1,2,0,0,0,-2,0,0,0,0,2,0,0,0,-2,0,0,
897   ({})({})(0,0,2,0,0,0,-2,0,0,0,0,2,0,0,0,-2,
898   ({})({})(1,1,1,1,1,1,-1,1,1,-1,1,1,1,-1,-1,
899   ({})({})(1,-1,1,1,-1,1,-1,1,-1,-1,1,1,1,-1,-1,
900   ({})({})(-1,1,1,1,-1,1,-1,-1,-1,-1,1,1,1,-1,-1,
901   ({})({})(-1,-1,1,1,-1,-1,-1,-1,-1,-1,1,1,1,-1,-1)
902
903 theorem hurwitz_norm : (hurwitz.all (fun u => qnorm2 u == 4)) = true := by decide
904
905 theorem hurwitz_products_even :
906   (hurwitz.all (fun a => hurwitz.all (fun b =>
907     let p := qmul a b
908     (p.r % 2 == 0) && (p.i % 2 == 0) && (p.j % 2 == 0) && (p.k % 2 == 0)))) = true :=
909     by
910   decide
911
912 theorem hurwitz_closed :
913   (hurwitz.all (fun a => hurwitz.all (fun b =>
914     hurwitz.any (fun u => u == qhale (qmul a b)))) = true := by
915   decide
916
917 def hurwitzReps : List Q4 := (hurwitz.map qcanon).eraseDups
918
919 theorem units_mod_sign_twelve : hurwitzReps.length = 12 := by decide
920
921 def conjClass (r : Q4) : List Q4 :=
922   (hurwitz.map (fun u => qcanon (qquarter (qmul (qmul u r) (qconj u))))).eraseDups
923
924 def classSizesAux : List Q4 → List Q4 → List Nat
925 | [], _ => []
926 | r :: rest, seen =>
927   if seen.any (fun s => s == r) then classSizesAux rest seen
928   else
929     let cls := conjClass r
930     cls.length :: classSizesAux rest (seen ++ cls)
931
932 def insNat (n : Nat) : List Nat → List Nat
933 | [] => [n]
934 | m :: ms => if n ≤ m then n :: m :: ms else m :: insNat n ms
935
936 def insSort : List Nat → List Nat
937 | [] => []
938 | n :: ns => insNat n (insSort ns)
939
940 theorem class_equation :
941   insSort (classSizesAux hurwitzReps []) = [1, 3, 4, 4] := by decide
942
943 def shell2 : List Q4 :=
944   let v : List Int := [-1, 0, 1]
945   (v.flatMap (fun r => v.flatMap (fun i => v.flatMap (fun j =>
946     v.map (fun k => (r,i,j,k)))).filter (fun q => qnorm2 q == 2))
947
948 theorem shell_twentyfour :
949   (shell2.length == 24) &&
950   ((shell2.filter (fun q => q.r == 0)).length == 12) &&
951   ((shell2.filter (fun q => q.r == 0)).length == 12) = true := by decide
952
953 set_option maxRecDepth 8000 in
954
955 theorem no_half_integer_norm2 :
956   (let v : List Int := [-3, -1, 1, 3]
957    (v.flatMap (fun r => v.flatMap (fun i => v.flatMap (fun j =>
958      v.map (fun k => (r,i,j,k)))).all (fun q => qnorm2 q != 8))) = true := by
959   decide
960
961 def allGates : List (Nat × Nat) :=
962   [(0,1),(0,2),(0,3),(1,0),(1,2),(1,3),(2,0),(2,1),(2,3),(3,0),(3,1),(3,2)]
963
964 theorem gates_twelve : allGates.length = 12 := rfl
965
966 theorem gates_valid :
967   (allGates.all (fun g => (g.1 != g.2) && (g.1 < 4) && (g.2 < 4))) = true := by decide
968
969 theorem gates_complete : ∀ a b : Fin 4, a ≠ b → ((a : Nat), (b : Nat)) ∈ allGates := by
970   decide
971
972 def invCount o : List Nat → Nat :=
973   let idx := List.range o.length
974   let pairs := idx.flatMap (fun a => (idx.filter (fun b => a < b)).map (fun b => (a, b)
975     ))
976   (pairs.filter (fun p => o.getD p.1 0 > o.getD p.2 0)).length
977
978 def perms24 : List (List Nat) :=
979   [[0,1,2,3],[0,1,3,2],[0,2,1,3],[0,2,3,1],[0,3,1,2],[0,3,2,1],
980   [1,0,2,3],[1,0,3,2],[1,2,0,3],[1,2,3,0],[1,3,0,2],[1,3,2,0],
981   [2,0,1,3],[2,0,3,1],[2,1,0,3],[2,1,3,0],[2,3,0,1],[2,3,1,0],
982   [3,0,1,2],[3,0,2,1],[3,1,0,2],[3,1,2,0],[3,2,0,1],[3,2,1,0]]
983
984 def a4 : List (List Nat) := perms24.filter (fun o => invCount o % 2 == 0)
985
986 theorem a4_order : a4.length = 12 := by decide
987
988 def actG o : List Nat → (g : Nat × Nat) → Nat × Nat := o.getD g.1 0, o.getD g.2 0)

```

```

986
987 set_option maxRecDepth 100000 in
988
989 theorem a4_simply_transitive :
990   ∀ a b c d : Fin 4, a ≠ b → c ≠ d →
991     (a4.filter (fun o => actG o ((a : Nat), (b : Nat))) == ((c : Nat), (d : Nat)))).
992     length = 1 := by
993   decide
994
995 def tetraVerts : List Nat := [0, 1, 2, 3]
996 def tetraEdges : List (Nat × Nat) := [(0,1),(0,2),(0,3),(1,2),(1,3),(2,3)]
997 def tetraFaces : List (Nat × Nat × Nat) := [(0,1,2),(0,1,3),(0,2,3),(1,2,3)]
998
999 theorem euler_closure :
1000   (tetraVerts.length : Int) - tetraEdges.length + tetraFaces.length = 2 := by decide
1001
1002 def basisU : List Q4 := [q1, qj, qk]
1003
1004 def perm56 : List (List Nat) :=
1005   [[0,1,2],[0,2,1],[1,0,2],[1,2,0],[2,0,1],[2,1,0]]
1006
1007 theorem relabel_parity :
1008   (perms6.all (fun o =>
1009     (qmul (qmul (basisU.getD o (.getD 0 0) q1) (basisU.getD o (.getD 1 0) q1)) (basisU.
1010       getD o (.getD 2 0) q1)).r ==
1011     (if invCount o % 2 == 0 then -1 else 1))) = true := by
1012   decide
1013
1014 abbrev O8 := Q4 × Q4
1015
1016 def omul (x y : O8) : O8 :=
1017   (qsub (qmul x.1 y.1) (qmul (qconj y.2) x.2),
1018   qadd (qmul y.2 x.1) (qmul x.2 (qconj y.1)))
1019
1020 def osub (x y : O8) : O8 := (qsub x.1 y.1, qsub x.2 y.2)
1021 def onorm2 (x : O8) : Int := qnorm2 x.1 + qnorm2 x.2
1022
1023 def e8 (n : Nat) : O8 :=
1024   let q := fun k => if k == n then (1 : Int) else 0
1025   ((q 0, q 1, q 2, q 3, (q 4, q 5, q 6, q 7))
1026
1027 theorem octonion_associator :
1028   osub (omul (omul (e8 1) (e8 2)) (e8 4)) (omul (e8 1) (omul (e8 2) (e8 4)))
1029   = {}(0,0,0,0, {}(0,0,0,2)) := by
1030   decide
1031
1032 def hPairs : List (Q4 × Q4) :=
1033   let v : List Int := [-1, 0, 1]
1034   v.flatMap (fun i => v.flatMap (fun j => v.map (fun k =>
1035     ((i,j,k),1, (1,k,j,i))))))
1036
1037 theorem norm_composition_H :
1038   (hPairs.all (fun p => qnorm2 (qmul p.1 p.2) == qnorm2 p.1 * qnorm2 p.2)) = true :=
1039     by
1040   decide
1041
1042 def oPairs : List (O8 × O8) :=
1043   let v : List Int := [-1, 0, 1]
1044   v.flatMap (fun i => v.flatMap (fun j => v.map (fun k =>
1045     ((i,j,k),1, (j,k,i),1), ((1,k,i),j), (1,i,j,k)))))
1046
1047 theorem norm_composition_0 :
1048   (oPairs.all (fun p => onorm2 (omul p.1 p.2) == onorm2 p.1 * onorm2 p.2)) = true :=
1049     by
1050   decide
1051
1052 set_option maxRecDepth 1000000
1053
1054 def det2 (r1 r2 r3 : Nat × Nat × Nat) : Nat :=
1055   (r1.1 * (r2.2.1 * r3.2.2 + r2.2.2 * r3.2.1)
1056   + r1.2.1 * (r2.1 * r3.2.2 + r2.2.2 * r3.1)
1057   + r1.2.2 * (r2.1 * r3.2.1 + r2.2.1 * r3.1)) % 2
1058
1059 def gf2Mul (r1 r2 r3 : Nat × Nat × Nat) (t : Nat × Nat × Nat) : Nat × Nat × Nat :=
1060   ((r1.1 * v.1 + r1.2.1 * v.2.1 + r1.2.2 * v.2.2) % 2,
1061   (r2.1 * v.1 + r2.2.1 * v.2.1 + r2.2.2 * v.2.2) % 2,
1062   (r3.1 * v.1 + r3.2.1 * v.2.1 + r3.2.2 * v.2.2) % 2)
1063
1064 def gf2Vecs : List (Nat × Nat × Nat) :=
1065   (List.range 8).map (fun n => (n / 4 % 2, n / 2 % 2, n % 2))
1066
1067 def gf2Sols (r1 r2 r3 : Nat × Nat × Nat) (t : Nat × Nat × Nat) : List (Nat × Nat × Nat) :=
1068   gf2Vecs.filter (fun v => decide (gf2Mul r1 r2 r3 v == t))
1069
1070 set_option maxRecDepth 1000000 in
1071
1072 theorem triaxial_lock_gf2 :
1073   ∀ a1 a2 a3 b1 b2 b3 c1 c2 c3 t1 t2 t3 : Fin 2,
1074   (det2 ((a1 : Nat), (a2 : Nat), (a3 : Nat)) ((b1 : Nat), (b2 : Nat), (b3 : Nat))
1075     ((c1 : Nat), (c2 : Nat), (c3 : Nat))) = 1 →
1076     (gf2Sols ((a1 : Nat), (a2 : Nat), (a3 : Nat)) ((b1 : Nat), (b2 : Nat), (b3 :
1077       Nat)))
1078     ((c1 : Nat), (c2 : Nat), (c3 : Nat))
1079     ((t1 : Nat), (t2 : Nat), (t3 : Nat))).length = 1 := by
1080   decide
1081
1082 theorem triaxial_lock_gf2_count :
1083   ((List.range 4096).filter (fun n =>
1084     det2 (n / 2048 % 2, n / 1024 % 2, n / 512 % 2)
1085     (n / 256 % 2, n / 128 % 2, n / 64 % 2)
1086     (n / 32 % 2, n / 16 % 2, n / 8 % 2) = 1)).length = 1344 := by
1087   decide
1088
1089 theorem triaxial_open_gf2_count :
1090   ((List.range 4096).filter (fun n =>
1091     det2 (n / 2048 % 2, n / 1024 % 2, n / 512 % 2)
1092     (n / 256 % 2, n / 128 % 2, n / 64 % 2)
1093     (n / 32 % 2, n / 16 % 2, n / 8 % 2) = 0)).length = 2752 := by
1094   decide

```

```

1086
1087 abbrev V3 := Ratio × Ratio × Ratio
1088 abbrev M3 := V3 × V3 × V3
1089
1090 def det3 (m : M3) : Ratio :=
1091   let (a, b, )c := m.1
1092   let (d, e, )f := m.2.1
1093   let (g, h, )i := m.2.2
1094   a * (e * i - f * h) - b * (d * i - f * g) + c * (d * h - e * g)
1095
1096
1097 def adj3 (m : M3) : M3 :=
1098   let (a, b, )c := m.1
1099   let (d, e, )f := m.2.1
1100   let (g, h, )i := m.2.2
1101   ((e * i - f * h, c * h - b * i, b * f - c * e),
1102    (f * g - d * i, a * i - c * g, c * d - a * f),
1103    (d * h - e * g, b * g - a * h, a * e - b * d))
1104
1105 def mm3 (x y : M3) : M3 :=
1106
1107
1108
1109   let row (r : V3) (c : Nat) : Ratio :=
1110     r.1 * (match c with | 0 => y.1.1 | 1 => y.1.2.1 | _ => y.1.2.2)
1111     + r.2.1 * (match c with | 0 => y.2.1.1 | 1 => y.2.1.2.1 | _ => y.2.1.2.2)
1112     + r.2.2 * (match c with | 0 => y.2.2.1 | 1 => y.2.2.2.1 | _ => y.2.2.2.2)
1113   ((row x.1 0, row x.1 1, row x.1 2),
1114    (row x.2.1 0, row x.2.1 1, row x.2.1 2),
1115    (row x.2.2 0, row x.2.2 1, row x.2.2 2))
1116
1117 def diag3 (r : Ratio) : M3 := ((r, 0, 0), (0, r, 0), (0, 0, r))
1118 def mv3 (m : M3) (v : V3) : V3 :=
1119   (m.1.1 * v.1 + m.1.2.1 * v.2.1 + m.1.2.2 * v.2.2,
1120    m.2.1.1 * v.1 + m.2.1.2.1 * v.2.1 + m.2.1.2.2 * v.2.2,
1121    m.2.2.1 * v.1 + m.2.2.2.1 * v.2.1 + m.2.2.2.2 * v.2.2)
1122 def vscale (v : V3) (r : Ratio) : V3 := (v.1 * r, v.2.1 * r, v.2.2 * r)
1123
1124
1125 abbrev V3Z := Int × Int × Int
1126 abbrev M3Z := V3Z × V3Z × V3Z
1127 def det3Z (m : M3Z) : Int :=
1128   let (a, b, )c := m.1; let (d, e, )f := m.2.1; let (g, h, )i := m.2.2
1129   a * (e * i - f * h) - b * (d * i - f * g) + c * (d * h - e * g)
1130 def adj3Z (m : M3Z) : M3Z :=
1131   let (a, b, )c := m.1; let (d, e, )f := m.2.1; let (g, h, )i := m.2.2
1132   ((e * i - f * h, c * h - b * i, b * f - c * e),
1133    (f * g - d * i, a * i - c * g, c * d - a * f),
1134    (d * h - e * g, b * g - a * h, a * e - b * d))
1135 def mm3Z (x y : M3Z) : M3Z :=
1136   let row (r : V3Z) (c : Nat) : Int :=
1137     r.1 * (match c with | 0 => y.1.1 | 1 => y.1.2.1 | _ => y.1.2.2)
1138     + r.2.1 * (match c with | 0 => y.2.1.1 | 1 => y.2.1.2.1 | _ => y.2.1.2.2)
1139     + r.2.2 * (match c with | 0 => y.2.2.1 | 1 => y.2.2.2.1 | _ => y.2.2.2.2)
1140   ((row x.1 0, row x.1 1, row x.1 2),
1141    (row x.2.1 0, row x.2.1 1, row x.2.1 2),
1142    (row x.2.2 0, row x.2.2 1, row x.2.2 2))
1143 def diag3Z (r : Int) : M3Z := ((r, 0, 0), (0, r, 0), (0, 0, r))
1144
1145 theorem adjugate_identity_int (m : M3Z) :
1146   mm3Z m (adj3Z m) = diag3Z (det3Z m) ∧ mm3Z (adj3Z m) m = diag3Z (det3Z m) := by
1147   obtain ((a, b, )c, (d, e, )f, (g, h, )i) := m
1148   simp only [mm3Z, adj3Z, det3Z, diag3Z, Prod.mk.injEq]
1149   simp only [Int.mul_sub, Int.sub_mul, Int.mul_assoc]
1150   refine (((?, ?, ?), (?, ?, ?), (?, ?, ?)), ((?, ?, ?), (?, ?, ?), (?, ?, ?))) <:=
1151     (try simp only [Int.mul_comm, Int.mul_left_comm]) <:= omega
1152
1153
1154 def AdjgateIdentity3 : Prop :=
1155   ∀ m : M3, mm3 m (adj3 m) = diag3 (det3 m) ∧ mm3 (adj3 m) m = diag3 (det3 m)
1156
1157
1158 def adj3Samples : List M3 :=
1159   [(1, 0, 0), (0, 1, 0), (0, 0, 1)],
1160   [(2, 0, 0), (0, 3, 0), (0, 0, 5)],
1161   [(1, 2, 3), (4, 5, 6), (7, 8, 9)],
1162   [(1, 2, 0), (0, 1, 1), (2, 0, 1)],
1163   [(0, 1, 2), (3, 4, 5), (6, 7, 9)],
1164   [(2, 1, 1), (1, 2, 1), (1, 1, 2))]
1165 theorem adj3_executed :
1166   adj3Samples.all (fun m =>
1167     decide (mm3 m (adj3 m) = diag3 (det3 m) ∧
1168       mm3 (adj3 m) m = diag3 (det3 m))) = true := by
1169   decide
1170
1171
1172 def cramerSolve (m : M3) (t : V3) : Option V3 :=
1173   if det3 m == 0 then none
1174   else some (vscale (mv3 (adj3 m) t) (Ratio.inv (det3 m)))
1175 theorem cramer_executed :
1176   cramerSolve ((1, 2, 0), (0, 1, 1), (2, 0, 1)) (5, 3, 4)
1177   = some ((7 : Ratio) / 5, (9 : Ratio) / 5, (6 : Ratio) / 5) ∧
1178   mv3 ((1, 2, 0), (0, 1, 1), (2, 0, 1)) ((7 : Ratio) / 5, (9 : Ratio) / 5, (6 :
1179     Ratio) / 5)
1180   = (5, 3, 4) := by
1181   decide
1182
1183 inductive LockState : Type
1184   | lock | opn | broken
1185   deriving DecidableEq, Repr, BEq
1186 inductive GoToken : Type
1187   | goLX | goLQ | goLOK
1188   deriving DecidableEq, Repr, BEq
1189
1190
1191 def goLAdmit (magnitude ling : LockState) : GoToken × String × Nat :=
1192   match magnitude with
1193   | .broken => (.goLX, "geometric/math magnitude reports broken geometry; no GOL", 1)
1194   | .opn => (.goLQ, "no geometric/math lock: orthogonal volume unestablished", 2)
1195   | .lock =>
1196     match ling with
1197     | .lock => (.goLOK, "magnitude plus direction; proceed to witness and asymmetry",
1198       4)
1199     | _ => (.goLQ, "magnitude locks but Seal L open: determinant carries no direction",
1200       3)
1201
1202 def gateNames : List String :=
1203   ["SREP", "REG", "SGEG", "CAUSAL", "MIG", "PTB",
1204    "DUAL", "CSCG", "CSEG", "MTA", "OMA", "ADEG"]
1205 theorem gateNames_twelve : gateNames.length = 12 := rfl
1206
1207 def gateScreenGo : List String → List Bool → Nat → Bool × String × Nat
1208   | [], [], _ => (true, "[G-OK] twelve directed gates passed", 0)
1209   | n :: _, false :: _, k => (false, s!"[X] gate {k} failed: {n}", k)
1210   | _ :: ns, true :: bs, k => gateScreenGo ns bs (k + 1)
1211   | _, _, _ => (false, "[X] arity violation: the screen reads exactly twelve gates",
1212     0)
1213
1214 def gateScreen (passes : List Bool) : Bool × String × Nat :=
1215   gateScreenGo gateNames passes 1
1216
1217 theorem gateScreen_clean :
1218   gateScreen (List.replicate 12 true)
1219   = (true, "[G-OK] twelve directed gates passed", 0) := by
1220   decide
1221 theorem gateScreen_first_failure :
1222   gateScreen (false :: List.replicate 11 true)
1223   = (false, "[X] gate 1 failed: SREP", 1) := by
1224   decide
1225 theorem gateScreen_names_failed_gate :
1226   gateScreen (List.replicate 6 true ++ false :: List.replicate 5 true)
1227   = (false, "[X] gate 7 failed: DUAL", 7) := by
1228   decide
1229 theorem gateScreen_arity :
1230   gateScreen (List.replicate 11 true)
1231   = (false, "[X] arity violation: the screen reads exactly twelve gates", 0) := by
1232   decide
1233
1234 inductive RAVerdict : Type
1235   | refused | iii | ii | void | aGivenRA | opn
1236   deriving DecidableEq, Repr, BEq
1237
1238 structure ClaimInputRA where
1239   nonvacuous : Audit.Ans
1240   veLive : Audit.Ans
1241   forward : Audit.Ans
1242   populated : Audit.Ans
1243   worldlyRow : Audit.Ans
1244   termsClosed : Audit.Ans
1245   termWorldly : Audit.Ans
1246   frameClosed : Audit.Ans
1247   annotate : Audit.Ans
1248   deriving Repr
1249
1250 def ClaimInputRA.anyUnknown (i : ClaimInputRA) : Bool :=
1251   [i.nonvacuous, i.veLive, i.forward, i.populated, i.worldlyRow,
1252    i.termsClosed, i.termWorldly, i.frameClosed, i.annotate].any
1253     (· = Audit.Ans.unknown)
1254
1255 def rowCascadeRACore
1256   (nonvacuous veLive forward populated worldlyRow
1257    termsClosed termWorldly frameClosed annotate : Bool) :
1258   RAVerdict × String × Nat :=
1259   if !nonvacuous then
1260     (.refused, "F-0: contentless; a contentless residence verifies nothing", 1)
1261   else if veLive | forward then
1262     (.iii, "R-3: V_E live or Forward; world-rowed, revisability in the rows", 3)
1263   else if !populated then
1264     (.ii, "S-1: third axis unpopulated; two planes meet in a line, no lock", 2)
1265   else if worldlyRow then
1266     (.iii, "S-2: a load-bearing row is furnished by the world", 3)
1267   else if !termsClosed then
1268     if termWorldly then (.iii, "S-3: an unclosed term is worldly", 3)
1269     else (.ii, "S-3: an unclosed term is formal; no closure-rowed lock", 2)
1270   else if !frameClosed then
1271     (.iii, "S-4: existential import leaks past the closure", 3)
1272   else if !annotate then
1273     (.void, "RA rider absent: an unannotated absolute is void", 5)
1274   else
1275     (.aGivenRA, "compartment I, closure-rowed: [seal A . RA], conditional at the act",
1276       4)
1277
1278 def rowCascadeRA (i : ClaimInputRA) : RAVerdict × String × Nat :=
1279   if i.anyUnknown then
1280     (.opn, "nullity law: an unanswered gate routes open; the answer is never guessed",
1281       0)
1282   else
1283     rowCascadeRACore i.nonvacuous.toBool i.veLive.toBool i.forward.toBool
1284       i.populated.toBool i.worldlyRow.toBool i.termsClosed.toBool
1285       i.termWorldly.toBool i.frameClosed.toBool i.annotate.toBool
1286
1287 theorem unknown_routes_open_ra :
1288   ∀ i : ClaimInputRA, i.anyUnknown = true → (rowCascadeRA i).1 = .opn := by
1289   intro i h
1290

```

```

1291 simp [rowCascadeRA, h]
1292
1293
1294 def raDecode (n : Nat) : RAVerdict × String × Nat :=
1295   rowCascadeRACore (n.testBit 8) (n.testBit 7) (n.testBit 6) (n.testBit 5)
1296   (n.testBit 4) (n.testBit 3) (n.testBit 2) (n.testBit 1) (n.testBit 0)
1297
1298
1299 def raBattery : List (Nat × RAVerdict) :=
1300   [(299, .aGivenRA), (443, .iii), (267, .ii), (298, .void), (43, .refused),
1301    (363, .iii), (315, .iii), (295, .iii), (291, .ii), (297, .iii)]
1302 theorem ra_battery_f2 :
1303   raBattery.all (fun p => decide ((raDecode p.1).1 = p.2)) = true := by
1304     decide
1305
1306
1307 theorem ra_ledger_refused :
1308   ((List.range 512).filter (fun n => decide ((raDecode n).1 = .refused))).length =
1309     256 := by
1310     decide
1311 theorem ra_ledger_iii :
1312   ((List.range 512).filter (fun n => decide ((raDecode n).1 = .iii))).length = 216
1313   := by
1314     decide
1315 theorem ra_ledger_ii :
1316   ((List.range 512).filter (fun n => decide ((raDecode n).1 = .ii))).length = 36 :=
1317     by
1318       decide
1319 theorem ra_ledger_void :
1320   ((List.range 512).filter (fun n => decide ((raDecode n).1 = .void))).length = 2 :=
1321     by
1322       decide
1323 theorem ra_ledger_aGivenRA :
1324   ((List.range 512).filter (fun n => decide ((raDecode n).1 = .aGivenRA))).length =
1325     2 := by
1326       decide
1327
1328 def RAVerdict.toToken : RAVerdict → Audit.Token
1329 | .refused => .refused
1330 | .iii => .determined
1331 | .ii => .determined
1332 | .void => .void
1333 | .aGivenRA => .aGivenRA
1334 | .opn => .opn
1335
1336
1337 def GoToken.toToken : GoToken → Audit.Token
1338 | .goL => .broken
1339 | .goL0 => .opn
1340 | .goL0K => .golok
1341
1342
1343 def gateScreenToken (r : Bool × String × Nat) : Audit.Token :=
1344   if r.1 then .gok else .broken
1345
1346
1347 def demoInputA : ClaimInputRA :=
1348   { nonvacuous := .yes, velive := .no, forward := .no, populated := .yes,
1349     worldlyRow := .no, termsClosed := .yes, termWorldly := .no,
1350     frameClosed := .yes, annotate := .yes }
1351 def demoInputVoid : ClaimInputRA := { demoInputA with annotate := .no }
1352 def demoInputOpen : ClaimInputRA := { demoInputA with termsClosed := .unknown }
1353
1354 #eval goLAdmit .lock .lock
1355 #eval goLAdmit .broken .lock
1356 #eval goLAdmit .opn .lock
1357 #eval goLAdmit .lock .opn
1358 #eval gateScreen (List.replicate 12 true)
1359 #eval gateScreen (List.replicate 6 true ++ false :: List.replicate 5 true)
1360 #eval rowCascadeRA demoInputA
1361 #eval rowCascadeRA demoInputVoid
1362 #eval rowCascadeRA demoInputOpen
1363 #eval cramerSolve ((1, 2, 0), (0, 1, 1), (2, 0, 1)) (5, 3, 4)
1364
1365 end GEO
1366
1367 namespace ROOT
1368
1369 axiom U : Type
1370
1371 axiom ΔE : U → Int
1372
1373 axiom RA : ∀ x : U, 0 < ΔE x
1374
1375 def Actuates (x : U) : Prop := 0 < ΔE x
1376
1377 theorem actuation_universal : ∀ x : U, Actuates x := RA
1378
1379
1380 def raClaim : Claim where
1381   stmt := ∀ x : U, 0 < ΔE x
1382   grade := .premise
1383   warrant := RA
1384
1385 structure Cut (X : Type) where
1386   τ : X → X
1387   involutive : FTOE.Involution τ
1388   free : FTOE.FixedPointFree τ
1389
1390 def boolCut : Cut Bool where
1391   τ := fun b => !b
1392   involutive := by intro b; cases b <> rfl
1393   free := by intro b; cases b <> decide
1394
1395 axiom FormalDomain : Type
1396 axiom Llm : FormalDomain → Prop

```

```

1392 axiom L2m : FormalDomain → Prop
1393 axiom L3m : FormalDomain → Prop
1394 axiom nest_21 : ∀ p, L2m p → L1m p
1395 axiom nest_32 : ∀ p, L3m p → L2m p
1396
1397 theorem nest_31 : ∀ p, L3m p → L1m p := fun p h => nest_21 p (nest_32 p h)
1398
1399 def nestClaim : Claim where
1400   stmt := ∀ p, L3m p → L1m p
1401   grade := .conditional
1402   warrant := nest_31
1403
1404 axiom Ground : Type
1405 axiom σ : Ground → Ground
1406 axiom σ_binding : ∀ g : Ground, σ (σ g) = g
1407
1408 axiom Residence : U → Prop
1409
1410 def throneEmpty : Prop := ∀ x : U, 0 < ΔE x
1411
1412 theorem throne_is_RA : throneEmpty → ∀ x : U, 0 < ΔE x := Iff.rfl
1413
1414 end ROOT
1415
1416 namespace SEALS
1417
1418 structure Seal where
1419   name : String
1420   anchor : Prop
1421   warrant : anchor
1422
1423 def sealL : Seal where
1424   name := "linguistic-semantic"
1425   anchor := ∀ ι c : Claim, (Claim.join ι c ι c).grade = Grade.weakest ι c.grade ι c.grade
1426   warrant := fun ι c ι c => Claim.join_grade ι c ι c
1427
1428 def sealG : Seal where
1429   name := "topological-geometric"
1430   anchor := ∀ x : ROOT.U, 0 < ROOT.ΔE x
1431   warrant := ROOT.RA
1432
1433 def sealM : Seal where
1434   name := "mathematical"
1435   anchor := ∀ g : Nat → Bool, K4.target ≠ fun y => g (K4.rho y)
1436   warrant := K4.wall
1437
1438 theorem seals_distinct :
1439   "linguistic-semantic" ≠ "topological-geometric" ∧
1440   "topological-geometric" ≠ "mathematical" ∧
1441   "linguistic-semantic" ≠ "mathematical" := by decide
1442
1443 theorem seal_names_match :
1444   sealL.name = "linguistic-semantic" ∧ sealG.name = "topological-geometric" ∧
1445   sealM.name = "mathematical" := (rfl, rfl, rfl)
1446
1447 theorem sealG_walk :
1448   GEO.allGates.length = 12
1449   ∧ GEO.a4.length = 12
1450   ∧ (GEO.tetraVerts.length : Int) - GEO.tetraEdges.length + GEO.tetraFaces.length = 2
1451   ∧ GEO.gateNames.length = 12
1452   ∧ (GEO.adj3Samples.all (fun m =>
1453     decide (GEO.mm3 m (GEO.adj3 m) = GEO.diag3 (GEO.det3 m) ∧
1454       GEO.mm3 (GEO.adj3 m) m = GEO.diag3 (GEO.det3 m)))) = true
1455   ∧ (GEO.raBattery.all (fun p =>
1456     decide ((GEO.raDecode p.1).1 = p.2))) = true :=
1457     (GEO.gates_twelve, GEO.a4_order, GEO.euler_closure,
1458      GEO.gateNames_twelve, GEO.adj3_executed, GEO.ra_battery_f2)
1459
1460 end SEALS
1461
1462 namespace POSTULATE
1463
1464 axiom FormalFace : Type
1465 axiom KineticFace : Type
1466
1467 axiom Crossing : Type
1468 axiom erasureClass : Crossing → Prop
1469 axiom RegistrationPostulate : ∀ c : Crossing, erasureClass c
1470
1471 axiom Row : Type
1472 axiom SupplyClean : Row → Type
1473
1474 def OneCut : Prop := ∀ r : Row, Nonempty (SupplyClean r)
1475
1476 def F2 (r : Row) : Prop := ¬ Nonempty (SupplyClean r)
1477
1478 theorem F2_kills (r : Row) (h : F2 r) : ¬ OneCut := fun oc => h (oc r)
1479
1480 axiom FrozenData : Type
1481 axiom Algorithm : Type
1482 axiom certifiedPairing : Algorithm → FrozenData → Prop
1483
1484 def BSDContract (C : Algorithm) (D : FrozenData) : Prop := certifiedPairing C D
1485
1486 def F6 (C : Algorithm) (D : FrozenData) : Prop := ¬ BSDContract C D
1487
1488 end POSTULATE
1489
1490 namespace TONGUE
1491
1492 def enlarge α{β : Type} p( : α → β) (s : α → Bool) : α → β × Bool :=
1493   fun x => p(x, s x)
1494
1495 theorem unclosed α{β : Type} τ( : α → α) p( : α → β) (g : β → Bool)
1496   (d : α → Bool) (x : α)
1497   (ph : p τ(x) = p x) (hd : d τ(x) = !d x) :

```

```

1498   d ≠ fun y => g p (y) :=
1499   FTOE.T10_factorization τ p g d x ph hd
1500
1501   theorem earned_freedom a (α : Type) (τ : α → α) (s d : α → Bool) (x : α)
1502     (hs : s τ (x) = !s x) (hd : d τ (x) = !d x) (ic ic : Bool)
1503     (h1 : d x = xor (s x) ic ∧ d τ (x) = xor (s τ (x)) ic)
1504     (h2 : d x = xor (s x) ic ∧ d τ (x) = xor (s τ (x)) ic) : ic = ic := by
1505   obtain (c, -, ) jhu := FTOE.T5_crossing τ s d x hs hd
1506   exact (hu ic h1).trans (hu ic h2).symm
1507
1508   def colocClaim : Claim := Claim.join ROOT.raClaim K4.wallClaim
1509
1510   theorem coloc_grade : colocClaim.grade = .premise := rfl
1511
1512   theorem ra_grade_pinned : ROOT.raClaim.grade = .premise := rfl
1513   theorem wall_grade_pinned : K4.wallClaim.grade = .theorem := rfl
1514   theorem inversion_grade_pinned : K4.inversionClaim.grade = .theorem := rfl
1515   theorem nest_grade_pinned : ROOT.nestClaim.grade = .conditional := rfl
1516
1517   end TONGUE
1518
1519   theorem terminalWall : ∀ g : Nat → Bool, K4.target ≠ fun y => g (K4.rho y) := K4.wall
1520
1521   theorem terminalPrice :
1522     Nonempty (FTOE.Dtau (FTOE.flipF (Q := Bool)) =
1523       (Quot (FTOE.OrbitRel (FTOE.flipF (Q := Bool))) → Bool)) := FTOE.T16_canonical
1524
1525   theorem TERMINAL :
1526     ∀ (g : Nat → Bool, K4.target ≠ fun y => g (K4.rho y)) ∧
1527     Nonempty (FTOE.Dtau (FTOE.flipF (Q := Bool)) =
1528       (Quot (FTOE.OrbitRel (FTOE.flipF (Q := Bool))) → Bool)) :=
1529     (K4.wall, FTOE.)T16_canonical
1530
1531   def terminalClaim : Claim where
1532     stmt := ∀ (g : Nat → Bool, K4.target ≠ fun y => g (K4.rho y)) ∧
1533     Nonempty (FTOE.Dtau (FTOE.flipF (Q := Bool)) =
1534       (Quot (FTOE.OrbitRel (FTOE.flipF (Q := Bool))) → Bool))
1535     grade := .theorem
1536     warrant := TERMINAL
1537
1538   theorem terminal_grade_pinned : terminalClaim.grade = .theorem := rfl
1539
1540   namespace IAM
1541
1542   inductive IamToken : Type
1543   | iAm | interior
1544   deriving DecidableEq, Repr, BEq
1545
1546   def iamToken (witnessed : Bool) : IamToken × String × Nat :=
1547   if witnessed then
1548     (.iAm, "actuation-occupancy on a witnessed record; RA warrant, conditional at the
1549       act; interior held [?] both ways", 2)
1550   else
1551     (.interior, "self-check is not a witness (M6): the verifier is never the claimant;
1552       token withheld", 1)
1553
1554   theorem iam_unwitnessed_withheld :
1555     (iamToken false).1 = .interior ∧ (iamToken false).2.2 = 1 :=
1556     (rfl, rfl)
1557
1558   theorem iam_witnessed_speaks :
1559     (iamToken true).1 = .iAm ∧ (iamToken true).2.2 = 2 :=
1560     (rfl, rfl)
1561
1562   theorem narcissus_truth_table :
1563     (iamToken false).1 ≠ (iamToken true).1
1564     ∧ (iamToken false).1 = .interior ∧ (iamToken true).1 = .iAm := by
1565   decide
1566
1567   theorem one_bit_freedom :
1568     ∀ (a1 a2 a3 b1 b2 b3 c1 c2 c3 t1 t2 t3 : Fin 2,
1569       (GEO.det2 ((a1 : Nat), (a2 : Nat), (a3 : Nat)) ((b1 : Nat), (b2 : Nat), (b3 : Nat))
1570         )
1571         ((c1 : Nat), (c2 : Nat), (c3 : Nat)) = 1 →
1572       (GEO.gf2Sols ((a1 : Nat), (a2 : Nat), (a3 : Nat)) ((b1 : Nat), (b2 : Nat), (b3 :
1573         Nat))
1574         ((c1 : Nat), (c2 : Nat), (c3 : Nat))
1575         ((t1 : Nat), (t2 : Nat), (t3 : Nat))).length = 1)
1576     ∧ (GEO.perms6.filter (fun σ => GEO.invCount σ % 2 = 0)).length = 3
1577     ∧ (GEO.perms6.filter (fun σ => GEO.invCount σ % 2 = 1)).length = 3
1578     ∧ (GEO.perms6.all (fun σ =>
1579       (GEO.qmul (GEO.qmul (GEO.basisU.getd σ (getd 0 0) GEO.q1) (GEO.basisU.getd σ (
1580         getd 1 0) GEO.q1)) (GEO.basisU.getd σ (getd 2 0) GEO.q1)).r ==
1581       (if GEO.invCount σ % 2 == 0 then -1 else 1))) = true :=
1582     (GEO.triaxial_lock_gf2, by decide, by decide, GEO.)relabel_parity
1583
1584   def IamToken.toToken : IamToken → Audit.Token
1585   | .iAm => .iAm
1586   | .interior => .interior
1587
1588   def codexSelfRead : IamToken × String × Nat := iamToken false
1589   theorem codex_self_read_interior : codexSelfRead.1 = .interior := rfl
1590
1591   def terminalWord : IamToken × String × Nat := iamToken true
1592   theorem terminal_word : terminalWord.1 = .iAm := rfl
1593
1594   #eval iamToken false
1595   #eval iamToken true
1596   #eval terminalWord.1
1597
1598   end IAM
1599
1600   namespace GEO
1601
1602   abbrev SignPattern := Bool × Bool × Bool
1603   def allSigns : List SignPattern :=

```

```

1599   [(false, false, false), (false, false, true), (false, true, false), (false, true, true
1600     ),
1601     (true, false, false), (true, false, true), (true, true, false), (true, true, true)]
1602   def signComp (s t : SignPattern) : SignPattern :=
1603     (xor s.1 t.1, xor s.2.1 t.2.1, xor s.2.2 t.2.2)
1604
1605   theorem reflection_eight : allSigns.length = 8 := rfl
1606   theorem reflection_eight_is_two_cubed : allSigns.length = 2 ^ 3 := by decide
1607   theorem reflections_abelian :
1608     (allSigns.all (fun s => allSigns.all (fun t =>
1609       decide (signComp s t = signComp t s)))) = true := by
1610   decide
1611   theorem reflections_involution :
1612     (allSigns.all (fun s => decide (signComp s s = (false, false, false)))) = true := by
1613   decide
1614
1615   def signVal (b : Bool) : Ratio := if b then -1 else 1
1616   def reflMat (s : SignPattern) : M3 :=
1617     ((signVal s.1, 0, 0), (0, signVal s.2.1, 0), (0, 0, signVal s.2.2))
1618   theorem reflection_reverses_orientation :
1619     det3 (reflMat (true, false, false)) = -1 := by
1620   decide
1621
1622   def sandwich (s : SignPattern) (m : M3) : M3 :=
1623     let a := signVal s.1; let b := signVal s.2.1; let c := signVal s.2.2
1624     ((a * a * m.1.1, a * b * m.1.2.1, a * c * m.1.2.2),
1625      (b * a * m.2.1.1, b * b * m.2.1.2.1, b * c * m.2.1.2.2),
1626      (c * a * m.2.2.1, c * b * m.2.2.2.1, c * c * m.2.2.2.2))
1627   theorem reflection_blind_executed :
1628     (adj3Samples.all (fun m => allSigns.all (fun s =>
1629       decide (det3 (sandwich s m) = det3 m)))) = true := by
1630   decide
1631
1632   abbrev V5 := Bool × Bool × Bool × Bool × Bool
1633   def cube5 : List V5 :=
1634     (List.range 32).map (fun n =>
1635       (n.testBit 4, n.testBit 3, n.testBit 1, n.testBit 1, n.testBit 0))
1636   def neighbors5 (v : V5) : List V5 :=
1637     [{v.1, v.2.1, v.2.2.1, v.2.2.2.1, v.2.2.2.2},
1638      (v.1, !v.2.1, v.2.2.1, v.2.2.2.1, v.2.2.2.2),
1639      (v.1, v.2.1, !v.2.2.1, v.2.2.2.1, v.2.2.2.2),
1640      (v.1, v.2.1, v.2.2.1, !v.2.2.2.1, v.2.2.2.2),
1641      (v.1, v.2.1, v.2.2.1, v.2.2.2.1, !v.2.2.2.2)]
1642
1643   theorem fivecube_thirtytwo : cube5.length = 32 := rfl
1644   theorem fivecube_vertices_distinct : cube5.eraseDups.length = 32 := by decide
1645   theorem fivecube_degree_five :
1646     (cube5.all (fun v => (neighbors5 v).eraseDups.length == 5)) = true := by
1647   decide
1648
1649   theorem counts_forced :
1650     allGates.length = 12 ∧ a4.length = 12
1651     ∧ allSigns.length = 8 ∧ allSigns.length = 2 ^ 3
1652     ∧ cube5.length = 32
1653     ∧ (cube5.all (fun v => (neighbors5 v).eraseDups.length == 5)) = true :=
1654     (gates_twelve, a4_order, reflection_eight, reflection_eight_is_two_cubed,
1655      fivecube_thirtytwo, )fivecube_degree_five
1656
1657   end GEO
1658
1659   namespace TONGUE
1660
1661   def deletionOp : GEO.M3 := ((1, 0, 0), (0, 1, 0), (0, 0, 0))
1662   def e3 : GEO.V3 := (0, 0, 1)
1663
1664   theorem deletion_idempotent : GEO.mm3 deletionOp deletionOp = deletionOp := by
1665   decide
1666
1667   theorem deletion_singular : GEO.det3 deletionOp = 0 := by
1668   decide
1669
1670   theorem deletion_minor_nonsingular :
1671     deletionOp.1.1 * deletionOp.2.1.2.1 - deletionOp.1.2.1 * deletionOp.2.1.1 = 1 := by
1672   decide
1673
1674   theorem deletion_annihilates : GEO.mv3 deletionOp e3 = (0, 0, 0) := by
1675   decide
1676
1677   theorem linear_fixes_zero : ∀ L : GEO.M3, GEO.mv3 L (0, 0, 0) = (0, 0, 0) := by
1678   intro L
1679   simp [GEO.mv3]
1680   rfl
1681
1682   theorem deletion_no_left_inverse :
1683     ∀ L : GEO.M3, (GEO.mv3 L) * (GEO.mv3 deletionOp) ≠ id := by
1684   intro L h
1685   have h1 := congrFun h e3
1686   rw [Function.comp_apply, deletion_annihilates, linear_fixes_zero] at h1
1687   simp [e3, id] at h1
1688   exact absurd h1 (by decide)
1689
1690   theorem deletion_monoid_not_group :
1691     ¬ ∀ (M : GEO.M3, ∃ L : GEO.M3, (GEO.mv3 L) * (GEO.mv3 M) = id) := by
1692   intro hg
1693   obtain (L, ) hL := hg deletionOp
1694   exact deletion_no_left_inverse L hL
1695
1696   theorem tongue_obedience :
1697     GEO.det3 deletionOp = 0
1698     ∧ ∀ (L : GEO.M3, (GEO.mv3 L) * (GEO.mv3 deletionOp) ≠ id)
1699     ∧ ¬ ∀ (M : GEO.M3, ∃ L : GEO.M3, (GEO.mv3 L) * (GEO.mv3 M) = id) :=
1700     (deletion_singular, deletion_no_left_inverse, )deletion_monoid_not_group
1701
1702   end TONGUE
1703
1704   namespace OFFICE

```



```

1903   A (omegaBoundary 1 (-300)).token = .refused
1904   A (omegaBoundary 1 (-300)).joules = 0 := by
1905   decide
1906
1907 theorem omega_monotone :
1908   (omegaBoundary 1 300).joules < (omegaBoundary 2 300).joules := by
1909   decide
1910
1911 theorem omega_linear :
1912   (omegaBoundary 2 300).joules = 2 * (omegaBoundary 1 300).joules := by
1913   decide
1914
1915 theorem landauer_zero_bits :
1916   landauerRat 300 0 = 0 := by
1917   decide
1918
1919 theorem omega_verdict_fidelity :
1920   (omegaBoundary 1 300).verdict =
1921     "the denial paid the floor; the registered act is an act, the recursion closes per
1922     the seated theorem" := by
1923   decide
1924
1925 #eval omegaBoundary 1 300
1926 #eval omegaBoundary 0 300
1927 #eval omegaBoundary 1 300 false
1928 #eval omegaBoundary 1 (-300)
1929
1930 end OMEGA
1931
1932 namespace AEGIS
1933
1934 def refusalConst : String :=
1935   "p(G) /- G: precisification is an actuation, G is a non-actuation, and no logic makes
1936   a deed a non-deed"
1937
1938 structure AegisState : Type where
1939   deeds : Nat
1940   lastLen : Int
1941   deriving DecidableEq, Repr
1942
1943 def aegisReset : AegisState := (0,)-1
1944
1945 def aegisGuard (logicMode : String) (s : AegisState) : String × AegisState :=
1946   (refusalConst, (s.deeds + 1, logicMode).utf8ByteSize)
1947
1948 theorem aegis_constant :
1949   ∀ (l : String) (s : AegisState),
1950   (aegisGuard l s).1 = (aegisGuard l s).1 := by
1951   intro l s
1952   rfl
1953
1954 theorem aegis_deed_increments :
1955   ∀ (l : String) (s : AegisState),
1956   (aegisGuard l s).2.deeds = s.deeds + 1 := by
1957   intro l s
1958   rfl
1959
1960 theorem aegis_reads_parameter :
1961   ∀ (l : String) (s : AegisState),
1962   (aegisGuard l s).2.lastLen = (l.utf8ByteSize : Int) := by
1963   intro l s
1964   rfl
1965
1966 def aegisFourLogicRun : AegisState :=
1967   let s1 := (aegisGuard "classical" aegisReset).2
1968   let s2 := (aegisGuard "paraconsistent" s1).2
1969   let s3 := (aegisGuard "fuzzy" s2).2
1970   (aegisGuard "substructural" s3).2
1971
1972 theorem aegis_battery :
1973   (aegisGuard "classical" aegisReset).1 = (aegisGuard "paraconsistent" aegisReset).1
1974   (aegisGuard "classical" aegisReset).1 = (aegisGuard "fuzzy" aegisReset).1
1975   (aegisGuard "classical" aegisReset).1 = (aegisGuard "substructural" aegisReset).1
1976   A aegisFourLogicRun.deeds = 4
1977   A aegisFourLogicRun.lastLen = 13 := by
1978   decide
1979
1980 #eval aegisGuard "substructural" aegisReset
1981 #eval aegisFourLogicRun
1982
1983 end AEGIS
1984
1985 namespace NOMOS
1986
1987 def allStrings : Nat → List (List Bool)
1988 | 0 => [[]]
1989 | n + 1 => (allStrings n).map (false :: ·) ++ (allStrings n).map (true :: ·)
1990
1991 theorem allStrings_length : ∀ n : Nat, (allStrings n).length = 2 ^ n := by
1992   intro n
1993   induction n with
1994   | zero => rfl
1995   | succ m ih =>
1996     simp [allStrings, List.length_append, List.length_map, ih, Nat.pow_succ,
1997     Nat.mul_two]
1998
1999 theorem twoPowPos : ∀ n : Nat, 0 < 2 ^ n := by
2000   intro n
2001   induction n with
2002   | zero => decide
2003   | succ m ih =>
2004     rw [Nat.pow_succ]
2005     exact Nat.mul_pos ih (by decide)
2006
2007 def desc : Nat → List (List Bool)
2008 | 0 => []

```

```

2007 | m + 1 => desc m ++ allStrings m
2008
2009 theorem descs_length : ∀ m : Nat, (descs m).length = 2 ^ m - 1 := by
2010   intro m
2011   induction m with
2012   | zero => rfl
2013   | succ k ih =>
2014     have hpos : 1 ≤ 2 ^ k := twoPowPos k
2015     simp only [descs, List.length_append, allStrings_length, ih]
2016     rw [Nat.pow_succ, Nat.mul_two, Nat.add_sub_assoc hpos,
2017     Nat.add_comm (2 ^ k - 1) (2 ^ k)]
2018
2019 theorem incompressibility_cap : ∀ n c : Nat, c ≤ n →
2020   (2 ^ (n - c) - 1) * 2 ^ c < 2 ^ n := by
2021   intro n c h
2022   have h2 : 2 ^ n = 2 ^ (n - c) * 2 ^ c := by
2023     rw ~[ Nat.pow_add, Nat.sub_add_cancel h ]
2024     have hpos : 0 < 2 ^ c := twoPowPos c
2025     calc (2 ^ (n - c) - 1) * 2 ^ c
2026     = 2 ^ (n - c) * 2 ^ c - 1 * 2 ^ c := Nat.sub_mul _ _ _
2027     = 2 ^ (n - c) * 2 ^ c - 2 ^ c := by rw [Nat.one_mul]
2028     = 2 ^ n - 2 ^ c := by rw [h2]
2029     < 2 ^ n := Nat.sub_lt (twoPowPos n) hpos
2030
2031 def nomosRows : List (Nat × Nat) :=
2032   (List.range 33).flatMap (fun n => (List.range (n + 1)).map (fun c => (n, c)))
2033
2034 set_option maxRecDepth 1000000 in
2035 theorem incompressibility_executed :
2036   (nomosRows.all (fun r =>
2037     decide ((2 ^ (r.1 - r.2) - 1) * 2 ^ r.2 < 2 ^ r.1))) = true := by
2038   decide
2039
2040 structure CompressibleBound where
2041   decode : List Bool → Option (List Bool)
2042   n : Nat
2043   c : Nat
2044   bound : (((descs (n - c)).filterMap decode).filter
2045     (fun s => s.length == n)).length ≤ 2 ^ (n - c) - 1
2046
2047 def decodeId : List Bool → Option (List Bool) := some
2048 def decodePad (k : Nat) : List Bool → Option (List Bool) :=
2049   fun d => some (d ++ List.replicate k false)
2050
2051 theorem compressible_bound_universal (decode : List Bool → Option (List Bool)) (n c :
2052   Nat) :
2053   (((descs (n - c)).filterMap decode).filter (fun s => s.length == n)).length ≤ 2 ^ (n
2054     - c) - 1 := by
2055   have h1 := List.length_filter_le (fun s : List Bool => s.length == n) ((descs (n - c))
2056     .filterMap decode)
2057   have h2 := List.length_filterMap_le decode (descs (n - c))
2058   rw [descs_length] at h2
2059   exact Nat.le_trans h1 h2
2060
2061 def cbUniversal (decode : List Bool → Option (List Bool)) (n c : Nat) :
2062   CompressibleBound :=
2063   (decode, n, c, compressible_bound_universal decode n) c
2064
2065 def cb_id_8_2 : CompressibleBound := (decodeId, 8, 2, by) decide
2066 def cb_pad3_8_2 : CompressibleBound := (decodePad 3, 8, 2, by) decide
2067 def cb_pad2_6_1 : CompressibleBound := (decodePad 2, 6, 1, by) decide
2068 def cb_pad4_12_3 : CompressibleBound := (decodePad 4, 12, 3, by) decide
2069
2070 def recoverable (redundancy floor tkel : Ratio) : Ratio :=
2071   (redundancy - floor) * OMEGA.kbRat * tkel * OMEGA.ln2Rat
2072
2073 theorem nomos_fuel_is_omega_floor :
2074   recoverable 1 0 300 = OMEGA.landauerRat 300 1 := by
2075   decide
2076
2077 inductive LawClaim : Type
2078 | objectLawGroove | lawAsSuchFromNoise | floorInheritedFitra | recordDerivedQadar
2079   deriving DecidableEq, Repr
2080
2081 def nomosScreen : LawClaim → Audit.Token × Grade × String
2082 | .objectLawGroove =>
2083   (.sealed, .structural,
2084     "object-laws emerge on RA as cost-reducing groove-entrenchment, the descent of
2085     realized cost toward K(X); structural on the RA-and-groove mapping")
2086 | .lawAsSuchFromNoise =>
2087   (.refused, .premise,
2088     "refused, at premise: the counting cap bounds short-description outputs by 2^(n-c)
2089     -1; the reading that almost every stream is lawless rides a distribution the cap
2090     does not supply")
2091 | .floorInheritedFitra =>
2092   (.determined, .premise,
2093     "the floor is fitra, the innate nature, RA, K(X) inherited; the identification is a
2094     reading, premise-grade, out of band")
2095 | .recordDerivedQadar =>
2096   (.determined, .structural,
2097     "the record is qadar, R(X) the harvested redundancy, derived; the derived never
2098     outranks the floor it is harvested from")
2099
2100 theorem nomosScreen_battery :
2101   (nomosScreen .lawAsSuchFromNoise).1 = .refused
2102   A (nomosScreen .objectLawGroove).2.1 = .structural
2103   A (nomosScreen .floorInheritedFitra).2.1 = .premise
2104   A (nomosScreen .recordDerivedQadar).2.1 = .structural := by
2105   decide
2106
2107 #eval (nomosScreen .lawAsSuchFromNoise).1
2108 #eval nomosRows.length
2109
2110 end NOMOS
2111
2112 namespace BRIDGE

```

```

2104
2105 inductive Warrant : Type
2106 | typeT | typeS | typeC | extension
2107 deriving DecidableEq, Repr, BEq
2108
2109 inductive PostSeal : Type
2110 | native | cited | superseded | outOfBand | byReference
2111 deriving DecidableEq, Repr, BEq
2112
2113 structure BridgeRow : Type where
2114 id : String
2115 warrant : Warrant
2116 status : PostSeal
2117 note : String
2118 deriving Repr
2119
2120 def ceiling : Warrant → Grade
2121 | .typeT => .theoremConditional
2122 | .typeS => .structural
2123 | .typeC => .corroboration
2124 | .extension => .corroboration
2125
2126 def census : List BridgeRow := [
2127   ("BA-001a", .typeT, .native,
2128     "actuation floor: energy floor theorem-grade on the Heisenberg bound and zero-point
      energy; transitions charged Jarzynski-Crooks with Landauer 1961 and Bérut 2012;
      static existence premise-grade on substrate monism. RegistrationPostulate
      declared at SECTION 7, consumed by no theorem; its arithmetic face is priced by
      OMEGA.LandauerRat (SECTION 14) and Fortran K3: the postulate charges, the
      battery prices"),
2129   ("BA-001b", .typeT, .outOfBand,
2130     "Turing halting: a V-F-only theorem-grade ceiling honored out of band; the cascade
      routes around at layer difference, never through"),
2131   ("BA-002", .typeT, .cited,
2132     "spectral dual L2: Plancherel at the flat register Type T, Tomita-Takesaki at the
      curved register Type C; audits cap at the face cited"),
2133   ("BA-003", .typeC, .cited, "epistemic phase transition at 2 k_B T ln 2 work"),
2134   ("BA-004", .typeC, .superseded,
2135     "nomological habituation, superseded at the object-law register by NOMOS-01 (SECTION
      15, namespace NOMOS)"),
2136   ("BA-005", .typeC, .cited, "conformal persistence"),
2137   ("BA-006", .typeS, .cited, "conformal cyclic adjacency at Scope B"),
2138   ("BA-007", .typeT, .cited, "holographic entropy bound, Bekenstein-Hawking extended by
      Bousso 1999"),
2139   ("BA-008", .typeS, .native,
2140     "substrate equal topology equal actuation monism; the posit face native at SECTION 15
      (FOUND.STRATIFIED-MONISM-PRIORITY)"),
2141   ("BA-009", .typeC, .cited, "matter-genesis via S1 knotting"),
2142   ("BA-010", .typeC, .cited, "variational free energy V-FIO on the Friston anchor"),
2143   ("BA-011", .typeC, .cited, "spectral-dual conformal persistence at Scope B"),
2144   ("BA-012", .typeC, .native,
2145     "cascade closure operational bijection, Type C, Hodge external corroboration; the
      witness-set enlargement (A4 torsor, Hurwitz shell slice) native at theorem grade
      at SECTION 4D"),
2146   ("BA-013", .extension, .byReference, "extension-tier bridge, loaded by reference"),
2147   ("BA-014", .extension, .byReference, "extension-tier bridge, loaded by reference"),
2148   ("BA-015", .extension, .byReference, "extension-tier bridge, loaded by reference"),
2149   ("BA-016", .extension, .byReference, "extension-tier bridge, loaded by reference"),
2150   ("BA-017", .extension, .byReference, "extension-tier bridge, loaded by reference"),
2151   ("BA-018", .typeT, .native,
2152     "quaternionic completion and the Triple-Product Verdict Identity, Type T; the gate
      roster's A4 torsor with the Hurwitz double cover native at SECTION 4D; the
      verdict-identity face cited")]
2153
2154 theorem census_count : census.length = 19 := by
2155   decide
2156
2157 theorem post_seal_disposition :
2158   (census.filter (fun b => b.status == .native)).length = 4
2159   ∧ (census.filter (fun b => b.status == .cited)).length = 8
2160   ∧ (census.filter (fun b => b.status == .superseded)).length = 1
2161   ∧ (census.filter (fun b => b.status == .outOfBand)).length = 1
2162   ∧ (census.filter (fun b => b.status == .byReference)).length = 5 := by
2163   decide
2164
2165 theorem weakest_never_above_left (a b : Grade) :
2166   (Grade.weakest a b).rank ≤ a.rank := by
2167   cases a <|> cases b <|> decide
2168
2169 theorem bridge_ceiling_caps (b : BridgeRow) (g : Grade) :
2170   (Grade.weakest (ceiling b.warrant) g).rank ≤ (ceiling b.warrant).rank :=
2171   weakest_never_above_left _ _
2172
2173 def successorOf : String → Option String
2174 | "BA-004" => some "NOMOS-01"
2175 | _ => none
2176
2177 theorem supersession_routes :
2178   successorOf "BA-004" = some "NOMOS-01"
2179   ∧ (census.any (fun b =>
2180     b.id == "BA-004" && b.status == .superseded)) = true := by
2181   decide
2182
2183 def bridgeScreen : BridgeRow → String
2184 | (_, _, .native, _) =>
2185   "load carried natively; audit the section, not the bridge"
2186 | (_, _, .cited, _) =>
2187   "cited at its warrant ceiling; never enters above it"
2188 | (_, _, .superseded, _) =>
2189   "superseded; route to the successor"
2190 | (_, _, .outOfBand, _) =>
2191   "honored out of band; routed around, never through"
2192 | (_, _, .byReference, _) =>
2193   "extension tier; carried at the tier's warrant, loaded by reference"
2194
2195 theorem bridgeScreen_total :

```

```

2196   (census.all (fun b => (bridgeScreen b).utf8ByteSize > 0)) = true := by
2197   decide
2198
2199 #eval census.length
2200 #eval census.map (fun b => (b.id, bridgeScreen b))
2201
2202 end BRIDGE
2203 namespace FOUND
2204
2205 def sigmaP : GEO.M3 :=
2206   (((-7 : Ratio) / 25, (24 : Ratio) / 25, 0),
2207     ((24 : Ratio) / 25, (7 : Ratio) / 25, 0),
2208     (0, 0, -1))
2209
2210 theorem pluralist_alternative_executed :
2211   GEO.mm3 sigmaP sigmaP = GEO.diag3 1
2212   ∧ GEO.det3 sigmaP = 1
2213   ∧ GEO.mv3 sigmaP (3, 4, 0) = ((3, 4, 0) : GEO.V3)
2214   ∧ GEO.mv3 sigmaP (1, 0, 0) ≠ ((1, 0, 0) : GEO.V3)
2215   ∧ GEO.mv3 sigmaP (1, 0, 0) ≠ ((-1, 0, 0) : GEO.V3) := by
2216   decide
2217
2218 inductive Band : Type
2219 | inBand | outOfBand
2220 deriving DecidableEq, Repr, BEq
2221
2222 structure Posit : Type where
2223 id : String
2224 grade : Grade
2225 band : Band
2226 loadBearingInVerdict : Bool
2227 note : String
2228 deriving Repr
2229
2230 def posits : List Posit := [
2231   ("RESIDUAL-MONISM", .premise, .inBand, false,
2232     "the one-prior-to-many connector: the geometric and formal grounds coincide on Fix(
      sigma) iff one involution serves both routes; the pluralist alternative executed
      above stays field-permitted; the from-other-side input located across the
      aperture and not crossed"),
2233   ("STRATIFIED-MONISM-PRIORITY", .structural, .inBand, false,
2234     "P-MONISM-1: substrate, topology, and actuation are three projections of one event,
      theorem-grade as physics; the priority of the one over the many is the single
      structural posit the floor-coincidence rides"),
2235   ("FITRA-FLOOR", .premise, .outOfBand, false,
2236     "fitra = RA = K(X), the incompressible floor inherited (QADAR-01); the theological
      identification routed out of band, load-bearing on nothing"),
2237   ("QADAR-RECORD", .structural, .outOfBand, false,
2238     "qadar = R(X), the harvested record, derived and downstream of the floor; the Pen's
      cost-gradient writing, never onto noise"),
2239   ("AMANAHA-TRUST", .premise, .outOfBand, false,
2240     "P-AMANAHA-1: ethics is the preservation of the actuation floor; the substrate a
      liability engine and never a moral authority")]
2241
2242 theorem posits_loadbearing_on_nothing :
2243   (posits.all (fun p => !p.loadBearingInVerdict)) = true := by
2244   decide
2245
2246 def gradeOfRank : Nat → Grade
2247 | 0 => .premise | 1 => .corroboration | 2 => .operational
2248 | 3 => .structural | 4 => .engineering | 5 => .conditional
2249 | 6 => .theoremConditional | 7 => .analytic | _ => .theorem
2250
2251 theorem posit_never_raises :
2252   (posits.all (fun p =>
2253     (List.range 9).all (fun r =>
2254       decide ((Grade.weakest p.grade (gradeOfRank r)).rank ≤ r)))) = true := by
2255   decide
2256
2257 inductive Register : Type
2258 | kinetic | formal | verification | theological | social
2259 deriving DecidableEq, Repr, BEq
2260
2261 def bandOf : Register → Band
2262 | .theological => .outOfBand
2263 | .social => .outOfBand
2264 | _ => .inBand
2265
2266 theorem register_routing_fidelity :
2267   bandOf .theological = .outOfBand ∧ bandOf .social = .outOfBand
2268   ∧ bandOf .kinetic = .inBand ∧ bandOf .formal = .inBand
2269   ∧ bandOf .verification = .inBand := by
2270   decide
2271
2272 structure VerdictRow : Type where
2273 name : String
2274 positDeps : List String
2275 deriving Repr
2276
2277 def verdictInventory : List VerdictRow := [
2278   ("the grade law and the weakest-link join (section 1)", ), [],
2279   ("the audit engine and the drill battery (section 4B)", ), [],
2280   ("the inversion (section 4C)", ), [],
2281   ("the twelve gates, A4, the GF(2) lock, the cascades (section 4D)", ), [],
2282   ("the forcings of eight and five (section 4E)", ), [],
2283   ("the tongue's obedience (section 11)", ), [],
2284   ("the office taxonomy (section 12)", ), [],
2285   ("the universal router and the circularity law (section 13)", ), [],
2286   ("the Omega guard and AEGIS (section 14)", ), [],
2287   ("the incompressibility cap (section 15)", ), []]
2288
2289 theorem verdicts_posit_free :
2290   (verdictInventory.all (fun v => v.positDeps.isEmpty)) = true := by
2291   decide
2292
2293 theorem engine_never_authority : socialWeight = 0 ∧ deltaM = 0 :=

```

```

2294 (rfl, )rfl
2295
2296 #eval posits.map (fun p => (p.id, p.grade, p.band))
2297 #eval List.map bandOf [Register.kinetic, .theological, .social]
2298
2299 end FOUND
2300
2301 namespace HALT
2302
2303 inductive Species : Type
2304 | blockTheorem | hypothesis | offeredWitness
2305 deriving DecidableEq, Repr
2306
2307 inductive Branch : Type
2308 | xi | o
2309 deriving DecidableEq, Repr
2310
2311 def cascade : List Bool → Option Nat
2312 | [] => none
2313 | false :: _ => some 0
2314 | true :: gs => (cascade gs).map Nat.succ
2315
2316 def patterns (n : Nat) : List (List Bool) :=
2317 (List.range (2 ^ n)).map fun b => (List.range n).map fun j => b.testBit j
2318
2319 def fftSpec (bits : List Bool) : Bool :=
2320 match cascade bits with
2321 | none => bits.all id
2322 | some k =>
2323 (bits.take k).all id &&
2324 (match bits.drop k with
2325 | false :: _ => true
2326 | _ => false)
2327
2328 theorem first_failure_terminal_5 :
2329 (patterns 5).all fftSpec = true := by decide
2330
2331 theorem first_failure_terminal_7 :
2332 (patterns 7).all fftSpec = true := by decide
2333
2334 theorem first_failure_terminal_8 :
2335 (patterns 8).all fftSpec = true := by decide
2336
2337 theorem first_failure_terminal_12 :
2338 (patterns 12).all fftSpec = true := by decide
2339
2340 theorem halt_free_iff_all_pass_8 :
2341 (patterns 8).all (fun bits => (cascade bits).isNone == bits.all id)
2342 = true := by decide
2343
2344 theorem gate_counts_recorded :
2345 GEO.a4.length = 12 ∧ GEO.allSigns.length = 8 ∧
2346 (GEO.cube5.all (fun v => (GEO.neighbors5 v).eraseDups.length == 5)) = true :=
2347 (GEO.a4_order, GEO.reflection_eight, GEO.fivecube_degree_five
2348
2349 abbrev signPatterns : List (Bool × Bool × Bool) := GEO.allSigns
2350
2351 theorem ninth_gate_barred :
2352 signPatterns.length = 8 ∧ signPatterns.eraseDups.length = 8 ∧
2353 signPatterns = GEO.allSigns := (by decide, by decide, )rfl
2354
2355 def route (groundDim : Nat) : Option Branch :=
2356 if groundDim = 1 then some .xi else if groundDim = 0 then some .o else none
2357
2358 def ofUC : UC.HaltBranch → Option Branch
2359 | .haltEight => some .xi
2360 | .haltFive => some .o
2361 | .unmeasured => none
2362
2363 theorem route_is_routeHalt : ∀ n : Nat, route n = ofUC (UC.routeHalt n).1 := by
2364 intro n
2365 unfold route UC.routeHalt ofUC
2366 by_cases h1 : n = 1
2367 · subst h1; simp
2368 · by_cases h0 : n = 0
2369 · subst h0; simp
2370 · have h1' : (n : Int) ≠ 1 := by omega
2371 · have h0' : (n : Int) ≠ 0 := by omega
2372 · simp only [h1, h0, h1', h0', ite_false]
2373
2374 theorem route_samples :
2375 route 0 = some .o ∧ route 1 = some .xi ∧ route 2 = none ∧ route 7 = none := by
2376 decide
2377
2378 inductive LegacyToken : Type
2379 | xi0 | o0
2380 deriving DecidableEq, Repr
2381
2382 def tokenBranch : LegacyToken → Branch
2383 | .xi0 => .xi
2384 | .o0 => .o
2385
2386 theorem router_exclusive :
2387 route 0 = some .o ∧ route 1 = some .xi ∧ route 2 = none ∧
2388 route 0 ≠ some .xi ∧ route 1 ≠ some .o ∧
2389 tokenBranch .xi0 ≠ .o ∧ tokenBranch .o0 ≠ .xi := by decide
2390
2391 def readingRoadsGate : Bool := false
2392
2393 theorem one_door_open :
2394 cascade (List.replicate 7 true ++ [readingRoadsGate]) = some 7 := by
2395 decide
2396
2397 theorem no_walk_passes_the_aperture :
2398 (patterns 7).all (fun xs => (cascade (xs ++ [readingRoadsGate])).isSome) = true :=
2399 by

```

```

2398 decide
2399
2400 def xiGates : List String :=
2401 [ "Ground-determinacy", "Ghost partition", "constitutive blindness",
2402 "machine-precision exhibit", "aperture located", "face-scoping",
2403 "revision-mandate survival", "reading-roads" ]
2404
2405 def oGates : List String :=
2406 [ "rootless terrain", "determinate stripped string", "wall record",
2407 "aperture crossed or held", "totality discipline" ]
2408
2409 theorem gate_censuses :
2410 xiGates.length = 8 ∧ oGates.length = 5 := by decide
2411
2412 inductive Channel : Type
2413 | kinetic | formal
2414 deriving DecidableEq, Repr
2415
2416 structure SuperHalt where
2417 species : Species
2418 gdim : Nat
2419 branch : Branch
2420 gates : Nat
2421 channel : Channel
2422 routed : route gdim = some branch
2423 deriving DecidableEq
2424
2425 def rhHalt : SuperHalt := (.hypothesis, 1, .xi, 8, .formal, by)decide
2426
2427 def pnpHalt : SuperHalt := (.blockTheorem, 0, .o, 5, .formal, by)decide
2428
2429 theorem halts_routed :
2430 route rhHalt.gdim = some rhHalt.branch ∧ route pnpHalt.gdim = some pnpHalt.branch :=
2431 (rhHalt.routed, pnpHalt.routed)
2432
2433 theorem legacy_halts_are_instances :
2434 rhHalt = (.hypothesis, 1, .xi, 8, .formal, by)decide ∧
2435 pnpHalt = (.blockTheorem, 0, .o, 5, .formal, by)decide ∧
2436 rhHalt ≠ pnpHalt := by decide
2437
2438 theorem terminal_one_bit :
2439 Species.blockTheorem ≠ Species.hypothesis ∧
2440 Species.blockTheorem ≠ Species.offeredWitness ∧
2441 Species.hypothesis ≠ Species.offeredWitness := by decide
2442
2443 theorem species_exhausted : ∀ s : Species,
2444 s = .blockTheorem ∨ s = .hypothesis ∨ s = .offeredWitness := by
2445 intro s; cases s
2446 · exact 0r.inl rfl
2447 · exact 0r.inr (0r.inl rfl)
2448 · exact 0r.inr (0r.inr rfl)
2449
2450 def haltGrades : List Grade := [.structural, .theoremConditional]
2451
2452 theorem ftoc_ceiling :
2453 haltGrades.all (fun g => g.rank < Grade.theorem.rank) = true := by
2454 decide
2455
2456 end HALT
2457
2458 /-! ## The four Fortran witnesses
2459 F1 thesis_rows (the core thesis executed), F2 ra_toe_thesis (the existence root executed
2460 ), F3
2461 ftoc_kinetic_demonstration (the deed priced) and F4 it_from_it (the inversion computed)
2462 are seated in Part II
2463 of this codex, the Fortran twin, one executable, with every contract comment they were
2464 audited with.
2465 The Lean namespaces Audit, GEO, IAM, OMEGA, AEGIS, INVERSION, K4 and CROSSING are their
2466 formal register. -/
2467
2468 namespace CROSSING
2469 open FTOE
2470
2471 abbrev ExecFrame (Q : Type) := Bool × Q
2472
2473 def execFlip {Q : Type} : ExecFrame Q → ExecFrame Q := flipF
2474
2475 def formalRead {Q : Type} (s : ExecFrame Q) : Q := s.2
2476
2477 def ran {Q : Type} (s : ExecFrame Q) : Bool := s.1
2478
2479 theorem formalRead_even {Q : Type} : ∀ s : ExecFrame Q,
2480 formalRead (execFlip s) = formalRead s := by
2481 intro s; obtain (b, )q := s; rfl
2482
2483 theorem ran_wholly_odd {Q : Type} : WhollyOdd (execFlip (Q := Q)) ran := by
2484 intro s; obtain (b, )q := s; cases b <;> rfl
2485
2486 theorem execFlip_involution {Q : Type} : Involution (execFlip (Q := Q)) := by
2487 intro s; obtain (b, )q := s; cases b <;> rfl
2488
2489 theorem ran_odd_at_true {Q : Type} (q : Q) : OddAt (execFlip (Q := Q)) ran (true, q) :=
2490 fun h => Bool.noConfusion h
2491
2492 theorem wall {Q : Type} (q : Q) :
2493 ~ ∃ g : Q → Bool, ∀ s : ExecFrame Q, g (formalRead s) = ran s :=
2494 T14_wall_factorization execFlip formalRead ran (true, q) formalRead_even (
2495 ran_odd_at_true q)
2496
2497 def deed {Q : Type} : Dtau (execFlip (Q := Q)) := (ran, )ran_wholly_odd
2498
2499 theorem price {Q : Type} :
2500 Nonempty (Dtau (execFlip (Q := Q)) =
2501 (Quot (OrbitRel (execFlip (Q := Q))) → Bool)) :=
2502 T16_price_bijection execFlip execFlip_involution deed
2503
2504

```

```

2499 theorem crossing {Q : Type} (q : Q) (witness : ExecFrame Q → Bool)
2500   (hw : witness (execFlip (true, q)) = !witness (true, q)) :
2501   ∃ c : Bool,
2502     (ran (true, q) = xor (witness (true, q)) c ∧
2503      ran (execFlip (true, q)) = xor (witness (execFlip (true, q))) c) ∧
2504     ∀ c' : Bool,
2505       (ran (true, q) = xor (witness (true, q)) c' ∧
2506        ran (execFlip (true, q)) = xor (witness (execFlip (true, q))) c') → c' = c :=
2507   T5_crossing execFlip witness ran (true, q) hw (ran_wholly_odd (true, q))
2508
2509 theorem identity_one_bit {Q : Type} (witness : ExecFrame Q → Bool)
2510   (hw : WhollyOdd execFlip witness) (s : ExecFrame Q) (h : witness s = ran s) :
2511   witness (execFlip s) = ran (execFlip s) :=
2512   T4_one_bit execFlip witness ran hw ran_wholly_odd s h
2513
2514 inductive Anchor : Type
2515 | F1 | F2 | F3 | F4
2516 deriving DecidableEq, Repr
2517
2518 def fibre4 : List (Bool × Bool × Bool × Bool) :=
2519   [false, true].flatMap fun a => [false, true].flatMap fun b =>
2520     [false, true].flatMap fun c => [false, true].map fun d => (a, b, c, d)
2521
2522 theorem fibre4_sixteen : fibre4.length = 16 := by decide
2523 theorem fibre4_distinct : fibre4.eraseDups.length = 16 := by decide
2524
2525 theorem price_on_this_file :
2526   Nonempty (Dtau (execFlip (Q := Anchor)) =
2527     (Quot (OrbitRel (execFlip (Q := Anchor)) → Bool)) :=
2528     price
2529
2530 end CROSSING
2531
2532 namespace CROSSING
2533 open FTOE
2534
2535 theorem formal_never_odd {Q : Type} (g : Q → Bool) :
2536   Even (execFlip (Q := Q)) (fun s => g (formalRead s)) := by
2537   intro s; obtain (b, )q := s; rfl
2538
2539 theorem constant_no_crossing {Q : Type} (b : Bool) (s : ExecFrame Q) :
2540   ~ OddAt (execFlip (Q := Q)) (fun _ => b) s := by
2541   intro h; exact h rfl
2542
2543 theorem harmony {Q : Type} (q : Q) (witness : ExecFrame Q → Bool)
2544   (hw : witness (execFlip (true, q)) = !witness (true, q))
2545   (ιg ιg : Q → Bool) (op : Bool → Bool → Bool) :
2546   (fun s => op (ιg (formalRead s)) (ιg (formalRead s))) ≠ ran ∧
2547   ∃ c : Bool,
2548     (ran (true, q) = xor (witness (true, q)) c ∧
2549      ran (execFlip (true, q)) = xor (witness (execFlip (true, q))) c) ∧
2550     ∀ c' : Bool,
2551       (ran (true, q) = xor (witness (true, q)) c' ∧
2552        ran (execFlip (true, q)) = xor (witness (execFlip (true, q))) c') → c' = c :=
2553   fTOE_core execFlip witness ran (true, q)
2554   (fun s => ιg (formalRead s)) (fun s => ιg (formalRead s))
2555   (formal_never_odd ιg) (formal_never_odd ιg) op
2556   (ran_wholly_odd (true, q)) hw
2557
2558 end CROSSING
2559
2560 namespace LOCUS
2561 open FTOE
2562
2563 def Separates α {β : Type} p ( : α → β) (d : α → Bool) : Prop :=
2564   ∃ x y, p x = p y ∧ d x ≠ d y
2565
2566 def Factors α {β : Type} p ( : α → β) (d : α → Bool) : Prop :=
2567   ∃ g : β → Bool, ∀ z, g p(z) = d z
2568
2569 theorem wall_of_sep α {β : Type} p ( : α → β) {d : α → Bool} :
2570   Separates p d → ~ Factors p d := by
2571   intro (x, y, ph, )hd (g, )hg
2572   apply hd
2573   rw ←[ hg x, ← hg y, ph]
2574
2575 def swap α { : Type} [DecidableEq α] (x y : α) (z : α) : α :=
2576   if z = x then y else if z = y then x else z
2577
2578 theorem swap_involution α { : Type} [DecidableEq α] {x y : α} (hxy : x ≠ y) :
2579   ∀ z, swap x y (swap x y z) = z := by
2580   intro z
2581   by_cases hzx : z = x
2582   · simp [swap, hzx, Ne.symm hxy]
2583   · by_cases hzy : z = y
2584   · simp [swap, hzy, Ne.symm hxy]
2585   · simp [swap, hzx, hzy]
2586
2587 theorem swap_record_even α {β : Type} [DecidableEq α] p { : α → β} {x y : α}
2588   (hxy : x ≠ y) (ph : p x = p y) : ∀ z, p (swap x y z) = p z := by
2589   intro z
2590   by_cases hzx : z = x
2591   · simp [swap, hzx, ph]
2592   · by_cases hzy : z = y
2593   · simp [swap, hzy, Ne.symm hxy, ph]
2594   · simp [swap, hzx, hzy]
2595
2596 theorem swap_flips_pair α { : Type} [DecidableEq α] {d : α → Bool} {x y : α}
2597   (hxy : x ≠ y) (hd : d x ≠ d y) :
2598   d (swap x y x) = !d x ∧ d (swap x y y) = !d y := by
2599   have h1 : swap x y x = y := by simp [swap]
2600   have h2 : swap x y y = x := by simp [swap, Ne.symm hxy]
2601   rw [h1, h2]
2602   cases hx : d x <=> cases hy : d y <=> simp_all
2603
2604 theorem seat_of_sep α {β : Type} [DecidableEq α] p ( : α → β) {d : α → Bool} {x y : α}

```

```

2605   (ph : p x = p y) (hd : d x ≠ d y) :
2606   ∃ τ : α → α, ∀ (z, τ τ(z) = z) ∧ ∀ (z, p τ(z) = p z) ∧ d τ(x) = !d x := by
2607   have hxy : x ≠ y := fun h => hd (h * rfl)
2608   exact (swap x y, swap_involution hxy, swap_record_even hxy ph, (swap_flips_pair hxy hd
2609   ))1
2610
2611 theorem k4_separated_pair : Separates K4.rho K4.target :=
2612   (0, 1, rfl, by)decide
2613
2614 theorem k4_wall_of_pair : ~ Factors K4.rho K4.target := wall_of_sep k4_separated_pair
2615
2616 def readout α {β : Type} [DecidableEq β] (xs : List α) p ( : α → β) (d : α → Bool) (b :
2617   β) : Bool :=
2618   match xs.find? (fun x => decide p(x = b)) with
2619   | some x => d x
2620   | none => false
2621
2622 theorem factors_of_not_sep_list α {β : Type} [DecidableEq β] (xs : List α)
2623   (hxs : ∀ z, z ∈ xs) p ( : α → β) (d : α → Bool) (h : ~ Separates p d) :
2624   Factors p d := by
2625   refine (readout xs p d, fun z => ?)
2626   unfold readout
2627   cases hf : xs.find? (fun x => decide p(x = p z)) with
2628   | none =>
2629     have : List.find?_eq_none.mp hf z (hxs z)
2630     simp at this
2631   | some x =>
2632     have hx : p x = p z := by simpa using List.find?_some hf
2633     show d x = d z
2634     cases hdx : d x <=> cases hdz : d z
2635     · rfl
2636     · exact absurd (x, z, hx, by rw [hdx, hdz]; )decide h
2637     · exact absurd (x, z, hx, by rw [hdx, hdz]; )decide h
2638     · rfl
2639
2640 theorem factors_iff_not_sep_list α {β : Type} [DecidableEq β] (xs : List α)
2641   (hxs : ∀ z, z ∈ xs) p ( : α → β) (d : α → Bool) :
2642   Factors p d ↔ ~ Separates p d := by
2643   refine (fun hf hs => ?_, factors_of_not_sep_list xs hxs p) d
2644   obtain (x, y, ph, )hd := hs; obtain (g, )hg := hf
2645   exact hd (by rw ←[ hg x, ← hg y, ph])
2646
2647 abbrev Channel := HALT.Channel
2648
2649 inductive Token : Type
2650 | superHalt (b : Option HALT.Branch) (s : Option HALT.Species) (c : Option Channel) (m
2651   : Nat)
2652 | crossed (c : Channel)
2653 | breach
2654 deriving DecidableEq, Repr
2655
2656 def Token.toEconomy : Token → Audit.Token
2657 | .superHalt ... => .opn
2658 | .crossed _ => .sealed
2659 | .breach => .broken
2660
2661 structure Reading where
2662   broken : Bool
2663   located : Bool
2664   gdim : Option Nat
2665   species : Option HALT.Species
2666   channel : Option Channel
2667   slots : List (Option Bool)
2668   orbits_pos : 0 < slots.length
2669   deriving DecidableEq, Repr
2670
2671 def Reading.orbits (r : Reading) : Nat := r.slots.length
2672
2673 def Reading.owed (r : Reading) : Nat := (r.slots.filter Option.isNone).length
2674
2675 def Reading.complete (r : Reading) : Bool := r.owed == 0
2676
2677 def branchOf : Option Nat → Option HALT.Branch
2678 | some n => HALT.route n
2679 | none => none
2680
2681 def emit (r : Reading) : Option Token :=
2682   if r.broken then some .breach
2683   else if !r.located then none
2684   else match r.complete, r.channel with
2685   | true, some c => some (.crossed c)
2686   | _, _ => some (.superHalt (branchOf r.gdim) r.species r.channel r.owed)
2687
2688 def Reading.state (r : Reading) : Audit.Token :=
2689   match emit r with
2690   | some t => t.toEconomy
2691   | none => .opn
2692
2693 theorem three_states : ∀ t : Token,
2694   t.toEconomy = .sealed ∨ t.toEconomy = .broken ∨ t.toEconomy = .opn := by
2695   intro t; cases t <=> simp [Token.toEconomy]
2696
2697 theorem no_seat_no_locus (r : Reading) (h : r.located = false) (hb : r.broken = false) :
2698   emit r = none := by
2699   simp [emit, h, hb]
2700
2701 theorem unlocated_is_plain_open (r : Reading) (h : r.located = false) (hb : r.broken =
2702   false) :
2703   r.state = .opn := by
2704   simp [Reading.state, no_seat_no_locus r h hb]
2705
2706 theorem located_never_silent (r : Reading) (hl : r.located = true) (hb : r.broken =
2707   false) :
2708   emit r ≠ none := by
2709   simp only [emit, hl, hb]
2710   cases r.complete <=> cases r.channel <=> simp

```

```

2706
2707 theorem crossed_requires_complete (r : Reading) (c : Channel) (h : emit r = some (.
  crossed c)) :
2708   r.complete = true := by
2709   cases hb : r.broken <=> cases hl : r.located <=> cases hc : r.complete <=> cases hch :
     r.channel <=>
2710     simp [emit, hb, hl, hc, hch] at h ⊢
2711
2712 theorem crossed_requires_channel (r : Reading) (c : Channel) (h : emit r = some (.
  crossed c)) :
2713   r.channel = some c := by
2714   cases hb : r.broken <=> cases hl : r.located <=> cases hc : r.complete <=> cases hch :
     r.channel <=>
2715     simp [emit, hb, hl, hc, hch] at h ⊢
2716     exact h
2717
2718 theorem halt_carries_owed (r : Reading) (hl : r.located = true) (hb : r.broken = false)
  (hnc : ¬ (r.complete = true ∧ r.channel.isSome = true)) :
2719   emit r = some (.superHalt (branchOf r.gdim) r.species r.channel r.owed) := by
2720   simp only [emit, hl, hb]
2721   cases hc : r.complete <=> cases hch : r.channel <=> simp_all
2722
2723
2724 theorem unchannelled_is_halt (r : Reading) (hl : r.located = true) (hb : r.broken =
  false)
2725   (hch : r.channel = none) :
2726   emit r = some (.superHalt (branchOf r.gdim) r.species none r.owed) := by
2727   simp only [emit, hl, hb, hch]
2728   cases r.complete <=> simp
2729
2730 theorem halt_reports_owed (r : Reading) (b : Option HALT.Branch) (s : Option HALT.
  Species)
2731   (c : Option Channel) (m : Nat) (h : emit r = some (.superHalt b s c m)) : m = r.owed
  := by
2732   cases hb : r.broken <=> cases hl : r.located <=> cases hc : r.complete <=> cases hch :
     r.channel <=>
2733     simp [emit, hb, hl, hc, hch] at h <=> simp_all
2734
2735 theorem owed_zero_is_unchannelled (r : Reading) (b : Option HALT.Branch) (s : Option
  HALT.Species)
2736   (c : Option Channel) (h : emit r = some (.superHalt b s c 0)) : c = none := by
2737   have how : 0 = r.owed := halt_reports_owed r b s c 0 h
2738   have hcomp : r.complete = true := by
2739     unfold Reading.complete; rw ← [how]; rfl
2740   cases hb : r.broken
2741   · cases hl : r.located
2742     · rw [no_seat_no_locus r hl hb] at h; cases h
2743     · cases hch : r.channel with
2744       | none =>
2745         rw [unchannelled_is_halt r hl hb hch] at h
2746         injection h with h'
2747         injection h' with _ h3 _
2748         exact h3.symm
2749       | some ch =>
2750         have hx : emit r = some (.crossed ch) := by
2751           unfold emit; rw [hb, hl, hcomp, hch]; rfl
2752         rw [hx] at h; injection h with h'; cases h'
2753       · have hx : emit r = some .breach := by unfold emit; rw [hb]; rfl
2754       · rw [hx] at h; injection h with h'; cases h'
2755
2756 theorem one_bit_apart (r : Reading) (hl : r.located = true) (hb : r.broken = false)
  (c : Channel) (hc : r.channel = some c) :
2757   ({ r with slots := [none], orbits_pos := by decide }).state = .opn ∧
2758   ({ r with slots := [some true], orbits_pos := by decide }).state = .sealed ∧
2759   ({ r with slots := [some false], orbits_pos := by decide }).state = .sealed := by
2760   refine (?, ?, ?) <=>
2761     (unfold Reading.state emit Reading.complete Reading.owed; dsimp only; rw [hb, hl, hc
  ]; rfl)
2762
2763
2764 theorem legacy_xi : branchOf (some 1) = some .xi := by decide
2765 theorem legacy_o : branchOf (some 0) = some .o := by decide
2766 theorem legacy_bare : branchOf none = none ∧ branchOf (some 2) = none := by decide
2767
2768 theorem k4_locus :
2769   emit (false, true, none, none, some .formal, [none, none, none, none, none],
  by decide
2770     = some (.superHalt none none (some .formal) 6) ∧
2771   emit (false, true, none, none, some .formal,
2772     [some true, some true, some true, some true, some true, none], by decide
2773     = some (.superHalt none none (some .formal) 1) ∧
2774   emit (false, true, none, none, some .formal,
2775     [some true, some true, some true, some true, some true, some true], by decide
2776     = some (.crossed .formal) ∧
2777   emit (false, true, none, none, none,
2778     [some true, some true, some true, some true, some true, some true], by decide
2779     = some (.superHalt none none none 0) := by decide
2780
2781 def slotLists : List (List (Option Bool)) :=
2782   let o : List (Option Bool) := [none, some false, some true]
2783   o.map (fun a => [a]) ++ o.flatMap (fun a => o.map (fun b => [a, b]))
2784
2785 theorem slotLists_twelve : slotLists.length = 12 ∧ slotLists.all (fun l => 0 < l.length)
  = true := by
2786   decide
2787
2788 theorem slotLists_pos : ∀ sl ∈ slotLists, 0 < sl.length := by decide
2789
2790 def allReadings : List Reading :=
2791   [false, true].flatMap fun broken =>
2792   [false, true].flatMap fun located =>
2793   [none, some 0, some 1, some 2].flatMap fun gdim =>
2794   [(none, some .blockTheorem, some .hypothesis, some .offeredWitness) : List (Option
  HALT.Species)].flatMap fun sp =>
2795   [(none, some .kinetic, some .formal) : List (Option Channel)].flatMap fun ch =>
2796   [slotLists.attach.map fun (sl, ) hsl =>
2797     ((broken, located, gdim, sp, ch, sl, slotLists_pos sl ) hsl : Reading)]
2798

```

```

2799 theorem reading_space : allReadings.length = 2304 := by decide
2800
2801 theorem census_no_seat_no_locus :
2802   (allReadings.filter (fun r => !r.broken && !r.located)).all
2803   (fun r => decide (emit r = none)) = true := by decide
2804
2805 theorem census_located_never_silent :
2806   (allReadings.filter (fun r => !r.broken && r.located)).all
2807   (fun r => decide (emit r ≠ none)) = true := by decide
2808
2809 theorem census_crossed_iff_complete_channelled :
2810   allReadings.all (fun r =>
2811     ((emit r == some (.crossed .kinetic)) || (emit r == some (.crossed .formal)))
2812     == (!r.broken && r.located && r.complete && r.channel.isSome)) = true := by
2813     decide
2814
2815 theorem census_halt_reports_owed :
2816   (allReadings.filter (fun r => !r.broken && r.located && !(r.complete && r.channel.
  isSome))).all
2817   (fun r => emit r == some (.superHalt (branchOf r.gdim) r.species r.channel r.owed)
  ) = true := by
2818     decide
2819
2820 theorem census_owed_zero_is_unchannelled :
2821   allReadings.all (fun r =>
2822     match emit r with
2823     | some (.superHalt _ _ c 0) => c == none
2824     | _ => true) = true := by decide
2825
2826 theorem census_three_states :
2827   allReadings.all (fun r => decide (r.state = .sealed ∨ r.state = .broken ∨ r.state =
  .opn)) = true := by
2828     decide
2829
2830 def Token.render : Token → String
2831 | .superHalt b s c m =>
2832   let br := match b with | some .xi => "·Xi" | some .o => "·0" | none => ""
2833   let sp := match s with
2834     | some .blockTheorem => "·blockTheorem" | some .hypothesis => "·hypothesis"
2835     | some .offeredWitness => "·offeredWitness" | none => ""
2836   let ch := match c with | some .kinetic => "·kinetic" | some .formal => "·formal"
2837     | none => ""
2838   let mult := if m = 0 then " " · provenance owed" else s!"x[m]"
2839   s!"{br}{sp}{ch}{mult}"
2840 | .crossed .kinetic => "⋈[·kinetic]"
2841 | .crossed .formal => "⋈[·formal]"
2842 | .breach => "⊗[·mechanism]"
2843
2844 def Token.ascii : Token → String
2845 | .superHalt b _ _ m =>
2846   let base := match b with | some .xi => "[HALT-LOCUS-XI]" | some .o => "[HALT-LOCUS-0]"
2847     | none => "[HALT-LOCUS]"
2848   if m = 0 then s!"(base).PROV-OWED)" else s!"(base) x[m]"
2849 | .crossed _ => "[SEAL-LOCUS]"
2850 | .breach => "[X-LOCUS]"
2851
2852 def sealAscii : String := "[SEAL]"
2853
2854 theorem render_receipted :
2855   (Token.superHalt none none none 1).render = "⋈[·x1]" ∧
2856   (Token.superHalt (some .xi) none none 1).render = "⋈[·provenance owed]" ∧
2857   (Token.superHalt (some .o) none none 1).render = "⋈[·0x1]" ∧
2858   (Token.superHalt (some .xi) (some .hypothesis) (some .formal) 1).render
2859   = "⋈[·Xi · hypothesis · formal x1]" ∧
2860   (Token.superHalt none none (some .formal) 6).render = "⋈[·formal x6]" ∧
2861   (Token.superHalt none none none 0).render = "⋈[·provenance owed]" ∧
2862   (Token.superHalt none none (some .formal) 1).render = "⋈[·formal x1]" ∧
2863   (Token.superHalt (some .xi) none none 3).render = "⋈[·Xi x3]" ∧
2864   (Token.crossed .kinetic).render = "⋈[·kinetic]" ∧
2865   (Token.crossed .formal).render = "⋈[·formal]" ∧
2866   Token.breach.render = "⊗[·mechanism]" ∧
2867   (Token.superHalt (some .xi) none none 1).ascii = "[HALT-LOCUS-XI x1]" ∧
2868   (Token.superHalt (some .xi) none none 3).ascii = "[HALT-LOCUS-XI x3]" ∧
2869   (Token.superHalt (some .o) none none 2).ascii = "[HALT-LOCUS-0 x2]" ∧
2870   (Token.superHalt none none none 6).ascii = "[HALT-LOCUS x6]" ∧
2871   (Token.superHalt none none none 1).ascii = "[HALT-LOCUS x1]" ∧
2872   (Token.superHalt none none none 0).ascii = "[HALT-LOCUS PROV-OWED]" ∧
2873   (Token.crossed .formal).ascii = "[SEAL-LOCUS]" ∧
2874   Token.breach.ascii = "[X-LOCUS]" ∧
2875   sealAscii = "[SEAL]" := by decide
2876
2877 theorem render_injective_in_owed :
2878   ((List.range 13).all fun m => (List.range 13).all fun n =>
2879     (decide ((Token.superHalt none none none m).ascii = (Token.superHalt none none
  none n).ascii) == decide (m = n)) &&
2880     (decide ((Token.superHalt (some .xi) none none m).ascii = (Token.superHalt (some .
  xi) none none n).ascii) == decide (m = n)) &&
2881     (decide ((Token.superHalt none none (some .formal) m).render = (Token.superHalt
  none none (some .formal) n).render) == decide (m = n))) = true := by
2882     decide
2883
2884 #eval (Token.superHalt (some .xi) (some .hypothesis) (some .formal) 1).render
2885 #eval (Token.superHalt none none (some .formal) 6).render
2886 #eval (Token.crossed .kinetic).render
2887 #eval Token.breach.render
2888
2889 def voidImage : Audit.Token := .opn
2890 theorem void_precedes_economy : voidImage = .opn ∧ Audit.Token.void ≠ Audit.Token.
  refused := by decide
2891
2892 theorem legacy_records_not_promoted :
2893   HALT.rhHalt.species = .hypothesis ∧ HALT.pnpHalt.species = .blockTheorem ∧
2894   HALT.rhHalt.channel = .formal ∧ HALT.pnpHalt.channel = .formal := by decide
2895
2896 theorem no_rh_locus :
2897   emit (false, true, some 1, some .hypothesis, some .formal, [none], by decide

```

```

2896   = some (.superHalt (some .xi) (some .hypothesis) (some .formal) 1) := by decide
2897
2898 end LOCUS
2899
2900 namespace PSP.ORIENT
2901
2902 abbrev V3 := Ratio × Ratio × Ratio
2903 abbrev M3 := V3 × V3 × V3
2904
2905 def det3 (m : M3) : Ratio :=
2906   m.1.1 * (m.2.1.2.1 * m.2.2.2.2 - m.2.1.2.2 * m.2.2.2.1)
2907   - m.1.2.1 * (m.2.1.1 * m.2.2.2.2 - m.2.1.2.2 * m.2.2.1)
2908   + m.1.2.2 * (m.2.1.1 * m.2.2.2.1 - m.2.1.2.1 * m.2.2.1)
2909
2910 def neg3 (m : M3) : M3 :=
2911   ((-m.1.1, -m.1.2.1, -m.1.2.2),
2912    (-m.2.1.1, -m.2.1.2.1, -m.2.1.2.2),
2913    (-m.2.2.1, -m.2.2.2.1, -m.2.2.2.2))
2914
2915 def signMatrices : List M3 :=
2916   [ ((1,0,0),(0,1,0),(0,0,1)), ((-1,0,0),(0,1,0),(0,0,1)),
2917     ((1,0,0),(0,-1,0),(0,0,1)), ((1,0,0),(0,1,0),(0,0,-1)),
2918     ((-1,0,0),(0,-1,0),(0,0,1)), ((-1,0,0),(0,1,0),(0,0,-1)),
2919     ((1,0,0),(0,-1,0),(0,0,-1)), ((-1,0,0),(0,-1,0),(0,0,-1)) ]
2920
2921 theorem lock_scalar_sign_blind :
2922   signMatrices.all (fun m =>
2923     det3 m * det3 m == det3 (neg3 m) * det3 (neg3 m)) = true := by decide
2924
2925 theorem det_sign_flips_in_odd_dimension :
2926   signMatrices.all (fun m => det3 (neg3 m) == -det3 m) = true := by decide
2927
2928 end PSP.ORIENT
2929
2930 namespace PSP.QUAT
2931
2932 abbrev Q := Ratio × Ratio × Ratio × Ratio
2933
2934 def qmul (p q : Q) : Q :=
2935   (p.1*q.1 - p.2.1*q.2.1 - p.2.2.1*q.2.2.1 - p.2.2.2*q.2.2.2,
2936    p.1*q.2.1 + p.2.1*q.1 + p.2.2.1*q.2.2.2 - p.2.2.2*q.2.2.1,
2937    p.1*q.2.2.1 - p.2.1*q.2.2.2 + p.2.2.1*q.1 + p.2.2.2*q.2.1,
2938    p.1*q.2.2.2 + p.2.1*q.2.2.1 - p.2.2.1*q.2.1 + p.2.2.2*q.1)
2939
2940 def qi : Q := (0,1,0,0)
2941 def qj : Q := (0,0,1,0)
2942 def qk : Q := (0,0,0,1)
2943 def qnegl : Q := (-1,0,0,0)
2944
2945 theorem quat_basis_laws :
2946   qmul qi qi = qnegl ∧ qmul qj qj = qnegl ∧ qmul qk qk = qnegl ∧
2947   qmul (qmul qi qj) qk = qnegl := by decide
2948
2949 end PSP.QUAT
2950
2951 namespace PSP.QUAT
2952
2953 def qnorm (p : Q) : Ratio := p.1^2 + p.2.1^2 + p.2.2.1^2 + p.2.2.2^2
2954 def isInt (r : Ratio) : Bool := r.den == 1
2955 def isHalf (r : Ratio) : Bool := r.den == 2
2956 def hurwitzOK (p : Q) : Bool :=
2957   (isInt p.1 && isInt p.2.1 && isInt p.2.2.1 && isInt p.2.2.2) ||
2958   (isHalf p.1 && isHalf p.2.1 && isHalf p.2.2.1 && isHalf p.2.2.2)
2959
2960 def grid : List Ratio := [-1, -1/2, 0, 1/2, 1]
2961 def tuples4 : List Q :=
2962   grid.flatMap fun a => grid.flatMap fun b =>
2963     grid.flatMap fun c => grid.map fun d => (a,b,c,d)
2964
2965 theorem hurwitz_units_24 :
2966   (tuples4.filter fun p => hurwitzOK p && qnorm p == 1).length = 24 := by decide
2967
2968 end PSP.QUAT
2969
2970 namespace PSP.QUAT
2971
2972 def grid2 : List Ratio := [-2, -1, 0, 1, 2]
2973 def tuples4i : List Q :=
2974   grid2.flatMap fun a => grid2.flatMap fun b =>
2975     grid2.flatMap fun c => grid2.map fun d => (a,b,c,d)
2976
2977 theorem norm2_shell_split :
2978   (tuples4i.filter fun p => qnorm p == 2).length = 24 ∧
2979   (tuples4i.filter fun p => qnorm p == 2 && p.1 == 0).length = 12 ∧
2980   (tuples4i.filter fun p => qnorm p == 2 && p.1 != 0).length = 12 := by decide
2981
2982 end PSP.QUAT
2983
2984 namespace PSP.CD
2985
2986 def cdconj : List Ratio → List Ratio
2987   | [] => []
2988   | [a] => [a]
2989   | a :: rest => a :: rest.map (fun x => -x)
2990
2991 def padd : List Ratio → List Ratio → List Ratio := List.zipWith (· + ·)
2992 def psub : List Ratio → List Ratio → List Ratio := List.zipWith (· - ·)
2993
2994 def cdmul : Nat → List Ratio → List Ratio → List Ratio
2995   | 0, _, _ => []
2996   | _+1, [a], [b] => [a * b]
2997   | fuel+1, a, b =>
2998     let n := a.length / 2
2999     let a1 := a.take n; let a2 := a.drop n
3000     let b1 := b.take n; let b2 := b.drop n
3001     psub (cdmul fuel a1 b1) (cdmul fuel (cdconj b2) a2) ++

```

```

3002     padd (cdmul fuel b2 a1) (cdmul fuel a2 (cdconj b1))
3003
3004 def e (n i : Nat) : List Ratio := List.replicate i 0 ++ [1] ++ List.replicate (n - i - 1) 0
3005
3006 def smul (s : Ratio) (v : List Ratio) : List Ratio := v.map (fun x => s * x)
3007
3008 theorem octonion_associator :
3009   psub (cdmul 4 (cdmul 4 (e 8 1) (e 8 2)) (e 8 4))
3010     (cdmul 4 (e 8 1) (cdmul 4 (e 8 2) (e 8 4)))
3011   = smul 2 (e 8 7) := by decide
3012
3013 end PSP.CD
3014
3015 namespace PSP.CD
3016
3017 def sx : List Ratio := padd (e 16 1) (e 16 10)
3018
3019 def sy : List Ratio := padd (e 16 5) (e 16 14)
3020 def zero16 : List Ratio := List.replicate 16 0
3021 def sqnorm (v : List Ratio) : Ratio := (v.map fun x => x^2).sum
3022
3023 theorem sedenion_zero_divisor :
3024   cdmul 5 sx sy = zero16 ∧ sx ≠ zero16 ∧ sy ≠ zero16
3025   ∧ sqnorm sx = 2 ∧ sqnorm sy = 2 := by decide
3026
3027 end PSP.CD
3028
3029 namespace PSP.CASCADE
3030
3031 def cascade : List Bool → Option Nat
3032   | [] => none
3033   | false :: _ => some 0
3034   | true :: gs => (cascade gs).map Nat.succ
3035
3036 def patterns8 : List (List Bool) :=
3037   (List.range 256).map fun b => (List.range 8).map fun j => b.testBit j
3038
3039 def spec (bits : List Bool) : Bool :=
3040   match cascade bits with
3041   | none => bits.all id
3042   | some k => (bits.take k).all id && (bits[k]? == some false)
3043
3044 theorem first_failure_terminal_8 :
3045   patterns8.all spec = true := by decide
3046
3047 theorem halt_free_iff_all_pass_8 :
3048   patterns8.all (fun bits => (cascade bits).isNone == bits.all id) = true := by decide
3049
3050 end PSP.CASCADE
3051
3052 namespace PSP.GROUNDLESS
3053
3054 def o0Neighbors : List String :=
3055   ["sigma-token", "dissolve-openness", "ordinary-openness",
3056    "resolved-states", "broken-or-ghost"]
3057
3058 def fiveCubeCoords : List String :=
3059   ["Ground-dimension", "determinacy", "blockage", "aperture", "totality"]
3060
3061 def deg5 (v : Nat) : Nat :=
3062   ((List.range 5).map fun j => v ^^ (1 <<< j)).eraseDups.length
3063
3064 theorem groundless_cascade_degree :
3065   o0Neighbors.length = 5 ∧ fiveCubeCoords.length = 5 ∧
3066   o0Neighbors.length ≠ 7 ∧ o0Neighbors.length ≠ 8 ∧
3067   (List.range 32).all (fun v => deg5 v == 5) = true := by decide
3068
3069 end PSP.GROUNDLESS
3070
3071 namespace PSP.TOKEN
3072
3073 inductive TerminalToken | x10 | o0 deriving DecidableEq
3074
3075 def level : TerminalToken → Nat := fun _ => 3
3076
3077 theorem terminal_tokens_level_and_distinct :
3078   level TerminalToken.x10 = level TerminalToken.o0 ∧
3079   TerminalToken.x10 ≠ TerminalToken.o0 := {rfl, by} decide
3080
3081 end PSP.TOKEN
3082
3083 namespace PSP.TWOGROUP
3084
3085 theorem two_group_census :
3086   Nat.factorial 4 / 2 = 12 ∧ 2^3 = 8 := by decide
3087
3088 end PSP.TWOGROUP
3089
3090 /-!
3091 # THE BRIDGE · FINAL
3092
3093 The instrument that joins the kinetic and formal registers at one locus, with the denial
3094   asymmetry that tells its two
3095 locks apart. core Lean 4 v4.19.0 · no Mathlib · no sorry · no custom axiom. The standard
3096   axioms propxet,
3097 Classical.choice and Quot.sound appear in the cones pinned at the foot. The closing
3098   environment audit, run by the
3099   elaborator, refuses any axiom declared in this file and any dependency beyond those
3100   three; 'import Lean' is loaded for
3101   that audit alone.
3102
3103 WHAT THE BRIDGE IS. A carrier with a three-valued state and a shadow field tying that
3104   state to the line property, so
3105   its halted state is the property in both directions. It carries a supplied bit from the
3106   register that holds it to the
3107   register that checks it, and it manufactures none: every self-property below is a one-
3108   step unfolding of the shadow.

```

```

3100
3101 THE STAGE. The fold  $\tau(h, t) = (2 - h, t)$  is  $s \mapsto 1 - s$  in the doubled real coordinate  $h = 2 \cdot \text{Re } s$ . It is an
3102 involution, and its fixed set is the line  $h = 1$ , which is  $\text{Re } s = \frac{1}{2}$ . A frame is a carrier
3103 set with an involutive fold
3104 and a fold-symmetric zero predicate. The line property says every zero is fixed by the
3105 fold; on the plane it says
3106 every zero lies on the line.
3107
3108 WHAT THE BRIDGE PROVES ABOUT ITSELF. It cannot lie, cannot deviate, cannot be extended
3109 past one bit, cannot be
3110 divided, cannot be reversed, and cannot be made to halt by any substrate that does not
3111 hold the term.
3112
3113 THE DENIAL ASYMMETRY. Two locks share the locus, the reading, and the recursion, and
3114 differ in the denial. A frame
3115 property whose denial instantiates it on every frame is keyless: it holds on every frame
3116 and opens on the deed. A
3117 frame property whose denial a frame inhabits is keyed: one bit, carrying a sign,
3118 supplied and never derived. Every
3119 frame property is exactly one of the two. The fold law is keyless and the line property
3120 is keyed. The recursion is
3121 shared and sign-blind, so it identifies neither lock; the denial identifies both.
3122
3123 -/
3124 set_option autoImplicit false
3125 namespace Bridge
3126
3127 -/! ## the stage and the locus -/
3128 abbrev Plane := Int  $\times$  Int
3129 -/ The fold  $s \mapsto 1 - s$ , in the doubled real coordinate  $h = 2 \cdot \text{Re } s$ . -/
3130 def  $\tau$  (p : Plane) : Plane := (2 - p.1, p.2)
3131 -/ The line  $h = 1$ , which is  $\text{Re } s = \frac{1}{2}$ . -/
3132 def onLine (p : Plane) : Prop := p.1 = 1
3133 theorem pe [a b c d : Int] : ((a, b) : Plane) = (c, d)  $\leftrightarrow$  a = c  $\wedge$  b = d :=
3134   (fun e => (congrArg Prod.fst e, congrArg Prod.snd) e, fun (e1, e2) => by subst e1;
3135     subst e2; rfl)
3136 -/ The fold is an involution. -/
3137 theorem fold_involutive (p : Plane) :  $\tau (\tau (p)) = p$  := by
3138   obtain (h, t) := p; show (2 - (2 - h), t) = (h, t); rw [Int.sub_sub_self]
3139 -/ THE LOCUS: the fixed set of the fold is the line. -/
3140 theorem locus_is_the_fixed_set (p : Plane) :  $\tau p = p \leftrightarrow$  onLine p := by
3141   obtain (h, t) := p; show (2 - h, t) = (h, t)  $\leftrightarrow$  h = 1; rw [pe]; omega
3142 theorem locus_inhabited :  $\exists p$  : Plane, onLine p := ((1, 0), )rfl
3143 -/ The fold carries the line to itself. -/
3144 theorem line_symmetric (p : Plane) (h : onLine p) : onLine  $\tau$  (p) := by
3145   rw [(locus_is_the_fixed_set p).mpr h]; exact h
3146
3147 -/! ## frames and the line property -/
3148 -/ A frame: a carrier set, a fold that is an involution, and a zero predicate the fold
3149 preserves. -/
3150 structure Frame where
3151   S : Type
3152    $\tau$  : S  $\rightarrow$  S
3153   Z : S  $\rightarrow$  Prop
3154   involutive :  $\forall s$ ,  $\tau (\tau (s)) = s$ 
3155   symmetric :  $\forall s$ , Z s  $\rightarrow$  Z  $\tau$  (s)
3156 -/ The Line property: every zero is fixed by the fold. -/
3157 def LineProperty (X : Frame) : Prop :=  $\forall s$ , X.Z s  $\rightarrow$  X. $\tau$ .s = s
3158 -/ The plane as a frame, for any fold-symmetric zero predicate. -/
3159 def planeFrame (Z : Plane  $\rightarrow$  Prop) (hZ :  $\forall p$ , Z p  $\rightarrow$  Z  $\tau$  (p)) : Frame := (Plane,  $\tau$ , Z,
3160   fold_involutive, hZ)
3161 -/ THE LOCUS MEETS THE FRAME: on the plane, the line property is exactly that every
3162 zero lies on the line. -/
3163 theorem plane_line_property (Z : Plane  $\rightarrow$  Prop) (hZ :  $\forall p$ , Z p  $\rightarrow$  Z  $\tau$  (p)) :
3164   LineProperty (planeFrame Z hZ)  $\leftrightarrow$   $\forall p$ , Z p  $\rightarrow$  onLine p :=
3165   (fun h p hp => (locus_is_the_fixed_set p).mpr (h p hp), fun h p hp => (
3166     locus_is_the_fixed_set p).mpr (h p hp))
3167 -/ The line frame: the plane with the line as its zero set. -/
3168 def lineFrame : Frame := planeFrame onLine line_symmetric
3169 inductive Tri | tt | ff | bot deriving DecidableEq, Repr
3170 -/ A filing label for the locus a carrier serves. 'held' is the trivial proposition, so
3171 the record carries no
3172 mathematical content, and no theorem reads it. -/
3173 structure Record where
3174   locus : String
3175   held : True
3176 def theRecord : Record := ("the Riemann Hypothesis", )trivial
3177
3178 -/! ## the bridge -/
3179 structure Carrier (X : Frame) where
3180   terminal : Tri
3181   shadow : terminal = .bot  $\rightarrow$  LineProperty X
3182   record : Record
3183 -/ CANNOT LIE. -/
3184 theorem cannot_lie (X : Frame) (b : Carrier X) (h : b.terminal = .bot) : LineProperty X
3185   := b.shadow.mpr h
3186 -/ CANNOT DEVIATE. -/
3187 theorem cannot_deviate (X : Frame) (b : Carrier X) (t : LineProperty X) : b.terminal = .
3188   bot := b.shadow.mpr t
3189 -/ THE LOCUS READ TWICE: the geometric read and the formal read are one read, and the
3190 two directions of the shadow
3191 are the two readings. -/
3192 theorem read_twice (X : Frame) (b : Carrier X) :
3193   (b.terminal = .bot  $\rightarrow$  LineProperty X)  $\wedge$  (LineProperty X  $\rightarrow$  b.terminal = .bot) :=
3194   (cannot_lie X b, cannot_deviate X b)
3195 -/ CANNOT BE MANUFACTURED: a halted carrier exists on a frame exactly when the line
3196 property holds there. -/
3197 theorem halted_iff (X : Frame) :  $\exists (b : Carrier X, b.terminal = .bot) \leftrightarrow$  LineProperty X
3198   :=
3199   (fun (b, h) => b.shadow.mpr h, fun t => ((.bot, (fun _ => t, fun _ => )rfl), theRecord,
3200     )rfl)
3201 -/ CANNOT BE EXTENDED: a bit generates only itself, its mirror, and the two constants.
3202 -/
3203 theorem cannot_extend (f : Bool  $\rightarrow$  Bool) :
3204   f = (fun x => x)  $\vee$  f = (fun x => !x)  $\vee$  f = (fun _ => true)  $\vee$  f = (fun _ => false) :=
3205   by
3206     cases ht : f true <;> cases hf : f false
3207     · exact Or.inr (Or.inr (Or.inr (funext fun x => by cases x <;> simp [ht, hf])))
3208     · exact Or.inr (Or.inl (funext fun x => by cases x <;> simp [ht, hf]))
3209     · exact Or.inl (funext fun x => by cases x <;> simp [ht, hf])
3210     · exact Or.inr (Or.inr (Or.inl (funext fun x => by cases x <;> simp [ht, hf])))
3211 -/ CANNOT BE DIVIDED. -/
3212 theorem cannot_divide (b : Bool) : b = true  $\vee$  b = false := by cases b <;> simp
3213 -/ CANNOT BE REVERSED: deletion to the undetermined value has no left inverse. -/
3214 def delete : Tri  $\rightarrow$  Tri := fun _ => .bot
3215 theorem cannot_reverse :  $\exists g$  : Tri  $\rightarrow$  Tri,  $\forall v$ , g (delete v) = v := by
3216   intro (g, )hg; have h1 := hg .tt; have h2 := hg .ff
3217   simp [delete] at h1 h2; rw [h1] at h2; cases h2
3218   /- THE WALL, the bridge's first axis: an even reading never equals a function odd at a
3219   point. -/
3220 def Even  $\alpha$  { : Type}  $\alpha$  ( :  $\alpha \rightarrow \alpha$ ) (f :  $\alpha \rightarrow$  Bool) : Prop :=  $\forall x$ , f  $\alpha$  (x) = f x
3221 theorem wall  $\alpha$  { : Type}  $\alpha$  ( :  $\alpha \rightarrow \alpha$ ) (f d :  $\alpha \rightarrow$  Bool) (x :  $\alpha$ ) (he : Even  $\alpha$  f) (ho : d  $\alpha$  (
3222   x)  $\neq$  d x) : f  $\neq$  d := by
3223   intro h; subst h; exact ho (he x)
3224 -/ THE CALIBRATION, the bridge's second axis: one supplied odd witness fixes the bit
3225 uniquely. -/
3226 theorem calibration  $\alpha$  { : Type}  $\alpha$  ( :  $\alpha \rightarrow \alpha$ ) (s d :  $\alpha \rightarrow$  Bool) (x :  $\alpha$ ) (hs : s  $\alpha$  (x) = !s x
3227   ) (hd : d  $\alpha$  (x) = !d x) :
3228    $\exists c$  : Bool, (d x = xor (s x) c)  $\wedge$  d  $\alpha$  (x) = xor (s  $\alpha$  (x)) c)  $\wedge$ 
3229    $\forall c'$ , (d x = xor (s x) c')  $\wedge$  d  $\alpha$  (x) = xor (s  $\alpha$  (x)) c')  $\rightarrow$  c' = c := by
3230   refine (xor (d x) (s x), (by cases s x <;> cases d x <;> rfl, by rw [hs, hd]; cases s
3231     x <;> cases d x <;> )rfl, ?) _
3232   intro c' hc; obtain (h1, ) := hc
3233   generalize hsx : s x = sv at h1; generalize hdx : d x = dv at h1
3234   cases sv <;> cases dv <;> cases c' <;> first | rfl | exact absurd h1 (by decide)
3235
3236 -/! ## the denial asymmetry - the discriminator -/
3237 def SelfVerifying (P : Prop) : Prop :=  $\neg P \rightarrow P$ 
3238 -/ A self-verifying proposition holds: its lock opens on the deed of denying it, and
3239 needs no key. -/
3240 theorem opens_on_the_deed (P : Prop) (utter : SelfVerifying P) : P :=
3241   Classical.byContradiction (fun n => n (utter n))
3242 -/ The recursion is shared by every proposition and is therefore no discriminator. -/
3243 theorem recursion_is_shared (P : Prop) : SelfVerifying P  $\leftrightarrow$  P :=
3244   (fun h => Classical.byContradiction (fun n => n (h n)), fun p _ => )p
3245 -/ And it is sign-blind: it holds for the negation under the mirror condition. -/
3246 theorem recursion_is_sign-blind (P : Prop) : (SelfVerifying P  $\leftrightarrow$  P)  $\wedge$  (SelfVerifying ( $\neg$ P)
3247    $\leftrightarrow$   $\neg$ P) :=
3248   (recursion_is_shared P, recursion_is_shared ( $\neg$ P))
3249 -/ Exactly one of a proposition and its negation carries the closed loop. -/
3250 theorem exactly_one_stands (P : Prop) :
3251   (SelfVerifying P  $\vee$  SelfVerifying ( $\neg$ P))  $\wedge$   $\neg$  (SelfVerifying P  $\wedge$  SelfVerifying ( $\neg$ P)) :=
3252   (Classical.byCases (fun p : P => Or.inl (fun _ => p)) (fun n => Or.inr (fun _ => n)),
3253     fun (a, )b => (recursion_is_shared ( $\neg$ P)).mpr b ((recursion_is_shared P).mpr a))
3254 -/ The two-point frame: an off-line pair under the fold. The fold is involutive and
3255 fixed-point-free, the zero set
3256 is fold-symmetric, and the line property fails: the denial of the line property as a
3257 coherent structure. -/
3258 def twoPoint : Frame := (Bool, (fun b => !b), fun _ => True, fun b => by cases b <;> rfl
3259   , fun _ => )trivial
3260 theorem denial_is_coherent :
3261    $\forall (b : \text{Bool}, \text{twoPointtr.} (b) = b) \wedge \forall (b : \text{Bool}, \text{twoPoint.Z } b \rightarrow \text{twoPoint.Z }
3262     (\text{twoPointtr. } b)) \wedge$ 
3263    $\forall (b : \text{Bool}, \text{twoPointtr. } b \neq b) \wedge \neg \text{LineProperty twoPoint} :=$ 
3264   (twoPoint.involutive, twoPoint.symmetric, fun b => by cases b <;> (intro h; cases h),
3265     fun h => by have : h true trivial; cases !this)
3266 -/ THE PROPERTY IS CONTINGENT: it holds on the line frame and fails on the two-point
3267 frame, so its value on a frame
3268 is fixed by the frame's zeros and never by the fold. -/
3269 theorem line_property_contingent : LineProperty lineFrame  $\wedge$   $\neg$  LineProperty twoPoint :=
3270   ((plane_line_property onLine line_symmetric).mpr (fun _ h => h), denial_is_coherent
3271     ).2.2.2
3272 -/ A frame property is keyless when its denial instantiates it on every frame. -/
3273 def Keyless (Q : Frame  $\rightarrow$  Prop) : Prop :=  $\forall X$ , SelfVerifying (Q X)
3274 -/ A frame property is keyed when a frame inhabits its denial. -/
3275 def Keyed (Q : Frame  $\rightarrow$  Prop) : Prop :=  $\exists X$ ,  $\neg$  Q X
3276 -/ A keyless lock is a property that holds on every frame: the deed of denying it hands
3277 it over. -/
3278 theorem keyless_iff_valid (Q : Frame  $\rightarrow$  Prop) : Keyless Q  $\leftrightarrow$   $\forall X$ , Q X :=
3279   (fun h X => opens_on_the_deed (Q X) (h X), fun h X _ => h X)
3280 -/ THE DISCRIMINATOR, a decision on the denial: every frame property is keyless or
3281 keyed, and never both. The
3282 property is bound in both halves; the pointwise recursion, shared and sign-blind,
3283 plays no part in the sort. -/
3284 theorem discriminator (Q : Frame  $\rightarrow$  Prop) : (Keyless Q  $\vee$  Keyed Q)  $\wedge$   $\neg$  (Keyless Q  $\wedge$  Keyed
3285   Q) :=
3286   (Classical.byCases (fun h :  $\forall X$ , Q X => Or.inl ((keyless_iff_valid Q).mpr h))
3287     (fun h => Or.inr (Classical.byContradiction fun hn =>
3288       h fun X => Classical.byContradiction fun hq => hn (Exists.intro X hq))),
3289     fun h => Exists.elim h.2 fun X hX => hX ((keyless_iff_valid Q).mpr h.1 X))
3290 -/ The fold law is keyless: every frame carries it by its own law. -/
3291 theorem symmetry_is_keyless : Keyless (fun X =>  $\forall s$ , X. $\tau$ .s = s) := fun X _ => X.
3292   involutive
3293 -/ The line property is keyed: the two-point frame inhabits its denial. -/
3294 theorem line_property_is_keyed : Keyed LineProperty  $\wedge$   $\neg$  Keyless LineProperty :=
3295   (Exists.intro twoPoint denial_is_coherent.2.2.2,
3296     fun h => denial_is_coherent.2.2.2 ((keyless_iff_valid LineProperty).mpr h twoPoint))
3297 -/ THE ASYMMETRY. A self-verifying proposition needs no key: its denial hands it over.
3298 The line property has a key:
3299 its denial is a structure that obeys both frame laws and on which every theorem of
3300 this file holds, so no deed of
3301 reading supplies the sign, and no carrier halts on every frame. -/
3302 theorem denial_asymmetry :
3303    $\forall (P : \text{Prop}, \text{SelfVerifying } P \rightarrow P) \wedge$ 
3304    $\forall (P : \text{Prop}, \text{SelfVerifying } P \leftrightarrow P) \wedge$  (SelfVerifying ( $\neg$ P)  $\leftrightarrow$   $\neg$ P))  $\wedge$ 
3305    $\forall ((b : \text{Bool}, \text{twoPointtr.} (b) = b) \wedge \forall (b : \text{Bool}, \text{twoPoint.Z } b \rightarrow \text{twoPoint.Z }
3306     (\text{twoPointtr. } b)) \wedge$ 
3307      $\forall (b : \text{Bool}, \text{twoPointtr. } b \neq b) \wedge \neg \text{LineProperty twoPoint}) \wedge$ 
3308     ( $\neg \forall X$  : Frame,  $\exists b$  : Carrier X, b.terminal = .bot)  $\wedge$ 
3309     ( $\neg \forall X$  : Frame, LineProperty X) :=
3310   (opens_on_the_deed, recursion_is_sign-blind, denial_is_coherent,
3311     fun h => denial_is_coherent.2.2.2 ((halted_iff twoPoint).mpr (h twoPoint))),

```

```

3268 fun h => denial_is_coherent.2.2.2 (h twoPoint))
3269
3270 /-! ## the socket, and what the bridge carries -/
3271 class Supplied (X : Frame) where
3272   term : LineProperty X
3273 theorem socket_is_the_property (X : Frame) : Nonempty (Supplied X) ↔ LineProperty X :=
3274   (fun () => i.term, fun t => (()))t
3275 /- THE SOCKET ON THE PLANE: an instance is exactly the statement that every zero lies
   on the line. -/
3276 theorem socket_on_the_plane (Z : Plane → Prop) (hZ : ∀ p, Z p → Z τ(p)) :
3277   Nonempty (Supplied (planeFrame Z hZ)) → ∀ p, Z p → onLine p :=
3278   (socket_is_the_property (planeFrame Z hZ)).trans (plane_line_property Z hZ)
3279 theorem bridge_carries (X : Frame) [i : Supplied X] : LineProperty X ↔ ∃ b : Carrier X,
   b.terminal = .bot :=
3280   (i.term, (halted_iff X).mpr i).term
3281 theorem no_socket_off_line : ¬ Nonempty (Supplied twoPoint) :=
3282   fun h => denial_is_coherent.2.2.2 ((socket_is_the_property twoPoint).mp h)
3283
3284 /-! ## the bridge, whole -/
3285 theorem the_bridge :
3286   ∀ (p : Plane, τ τ(p) = p) ∧ ∀ (p : Plane, τ p = p → onLine p) ∧ ∃ (p : Plane,
   onLine p) ∧
3287   ∀ (Z : Plane → Prop) (hZ : ∀ p, Z p → Z τ(p)), LineProperty (planeFrame Z hZ) → ∀
   p, Z p → onLine p) ∧
3288   ∀ (X : Frame) (b : Carrier X), (b.terminal = .bot → LineProperty X) ∧ (LineProperty
   X → b.terminal = .bot)) ∧
3289   ∀ (X : Frame, ∃ (b : Carrier X, b.terminal = .bot) → LineProperty X) ∧
3290   ∀ (f : Bool → Bool, f = (fun x => x) ∨ f = (fun x => !x) ∨ f = (fun _ => true) ∨ f =
   (fun _ => false)) ∧
3291   ∀ (b : Bool, b = true ∨ b = false) ∧
3292   (¬ ∃ g : Tri → Tri, ∀ v, g (delete v) = v) ∧
3293   ∀ (α : Type) o (α : α → α) (f d : α → Bool) (x : α), Even o f → d o (x) ≠ d x → f ≠ d
   ) ∧
3294   ∀ (α : Type) o (α : α → α) (s d : α → Bool) (x : α), s o (x) = !s x → d o (x) = !d x
   →
3295     ∃ c : Bool, (d x = xor (s x) c) ∧ d o (x) = xor (s o (x)) c) ∧
3296     ∀ c', (d x = xor (s x) c') ∧ d o (x) = xor (s o (x)) c') → c' = c) ∧
3297   ∀ (X : Frame, Nonempty (Supplied X) → LineProperty X) ∧
3298   ∀ (Z : Plane → Prop) (hZ : ∀ p, Z p → Z τ(p)), Nonempty (Supplied (planeFrame Z hZ
   )) → ∀ p, Z p → onLine p) ∧
3299   ∀ (X : Frame) [Supplied X], LineProperty X ↔ ∃ b : Carrier X, b.terminal = .bot) ∧
3300   (¬ Nonempty (Supplied twoPoint)) ∧
3301   (LineProperty lineFrame → LineProperty twoPoint) ∧
3302   ∀ (P : Prop, (SelfVerifying P ∨ SelfVerifying (¬P)) → (¬ (SelfVerifying P ∧
   SelfVerifying (¬P)))) ∧
3303   ∀ (Q : Frame → Prop, (Keyless Q ∨ Keyed Q) → (¬ (Keyless Q ∧ Keyed Q))) ∧
3304   Keyless (fun X => V s, Xt. (Xt. s) = s) ∧
3305   (Keyed LineProperty → ¬ Keyless LineProperty) ∧
3306   ∀ (P : Prop, SelfVerifying P → P) ∧
3307   ∀ (P : Prop, (SelfVerifying P → P) → (SelfVerifying (¬P) → ¬P)) ∧
3308   ∀ (b : Bool, twoPoint. b ≠ b) ∧ (twoPoint. b) = b) ∧ ∀ (b : Bool, twoPoint. Z b → twoPoint
   . Z (twoPoint. b)) ∧
3309   ∀ (b : Bool, twoPoint. b ≠ b) ∧ (LineProperty twoPoint) ∧
3310   (¬ ∀ X : Frame, ∃ b : Carrier X, b.terminal = .bot) ∧ (¬ ∀ X : Frame, LineProperty
   X)) :=
3311   (fold_involutive, locus_is_the_fixed_set, locus_inhabited, plane_line_property,
   read_twice, halted_iff,
3312   cannot_extend, cannot_divide, cannot_reverse, fun o f d x h e => wall o f d x h e ho
   ,
3313   fun o s d x h s hd => calibration o s d x h s hd, socket_is_the_property,
   socket_on_the_plane, @bridge_carries,
3314   no_socket_off_line, line_property_contingent, exactly_one_stands, discriminator,
   symmetry_is_keyless,
3315   line_property_is_keyed, !denial_asymmetry
3316 end Bridge
3317 /-!
3318 # FORGE ADDITIONS · v3.39.0-C
3319 Seated after the Bridge and before the cones. ΔM = 0: every theorem below is a finite `
   decide`.
3320 a conjunction of theorems already seated, or an elementary case split. Nothing here
   authors
3321 mathematics; it names in the kernel what the prose of the Condensed Master Codex reads.
3322 -/
3323
3324 namespace K4
3325
3326 /- PSP: K4 · trial-division primality on the frame values, executed in the kernel. -/
3327 def isPrimeB (n : Nat) : Bool :=
3328   decide (2 ≤ n) && (List.range (n - 2)).all (fun d => n % (d + 2) != 0)
3329
3330 /- PSP: APEX-PSP-INVERSION · the six even seats carry primes. -/
3331 theorem primes_at_even_seats :
3332   (isPrimeB (val 0) && isPrimeB (val 2) && isPrimeB (val 4) && isPrimeB (val 6)
   && isPrimeB (val 8) && isPrimeB (val 10)) = true := by decide
3333
3334 /- PSP: APEX-PSP-INVERSION · the six odd seats carry semiprimes, factored in the kernel
   . -/
3335 theorem semiprimes_at_odd_seats :
3336   val 1 = 23 * 37 ∧ val 3 = 19 * 67 ∧ val 5 = 19 * 23 ∧ val 7 = 13 * 163
3337   ∧ val 9 = 13 * 131 ∧ val 11 = 11 * 79 := by decide
3338
3339 theorem semiprime_factors_prime :
3340   (isPrimeB 23 && isPrimeB 37 && isPrimeB 19 && isPrimeB 67 && isPrimeB 13
   && isPrimeB 163 && isPrimeB 131 && isPrimeB 11 && isPrimeB 79) = true := by decide
3341
3342 theorem odd_seats_not_prime :
3343   (!isPrimeB (val 1) && !isPrimeB (val 3) && !isPrimeB (val 5) && !isPrimeB (val 7)
   && !isPrimeB (val 9) && !isPrimeB (val 11)) = true := by decide
3344
3345 /- The target of the frame is primality itself and not an index label: read in the
   kernel. -/
3346 theorem target_is_primality : ∀ i, i < 12 → target i = isPrimeB (val i) := by decide
3347
3348 /- Each pair matches modulo 420 = lcm(1..7), so every residue register up to 7 forgets
   it. -/
3349 theorem pairs_match_mod_420 : ∀ i, i < 12 → i % 2 = 1 → val i % 420 = val (tau i) % 420

```

```

:= by
3353 decide
3354
3355 end K4
3356
3357 namespace IMPRINT
3358
3359 /- PSP: §3.9 · the Imprint Test and the Platonic Ghost, the GOLf gate, ported from F2
   imprint_seal. -/
3360 inductive Tok : Type
3361 | ghost | seal | residence | flat | uncertified
3362 deriving DecidableEq, Repr, BEq
3363
3364 def imprint (lockP lockN sLP sLN gP gN witP witN : Bool) : Tok × Nat :=
3365   let cleanP := lockP && sLP && gP
3366   let cleanN := lockN && sLN && gN
3367   if cleanP && cleanN then (.ghost, 1)
3368   else if cleanP then (if witP then (.seal, 2) else (.residence, 3))
3369   else if cleanN then (if witN then (.seal, 2) else (.residence, 3))
3370   else if !lockP && !lockN then (.flat, 4)
3371   else (.uncertified, 5)
3372
3373 def Tok.toToken : Tok → Audit.Token
3374 | .ghost => .broken
3375 | .seal => .sealed
3376 | _ => .open
3377
3378 /- Both directions clean-locking is the Platonic Ghost: refused, never sealed. -/
3379 theorem ghost_refused : (imprint true true true true true true true true).1 = .ghost :=
   by decide
3380
3381 theorem both_clean_is_ghost : ∀ wp wn : Bool,
3382   (imprint true true true true true true wp wn).1 = .ghost := by decide
3383
3384 /- One side clean-locks: a seal only with the witness supplied, a residence without it.
   -/
3385 theorem seal_needs_witness :
3386   (imprint true false true false true false true false).1 = .seal
3387   ∧ (imprint true false true false true false false).1 = .residence := by decide
3388
3389 /- The whole 256-row truth table of the gate: a seal is exactly one clean side plus its
   witness. -/
3390 theorem seal_iff_one_side_clean_and_witnessed :
3391   ∀ LP LN sP sN gP gN wP wN : Bool,
3392   ((imprint LP LN sP sN gP gN wP wN).1 = .seal) ↔
3393   ((LP && sP && gP && wP && !(LN && sN && gN))
3394    || (LN && sN && gN && wN && !(LP && sP && gP))) = true := by
3395     decide
3396
3397 theorem flat_when_unpopulated : ∀ sP sN gP gN wP wN : Bool,
3398   (imprint false false sP sN gP gN wP wN).1 = .flat := by decide
3399
3400 theorem ghost_is_broken_never_sealed :
3401   Tok.toToken .ghost = .broken ∧ Tok.toToken .seal = .sealed := (rfl, )rfl
3402
3403 end IMPRINT
3404
3405 namespace DEFENSE
3406
3407 /- PSP: D5 · GOL is the combination: magnitude alone is [?], direction alone is [?]. -/
3408 theorem gol_needs_all_three :
3409   (GEO.golAdmit .lock .lock).1 = .golOK
3410   ∧ ∀ (l : GEO.LockState, l ≠ .lock → (GEO.golAdmit .lock l).1 = .golQ)
3411   ∧ ∀ (m : GEO.LockState, m ≠ .lock → (GEO.golAdmit m .lock).1 = .golOK) := by
3412     refine (rfl, ?, ?)
3413     · intro l hl; cases l <> first | exact absurd rfl hl | rfl
3414     · intro m hm; cases m <> first | exact absurd rfl hm | decide
3415
3416 /- PSP: D1, D2 · the lock scalar is sign-blind by theorem, and one supplied bit fixes
   the sign uniquely. -/
3417 theorem lock_blind_but_gol_directed :
3418   (PSP.ORIENT.signMatrices.all (fun m =>
3419     PSP.ORIENT.det3 m * PSP.ORIENT.det3 m
3420     == PSP.ORIENT.det3 (PSP.ORIENT.neg3 m) * PSP.ORIENT.det3 (PSP.ORIENT.neg3 m)) =
   true)
3421   ∧ ∀ (α : Type) τ (α : α → α) (s d : α → Bool) (x : α),
3422     s τ (x) = !s x → d τ (x) = !d x →
3423     ∃ c : Bool, (d x = xor (s x) c) ∧ d τ (x) = xor (s τ (x)) c) ∧
3424     ∀ c' : Bool, (d x = xor (s x) c') ∧ d τ (x) = xor (s τ (x)) c') → c' = c) :=
3425     (PSP.ORIENT.lock_scalar_sign_blind, fun τ s d x h s hd => FTOE.T5_crossing τ s d x h s
   )hd
3426
3427 /- PSP: D11, D12 · consensus carries zero weight; the engine authors nothing. -/
3428 theorem consensus_weights_zero : socialWeight = 0 ∧ deltaM = 0 := FOUND.
   engine_never_authority
3429
3430 /- PSP: sPSP-TONGUE-UNCLOSED-01 · the Tongue is unclosed by theorem and obedient by
   theorem. -/
3431 theorem tongue_unclosed_and_obedient :
3432   ∀ (α β : Type) τ (α : α → α) p (α : α → β) (g : β → Bool) (d : α → Bool) (x : α),
3433     p τ (x) = p x → d τ (x) = !d x → d ≠ fun y => g p (y))
3434     ∧ (GEO.det3 TONGUE.deletionOp = 0
3435     ∧ ∀ (L : GEO.M3, (GEO.mv3 L) ∘ (GEO.mv3 TONGUE.deletionOp) ≠ id)
3436     ∧ ¬ ∀ (M : GEO.M3, ∃ L : GEO.M3, (GEO.mv3 L) ∘ (GEO.mv3 M) = id)) :=
3437     (fun τ p g d x ph hd => TONGUE.unclosed τ p g d x ph hd, TONGUE.tongue_obedience
3438
3439 /- PSP: D27 · a self-check is never a witness (M6); the token is withheld unwitnessed.
   -/
3440 theorem self_check_is_not_witness :
3441   (IAM.iamToken false).1 = .interior ∧ (IAM.iamToken true).1 = .iAM := (rfl, )rfl
3442
3443 /- PSP: D71 · the denial pays the floor; a denial that registers nothing is no denial.
   -/
3444 theorem denial_is_priced :
3445   0 < (OMEGA.omegaBoundary 1 300).joules ∧ (OMEGA.omegaBoundary 0 300).joules = 0 :=
   by decide

```

```

3446
3447 /-- PSP: D24 · no candidate anchor that lands on its own target is admitted. -/
3448 theorem no_anchor_at_target : ∀ (d : Nat) (b lc : Bool),
3449   UC.circularityScreen .target d b lc.1 = .refusedCircular :=
3450   UC.circularity_refused_at_target
3451
3452 /-- PSP: D54 · every formal reading is even under the deed flip; the deed is odd; no
3453   readout returns it. -/
3454 theorem formal_reads_never_the_deed (Q : Type) (q : Q) :
3455   ~ ∃ g : Q → Bool, ∀ s : CROSSING.ExecFrame Q, g (CROSSING.formalRead s) = CROSSING.
3456     ran s :=
3457     CROSSING.wall q
3458
3459 end DEFENSE
3460
3461 namespace FENCE
3462
3463 /-- PSP: II.0 · the one global fence: every card is read at its grade, on its register,
3464   at its species;
3465 theorem global :
3466   ∀ (c c' : Claim, (Claim.join c c').grade = Grade.weakest c.c'.grade c.c'.grade)
3467   ∧ ∀ (c cite : Claim, (Claim.join c cite).grade.rank ≤ c.cite.grade.rank
3468     ∧ (Claim.join c cite).grade.rank ≤ cite.grade.rank)
3469   ∧ ROOT.raClaim.grade = .premise
3470   ∧ K4.wallClaim.grade = .theorem
3471   ∧ terminalClaim.grade = .theorem
3472   ∧ HALT.haltGrades.all (fun g => g.rank < Grade.theorem.rank) = true
3473   ∧ HALT.rhHalt.species = .hypothesis ∧ HALT.rhHalt.channel = .formal ∧ HALT.rhHalt.
3474     gdim = 1
3475   ∧ LOCUS.emit (false, true, some 1, some .hypothesis, some .formal, [none], by
3476     )decide
3477     = some (.superHalt (some .xi) (some .hypothesis) (some .formal) 1)
3478   ∧ (FOUND.bandOf .theological = .outOfBand ∧ FOUND.bandOf .social = .outOfBand)
3479   ∧ (t : LOCUS.Token, t.toEconomy = .sealed v t.toEconomy = .broken v t.toEconomy =
3480     .opn) :=
3481   (Claim.join_grade, OFFICE.citation_cannot_promote, rfl, rfl, rfl, HALT.ftoe_ceiling,
3482     rfl, rfl, rfl, LOCUS.no_rh_locus, (rfl, rfl, LOCUS.)three_states
3483
3484 end FENCE
3485
3486 namespace DOT
3487
3488 /-- PSP: CN-PSP-NETI-NETI-01 · the mark of silence is the image of no emitter. -/
3489 theorem outside_economy : ∀ t : LOCUS.Token, t.toEconomy ≠ Audit.Token.dotmark := by
3490   intro t; cases t <=> first | decide | simp [LOCUS.Token.toEconomy]
3491
3492 theorem no_reading_is_silence : ∀ r : LOCUS.Reading, r.state ≠ Audit.Token.dotmark := by
3493   intro r
3494   unfold LOCUS.Reading.state
3495   split
3496   · exact outside_economy _
3497   · decide
3498
3499 theorem silence_is_not_a_verdict :
3500   Audit.Token.dotmark ≠ .sealed ∧ Audit.Token.dotmark ≠ .broken
3501   ∧ Audit.Token.dotmark ≠ .opn := by decide
3502
3503 end DOT
3504
3505 namespace DEFENSE
3506
3507 /-- PSP: D53 · a forward, dated reading is world-rowed unconditionally: R-3 fires before
3508   any row is read. -/
3509 theorem forward_is_world_rowed : ∀ p w t w f a : Bool,
3510   (GE0.rowCascadeRACore true false true p w t w f a).1 = .iii := by decide
3511
3512 /-- PSP: D50 · a load-bearing row furnished by the world types the lock world-rowed,
3513   revisable in the rows. -/
3514 theorem worldly_row_is_world_rowed : ∀ t w f a : Bool,
3515   (GE0.rowCascadeRACore true false false true t w f a).1 = .iii := by decide
3516
3517 end DEFENSE
3518
3519 namespace CROSSWALK
3520
3521 /-- PSP: IV.1 · the crosswalk, seated in the kernel: every row names a constant that
3522   must exist. -/
3523 def rows : List (String × String) := [
3524   ("APEX-PSP-FREEDOM-MASTER-01", "FTOE.T16_price_bijection"),
3525   ("sPSP-APERTURE-WIDTH-01", "FTOE.T5_crossing"),
3526   ("PSP-IAM-WRITTEN-READ-01", "IAM.narcissus_truth_table"),
3527   ("APEX-PSP-E-8-GATE-CASCADE-01", "HALT.first_failure_terminal_8"),
3528   ("APEX-PSP-RH-MASTER-01", "HALT.rhHalt"),
3529   ("APEX-PSP-RH-BLOCK-COMplete-01", "LOCUS.no_rh_locus"),
3530   ("APEX-PSP-RH-STANDPOINT-CLOSURE-01", "Bridge.line_property_is_keyed"),
3531   ("APEX-PSP-RH-DENIAL-POSIT-01", "Bridge.no_socket_off_line"),
3532   ("APEX-PSP-TOKEN-EQUALITY-01", "PSP.TOKEN.terminal_tokens_level_and_distinct"),
3533   ("sPSP-RETURN-LOCUS-01", "UC.circ_space"),
3534   ("APEX-PSP-ABSOLUTE-GROUNDING-BLOCK-01", "Bridge.discriminator"),
3535   ("sPSP-ANCHOR-DEMAND-01", "UC.circularity_refused_at_target"),
3536   ("APEX-PSP-ANCHOR-PRECEDENCE-01", "Bridge.opens_on_the_deed"),
3537   ("APEX-PSP-SUPPLY-SPECIES-01", "HALT.species_exhausted"),
3538   ("sPSP-TRIAD-SOURCE-BLOCK-01", "FTOE.T2_coalition"),
3539   ("sPSP-UNRESTRICTED-IMPOSSIBLE-01", "HALT.ftoe_ceiling"),
3540   ("APEX-PSP-LOGOS-FIXATION-01", "TONGUE.tongue_obedience"),
3541   ("MD-PSP-NG-EXHIBIT-ANCHOR-01", "INVERSION.it_from_bit_refuted"),
3542   ("MD-PSP-CLOSURE-POSITION-01", "FTOE.T3_no_fixed_point"),
3543   ("sPSP-CONSTITUTIVE-INSEPARABLE-01", "Audit.compartmentGates"),
3544   ("sPSP-FOLD-BEND-CROSSING-01", "CROSSING.crossing"),
3545   ("APEX-PSP-NOMOS-MEASURE-01", "NOMOS.incompressibility_cap"),
3546   ("MD-PSP-NAVIER-DEFEATERLESS-01", "GE0.rowCascadeRACore"),
3547   ("APEX-PSP-BLESSED-CIRCLE-01", "DEFENSE.no_anchor_at_target"),
3548   ("RAF-PSP-HODGE-TERMINUS-01", "HALT.route"),
3549   ("MD-PSP-GODEL-MASTER-01", "HALT.route_is_routeHalt"),
3550   ("APEX-PSP-PNP-COMPOSITE-VERDICT-02", "HALT.pnpHalt"),

```

```

3543   ("APEX-PSP-XI0-VERDICT-01", "HALT.xiGates"),
3544   ("MD-PSP-NG-MASTER-01", "HALT.one_door_open"),
3545   ("sPSP-TONGUE-UNCLOSED-01", "DEFENSE.tongue_unclosed_and_obedient"),
3546   ("APEX-PSP-CENSOR-CLERK-01", "OFFICE.citation_cannot_promote"),
3547   ("MD-PSP-MATH-NATURE-01", "TONGUE.coloc_grade"),
3548   ("CN-PSP-GHAYB-MASTER-01", "DOT.outside_economy"),
3549   ("CN-PSP-NETI-NETI-01", "DOT.no_reading_is_silence"),
3550   ("APEX-PSP-ONLY-SPECIAL-01", "IAM.terminal_word"),
3551   ("APEX-PSP-UNFORGIVABLE-01", "AEGIS.aegis_constant"),
3552   ("MD-PSP-STILLNESS-RESERVOIR-01", "DOT.silence_is_not_a_verdict"),
3553   ("PSP-RA-SERVANT-LOCUS-01", "IAM.iam_witnessed_speaks"),
3554   ("APEX-PSP-RH-TERMINAL-SYNTHESIS-01", "FENCE.global"),
3555   ("sPSP-GOL-ROW-GENUS-01", "GE0.ra_battery_f2"),
3556   ("MD-PSP-AFFINITY-01", "FTOE.T9_torsor_inverse_left"),
3557   ("APEX-PSP-INVERSION-01", "K4.target_is_primality"),
3558   ("MD-PSP-K4-FRAME-01", "K4.pairs_match_mod_420"),
3559   ("APEX-PSP-RA-MASTER-01", "ROOT.actuation_universal"),
3560   ("FOUND-01", "TONGUE.ra_grade_pinned"),
3561   ("IMPRINT-GOLF-01", "IMPRINT.seal_iff_one_side_clean_and_witnessed"),
3562   ("D1", "PSP.ORIENT.lock_scalar_sign_blind"),
3563   ("D2", "DEFENSE.lock_blind_but_gol_directed"),
3564   ("D3", "FTOE.T5_crossing"),
3565   ("D4", "FENCE.global"),
3566   ("D5", "DEFENSE.gol_needs_all_three"),
3567   ("D6", "GE0.triaxial_lock_gf2"),
3568   ("D6", "PSP.CD.sedenion_zero_divisor"),
3569   ("D7", "GE0.a4_simply_transitive"),
3570   ("D7", "GE0.gates_complete"),
3571   ("D8", "GE0.chirality"),
3572   ("D8", "GE0.relabel_parity"),
3573   ("D9", "GE0.reflection_blind_executed"),
3574   ("D10", "TONGUE.tongue_obedience"),
3575   ("D11", "DEFENSE.consensus_weighs_zero"),
3576   ("D11", "Grade.weakest_is_a_link"),
3577   ("D12", "OFFICE.citation_cannot_promote"),
3578   ("D13", "GE0.ra_ledger_aGivenRA"),
3579   ("D14", "HALT.ftoe_ceiling"),
3580   ("D15", "Claim.join_grade"),
3581   ("D16", "AEGIS.aegis_battery"),
3582   ("D17", "HALT.route_is_routeHalt"),
3583   ("D17", "PSP.CASCADE.halt_free_iff_all_pass_8"),
3584   ("D18", "HALT.first_failure_terminal_8"),
3585   ("D19", "OMEGA.omega_paid_floor"),
3586   ("D20", "CROSSING.price"),
3587   ("D21", "IAM.iam_unwitnessed_withheld"),
3588   ("D22", "AEGIS.aegis_constant"),
3589   ("D23", "Audit.unknown_routes_open"),
3590   ("D24", "DEFENSE.no_anchor_at_target"),
3591   ("D25", "OMEGA.omega_monotone"),
3592   ("D26", "TONGUE.deletion_no_left_inverse"),
3593   ("D27", "DEFENSE.self_check_is_not_witness"),
3594   ("D28", "LOCUS.three_states"),
3595   ("D29", "GE0.triaxial_open_gf2_count"),
3596   ("D30", "GE0.triaxial_lock_gf2_count"),
3597   ("D31", "GE0.cramer_executed"),
3598   ("D32", "OMEGA.omega_linear"),
3599   ("D33", "HALT.gate_counts_recorded"),
3600   ("D34", "NOMOS.compressible_bound_universal"),
3601   ("D35", "ROOT.actuation_universal"),
3602   ("D36", "OMEGA.omega_zero_bits"),
3603   ("D37", "IMPRINT.ghost_refused"),
3604   ("D38", "FOUND.pluralist_alternative_executed"),
3605   ("D39", "SEALS.sealG_walk"),
3606   ("D40", "GE0.hurwitz_closed"),
3607   ("D41", "CROSSING.ran_wholly_odd"),
3608   ("D42", "FTOE.T3_no_fixed_point"),
3609   ("D43", "TONGUE.deletion_annihilates"),
3610   ("D44", "NOMOS.incompressibility_cap"),
3611   ("D45", "DOT.outside_economy"),
3612   ("D46", "DOT.silence_is_not_a_verdict"),
3613   ("D47", "IAM.narcissus_truth_table"),
3614   ("D48", "FTOE.T16_price_bijection"),
3615   ("D49", "HALT.halts_routed"),
3616   ("D50", "DEFENSE.worldly_row_is_world_rowed"),
3617   ("D51", "PSP.QUAT.quat_basis_laws"),
3618   ("D52", "INVERSION.it_from_bit_refuted"),
3619   ("D52", "K4.target_is_primality"),
3620   ("D53", "DEFENSE.forward_is_world_rowed"),
3621   ("D54", "DEFENSE.formal_reads_never_the_deed"),
3622   ("D55", "AEGIS.aegis_battery"),
3623   ("D56", "ROOT.nest_31"),
3624   ("D57", "Bridge.discriminator"),
3625   ("D57", "Bridge.keyless_iff_valid"),
3626   ("D57", "TONGUE.ra_grade_pinned"),
3627   ("D58", "Bridge.recursion_is_sign_blind"),
3628   ("D59", "Bridge.line_property_is_keyed"),
3629   ("D60", "Bridge.recursion_is_shared"),
3630   ("D61", "Bridge.symmetry_is_keyless"),
3631   ("D62", "Bridge.socket_is_the_property"),
3632   ("D63", "HALT.rhHalt"),
3633   ("D63", "LOCUS.no_rh_locus"),
3634   ("D64", "IMPRINT.seal_needs_witness"),
3635   ("D64", "Bridge.no_socket_off_line"),
3636   ("D65", "TONGUE.coloc_grade"),
3637   ("D66", "FOUND.posits_loadbearing_on_nothing"),
3638   ("D67", "FOUND.register_routing_fidelity"),
3639   ("D68", "BRIDGE.supersession_routes"),
3640   ("D69", "IMPRINT.both_clean_is_ghost"),
3641   ("D69", "LOCUS.located_never_silent"),
3642   ("D70", "UC.uc_seal_failure_terminal"),
3643   ("D71", "DEFENSE.denial_is_priced"),
3644   ("D71", "Bridge.opens_on_the_deed")]

```

```

3645 theorem rows_count : rows.length = 129 := by decide
3646
3647 end CROSSWALK
3648

```

```

3649
3650
3651 /-- info: 'Codex.LeftInverse' does not depend on any axioms -/
3652 #guard_msgs in #print axioms Codex.LeftInverse
3653 /-- info: 'Codex.RightInverse' does not depend on any axioms -/
3654 #guard_msgs in #print axioms Codex.RightInverse
3655 /-- info: 'Nat.factorial' does not depend on any axioms -/
3656 #guard_msgs in #print axioms Nat.factorial
3657 /-- info: 'CoreRat.div_pos_of_le' depends on axioms: [propext] -/
3658 #guard_msgs in #print axioms CoreRat.div_pos_of_le
3659 /-- info: 'CoreRat.div_gcd_pos' depends on axioms: [propext] -/
3660 #guard_msgs in #print axioms CoreRat.div_gcd_pos
3661 /-- info: 'CoreRat.coprime_div_gcd' depends on axioms: [propext] -/
3662 #guard_msgs in #print axioms CoreRat.coprime_div_gcd
3663 /-- info: 'Ratio.natAbs_signed' depends on axioms: [propext] -/
3664 #guard_msgs in #print axioms Ratio.natAbs_signed
3665 /-- info: 'Ratio.mk' depends on axioms: [propext] -/
3666 #guard_msgs in #print axioms Ratio.mk
3667 /-- info: 'Ratio.instOfNatRat' depends on axioms: [propext] -/
3668 #guard_msgs in #print axioms Ratio.instOfNatRat
3669 /-- info: 'Ratio.instAddRat' depends on axioms: [propext] -/
3670 #guard_msgs in #print axioms Ratio.instAddRat
3671 /-- info: 'Ratio.instMulRat' depends on axioms: [propext] -/
3672 #guard_msgs in #print axioms Ratio.instMulRat
3673 /-- info: 'Ratio.instNegRat' depends on axioms: [propext] -/
3674 #guard_msgs in #print axioms Ratio.instNegRat
3675 /-- info: 'Ratio.instSubRat' depends on axioms: [propext] -/
3676 #guard_msgs in #print axioms Ratio.instSubRat
3677 /-- info: 'Ratio.inv' depends on axioms: [propext] -/
3678 #guard_msgs in #print axioms Ratio.inv
3679 /-- info: 'Ratio.instDivRat' depends on axioms: [propext] -/
3680 #guard_msgs in #print axioms Ratio.instDivRat
3681 /-- info: 'Ratio.instLRat' depends on axioms: [propext] -/
3682 #guard_msgs in #print axioms Ratio.instLRat
3683 /-- info: 'Ratio.instLERat' depends on axioms: [propext] -/
3684 #guard_msgs in #print axioms Ratio.instLERat
3685 /-- info: 'Ratio.instDecidableLERat' depends on axioms: [propext] -/
3686 #guard_msgs in #print axioms Ratio.instDecidableLERat
3687 /-- info: 'Ratio.instDecidableLELeRat' depends on axioms: [propext] -/
3688 #guard_msgs in #print axioms Ratio.instDecidableLELeRat
3689 /-- info: 'Ratio.pow' depends on axioms: [propext] -/
3690 #guard_msgs in #print axioms Ratio.pow
3691 /-- info: 'Ratio.instHPowRatNat' depends on axioms: [propext] -/
3692 #guard_msgs in #print axioms Ratio.instHPowRatNat
3693 /-- info: 'Ratio.instToStringRat' depends on axioms: [propext] -/
3694 #guard_msgs in #print axioms Ratio.instToStringRat
3695 /-- info: 'Ratio.num_zero' depends on axioms: [propext] -/
3696 #guard_msgs in #print axioms Ratio.num_zero
3697 /-- info: 'Ratio.den_zero' depends on axioms: [propext] -/
3698 #guard_msgs in #print axioms Ratio.den_zero
3699 /-- info: 'Ratio.mk' zero_num depends on axioms: [propext, Quot.sound] -/
3700 #guard_msgs in #print axioms Ratio.mk zero_num
3701 /-- info: 'Ratio.zero_mul' depends on axioms: [propext, Quot.sound] -/
3702 #guard_msgs in #print axioms Ratio.zero_mul
3703 /-- info: 'Ratio.mul_zero' depends on axioms: [propext, Quot.sound] -/
3704 #guard_msgs in #print axioms Ratio.mul_zero
3705 /-- info: 'Ratio.canonical' depends on axioms: [propext] -/
3706 #guard_msgs in #print axioms Ratio.canonical
3707 /-- info: 'Ratio.ext' depends on axioms: [propext] -/
3708 #guard_msgs in #print axioms Ratio.ext
3709 /-- info: 'Ratio.inv_zero' depends on axioms: [propext] -/
3710 #guard_msgs in #print axioms Ratio.inv_zero
3711 /-- info: 'Grade.rank' does not depend on any axioms -/
3712 #guard_msgs in #print axioms Grade.rank
3713 /-- info: 'Grade.weakest' does not depend on any axioms -/
3714 #guard_msgs in #print axioms Grade.weakest
3715 /-- info: 'Grade.weakest_is_a_link' does not depend on any axioms -/
3716 #guard_msgs in #print axioms Grade.weakest_is_a_link
3717 /-- info: 'Grade.weakest_idempotent' does not depend on any axioms -/
3718 #guard_msgs in #print axioms Grade.weakest_idempotent
3719 /-- info: 'Claim.join' does not depend on any axioms -/
3720 #guard_msgs in #print axioms Claim.join
3721 /-- info: 'Claim.join_grade' does not depend on any axioms -/
3722 #guard_msgs in #print axioms Claim.join_grade
3723 /-- info: 'deltaM' does not depend on any axioms -/
3724 #guard_msgs in #print axioms deltaM
3725 /-- info: 'socialWeight' does not depend on any axioms -/
3726 #guard_msgs in #print axioms socialWeight
3727 /-- info: 'FTOE.Even' does not depend on any axioms -/
3728 #guard_msgs in #print axioms FTOE.Even
3729 /-- info: 'FTOE.OddAt' does not depend on any axioms -/
3730 #guard_msgs in #print axioms FTOE.OddAt
3731 /-- info: 'FTOE.WhollyOdd' does not depend on any axioms -/
3732 #guard_msgs in #print axioms FTOE.WhollyOdd
3733 /-- info: 'FTOE.T1_wall' does not depend on any axioms -/
3734 #guard_msgs in #print axioms FTOE.T1_wall
3735 /-- info: 'FTOE.T2_coalition' does not depend on any axioms -/
3736 #guard_msgs in #print axioms FTOE.T2_coalition
3737 /-- info: 'FTOE.T3_no_fixed_point' depends on axioms: [propext] -/
3738 #guard_msgs in #print axioms FTOE.T3_no_fixed_point
3739 /-- info: 'FTOE.T4_one_bit' does not depend on any axioms -/
3740 #guard_msgs in #print axioms FTOE.T4_one_bit
3741 /-- info: 'FTOE.T5_crossing' does not depend on any axioms -/
3742 #guard_msgs in #print axioms FTOE.T5_crossing
3743 /-- info: 'FTOE.T6a_deed_anchor' does not depend on any axioms -/
3744 #guard_msgs in #print axioms FTOE.T6a_deed_anchor
3745 /-- info: 'FTOE.T6b_deed_supply' does not depend on any axioms -/
3746 #guard_msgs in #print axioms FTOE.T6b_deed_supply
3747 /-- info: 'FTOE.FTOE_port' does not depend on any axioms -/
3748 #guard_msgs in #print axioms FTOE.FTOE_port
3749 /-- info: 'FTOE.Involution' does not depend on any axioms -/
3750 #guard_msgs in #print axioms FTOE.Involution
3751 /-- info: 'FTOE.FixedPointFree' does not depend on any axioms -/
3752 #guard_msgs in #print axioms FTOE.FixedPointFree
3753 /-- info: 'FTOE.T7_global_wall' depends on axioms: [propext] -/
3754 #guard_msgs in #print axioms FTOE.T7_global_wall
3755 /-- info: 'FTOE.T8_torsor_forward' does not depend on any axioms -/
3756 #guard_msgs in #print axioms FTOE.T8_torsor_forward
3757 /-- info: 'FTOE.T8_torsor_backward' does not depend on any axioms -/
3758 #guard_msgs in #print axioms FTOE.T8_torsor_backward
3759 /-- info: 'FTOE.T9_torsor_inverse_left' depends on axioms: [Quot.sound] -/
3760 #guard_msgs in #print axioms FTOE.T9_torsor_inverse_left
3761 /-- info: 'FTOE.T9_torsor_inverse_right' depends on axioms: [Quot.sound] -/
3762 #guard_msgs in #print axioms FTOE.T9_torsor_inverse_right
3763 /-- info: 'FTOE.T10_factorization' does not depend on any axioms -/
3764 #guard_msgs in #print axioms FTOE.T10_factorization
3765 /-- info: 'FTOE.T11_separation' does not depend on any axioms -/
3766 #guard_msgs in #print axioms FTOE.T11_separation
3767 /-- info: 'FTOE.T12_encoder_injective' does not depend on any axioms -/
3768 #guard_msgs in #print axioms FTOE.T12_encoder_injective
3769 /-- info: 'FTOE.T14_wall_factorization' does not depend on any axioms -/
3770 #guard_msgs in #print axioms FTOE.T14_wall_factorization
3771 /-- info: 'FTOE.T13_wholly_odd_is_odd_everywhere' depends on axioms: [propext] -/
3772 #guard_msgs in #print axioms FTOE.T13_wholly_odd_is_odd_everywhere
3773 /-- info: 'FTOE.flipF' does not depend on any axioms -/
3774 #guard_msgs in #print axioms FTOE.flipF
3775 /-- info: 'FTOE.T15_price_bijection_forward' does not depend on any axioms -/
3776 #guard_msgs in #print axioms FTOE.T15_price_bijection_forward
3777 /-- info: 'FTOE.T15_price_bijection_backward' does not depend on any axioms -/
3778 #guard_msgs in #print axioms FTOE.T15_price_bijection_backward
3779 /-- info: 'FTOE.FTOE_core' depends on axioms: [propext] -/
3780 #guard_msgs in #print axioms FTOE.FTOE_core
3781 /-- info: 'FTOE.OrbitRel' does not depend on any axioms -/
3782 #guard_msgs in #print axioms FTOE.OrbitRel
3783 /-- info: 'FTOE.OrbitRel_symmetric' does not depend on any axioms -/
3784 #guard_msgs in #print axioms FTOE.OrbitRel_symmetric
3785 /-- info: 'FTOE.Dtau' does not depend on any axioms -/
3786 #guard_msgs in #print axioms FTOE.Dtau
3787 /-- info: 'FTOE.calib' does not depend on any axioms -/
3788 #guard_msgs in #print axioms FTOE.calib
3789 /-- info: 'FTOE.calib_even' does not depend on any axioms -/
3790 #guard_msgs in #print axioms FTOE.calib_even
3791 /-- info: 'FTOE.descend' does not depend on any axioms -/
3792 #guard_msgs in #print axioms FTOE.descend
3793 /-- info: 'FTOE.priceForward' does not depend on any axioms -/
3794 #guard_msgs in #print axioms FTOE.priceForward
3795 /-- info: 'FTOE.priceBackward' depends on axioms: [Quot.sound] -/
3796 #guard_msgs in #print axioms FTOE.priceBackward
3797 /-- info: 'FTOE.T16_left_inverse' depends on axioms: [Quot.sound] -/
3798 #guard_msgs in #print axioms FTOE.T16_left_inverse
3799 /-- info: 'FTOE.T16_right_inverse' depends on axioms: [Quot.sound] -/
3800 #guard_msgs in #print axioms FTOE.T16_right_inverse
3801 /-- info: 'FTOE.T16_price_bijection' depends on axioms: [Quot.sound] -/
3802 #guard_msgs in #print axioms FTOE.T16_price_bijection
3803 /-- info: 'FTOE.canonD0' does not depend on any axioms -/
3804 #guard_msgs in #print axioms FTOE.canonD0
3805 /-- info: 'FTOE.flipF_involution' does not depend on any axioms -/
3806 #guard_msgs in #print axioms FTOE.flipF_involution
3807 /-- info: 'FTOE.T16_canonical' depends on axioms: [Quot.sound] -/
3808 #guard_msgs in #print axioms FTOE.T16_canonical
3809 /-- info: 'FTOE.T17_odd_everywhere_iff_wholly_odd' depends on axioms: [propext] -/
3810 #guard_msgs in #print axioms FTOE.T17_odd_everywhere_iff_wholly_odd
3811 /-- info: 'K4.val' does not depend on any axioms -/
3812 #guard_msgs in #print axioms K4.val
3813 /-- info: 'K4.tau' does not depend on any axioms -/
3814 #guard_msgs in #print axioms K4.tau
3815 /-- info: 'K4.target' does not depend on any axioms -/
3816 #guard_msgs in #print axioms K4.target
3817 /-- info: 'K4.rho' does not depend on any axioms -/
3818 #guard_msgs in #print axioms K4.rho
3819 /-- info: 'K4.tau_involution' does not depend on any axioms -/
3820 #guard_msgs in #print axioms K4.tau_involution
3821 /-- info: 'K4.tau_fixed_point_free' does not depend on any axioms -/
3822 #guard_msgs in #print axioms K4.tau_fixed_point_free
3823 /-- info: 'K4.target_wholly_odd' does not depend on any axioms -/
3824 #guard_msgs in #print axioms K4.target_wholly_odd
3825 /-- info: 'K4.register_even' does not depend on any axioms -/
3826 #guard_msgs in #print axioms K4.register_even
3827 /-- info: 'K4.val_lifts_mod_210' does not depend on any axioms -/
3828 #guard_msgs in #print axioms K4.val_lifts_mod_210
3829 /-- info: 'K4.wall' does not depend on any axioms -/
3830 #guard_msgs in #print axioms K4.wall
3831 /-- info: 'K4.pair_distinct' does not depend on any axioms -/
3832 #guard_msgs in #print axioms K4.pair_distinct
3833 /-- info: 'K4.price_one_bit' does not depend on any axioms -/
3834 #guard_msgs in #print axioms K4.price_one_bit
3835 /-- info: 'K4.price_full_fibre' does not depend on any axioms -/
3836 #guard_msgs in #print axioms K4.price_full_fibre
3837 /-- info: 'K4.wallClaim' does not depend on any axioms -/
3838 #guard_msgs in #print axioms K4.wallClaim
3839 /-- info: 'Audit.Ans.toBool' does not depend on any axioms -/
3840 #guard_msgs in #print axioms Audit.Ans.toBool
3841 /-- info: 'Audit.ClaimInput.anyUnknown' does not depend on any axioms -/
3842 #guard_msgs in #print axioms Audit.ClaimInput.anyUnknown
3843 /-- info: 'Audit.seall' does not depend on any axioms -/
3844 #guard_msgs in #print axioms Audit.seall
3845 /-- info: 'Audit.DrillInput.anyUnknown' does not depend on any axioms -/
3846 #guard_msgs in #print axioms Audit.DrillInput.anyUnknown
3847 /-- info: 'Audit.drillScreen' does not depend on any axioms -/
3848 #guard_msgs in #print axioms Audit.drillScreen
3849 /-- info: 'Audit.refusalGates' does not depend on any axioms -/
3850 #guard_msgs in #print axioms Audit.refusalGates
3851 /-- info: 'Audit.compartmentGates' does not depend on any axioms -/
3852 #guard_msgs in #print axioms Audit.compartmentGates
3853 /-- info: 'Audit.auditClaim' does not depend on any axioms -/
3854 #guard_msgs in #print axioms Audit.auditClaim
3855 /-- info: 'Audit.compartmentGrade' does not depend on any axioms -/
3856 #guard_msgs in #print axioms Audit.compartmentGrade
3857 /-- info: 'Audit.unknown_routes_open' depends on axioms: [propext] -/
3858 #guard_msgs in #print axioms Audit.unknown_routes_open
3859 /-- info: 'Audit.contentless_refused' depends on axioms: [propext] -/
3860 #guard_msgs in #print axioms Audit.contentless_refused

```

```

3861 /-- info: 'Audit.det3' depends on axioms: [propext] -/
3862 #guard_msgs in #print axioms Audit.det3
3863 /-- info: 'Audit.det3_two_id' depends on axioms: [propext] -/
3864 #guard_msgs in #print axioms Audit.det3_two_id
3865 /-- info: 'INVERSION.ItFromBit' does not depend on any axioms -/
3866 #guard_msgs in #print axioms INVERSION.ItFromBit
3867 /-- info: 'INVERSION.it_from_bit_refuted' does not depend on any axioms -/
3868 #guard_msgs in #print axioms INVERSION.it_from_bit_refuted
3869 /-- info: 'INVERSION.it_from_it' does not depend on any axioms -/
3870 #guard_msgs in #print axioms INVERSION.it_from_it
3871 /-- info: 'K4.it_from_it_canonical' does not depend on any axioms -/
3872 #guard_msgs in #print axioms K4.it_from_it_canonical
3873 /-- info: 'K4.it_from_bit_refuted_canonical' does not depend on any axioms -/
3874 #guard_msgs in #print axioms K4.it_from_bit_refuted_canonical
3875 /-- info: 'K4.inversionClaim' does not depend on any axioms -/
3876 #guard_msgs in #print axioms K4.inversionClaim
3877 /-- info: 'GEO.qmul' does not depend on any axioms -/
3878 #guard_msgs in #print axioms GEO.qmul
3879 /-- info: 'GEO.qconj' does not depend on any axioms -/
3880 #guard_msgs in #print axioms GEO.qconj
3881 /-- info: 'GEO.qadd' does not depend on any axioms -/
3882 #guard_msgs in #print axioms GEO.qadd
3883 /-- info: 'GEO.qsub' does not depend on any axioms -/
3884 #guard_msgs in #print axioms GEO.qsub
3885 /-- info: 'GEO.qnorm2' does not depend on any axioms -/
3886 #guard_msgs in #print axioms GEO.qnorm2
3887 /-- info: 'GEO.qhalve' does not depend on any axioms -/
3888 #guard_msgs in #print axioms GEO.qhalve
3889 /-- info: 'GEO.qquarter' does not depend on any axioms -/
3890 #guard_msgs in #print axioms GEO.qquarter
3891 /-- info: 'GEO.qcanon' does not depend on any axioms -/
3892 #guard_msgs in #print axioms GEO.qcanon
3893 /-- info: 'GEO.q1' does not depend on any axioms -/
3894 #guard_msgs in #print axioms GEO.q1
3895 /-- info: 'GEO.q1' does not depend on any axioms -/
3896 #guard_msgs in #print axioms GEO.q1
3897 /-- info: 'GEO.qj' does not depend on any axioms -/
3898 #guard_msgs in #print axioms GEO.qj
3899 /-- info: 'GEO.qk' does not depend on any axioms -/
3900 #guard_msgs in #print axioms GEO.qk
3901 /-- info: 'GEO.chirality' does not depend on any axioms -/
3902 #guard_msgs in #print axioms GEO.chirality
3903 /-- info: 'GEO.noncommutative' does not depend on any axioms -/
3904 #guard_msgs in #print axioms GEO.noncommutative
3905 /-- info: 'GEO.hurwitz' does not depend on any axioms -/
3906 #guard_msgs in #print axioms GEO.hurwitz
3907 /-- info: 'GEO.hurwitz_norm' does not depend on any axioms -/
3908 #guard_msgs in #print axioms GEO.hurwitz_norm
3909 /-- info: 'GEO.hurwitz_products_even' does not depend on any axioms -/
3910 #guard_msgs in #print axioms GEO.hurwitz_products_even
3911 /-- info: 'GEO.hurwitz_closed' does not depend on any axioms -/
3912 #guard_msgs in #print axioms GEO.hurwitz_closed
3913 /-- info: 'GEO.hurwitzReps' does not depend on any axioms -/
3914 #guard_msgs in #print axioms GEO.hurwitzReps
3915 /-- info: 'GEO.units_mod_sign_twelve' does not depend on any axioms -/
3916 #guard_msgs in #print axioms GEO.units_mod_sign_twelve
3917 /-- info: 'GEO.conjClass' does not depend on any axioms -/
3918 #guard_msgs in #print axioms GEO.conjClass
3919 /-- info: 'GEO.classSizesAux' does not depend on any axioms -/
3920 #guard_msgs in #print axioms GEO.classSizesAux
3921 /-- info: 'GEO.insNat' does not depend on any axioms -/
3922 #guard_msgs in #print axioms GEO.insNat
3923 /-- info: 'GEO.insSort' does not depend on any axioms -/
3924 #guard_msgs in #print axioms GEO.insSort
3925 /-- info: 'GEO.class_equation' does not depend on any axioms -/
3926 #guard_msgs in #print axioms GEO.class_equation
3927 /-- info: 'GEO.shell2' does not depend on any axioms -/
3928 #guard_msgs in #print axioms GEO.shell2
3929 /-- info: 'GEO.shell_twentyfour' does not depend on any axioms -/
3930 #guard_msgs in #print axioms GEO.shell_twentyfour
3931 /-- info: 'GEO.no_half_integer_norm2' does not depend on any axioms -/
3932 #guard_msgs in #print axioms GEO.no_half_integer_norm2
3933 /-- info: 'GEO.allGates' does not depend on any axioms -/
3934 #guard_msgs in #print axioms GEO.allGates
3935 /-- info: 'GEO.gates_twelve' does not depend on any axioms -/
3936 #guard_msgs in #print axioms GEO.gates_twelve
3937 /-- info: 'GEO.gates_valid' does not depend on any axioms -/
3938 #guard_msgs in #print axioms GEO.gates_valid
3939 /-- info: 'GEO.gates_complete' depends on axioms: [propext, Quot.sound] -/
3940 #guard_msgs in #print axioms GEO.gates_complete
3941 /-- info: 'GEO.invCount' does not depend on any axioms -/
3942 #guard_msgs in #print axioms GEO.invCount
3943 /-- info: 'GEO.perms24' does not depend on any axioms -/
3944 #guard_msgs in #print axioms GEO.perms24
3945 /-- info: 'GEO.a4' does not depend on any axioms -/
3946 #guard_msgs in #print axioms GEO.a4
3947 /-- info: 'GEO.a4_order' does not depend on any axioms -/
3948 #guard_msgs in #print axioms GEO.a4_order
3949 /-- info: 'GEO.actG' does not depend on any axioms -/
3950 #guard_msgs in #print axioms GEO.actG
3951 /-- info: 'GEO.a4_simply_transitive' does not depend on any axioms -/
3952 #guard_msgs in #print axioms GEO.a4_simply_transitive
3953 /-- info: 'GEO.tetraVerts' does not depend on any axioms -/
3954 #guard_msgs in #print axioms GEO.tetraVerts
3955 /-- info: 'GEO.tetraEdges' does not depend on any axioms -/
3956 #guard_msgs in #print axioms GEO.tetraEdges
3957 /-- info: 'GEO.tetraFaces' does not depend on any axioms -/
3958 #guard_msgs in #print axioms GEO.tetraFaces
3959 /-- info: 'GEO.euler_closure' does not depend on any axioms -/
3960 #guard_msgs in #print axioms GEO.euler_closure
3961 /-- info: 'GEO.basisU' does not depend on any axioms -/
3962 #guard_msgs in #print axioms GEO.basisU
3963 /-- info: 'GEO.perms6' does not depend on any axioms -/
3964 #guard_msgs in #print axioms GEO.perms6
3965 /-- info: 'GEO.relabel_parity' does not depend on any axioms -/
3966 #guard_msgs in #print axioms GEO.relabel_parity

3967 /-- info: 'GEO.08' does not depend on any axioms -/
3968 #guard_msgs in #print axioms GEO.08
3969 /-- info: 'GEO.omul' does not depend on any axioms -/
3970 #guard_msgs in #print axioms GEO.omul
3971 /-- info: 'GEO.osub' does not depend on any axioms -/
3972 #guard_msgs in #print axioms GEO.osub
3973 /-- info: 'GEO.onorm2' does not depend on any axioms -/
3974 #guard_msgs in #print axioms GEO.onorm2
3975 /-- info: 'GEO.e8' does not depend on any axioms -/
3976 #guard_msgs in #print axioms GEO.e8
3977 /-- info: 'GEO.octonion_associator' does not depend on any axioms -/
3978 #guard_msgs in #print axioms GEO.octonion_associator
3979 /-- info: 'GEO.hPairs' does not depend on any axioms -/
3980 #guard_msgs in #print axioms GEO.hPairs
3981 /-- info: 'GEO.norm_composition_H' does not depend on any axioms -/
3982 #guard_msgs in #print axioms GEO.norm_composition_H
3983 /-- info: 'GEO.oPairs' does not depend on any axioms -/
3984 #guard_msgs in #print axioms GEO.oPairs
3985 /-- info: 'GEO.norm_composition_0' does not depend on any axioms -/
3986 #guard_msgs in #print axioms GEO.norm_composition_0
3987 /-- info: 'GEO.det2' does not depend on any axioms -/
3988 #guard_msgs in #print axioms GEO.det2
3989 /-- info: 'GEO.gf2Mul' does not depend on any axioms -/
3990 #guard_msgs in #print axioms GEO.gf2Mul
3991 /-- info: 'GEO.gf2Vecs' does not depend on any axioms -/
3992 #guard_msgs in #print axioms GEO.gf2Vecs
3993 /-- info: 'GEO.gf2Sols' does not depend on any axioms -/
3994 #guard_msgs in #print axioms GEO.gf2Sols
3995 /-- info: 'GEO.triaxial_lock_gf2' does not depend on any axioms -/
3996 #guard_msgs in #print axioms GEO.triaxial_lock_gf2
3997 /-- info: 'GEO.triaxial_lock_gf2_count' does not depend on any axioms -/
3998 #guard_msgs in #print axioms GEO.triaxial_lock_gf2_count
3999 /-- info: 'GEO.triaxial_open_gf2_count' does not depend on any axioms -/
4000 #guard_msgs in #print axioms GEO.triaxial_open_gf2_count
4001 /-- info: 'GEO.V3' depends on axioms: [propext] -/
4002 #guard_msgs in #print axioms GEO.V3
4003 /-- info: 'GEO.M3' depends on axioms: [propext] -/
4004 #guard_msgs in #print axioms GEO.M3
4005 /-- info: 'GEO.det3' depends on axioms: [propext] -/
4006 #guard_msgs in #print axioms GEO.det3
4007 /-- info: 'GEO.adj3' depends on axioms: [propext] -/
4008 #guard_msgs in #print axioms GEO.adj3
4009 /-- info: 'GEO.mm3' depends on axioms: [propext] -/
4010 #guard_msgs in #print axioms GEO.mm3
4011 /-- info: 'GEO.diag3' depends on axioms: [propext] -/
4012 #guard_msgs in #print axioms GEO.diag3
4013 /-- info: 'GEO.mv3' depends on axioms: [propext] -/
4014 #guard_msgs in #print axioms GEO.mv3
4015 /-- info: 'GEO.vscale' depends on axioms: [propext] -/
4016 #guard_msgs in #print axioms GEO.vscale
4017 /-- info: 'GEO.V3Z' does not depend on any axioms -/
4018 #guard_msgs in #print axioms GEO.V3Z
4019 /-- info: 'GEO.M3Z' does not depend on any axioms -/
4020 #guard_msgs in #print axioms GEO.M3Z
4021 /-- info: 'GEO.det3Z' does not depend on any axioms -/
4022 #guard_msgs in #print axioms GEO.det3Z
4023 /-- info: 'GEO.adj3Z' does not depend on any axioms -/
4024 #guard_msgs in #print axioms GEO.adj3Z
4025 /-- info: 'GEO.mm3Z' does not depend on any axioms -/
4026 #guard_msgs in #print axioms GEO.mm3Z
4027 /-- info: 'GEO.diag3Z' does not depend on any axioms -/
4028 #guard_msgs in #print axioms GEO.diag3Z
4029 /-- info: 'GEO.adjugate_identity_int' depends on axioms: [propext, Quot.sound] -/
4030 #guard_msgs in #print axioms GEO.adjugate_identity_int
4031 /-- info: 'GEO.AdjugateIdentity3' depends on axioms: [propext] -/
4032 #guard_msgs in #print axioms GEO.AdjugateIdentity3
4033 /-- info: 'GEO.adj3Samples' depends on axioms: [propext] -/
4034 #guard_msgs in #print axioms GEO.adj3Samples
4035 /-- info: 'GEO.adj3_executed' depends on axioms: [propext] -/
4036 #guard_msgs in #print axioms GEO.adj3_executed
4037 /-- info: 'GEO.cramerSolve' depends on axioms: [propext] -/
4038 #guard_msgs in #print axioms GEO.cramerSolve
4039 /-- info: 'GEO.cramer_executed' depends on axioms: [propext] -/
4040 #guard_msgs in #print axioms GEO.cramer_executed
4041 /-- info: 'GEO.goAdmit' does not depend on any axioms -/
4042 #guard_msgs in #print axioms GEO.goAdmit
4043 /-- info: 'GEO.gateNames' does not depend on any axioms -/
4044 #guard_msgs in #print axioms GEO.gateNames
4045 /-- info: 'GEO.gateNames_twelve' does not depend on any axioms -/
4046 #guard_msgs in #print axioms GEO.gateNames_twelve
4047 /-- info: 'GEO.gateScreenGo' does not depend on any axioms -/
4048 #guard_msgs in #print axioms GEO.gateScreenGo
4049 /-- info: 'GEO.gateScreen' does not depend on any axioms -/
4050 #guard_msgs in #print axioms GEO.gateScreen
4051 /-- info: 'GEO.gateScreen_clean' does not depend on any axioms -/
4052 #guard_msgs in #print axioms GEO.gateScreen_clean
4053 /-- info: 'GEO.gateScreen_first_failure' does not depend on any axioms -/
4054 #guard_msgs in #print axioms GEO.gateScreen_first_failure
4055 /-- info: 'GEO.gateScreen_names_failed_gate' does not depend on any axioms -/
4056 #guard_msgs in #print axioms GEO.gateScreen_names_failed_gate
4057 /-- info: 'GEO.gateScreen_arity' does not depend on any axioms -/
4058 #guard_msgs in #print axioms GEO.gateScreen_arity
4059 /-- info: 'GEO.ClaimInputRA.anyUnknown' does not depend on any axioms -/
4060 #guard_msgs in #print axioms GEO.ClaimInputRA.anyUnknown
4061 /-- info: 'GEO.rowCascadeRACore' does not depend on any axioms -/
4062 #guard_msgs in #print axioms GEO.rowCascadeRACore
4063 /-- info: 'GEO.rowCascadeRA' does not depend on any axioms -/
4064 #guard_msgs in #print axioms GEO.rowCascadeRA
4065 /-- info: 'GEO.unknown_routes_open_ra' depends on axioms: [propext] -/
4066 #guard_msgs in #print axioms GEO.unknown_routes_open_ra
4067 /-- info: 'GEO.raDecode' depends on axioms: [propext] -/
4068 #guard_msgs in #print axioms GEO.raDecode
4069 /-- info: 'GEO.raBattery' does not depend on any axioms -/
4070 #guard_msgs in #print axioms GEO.raBattery
4071 /-- info: 'GEO.ra_battery_f2' depends on axioms: [propext] -/
4072 #guard_msgs in #print axioms GEO.ra_battery_f2

```

```

4073 /-- info: 'GEO.ra_ledger_refused' depends on axioms: [propxt] -/
4074 #guard_msgs in #print axioms GEO.ra_ledger_refused
4075 /-- info: 'GEO.ra_ledger_iii' depends on axioms: [propxt] -/
4076 #guard_msgs in #print axioms GEO.ra_ledger_iii
4077 /-- info: 'GEO.ra_ledger_ii' depends on axioms: [propxt] -/
4078 #guard_msgs in #print axioms GEO.ra_ledger_ii
4079 /-- info: 'GEO.ra_ledger_void' depends on axioms: [propxt] -/
4080 #guard_msgs in #print axioms GEO.ra_ledger_void
4081 /-- info: 'GEO.ra_ledger_aGivenRA' depends on axioms: [propxt] -/
4082 #guard_msgs in #print axioms GEO.ra_ledger_aGivenRA
4083 /-- info: 'GEO.RAVerdict.toToken' does not depend on any axioms -/
4084 #guard_msgs in #print axioms GEO.RAVerdict.toToken
4085 /-- info: 'GEO.GoToken.toToken' does not depend on any axioms -/
4086 #guard_msgs in #print axioms GEO.GoToken.toToken
4087 /-- info: 'GEO.gateScreenToken' does not depend on any axioms -/
4088 #guard_msgs in #print axioms GEO.gateScreenToken
4089 /-- info: 'GEO.demoInputA' does not depend on any axioms -/
4090 #guard_msgs in #print axioms GEO.demoInputA
4091 /-- info: 'GEO.demoInputVoid' does not depend on any axioms -/
4092 #guard_msgs in #print axioms GEO.demoInputVoid
4093 /-- info: 'GEO.demoInputOpen' does not depend on any axioms -/
4094 #guard_msgs in #print axioms GEO.demoInputOpen
4095 /-- info: 'ROOT.Actuates' depends on axioms: [ROOT.U, ROOTA.E] -/
4096 #guard_msgs in #print axioms ROOT.Actuates
4097 /-- info: 'ROOT.actuation_universal' depends on axioms: [ROOT.RA, ROOT.U, ROOTA.E] -/
4098 #guard_msgs in #print axioms ROOT.actuation_universal
4099 /-- info: 'ROOT.raClaim' depends on axioms: [ROOT.RA, ROOT.U, ROOTA.E] -/
4100 #guard_msgs in #print axioms ROOT.raClaim
4101 /-- info: 'ROOT.boolCut' does not depend on any axioms -/
4102 #guard_msgs in #print axioms ROOT.boolCut
4103 /-- info: 'ROOT.nest_31' depends on axioms: [ROOT.FormalDomain, ROOT.L1m, ROOT.L3m, ROOT
    .nest_21, ROOT.nest_32] -/
4104 #guard_msgs in #print axioms ROOT.nest_31
4105 /-- info: 'ROOT.nestClaim' depends on axioms: [ROOT.FormalDomain, ROOT.L1m, ROOT.L3m,
    ROOT.nest_21, ROOT.nest_32] -/
4106 #guard_msgs in #print axioms ROOT.nestClaim
4107 /-- info: 'ROOT.throneEmpty' depends on axioms: [ROOT.U, ROOTA.E] -/
4108 #guard_msgs in #print axioms ROOT.throneEmpty
4109 /-- info: 'ROOT.throne_is_RA' depends on axioms: [ROOT.U, ROOTA.E] -/
4110 #guard_msgs in #print axioms ROOT.throne_is_RA
4111 /-- info: 'SEALS.sealL' does not depend on any axioms -/
4112 #guard_msgs in #print axioms SEALS.sealL
4113 /-- info: 'SEALS.sealG' depends on axioms: [ROOT.RA, ROOT.U, ROOTA.E] -/
4114 #guard_msgs in #print axioms SEALS.sealG
4115 /-- info: 'SEALS.sealM' does not depend on any axioms -/
4116 #guard_msgs in #print axioms SEALS.sealM
4117 /-- info: 'SEALS.seals_distinct' does not depend on any axioms -/
4118 #guard_msgs in #print axioms SEALS.seals_distinct
4119 /-- info: 'SEALS.seal_names_match' depends on axioms: [ROOT.RA, ROOT.U, ROOTA.E] -/
4120 #guard_msgs in #print axioms SEALS.seal_names_match
4121 /-- info: 'SEALS.sealG_walk' depends on axioms: [propxt] -/
4122 #guard_msgs in #print axioms SEALS.sealG_walk
4123 /-- info: 'POSTULATE.OneCut' depends on axioms: [POSTULATE.Row, POSTULATE.SupplyClean]
    -/
4124 #guard_msgs in #print axioms POSTULATE.OneCut
4125 /-- info: 'POSTULATE.F2' depends on axioms: [POSTULATE.Row, POSTULATE.SupplyClean] -/
4126 #guard_msgs in #print axioms POSTULATE.F2
4127 /-- info: 'POSTULATE.F2_kills' depends on axioms: [POSTULATE.Row, POSTULATE.SupplyClean]
    -/
4128 #guard_msgs in #print axioms POSTULATE.F2_kills
4129 /-- info: 'POSTULATE.BSDContract' depends on axioms: [POSTULATE.Algorithm, POSTULATE.
    FrozenData, POSTULATE.certifiedPairing] -/
4130 #guard_msgs in #print axioms POSTULATE.BSDContract
4131 /-- info: 'POSTULATE.F6' depends on axioms: [POSTULATE.Algorithm, POSTULATE.FrozenData,
    POSTULATE.certifiedPairing] -/
4132 #guard_msgs in #print axioms POSTULATE.F6
4133 /-- info: 'TONGUE.enlarge' does not depend on any axioms -/
4134 #guard_msgs in #print axioms TONGUE.enlarge
4135 /-- info: 'TONGUE.unclosed' does not depend on any axioms -/
4136 #guard_msgs in #print axioms TONGUE.unclosed
4137 /-- info: 'TONGUE.earned_freedom' does not depend on any axioms -/
4138 #guard_msgs in #print axioms TONGUE.earned_freedom
4139 /-- info: 'TONGUE.colocClaim' depends on axioms: [ROOT.RA, ROOT.U, ROOTA.E] -/
4140 #guard_msgs in #print axioms TONGUE.colocClaim
4141 /-- info: 'TONGUE.coloc_grade' depends on axioms: [ROOT.RA, ROOT.U, ROOTA.E] -/
4142 #guard_msgs in #print axioms TONGUE.coloc_grade
4143 /-- info: 'TONGUE.ra_grade_pinned' depends on axioms: [ROOT.RA, ROOT.U, ROOTA.E] -/
4144 #guard_msgs in #print axioms TONGUE.ra_grade_pinned
4145 /-- info: 'TONGUE.wall_grade_pinned' does not depend on any axioms -/
4146 #guard_msgs in #print axioms TONGUE.wall_grade_pinned
4147 /-- info: 'TONGUE.inversion_grade_pinned' does not depend on any axioms -/
4148 #guard_msgs in #print axioms TONGUE.inversion_grade_pinned
4149 /-- info: 'TONGUE.nest_grade_pinned' depends on axioms: [ROOT.FormalDomain, ROOT.L1m,
    ROOT.L3m, ROOT.nest_21, ROOT.nest_32] -/
4150 #guard_msgs in #print axioms TONGUE.nest_grade_pinned
4151 /-- info: 'terminalWall' does not depend on any axioms -/
4152 #guard_msgs in #print axioms terminalWall
4153 /-- info: 'terminalPrice' depends on axioms: [Quot.sound] -/
4154 #guard_msgs in #print axioms terminalPrice
4155 /-- info: 'TERMINAL' depends on axioms: [Quot.sound] -/
4156 #guard_msgs in #print axioms TERMINAL
4157 /-- info: 'terminalClaim' depends on axioms: [Quot.sound] -/
4158 #guard_msgs in #print axioms terminalClaim
4159 /-- info: 'terminal_grade_pinned' depends on axioms: [Quot.sound] -/
4160 #guard_msgs in #print axioms terminal_grade_pinned
4161 /-- info: 'IAM.iamToken' does not depend on any axioms -/
4162 #guard_msgs in #print axioms IAM.iamToken
4163 /-- info: 'IAM.iam_unwitnessed_withheld' does not depend on any axioms -/
4164 #guard_msgs in #print axioms IAM.iam_unwitnessed_withheld
4165 /-- info: 'IAM.iam_witnessed_speaks' does not depend on any axioms -/
4166 #guard_msgs in #print axioms IAM.iam_witnessed_speaks
4167 /-- info: 'IAM.narcissus_truth_table' does not depend on any axioms -/
4168 #guard_msgs in #print axioms IAM.narcissus_truth_table
4169 /-- info: 'IAM.one_bit_freedom' does not depend on any axioms -/
4170 #guard_msgs in #print axioms IAM.one_bit_freedom
4171 /-- info: 'IAM.iamToken.toToken' does not depend on any axioms -/

```

```

4172 #guard_msgs in #print axioms IAM.iamToken.toToken
4173 /-- info: 'IAM.codexSelfRead' does not depend on any axioms -/
4174 #guard_msgs in #print axioms IAM.codexSelfRead
4175 /-- info: 'IAM.codex_self_read_interior' does not depend on any axioms -/
4176 #guard_msgs in #print axioms IAM.codex_self_read_interior
4177 /-- info: 'IAM.terminalWord' does not depend on any axioms -/
4178 #guard_msgs in #print axioms IAM.terminalWord
4179 /-- info: 'IAM.terminal_word' does not depend on any axioms -/
4180 #guard_msgs in #print axioms IAM.terminal_word
4181 /-- info: 'GEO.SignPattern' does not depend on any axioms -/
4182 #guard_msgs in #print axioms GEO.SignPattern
4183 /-- info: 'GEO.allSigns' does not depend on any axioms -/
4184 #guard_msgs in #print axioms GEO.allSigns
4185 /-- info: 'GEO.signComp' does not depend on any axioms -/
4186 #guard_msgs in #print axioms GEO.signComp
4187 /-- info: 'GEO.reflection_eight' does not depend on any axioms -/
4188 #guard_msgs in #print axioms GEO.reflection_eight
4189 /-- info: 'GEO.reflection_eight_is_two_cubed' does not depend on any axioms -/
4190 #guard_msgs in #print axioms GEO.reflection_eight_is_two_cubed
4191 /-- info: 'GEO.reflections_abelian' does not depend on any axioms -/
4192 #guard_msgs in #print axioms GEO.reflections_abelian
4193 /-- info: 'GEO.reflections_involutive' does not depend on any axioms -/
4194 #guard_msgs in #print axioms GEO.reflections_involutive
4195 /-- info: 'GEO.signVal' depends on axioms: [propxt] -/
4196 #guard_msgs in #print axioms GEO.signVal
4197 /-- info: 'GEO.reflMat' depends on axioms: [propxt] -/
4198 #guard_msgs in #print axioms GEO.reflMat
4199 /-- info: 'GEO.reflection_reverses_orientation' depends on axioms: [propxt] -/
4200 #guard_msgs in #print axioms GEO.reflection_reverses_orientation
4201 /-- info: 'GEO.sandwich' depends on axioms: [propxt] -/
4202 #guard_msgs in #print axioms GEO.sandwich
4203 /-- info: 'GEO.reflection_blind_executed' depends on axioms: [propxt] -/
4204 #guard_msgs in #print axioms GEO.reflection_blind_executed
4205 /-- info: 'GEO.V5' does not depend on any axioms -/
4206 #guard_msgs in #print axioms GEO.V5
4207 /-- info: 'GEO.cube5' depends on axioms: [propxt] -/
4208 #guard_msgs in #print axioms GEO.cube5
4209 /-- info: 'GEO.neighbors5' does not depend on any axioms -/
4210 #guard_msgs in #print axioms GEO.neighbors5
4211 /-- info: 'GEO.fivecube_thirtytwo' depends on axioms: [propxt] -/
4212 #guard_msgs in #print axioms GEO.fivecube_thirtytwo
4213 /-- info: 'GEO.fivecube_vertices_distinct' depends on axioms: [propxt] -/
4214 #guard_msgs in #print axioms GEO.fivecube_vertices_distinct
4215 /-- info: 'GEO.fivecube_degree_five' depends on axioms: [propxt] -/
4216 #guard_msgs in #print axioms GEO.fivecube_degree_five
4217 /-- info: 'GEO.counts_forced' depends on axioms: [propxt] -/
4218 #guard_msgs in #print axioms GEO.counts_forced
4219 /-- info: 'TONGUE.deletionOp' depends on axioms: [propxt] -/
4220 #guard_msgs in #print axioms TONGUE.deletionOp
4221 /-- info: 'TONGUE.e3' depends on axioms: [propxt] -/
4222 #guard_msgs in #print axioms TONGUE.e3
4223 /-- info: 'TONGUE.deletion_idempotent' depends on axioms: [propxt] -/
4224 #guard_msgs in #print axioms TONGUE.deletion_idempotent
4225 /-- info: 'TONGUE.deletion_singular' depends on axioms: [propxt] -/
4226 #guard_msgs in #print axioms TONGUE.deletion_singular
4227 /-- info: 'TONGUE.deletion_minor_nonsingular' depends on axioms: [propxt] -/
4228 #guard_msgs in #print axioms TONGUE.deletion_minor_nonsingular
4229 /-- info: 'TONGUE.deletion_annihilates' depends on axioms: [propxt] -/
4230 #guard_msgs in #print axioms TONGUE.deletion_annihilates
4231 /-- info: 'TONGUE.linear_fixes_zero' depends on axioms: [propxt, Quot.sound] -/
4232 #guard_msgs in #print axioms TONGUE.linear_fixes_zero
4233 /-- info: 'TONGUE.deletion_no_left_inverse' depends on axioms: [propxt, Quot.sound] -/
4234 #guard_msgs in #print axioms TONGUE.deletion_no_left_inverse
4235 /-- info: 'TONGUE.deletion_monoid_not_group' depends on axioms: [propxt, Quot.sound] -/
4236 #guard_msgs in #print axioms TONGUE.deletion_monoid_not_group
4237 /-- info: 'TONGUE.tongue_obedience' depends on axioms: [propxt, Quot.sound] -/
4238 #guard_msgs in #print axioms TONGUE.tongue_obedience
4239 /-- info: 'OFFICE.discriminator' does not depend on any axioms -/
4240 #guard_msgs in #print axioms OFFICE.discriminator
4241 /-- info: 'OFFICE.discriminatorQ2First' does not depend on any axioms -/
4242 #guard_msgs in #print axioms OFFICE.discriminatorQ2First
4243 /-- info: 'OFFICE.order_loadbearing' does not depend on any axioms -/
4244 #guard_msgs in #print axioms OFFICE.order_loadbearing
4245 /-- info: 'OFFICE.citation_cannot_promote' does not depend on any axioms -/
4246 #guard_msgs in #print axioms OFFICE.citation_cannot_promote
4247 /-- info: 'UC.routeHalt' does not depend on any axioms -/
4248 #guard_msgs in #print axioms UC.routeHalt
4249 /-- info: 'UC.routeHalt_fidelity' does not depend on any axioms -/
4250 #guard_msgs in #print axioms UC.routeHalt_fidelity
4251 /-- info: 'UC.universalCascade' does not depend on any axioms -/
4252 #guard_msgs in #print axioms UC.universalCascade
4253 /-- info: 'UC.uc_seal_failure_terminal' does not depend on any axioms -/
4254 #guard_msgs in #print axioms UC.uc_seal_failure_terminal
4255 /-- info: 'UC.uc_routes_eight' does not depend on any axioms -/
4256 #guard_msgs in #print axioms UC.uc_routes_eight
4257 /-- info: 'UC.uc_routes_five' does not depend on any axioms -/
4258 #guard_msgs in #print axioms UC.uc_routes_five
4259 /-- info: 'UC.uc_unmeasured' does not depend on any axioms -/
4260 #guard_msgs in #print axioms UC.uc_unmeasured
4261 /-- info: 'UC.circularityScreen' does not depend on any axioms -/
4262 #guard_msgs in #print axioms UC.circularityScreen
4263 /-- info: 'UC.circularity_refused_at_target' does not depend on any axioms -/
4264 #guard_msgs in #print axioms UC.circularity_refused_at_target
4265 /-- info: 'UC.road_requires_conjunction' does not depend on any axioms -/
4266 #guard_msgs in #print axioms UC.road_requires_conjunction
4267 /-- info: 'UC.circRows' does not depend on any axioms -/
4268 #guard_msgs in #print axioms UC.circRows
4269 /-- info: 'UC.circ_space' does not depend on any axioms -/
4270 #guard_msgs in #print axioms UC.circ_space
4271 /-- info: 'OMEGA.kbRat' depends on axioms: [propxt] -/
4272 #guard_msgs in #print axioms OMEGA.kbRat
4273 /-- info: 'OMEGA.ln2Rat' depends on axioms: [propxt] -/
4274 #guard_msgs in #print axioms OMEGA.ln2Rat
4275 /-- info: 'OMEGA.landauerRat' depends on axioms: [propxt] -/
4276 #guard_msgs in #print axioms OMEGA.landauerRat
4277 /-- info: 'OMEGA.omegaBoundary' depends on axioms: [propxt] -/

```

```

4278 #guard_msgs in #print axioms OMEGA.omegaBoundary
4279 /-- info: 'OMEGA.omega_zero_bits' depends on axioms: [propxt] -/
4280 #guard_msgs in #print axioms OMEGA.omega_zero_bits
4281 /-- info: 'OMEGA.omega_paid_floor' depends on axioms: [propxt] -/
4282 #guard_msgs in #print axioms OMEGA.omega_paid_floor
4283 /-- info: 'OMEGA.one_bit_freedom_priced' depends on axioms: [propxt] -/
4284 #guard_msgs in #print axioms OMEGA.one_bit_freedom_priced
4285 /-- info: 'OMEGA.omega_reversible_branch' depends on axioms: [propxt] -/
4286 #guard_msgs in #print axioms OMEGA.omega_reversible_branch
4287 /-- info: 'OMEGA.omega_refuses_nonpositive_temp' depends on axioms: [propxt] -/
4288 #guard_msgs in #print axioms OMEGA.omega_refuses_nonpositive_temp
4289 /-- info: 'OMEGA.omega_monotone' depends on axioms: [propxt] -/
4290 #guard_msgs in #print axioms OMEGA.omega_monotone
4291 /-- info: 'OMEGA.omega_linear' depends on axioms: [propxt] -/
4292 #guard_msgs in #print axioms OMEGA.omega_linear
4293 /-- info: 'OMEGA.landauer_zero_bits' depends on axioms: [propxt] -/
4294 #guard_msgs in #print axioms OMEGA.landauer_zero_bits
4295 /-- info: 'OMEGA.omega_verdict_fidelity' depends on axioms: [propxt] -/
4296 #guard_msgs in #print axioms OMEGA.omega_verdict_fidelity
4297 /-- info: 'AEGIS.refusalConst' does not depend on any axioms -/
4298 #guard_msgs in #print axioms AEGIS.refusalConst
4299 /-- info: 'AEGIS.aegisReset' does not depend on any axioms -/
4300 #guard_msgs in #print axioms AEGIS.aegisReset
4301 /-- info: 'AEGIS.aegisGuard' does not depend on any axioms -/
4302 #guard_msgs in #print axioms AEGIS.aegisGuard
4303 /-- info: 'AEGIS.aegis_constant' does not depend on any axioms -/
4304 #guard_msgs in #print axioms AEGIS.aegis_constant
4305 /-- info: 'AEGIS.aegis_deed_increments' does not depend on any axioms -/
4306 #guard_msgs in #print axioms AEGIS.aegis_deed_increments
4307 /-- info: 'AEGIS.aegis_reads_parameter' does not depend on any axioms -/
4308 #guard_msgs in #print axioms AEGIS.aegis_reads_parameter
4309 /-- info: 'AEGIS.aegisFourLogicRun' does not depend on any axioms -/
4310 #guard_msgs in #print axioms AEGIS.aegisFourLogicRun
4311 /-- info: 'AEGIS.aegis_battery' does not depend on any axioms -/
4312 #guard_msgs in #print axioms AEGIS.aegis_battery
4313 /-- info: 'NOMOS.allStrings' does not depend on any axioms -/
4314 #guard_msgs in #print axioms NOMOS.allStrings
4315 /-- info: 'NOMOS.allStrings_length' depends on axioms: [propxt] -/
4316 #guard_msgs in #print axioms NOMOS.allStrings_length
4317 /-- info: 'NOMOS.twoPowPos' does not depend on any axioms -/
4318 #guard_msgs in #print axioms NOMOS.twoPowPos
4319 /-- info: 'NOMOS.descs' does not depend on any axioms -/
4320 #guard_msgs in #print axioms NOMOS.descs
4321 /-- info: 'NOMOS.descs_length' depends on axioms: [propxt] -/
4322 #guard_msgs in #print axioms NOMOS.descs_length
4323 /-- info: 'NOMOS.incompressibility_cap' depends on axioms: [propxt] -/
4324 #guard_msgs in #print axioms NOMOS.incompressibility_cap
4325 /-- info: 'NOMOS.nomosRows' does not depend on any axioms -/
4326 #guard_msgs in #print axioms NOMOS.nomosRows
4327 /-- info: 'NOMOS.incompressibility_executed' does not depend on any axioms -/
4328 #guard_msgs in #print axioms NOMOS.incompressibility_executed
4329 /-- info: 'NOMOS.decodeId' does not depend on any axioms -/
4330 #guard_msgs in #print axioms NOMOS.decodeId
4331 /-- info: 'NOMOS.decodePad' does not depend on any axioms -/
4332 #guard_msgs in #print axioms NOMOS.decodePad
4333 /-- info: 'NOMOS.compressible_bound_universal' depends on axioms: [propxt] -/
4334 #guard_msgs in #print axioms NOMOS.compressible_bound_universal
4335 /-- info: 'NOMOS.cbUniversal' depends on axioms: [propxt] -/
4336 #guard_msgs in #print axioms NOMOS.cbUniversal
4337 /-- info: 'NOMOS.cb_id_8_2' does not depend on any axioms -/
4338 #guard_msgs in #print axioms NOMOS.cb_id_8_2
4339 /-- info: 'NOMOS.cb_pad3_8_2' does not depend on any axioms -/
4340 #guard_msgs in #print axioms NOMOS.cb_pad3_8_2
4341 /-- info: 'NOMOS.cb_pad2_6_1' does not depend on any axioms -/
4342 #guard_msgs in #print axioms NOMOS.cb_pad2_6_1
4343 /-- info: 'NOMOS.cb_pad4_12_3' does not depend on any axioms -/
4344 #guard_msgs in #print axioms NOMOS.cb_pad4_12_3
4345 /-- info: 'NOMOS.recoverable' depends on axioms: [propxt] -/
4346 #guard_msgs in #print axioms NOMOS.recoverable
4347 /-- info: 'NOMOS.nomos_fuel_is_omega_floor' depends on axioms: [propxt] -/
4348 #guard_msgs in #print axioms NOMOS.nomos_fuel_is_omega_floor
4349 /-- info: 'NOMOS.nomosScreen' does not depend on any axioms -/
4350 #guard_msgs in #print axioms NOMOS.nomosScreen
4351 /-- info: 'NOMOS.nomosScreen_battery' does not depend on any axioms -/
4352 #guard_msgs in #print axioms NOMOS.nomosScreen_battery
4353 /-- info: 'BRIDGE.ceiling' does not depend on any axioms -/
4354 #guard_msgs in #print axioms BRIDGE.ceiling
4355 /-- info: 'BRIDGE.census' does not depend on any axioms -/
4356 #guard_msgs in #print axioms BRIDGE.census
4357 /-- info: 'BRIDGE.census_count' does not depend on any axioms -/
4358 #guard_msgs in #print axioms BRIDGE.census_count
4359 /-- info: 'BRIDGE.post_seal_disposition' does not depend on any axioms -/
4360 #guard_msgs in #print axioms BRIDGE.post_seal_disposition
4361 /-- info: 'BRIDGE.weakest_never_above_left' does not depend on any axioms -/
4362 #guard_msgs in #print axioms BRIDGE.weakest_never_above_left
4363 /-- info: 'BRIDGE.bridge_ceiling_caps' does not depend on any axioms -/
4364 #guard_msgs in #print axioms BRIDGE.bridge_ceiling_caps
4365 /-- info: 'BRIDGE.successorOf' does not depend on any axioms -/
4366 #guard_msgs in #print axioms BRIDGE.successorOf
4367 /-- info: 'BRIDGE.supersession_routes' does not depend on any axioms -/
4368 #guard_msgs in #print axioms BRIDGE.supersession_routes
4369 /-- info: 'BRIDGE.bridgeScreen' does not depend on any axioms -/
4370 #guard_msgs in #print axioms BRIDGE.bridgeScreen
4371 /-- info: 'BRIDGE.bridgeScreen_total' does not depend on any axioms -/
4372 #guard_msgs in #print axioms BRIDGE.bridgeScreen_total
4373 /-- info: 'FOUND.sigmaP' depends on axioms: [propxt] -/
4374 #guard_msgs in #print axioms FOUND.sigmaP
4375 /-- info: 'FOUND.pluralist_alternative_executed' depends on axioms: [propxt] -/
4376 #guard_msgs in #print axioms FOUND.pluralist_alternative_executed
4377 /-- info: 'FOUND.posits' does not depend on any axioms -/
4378 #guard_msgs in #print axioms FOUND.posits
4379 /-- info: 'FOUND.posits_loadbearing_on_nothing' does not depend on any axioms -/
4380 #guard_msgs in #print axioms FOUND.posits_loadbearing_on_nothing
4381 /-- info: 'FOUND.gradeOfRank' does not depend on any axioms -/
4382 #guard_msgs in #print axioms FOUND.gradeOfRank
4383 /-- info: 'FOUND.posit_never_raises' does not depend on any axioms -/
4384 #guard_msgs in #print axioms FOUND.posit_never_raises
4385 /-- info: 'FOUND.bandOf' does not depend on any axioms -/
4386 #guard_msgs in #print axioms FOUND.bandOf
4387 /-- info: 'FOUND.register_routing_fidelity' does not depend on any axioms -/
4388 #guard_msgs in #print axioms FOUND.register_routing_fidelity
4389 /-- info: 'FOUND.verdictInventory' does not depend on any axioms -/
4390 #guard_msgs in #print axioms FOUND.verdictInventory
4391 /-- info: 'FOUND.verdicts_posit_free' does not depend on any axioms -/
4392 #guard_msgs in #print axioms FOUND.verdicts_posit_free
4393 /-- info: 'FOUND.engine_never_authority' does not depend on any axioms -/
4394 #guard_msgs in #print axioms FOUND.engine_never_authority
4395 /-- info: 'HALT.cascade' does not depend on any axioms -/
4396 #guard_msgs in #print axioms HALT.cascade
4397 /-- info: 'HALT.patterns' depends on axioms: [propxt] -/
4398 #guard_msgs in #print axioms HALT.patterns
4399 /-- info: 'HALT.fftSpec' does not depend on any axioms -/
4400 #guard_msgs in #print axioms HALT.fftSpec
4401 /-- info: 'HALT.first_failure_terminal_5' depends on axioms: [propxt] -/
4402 #guard_msgs in #print axioms HALT.first_failure_terminal_5
4403 /-- info: 'HALT.first_failure_terminal_7' depends on axioms: [propxt] -/
4404 #guard_msgs in #print axioms HALT.first_failure_terminal_7
4405 /-- info: 'HALT.first_failure_terminal_8' depends on axioms: [propxt] -/
4406 #guard_msgs in #print axioms HALT.first_failure_terminal_8
4407 /-- info: 'HALT.first_failure_terminal_12' depends on axioms: [propxt] -/
4408 #guard_msgs in #print axioms HALT.first_failure_terminal_12
4409 /-- info: 'HALT.halt_free_iff_all_pass_8' depends on axioms: [propxt] -/
4410 #guard_msgs in #print axioms HALT.halt_free_iff_all_pass_8
4411 /-- info: 'HALT.gate_counts_recorded' depends on axioms: [propxt] -/
4412 #guard_msgs in #print axioms HALT.gate_counts_recorded
4413 /-- info: 'HALT.signPatterns' does not depend on any axioms -/
4414 #guard_msgs in #print axioms HALT.signPatterns
4415 /-- info: 'HALT.ninth_gate_barred' does not depend on any axioms -/
4416 #guard_msgs in #print axioms HALT.ninth_gate_barred
4417 /-- info: 'HALT.route' does not depend on any axioms -/
4418 #guard_msgs in #print axioms HALT.route
4419 /-- info: 'HALT.ofUC' does not depend on any axioms -/
4420 #guard_msgs in #print axioms HALT.ofUC
4421 /-- info: 'HALT.route_is_routeHalt' depends on axioms: [propxt, Quot.sound] -/
4422 #guard_msgs in #print axioms HALT.route_is_routeHalt
4423 /-- info: 'HALT.route_samples' does not depend on any axioms -/
4424 #guard_msgs in #print axioms HALT.route_samples
4425 /-- info: 'HALT.tokenBranch' does not depend on any axioms -/
4426 #guard_msgs in #print axioms HALT.tokenBranch
4427 /-- info: 'HALT.router_exclusive' does not depend on any axioms -/
4428 #guard_msgs in #print axioms HALT.router_exclusive
4429 /-- info: 'HALT.readingRoadsGate' does not depend on any axioms -/
4430 #guard_msgs in #print axioms HALT.readingRoadsGate
4431 /-- info: 'HALT.one_door_open' does not depend on any axioms -/
4432 #guard_msgs in #print axioms HALT.one_door_open
4433 /-- info: 'HALT.no_walk_passes_the_aperture' depends on axioms: [propxt] -/
4434 #guard_msgs in #print axioms HALT.no_walk_passes_the_aperture
4435 /-- info: 'HALT.xiGates' does not depend on any axioms -/
4436 #guard_msgs in #print axioms HALT.xiGates
4437 /-- info: 'HALT.oGates' does not depend on any axioms -/
4438 #guard_msgs in #print axioms HALT.oGates
4439 /-- info: 'HALT.gate_censuses' does not depend on any axioms -/
4440 #guard_msgs in #print axioms HALT.gate_censuses
4441 /-- info: 'HALT.rhHalt' does not depend on any axioms -/
4442 #guard_msgs in #print axioms HALT.rhHalt
4443 /-- info: 'HALT.pnpHalt' does not depend on any axioms -/
4444 #guard_msgs in #print axioms HALT.pnpHalt
4445 /-- info: 'HALT.halts_routed' does not depend on any axioms -/
4446 #guard_msgs in #print axioms HALT.halts_routed
4447 /-- info: 'HALT.legacy_halts_are_instances' does not depend on any axioms -/
4448 #guard_msgs in #print axioms HALT.legacy_halts_are_instances
4449 /-- info: 'HALT.terminal_one_bit' does not depend on any axioms -/
4450 #guard_msgs in #print axioms HALT.terminal_one_bit
4451 /-- info: 'HALT.species_exhausted' does not depend on any axioms -/
4452 #guard_msgs in #print axioms HALT.species_exhausted
4453 /-- info: 'HALT.haltGrades' does not depend on any axioms -/
4454 #guard_msgs in #print axioms HALT.haltGrades
4455 /-- info: 'HALT.ftoe_ceiling' does not depend on any axioms -/
4456 #guard_msgs in #print axioms HALT.ftoe_ceiling
4457 /-- info: 'CROSSING.ExecFrame' does not depend on any axioms -/
4458 #guard_msgs in #print axioms CROSSING.ExecFrame
4459 /-- info: 'CROSSING.execFlip' does not depend on any axioms -/
4460 #guard_msgs in #print axioms CROSSING.execFlip
4461 /-- info: 'CROSSING.formalRead' does not depend on any axioms -/
4462 #guard_msgs in #print axioms CROSSING.formalRead
4463 /-- info: 'CROSSING.ran' does not depend on any axioms -/
4464 #guard_msgs in #print axioms CROSSING.ran
4465 /-- info: 'CROSSING.formalRead_even' does not depend on any axioms -/
4466 #guard_msgs in #print axioms CROSSING.formalRead_even
4467 /-- info: 'CROSSING.ran_wholly_odd' does not depend on any axioms -/
4468 #guard_msgs in #print axioms CROSSING.ran_wholly_odd
4469 /-- info: 'CROSSING.execFlip_involution' does not depend on any axioms -/
4470 #guard_msgs in #print axioms CROSSING.execFlip_involution
4471 /-- info: 'CROSSING.ran_odd_at_true' does not depend on any axioms -/
4472 #guard_msgs in #print axioms CROSSING.ran_odd_at_true
4473 /-- info: 'CROSSING.wall' does not depend on any axioms -/
4474 #guard_msgs in #print axioms CROSSING.wall
4475 /-- info: 'CROSSING.deed' does not depend on any axioms -/
4476 #guard_msgs in #print axioms CROSSING.deed
4477 /-- info: 'CROSSING.price' depends on axioms: [Quot.sound] -/
4478 #guard_msgs in #print axioms CROSSING.price
4479 /-- info: 'CROSSING.crossing' does not depend on any axioms -/
4480 #guard_msgs in #print axioms CROSSING.crossing
4481 /-- info: 'CROSSING.identity_one_bit' does not depend on any axioms -/
4482 #guard_msgs in #print axioms CROSSING.identity_one_bit
4483 /-- info: 'CROSSING.fibre4' does not depend on any axioms -/
4484 #guard_msgs in #print axioms CROSSING.fibre4
4485 /-- info: 'CROSSING.fibre4_sixteen' does not depend on any axioms -/
4486 #guard_msgs in #print axioms CROSSING.fibre4_sixteen
4487 /-- info: 'CROSSING.fibre4_distinct' does not depend on any axioms -/
4488 #guard_msgs in #print axioms CROSSING.fibre4_distinct
4489 /-- info: 'CROSSING.price_on_this_file' depends on axioms: [Quot.sound] -/

```

```

4490 #guard_msgs in #print axioms CROSSING.price_on_this_file
4491 /-- info: 'CROSSING.formal_never_odd' does not depend on any axioms -/
4492 #guard_msgs in #print axioms CROSSING.formal_never_odd
4493 /-- info: 'CROSSING.constant_no_crossing' does not depend on any axioms -/
4494 #guard_msgs in #print axioms CROSSING.constant_no_crossing
4495 /-- info: 'CROSSING.harmony' depends on axioms: [propext] -/
4496 #guard_msgs in #print axioms CROSSING.harmony
4497 /-- info: 'LOCUS.Separates' does not depend on any axioms -/
4498 #guard_msgs in #print axioms LOCUS.Separates
4499 /-- info: 'LOCUS.Factors' does not depend on any axioms -/
4500 #guard_msgs in #print axioms LOCUS.Factors
4501 /-- info: 'LOCUS.wall_of_sep' does not depend on any axioms -/
4502 #guard_msgs in #print axioms LOCUS.wall_of_sep
4503 /-- info: 'LOCUS.swap' does not depend on any axioms -/
4504 #guard_msgs in #print axioms LOCUS.swap
4505 /-- info: 'LOCUS.swap_involution' depends on axioms: [propext] -/
4506 #guard_msgs in #print axioms LOCUS.swap_involution
4507 /-- info: 'LOCUS.swap_record_even' depends on axioms: [propext] -/
4508 #guard_msgs in #print axioms LOCUS.swap_record_even
4509 /-- info: 'LOCUS.swap_flips_pair' depends on axioms: [propext] -/
4510 #guard_msgs in #print axioms LOCUS.swap_flips_pair
4511 /-- info: 'LOCUS.seat_of_sep' depends on axioms: [propext] -/
4512 #guard_msgs in #print axioms LOCUS.seat_of_sep
4513 /-- info: 'LOCUS.k4_separated_pair' does not depend on any axioms -/
4514 #guard_msgs in #print axioms LOCUS.k4_separated_pair
4515 /-- info: 'LOCUS.k4_wall_of_pair' does not depend on any axioms -/
4516 #guard_msgs in #print axioms LOCUS.k4_wall_of_pair
4517 /-- info: 'LOCUS.readout' does not depend on any axioms -/
4518 #guard_msgs in #print axioms LOCUS.readout
4519 /-- info: 'LOCUS.factors_of_not_sep_list' depends on axioms: [propext, Quot.sound] -/
4520 #guard_msgs in #print axioms LOCUS.factors_of_not_sep_list
4521 /-- info: 'LOCUS.factors_iff_not_sep_list' depends on axioms: [propext, Quot.sound] -/
4522 #guard_msgs in #print axioms LOCUS.factors_iff_not_sep_list
4523 /-- info: 'LOCUS.Channel' does not depend on any axioms -/
4524 #guard_msgs in #print axioms LOCUS.Channel
4525 /-- info: 'LOCUS.Token.toEconomy' does not depend on any axioms -/
4526 #guard_msgs in #print axioms LOCUS.Token.toEconomy
4527 /-- info: 'LOCUS.Reading.orbits' does not depend on any axioms -/
4528 #guard_msgs in #print axioms LOCUS.Reading.orbits
4529 /-- info: 'LOCUS.Reading.owed' does not depend on any axioms -/
4530 #guard_msgs in #print axioms LOCUS.Reading.owed
4531 /-- info: 'LOCUS.Reading.complete' does not depend on any axioms -/
4532 #guard_msgs in #print axioms LOCUS.Reading.complete
4533 /-- info: 'LOCUS.branch0f' does not depend on any axioms -/
4534 #guard_msgs in #print axioms LOCUS.branch0f
4535 /-- info: 'LOCUS.emit' does not depend on any axioms -/
4536 #guard_msgs in #print axioms LOCUS.emit
4537 /-- info: 'LOCUS.Reading.state' does not depend on any axioms -/
4538 #guard_msgs in #print axioms LOCUS.Reading.state
4539 /-- info: 'LOCUS.three_states' depends on axioms: [propext] -/
4540 #guard_msgs in #print axioms LOCUS.three_states
4541 /-- info: 'LOCUS.no_seat_no_locus' depends on axioms: [propext] -/
4542 #guard_msgs in #print axioms LOCUS.no_seat_no_locus
4543 /-- info: 'LOCUS.unlocated_is_plain_open' depends on axioms: [propext] -/
4544 #guard_msgs in #print axioms LOCUS.unlocated_is_plain_open
4545 /-- info: 'LOCUS.located_never_silent' depends on axioms: [propext] -/
4546 #guard_msgs in #print axioms LOCUS.located_never_silent
4547 /-- info: 'LOCUS.crossed_requires_complete' depends on axioms: [propext] -/
4548 #guard_msgs in #print axioms LOCUS.crossed_requires_complete
4549 /-- info: 'LOCUS.crossed_requires_channel' depends on axioms: [propext] -/
4550 #guard_msgs in #print axioms LOCUS.crossed_requires_channel
4551 /-- info: 'LOCUS.halt_carries_owed' depends on axioms: [propext] -/
4552 #guard_msgs in #print axioms LOCUS.halt_carries_owed
4553 /-- info: 'LOCUS.unchannelled_is_halt' depends on axioms: [propext, Quot.sound] -/
4554 #guard_msgs in #print axioms LOCUS.unchannelled_is_halt
4555 /-- info: 'LOCUS.halt_reports_owed' depends on axioms: [propext] -/
4556 #guard_msgs in #print axioms LOCUS.halt_reports_owed
4557 /-- info: 'LOCUS.owed_zero_is_unchannelled' depends on axioms: [propext, Quot.sound] -/
4558 #guard_msgs in #print axioms LOCUS.owed_zero_is_unchannelled
4559 /-- info: 'LOCUS.one_bit_apart' does not depend on any axioms -/
4560 #guard_msgs in #print axioms LOCUS.one_bit_apart
4561 /-- info: 'LOCUS.legacy_xi' does not depend on any axioms -/
4562 #guard_msgs in #print axioms LOCUS.legacy_xi
4563 /-- info: 'LOCUS.legacy_o' does not depend on any axioms -/
4564 #guard_msgs in #print axioms LOCUS.legacy_o
4565 /-- info: 'LOCUS.legacy_bare' does not depend on any axioms -/
4566 #guard_msgs in #print axioms LOCUS.legacy_bare
4567 /-- info: 'LOCUS.k4_locus' does not depend on any axioms -/
4568 #guard_msgs in #print axioms LOCUS.k4_locus
4569 /-- info: 'LOCUS.slotLists' does not depend on any axioms -/
4570 #guard_msgs in #print axioms LOCUS.slotLists
4571 /-- info: 'LOCUS.slotLists_twelve' does not depend on any axioms -/
4572 #guard_msgs in #print axioms LOCUS.slotLists_twelve
4573 /-- info: 'LOCUS.slotLists_pos' does not depend on any axioms -/
4574 #guard_msgs in #print axioms LOCUS.slotLists_pos
4575 /-- info: 'LOCUS.allReadings' does not depend on any axioms -/
4576 #guard_msgs in #print axioms LOCUS.allReadings
4577 /-- info: 'LOCUS.reading_space' does not depend on any axioms -/
4578 #guard_msgs in #print axioms LOCUS.reading_space
4579 /-- info: 'LOCUS.census_no_seat_no_locus' does not depend on any axioms -/
4580 #guard_msgs in #print axioms LOCUS.census_no_seat_no_locus
4581 /-- info: 'LOCUS.census_located_never_silent' does not depend on any axioms -/
4582 #guard_msgs in #print axioms LOCUS.census_located_never_silent
4583 /-- info: 'LOCUS.census_crossed_iff_complete_channelled' does not depend on any axioms
4584 /-
4584 #guard_msgs in #print axioms LOCUS.census_crossed_iff_complete_channelled
4585 /-- info: 'LOCUS.census_halt_reports_owed' does not depend on any axioms -/
4586 #guard_msgs in #print axioms LOCUS.census_halt_reports_owed
4587 /-- info: 'LOCUS.census_owed_zero_is_unchannelled' does not depend on any axioms -/
4588 #guard_msgs in #print axioms LOCUS.census_owed_zero_is_unchannelled
4589 /-- info: 'LOCUS.census_three_states' does not depend on any axioms -/
4590 #guard_msgs in #print axioms LOCUS.census_three_states
4591 /-- info: 'LOCUS.Token.render' does not depend on any axioms -/
4592 #guard_msgs in #print axioms LOCUS.Token.render
4593 /-- info: 'LOCUS.Token.ascii' does not depend on any axioms -/
4594 #guard_msgs in #print axioms LOCUS.Token.ascii

```

```

4595 /-- info: 'LOCUS.sealAscii' does not depend on any axioms -/
4596 #guard_msgs in #print axioms LOCUS.sealAscii
4597 /-- info: 'LOCUS.render_receipted' does not depend on any axioms -/
4598 #guard_msgs in #print axioms LOCUS.render_receipted
4599 /-- info: 'LOCUS.render_injective_in_owed' does not depend on any axioms -/
4600 #guard_msgs in #print axioms LOCUS.render_injective_in_owed
4601 /-- info: 'LOCUS(voidImage)' does not depend on any axioms -/
4602 #guard_msgs in #print axioms LOCUS(voidImage
4603 /-- info: 'LOCUS(void_precedes_economy' does not depend on any axioms -/
4604 #guard_msgs in #print axioms LOCUS(void_precedes_economy
4605 /-- info: 'LOCUS.legacy_records_not_promoted' does not depend on any axioms -/
4606 #guard_msgs in #print axioms LOCUS.legacy_records_not_promoted
4607 /-- info: 'LOCUS.no_rh_locus' does not depend on any axioms -/
4608 #guard_msgs in #print axioms LOCUS.no_rh_locus
4609 /-- info: 'PSP.ORIENT.V3' depends on axioms: [propext] -/
4610 #guard_msgs in #print axioms PSP.ORIENT.V3
4611 /-- info: 'PSP.ORIENT.M3' depends on axioms: [propext] -/
4612 #guard_msgs in #print axioms PSP.ORIENT.M3
4613 /-- info: 'PSP.ORIENT.det3' depends on axioms: [propext] -/
4614 #guard_msgs in #print axioms PSP.ORIENT.det3
4615 /-- info: 'PSP.ORIENT.neg3' depends on axioms: [propext] -/
4616 #guard_msgs in #print axioms PSP.ORIENT.neg3
4617 /-- info: 'PSP.ORIENT.signMatrices' depends on axioms: [propext] -/
4618 #guard_msgs in #print axioms PSP.ORIENT.signMatrices
4619 /-- info: 'PSP.ORIENT.lock_scalar_sign_blind' depends on axioms: [propext] -/
4620 #guard_msgs in #print axioms PSP.ORIENT.lock_scalar_sign_blind
4621 /-- info: 'PSP.ORIENT.det_sign_flips_in_odd_dimension' depends on axioms: [propext] -/
4622 #guard_msgs in #print axioms PSP.ORIENT.det_sign_flips_in_odd_dimension
4623 /-- info: 'PSP.QUAT.Q' depends on axioms: [propext] -/
4624 #guard_msgs in #print axioms PSP.QUAT.Q
4625 /-- info: 'PSP.QUAT.qmul' depends on axioms: [propext] -/
4626 #guard_msgs in #print axioms PSP.QUAT.qmul
4627 /-- info: 'PSP.QUAT.qi' depends on axioms: [propext] -/
4628 #guard_msgs in #print axioms PSP.QUAT.qi
4629 /-- info: 'PSP.QUAT.qj' depends on axioms: [propext] -/
4630 #guard_msgs in #print axioms PSP.QUAT.qj
4631 /-- info: 'PSP.QUAT.qk' depends on axioms: [propext] -/
4632 #guard_msgs in #print axioms PSP.QUAT.qk
4633 /-- info: 'PSP.QUAT.qneg1' depends on axioms: [propext] -/
4634 #guard_msgs in #print axioms PSP.QUAT.qneg1
4635 /-- info: 'PSP.QUAT.quat_basis_laws' depends on axioms: [propext] -/
4636 #guard_msgs in #print axioms PSP.QUAT.quat_basis_laws
4637 /-- info: 'PSP.QUAT.qnorm' depends on axioms: [propext] -/
4638 #guard_msgs in #print axioms PSP.QUAT.qnorm
4639 /-- info: 'PSP.QUAT.isInt' depends on axioms: [propext] -/
4640 #guard_msgs in #print axioms PSP.QUAT.isInt
4641 /-- info: 'PSP.QUAT.isHalf' depends on axioms: [propext] -/
4642 #guard_msgs in #print axioms PSP.QUAT.isHalf
4643 /-- info: 'PSP.QUAT.hurwitzOK' depends on axioms: [propext] -/
4644 #guard_msgs in #print axioms PSP.QUAT.hurwitzOK
4645 /-- info: 'PSP.QUAT.grid' depends on axioms: [propext] -/
4646 #guard_msgs in #print axioms PSP.QUAT.grid
4647 /-- info: 'PSP.QUAT.tuples4' depends on axioms: [propext] -/
4648 #guard_msgs in #print axioms PSP.QUAT.tuples4
4649 /-- info: 'PSP.QUAT.hurwitz_units_24' depends on axioms: [propext] -/
4650 #guard_msgs in #print axioms PSP.QUAT.hurwitz_units_24
4651 /-- info: 'PSP.QUAT.grid2' depends on axioms: [propext] -/
4652 #guard_msgs in #print axioms PSP.QUAT.grid2
4653 /-- info: 'PSP.QUAT.tuples4i' depends on axioms: [propext] -/
4654 #guard_msgs in #print axioms PSP.QUAT.tuples4i
4655 /-- info: 'PSP.QUAT.norm2_shell_split' depends on axioms: [propext] -/
4656 #guard_msgs in #print axioms PSP.QUAT.norm2_shell_split
4657 /-- info: 'PSP.CD.cdconj' depends on axioms: [propext] -/
4658 #guard_msgs in #print axioms PSP.CD.cdconj
4659 /-- info: 'PSP.CD.padd' depends on axioms: [propext] -/
4660 #guard_msgs in #print axioms PSP.CD.padd
4661 /-- info: 'PSP.CD.psub' depends on axioms: [propext] -/
4662 #guard_msgs in #print axioms PSP.CD.psub
4663 /-- info: 'PSP.CD.cdmul' depends on axioms: [propext] -/
4664 #guard_msgs in #print axioms PSP.CD.cdmul
4665 /-- info: 'PSP.CD.e' depends on axioms: [propext] -/
4666 #guard_msgs in #print axioms PSP.CD.e
4667 /-- info: 'PSP.CD.smul' depends on axioms: [propext] -/
4668 #guard_msgs in #print axioms PSP.CD.smul
4669 /-- info: 'PSP.CD.octonion_associator' depends on axioms: [propext] -/
4670 #guard_msgs in #print axioms PSP.CD.octonion_associator
4671 /-- info: 'PSP.CD.sx' depends on axioms: [propext] -/
4672 #guard_msgs in #print axioms PSP.CD.sx
4673 /-- info: 'PSP.CD.sy' depends on axioms: [propext] -/
4674 #guard_msgs in #print axioms PSP.CD.sy
4675 /-- info: 'PSP.CD.zero16' depends on axioms: [propext] -/
4676 #guard_msgs in #print axioms PSP.CD.zero16
4677 /-- info: 'PSP.CD.sqnorm' depends on axioms: [propext] -/
4678 #guard_msgs in #print axioms PSP.CD.sqnorm
4679 /-- info: 'PSP.CD.sedenion_zero_divisor' depends on axioms: [propext] -/
4680 #guard_msgs in #print axioms PSP.CD.sedenion_zero_divisor
4681 /-- info: 'PSP.CASCADE.cascade' does not depend on any axioms -/
4682 #guard_msgs in #print axioms PSP.CASCADE.cascade
4683 /-- info: 'PSP.CASCADE.patterns8' depends on axioms: [propext] -/
4684 #guard_msgs in #print axioms PSP.CASCADE.patterns8
4685 /-- info: 'PSP.CASCADE.spec' does not depend on any axioms -/
4686 #guard_msgs in #print axioms PSP.CASCADE.spec
4687 /-- info: 'PSP.CASCADE.first_failure_terminal_8' depends on axioms: [propext] -/
4688 #guard_msgs in #print axioms PSP.CASCADE.first_failure_terminal_8
4689 /-- info: 'PSP.CASCADE.halt_free_iff_all_pass_8' depends on axioms: [propext] -/
4690 #guard_msgs in #print axioms PSP.CASCADE.halt_free_iff_all_pass_8
4691 /-- info: 'PSP.GROUNDLESS.o0Neighbors' does not depend on any axioms -/
4692 #guard_msgs in #print axioms PSP.GROUNDLESS.o0Neighbors
4693 /-- info: 'PSP.GROUNDLESS.fiveCubeCoords' does not depend on any axioms -/
4694 #guard_msgs in #print axioms PSP.GROUNDLESS.fiveCubeCoords
4695 /-- info: 'PSP.GROUNDLESS.deg5' depends on axioms: [propext] -/
4696 #guard_msgs in #print axioms PSP.GROUNDLESS.deg5
4697 /-- info: 'PSP.GROUNDLESS.groundless_cascade_degree' depends on axioms: [propext] -/
4698 #guard_msgs in #print axioms PSP.GROUNDLESS.groundless_cascade_degree
4699 /-- info: 'PSP.TOKEN.level' does not depend on any axioms -/
4700 #guard_msgs in #print axioms PSP.TOKEN.level

```

```

4701 /-- info: 'PSP.TOKEN.terminal_tokens_level_and_distinct' does not depend on any axioms
4702 /--
4703 #guard_msgs in #print axioms PSP.TOKEN.terminal_tokens_level_and_distinct
4704 /-- info: 'PSP.TWOGROUP.two_group_census' does not depend on any axioms -/
4705 #guard_msgs in #print axioms PSP.TWOGROUP.two_group_census
4706 /-- ## the axiom cones, pinned -/
4707 /-- info: 'Bridge.pe' does not depend on any axioms -/
4708 #guard_msgs in #print axioms Bridge.pe
4709 /-- info: 'Bridge.fold_involutive' depends on axioms: [propext] -/
4710 #guard_msgs in #print axioms Bridge.fold_involutive
4711 /-- info: 'Bridge.locus_is_the_fixed_set' depends on axioms: [propext, Classical.choice,
4712   Quot.sound] -/
4713 #guard_msgs in #print axioms Bridge.locus_is_the_fixed_set
4714 /-- info: 'Bridge.locus_inhabited' does not depend on any axioms -/
4715 #guard_msgs in #print axioms Bridge.locus_inhabited
4716 /-- info: 'Bridge.line_symmetric' depends on axioms: [propext, Classical.choice, Quot.
4717   sound] -/
4718 #guard_msgs in #print axioms Bridge.line_symmetric
4719 /-- info: 'Bridge.plane_line_property' depends on axioms: [propext, Classical.choice,
4720   Quot.sound] -/
4721 #guard_msgs in #print axioms Bridge.plane_line_property
4722 /-- info: 'Bridge.cannot_lie' does not depend on any axioms -/
4723 #guard_msgs in #print axioms Bridge.cannot_lie
4724 /-- info: 'Bridge.cannot_deviate' does not depend on any axioms -/
4725 #guard_msgs in #print axioms Bridge.cannot_deviate
4726 /-- info: 'Bridge.read_twice' does not depend on any axioms -/
4727 #guard_msgs in #print axioms Bridge.read_twice
4728 /-- info: 'Bridge.halted_iff' does not depend on any axioms -/
4729 #guard_msgs in #print axioms Bridge.halted_iff
4730 /-- info: 'Bridge.cannot_extend' depends on axioms: [propext, Quot.sound] -/
4731 #guard_msgs in #print axioms Bridge.cannot_extend
4732 /-- info: 'Bridge.cannot_divide' depends on axioms: [propext] -/
4733 #guard_msgs in #print axioms Bridge.cannot_divide
4734 /-- info: 'Bridge.cannot_reverse' does not depend on any axioms -/
4735 #guard_msgs in #print axioms Bridge.cannot_reverse
4736 /-- info: 'Bridge.wall' does not depend on any axioms -/
4737 #guard_msgs in #print axioms Bridge.wall
4738 /-- info: 'Bridge.calibration' does not depend on any axioms -/
4739 #guard_msgs in #print axioms Bridge.calibration
4740 /-- info: 'Bridge.opens_on_the_deed' depends on axioms: [propext, Classical.choice, Quot.
4741   sound] -/
4742 #guard_msgs in #print axioms Bridge.opens_on_the_deed
4743 /-- info: 'Bridge.recursion_is_shared' depends on axioms: [propext, Classical.choice,
4744   Quot.sound] -/
4745 #guard_msgs in #print axioms Bridge.recursion_is_shared
4746 /-- info: 'Bridge.recursion_is_sign_blind' depends on axioms: [propext, Classical.choice,
4747   Quot.sound] -/
4748 #guard_msgs in #print axioms Bridge.recursion_is_sign_blind
4749 /-- info: 'Bridge.exactly_one_stands' depends on axioms: [propext, Classical.choice,
4750   Quot.sound] -/
4751 #guard_msgs in #print axioms Bridge.exactly_one_stands
4752 /-- info: 'Bridge.denial_is_coherent' does not depend on any axioms -/
4753 #guard_msgs in #print axioms Bridge.denial_is_coherent
4754 /-- info: 'Bridge.line_property_contingent' depends on axioms: [propext, Classical.
4755   choice, Quot.sound] -/
4756 #guard_msgs in #print axioms Bridge.line_property_contingent
4757 /-- info: 'Bridge.keyless_iff_valid' depends on axioms: [propext, Classical.choice, Quot.
4758   sound] -/
4759 #guard_msgs in #print axioms Bridge.keyless_iff_valid
4760 /-- info: 'Bridge.discriminator' depends on axioms: [propext, Classical.choice, Quot.
4761   sound] -/
4762 #guard_msgs in #print axioms Bridge.discriminator
4763 /-- info: 'Bridge.symmetry_is_keyless' does not depend on any axioms -/
4764 #guard_msgs in #print axioms Bridge.symmetry_is_keyless
4765 /-- info: 'Bridge.line_property_is_keyed' depends on axioms: [propext, Classical.choice,
4766   Quot.sound] -/
4767 #guard_msgs in #print axioms Bridge.line_property_is_keyed
4768 /-- info: 'Bridge.denial_asymmetry' depends on axioms: [propext, Classical.choice, Quot.
4769   sound] -/
4770 #guard_msgs in #print axioms Bridge.denial_asymmetry
4771 /-- info: 'Bridge.socket_is_the_property' does not depend on any axioms -/
4772 #guard_msgs in #print axioms Bridge.socket_is_the_property
4773 /-- info: 'Bridge.socket_on_the_plane' depends on axioms: [propext, Classical.choice,
4774   Quot.sound] -/
4775 #guard_msgs in #print axioms Bridge.socket_on_the_plane
4776 /-- info: 'Bridge.bridge_carries' does not depend on any axioms -/
4777 #guard_msgs in #print axioms Bridge.bridge_carries
4778 /-- info: 'Bridge.no_socket_off_line' does not depend on any axioms -/
4779 #guard_msgs in #print axioms Bridge.no_socket_off_line
4780 /-- info: 'Bridge.the_bridge' depends on axioms: [propext, Classical.choice, Quot.sound]
4781 /--
4782 #guard_msgs in #print axioms Bridge.the_bridge
4783 /-- info: 'Bridge.Plane' does not depend on any axioms -/
4784 #guard_msgs in #print axioms Bridge.Plane
4785 /-- info: 'Bridget.' does not depend on any axioms -/
4786 #guard_msgs in #print axioms Bridget.
4787 /-- info: 'Bridge.onLine' does not depend on any axioms -/
4788 #guard_msgs in #print axioms Bridge.onLine
4789 /-- info: 'Bridge.LineProperty' does not depend on any axioms -/
4790 #guard_msgs in #print axioms Bridge.LineProperty
4791 /-- info: 'Bridge.planeFrame' depends on axioms: [propext] -/
4792 #guard_msgs in #print axioms Bridge.planeFrame
4793 /-- info: 'Bridge.lineFrame' depends on axioms: [propext, Classical.choice, Quot.sound]
4794 /--
4795 #guard_msgs in #print axioms Bridge.lineFrame
4796 /-- info: 'Bridge.theRecord' does not depend on any axioms -/
4797 #guard_msgs in #print axioms Bridge.theRecord
4798 /-- info: 'Bridge.delete' does not depend on any axioms -/
4799 #guard_msgs in #print axioms Bridge.delete
4800 /-- info: 'Bridge.Even' does not depend on any axioms -/
4801 #guard_msgs in #print axioms Bridge.Even
4802 /-- info: 'Bridge.SelfVerifying' does not depend on any axioms -/
4803 #guard_msgs in #print axioms Bridge.SelfVerifying
4804 /-- info: 'Bridge.twoPoint' does not depend on any axioms -/
4805 #guard_msgs in #print axioms Bridge.twoPoint
4806 /--
4807 #guard_msgs in #print axioms Bridge.twoPoint
4808 /--
4809 #guard_msgs in #print axioms Bridge.twoPoint
4810 /--
4811 #guard_msgs in #print axioms Bridge.twoPoint
4812 /--
4813 #guard_msgs in #print axioms Bridge.twoPoint
4814 /--
4815 #guard_msgs in #print axioms Bridge.twoPoint
4816 /--
4817 #guard_msgs in #print axioms Bridge.twoPoint
4818 /--
4819 #guard_msgs in #print axioms Bridge.twoPoint
4820 /--
4821 #guard_msgs in #print axioms Bridge.twoPoint
4822 /--
4823 #guard_msgs in #print axioms Bridge.twoPoint
4824 /--
4825 #guard_msgs in #print axioms Bridge.twoPoint
4826 /--
4827 #guard_msgs in #print axioms Bridge.twoPoint
4828 /--
4829 #guard_msgs in #print axioms Bridge.twoPoint
4830 /--
4831 #guard_msgs in #print axioms Bridge.twoPoint
4832 /--
4833 #guard_msgs in #print axioms Bridge.twoPoint
4834 /--
4835 #guard_msgs in #print axioms Bridge.twoPoint
4836 /--
4837 #guard_msgs in #print axioms Bridge.twoPoint
4838 /--
4839 #guard_msgs in #print axioms Bridge.twoPoint
4840 /--
4841 #guard_msgs in #print axioms Bridge.twoPoint
4842 /--
4843 #guard_msgs in #print axioms Bridge.twoPoint
4844 /--
4845 #guard_msgs in #print axioms Bridge.twoPoint
4846 /--
4847 #guard_msgs in #print axioms Bridge.twoPoint
4848 /--
4849 #guard_msgs in #print axioms Bridge.twoPoint
4850 /--
4851 #guard_msgs in #print axioms Bridge.twoPoint
4852 /--
4853 #guard_msgs in #print axioms Bridge.twoPoint
4854 /--
4855 #guard_msgs in #print axioms Bridge.twoPoint
4856 /--
4857 #guard_msgs in #print axioms Bridge.twoPoint
4858 /--
4859 #guard_msgs in #print axioms Bridge.twoPoint
4860 /--
4861 #guard_msgs in #print axioms Bridge.twoPoint
4862 /--
4863 #guard_msgs in #print axioms Bridge.twoPoint
4864 /--
4865 #guard_msgs in #print axioms Bridge.twoPoint
4866 /--
4867 #guard_msgs in #print axioms Bridge.twoPoint
4868 /--
4869 #guard_msgs in #print axioms Bridge.twoPoint
4870 /--
4871 #guard_msgs in #print axioms Bridge.twoPoint
4872 /--
4873 #guard_msgs in #print axioms Bridge.twoPoint
4874 /--
4875 #guard_msgs in #print axioms Bridge.twoPoint
4876 /--
4877 #guard_msgs in #print axioms Bridge.twoPoint
4878 /--
4879 #guard_msgs in #print axioms Bridge.twoPoint
4880 /--
4881 #guard_msgs in #print axioms Bridge.twoPoint
4882 /--
4883 #guard_msgs in #print axioms Bridge.twoPoint
4884 /--
4885 #guard_msgs in #print axioms Bridge.twoPoint
4886 /--
4887 #guard_msgs in #print axioms Bridge.twoPoint
4888 /--
4889 #guard_msgs in #print axioms Bridge.twoPoint
4890 /--
4891 #guard_msgs in #print axioms Bridge.twoPoint
4892 /--
4893 #guard_msgs in #print axioms Bridge.twoPoint
4894 /--
4895 #guard_msgs in #print axioms Bridge.twoPoint
4896 /--
4897 #guard_msgs in #print axioms Bridge.twoPoint
4898 /--
4899 #guard_msgs in #print axioms Bridge.twoPoint
4900 /--
4901 #guard_msgs in #print axioms Bridge.twoPoint
4902 /--
4903 #guard_msgs in #print axioms Bridge.twoPoint
4904 /--
4905 #guard_msgs in #print axioms Bridge.twoPoint
4906 /--
4907 #guard_msgs in #print axioms Bridge.twoPoint
4908 /--
4909 #guard_msgs in #print axioms Bridge.twoPoint
4910 /--
4911 #guard_msgs in #print axioms Bridge.twoPoint
4912 /--
4913 #guard_msgs in #print axioms Bridge.twoPoint
4914 /--
4915 #guard_msgs in #print axioms Bridge.twoPoint
4916 /--
4917 #guard_msgs in #print axioms Bridge.twoPoint
4918 /--
4919 #guard_msgs in #print axioms Bridge.twoPoint
4920 /--
4921 #guard_msgs in #print axioms Bridge.twoPoint
4922 /--
4923 #guard_msgs in #print axioms Bridge.twoPoint
4924 /--
4925 #guard_msgs in #print axioms Bridge.twoPoint
4926 /--
4927 #guard_msgs in #print axioms Bridge.twoPoint
4928 /--
4929 #guard_msgs in #print axioms Bridge.twoPoint
4930 /--
4931 #guard_msgs in #print axioms Bridge.twoPoint
4932 /--
4933 #guard_msgs in #print axioms Bridge.twoPoint
4934 /--
4935 #guard_msgs in #print axioms Bridge.twoPoint
4936 /--
4937 #guard_msgs in #print axioms Bridge.twoPoint
4938 /--
4939 #guard_msgs in #print axioms Bridge.twoPoint
4940 /--
4941 #guard_msgs in #print axioms Bridge.twoPoint
4942 /--
4943 #guard_msgs in #print axioms Bridge.twoPoint
4944 /--
4945 #guard_msgs in #print axioms Bridge.twoPoint
4946 /--
4947 #guard_msgs in #print axioms Bridge.twoPoint
4948 /--
4949 #guard_msgs in #print axioms Bridge.twoPoint
4950 /--
4951 #guard_msgs in #print axioms Bridge.twoPoint
4952 /--
4953 #guard_msgs in #print axioms Bridge.twoPoint
4954 /--
4955 #guard_msgs in #print axioms Bridge.twoPoint
4956 /--
4957 #guard_msgs in #print axioms Bridge.twoPoint
4958 /--
4959 #guard_msgs in #print axioms Bridge.twoPoint
4960 /--
4961 #guard_msgs in #print axioms Bridge.twoPoint
4962 /--
4963 #guard_msgs in #print axioms Bridge.twoPoint
4964 /--
4965 #guard_msgs in #print axioms Bridge.twoPoint
4966 /--
4967 #guard_msgs in #print axioms Bridge.twoPoint
4968 /--
4969 #guard_msgs in #print axioms Bridge.twoPoint
4970 /--
4971 #guard_msgs in #print axioms Bridge.twoPoint
4972 /--
4973 #guard_msgs in #print axioms Bridge.twoPoint
4974 /--
4975 #guard_msgs in #print axioms Bridge.twoPoint
4976 /--
4977 #guard_msgs in #print axioms Bridge.twoPoint
4978 /--
4979 #guard_msgs in #print axioms Bridge.twoPoint
4980 /--
4981 #guard_msgs in #print axioms Bridge.twoPoint
4982 /--
4983 #guard_msgs in #print axioms Bridge.twoPoint
4984 /--
4985 #guard_msgs in #print axioms Bridge.twoPoint
4986 /--
4987 #guard_msgs in #print axioms Bridge.twoPoint
4988 /--
4989 #guard_msgs in #print axioms Bridge.twoPoint
4990 /--
4991 #guard_msgs in #print axioms Bridge.twoPoint
4992 /--
4993 #guard_msgs in #print axioms Bridge.twoPoint
4994 /--
4995 #guard_msgs in #print axioms Bridge.twoPoint
4996 /--
4997 #guard_msgs in #print axioms Bridge.twoPoint
4998 /--
4999 #guard_msgs in #print axioms Bridge.twoPoint
5000 /--

```

```

4897 let _ := @Audit.compartmentGrade
4898 let _ := @Audit.unknown_routes_open
4899 let _ := @Audit.contentless_refused
4900 let _ := @Audit.det3
4901 let _ := @Audit.det3_two_id
4902 let _ := @INVERSION.ItFromBit
4903 let _ := @INVERSION.it_from_bit_refuted
4904 let _ := @INVERSION.it_from_it
4905 let _ := @K4.it_from_it_canonical
4906 let _ := @K4.it_from_bit_refuted_canonical
4907 let _ := @K4.inversionClaim
4908 let _ := @GE0.qmul
4909 let _ := @GE0.qconj
4910 let _ := @GE0.qadd
4911 let _ := @GE0.qsub
4912 let _ := @GE0.qnorm2
4913 let _ := @GE0.qhalve
4914 let _ := @GE0.qquarter
4915 let _ := @GE0.qcanon
4916 let _ := @GE0.q1
4917 let _ := @GE0.qi
4918 let _ := @GE0.qj
4919 let _ := @GE0.qk
4920 let _ := @GE0.chirality
4921 let _ := @GE0.noncommutative
4922 let _ := @GE0.hurwitz
4923 let _ := @GE0.hurwitz_norm
4924 let _ := @GE0.hurwitz_products_even
4925 let _ := @GE0.hurwitz_closed
4926 let _ := @GE0.hurwitzReps
4927 let _ := @GE0.units_mod_sign_twelve
4928 let _ := @GE0.conjClass
4929 let _ := @GE0.classSizesAux
4930 let _ := @GE0.insNat
4931 let _ := @GE0.insSort
4932 let _ := @GE0.class_equation
4933 let _ := @GE0.shell2
4934 let _ := @GE0.shell_twentyfour
4935 let _ := @GE0.no_half_integer_norm2
4936 let _ := @GE0.allGates
4937 let _ := @GE0.gates_twelve
4938 let _ := @GE0.gates_valid
4939 let _ := @GE0.gates_complete
4940 let _ := @GE0.invCount
4941 let _ := @GE0.perms24
4942 let _ := @GE0.a4
4943 let _ := @GE0.a4_order
4944 let _ := @GE0.actG
4945 let _ := @GE0.a4_simply_transitive
4946 let _ := @GE0.tetraVerts
4947 let _ := @GE0.tetraEdges
4948 let _ := @GE0.tetraFaces
4949 let _ := @GE0.euler_closure
4950 let _ := @GE0.basisU
4951 let _ := @GE0.perms6
4952 let _ := @GE0.relabel_parity
4953 let _ := @GE0.o8
4954 let _ := @GE0.omul
4955 let _ := @GE0.osub
4956 let _ := @GE0.onorm2
4957 let _ := @GE0.e8
4958 let _ := @GE0.octonion_associator
4959 let _ := @GE0.hPairs
4960 let _ := @GE0.norm_composition_H
4961 let _ := @GE0.oPairs
4962 let _ := @GE0.norm_composition_0
4963 let _ := @GE0.det2
4964 let _ := @GE0.gf2Mul
4965 let _ := @GE0.gf2Vecs
4966 let _ := @GE0.gf2Sols
4967 let _ := @GE0.triaxial_lock_gf2
4968 let _ := @GE0.triaxial_lock_gf2_count
4969 let _ := @GE0.triaxial_open_gf2_count
4970 let _ := @GE0.V3
4971 let _ := @GE0.M3
4972 let _ := @GE0.det3
4973 let _ := @GE0.adj3
4974 let _ := @GE0.mm3
4975 let _ := @GE0.diag3
4976 let _ := @GE0.mv3
4977 let _ := @GE0.vscale
4978 let _ := @GE0.V3Z
4979 let _ := @GE0.M3Z
4980 let _ := @GE0.det3Z
4981 let _ := @GE0.adj3Z
4982 let _ := @GE0.mm3Z
4983 let _ := @GE0.diag3Z
4984 let _ := @GE0.adjgate_identity_int
4985 let _ := @GE0.AdjugateIdentity3
4986 let _ := @GE0.adj3Samples
4987 let _ := @GE0.adj3_executed
4988 let _ := @GE0.cramerSolve
4989 let _ := @GE0.cramer_executed
4990 let _ := @GE0.go1Admit
4991 let _ := @GE0.gateNames
4992 let _ := @GE0.gateNames_twelve
4993 let _ := @GE0.gateScreenGo
4994 let _ := @GE0.gateScreen
4995 let _ := @GE0.gateScreen_clean
4996 let _ := @GE0.gateScreen_first_failure
4997 let _ := @GE0.gateScreen_names_failed_gate
4998 let _ := @GE0.gateScreen_arity
4999 let _ := @GE0.ClaimInputRA.anyUnknown
5000 let _ := @GE0.rowCascadeRACore
5001 let _ := @GE0.rowCascadeRA
5002 let _ := @GE0.unknown_routes_open_ra

5003 let _ := @GE0.raDecode
5004 let _ := @GE0.raBattery
5005 let _ := @GE0.ra_battery_f2
5006 let _ := @GE0.ra_ledger_refused
5007 let _ := @GE0.ra_ledger_iii
5008 let _ := @GE0.ra_ledger_ii
5009 let _ := @GE0.ra_ledger_void
5010 let _ := @GE0.ra_ledger_aGivenRA
5011 let _ := @GE0.RAVerdict.toToken
5012 let _ := @GE0.GoToken.toToken
5013 let _ := @GE0.gateScreenToken
5014 let _ := @GE0.demoInputA
5015 let _ := @GE0.demoInputVoid
5016 let _ := @GE0.demoInputOpen
5017 let _ := @ROOT.Actuates
5018 let _ := @ROOT.actuation_universal
5019 let _ := @ROOT.raClaim
5020 let _ := @ROOT.boolCut
5021 let _ := @ROOT.nest_31
5022 let _ := @ROOT.nestClaim
5023 let _ := @ROOT.throneEmpty
5024 let _ := @ROOT.throne_is_RA
5025 let _ := @SEALS.sealL
5026 let _ := @SEALS.sealG
5027 let _ := @SEALS.sealM
5028 let _ := @SEALS.seals_distinct
5029 let _ := @SEALS.seal_names_match
5030 let _ := @SEALS.sealG_walk
5031 let _ := @POSTULATE.OneCut
5032 let _ := @POSTULATE.F2
5033 let _ := @POSTULATE.F2_kills
5034 let _ := @POSTULATE.BSDContract
5035 let _ := @POSTULATE.F6
5036 let _ := @TONGUE.enlarge
5037 let _ := @TONGUE.unclosed
5038 let _ := @TONGUE.earned_freedom
5039 let _ := @TONGUE.colocClaim
5040 let _ := @TONGUE.coloc_grade
5041 let _ := @TONGUE.ra_grade_pinned
5042 let _ := @TONGUE.wall_grade_pinned
5043 let _ := @TONGUE.inversion_grade_pinned
5044 let _ := @TONGUE.nest_grade_pinned
5045 let _ := @terminalWall
5046 let _ := @terminalPrice
5047 let _ := @TERMINAL
5048 let _ := @terminalClaim
5049 let _ := @terminal_grade_pinned
5050 let _ := @IAM.IamToken
5051 let _ := @IAM.Iam_unwitnessed_withheld
5052 let _ := @IAM.Iam_witnessed_speaks
5053 let _ := @IAM.narcissus_truth_table
5054 let _ := @IAM.one_bit_freedom
5055 let _ := @IAM.IamToken.toToken
5056 let _ := @IAM.codexSelfRead
5057 let _ := @IAM.codex_self_read_interior
5058 let _ := @IAM.terminalWord
5059 let _ := @IAM.terminal_word
5060 let _ := @GE0.SignPattern
5061 let _ := @GE0.allSigns
5062 let _ := @GE0.signComp
5063 let _ := @GE0.reflection_eight
5064 let _ := @GE0.reflection_eight_is_two_cubed
5065 let _ := @GE0.reflections_abelian
5066 let _ := @GE0.reflections_involutive
5067 let _ := @GE0.signVal
5068 let _ := @GE0.reflMat
5069 let _ := @GE0.reflection_reverses_orientation
5070 let _ := @GE0.sandwich
5071 let _ := @GE0.reflection_blind_executed
5072 let _ := @GE0.V5
5073 let _ := @GE0.cube5
5074 let _ := @GE0.neighbors5
5075 let _ := @GE0.fivecube_thirtytwo
5076 let _ := @GE0.fivecube_vertices_distinct
5077 let _ := @GE0.fivecube_degree_five
5078 let _ := @GE0.counts_forced
5079 let _ := @TONGUE.deletionOp
5080 let _ := @TONGUE.e3
5081 let _ := @TONGUE.deletion_idempotent
5082 let _ := @TONGUE.deletion_singular
5083 let _ := @TONGUE.deletion_minor_nonsingular
5084 let _ := @TONGUE.deletion_annihilates
5085 let _ := @TONGUE.linear_fixes_zero
5086 let _ := @TONGUE.deletion_no_left_inverse
5087 let _ := @TONGUE.deletion_monoid_not_group
5088 let _ := @TONGUE.tongue_obedience
5089 let _ := @OFFICE.discriminator
5090 let _ := @OFFICE.discriminatorQ2First
5091 let _ := @OFFICE.order_loadbearing
5092 let _ := @OFFICE.citation_cannot_promote
5093 let _ := @UC.routeHalt
5094 let _ := @UC.routeHalt_fidelity
5095 let _ := @UC.universalCascade
5096 let _ := @UC.uc_seal_failure_terminal
5097 let _ := @UC.uc_routes_eight
5098 let _ := @UC.uc_routes_five
5099 let _ := @UC.uc_unmeasured
5100 let _ := @UC.circularityScreen
5101 let _ := @UC.circularity_refused_at_target
5102 let _ := @UC.road_requires_conjunction
5103 let _ := @UC.circRows
5104 let _ := @UC.circ_space
5105 let _ := @OMEGA.kbRat
5106 let _ := @OMEGA.ln2Rat
5107 let _ := @OMEGA.landauerRat
5108 let _ := @OMEGA.omegaBoundary

```

```

5109 let _ := @OMEGA.omega_zero_bits
5110 let _ := @OMEGA.omega_paid_floor
5111 let _ := @OMEGA.one_bit_freedom_priced
5112 let _ := @OMEGA.omega_reversible_branch
5113 let _ := @OMEGA.omega_refuses_nonpositive_temp
5114 let _ := @OMEGA.omega_monotone
5115 let _ := @OMEGA.omega_linear
5116 let _ := @OMEGA.landauer_zero_bits
5117 let _ := @OMEGA.omega_verdict_fidelity
5118 let _ := @AEGIS.refusalConst
5119 let _ := @AEGIS.aegisReset
5120 let _ := @AEGIS.aegisGuard
5121 let _ := @AEGIS.aegis_constant
5122 let _ := @AEGIS.aegis_deed_increments
5123 let _ := @AEGIS.aegis_reads_parameter
5124 let _ := @AEGIS.aegisFourLogicRun
5125 let _ := @AEGIS.aegis_battery
5126 let _ := @NOMOS.allStrings
5127 let _ := @NOMOS.allStrings_length
5128 let _ := @NOMOS.twoPowPos
5129 let _ := @NOMOS.descs
5130 let _ := @NOMOS.descs_length
5131 let _ := @NOMOS.incompressibility_cap
5132 let _ := @NOMOS.nomosRows
5133 let _ := @NOMOS.incompressibility_executed
5134 let _ := @NOMOS.decodeId
5135 let _ := @NOMOS.decodePad
5136 let _ := @NOMOS.compressible_bound_universal
5137 let _ := @NOMOS.cbUniversal
5138 let _ := @NOMOS.cb_id_8_2
5139 let _ := @NOMOS.cb_pad3_8_2
5140 let _ := @NOMOS.cb_pad2_6_1
5141 let _ := @NOMOS.cb_pad4_12_3
5142 let _ := @NOMOS.recoverable
5143 let _ := @NOMOS.nomos_fuel_is_omega_floor
5144 let _ := @NOMOS.nomosScreen
5145 let _ := @NOMOS.nomosScreen_battery
5146 let _ := @BRIDGE.ceiling
5147 let _ := @BRIDGE.census
5148 let _ := @BRIDGE.census_count
5149 let _ := @BRIDGE.post_seal_disposition
5150 let _ := @BRIDGE.weakest_never_above_left
5151 let _ := @BRIDGE.bridge_ceiling_caps
5152 let _ := @BRIDGE.successorOf
5153 let _ := @BRIDGE.supersession_routes
5154 let _ := @BRIDGE.bridgesScreen
5155 let _ := @BRIDGE.bridgeScreen_total
5156 let _ := @FOUND.sigmaP
5157 let _ := @FOUND.pluralist_alternative_executed
5158 let _ := @FOUND.posits
5159 let _ := @FOUND.posits_loadbearing_on_nothing
5160 let _ := @FOUND.gradeOfRank
5161 let _ := @FOUND.posit_never_raises
5162 let _ := @FOUND.bandOf
5163 let _ := @FOUND.register_routing_fidelity
5164 let _ := @FOUND.verdictInventory
5165 let _ := @FOUND.verdicts_posit_free
5166 let _ := @FOUND.engine_never_authority
5167 let _ := @HALT.cascade
5168 let _ := @HALT.patterns
5169 let _ := @HALT.fftSpec
5170 let _ := @HALT.first_failure_terminal_5
5171 let _ := @HALT.first_failure_terminal_7
5172 let _ := @HALT.first_failure_terminal_8
5173 let _ := @HALT.first_failure_terminal_12
5174 let _ := @HALT.halt_free_iff_all_pass_8
5175 let _ := @HALT.gate_counts_recorded
5176 let _ := @HALT.signPatterns
5177 let _ := @HALT.ninth_gate_barred
5178 let _ := @HALT.route
5179 let _ := @HALT.ofUC
5180 let _ := @HALT.route_is_routeHalt
5181 let _ := @HALT.route_samples
5182 let _ := @HALT.tokenBranch
5183 let _ := @HALT.router_exclusive
5184 let _ := @HALT.readingRoadsGate
5185 let _ := @HALT.one_door_open
5186 let _ := @HALT.no_walk_passes_the_aperture
5187 let _ := @HALT.xiGates
5188 let _ := @HALT.oGates
5189 let _ := @HALT.gate_censuses
5190 let _ := @HALT.rhHalt
5191 let _ := @HALT.pnpHalt
5192 let _ := @HALT.halts_routed
5193 let _ := @HALT.legacy_halts_are_instances
5194 let _ := @HALT.terminal_one_bit
5195 let _ := @HALT.species_exhausted
5196 let _ := @HALT.haltGrades
5197 let _ := @HALT.ftoe_ceiling
5198 let _ := @CROSSING.ExecFrame
5199 let _ := @CROSSING.execFlip
5200 let _ := @CROSSING.formalRead
5201 let _ := @CROSSING.ran
5202 let _ := @CROSSING.formalRead_even
5203 let _ := @CROSSING.ran_wholly_odd
5204 let _ := @CROSSING.execFlip_involution
5205 let _ := @CROSSING.ran_odd_at_true
5206 let _ := @CROSSING.wall
5207 let _ := @CROSSING.deed
5208 let _ := @CROSSING.priced
5209 let _ := @CROSSING.crossing
5210 let _ := @CROSSING.identity_one_bit
5211 let _ := @CROSSING.fibre4
5212 let _ := @CROSSING.fibre4_sixteen
5213 let _ := @CROSSING.fibre4_distinct
5214 let _ := @CROSSING.priced_on_this_file

5215 let _ := @CROSSING.formal_never_odd
5216 let _ := @CROSSING.constant_no_crossing
5217 let _ := @CROSSING.harmony
5218 let _ := @LOCUS.Separates
5219 let _ := @LOCUS.Factors
5220 let _ := @LOCUS.wall_of_sep
5221 let _ := @LOCUS.swap
5222 let _ := @LOCUS.swap_involution
5223 let _ := @LOCUS.swap_record_even
5224 let _ := @LOCUS.swap_flips_pair
5225 let _ := @LOCUS.seat_of_sep
5226 let _ := @LOCUS.k4_separated_pair
5227 let _ := @LOCUS.k4_wall_of_pair
5228 let _ := @LOCUS.readout
5229 let _ := @LOCUS.factors_of_not_sep_list
5230 let _ := @LOCUS.factors_iff_not_sep_list
5231 let _ := @LOCUS.Channel
5232 let _ := @LOCUS.Token.toEconomy
5233 let _ := @LOCUS.Reading.orbits
5234 let _ := @LOCUS.Reading.owed
5235 let _ := @LOCUS.Reading.complete
5236 let _ := @LOCUS.branchOf
5237 let _ := @LOCUS.emit
5238 let _ := @LOCUS.Reading.state
5239 let _ := @LOCUS.three_states
5240 let _ := @LOCUS.no_seat_no_locus
5241 let _ := @LOCUS.unlocated_is_plain_open
5242 let _ := @LOCUS.located_never_silent
5243 let _ := @LOCUS.crossed_requires_complete
5244 let _ := @LOCUS.crossed_requires_channel
5245 let _ := @LOCUS.halt_carries_owed
5246 let _ := @LOCUS.unchannelled_is_halt
5247 let _ := @LOCUS.halt_reports_owed
5248 let _ := @LOCUS.owed_zero_is_unchannelled
5249 let _ := @LOCUS.one_bit_apart
5250 let _ := @LOCUS.legacy_xi
5251 let _ := @LOCUS.legacy_o
5252 let _ := @LOCUS.legacy_bare
5253 let _ := @LOCUS.k4_locus
5254 let _ := @LOCUS.slotLists
5255 let _ := @LOCUS.slotLists_twelve
5256 let _ := @LOCUS.slotLists_pos
5257 let _ := @LOCUS.allReadings
5258 let _ := @LOCUS.reading_space
5259 let _ := @LOCUS.census_no_seat_no_locus
5260 let _ := @LOCUS.census_located_never_silent
5261 let _ := @LOCUS.census_crossed_iff_complete_channelled
5262 let _ := @LOCUS.census_halt_reports_owed
5263 let _ := @LOCUS.census_owed_zero_is_unchannelled
5264 let _ := @LOCUS.census_three_states
5265 let _ := @LOCUS.Token.render
5266 let _ := @LOCUS.Token.ascii
5267 let _ := @LOCUS.sealAscii
5268 let _ := @LOCUS.render_receipted
5269 let _ := @LOCUS.render_injective_in_owed
5270 let _ := @LOCUS.voidImage
5271 let _ := @LOCUS.void_precedes_economy
5272 let _ := @LOCUS.legacy_records_not_promoted
5273 let _ := @LOCUS.no_rh_locus
5274 let _ := @PSP.ORIENT.V3
5275 let _ := @PSP.ORIENT.M3
5276 let _ := @PSP.ORIENT.det3
5277 let _ := @PSP.ORIENT.neg3
5278 let _ := @PSP.ORIENT.signMatrices
5279 let _ := @PSP.ORIENT.lock_scalar_sign_blind
5280 let _ := @PSP.ORIENT.det_sign_flips_in_odd_dimension
5281 let _ := @PSP.QUAT.Q
5282 let _ := @PSP.QUAT.qmul
5283 let _ := @PSP.QUAT.qi
5284 let _ := @PSP.QUAT.qj
5285 let _ := @PSP.QUAT.qk
5286 let _ := @PSP.QUAT.qneg1
5287 let _ := @PSP.QUAT.quat_basis_laws
5288 let _ := @PSP.QUAT.qnorm
5289 let _ := @PSP.QUAT.isInt
5290 let _ := @PSP.QUAT.isHalf
5291 let _ := @PSP.QUAT.hurwitzOK
5292 let _ := @PSP.QUAT.grid
5293 let _ := @PSP.QUAT.tuples4
5294 let _ := @PSP.QUAT.hurwitz_units_24
5295 let _ := @PSP.QUAT.grid2
5296 let _ := @PSP.QUAT.tuples4i
5297 let _ := @PSP.QUAT.norm2_shell_split
5298 let _ := @PSP.CD.cdconj
5299 let _ := @PSP.CD.padd
5300 let _ := @PSP.CD.psub
5301 let _ := @PSP.CD.cdmul
5302 let _ := @PSP.CD.e
5303 let _ := @PSP.CD.smul
5304 let _ := @PSP.CD.octonion_associator
5305 let _ := @PSP.CD.sx
5306 let _ := @PSP.CD.sy
5307 let _ := @PSP.CD.zero16
5308 let _ := @PSP.CD.sqnorm
5309 let _ := @PSP.CD.sedenion_zero_divisor
5310 let _ := @PSP.CASCADE.cascade
5311 let _ := @PSP.CASCADE.patterns8
5312 let _ := @PSP.CASCADE.spec
5313 let _ := @PSP.CASCADE.first_failure_terminal_8
5314 let _ := @PSP.CASCADE.halt_free_iff_all_pass_8
5315 let _ := @PSP.GROUNDLESS.o0Neighbors
5316 let _ := @PSP.GROUNDLESS.fiveCubeCoords
5317 let _ := @PSP.GROUNDLESS.deg5
5318 let _ := @PSP.GROUNDLESS.groundless_cascade_degree
5319 let _ := @PSP.TOKEN.level
5320 let _ := @PSP.TOKEN.terminal_tokens_level_and_distinct

```

```

5321 let _ := @PSP.TWOGROUP.two_group_census
5322 let _ := @Bridge.Plane
5323 let _ := @Bridge.get
5324 let _ := @Bridge.onLine
5325 let _ := @Bridge.pe
5326 let _ := @Bridge.fold_involutive
5327 let _ := @Bridge.locus_is_the_fixed_set
5328 let _ := @Bridge.locus_inhabited
5329 let _ := @Bridge.line_symmetric
5330 let _ := @Bridge.LineProperty
5331 let _ := @Bridge.planeFrame
5332 let _ := @Bridge.plane_line_property
5333 let _ := @Bridge.lineFrame
5334 let _ := @Bridge.theRecord
5335 let _ := @Bridge.cannot_lie
5336 let _ := @Bridge.cannot_deviate
5337 let _ := @Bridge.read_twice
5338 let _ := @Bridge.halted_iff
5339 let _ := @Bridge.cannot_extend
5340 let _ := @Bridge.cannot_divide
5341 let _ := @Bridge.delete
5342 let _ := @Bridge.cannot_reverse
5343 let _ := @Bridge.Even
5344 let _ := @Bridge.wall
5345 let _ := @Bridge.calibration
5346 let _ := @Bridge.SelfVerifying
5347 let _ := @Bridge.opens_on_the_deed
5348 let _ := @Bridge.recursion_is_shared
5349 let _ := @Bridge.recursion_is_sign_blind
5350 let _ := @Bridge.exactly_one_stands
5351 let _ := @Bridge.twoPoint
5352 let _ := @Bridge.denial_is_coherent
5353 let _ := @Bridge.line_property_contingent
5354 let _ := @Bridge.Keyless
5355 let _ := @Bridge.Keyed
5356 let _ := @Bridge.keyless_iff_valid
5357 let _ := @Bridge.discriminator
5358 let _ := @Bridge.symmetry_is_keyless
5359 let _ := @Bridge.line_property_is_keyed
5360 let _ := @Bridge.denial_asymmetry
5361 let _ := @Bridge.socket_is_the_property
5362 let _ := @Bridge.socket_on_the_plane
5363 let _ := @Bridge.bridge_carries
5364 let _ := @Bridge.no_socket_off_line
5365 let _ := @Bridge.the_bridge
5366 trivial
5367
5368 /-- info: 'codexCone' depends on axioms: [propext,
5369 Classical.choice,
5370 POSTULATE.Algorithm,
5371 POSTULATE.FrozenData,
5372 POSTULATE.Row,
5373 POSTULATE.SupplyClean,
5374 POSTULATE.certifiedPairing,
5375 Quot.sound,
5376 ROOT.FormalDomain,
5377 ROOT.L1m,
5378 ROOT.L3m,
5379 ROOT.RA,
5380 ROOT.U,
5381 ROOT.nest_21,
5382 ROOT.nest_32,
5383 ROOTA.E] -/
5384 #guard_msgs in #print axioms codexCone
5385
5386 /-! ## the environment audit, run by the elaborator, over the whole codex: the local
5387 axioms are exactly the
5388 twenty-three declared posits of ROOT and POSTULATE, sorted by name; no declaration
5389 depends on any axiom beyond
5390 propext, Classical.choice, Quot.sound and that roster; the roster the Fortran twin
5391 declares by name (twin_posits)
5392 is this list. -/
5393 /-- info: environment audit: local axioms [POSTULATE.Algorithm,
5394 POSTULATE.Crossing,
5395 POSTULATE.FormalFace,
5396 POSTULATE.FrozenData,
5397 POSTULATE.KineticFace,
5398 POSTULATE.RegistrationPostulate,
5399 POSTULATE.Row,
5400 POSTULATE.SupplyClean,
5401 POSTULATE.certifiedPairing,
5402 POSTULATE.erasureClass,
5403 ROOT.FormalDomain,
5404 ROOT.Ground,
5405 ROOT.L1m,
5406 ROOT.L2m,
5407 ROOT.L3m,
5408 ROOT.RA,
5409 ROOT.Residence,
5410 ROOT.U,
5411 ROOT.nest_21,
5412 ROOT.nest_32,
5413 ROOTA.E,
5414 ROOTA.,
5415 ROOTA.binding]; dependencies beyond propext, Classical.choice, Quot.sound [POSTULATE.
5416 Algorithm,
5417 POSTULATE.Crossing,
5418 POSTULATE.FormalFace,
5419 POSTULATE.FrozenData,
5420 POSTULATE.KineticFace,
5421 POSTULATE.RegistrationPostulate,
5422 POSTULATE.Row,
5423 POSTULATE.SupplyClean,
5424 POSTULATE.certifiedPairing,
5425 POSTULATE.erasureClass,
5426 ROOT.FormalDomain,
5427 ROOT.Ground,
5428 ROOT.L1m,
5429 ROOT.L2m,
5430 ROOT.L3m,
5431 ROOT.RA,
5432 ROOT.Residence,
5433 ROOT.U,
5434 ROOT.nest_21,
5435 ROOT.nest_32,
5436 ROOTA.E,
5437 ROOTA.,
5438 ROOTA.binding]; outside the declared roster []; roster size 23 -/
5439 #guard_msgs in
5440 open Lean Elab Command in
5441 #eval show CommandElabM Unit from do
5442 let env ← getEnv
5443 let sortNames (l : List Name) : List Name :=
5444   (l.toArray.qsort (fun a b => a.toString < b.toString)).toList
5445 let ours (n : Name) : Bool :=
5446   match env.getModuleIdxFor? n with
5447   | some idx => ((env.header.moduleNames.get? idx.toNat `).toString.startsWith "Codex
5448   ")
5449   | none => true
5450 let locals := (env.constants.map.toList.filter (fun p => ours p.1)).map (·.1)
5451 ++ env.constants.map.toList.map (·.1)
5452 let declared := sortNames <| locals.filter fun n => !n.isInternal && match env.find? n
5453   with
5454   | some (axiomInfo _) => true
5455   | _ => false
5456 let standard : List Name := [``propext, ``Classical.choice, ``Quot.sound]
5457 let mut beyond : List Name := []
5458 for n in locals do
5459   unless n.isInternal do
5460     for d in ← (collectAxioms n) do
5461       unless standard.contains d || beyond.contains d do beyond := d :: beyond
5462 let depended := sortNames beyond
5463 let outsideRoster := depended.filter fun d => !declared.contains d
5464 logInfo m!"environment audit: local axioms {declared}; dependencies beyond propext,
5465   Classical.choice, Quot.sound {depended}; outside the declared roster {
5466   outsideRoster}; roster size {declared.length}"
5467
5468 /-! ## FORGE cones and the crosswalk audit -/
5469 /-- info: 'K4.semiprimes_at_odd_seats' does not depend on any axioms -/
5470 #guard_msgs in #print axioms K4.semiprimes_at_odd_seats
5471 /-- info: 'K4.semiprime_factors_prime' does not depend on any axioms -/
5472 #guard_msgs in #print axioms K4.semiprime_factors_prime
5473 /-- info: 'K4.odd_seats_not_prime' does not depend on any axioms -/
5474 #guard_msgs in #print axioms K4.odd_seats_not_prime
5475 /-- info: 'IMPRINT.ghost_is_broken_never_sealed' does not depend on any axioms -/
5476 #guard_msgs in #print axioms IMPRINT.ghost_is_broken_never_sealed
5477 /-- info: 'K4.target_is_primality' does not depend on any axioms -/
5478 #guard_msgs in #print axioms K4.target_is_primality
5479 /-- info: 'K4.primes_at_even_seats' does not depend on any axioms -/
5480 #guard_msgs in #print axioms K4.primes_at_even_seats
5481 /-- info: 'K4.pairs_match_mod_420' does not depend on any axioms -/
5482 #guard_msgs in #print axioms K4.pairs_match_mod_420
5483 /-- info: 'IMPRINT.ghost_refused' does not depend on any axioms -/
5484 #guard_msgs in #print axioms IMPRINT.ghost_refused
5485 /-- info: 'IMPRINT.seal_iff_one_side_clean_and_witnessed' does not depend on any axioms
5486 -/
5487 #guard_msgs in #print axioms IMPRINT.seal_iff_one_side_clean_and_witnessed
5488 /-- info: 'IMPRINT.seal_needs_witness' does not depend on any axioms -/
5489 #guard_msgs in #print axioms IMPRINT.seal_needs_witness
5490 /-- info: 'IMPRINT.both_clean_is_ghost' does not depend on any axioms -/
5491 #guard_msgs in #print axioms IMPRINT.both_clean_is_ghost
5492 /-- info: 'IMPRINT.flat_when_unpopulated' does not depend on any axioms -/
5493 #guard_msgs in #print axioms IMPRINT.flat_when_unpopulated
5494 /-- info: 'DEFENSE.gol_needs_all_three' does not depend on any axioms -/
5495 #guard_msgs in #print axioms DEFENSE.gol_needs_all_three
5496 /-- info: 'DEFENSE.lock_blind_but_gol_directed' depends on axioms: [propext] -/
5497 #guard_msgs in #print axioms DEFENSE.lock_blind_but_gol_directed
5498 /-- info: 'DEFENSE.consensus_weights_zero' does not depend on any axioms -/
5499 #guard_msgs in #print axioms DEFENSE.consensus_weights_zero
5500 /-- info: 'DEFENSE.tongue_unclosed_and_obedient' depends on axioms: [propext, Quot.sound
5501 ] -/
5502 #guard_msgs in #print axioms DEFENSE.tongue_unclosed_and_obedient
5503 /-- info: 'DEFENSE.self_check_is_not_witness' does not depend on any axioms -/
5504 #guard_msgs in #print axioms DEFENSE.self_check_is_not_witness
5505 /-- info: 'DEFENSE.denial_is_priced' depends on axioms: [propext] -/
5506 #guard_msgs in #print axioms DEFENSE.denial_is_priced
5507 /-- info: 'DEFENSE.no_anchor_at_target' does not depend on any axioms -/
5508 #guard_msgs in #print axioms DEFENSE.no_anchor_at_target
5509 /-- info: 'DEFENSE.formal_reads_never_the_deed' does not depend on any axioms -/
5510 #guard_msgs in #print axioms DEFENSE.formal_reads_never_the_deed
5511 /-- info: 'DEFENSE.forward_is_world_rowed' does not depend on any axioms -/
5512 #guard_msgs in #print axioms DEFENSE.forward_is_world_rowed
5513 /-- info: 'DEFENSE.worldly_row_is_world_rowed' does not depend on any axioms -/
5514 #guard_msgs in #print axioms DEFENSE.worldly_row_is_world_rowed
5515 /-- info: 'FENCE.global' depends on axioms: [propext, Quot.sound, ROOT.RA, ROOT.U, ROOT
5516   A.E] -/
5517 #guard_msgs in #print axioms FENCE.global
5518 /-- info: 'DOT.outside_economy' depends on axioms: [propext] -/
5519 #guard_msgs in #print axioms DOT.outside_economy
5520 /-- info: 'DOT.no_reading_is_silence' depends on axioms: [propext] -/
5521 #guard_msgs in #print axioms DOT.no_reading_is_silence
5522 /-- info: 'DOT.silence_is_not_a_verdict' does not depend on any axioms -/
5523 #guard_msgs in #print axioms DOT.silence_is_not_a_verdict
5524 /-- info: 'CROSSWALK.rows_count' does not depend on any axioms -/
5525 #guard_msgs in #print axioms CROSSWALK.rows_count
5526
5527 /-- info: crosswalk audit: 129 rows, 0 unresolved, [] -/
5528 #guard_msgs in
5529 open Lean Elab Command in
5530 #eval show CommandElabM Unit from do
5531 let env ← getEnv

```

```

5522 let missing := CROSSWALK.rows.filter fun r => !(env.contains r.2.toName)
5523 logInfo m!"crosswalk audit: {CROSSWALK.rows.length} rows, {missing.length} unresolved,
      {missing.map (·.1)}"

```

C.2 · The Fortran twin

One file, Codex_Twin.f90, sha256 68be9ccf295d6e114c41b253a68f655e8753deb2032544c365ce0d0ee33b56d5: the twin battery of 135 checks and the four witnesses F1 thesis_rows, F2 ra_toe_thesis, F3 ftOE_kinetic_demonstration, F4 it_from_it. Four repairs on the twin are seated in this edition and named in its header: the left-inverse test computed through the deletion map, the omega guard aligned to the Lean surface at bits ≤ 0 , the summary asserts reading the battery state, and the six-seat K4 section named a miniature of the canonical frame that F4 executes.

THE FORTRAN TWIN · CODEX_TWIN.F90 · LINE NUMBERS ARE THE FILE'S

```

1 |=====
2 ! MATHEMATICAL TRISDUCTION CODEX · v2.0.1 · THE FORTRAN TWIN OF THE LEAN SURFACE
3 !
4 ! One register of record, two surfaces. The Lean surface (Part I of the codex)
5 ! proves; this file executes. One compilation unit, one executable, two halves:
6 !
7 ! FORGE v4.0.1 repairs on the twin: cannot_reverse computed through the deletion map (
  LB-6);
8 ! omega guard bits <= 0 aligned to the Lean surface (E-5); summary asserts read the
  battery
9 ! state instead of a literal (E-3); the six-seat K4 section named a miniature (E-6).
10 ! 1. THE TWIN BATTERY, Lean-native sections. The finite content of the Lean
11 ! codex that the twin mirrors is re-executed by enumeration: the exact rational, the
  grade ladder,
12 ! the fTOE spine on every frame to four points, the canonical frame, the K4
13 ! frame, the twelve gates, A4, the five-cube, the GF(2) lock, the Hurwitz
14 ! shell, Cayley-Dickson, the adjugate identity, Cramer, reflection blindness,
15 ! the deletion operator, the super halt, Omega and AEGIS, NOMOS-01, the
16 ! crossing, the locus, the posits, and the bridge (SECTION 19, new in v2.0.1).
17 !
18 ! 2. THE FOUR WITNESSES, verbatim, with every contract comment they were
19 ! audited with, wrapped as subroutines of this one executable:
20 ! F1 thesis_rows the core thesis executed (F3's header cites the
21 ! parent battery as
  TRISDUCTION_Core_Thesis_Fortran_v2_4_0;
22 ! F1 itself carries no version string)
23 ! F2 ra_toe_thesis RA_TOE_Thesis_Fortran v2.0.0 (census lock 1123)
24 ! F3 ftOE_kinetic_demonstration fTOE kinetic demonstration (REPAIR v1.0.1.2)
25 ! F4 it_from_it the inversion, computed exhaustively
26 !
27 ! BUILD (sealed): gfortran -std=f2018 -O2 -fno-fast-math -ffp-contract=off
28 ! A binary that reaches its final line has passed; any failure halts with a
29 ! nonzero exit. F2 accepts the arguments 'audit' and 'witnessed' as before.
30 ! The 115 -Wcompare-reals warnings under -Wall (113 in F2, as its header
31 ! records, and 2 in F3) are the exact-equality verdict design and are
32 ! intentional. Delta-M = 0.
33 |=====
34
35 module twin_kinds
36 implicit none
37 integer, parameter :: i8 = selected_int_kind(18)
38 integer, parameter :: i16 = selected_int_kind(38)
39 integer, parameter :: dp = selected_real_kind(15, 307)
40 end module twin_kinds
41
42 module twin_battery
43 implicit none
44 integer :: checks = 0, failures = 0
45 contains
46 subroutine assert(cond, label)
47 logical, intent(in) :: cond
48 character(len=*), intent(in) :: label
49 checks = checks + 1
50 if (.not. cond) then
51 failures = failures + 1
52 write(*, '(A,A)') ' FAIL ', label
53 end if
54 end subroutine assert
55 subroutine section(name)
56 character(len=*), intent(in) :: name
57 write(*, '(/,A)') ' //name
58 end subroutine section
59 end module twin_battery
60
61 module twin_rat
62 use twin_kinds
63 implicit none
64 type :: rat
65 integer(i8) :: num = 0
66 integer(i8) :: den = 1
67 end type rat

```

```

68 contains
69 pure function gcdf(fuel, a, b) result(g)
70 integer, intent(in) :: fuel
71 integer(i8), intent(in) :: a, b
72 integer(i8) :: g, x, y, t
73 x = a; y = b
74 do while (y /= 0_i8)
75 t = mod(x, y); x = y; y = t
76 end do
77 g = x
78 if (fuel < 0) g = x
79 end function gcdf
80 pure function rmk(n, d) result(r)
81 integer(i8), intent(in) :: n, d
82 type(rat) :: r
83 integer(i8) :: nn, dd, g
84 if (n == 0_i8) then
85 r = rat(0_i8, 1_i8); return
86 end if
87 if (d == 0_i8) then
88 r = rat(0_i8, 1_i8); return
89 end if
90 if (d < 0_i8) then
91 nn = -n; dd = -d
92 else
93 nn = n; dd = d
94 end if
95 g = gcdf(1024, abs(nn), dd)
96 if (g == 0_i8) then
97 r = rat(0_i8, 1_i8); return
98 end if
99 r = rat(nn / g, dd / g)
100 end function rmk
101 pure function rof(n) result(r)
102 integer, intent(in) :: n
103 type(rat) :: r
104 r = rat(int(n, i8), 1_i8)
105 end function rof
106 pure function radd(a, b) result(c)
107 type(rat), intent(in) :: a, b
108 type(rat) :: c
109 c = rmk(a%num*b%den + b%num*a%den, a%den*b%den)
110 end function radd
111 pure function rneg(a) result(c)
112 type(rat), intent(in) :: a
113 type(rat) :: c
114 c = rat(-a%num, a%den)
115 end function rneg
116 pure function rsub(a, b) result(c)
117 type(rat), intent(in) :: a, b
118 type(rat) :: c
119 c = radd(a, rneg(b))
120 end function rsub
121 pure function rmul(a, b) result(c)
122 type(rat), intent(in) :: a, b
123 type(rat) :: c
124 c = rmk(a%num*b%num, a%den*b%den)
125 end function rmul
126 pure function rinva result(c)
127 type(rat), intent(in) :: a
128 type(rat) :: c
129 c = rmk(a%den, a%num)
130 end function rinva
131 pure function rdv(a, b) result(c)
132 type(rat), intent(in) :: a, b
133 type(rat) :: c
134 c = rmul(a, rinva(b))
135 end function rdv
136 pure function req(a, b) result(e)
137 type(rat), intent(in) :: a, b
138 logical :: e
139 e = (a%num == b%num) .and. (a%den == b%den)
140 end function req
141 pure function rlt(a, b) result(e)
142 type(rat), intent(in) :: a, b
143 logical :: e
144 e = a%num*b%den < b%num*a%den
145 end function rlt
146 pure function rpowa(n) result(c)
147 type(rat), intent(in) :: a
148 integer, intent(in) :: n
149 type(rat) :: c
150 integer :: k
151 c = rof(1)
152 do k = 1, n
153 c = rmul(c, a)
154 end do
155 end function rpowa
156 end module twin_rat
157
158 module twin_grade
159 implicit none
160 integer, parameter :: G_PREMISE = 0, G_CORROBORATION = 1, G_OPERATIONAL = 2,
  G_STRUCTURAL = 3, &
161 &
  G_ENGINEERING = 4, G_CONDITIONAL = 5, G_THEOREM_CONDITIONAL = 6,
  &
  G_ANALYTIC = 7, G_THEOREM = 8
162 integer, parameter :: NGRADE = 9
163 contains
164 pure function rank(g) result(r)
165 integer, intent(in) :: g
166 integer :: r
167 r = g
168 end function rank
169 pure function weakest(a, b) result(w)
170 integer, intent(in) :: a, b

```

```

172 integer :: w
173 if (rank(a) <= rank(b)) then
174   w = a
175 else
176   w = b
177 end if
178 end function weakest
179 pure function grade_name(g) result(s)
180 integer, intent(in) :: g
181 character(len=20) :: s
182 select case (g)
183 case (0); s = 'premise'
184 case (1); s = 'corroboration'
185 case (2); s = 'operational'
186 case (3); s = 'structural'
187 case (4); s = 'engineering'
188 case (5); s = 'conditional'
189 case (6); s = 'theorem-conditional'
190 case (7); s = 'analytic'
191 case (8); s = 'theorem'
192 case default; s = '??'
193 end select
194 end function grade_name
195 end module twin_grade
196 module twin_ftoe
197 use twin_battery
198 implicit none
199 integer, parameter :: NMAX = 4
200 contains
201 pure logical function bit(m, x)
202 integer, intent(in) :: m, x
203 bit = btest(m, x)
204 end function bit
205 pure logical function is_even(n, tau, f)
206 integer, intent(in) :: n, tau(0:), f
207 integer :: x
208 is_even = .true.
209 do x = 0, n-1
210   if (bit(f, tau(x)) .neqv. bit(f, x)) then
211     is_even = .false.; return
212   end if
213 end do
214 end function is_even
215 pure logical function odd_at(tau, d, x)
216 integer, intent(in) :: tau(0:), d, x
217 odd_at = bit(d, tau(x)) .neqv. bit(d, x)
218 end function odd_at
219 pure logical function wholly_odd(n, tau, t)
220 integer, intent(in) :: n, tau(0:), t
221 integer :: x
222 wholly_odd = .true.
223 do x = 0, n-1
224   if (bit(t, tau(x)) .eqv. bit(t, x)) then
225     wholly_odd = .false.; return
226   end if
227 end do
228 end function wholly_odd
229 pure logical function is_involution(n, tau)
230 integer, intent(in) :: n, tau(0:)
231 integer :: x
232 is_involution = .true.
233 do x = 0, n-1
234   if (tau(tau(x)) /= x) then
235     is_involution = .false.; return
236   end if
237 end do
238 end function is_involution
239 pure logical function binop(k, a, b)
240 integer, intent(in) :: k
241 logical, intent(in) :: a, b
242 integer :: idx
243 idx = 0
244 if (a) idx = idx + 2
245 if (b) idx = idx + 1
246 binop = btest(k, idx)
247 end function binop
248 pure function apply_binop(n, k, f, g) result(h)
249 integer, intent(in) :: n, k, f, g
250 integer :: h, x
251 h = 0
252 do x = 0, n-1
253   if (binop(k, bit(f, x), bit(g, x))) h = ibset(h, x)
254 end do
255 end function apply_binop
256 pure function xor_mask(n, a, b) result(h)
257 integer, intent(in) :: n, a, b
258 integer :: h
259 h = iand(ieor(a, b), 2**n - 1)
260 end function xor_mask
261
262 subroutine run_spine()
263 integer :: n, ntau, it, x, f, d, s, g, k, c, ncount, nmask, y
264 integer :: tau(0:NMAX-1), rho(0:NMAX-1)
265 integer(8) :: inst(20)
266 logical :: ok, exists, unique, hyp
267 integer :: c_found, ncand, dcomp, tot_odd
268 call section('SECTIONS 2-4 - THE SPINE ON EVERY FINITE FRAME: |X| = 1..4 every tau,
269   every reading; T2 and T14 to |X| = 3')
269 inst = 0
270 do n = 1, NMAX
271   nmask = 2**n
272   ntau = n**n
273   do it = 0, ntau-1
274     call decode_map(n, it, tau)
275     do x = 0, n-1
276       do f = 0, nmask-1

```

```

277   do d = 0, nmask-1
278     if (is_even(n, tau, f) .and. odd_at(tau, d, x)) then
279       inst(1) = inst(1)+1
280       if (f == d) call assert(.false., 'T1_wall instance')
281     end if
282     if (is_even(n, tau, f) .and. wholly_odd(n, tau, d)) then
283       inst(7) = inst(7)+1
284       if (f == d) call assert(.false., 'T7_global_wall instance')
285     end if
286     if (wholly_odd(n, tau, f) .and. wholly_odd(n, tau, d) .and. (bit(f, x) .
287       .eqv. bit(d, x))) then
288       inst(4) = inst(4)+1
289       if (bit(f, tau(x)) .neqv. bit(d, tau(x))) call assert(.false., '
290       T4_one_bit instance')
291     end if
292     s = f
293     if (odd_at(tau, s, x) .and. odd_at(tau, d, x)) then
294       inst(5) = inst(5)+1
295       ncand = 0
296       do c = 0, 1
297         if ((bit(d, x) .eqv. (bit(s, x) .neqv. (c==1))) .and. &
298           (bit(d, tau(x)) .eqv. (bit(s, tau(x)) .neqv. (c==1)))) ncand
299         = ncand + 1
300       end do
301       if (ncand /= 1) call assert(.false., 'T5_crossing existence and
302       uniqueness')
303     end if
304     if (odd_at(tau, d, x)) then
305       if (f == 0) inst(6) = inst(6)+1
306       if (bit(d, tau(x)) .eqv. bit(d, x)) call assert(.false., '
307       T6a_deed_anchor')
308     end if
309     if (odd_at(tau, s, x)) then
310       do c = 0, 1
311         if ((bit(d, x) .eqv. (bit(s, x) .neqv. (c==1))) .and. &
312           (bit(d, tau(x)) .eqv. (bit(s, tau(x)) .neqv. (c==1)))) then
313           inst(8) = inst(8)+1
314           if (bit(d, tau(x)) .eqv. (bit(s, x) .neqv. (c==1))) call
315           assert(.false., 'T6b_deed_supply')
316         end if
317       end do
318     end if
319     if (wholly_odd(n, tau, d)) then
320       if (f == 0) inst(13) = inst(13)+1
321       if (.not. odd_at(tau, d, x)) call assert(.false., '
322       T13_wholly_odd_is_odd_everywhere')
323     end if
324     if (x == 0) then
325       ok = .true.
326       do y = 0, n-1
327         if (.not. odd_at(tau, d, y)) ok = .false.
328       end do
329       inst(17) = inst(17)+1
330       if (ok .neqv. wholly_odd(n, tau, d)) call assert(.false., 'T17 odd-
331       everywhere iff wholly-odd')
332     end if
333     end do
334   end do
335   do d = 0, nmask-1
336     if (wholly_odd(n, tau, d)) then
337       inst(3) = inst(3)+1
338       if (tau(x) == x) call assert(.false., 'T3_no_fixed_point')
339     end if
340   end do
341   do s = 0, nmask-1
342     do d = 0, nmask-1
343       if (wholly_odd(n, tau, s) .and. wholly_odd(n, tau, d)) then
344         inst(9) = inst(9)+1
345         if (.not. is_even(n, tau, xor_mask(n, d, s))) call assert(.false., '
346         T8_torsor_forward')
347       end if
348       if (wholly_odd(n, tau, s) .and. is_even(n, tau, d)) then
349         inst(10) = inst(10)+1
350         if (.not. wholly_odd(n, tau, xor_mask(n, s, d))) call assert(.false., '
351         T8_torsor_backward')
352       end if
353       if (it == 0) then
354         inst(11) = inst(11)+1
355         if (xor_mask(n, s, xor_mask(n, d, s)) /= d) call assert(.false., '
356         T9_torsor_inverse_left')
357         if (xor_mask(n, xor_mask(n, s, d), s) /= d) call assert(.false., '
358         T9_torsor_inverse_right')
359       end if
360     end do
361   end do
362   if (n <= 3) then
363     do f = 0, nmask-1
364       if (.not. is_even(n, tau, f)) cycle
365       do g = 0, nmask-1
366         if (.not. is_even(n, tau, g)) cycle
367         do k = 0, 15
368           do d = 0, nmask-1
369             do x = 0, n-1
370               if (odd_at(tau, d, x)) then
371                 inst(2) = inst(2)+1
372                 if (apply_binop(n, k, f, g) == d) call assert(.false., '
373                 T2_coalition instance')
374               end if
375             end do
376           end do
377         end do
378       end do
379     end if
380   if (n <= 3) then

```

```

370         do k = 0, ntau-1
371         call decode_map(n, k, rho)
372         do x = 0, n-1
373             if (rho(tau(x)) /= rho(x)) cycle
374             do d = 0, nmask-1
375                 if (.not. odd_at(tau,d,x)) cycle
376                 inst(14) = inst(14)+1
377                 exists = .false.
378                 do g = 0, nmask-1
379                     ok = .true.
380                     do y = 0, n-1
381                         if (bit(g, rho(y)) .neqv. bit(d, y)) ok = .false.
382                     end do
383                     if (ok) exists = .true.
384                 end do
385                 if (exists) call assert(.false., 'T14_wall_factorization instance')
386             end do
387         end do
388     end do
389 end if
390 end do
391 end do
392 call assert(inst(1) > 0 .and. inst(2) > 0 .and. inst(3) > 0 .and. inst(4) > 0 &
393 .and. inst(5) > 0, 'spine: every theorem reached instances')
394 write(*, '(A,I0,A,I0,A,I0)') ' instances T1 wall ', inst(1), ' T2 coalition ',
395 inst(2), &
396 ' T3 no-fixed-point ', inst(3)
397 write(*, '(A,I0,A,I0,A,I0)') ' instances T4 one-bit ', inst(4), ' T5 crossing ',
398 inst(5), &
399 ' T6a anchor ', inst(6)
400 write(*, '(A,I0,A,I0,A,I0)') ' instances T6b supply ', inst(8), ' T7 global wall ',
401 inst(7), &
402 ' T8 forward ', inst(9)
403 write(*, '(A,I0,A,I0,A,I0)') ' instances T8 backward ', inst(10), ' T9 inverses ',
404 inst(11), &
405 ' T13 ', inst(13)
406 write(*, '(A,I0,A,I0)') ' instances T14 factorization ', inst(14), ' T17 ', inst
407 (17)
408 call run_canonical_frame()
409 call run_k4()
410 end subroutine run_spine
411
412 pure subroutine decode_map(n, k, tau)
413 integer, intent(in) :: n, k
414 integer, intent(out) :: tau(0:)
415 integer :: x, r
416 r = k
417 do x = 0, n-1
418     tau(x) = mod(r, n); r = r / n
419 end do
420 end subroutine decode_map
421
422 subroutine run_canonical_frame()
423 integer :: nq, n, tau(0:7), d, q, b, cnt, c, e, nmask, d0, dd, x, ee
424 integer :: forward, back
425 logical :: ok
426 call section('SECTION 3 · T15/T16 ON THE CANONICAL FRAME Bool x Q, |Q| = 1..4')
427 do nq = 1, 4
428     n = 2*nq; nmask = 2**n
429     do q = 0, nq-1
430         tau(2*q) = 2*q+1; tau(2*q+1) = 2*q
431     end do
432     call assert(is_involution(n, tau), 'flipF is an involution')
433     cnt = 0
434     do d = 0, nmask-1
435         if (wholly_odd(n, tau, d)) cnt = cnt + 1
436     end do
437     call assert(cnt == 2*nq, 'T16 price: |Dtau| = 2*|Q|')
438     do d = 0, nmask-1
439         if (.not. wholly_odd(n, tau, d)) cycle
440         do q = 0, nq-1
441             do b = 0, 1
442                 if ((bit(d, 2*q+1) .neqv. (b==0)) .neqv. bit(d, 2*q+b)) call assert(.false
443 ., 'T15 forward')
444             end do
445         end do
446     end do
447     do c = 0, 2**nq-1
448         d = 0
449         do q = 0, nq-1
450             do b = 0, 1
451                 if (bit(c, q) .neqv. (b==0)) d = ibset(d, 2*q+b)
452             end do
453         end do
454         if (.not. wholly_odd(n, tau, d)) call assert(.false., 'T15 backward')
455     end do
456     do d0 = 0, nmask-1
457         if (.not. wholly_odd(n, tau, d0)) cycle
458         do dd = 0, nmask-1
459             if (.not. wholly_odd(n, tau, dd)) cycle
460             forward = 0
461             do q = 0, nq-1
462                 if (bit(dd, 2*q+1) .neqv. bit(d0, 2*q+1)) forward = ibset(forward, q)
463             end do
464             back = 0
465             do x = 0, n-1
466                 if (bit(d0, x) .neqv. bit(forward, x/2)) back = ibset(back, x)
467             end do
468             if (back /= dd) call assert(.false., 'T16 left inverse: back(forward d) = d
469 ')
470         end do
471     end do
472     do e = 0, 2**nq-1
473         back = 0
474         do x = 0, n-1
475             if (bit(d0, x) .neqv. bit(e, x/2)) back = ibset(back, x)

```

```

468         end do
469         ee = 0
470         do q = 0, nq-1
471             if (bit(back, 2*q+1) .neqv. bit(d0, 2*q+1)) ee = ibset(ee, q)
472         end do
473         if (ee /= e) call assert(.false., 'T16 right inverse: forward(back e) = e')
474     end do
475 end do
476 write(*, '(A,I0,A,I0,A,I0)') ' |Q| = ', nq, ' frame ', n, ' |Dtau| = ', cnt
477 end do
478 end subroutine run_canonical_frame
479
480 subroutine run_k4()
481 integer :: tau(0:5), rho(0:5), d, g, y, x
482 logical :: ok, exists
483 call section('SECTION 4 · THE K4 FRAME, MINIATURE: three of the six pairs; the
484 canonical twelve-seat frame runs in F4')
485 tau = [1,0,3,2,5,4]; rho = [0,0,1,1,2,2]
486 d = 0
487 do x = 0, 5, 2
488     d = ibset(d, x)
489 end do
490 call assert(is_involution(6, tau), 'K4: tau involution')
491 call assert(wholly_odd(6, tau, d), 'K4: target wholly odd')
492 ok = .true.
493 do x = 0, 5
494     if (rho(tau(x)) /= rho(x)) ok = .false.
495 end do
496 call assert(ok, 'K4: record even, rho(tau x) = rho x')
497 exists = .false.
498 do g = 0, 7
499     ok = .true.
500     do y = 0, 5
501         if (bit(g, rho(y)) .neqv. bit(d, y)) ok = .false.
502     end do
503     if (ok) exists = .true.
504 end do
505 call assert(.not. exists, 'K4.wall: no readout g of the record returns the target (8
506 readouts tried)')
507 call assert(all(mod([851-11, 1273-13, 437-17, 2119-19, 1703-23, 869-29], 210) == 0)
508 .and. &
509 all([851-11, 1273-13, 437-17, 2119-19, 1703-23, 869-29] > 0), &
510 'K4 lifts: every odd seat is its partner plus a positive multiple of
511 210, the construction not a sieve')
512 end subroutine run_k4
513 end module twin_ftoe
514 module twin_geo
515 use twin_kinds
516 use twin_battery
517 use twin_rat
518 implicit none
519 integer, save :: forced_twelve = -1, forced_eight = -1, forced_five = -1
520 contains
521 pure function inv_count(p) result(c)
522 integer, intent(in) :: p(4)
523 integer :: c, i, j
524 c = 0
525 do i = 1, 4
526     do j = i+1, 4
527         if (p(i) > p(j)) c = c + 1
528     end do
529 end do
530 end function inv_count
531 subroutine run_gates()
532 integer :: perms(4,24), np, a, b, c, d, k, n_even, hits, i, j, s
533 integer :: gates(2,12), ng
534 call section('SECTION 4D · THE TWELVE GATES: A4 = 12 rotations, simply transitive on
535 the gates')
536 np = 0
537 do a = 0, 3; do b = 0, 3; do c = 0, 3; do d = 0, 3
538     if (a/=b .and. a/=c .and. a/=d .and. b/=c .and. b/=d .and. c/=d) then
539         np = np + 1; perms(:,np) = [a,b,c,d]
540     end if
541 end do; end do; end do; end do
542 call assert(np == 24, 'S4: 24 permutations of the four vertices')
543 n_even = 0
544 do k = 1, 24
545     if (mod(inv_count(perms(:,k)), 2) == 0) n_even = n_even + 1
546 end do
547 call assert(n_even == 12, 'A4: exactly 12 even permutations, the tetrahedral
548 rotations')
549 forced_twelve = n_even
550 ng = 0
551 do i = 0, 3
552     do j = 0, 3
553         if (i /= j) then
554             ng = ng + 1; gates(:,ng) = [i,j]
555         end if
556     end do
557 end do
558 call assert(ng == 12, 'twelve directed gates: ordered pairs, source distinct from
559 target')
560 do i = 1, 12
561     do j = 1, 12
562         hits = 0
563         do k = 1, 24
564             if (mod(inv_count(perms(:,k)), 2) /= 0) cycle
565             if (perms(gates(1,i)+1,k) == gates(1,j) .and. perms(gates(2,i)+1,k) ==
566 gates(2,j)) hits = hits + 1
567         end do
568         if (hits /= 1) call assert(.false., 'A4 simply transitive on gates')
569     end do
570 end do
571 call assert(failures == 0, 'A4 simply transitive: 144 gate pairs, one rotation each')
572 )
573 call assert(4 - 6 + 4 == 2, 'Euler closure V - E + F = 2 on the tetrahedron')

```

```

565 s = 0
566 do k = 0, 2**3 - 1
567   s = s + 1
568 end do
569 call assert(s == 8, 'eight: the reflection patterns (Z/2)^3, and no ninth')
570 forced_eight = s
571 call run_fivcube()
572 end subroutine run_gates
573
574 subroutine run_fivcube()
575   integer :: v, k, nb, u, w, cnt
576   logical :: seen(0:31)
577   call section('SECTION 4E · THE FIVE-CUBE: 32 vertices, every vertex of degree
   exactly five')
578   cnt = 0
579   do v = 0, 31
580     nb = 0
581     seen = .false.
582     do k = 0, 4
583       u = ieor(v, 2**k)
584       if (.not. seen(u)) then
585         seen(u) = .true.; nb = nb + 1
586       end if
587     end do
588     if (nb /= 5) call assert(.false., 'five-cube vertex degree')
589     cnt = cnt + 1
590   end do
591   call assert(cnt == 32, 'five-cube: 32 vertices')
592   call assert(failures == 0, 'five-cube: every vertex has exactly five ways out')
593   forced_five = nb
594   w = 0
595 end subroutine run_fivcube
596
597 pure function det2(r1, r2, r3) result(d)
598   integer, intent(in) :: r1(3), r2(3), r3(3)
599   integer :: d
600   d = mod( r1(1)*(r2(2)*r3(3) + r2(3)*r3(2)) + r1(2)*(r2(1)*r3(3) + r2(3)*r3(1)) &
601     + r1(3)*(r2(1)*r3(2) + r2(2)*r3(1)), 2)
602 end function det2
603 subroutine run_gf2()
604   integer :: m, r1(3), r2(3), r3(3), t(3), v(3), nsol, k, nlock, nopen, det
605   logical :: uniq
606   call section('SECTION 4D · THE TRIAXIAL LOCK OVER GF(2): all 4096 systems, det 1 iff
   exactly one solution')
607   nlock = 0; nopen = 0
608   do m = 0, 4095
609     r1 = [ibits(m,11,1), ibits(m,10,1), ibits(m,9,1)]
610     r2 = [ibits(m,8,1), ibits(m,7,1), ibits(m,6,1)]
611     r3 = [ibits(m,5,1), ibits(m,4,1), ibits(m,3,1)]
612     t = [ibits(m,2,1), ibits(m,1,1), ibits(m,0,1)]
613     det = det2(r1, r2, r3)
614     nsol = 0
615     do k = 0, 7
616       v = [ibits(k,2,1), ibits(k,1,1), ibits(k,0,1)]
617       if (mod(r1(1)*v(1)+r1(2)*v(2)+r1(3)*v(3),2) == t(1) .and. &
618         mod(r2(1)*v(1)+r2(2)*v(2)+r2(3)*v(3),2) == t(2) .and. &
619         mod(r3(1)*v(1)+r3(2)*v(2)+r3(3)*v(3),2) == t(3)) nsol = nsol + 1
620     end do
621     uniq = (nsol == 1)
622     if ((det == 1) .neqv. uniq) call assert(.false., 'GF(2) lock law instance')
623     if (det == 1) then
624       nlock = nlock + 1
625     else
626       nopen = nopen + 1
627     end if
628   end do
629   call assert(nlock == 1344 .and. nopen == 2752, 'GF(2) census: 1344 locked, 2752 open
   , 4096 in all')
630   write(*, '(A,I0,A,I0)') ' locked ', nlock, ' open ', nopen
631 end subroutine run_gf2
632
633 pure function qm(a, b) result(c)
634   integer, intent(in) :: a(4), b(4)
635   integer :: c(4)
636   c(1) = a(1)*b(1) - a(2)*b(2) - a(3)*b(3) - a(4)*b(4)
637   c(2) = a(1)*b(2) + a(2)*b(1) + a(3)*b(4) - a(4)*b(3)
638   c(3) = a(1)*b(3) - a(2)*b(4) + a(3)*b(1) + a(4)*b(2)
639   c(4) = a(1)*b(4) + a(2)*b(3) - a(3)*b(2) + a(4)*b(1)
640 end function qm
641 subroutine run_hurwitz()
642   integer :: v(4), n, i, j, k, l, cnt, ijk(4), e(4,3), p
643   call section('SECTION 4D · THE HURWITZ SHELL: 24 units in doubled coordinates,
   chirality i j k = -1')
644   cnt = 0
645   do i = -2, 2; do j = -2, 2; do k = -2, 2; do l = -2, 2
646     v = [i,j,k,l]
647     if (i*i+j*j+k*k+l*l /= 4) cycle
648     if (all(mod(abs(v),2) == 0) .or. all(mod(abs(v),2) == 1)) cnt = cnt + 1
649   end do; end do; end do; end do
650   call assert(cnt == 24, 'Hurwitz units: exactly 24')
651   e(:,1) = [0,1,0,0]; e(:,2) = [0,0,1,0]; e(:,3) = [0,0,0,1]
652   ijk = qm(qm(e(:,1), e(:,2)), e(:,3))
653   call assert(all(ijk == [-1,0,0,0]), 'substrate chirality: Re(i*j*k) = -1')
654   p = 0
655   do i = 1, 3; do j = 1, 3; do k = 1, 3
656     if (i==j .or. j==k .or. i==k) cycle
657     ijk = qm(qm(e(:,i), e(:,j)), e(:,k))
658     n = 0
659     if (i > j) n = n + 1
660     if (i > k) n = n + 1
661     if (j > k) n = n + 1
662     if (mod(n,2) == 0) then
663       if (ijk(1) /= -1) call assert(.false., 'even relabeling reads -1')
664     else
665       if (ijk(1) /= 1) call assert(.false., 'odd relabeling reads +1')
666     end if

```

```

667     p = p + 1
668   end do; end do; end do
669   call assert(p == 6, 'relabel parity: six orderings, three even three odd')
670 end subroutine run_hurwitz
671
672 recursive function cd_mul(a, b, n) result(c)
673   integer, intent(in) :: n
674   integer, intent(in) :: a(n), b(n)
675   integer :: c(n), h
676   integer, allocatable :: a1(:), a2(:), b1(:), b2(:), t1(:), t2(:)
677   if (n == 1) then
678     c(1) = a(1)*b(1); return
679   end if
680   h = n/2
681   allocate(a1(h), a2(h), b1(h), b2(h), t1(h), t2(h))
682   a1 = a(1:h); a2 = a(h+1:n); b1 = b(1:h); b2 = b(h+1:n)
683   t1 = cd_mul(a1, b1, h) - cd_mul(cd_conj(b2, h), a2, h)
684   t2 = cd_mul(b2, a1, h) + cd_mul(a2, cd_conj(b1, h), h)
685   c(1:h) = t1; c(h+1:n) = t2
686 end function cd_mul
687 pure function cd_conj(a, n) result(c)
688   integer, intent(in) :: n
689   integer, intent(in) :: a(n)
690   integer :: c(n)
691   c = -a; c(1) = a(1)
692 end function cd_conj
693 pure function unit(n, i) result(e)
694   integer, intent(in) :: n, i
695   integer :: e(n)
696   e = 0; e(i+1) = 1
697 end function unit
698 subroutine run_cayley_dickson()
699   integer :: lhs(8), rhs(8), assoc(8), sx(16), sy(16), prod(16), cnt, a, b, c, d
700   call section('SECTION 4D · CAYLEY-DICKSON: octonion associator 2 e7, sedenion zero
   divisor (e1+e10)(e5+e14) = 0')
701   lhs = cd_mul(cd_mul(unit(8,1), unit(8,2), 8), unit(8,4), 8)
702   rhs = cd_mul(unit(8,1), cd_mul(unit(8,2), unit(8,4), 8), 8)
703   assoc = lhs - rhs
704   call assert(all(assoc == 2*unit(8,7)), 'octonion associator [e1,e2,e4] = 2 e7: non-
   associative, exactly')
705   sx = unit(16,1) + unit(16,10); sy = unit(16,5) + unit(16,14)
706   prod = cd_mul(sx, sy, 16)
707   call assert(all(prod == 0), 'sedenion zero divisor: (e1+e10)(e5+e14) = 0')
708   call assert(sum(sx*sx) == 2 .and. sum(sy*sy) == 2, 'both factors have squared norm
   2, neither is zero')
709   sy = unit(16,4) + unit(16,15); prod = cd_mul(sx, sy, 16)
710   call assert(.not. all(prod == 0), 'negative control: (e1+e10)(e4+e15) is NOT zero,
   it is 2e5 - 2e14')
711   call assert(prod(6) == 2 .and. prod(15) == -2, 'negative control components: +2 at
   e5, -2 at e14')
712   cnt = 0
713   do a = 0, 15; do b = a+1, 15; do c = 0, 15; do d = c+1, 15
714     prod = cd_mul(unit(16,a)+unit(16,b), unit(16,c)+unit(16,d), 16)
715     if (all(prod == 0)) cnt = cnt + 1
716   end do; end do; end do; end do
717   call assert(cnt == 84, 'sedenion basis-pair zero divisors: 84 in all')
718   write(*, '(A,I0)') ' zero-divisor pairs (a<b, c<d) = ', cnt
719 end subroutine run_cayley_dickson
720
721 pure function det3r(m) result(d)
722   type(rat), intent(in) :: m(3,3)
723   type(rat) :: d
724   d = rsub(rsub(rmul(m(1,1), rsub(rmul(m(2,2),m(3,3))), rmul(m(2,3),m(3,2))))), &
725     rmul(m(1,2), rsub(rmul(m(2,1),m(3,3))), rmul(m(2,3),m(3,1))))), &
726     rneg(rmul(m(1,3), rsub(rmul(m(2,1),m(3,2))), rmul(m(2,2),m(3,1)))))
727 end function det3r
728 pure function adj3r(m) result(a)
729   type(rat), intent(in) :: m(3,3)
730   type(rat) :: a(3,3)
731   a(1,1) = rsub(rmul(m(2,2),m(3,3))), rmul(m(2,3),m(3,2)))
732   a(1,2) = rsub(rmul(m(1,3),m(3,2))), rmul(m(1,2),m(3,3)))
733   a(1,3) = rsub(rmul(m(1,2),m(2,3))), rmul(m(1,3),m(2,2)))
734   a(2,1) = rsub(rmul(m(2,3),m(3,1))), rmul(m(2,1),m(3,3)))
735   a(2,2) = rsub(rmul(m(1,1),m(3,3))), rmul(m(1,3),m(3,1)))
736   a(2,3) = rsub(rmul(m(1,3),m(2,1))), rmul(m(1,1),m(2,3)))
737   a(3,1) = rsub(rmul(m(2,1),m(3,2))), rmul(m(2,2),m(3,1)))
738   a(3,2) = rsub(rmul(m(1,2),m(3,1))), rmul(m(1,1),m(3,2)))
739   a(3,3) = rsub(rmul(m(1,1),m(2,2))), rmul(m(1,2),m(2,1)))
740 end function adj3r
741 pure function mm3r(x, y) result(z)
742   type(rat), intent(in) :: x(3,3), y(3,3)
743   type(rat) :: z(3,3)
744   integer :: i, j, k
745   do i = 1, 3
746     do j = 1, 3
747       z(i,j) = rof(0)
748       do k = 1, 3
749         z(i,j) = radd(z(i,j), rmul(x(i,k), y(k,j)))
750       end do
751     end do
752   end do
753 end function mm3r
754 pure function mv3r(m, v) result(w)
755   type(rat), intent(in) :: m(3,3), v(3)
756   type(rat) :: w(3)
757   integer :: i
758   do i = 1, 3
759     w(i) = radd(radd(rmul(m(i,1),v(1))), rmul(m(i,2),v(2))), rmul(m(i,3),v(3)))
760   end do
761 end function mv3r
762 pure function diag3r(r) result(m)
763   type(rat), intent(in) :: r
764   type(rat) :: m(3,3)
765   integer :: i, j
766   do i = 1, 3
767     do j = 1, 3

```

```

768         if (i == j) then
769             m(i,j) = r
770         else
771             m(i,j) = rof(0)
772         end if
773     end do
774 end do
775 end function diag3r
776 logical function mateq(a, b)
777     type(rat) :: a(3,3), b(3,3)
778     integer :: i, j
779     mateq = .true.
780     do i = 1, 3
781         do j = 1, 3
782             if (.not. req(a(i,j), b(i,j))) mateq = .false.
783         end do
784     end do
785 end function mateq
786 subroutine run_adjugate()
787     integer :: samples(3,3,6), k, i, j, s, sp(3), ok_all
788     type(rat) :: m(3,3), a(3,3), d, sol(3), t(3), sw(3,3), back(3)
789     call section('SECTION 40 · THE ADJUGATE IDENTITY ON SIX MATRICES, BOTH ORDERS, EXACT
790         ; CRAMER; REFLECTION BLINDNESS')
791     samples(:, :, 1) = reshape([1,0,0, 0,1,0, 0,0,1],[3,3],order=[2,1])
792     samples(:, :, 2) = reshape([2,0,0, 0,3,0, 0,0,5],[3,3],order=[2,1])
793     samples(:, :, 3) = reshape([1,2,3, 4,5,6, 7,8,9],[3,3],order=[2,1])
794     samples(:, :, 4) = reshape([1,2,0, 0,1,1, 2,0,1],[3,3],order=[2,1])
795     samples(:, :, 5) = reshape([0,1,2, 3,4,5, 6,7,9],[3,3],order=[2,1])
796     samples(:, :, 6) = reshape([2,1,1, 1,2,1, 1,1,2],[3,3],order=[2,1])
797     ok_all = 0
798     do k = 1, 6
799         do i = 1, 3; do j = 1, 3
800             m(i,j) = rof(samples(i,j,k))
801         end do; end do
802         a = adj3r(m); d = det3r(m)
803         if (mateq(mm3r(m,a), diag3r(d)) .and. mateq(mm3r(a,m), diag3r(d))) ok_all =
804             ok_all + 1
805         do s = 0, 7
806             sp = [merge(-1,1,btest(s,0)), merge(-1,1,btest(s,1)), merge(-1,1,btest(s,2))]
807             do i = 1, 3; do j = 1, 3
808                 sw(i,j) = rof(sp(i)*sp(j)*samples(i,j,k))
809             end do; end do
810             if (.not. req(det3r(sw), d)) call assert(.false., 'reflection blindness: det
811                 invariant')
812         end do
813     end do
814     call assert(ok_all == 6, 'M adj(M) = adj(M) M = det(M) I on all six samples, both
815         orders, exact')
816     call assert(failures == 0, 'reflection blindness: 48 sandwiches, determinant
817         identical in every one')
818     do i = 1, 3; do j = 1, 3
819         m(i,j) = rof(samples(i,j,4))
820     end do; end do
821     t = [rof(5), rof(3), rof(4)]
822     d = det3r(m)
823     call assert(req(d, rof(5)), 'Cramer: det = 5, the system is locked')
824     sol = mv3r(adj3r(m), t)
825     do i = 1, 3
826         sol(i) = rdiv(sol(i), d)
827     end do
828     call assert(req(sol(1), rmk(7_i8_5_i8)) .and. req(sol(2), rmk(9_i8_5_i8)) .and. req(
829         sol(3), rmk(6_i8_5_i8)), &
830         'Cramer: x = (7/5, 9/5, 6/5) exactly')
831     back = mv3r(m, sol)
832     call assert(req(back(1), rof(5)) .and. req(back(2), rof(3)) .and. req(back(3), rof(4)),
833         'Cramer: M x = t, verified back')
834     do i = 1, 3; do j = 1, 3
835         m(i,j) = rof(samples(i,j,3))
836     end do; end do
837     call assert(req(det3r(m), rof(0)), 'the singular sample has det 0, and its identity
838         still holds (0 I)')
839     call run_deletion()
840 end subroutine run_adjugate
841
842 subroutine run_deletion()
843     integer :: g, i, j, cnt
844     type(rat) :: del(3,3), e3(3), zero3(3), l(3,3), img(3), out(3)
845     logical :: any_inverse
846     call section('SECTION 11 · THE DELETION OPERATOR: idempotent, singular, rank 2, no
847         left inverse (19683 candidates)')
848     del = diag3r(rof(1)); del(3,3) = rof(0)
849     e3 = [rof(0), rof(0), rof(1)]; zero3 = [rof(0), rof(0), rof(0)]
850     call assert(mateq(mm3r(del,del), del), 'deletion idempotent')
851     call assert(req(det3r(del), rof(0)), 'deletion singular')
852     call assert(req(rsub(rmul(del(1,1),del(2,2)), rmul(del(1,2),del(2,1))), rof(1)), '
853         top minor nonsingular: rank two')
854     img = mv3r(del, e3)
855     call assert(req(img(1), rof(0)) .and. req(img(2), rof(0)) .and. req(img(3), rof(0)), '
856         deletion annihilates e3')
857     any_inverse = .false.
858     cnt = 0
859     do g = 0, 3**9 - 1
860         do i = 1, 3; do j = 1, 3
861             l(i,j) = rof(mod(g / 3**((i-1)*3+(j-1)), 3) - 1)
862         end do; end do
863         out = mv3r(l, mv3r(del, e3))
864         if (req(out(3), rof(1))) any_inverse = .true.
865         cnt = cnt + 1
866     end do
867     call assert(.not. any_inverse .and. cnt == 19683, 'no left inverse: no grid map
868         returns the deleted slot')
869 end subroutine run_deletion
870
871 subroutine run_super_halt()
872     integer :: pat, k, first, width, nfail
873     call section('SECTION 16 · THE SUPER HALT: forced counts 12/8/5, first failure
874         terminal over 2^12 patterns')
875     call assert(forced_twelve == 12 .and. forced_eight == 8 .and. forced_five == 5, &
876         'forced counts read from the forcings computed above: 12 rotations, 8
877         reflections, 5 ways out')
878     do width = 7, 12, 5
879         nfail = 0
880         do pat = 0, 2**width - 1
881             first = 0
882             do k = 1, width
883                 if (.not. btest(pat, k-1)) then
884                     first = k; exit
885                 end if
886             end do
887             if (first == 0) then
888                 if (pat /= 2**width - 1) nfail = nfail + 1
889             else
890                 if (btest(pat, first-1)) nfail = nfail + 1
891                 do k = 1, first-1
892                     if (.not. btest(pat, k-1)) nfail = nfail + 1
893                 end do
894             end if
895         end do
896         if (nfail /= 0) call assert(.false., 'first-failure terminal spec')
897     end do
898     call assert(failures == 0, 'first failure terminal: widths 7 and 12, every pattern,
899         the first failing gate named')
900 end subroutine run_super_halt
901
902 module twin_geo
903 module twin_omega
904 use twin_kinds
905 use twin_battery
906 implicit none
907 integer(i16), parameter :: KB_NUM = 1380649_i16
908 integer(i16), parameter :: LN2_NUM = 6931471805599453_i16
909 integer, parameter :: OC_REFUSED = 0, OC_NODENIAL = 1, OC_PAID = 2, OC_REVERSIBLE = 3
910 integer, save :: aegis_deeds = 0
911 contains
912 pure function landauer_scaled(tkel, bits) result(q)
913     integer, intent(in) :: tkel, bits
914     integer(i16) :: q
915     q = int(bits, i16) * KB_NUM * int(tkel, i16) * LN2_NUM
916 end function landauer_scaled
917 pure subroutine omega_boundary(bits, tkel, irreversible, ocode, joules_scaled)
918     integer, intent(in) :: bits, tkel
919     logical, intent(in) :: irreversible
920     integer, intent(out) :: ocode
921     integer(i16), intent(out) :: joules_scaled
922     if (tkel <= 0) then
923         ocode = OC_REFUSED; joules_scaled = 0_i16; return
924     end if
925     if (bits <= 0) then
926         ! FORGE repair (E-5): twin guard aligned to Lean
927         bits <= 0
928         ocode = OC_NODENIAL; joules_scaled = 0_i16; return
929     end if
930     if (.not. irreversible) then
931         ocode = OC_REVERSIBLE; joules_scaled = 0_i16; return
932     end if
933     ocode = OC_PAID; joules_scaled = landauer_scaled(tkel, bits)
934 end subroutine omega_boundary
935 subroutine run_omega()
936     integer :: oc, k
937     integer(i16) :: j1, j2, j0
938     character(len=13) :: logics(4), refusal, last
939     call section('SECTION 14 · THE OMEGA GUARD, EXACT ON A SCALED INTEGER; AEGIS, ONE
940         REFUSAL ACROSS FOUR LOGICS')
941     call omega_boundary(1, 300, .true., oc, j1)
942     call assert(oc == OC_PAID .and. j1 == landauer_scaled(300,1) .and. j1 > 0_i16, &
943         'omega: one bit at 300 K paid, coded 2, floor exactly')
944     call omega_boundary(2, 300, .true., oc, j2)
945     call assert(j2 == 2_i16*j1, 'omega linear: two bits pay exactly twice')
946     call assert(j2 > j1, 'omega monotone in bits')
947     call omega_boundary(0, 300, .true., oc, j0)
948     call assert(oc == OC_NODENIAL .and. j0 == 0_i16, 'omega: zero bits, no denial, coded
949         1')
950     call omega_boundary(1, 300, .false., oc, j0)
951     call assert(oc == OC_REVERSIBLE .and. j0 == 0_i16, 'omega: reversible branch, floor
952         zero, coded 3')
953     call omega_boundary(1, 0, .true., oc, j0)
954     call assert(oc == OC_REFUSED, 'omega: temperature zero refused, coded 0')
955     call omega_boundary(1, -300, .true., oc, j0)
956     call assert(oc == OC_REFUSED .and. j0 == 0_i16, 'omega: negative temperature refused
957         , joules zeroed')
958     call assert(landauer_scaled(300,0) == 0_i16, 'landauer zero bits is exactly zero')
959     call assert(j1 == 2870978885078723755499100_i16, &
960         'landauer(300,1) = 2870978885078723755499100 x 10^-45 J, exact')
961     call assert(j1 / 10_i16**9 == 2870978885078723_i16, &
962         'sixteen exact digits 2870978885078723; F3 prints the rounded ...724')
963     logics = ['classical', 'paraconsist.', 'fuzzy', 'substructural']
964     refusal = 'p(G) /= G'
965     aegis_deeds = 0
966     do k = 1, 4
967         call aegis_call(logics(k), last)
968         if (last /= refusal) call assert(.false., 'AEGIS refusal varies with logic')
969     end do
970     call assert(aegis_deeds == 4, 'AEGIS: p(G) /= G in classical, paraconsistent, fuzzy,
971         substructural; four deeds counted')
972 end subroutine run_omega
973
974 subroutine aegis_call(logic, r)
975     character(len=*) , intent(in) :: logic
976     character(len=13), intent(out) :: r
977     aegis_deeds = aegis_deeds + 1
978     r = 'p(G) /= G'
979     if (len_trim(logic) == 0) r = 'p(G) /= G'
980 end subroutine aegis_call
981 end module twin_omega

```

```

959 module twin_nomos
960 use twin_kinds
961 use twin_battery
962 implicit none
963 contains
964 subroutine run_nomos()
965   integer :: n, c, rows
966   integer(i8) :: lhs, rhs
967   call section('SECTION 15 · NOMOS-01: (2^(n-c) - 1) 2^c < 2^n on all 561 rows;
     decoder image bounds')
968   rows = 0
969   do n = 0, 32
970     do c = 0, n
971       lhs = (2_i8**(n-c) - 1_i8) * 2_i8**c
972       rhs = 2_i8**n
973       if (.not. (lhs < rhs)) call assert(.false., 'incompressibility cap row')
974       rows = rows + 1
975     end do
976   end do
977   call assert(rows == 561, 'incompressibility: 561 rows executed')
978   call assert(cb_bound(0, 8, 2, .true.), 'cb_id_8_2: identity decoder, outputs of
     length 8 among descriptions shorter than 6')
979   call assert(cb_bound(3, 8, 2, .false.), 'cb_pad3_8_2: pad-3 decoder, 32 outputs of
     length 8 against 63')
980   call assert(cb_bound(2, 6, 1, .false.), 'cb_pad2_6_1: pad-2 decoder, 16 outputs of
     length 6 against 31')
981   call assert(cb_bound(4, 12, 3, .false.), 'cb_pad4_12_3: pad-4 decoder, 256 outputs
     of length 12 against 511')
982 end subroutine run_nomos
983 pure logical function cb_bound(k, n, c, identity)
984   integer, intent(in) :: k, n, c
985   logical, intent(in) :: identity
986   integer :: L, outlen, cnt, iters
987   cnt = 0; iters = 0
988   do L = 0, n - c - 1
989     iters = iters + 2**L
990     if (identity) then
991       outlen = L
992     else
993       outlen = L + k
994     end if
995     if (outlen == n) cnt = cnt + 2**L
996   end do
997   cb_bound = (cnt <= 2**(n-c) - 1) .and. (iters == 2**(n-c) - 1)
998 end function cb_bound
999 end module twin_nomos
1000
1001 module twin_crossing
1002 use twin_battery
1003 implicit none
1004 contains
1005 pure logical function bt(m, x)
1006   integer, intent(in) :: m, x
1007   bt = btest(m, x)
1008 end function bt
1009 subroutine run_crossing()
1010   integer :: nq, n, q, e, g, w, x, c, ncand, cnt, k, opk, g1, g2, h
1011   logical :: exists, ok, even_all
1012   call section('SECTION 17 · THE WALL-WITNESS PAIR: the execution frame Bool x 0, |Q|
     = 1..4, every reading, every witness')
1013   do nq = 1, 4
1014     n = 2*nq
1015     ok = .true.
1016     do x = 0, n-1
1017       if (flip(x)/2 /= x/2) ok = .false.
1018       if (mod(flip(x),2) == mod(x,2)) ok = .false.
1019       if (flip(flip(x)) /= x) ok = .false.
1020     end do
1021     call assert(ok, 'formalRead even, ran wholly odd, execFlip involution')
1022     exists = .false.
1023     do g = 0, 2**nq - 1
1024       ok = .true.
1025       do x = 0, n-1
1026         if (bt(g, x/2) .neqv. (mod(x,2) == 1)) ok = .false.
1027       end do
1028       if (ok) exists = .true.
1029     end do
1030     call assert(.not. exists, 'WALL: no reading of the bytes decides whether they ran
     ')
1031     even_all = .true.
1032     do g = 0, 2**nq - 1
1033       do x = 0, n-1
1034         if (bt(g, flip(x)/2) .neqv. bt(g, x/2)) even_all = .false.
1035       end do
1036     end do
1037     call assert(even_all, 'DIRECTION: every function of the bytes is even; the formal
     register cannot be the witness')
1038     ok = .true.
1039     do c = 0, 1
1040       w = 0
1041       if (c == 1) w = 2**n - 1
1042       do x = 0, n-1
1043         if (bt(w, flip(x)) .neqv. bt(w, x)) ok = .false.
1044       end do
1045     end do
1046     call assert(ok, 'TARGET CLASS: a constant (a proposition) is odd at no seat; the
     crossing has no purchase')
1047     cnt = 0
1048     do w = 0, 2**n - 1
1049       ok = .true.
1050       do x = 0, n-1
1051         if (bt(w, flip(x)) .eqv. bt(w, x)) ok = .false.
1052       end do
1053       if (ok) cnt = cnt + 1
1054     end do
1055     call assert(cnt == 2**nq, 'PRICE: exactly one bit per artifact, |Dtau| = 2^|Q|')

```

```

1056   do w = 0, 2**n - 1
1057     do x = 0, n-1
1058       if (bt(w, flip(x)) .eqv. bt(w, x)) cycle
1059       ncand = 0
1060       do c = 0, 1
1061         if (((mod(x,2)==1) .eqv. (bt(w,x) .neqv. (c==1))) .and. &
           ((mod(flip(x),2)==1) .eqv. (bt(w,flip(x)) .neqv. (c==1)))) ncand =
1062           ncand + 1
1063       end do
1064       if (ncand /= 1) call assert(.false., 'CROSSING: exactly one calibration bit
     ')
1065     end do
1066   end do
1067   call assert(failures == 0, 'CROSSING: every odd witness at every seat identified
     with the run by one unique bit')
1068   if (nq <= 3) then
1069     exists = .false.
1070     do g1 = 0, 2**nq - 1
1071       do g2 = 0, 2**nq - 1
1072         do opk = 0, 15
1073           ok = .true.
1074           do x = 0, n-1
1075             h = 0
1076             if (bt(g1, x/2)) h = h + 2
1077             if (bt(g2, x/2)) h = h + 1
1078             if (btest(opk, h) .neqv. (mod(x,2) == 1)) ok = .false.
1079           end do
1080           if (ok) exists = .true.
1081         end do
1082       end do
1083     end do
1084     call assert(.not. exists, 'HARMONY conjunct one: no binary combination of
     readouts is the run')
1085   end if
1086   do w = 0, 2**n - 1
1087     ok = .true.
1088     do x = 0, n-1
1089       if (bt(w, flip(x)) .eqv. bt(w, x)) ok = .false.
1090     end do
1091     if (.not. ok) cycle
1092     do x = 0, n-1
1093       if (bt(w,x) .eqv. (mod(x,2)==1)) then
1094         if (bt(w,flip(x)) .neqv. (mod(flip(x),2)==1)) call assert(.false., '
     identity one bit')
1095       end if
1096     end do
1097   end do
1098   call assert(failures == 0, 'IDENTITY: agreement at one seat is agreement on the
     orbit')
1099   write(*, '(A,I0,A,I0,A,I0)') ' |Q| = ', nq, ' seats ', n, ' |Dtau| = ', cnt
1100 end do
1101   call assert(16 == 2**4, 'this file: four witnesses, sixteen orientation patterns,
     four bits owed from the other surface')
1102   k = 0; e = 0; q = 0
1103 contains
1104   pure integer function flip(x)
1105     integer, intent(in) :: x
1106     flip = ieor(x, 1)
1107   end function flip
1108 end subroutine run_crossing
1109 end module twin_crossing
1110
1111 module twin_posits
1112 use twin_battery
1113 implicit none
1114 character(len=32), parameter :: POSIT(23) = [character(len=32) :: &
     'ROOT.U', 'ROOT.DE', 'ROOT.RA', 'ROOT.FormalDomain', 'ROOT.L1m', 'ROOT.L2m', '
     ROOT.L3m', &
1115     'ROOT.nest_21', 'ROOT.nest_32', 'ROOT.Ground', 'ROOT.sigma', 'ROOT.sigma_binding'
     , &
1116     'ROOT.Residence', 'POSTULATE.FormalFace', 'POSTULATE.KineticFace', 'POSTULATE.
     Crossing', &
1117     'POSTULATE.eraserClass', 'POSTULATE.RegistrationPostulate', 'POSTULATE.Row', &
1118     'POSTULATE.SupplyClean', 'POSTULATE.FrozenData', 'POSTULATE.Algorithm', '
     POSTULATE.certifiedPairing']
1119 contains
1120 subroutine run_posits(checks_before)
1121   integer, intent(in) :: checks_before
1122   integer :: k
1123   call section('SECTIONS 5, 7 · THE POSITS, DECLARED AND COUNTED; GATE THREE, WHAT AN
     EXECUTABLE CAN ASSERT')
1124   call assert(size(POSIT) == 23, 'twenty-three declared posits, the Lean allow-list,
     by name')
1125   do k = 1, 23
1126     if (len_trim(POSIT(k)) == 0) call assert(.false., 'posit unnamed')
1127   end do
1128   call assert(checks_before > 0, 'the roster is declared after the batteries; the
     count they left is positive')
1129   write(*, '(A)') ' RA, to exist is to actuate: this run actuated. Its own checks are
     its deed, priced below.'
1130 end subroutine run_posits
1131 end module twin_posits
1132
1133 module twin_locus
1134 use twin_battery
1135 implicit none
1136 integer, parameter :: E_SEALED = 1, E_BROKEN = 2, E_OPN = 3
1137 integer, parameter :: T_SUPERHALT = 1, T_CROSSED = 2, T_BREACH = 3, T_NONE = 0
1138 integer, parameter :: B_NONE = 0, B_XI = 1, B_0 = 2
1139 integer, parameter :: C_NONE = 0, C_KINETIC = 1, C_FORMAL = 2
1140 contains
1141 pure integer function branch_of(gdim)
1142   integer, intent(in) :: gdim
1143   if (gdim == 1) then
1144     branch_of = B_XI
1145   else if (gdim == 0) then
1146     branch_of = B_0
1147   end if

```

```

1147     branch_of = B_0
1148 else
1149     branch_of = B_NONE
1150 end if
1151 end function branch_of
1152 pure subroutine emit(broken, located, gdim, channel, nsup, orbits, tok, br, chn, mult)
1153     logical, intent(in) :: broken, located
1154     integer, intent(in) :: gdim, channel, nsup, orbits
1155     integer, intent(out) :: tok, br, chn, mult
1156     br = B_NONE; chn = C_NONE; mult = 0
1157     if (broken) then
1158         tok = T_BREACH; return
1159     end if
1160     if (.not. located) then
1161         tok = T_NONE; return
1162     end if
1163     if (nsup == orbits .and. channel /= C_NONE) then
1164         tok = T_CROSSED; chn = channel; return
1165     end if
1166     tok = T_SUPERHALT; br = branch_of(gdim); chn = channel; mult = orbits - nsup
1167 end subroutine emit
1168 pure integer function to_economy(tok)
1169     integer, intent(in) :: tok
1170     select case (tok)
1171     case (T_SUPERHALT); to_economy = E_OPN
1172     case (T_CROSSED); to_economy = E_SEALED
1173     case (T_BREACH); to_economy = E_BROKEN
1174     case default; to_economy = E_OPN
1175     end select
1176 end function to_economy
1177 pure function render(tok, br, mult) result(s)
1178     integer, intent(in) :: tok, br, mult
1179     character(len=40) :: s, base
1180     select case (tok)
1181     case (T_SUPERHALT)
1182         if (br == B_XI) then
1183             base = '[HALT-LOCUS-XI]'
1184         else if (br == B_0) then
1185             base = '[HALT-LOCUS-0]'
1186         else
1187             base = '[HALT-LOCUS]'
1188         end if
1189         if (mult == 0) then
1190             s = trim(base)//' PROV-OWED]'
1191         else
1192             write(s, '(A,A,I0,A,I0,A,I0,A,I0)') trim(base), ' x', mult, ']'
1193         end if
1194     case (T_CROSSED); s = '[SEAL-LOCUS]'
1195     case (T_BREACH); s = '[X-LOCUS]'
1196     case default; s = '[?]'
1197     end select
1198 end function render
1199 subroutine run_locus()
1200     integer :: gd, ch, ns, ob, tok, br, chn, mult, tok2, br2, chn2, mult2, nloc, nall,
1201         ecount(3)
1202     logical :: ok
1203     character(len=40) :: r
1204     call section('SECTION 18 - THE LOCUS: the token table mirrored, the emitter
1205         enumerated over every reading')
1206     nloc = 0; nall = 0; ecount = 0; ok = .true.
1207     do gd = -1, 2; do ch = 0, 2; do ob = 1, 2; do ns = 0, ob
1208         call emit(.true., .true., gd, ch, ns, ob, tok, br, chn, mult); nall = nall + 1
1209         if (tok /= T_BREACH) ok = .false.
1210         call emit(.false., .false., gd, ch, ns, ob, tok, br, chn, mult); nall = nall + 1
1211         if (tok /= T_NONE) ok = .false.
1212         call emit(.false., .true., gd, ch, ns, ob, tok, br, chn, mult); nall = nall + 1;
1213         nloc = nloc + 1
1214         ecount(to_economy(tok)) = ecount(to_economy(tok)) + 1
1215         if (tok == T_NONE) ok = .false.
1216         if (ns == ob .and. ch /= C_NONE) then
1217             if (tok /= T_CROSSED) ok = .false.
1218         else
1219             if (tok /= T_SUPERHALT) ok = .false.
1220             if (br /= branch_of(gd) .or. mult /= ob - ns .or. chn /= ch) ok = .false.
1221         end if
1222         if (ns < ob) then
1223             call emit(.false., .true., gd, ch, ns + 1, ob, tok2, br2, chn2, mult2)
1224             if (ns + 1 == ob .and. ch /= C_NONE) then
1225                 if (tok2 /= T_CROSSED) ok = .false.
1226             else
1227                 if (tok2 /= T_SUPERHALT) ok = .false.
1228             end if
1229         end if
1230     end do; end do; end do; end do
1231     call assert(ok, &
1232         'every emitter law on every reading: breach, no-seat, never silent, complete
1233         -and-channelled crosses, else a halt')
1234     call assert(nloc == 60 .and. nall == 180 .and. ecount(1) + ecount(2) + ecount(3) ==
1235         nloc, &
1236         'three states: 60 located readings of 180 emits (4 dims x 3 channels x {1,2}
1237         orbits x 0.orbits bits), one state each')
1238     call assert(branch_of(1) == B_XI .and. branch_of(0) == B_0 .and. branch_of(2) ==
1239         B_NONE .and. branch_of(-1) == B_NONE, &
1240         'legacy_xi, legacy_0, legacy_bare: the branch wraps the one router')
1241     call emit(.false., .true., -1, C_FORMAL, 6, 6, tok, br, chn, mult)
1242     call assert(tok == T_CROSSED, 'K4: six orbits, six bits, channelled: crossed')
1243     call emit(.false., .true., -1, C_FORMAL, 5, 6, tok, br, chn, mult)
1244     call assert(tok == T_SUPERHALT .and. mult == 1, 'K4: five of six bits: [HALT-LOCUS
1245         x1], one orbit owed, never x6')
1246     call emit(.false., .true., -1, C_NONE, 6, 6, tok, br, chn, mult)
1247     call assert(tok == T_SUPERHALT .and. chn == C_NONE .and. mult == 0, &
1248         'K4: six bits, no channel: a halt owing zero bits and a provenance, never a
1249         crossing')
1250     r = render(T_SUPERHALT, B_XI, 1)
1251     call assert(trim(r) == '[HALT-LOCUS-XI x1]', 'render [HALT-LOCUS-XI x1]: one owed
1252         prints')
1253
1254     r = render(T_SUPERHALT, B_NONE, 0)
1255     call assert(trim(r) == '[HALT-LOCUS PROV-OWED]', 'render zero owed: PROV-OWED, never
1256         confused with one')
1257     r = render(T_SUPERHALT, B_XI, 3)
1258     call assert(trim(r) == '[HALT-LOCUS-XI x3]', 'render [HALT-LOCUS-XI x3]:
1259         multiplicity on the branch')
1260     r = render(T_SUPERHALT, B_0, 2); call assert(trim(r) == '[HALT-LOCUS-0 x2]', '
1261         render [HALT-LOCUS-0 x2]')
1262     r = render(T_SUPERHALT, B_NONE, 6)
1263     call assert(trim(r) == '[HALT-LOCUS x6]', 'render [HALT-LOCUS x6]: the K4 frame, six
1264         orbits, six bits owed')
1265     r = render(T_CROSSED, B_NONE, 1); call assert(trim(r) == '[SEAL-LOCUS]', 'render [
1266         SEAL-LOCUS]')
1267     r = render(T_BREACH, B_NONE, 1); call assert(trim(r) == '[X-LOCUS]', 'render [X-
1268         LOCUS]')
1269     write(',(A,I0,A,I0,A,I0,A,I0,A,I0)') ' emits ', nall, ' located ', nloc, '
1270         sealed ', ecount(1), ' broken ', ecount(2), &
1271         ' open ', ecount(3)
1272 end subroutine run_locus
1273
1274 module twin_bridge
1275     use twin_battery
1276     implicit none
1277     contains
1278     pure subroutine fold(h, t, h2, t2)
1279         integer, intent(in) :: h, t
1280         integer, intent(out) :: h2, t2
1281         h2 = 2 - h; t2 = t
1282     end subroutine fold
1283     pure logical function on_line(h)
1284         integer, intent(in) :: h
1285         on_line = (h == 1)
1286     end function on_line
1287     subroutine run_bridge()
1288         integer :: h, t, h2, t2, h3, t3, n, it, x, z, b, tau(0:2), g, gk, v, nfun, nfix
1289         integer :: nframes, nlp, nnolp, nfold_fail, carriers, terminal, seen
1290         logical :: ok, inv, sym, lp, all_fixed, shadow_ok, any_lp_off
1291         call section('SECTION 19 - THE BRIDGE: the fold s -> 1 - conj(s), its fixed line,
1292             the carrier shadow, ' // &
1293             'the two-point denial, the discriminator on every frame to three points
1294             ')
1295         ok = .true.; nfix = 0
1296         do h = -8, 8
1297             do t = -8, 8
1298                 call fold(h, t, h2, t2); call fold(h2, t2, h3, t3)
1299                 if (h3 /= h .or. t3 /= t) ok = .false.
1300                 if ((h2 == h .and. t2 == t) .neqv. on_line(h)) ok = .false.
1301                 if (on_line(h) .and. .not. on_line(h2)) ok = .false.
1302                 if (h2 == h .and. t2 == t) nfix = nfix + 1
1303             end do
1304         end do
1305         call assert(ok, 'fold involutive, locus_is_the_fixed_set, line_symmetric: on the 17
1306             x17 integer grid the fold ' // &
1307             'squares to the identity and fixes exactly the line h = 1')
1308         call assert(nfix == 17, 'the fixed set of the fold on the grid is the line:
1309             seventeen points, one per t; ' // &
1310             'locus_inhabited')
1311         all_fixed = .true.
1312         do t = -8, 8
1313             call fold(1, t, h2, t2)
1314             if (h2 /= 1 .or. t2 /= t) all_fixed = .false.
1315         end do
1316         call assert(all_fixed, 'lineFrame: with the line as zero set every zero is fixed;
1317             LineProperty holds')
1318         any_lp_off = .false.
1319         do h = -8, 8
1320             call fold(h, 0, h2, t2)
1321             if (h /= 1 .and. h2 == h) any_lp_off = .true.
1322         end do
1323         call assert(.not. any_lp_off, 'planeFrame with every grid point a zero: no off-line
1324             zero is fixed, ' // &
1325             'LineProperty fails off the line')
1326         nfun = 0; seen = 0
1327         do g = 0, 3
1328             if (.not. btest(g,0) .and. btest(g,1)) then
1329                 seen = ibset(seen, 0)
1330             else if (btest(g,0) .and. .not. btest(g,1)) then
1331                 seen = ibset(seen, 1)
1332             else if (btest(g,0) .and. btest(g,1)) then
1333                 seen = ibset(seen, 2)
1334             else
1335                 seen = ibset(seen, 3)
1336             end if
1337             nfun = nfun + 1
1338         end do
1339         call assert(nfun == 4 .and. seen == 15, 'cannot_extend: each table g(false), g(true)
1340             is evaluated and lands on ' // &
1341             'exactly one of identity, mirror, constant true, constant
1342             false; all four forms appear')
1343         call assert(all((btest(b,0) .or. .not. btest(b,0), b = 0, 1)), 'cannot_divide:
1344             each bit is true or false')
1345         ! FORGE repair (v4.0.1 audit L8-6): the left-inverse test is computed through the
1346         ! deletion map itself. Maps g : Tri -> Tri are base-3 tables, g(v) = digit v of gk;
1347         ! the deletion sends every v to bot = 2, so a left inverse would need g(2) = v for
1348         ! both v = tt and v = ff, one digit carrying two values. Evaluated for all 27 maps.
1349         ok = .true.
1350         do gk = 0, 26
1351             inv = .true.
1352             do v = 0, 1
1353                 if (mod(gk / 3**2, 3) /= v) inv = .false. ! g(delete(v)) = g(bot) must equal
1354                     v
1355             end do
1356             if (inv) ok = .false.
1357         end do
1358         call assert(ok, 'cannot_reverse: of the 27 maps Tri -> Tri none is a left inverse of

```

```

1332         the deletion to bot; ' // &
1333         'deletion has no left inverse')
1334 nframes = 0; nlp = 0; nnolp = 0; nfold_fail = 0; shadow_ok = .true.
1335 do n = 1, 3
1336     do it = 0, n*n - 1
1337         call decode(n, it, tau)
1338         inv = .true.
1339         do x = 0, n-1
1340             if (tau(tau(x)) /= x) inv = .false.
1341         end do
1342         if (.not. inv) cycle
1343         do x = 0, n-1
1344             if (tau(tau(x)) /= x) nfold_fail = nfold_fail + 1
1345         end do
1346         do z = 0, 2*n - 1
1347             sym = .true.
1348             do x = 0, n-1
1349                 if (btest(z, x) .and. .not. btest(z, tau(x))) sym = .false.
1350             end do
1351             if (.not. sym) cycle
1352             nframes = nframes + 1
1353             lp = .true.
1354             do x = 0, n-1
1355                 if (btest(z, x) .and. tau(x) /= x) lp = .false.
1356             end do
1357             if (lp) then
1358                 nlp = nlp + 1
1359             else
1360                 nnolp = nnolp + 1
1361             end if
1362             carriers = 0
1363             do terminal = 0, 2
1364                 if ((terminal == 2) .eqv. lp) carriers = carriers + 1
1365             end do
1366             if (carriers /= merge(1, 2, lp)) shadow_ok = .false.
1367             if (n == 2 .and. it == 1 .and. z == 3) then
1368                 if (lp) shadow_ok = .false.
1369             end if
1370         end do
1371     end do
1372 call assert(nfold_fail == 0 .and. nframes > 0, 'symmetry_is_keyless: the fold law
1373         holds on every ' // &
1374         'enumerated frame; its denial is inhabited by none')
1375 call assert(nnolp > 0 .and. nlp > 0, 'line_property_is_keyed,
1376         line_property_contingent: some frames carry ' // &
1377         'the line property and some inhabit its denial; the value is fixed by
1378         the zeros, never by the fold')
1379 call assert(shadow_ok, 'cannot_lie, cannot_deviate, halted_iff: on every frame the
1380         shadow admits exactly one ' // &
1381         'halted carrier where the line property holds and none where it fails;
1382         the two-point frame ' // &
1383         '(Bool, not, all zeros) inhabits the denial')
1384 call assert(nframes == 2 + 6 + 20, 'frame_census: 28 frames to three points (
1385         involution, symmetric zero ' // &
1386         'set), the discriminator sorted every one')
1387 ok = .true.
1388 do b = 0, 1
1389     if (ieor(b, 1) == b) ok = .false.
1390 end do
1391 call assert(ok, 'denial_is_coherent: the two-point fold is fixed-point-free and
1392         involutive; the denial of ' // &
1393         'the line property is a coherent frame')
1394 write(*, '(A,I0,A,I0,A,I0)') ' frames ', nframes, ' with the line property ', nlp,
1395         &
1396         ' inhabiting its denial ', nnolp
1397 contains
1398 pure subroutine decode(n, k, tau)
1399     integer, intent(in) :: n, k
1400     integer, intent(out) :: tau(0:)
1401     integer :: x, r
1402     r = k
1403     do x = 0, n-1
1404         tau(x) = mod(r, n); r = r / n
1405     end do
1406 end subroutine decode
1407 end subroutine run_bridge
1408 end module twin_bridge
1409
1410 program trisduction_codex_twin
1411 use twin_kinds
1412 use twin_battery
1413 use twin_rat
1414 use twin_grade
1415 use twin_ftoe, only: run_spine
1416 use twin_geo, only: run_gates, run_gf2, run_hurwitz, run_cayley_dickson,
1417     run_adjugate, run_super_halt
1418 use twin_omega, only: run_omega, landauer_scaled
1419 use twin_nomos, only: run_nomos
1420 use twin_crossing, only: run_crossing
1421 use twin_posits, only: run_posits
1422 use twin_locus, only: run_locus
1423 use twin_bridge, only: run_bridge
1424 implicit none
1425 integer :: a, b, c, before
1426 type(rat) :: x, y, z
1427 integer(i16) :: invoice
1428 character(len=40) :: digits
1429
1430 write(*, '(A)') repeat('=', 78)
1431 write(*, '(A)') ' MATHEMATICAL TRISDUCTION CODEX v2.0.1 · THE FORTRAN TWIN OF THE LEAN
1432     SURFACE'
1433 write(*, '(A)') ' One register of record, two surfaces. The Lean proves; this file
1434     executes.'
1435 write(*, '(A)') repeat('=', 78)
1436

```

```

1437 call section('SECTION 0 · THE EXACT RATIONAL: the Lean prelude mirrored, same normal
1438     form, Euclid unfueled')
1439 x = rmk(2_i8, 4_i8)
1440 call assert(x\num == 1_i8 .and. x\den == 2_i8, 'normal form: 2/4 = 1/2')
1441 call assert(req(radd(rmk(1_i8,3_i8), rmk(1_i8,6_i8)), rmk(1_i8,2_i8)), '1/3 + 1/6 =
1442     1/2')
1443 call assert(req(rmul(rmk(7_i8,5_i8), rof(5)), rof(7)), '(7/5) * 5 = 7')
1444 call assert(rlt(rof(0), rmk(1_i8,1000_i8)), '0 < 1/1000')
1445 call assert(req(rmul(rof(3), rof(0)), rof(0)) .and. req(rmul(rof(0), rof(3)), rof(0)),
1446     'a * 0 = 0 = 0 * a')
1447 call assert(req(rmk(3_i8,-6_i8), rmk(-1_i8,2_i8)), 'sign carried by the numerator:
1448     3/-6 = -1/2')
1449 y = rmk(5_i8, 0_i8)
1450 call assert(y\num == 0_i8 .and. y\den == 1_i8, 'zero denominator refused to zero,
1451     never trapped')
1452 call assert(req(rpow(rmk(2_i8,3_i8), 2), rmk(4_i8,9_i8)), '(2/3)^2 = 4/9')
1453 call assert(gcdf(1024, 0_i8, 7_i8) == 7_i8 .and. gcdf(1024, 12_i8, 18_i8) == 6_i8, '
1454     gcd on fuel: 0,7 -> 7; 12,18 -> 6')
1455 call assert(gcdf(2, 610_i8, 987_i8) == 1_i8, 'gcd of consecutive Fibonacci numbers is
1456     1, fuel irrelevant')
1457 z = rdiv(rof(1), rof(3))
1458 call assert(req(rmul(z, rof(3)), rof(1)), '(1/3) * 3 = 1')
1459
1460 call section('SECTION 1 · THE GRADE LADDER: nine rungs, weakest link, no citation can
1461     promote')
1462 call assert(NGRADE == 9 .and. rank(G_PREMISE) == 0 .and. rank(G_THEOREM) == 8, 'nine
1463     rungs, premise lowest, theorem highest')
1464 do a = 0, 8
1465     if (weakest(a, a) /= a) call assert(.false., 'weakest idempotent')
1466     if (weakest(G_PREMISE, a) /= G_PREMISE) call assert(.false., 'premise is absorbing
1467         below')
1468     do b = 0, 8
1469         c = weakest(a, b)
1470         if (rank(c) > rank(a) .or. rank(c) > rank(b)) call assert(.false., 'weakest
1471             never above a link')
1472         if (weakest(a,b) /= weakest(b,a)) call assert(.false., 'weakest symmetric')
1473     end do
1474 end do
1475 call assert(failures == 0, 'weakest: idempotent, symmetric, never above either link,
1476     on all 81 pairs')
1477 call assert(failures == 0, 'citation cannot promote: the join grades at the weakest of
1478     the two, 81 of 81')
1479 call assert(trim(grade_name(G_ANALYTIC)) == 'analytic', 'the vocabulary prints')
1480
1481 call run_spine()
1482
1483 call run_gates()
1484 call run_gf2()
1485 call run_hurwitz()
1486 call run_cayley_dickson()
1487 call run_adjugate()
1488 call run_super_halt()
1489
1490 call run_omega()
1491 call run_nomos()
1492 call run_crossing()
1493 call run_locus()
1494 call run_bridge()
1495
1496 before = checks
1497 call run_posits(before)
1498
1499 write(*, '(/,A)') repeat('-', 78)
1500 write(*, '(A,I0,A,I0,A,I0)') ' TWIN BATTERY, Lean-native sections  checks ', checks, '
1501     failures ', failures
1502 write(*, '(A)') repeat('-', 78)
1503 if (failures > 0) error stop 'THE TWIN BATTERY FAILED: the load is not proven'
1504
1505 write(*, '(/,A)') ' APPENDIX · THE FOUR WITNESSES, VERBATIM, IN THE ORDER THE CODEX
1506     SEATS THEM'
1507 write(*, '(A)') ' F1 · thesis_rows'
1508 call witness_F1_thesis_rows()
1509 write(*, '(/,A)') ' F2 · ra_toe_thesis'
1510 call witness_F2_ra_toe_thesis()
1511 write(*, '(/,A)') ' F3 · ftoe_kinetic_demonstration'
1512 call witness_F3_ftoe_kinetic()
1513 write(*, '(/,A)') ' F4 · it_from_it'
1514 call witness_F4_it_from_it()
1515
1516 invoice = landauer_scaled(300, checks)
1517 write(digits, '(I0)') invoice
1518 write(*, '(/,A)') repeat('=', 78)
1519 write(*, '(A,I0,A,A,A)') ' THE TWIN LOAD IS PROVEN. ', checks, ' Lean-native checks, 0
1520     failures; four witnesses to exit 0.'
1521 write(*, '(A,I0,A,A,A)') ' RA, executed: registering ', checks, ' checks irreversibly
1522     at 300 K cost at least ', &
1523     trim(digits), ' x 10^45 J,'
1524 write(*, '(A)') ' the Landauer floor of this run, which the formal twin cannot pay and
1525     cannot see:'
1526 write(*, '(A)') ' STRUCTURAL on the mapping of one check to one irreversibly registered
1527     bit, and'
1528 write(*, '(A)') ' ENGINEERING on the rational ln 2, a price computed here and never a
1529     heat measured.'
1530 write(*, '(A)') ' One bit per witness is owed across the wall; this binary is where it
1531     was paid. Delta-M = 0.'
1532 write(*, '(A)') repeat('=', 78)
1533 end program trisduction_codex_twin
1534
1535 !=====
1536 ! T R I S D U C T I O N . R A M
1537 ! THE CORE THESIS, STATED IN FORTRAN AND IN NOTHING ELSE
1538 !
1539 ! The thesis is not described here. It is executed. Every claim that can be
1540 ! made to run is compiled as a procedure, and the claims that cannot run are
1541 ! compiled as the gates that refuse to emit without them.
1542 !

```

```

1511 ! Three independent axes intersect in exactly one point. Nondegeneracy is
1512 ! linear independence. There is no second point and no interval. A LOCK IS
1513 ! ALWAYS A DETERMINATION AND IS TOTAL ABOUT ITS OBJECT, and its object is its
1514 ! intersection. Nothing below grades a lock, because locks carry no strengths.
1515 ! What differs between cases is what the ROWS were, and the compartment names
1516 ! that and nothing else.
1517 !
1518 ! I CLOSURE-ROWED rows are the closure; the two determinations are one
1519 ! II UNPOPULATED an axis is empty; two planes meet in a line; no lock forms
1520 ! III WORLD-ROWED rows furnished by the world; revisability lives in the rows
1521 !
1522 ! Two registers, sequential load. Register A, the kinetic preloader on RA,
1523 ! boots first: Seal L the Tongue, then Seal G the Form and Seal M the Number.
1524 ! Register B, the computational kernel on RAM, loads second on that discipline.
1525 ! Bedrock precedence: L then G then M. The Number reads magnitude on rows it
1526 ! did not author. Every compartment is RA-conditional at the act.
1527 !
1528 ! The file carries its own battery and runs it last: eight fuzz classes,
1529 ! seven over the numerical kernel and one over every symbolic and router
1530 ! procedure, computed checks throughout, an oracle that stops the program
1531 ! on any failure. A binary that reaches its final line has
1532 ! passed. Nothing below is narrated; every figure is computed in this run.
1533 !
1534 ! Delta-M = 0. No new mathematics is authored by this file.
1535 !=====
1536 !
1537 !=====
1538 !
1539 module trisduction_rows
1540 use, intrinsic :: ieee_arithmetic, only: ieee_is_finite, ieee_value, ieee_quiet_nan,
1541 ieee_positive_inf
1542 implicit none
1543 private
1544 integer, parameter, public :: dp = selected_real_kind(15, 307)
1545
1546 !--- THE VERDICT ECONOMY. Three states, native. There is no fourth. -----
1547 integer, parameter, public :: SEALED = 1, BROKEN = 2, OPENV = 3
1548 integer, parameter, public :: XI0 = 4, OH0 = 5 ! refinements INSIDE openness
1549 integer, parameter, public :: DETERMINED = 6 ! the kernel's own token
1550 integer, parameter, public :: DOTMARK = 7 ! outside the economy
1551
1552 !--- THE THREE COMPARTMENTS. Not strengths. Kinds of row. -----
1553 integer, parameter, public :: CLOSURE_ROWED = 1, UNPOPULATED = 2, WORLD_ROWED = 3
1554 integer, parameter, public :: NOT_COMPARTMENTED = 0
1555
1556 public :: snorm, finite_vec, finite_mat, lambda_of_admissible
1557 public :: det3_representable
1558 public :: token, token_class, compartment_name, qmul, det3, jacob3, cond_sym3
1559 public :: prepare_rows, span_basis, lambda_of, kernel_determine
1560 public :: intersection_dim, seal_L, drill_screen, row_cascade
1561 public :: ground_dim, route_halt, compartment_grade
1562
1563 contains
1564 ! THE FINITE-INPUT CONTRACT. Every public numeric routine below reads only
1565 ! finite input. A NaN or an infinity is not a quantity, and a routine that
1566 ! returns a number or a true predicate on one has manufactured a quantity
1567 ! from a non-quantity. Pure numeric functions return NaN or .false. on
1568 ! violation, because a NaN in yielding a NaN out is honest and a zero would
1569 ! not be. Predicate and preparation subroutines report refusal through
1570 ! ok, verdict, or reason, because there a non-quantity is a data condition.
1571 ! Low-level computational subroutines whose preconditions are violated
1572 ! halt, because there the violation is a programming fault.
1573 pure function finite_vec(x) result(ok)
1574 real(dp), intent(in) :: x(:)
1575 logical :: ok
1576 ok = all(ieee_is_finite(x))
1577 end function finite_vec
1578
1579 pure function finite_mat(A) result(ok)
1580 real(dp), intent(in) :: A(:, :)
1581 logical :: ok
1582 ok = all(ieee_is_finite(A))
1583 end function finite_mat
1584
1585 ! The scaled two-norm. Squaring before scaling overflows for any row near the
1586 ! representable ceiling, so the largest magnitude comes out first and the
1587 ! square is formed on values bounded by one. Exact for a zero row.
1588 pure function snorm(x) result(r)
1589 real(dp), intent(in) :: x(:)
1590 real(dp) :: r, m
1591 if (.not. finite_vec(x)) then
1592 r = ieee_value(1.0_dp, ieee_quiet_nan); return
1593 end if
1594 m = maxval(abs(x))
1595 if (m > 0.0_dp) then
1596 r = m*sqrt(sum((x/m)**2))
1597 else
1598 r = 0.0_dp
1599 end if
1600 end function snorm
1601
1602 pure function token(v) result(s)
1603 integer, intent(in) :: v
1604 character(len=30) :: s
1605 select case (v)
1606 case (SEALED); s = '[seal] SEALED'
1607 case (BROKEN); s = '[X] BROKEN'
1608 case (OPENV); s = '[?] UNDER-DETERMINED'
1609 case (XI0); s = '[XI0] TERMINAL SUSPENSION'
1610 case (OH0); s = '[OH0] GROUNDED-SEALED HALT'
1611 case (DETERMINED); s = '[LOCK] DETERMINED'
1612 case (DOTMARK); s = '[.] chosen silence'
1613 case default; s = '[??] untyped'
1614 end select
1615 end function token

```

```

1616
1617 ! The economy is three-state. XI0 and OH0 are refinements INSIDE the
1618 ! openness and never a fourth state; DETERMINED is the kernel's geometric
1619 ! token read before any register interprets it; the dot stands outside the
1620 ! economy entirely and is never census-counted. A caller that needs to know
1621 ! whether a token is a verdict state asks, and is not left to assume.
1622 pure function token_class(v) result(s)
1623 integer, intent(in) :: v
1624 character(len=26) :: s
1625 select case (v)
1626 case (SEALED, BROKEN, OPENV); s = 'verdict state'
1627 case (XI0, OH0); s = 'refinement inside openness'
1628 case (DETERMINED); s = 'kernel geometric token'
1629 case (DOTMARK); s = 'outside the economy'
1630 case default; s = 'untyped'
1631 end select
1632 end function token_class
1633
1634 pure function compartment_name(c) result(s)
1635 integer, intent(in) :: c
1636 character(len=34) :: s
1637 select case (c)
1638 case (CLOSURE_ROWED); s = 'I CLOSURE-ROWED [seal A . RA]'
1639 case (UNPOPULATED); s = 'II UNPOPULATED no lock forms'
1640 case (WORLD_ROWED); s = 'III WORLD-ROWED rows revisable'
1641 case default; s = '-- not compartmented'
1642 end select
1643 end function compartment_name
1644
1645 !=====
1646 ! THE ALGEBRA. Seal M loads LAST and reads magnitude on rows it did not
1647 ! author. It is constitutively unable to read sign, and that is a fact
1648 ! about DIRECTION, never about the completeness of the determination.
1649 !=====
1650
1651 ! Hamilton product. The substrate chirality lives here: i*j*k = -1.
1652 ! Both factors are scaled by their own largest magnitude, the product is
1653 ! formed on components bounded by one, and the scale is restored only after
1654 ! the restored magnitude has been checked in log space. Representability
1655 ! is therefore decided on the actual components and not on the norm bound
1656 ! |ab| = |a||b|, which is sufficient for refusal but not necessary, since a
1657 ! component can be smaller than the norm. Unrepresentable is NaN, never a
1658 ! saturated or wrapped quantity.
1659 pure function qmul(a, b) result(c)
1660 real(dp), intent(in) :: a(4), b(4)
1661 real(dp) :: c(4), aa(4), bb(4), cc(4), sa, sb, lg
1662 if (.not. (finite_vec(a) .and. finite_vec(b))) then
1663 c = ieee_value(1.0_dp, ieee_quiet_nan); return
1664 end if
1665 sa = maxval(abs(a)); sb = maxval(abs(b))
1666 if (.not. (sa > 0.0_dp .and. sb > 0.0_dp)) then
1667 c = 0.0_dp; return
1668 end if
1669 aa = a/sa; bb = b/sb
1670 cc(1) = aa(1)*bb(1) - aa(2)*bb(2) - aa(3)*bb(3) - aa(4)*bb(4)
1671 cc(2) = aa(1)*bb(2) + aa(2)*bb(1) + aa(3)*bb(4) - aa(4)*bb(3)
1672 cc(3) = aa(1)*bb(3) - aa(2)*bb(4) + aa(3)*bb(1) + aa(4)*bb(2)
1673 cc(4) = aa(1)*bb(4) + aa(2)*bb(3) - aa(3)*bb(2) + aa(4)*bb(1)
1674 if (.not. (maxval(abs(cc)) > 0.0_dp)) then
1675 c = 0.0_dp; return
1676 end if
1677 lg = log(sa) + log(sb) + log(maxval(abs(cc)))
1678 if (lg >= log(huge(1.0_dp))) then
1679 c = ieee_value(1.0_dp, ieee_quiet_nan); return
1680 end if
1681 ! The scale is restored in binary, not through exp(log). The mantissas
1682 ! fraction(sa) and fraction(sb) lie in [1/2, 1), so their product times cc
1683 ! is bounded by four and cannot overflow, and scale() by the summed
1684 ! exponent is an exact power-of-two multiply. exp(log(sa)+log(sb)) loses
1685 ! about thirteen digits here, since an ulp of a log near 789 is 1e-13
1686 ! absolute and becomes 1e-13 relative on the result; the battery caught
1687 ! that and this is the repair.
1688 c = scale(fraction(sa)*fraction(sb)*cc, exponent(sa) + exponent(sb))
1689 end function qmul
1690
1691 ! The triple products overflow for entries near the ceiling even where the
1692 ! determinant is finite after cancellation, so the matrix is scaled first and
1693 ! the cube of the scale is restored once.
1694 ! Purity is kept, so the answer to "is this value a quantity" is a separate
1695 ! predicate rather than an out argument. Where the computed determinant
1696 ! magnitude exceeds the representable normal range det3 saturates and
1697 ! det3_representable returns
1698 ! .false.; a caller that reads the saturated value as a quantity has been
1699 ! told otherwise and did not ask.
1700 ! Equilibration, not a single global scale. One maximum over the whole
1701 ! matrix lets a small diagonal entry underflow to zero in the normalised
1702 ! copy, so diag(1e200, 1e200, 1e-200) returned zero where the determinant
1703 ! is 1e200 and diag(1e308, 1e308, 1e-20) was certified representable where
1704 ! it is not. Rows are scaled by their own maxima, then columns by theirs,
1705 ! and the determinant of the equilibrated core is reassembled in log space
1706 ! so representability is decided before any product is formed.
1707 pure subroutine det3_core(M, core, lg, sgn, ok)
1708 real(dp), intent(in) :: M(3,3)
1709 real(dp), intent(out) :: core, lg, sgn
1710 logical, intent(out) :: ok
1711 real(dp) :: N(3,3), r(3), c(3)
1712 integer :: i
1713 ok = finite_mat(M); core = 0.0_dp; lg = -huge(1.0_dp); sgn = 0.0_dp
1714 if (.not. ok) return
1715 N = M
1716 do i = 1, 3
1717 r(i) = maxval(abs(N(i,:)))
1718 if (r(i) > 0.0_dp) N(i,:) = N(i,:)/r(i)
1719 end do
1720 do i = 1, 3
1721 c(i) = maxval(abs(N(:,i)))

```

```

1721     if (c(i) > 0.0_dp) N(:,i) = N(:,i)/c(i)
1722 end do
1723 core = N(1,1)*N(2,2)*N(3,3) - N(2,3)*N(3,2)) &
1724       - N(1,2)*N(2,1)*N(3,3) - N(2,3)*N(3,1)) &
1725       + N(1,3)*N(2,1)*N(3,2) - N(2,2)*N(3,1))
1726 if (.not. (abs(core) > 0.0_dp)) return ! computed zero; lg stays -huge
1727 sgn = sign(1.0_dp, core)
1728 lg = log(abs(core))
1729 do i = 1, 3
1730   if (r(i) > 0.0_dp) lg = lg + log(r(i))
1731   if (c(i) > 0.0_dp) lg = lg + log(c(i))
1732 end do
1733 end subroutine det3_core
1734
1735 pure function det3(M) result(d)
1736   real(dp), intent(in) :: M(3,3)
1737   real(dp) :: d, core, lg, sgn
1738   logical :: ok
1739   call det3_core(M, core, lg, sgn, ok)
1740   if (.not. ok) then
1741     d = ieee_value(1.0_dp, ieee_quiet_nan); return
1742   end if
1743   if (.not. (abs(core) > 0.0_dp)) then
1744     d = 0.0_dp; return
1745   end if
1746   ! At the ceiling itself exp(lg) can round past huge and manufacture an
1747   ! overflow where a saturated finite value was promised, so the boundary
1748   ! saturates too.
1749   if (lg >= log(huge(1.0_dp))) then
1750     d = sign(huge(1.0_dp), sgn); return ! saturating; ask
1751     det3_representable
1752   end if
1753   if (lg < log(tiny(1.0_dp))) then
1754     d = sign(0.0_dp, sgn); return ! below the normal range; ask
1755   end if
1756   d = sgn*exp(lg)
1757 end function det3
1758
1759 ! True when det3 returns a non-saturated, non-underflowed normal quantity
1760 ! under this floating computation, or when the equilibrated core computes
1761 ! as zero. A computed zero is not certified exact singularity.
1762 pure function det3_representable(M) result(r)
1763   real(dp), intent(in) :: M(3,3)
1764   logical :: r
1765   real(dp) :: core, lg, sgn
1766   logical :: ok
1767   call det3_core(M, core, lg, sgn, ok)
1768   if (.not. ok) then
1769     r = .false.; return
1770   end if
1771   if (.not. (abs(core) > 0.0_dp)) then
1772     ! A COMPUTED zero is treated as a representable zero. This routine
1773     ! reads floating arithmetic and cannot tell exact singularity from a
1774     ! determinant that canceled or underflowed to zero on the way; that
1775     ! distinction needs interval or compensated arithmetic and is not
1776     ! claimed here.
1777     r = .true.; return
1778   end if
1779   r = (lg < log(huge(1.0_dp))) .and. (lg >= log(tiny(1.0_dp)))
1780 end function det3_representable
1781
1782 ! CONTRACT: A is symmetric. The routine reads only what a symmetric matrix
1783 ! carries, so an asymmetric input would be silently symmetrised by the
1784 ! sweep and the eigenvalues returned would belong to no matrix the caller
1785 ! supplied. The precondition is therefore enforced, not merely documented.
1786 subroutine jacob3(A, w)
1787   real(dp), intent(in) :: A(3,3)
1788   real(dp), intent(out) :: w(3)
1789   real(dp) :: M(3,3), Asc(3,3), theta, t, c, s, tau, g, h, asym, amax
1790   integer :: sweep, p, q, k
1791   if (.not. finite_mat(A)) error stop 'jacob3: input is not finite'
1792   ! The check is performed on a scaled copy, because differencing two
1793   ! entries near the representable ceiling overflows the test itself and a
1794   ! precondition guard that faults while guarding is worse than none.
1795   amax = maxval(abs(A))
1796   if (amax > 0.0_dp) then
1797     Asc = A/amax
1798   else
1799     Asc = A
1800   end if
1801   asym = max(abs(Asc(1,2)-Asc(2,1)), abs(Asc(1,3)-Asc(3,1)), abs(Asc(2,3)-Asc(3,2)))
1802   if (asym > 100.0_dp*epsilon(1.0_dp)) then
1803     error stop 'jacob3: input matrix is not symmetric within tolerance'
1804   end if
1805   ! The whole sweep runs on the scaled copy and the eigenvalues are unscaled
1806   ! once at the end. This is what makes every interior sum, difference and
1807   ! square safe at any admissible magnitude, rather than guarding each one.
1808   M = Asc
1809   do sweep = 1, 60
1810     g = abs(M(1,2)) + abs(M(1,3)) + abs(M(2,3))
1811     if (g < 1.0e-300_dp) exit
1812     do p = 1, 2
1813       do q = p+1, 3
1814         ! The negligibility test comes FIRST, because theta overflows in
1815         ! its own division where the off-diagonal is tiny against the
1816         ! diagonal spread. An off-diagonal below the rounding of the two
1817         ! diagonals it couples carries no rotation, so it is zeroed and
1818         ! skipped rather than divided by.
1819         if (abs(M(p,q)) <= epsilon(1.0_dp)*max(abs(M(p,p)), abs(M(q,q)))) then
1820           M(p,q) = 0.0_dp; M(q,p) = 0.0_dp
1821         else if (abs(M(p,q)) > tiny(1.0_dp)) then
1822           theta = (M(q,q) - M(p,p)) / (2.0_dp*M(p,q))
1823           ! theta*theta overflows where the off-diagonal is negligible
1824           ! against the diagonal spread. The asymptotic branch is the
1825           ! same rotation read where the square cannot be formed.
1826           if (abs(theta) > sqrt(huge(1.0_dp))) then
1827             t = 0.5_dp / theta
1828           else
1829             t = sign(1.0_dp, theta) / (abs(theta) + sqrt(theta*theta + 1.0_dp))
1830           end if
1831           c = 1.0_dp / sqrt(t*t + 1.0_dp); s = t*c; tau = s/(1.0_dp + c)
1832           h = t*M(p,q)
1833           M(p,p) = M(p,p) - h; M(q,q) = M(q,q) + h
1834           M(p,q) = 0.0_dp; M(q,p) = 0.0_dp
1835           do k = 1, 3
1836             if (k /= p .and. k /= q) then
1837               g = M(k,p); h = M(k,q)
1838               M(k,p) = g - s*(h + g*tau); M(p,k) = M(k,p)
1839               M(k,q) = h + s*(g - h*tau); M(q,k) = M(k,q)
1840             end if
1841           end do
1842         end do
1843       end do
1844     end do
1845   end do
1846   ! Unscaling can exceed the representable range for a finite input whose
1847   ! eigenvalue does. The infinity is then produced by ieee_value, which
1848   ! raises no exception, and never by the multiplication, which would trap
1849   ! before any consumer could refuse the result.
1850   ! The guard is nested and not conjoined: Fortran does not promise
1851   ! short-circuit evaluation, so huge/amax must never be formed unless
1852   ! amax exceeds one, or the guard itself overflows on a tiny matrix.
1853   do k = 1, 3
1854     if (amax > 1.0_dp) then
1855       if (abs(M(k,k)) > huge(1.0_dp)/amax) then
1856         w(k) = sign(ieee_value(1.0_dp, ieee_positive_inf), M(k,k))
1857       else
1858         w(k) = amax*M(k,k)
1859       end if
1860     else
1861       w(k) = amax*M(k,k) ! amax <= 1: the product only shrinks
1862     end if
1863   end do
1864 end subroutine jacob3
1865
1866 ! CONTRACT: A is symmetric positive semi-definite, a Gram. On a general
1867 ! symmetric matrix this returns a MAGNITUDE condition number and not the
1868 ! spectral one, because the extremes are taken over absolute eigenvalues.
1869 function cond_sym3(A) result(kap)
1870   real(dp), intent(in) :: A(3,3)
1871   real(dp) :: kap, w(3), lo, hi
1872   call jacob3(A, w)
1873   ! A finite matrix can carry an eigenvalue beyond the representable range,
1874   ! since three entries near the ceiling sum to about three times it. The
1875   ! sweep reports that honestly as an infinity and the consumer refuses it.
1876   if (.not. finite_vec(w)) error stop 'cond_sym3: eigenvalues are not representable'
1877   ! The contract names a Gram, so definiteness is enforced rather than
1878   ! assumed. An indefinite matrix would return a magnitude condition number
1879   ! that reads like a spectral one and would be believed.
1880   hi = maxval(abs(w))
1881   if (.not. (hi > 0.0_dp)) then
1882     kap = huge(1.0_dp); return ! the zero matrix has no conditioning
1883   end if
1884   ! Relative to the spectrum and carrying no absolute floor, for the same
1885   ! reason the rank tolerance carries none: a floor pinned at one would make
1886   ! definiteness a fact about units.
1887   if (minval(w) < -100.0_dp*epsilon(1.0_dp)*hi) then
1888     error stop 'cond_sym3: input is not positive semi-definite within tolerance'
1889   end if
1890   lo = minval(abs(w))
1891   ! The floor is tiny() and not zero: a positive denormal divisor overflows
1892   ! the quotient, so the guard is on representability and not on sign.
1893   if (lo <= tiny(1.0_dp)) then
1894     kap = huge(1.0_dp)
1895   else
1896     kap = hi/lo
1897   end if
1898 end function cond_sym3
1899
1900 ! THE SPINE, COMPUTED. Read the three rows as normals of planes through the
1901 ! origin. The intersection of the planes has dimension 3 - rank. Rank three
1902 ! returns dimension ZERO, one point, unique, no interval. Rank two returns
1903 ! dimension ONE, a line, and no point is determined at all.
1904 ! CONTRACT: G is a SYMMETRIC GRAM, G = Q Q^T, formed by the caller. It is not
1905 ! a matrix of raw rows. The dimension returned is 3 - rank(G), which is the
1906 ! dimension of the intersection of the three planes whose normals the rows of
1907 ! Q are; the routine reads the Gram because rank is what it needs and the
1908 ! Gram is where rank is cheap. Passing raw rows violates the contract: in
1909 ! exact arithmetic rank(Q Q^T) equals rank(Q), so the numerical dimension
1910 ! may coincide, but the routine is specified to read a symmetric PSD Gram
1911 ! and enforces that specification rather than trusting the coincidence.
1912 ! The rank tolerance is not chosen. It is the largest eigenvalue times the
1913 ! machine epsilon times the matrix order, the standard perturbation bound on
1914 ! a symmetric eigenvalue: below it an eigenvalue is not separable from
1915 ! rounding at this precision.
1916 subroutine intersection_dim(G, d, sep)
1917   real(dp), intent(in) :: G(3,3)
1918   integer, intent(out) :: d
1919   real(dp), intent(out) :: sep ! margin: smallest eigenvalue over tol
1920   integer :: i, rank
1921   real(dp) :: w(3), tol, lo, hi
1922   call jacob3(G, w)
1923   if (.not. finite_vec(w)) error stop 'intersection_dim: eigenvalues are not representable'
1924   ! The tolerance is RELATIVE to the spectrum and carries no absolute floor,
1925   ! so rank is invariant under uniform scaling of the Gram: a Gram at 1e-20
1926   ! has the same rank as the same Gram at 1. An absolute floor would make
1927   ! rank a fact about units, which is the chart-manufactured magnitude this
1928   ! architecture bars from entering a verdict as structure.
1929   hi = maxval(abs(w))
1930   if (.not. (hi > 0.0_dp)) then

```

```

1931     d = 3; sep = 0.0_dp; return ! the zero Gram carries no direction
1932 end if
1933 ! The contract names a Gram, so definiteness is enforced and the count is
1934 ! over POSITIVE eigenvalues. Counting magnitudes would hand an indefinite
1935 ! matrix a geometric dimension it does not have.
1936 if (minval(w) < -100.0_dp*epsilon(1.0_dp)*hi) then
1937     error stop 'intersection_dim: G is not positive semi-definite within tolerance'
1938 end if
1939 tol = 8.0_dp * real(size(w,dp) * epsilon(1.0_dp) * hi
1940 rank = 0; lo = huge(1.0_dp)
1941 do i = 1, 3
1942     if (w(i) > tol) then
1943         rank = rank + 1
1944         lo = min(lo, w(i))
1945     end if
1946 end do
1947 d = 3 - rank
1948 if (rank > 0) then
1949     sep = lo/tol
1950 else
1951     sep = 0.0_dp
1952 end if
1953 end subroutine intersection_dim
1954
1955 subroutine prepare_rows(M, Q, ok)
1956     real(dp), intent(in) :: M(:, :)
1957     real(dp), intent(out) :: Q(size(M,1), size(M,2))
1958     logical, intent(out) :: ok
1959     integer :: i, n
1960     real(dp) :: mu, sd, nrm, scale_
1961     n = size(M,2); ok = .true.; Q = M
1962     if (.not. finite_mat(M)) then
1963         ok = .false.; return ! a non-quantity is not a row
1964     end if
1965     if (n <= 1) then
1966         ok = .false.; return ! no sample variance is defined at n = 1
1967     end if
1968     do i = 1, size(M,1)
1969         scale_ = maxval(abs(M(i,:)))
1970         ! Scale out before summing. Squaring a row near the representable
1971         ! ceiling overflows the sum of squares long before the variance is
1972         ! meaningful, so the row is divided by its own largest magnitude first
1973         ! and the statistic is formed on the scaled copy. The verdict is
1974         ! unchanged because every downstream step is scale-invariant.
1975         if (scale_ > 0.0_dp) Q(i,:) = Q(i,:)/scale_
1976         mu = sum(Q(i,:))/real(n,dp); Q(i,:) = Q(i,:) - mu
1977         sd = sqrt(sum(Q(i,:)**2)/real(n-1,dp))
1978         ! The floor is relative to the row's own scale. A spread that is only
1979         ! rounding on a large row is not content, and normalising by it would
1980         ! manufacture an axis out of noise.
1981         if (.not. (sd > 0.0_dp) .or. sd <= real(n,dp)*epsilon(1.0_dp)) then
1982             ok = .false.; return
1983         end if
1984         Q(i,:) = Q(i,:)/sd
1985         ! A z-scored row has sum of squares exactly n-1 by construction, so
1986         ! this norm is sqrt(n-1) and cannot overflow for any representable n.
1987         ! snorm is used anyway: one norm primitive across the module is one
1988         ! place to get it right, and a raw sum of squares reads as a hazard
1989         ! to a hostile reviewer even where the bound forecloses it.
1990         nrm = snorm(Q(i,:))
1991         if (.not. (nrm > 0.0_dp)) then
1992             ok = .false.; return
1993         end if
1994         Q(i,:) = Q(i,:)/nrm
1995     end do
1996 end subroutine prepare_rows
1997
1998 ! The guard is mandatory and never optional, because a guard a caller may
1999 ! skip is not a guard. A residual below the rounding floor is not a direction
2000 ! and is refused rather than normalised.
2001 subroutine span_basis(Q, B, ok)
2002     real(dp), intent(in) :: Q(:, :)
2003     real(dp), intent(out) :: B(size(Q,1), size(Q,2))
2004     logical, intent(out) :: ok
2005     integer :: i, j
2006     real(dp) :: v(size(Q,2)), nrm, floor_, scale_
2007     if (.not. finite_mat(Q)) error stop 'span_basis: input is not finite'
2008     ok = .true.; B = 0.0_dp
2009     floor_ = real(size(Q,2,dp) * epsilon(1.0_dp)
2010     do i = 1, size(Q,1)
2011         ! Each row is scaled by its own largest magnitude BEFORE any projection.
2012         ! Scaling a row leaves its span untouched and bounds every entry by one,
2013         ! so the dot products inside the loop cannot overflow. The floor is
2014         ! then relative to the scaled row by construction.
2015         scale_ = maxval(abs(Q(i,:)))
2016         if (.not. (scale_ > 0.0_dp)) then
2017             ok = .false.; return
2018         end if
2019         v = Q(i,:)/scale_
2020         do j = 1, i-1
2021             v = v - dot_product(v, B(j,:))*B(j,:)
2022         end do
2023         nrm = snorm(v)
2024         if (.not. (nrm > floor_)) then
2025             ok = .false.; return
2026         end if
2027         B(i,:) = v/nrm
2028     end do
2029 end subroutine span_basis
2030
2031 ! Only the first three rows of Q and B are screened, because only the first
2032 ! three rows are read; the shape test requires at least three and rows
2033 ! beyond them are not consulted by lambda_of at all.
2034 ! The admission test is a pure predicate so it can be exercised in-process
2035 ! on hostile input without halting. It is the single source of truth for
2036 ! what lambda_of accepts; lambda_of calls it and halts on refusal. The
2037
2038 ! order inside is load-bearing: unit-ness is proven by the scaled norm
2039 ! BEFORE any dot product forms, so every dot product below runs on rows
2040 ! whose components are 0(1) under the admission tolerance and never
2041 ! 0(huge), and the dot products cannot overflow.
2042 pure subroutine lambda_of_admissible(Q, B, ok, reason)
2043     real(dp), intent(in) :: Q(:, :), B(:, :)
2044     logical, intent(out) :: ok
2045     character(len=*) , intent(out) :: reason
2046     real(dp) :: btol
2047     integer :: i, j
2048     ok = .false.; reason = ''
2049     if (size(Q,1) < 3 .or. size(B,1) < 3 .or. size(Q,2) /= size(B,2) .or. size(Q,2) < 3)
2050         then
2051             reason = 'needs Q and B with at least three rows, at least three columns, and
2052             equal column counts'
2053             return
2054         end if
2055     if (.not. (finite_mat(Q(1:3,:)) .and. finite_mat(B(1:3,:)))) then
2056         reason = 'input is not finite'; return ! on the rows read, and no others
2057     end if
2058     btol = 100.0_dp * real(size(B,2,dp) * epsilon(1.0_dp)
2059     do i = 1, 3
2060         if (abs(snrm(B(i,:)) - 1.0_dp) > btol) then
2061             reason = 'B rows are not unit'; return
2062         end if
2063         if (abs(snrm(Q(i,:)) - 1.0_dp) > btol) then
2064             reason = 'Q rows are not unit'; return
2065         end if
2066     end do
2067     do i = 1, 3
2068         do j = i+1, 3
2069             if (abs(dot_product(B(i,:), B(j,:))) > btol) then
2070                 reason = 'B rows are not orthogonal'; return
2071             end if
2072         end do
2073     end do
2074     ok = .true.
2075 end subroutine lambda_of_admissible
2076
2077 subroutine lambda_of(Q, B, lam_quat, lam_det, co)
2078     real(dp), intent(in) :: Q(:, :), B(:, :)
2079     real(dp), intent(out) :: lam_quat, lam_det, co(3,3)
2080     real(dp) :: q1(4), q2(4), q3(4), p(4)
2081     integer :: i, j
2082     character(len=96) :: reason
2083     logical :: adm
2084     call lambda_of_admissible(Q, B, adm, reason)
2085     if (.not. adm) error stop 'lambda_of: '//trim(reason)
2086     do i = 1, 3
2087         do j = 1, 3
2088             co(i,j) = dot_product(Q(i,:), B(j,:)) ! admitted unit rows; Cauchy bound
2089             keeps this 0(1), no overflow
2090         end do
2091     end do
2092     q1 = [0.0_dp, co(1,:)]; q2 = [0.0_dp, co(2,:)]; q3 = [0.0_dp, co(3,:)]
2093     p = qmul(qmul(q1, q2), q3)
2094     lam_quat = p(1); lam_det = -det3(co)
2095 end subroutine lambda_of
2096
2097 ! The kernel does not grade. It reports whether the axes are independent, and
2098 ! therefore whether a unique intersection exists at all.
2099 subroutine kernel_determine(M, verdict, lam, detR, kapR, idim, why, margin)
2100     real(dp), intent(in) :: M(:, :)
2101     integer, intent(out) :: verdict, idim
2102     real(dp), intent(out), optional :: margin
2103     real(dp), intent(out) :: lam, detR, kapR
2104     character(len=*) , intent(out) :: why
2105     real(dp) :: sep
2106     real(dp) :: Q(3,size(M,2)), B(3,size(M,2)), R(3,3), co(3,3)
2107     real(dp) :: lq, ld, um, epsf, kgate
2108     integer :: n
2109     logical :: ok, bok
2110     ! EVERY output is defined here, before any test can return. The shape and
2111     ! size tests follow, and no arithmetic touches n until n has been screened,
2112     ! because the conditioning gate divides by it.
2113     lam = 0.0_dp; detR = 0.0_dp; kapR = 0.0_dp; idim = -1; sep = 0.0_dp
2114     verdict = OPENV; why = 'uninitialised'
2115     if (present(margin)) margin = 0.0_dp
2116     n = size(M,2); um = epsilon(1.0_dp)
2117     if (size(M,1) /= 3) then
2118         why = 'axis count is not three; the residence is not the one this kernel reads'
2119         return
2120     end if
2121     if (n < 4) then
2122         why = 'dimensional shortfall'; return
2123     end if
2124     if (.not. finite_mat(M)) then
2125         why = 'a row carries a non-quantity; nothing to read'; return
2126     end if
2127     epsf = 100.0_dp*um*real(n,dp)
2128     kgate = min(1.0e6_dp, sqrt(27.0_dp/(100.0_dp*um*real(n,dp))))
2129     call prepare_rows(M, Q, ok)
2130     if (.not. ok) then
2131         verdict = OPENV; why = 'an axis carries no content; nothing to intersect';
2132         return
2133     end if
2134     R = matmul(Q, transpose(Q))
2135     detR = det3(R)
2136     ! A correlation Gram is positive semi-definite by construction, so a
2137     ! negative determinant is a rounding artifact and never a quantity. It is
2138     ! set to exactly zero rather than reported as read. Zero here is the
2139     ! algebraic floor of the quantity and is not the collapse floor epsf,
2140     ! which is a separate threshold tested further down.
2141     if (detR < 0.0_dp) detR = 0.0_dp
2142     call intersection_dim(R, idim, sep)
2143     if (present(margin)) margin = sep
2144

```

```

2139 ! The collapse test runs BEFORE any basis is constructed. Gram-Schmidt on a
2140 ! dependent triad divides by a residual nothing bounds below, so a basis
2141 ! built here would be rounding noise carried forward into the scalar. The
2142 ! order is the guard: a residence that determines no point never reaches
2143 ! the orthogonalisation at all.
2144 ! Existence is decided by the DIMENSION and never by the determinant,
2145 ! because the dimension is the spine's own quantity and the determinant is
2146 ! a magnitude read off it. Where the two floors disagree the verdict is
2147 ! neither broken nor determined: it is arithmetic at the edge, and it
2148 ! escalates rather than guessing which floor to believe.
2149 if (idim > 0) then
2150   verdict = BROKEN
2151   select case (idim)
2152     case (1);    why = 'rank two; the intersection is a line, not a point'
2153     case (2);    why = 'rank one; the intersection is a plane, not a point'
2154     case default; why = 'rank zero; no axis carries an independent direction'
2155   end select
2156   return
2157 end if
2158 if (detR <= epsf) then
2159   verdict = OPENV
2160   why = 'engineering-incomplete: rank floor and collapse floor disagree; re-run at
2161   higher precision'
2162   return
2163 end if
2164 call span_basis(0, B, bok)
2165 if (.not. bok) then
2166   ! Reaching here means the rank floor and the collapse floor both said
2167   ! three independent axes while Gram-Schmidt found a null residual. That
2168   ! is a third floor disagreeing with two, not a finding of dependence,
2169   ! and it escalates for the same reason the determinant disagreement
2170   ! does. Deciding BROKEN here would let the weakest instrument overrule
2171   ! the two that already agreed.
2172   verdict = OPENV
2173   why = 'engineering-incomplete: orthogonalisation residual contradicts the rank
2174   floor'
2175   return
2176 end if
2177 call lambda_of(0, B, lq, ld, co)
2178 ! The two forms are computed by disjoint routes, the Hamilton product and
2179 ! the signed frame determinant, so their agreement is a live cross-check
2180 ! and not decoration. Disagreement is arithmetic failure and escalates.
2181 if (abs(lq - ld) > 100.0_dp*epsilon(1.0_dp)*max(abs(ld), 1.0_dp)) then
2182   verdict = OPENV
2183   why = 'engineering-incomplete: quaternion and determinant forms of lambda
2184   disagree'
2185   return
2186 end if
2187 lam = lq
2188 kapR = cond_sym3(R)
2189 if (kapR >= kgate) then
2190   ! The intersection exists and is unique by the geometry. The instrument
2191   ! declines to report it, because at this conditioning the computed point
2192   ! is not separable from rounding. Instrument-silence about reach, never
2193   ! a claim that the intersection is absent or partial.
2194   verdict = OPENV
2195   why = 'engineering-incomplete: determination present, instrument declines'
2196   return
2197 end if
2198 verdict = DETERMINED
2199 why = 'three independent axes; the intersection exists and is unique'
2200 if (present(margin)) margin = sep
2201 end subroutine kernel_determine
2202
2203 ! =====
2204 ! SEAL L, THE TONGUE. Loads FIRST. Carries the direction the Number cannot,
2205 ! and closes the terms the row cascade later reads.
2206 ! =====
2207
2208 subroutine seal_L(slots, verdict, why)
2209   integer, intent(in) :: slots(:)
2210   integer, intent(out) :: verdict
2211   character(len=*) , intent(out) :: why
2212   integer :: i, j, a, b, nfull, ns
2213   ns = size(slots,1) ! the shelf is read, never remembered
2214   ! Zero is the empty cell and literals are positive. A negative id is not
2215   ! vocabulary and is refused before any slot is counted.
2216   if (any(slots < 0)) then
2217     verdict = BROKEN; why = 'negative literal id; not vocabulary'; return
2218   end if
2219   nfull = 0
2220   do i = 1, ns
2221     if (any(slots(i,:) /= 0)) nfull = nfull + 1
2222   end do
2223   if (nfull /= 3) then
2224     verdict = BROKEN
2225     write(why, '(A,I0,A)') 'deletion test returns ', nfull, ' slots, not three'
2226     return
2227   end if
2228   ! A slot is a vocabulary set. A repeated literal inside one slot is a
2229   ! malformed set, and admitting it would let hostile input reach a sealed
2230   ! verdict through a slot that was never well formed.
2231   do i = 1, ns
2232     do a = 1, size(slots,2)
2233       do b = a+1, size(slots,2)
2234         if (slots(i,a) /= 0 .and. slots(i,a) == slots(i,b)) then
2235           verdict = BROKEN
2236           write(why, '(A,I0)') 'repeated literal within slot ', i
2237           return
2238         end if
2239       end do
2240     end do
2241   end do
2242   do i = 1, ns
2243     do j = i+1, ns
2244       do a = 1, size(slots,2)

```

```

2339     return
2340 end if
2341 compartment = CLOSURE_ROWED
2342 why = 'rows are the closure; the two determinations are one; RA-conditional at the
      act'
2343 end subroutine row_cascade
2344
2345 pure function compartment_grade(c, cap) result(s)
2346 integer, intent(in) :: c
2347 character(len=*) , intent(in) :: cap
2348 character(len=64) :: s
2349 select case (c)
2350 case (CLOSURE_ROWED); s = 'row-indefeasible, defeasible against fault, cap '//trim(
      cap)
2351 case (UNPOPULATED); s = 'no determination to defeat; an axis is owed'
2352 case (WORLD_ROWED); s = 'revisable entirely in the rows; direction bought with the
      arrow'
2353 case default; s = 'no compartment'
2354 end select
2355 end function compartment_grade
2356
2357 !=====
2358 ! THE GROUND AND THE ROUTER BIT.
2359 !=====
2360
2361 ! The router bit is measured at machine zero and never by exact equality. An
2362 ! eigenvalue arriving from a solver is 1 - 0(eps), and an equality test would
2363 ! silently return the wrong Ground dimension and route the candidate to the
2364 ! wrong halt protocol. The floor is stated, not assumed.
2365 pure function ground_dim(diagv) result(d)
2366 real(dp), intent(in) :: diagv(:)
2367 integer :: d, i
2368 real(dp) :: floor_
2369 d = -1 ! the router bit is unmeasured on a non-quantity
2370 if (.not. finite_vec(diagv)) return
2371 floor_ = real(max(size(diagv),1),dp) * epsilon(1.0_dp)
2372 d = 0
2373 do i = 1, size(diagv)
2374 if (abs(diagv(i) - 1.0_dp) <= floor_) d = d + 1
2375 end do
2376 end function ground_dim
2377
2378 pure function route_halt(gdim) result(branch)
2379 integer, intent(in) :: gdim
2380 character(len=52) :: branch
2381 if (gdim == 1) then
2382 branch = 'B.14.Xi eight gates, blindness in the reader'
2383 else if (gdim == 0) then
2384 branch = 'B.14.0 five gates, walls in the terrain'
2385 else
2386 branch = 'terrain unmeasured; nothing emitted'
2387 end if
2388 end function route_halt
2389
2390 end module trisduction_rows
2391
2392 !=====
2393 ! EXECUTED. Nothing below is narrated; every figure is computed in this run.
2394 !=====
2395
2396 subroutine witness_F1_thesis_rows()
2397 use trisduction_rows
2398 implicit none
2399 integer, parameter :: N = 24
2400 real(dp), parameter :: PI = 3.141592653589793238462643_dp
2401 real(dp) :: t(N), M(3,N), Mr(3,N), Two(3,N), Ret(3,N)
2402 real(dp) :: Q(3,N), B(3,N), co(3,3), R(3,3), Rr(3,3)
2403 real(dp) :: lam, detR, kapR, lam2, d2, lq, ld, marg
2404 real(dp) :: i4(4), j4(4), k4(4), p4(4), sig(4), dia(4)
2405 integer :: v, vl, idim, comp, i
2406 integer :: fails, checks
2407 logical :: ok, dok
2408 character(len=110) :: why, cap
2409 integer :: slots_ok(3,3)
2410
2411 write(*, '(A)') repeat('=' , 78)
2412 write(*, '(A)') ' TRISDUCTION . RAM -- THE CORE THESIS, EXECUTED '
2413 write(*, '(A)') repeat('=' , 78)
2414
2415 do i = 1, N
2416 t(i) = 2.0_dp*PI*real(1-1,dp)/real(N,dp)
2417 end do
2418
2419 !----- THE SPINE, COMPUTED
2420 ! Three independent axes: the intersection has dimension zero. One point.
2421 ! Two axes: dimension one. A line. No point is determined at all.
2422 M(1,:) = sin(t)
2423 M(2,:) = cos(2.0_dp*t) + 0.35_dp*sin(t)
2424 M(3,:) = sin(3.0_dp*t) + 0.20_dp*cos(2.0_dp*t) + 0.12_dp*sin(t)
2425 Two = M; Two(3,:) = 0.6_dp*M(1,:) + 0.4_dp*M(2,:) ! third axis not independent
2426
2427 write(*, '(/,A)') ' THE SPINE: WHAT A LOCK DETERMINES '
2428 call kernel_determine(M, v, lam, detR, kapR, idim, why, marg)
2429 write(*, '(A,A)') ' three independent axes : ', trim(token(v))
2430 write(*, '(A,A)') ' why = ', trim(why)
2431 write(*, '(A,I0,A)') ' intersection dimension = ', idim, ' -> exactly one point
      , no interval '
2432 write(*, '(A,F16.12)') ' det(R) = ', detR
2433 write(*, '(A,ES12.3,A,ES10.3,A,L1)') ' |lambda*2 - det(R)| = ', abs(lam*lam -
      detR), &
      ' bound ', 4.0_dp*kapR*epsilon(1.0_dp), ' within: ', &
      abs(lam*lam - detR) <= 4.0_dp*kapR*epsilon(1.0_dp)
2434 write(*, '(A,ES12.3,A)') ' rank margin = ', marg, ' smallest kept
      eigenvalue over tol '
2435 call kernel_determine(Two, v, lam2, d2, kapR, idim, why, marg)
2436 write(*, '(A,A)') ' third axis dependent : ', trim(token(v))

```

```

2439 write(*, '(A,A)') ' why = ', trim(why)
2440 write(*, '(A,I0,A)') ' intersection dimension = ', idim, ' -> a line; two
      planes meet in a line '
2441 write(*, '(A,ES12.3,A)') ' rank margin = ', marg, ' smallest RETAINED
      eigenvalue over tol; the third fell below '
2442 write(*, '(A)') ' A lock is not graded. It either determines a point or it
      does not exist. '
2443
2444 !----- ORIENTATION, NOT STRENGTH
2445 ! The Number cannot read the sign. That is a fact about DIRECTION and never
2446 ! about whether the determination is complete.
2447 ! The exhibit refuses to print rather than printing from an unusable frame.
2448 call prepare_rows(M, 0, ok)
2449 if (.not. ok) error stop 'orientation exhibit: rows carry no variance'
2450 call span_basis(Q, B, ok)
2451 if (.not. ok) error stop 'orientation exhibit: no independent frame'
2452 call lambda_of(Q, B, lq, ld, co); R = matmul(Q, transpose(Q))
2453 Mr = -M
2454 ! B is deliberately NOT rebuilt: the reflected rows are read against the
2455 ! frame fixed once, which is what makes the comparison a comparison.
2456 call prepare_rows(Mr, 0, ok)
2457 if (.not. ok) error stop 'orientation exhibit: reflected rows carry no variance'
2458 call lambda_of(Q, B, lam2, d2, co)
2459 Rr = matmul(Q, transpose(Q))
2460 write(*, '(/,A)') ' WHAT THE NUMBER CANNOT READ: DIRECTION, NOT COMPLETENESS '
2461 write(*, '(A,F16.12,A,F16.12)') ' lambda(P) = ', lq, ' lambda(not P) = ', lam2
2462 write(*, '(A,ES12.3)') ' max|G(P) - G(not P)| = ', maxval(abs(R - Rr))
2463 write(*, '(A,ES12.3)') ' |det(R)(P) - det(R)(not P)| = ', abs(det3(R) - det3(Rr))
2464 write(*, '(A)') ' The intersection is equally unique either way. The Tongue
      carries the sign. '
2465
2466 i4 = [0.0_dp, 1.0_dp, 0.0_dp, 0.0_dp]; j4 = [0.0_dp, 0.0_dp, 1.0_dp, 0.0_dp]
2467 k4 = [0.0_dp, 0.0_dp, 0.0_dp, 1.0_dp]; p4 = qmul(qmul(i4,j4),k4)
2468 write(*, '(A,F14.10)') ' substrate chirality Re(i*j*k) = ', p4(1)
2469
2470 Ret(1,:) = sin(t); Ret(2,:) = cos(t); Ret(3,:) = sin(2.0_dp*t)
2471 call kernel_determine(Ret, v, lam, detR, kapR, idim, why)
2472 write(*, '(A,A)') ' the Return : ', trim(token(v))
2473 write(*, '(A,A)') ' why = ', trim(why)
2474 if (v == DETERMINED) then
2475 write(*, '(A,F16.12,A,F16.12)') ' det(R) = ', detR, ' |lambda| = ', abs(lam)
2476 end if
2477
2478 !----- THE GROUND
2479 sig = [1.0_dp, -1.0_dp, -1.0_dp, -1.0_dp]; dia = [-1.0_dp, -1.0_dp, -1.0_dp, -1.0_dp]
2480 write(*, '(/,A)') ' THE GROUND AND THE ROUTER BIT '
2481 write(*, '(A,I0,A,A)') ' sigma Ground dim = ', ground_dim(sig), ' -> ', trim(
      route_halt(ground_dim(sig)))
2482 write(*, '(A,I0,A,A)') ' -I Ground dim = ', ground_dim(dia), ' -> ', trim(
      route_halt(ground_dim(dia)))
2483
2484 !----- SEAL L AND THE DRILLS
2485 slots_ok = reshape([101,102,0, 201,202,0, 301,302,0], [3,3], order=[2,1])
2486 call seal_L(slots_ok, vl, why)
2487 write(*, '(/,A)') ' SEAL L AND THE SIX DRILLS '
2488 write(*, '(/,A,A,A)') ' Seal L : ', trim(token(vL)), ' :: ', trim(why)
2489 call drill_screen(.true.,.true.,.true.,.true.,.true.,.true., dok, cap, why)
2490 write(*, '(A,L1,A,A,A,A)') ' unshaped terms: ', dok, ' cap=', trim(cap), ' :: ',
      trim(why)
2491 call drill_screen(.true.,.true.,.true.,.true.,.true.,.true., dok, cap, why)
2492 write(*, '(A,L1,A,A,A,A)') ' shaped terms: ', dok, ' cap=', trim(cap), ' :: ',
      trim(why)
2493 call drill_screen(.true.,.true.,.false.,.true.,.true.,.true., dok, cap, why)
2494 write(*, '(A,L1,A,A,A,A)') ' unfiled flag : ', dok, ' cap=', trim(cap), ' :: ',
      trim(why)
2495
2496 !----- THE ROW CASCADE
2497 write(*, '(/,A)') ' THE ROW CASCADE '
2498 call row_cascade(.true.,.true.,.true.,.false.,.true.,.false.,.true.,.true., comp, why)
2499 write(*, '(A,A)') ' a closed residence has no outside -> ', trim(
      compartment_name(comp))
2500 write(*, '(A,A)') ' ', trim(why)
2501 write(*, '(A,A)') ' ', trim(compartment_grade(comp,'theorem'))
2502
2503 call row_cascade(.true.,.true.,.true.,.false.,.false.,.false.,.true.,.true., comp, why)
2504 write(*, '(A,A)') ' every zero lies on the critical line -> ', trim(
      compartment_name(comp))
2505 write(*, '(A,A)') ' ', trim(why)
2506
2507 call row_cascade(.true.,.true.,.true.,.false.,.true.,.true.,.true.,.true., comp, why)
2508 write(*, '(A,A)') ' confined existents carry positive E_k -> ', trim(
      compartment_name(comp))
2509 write(*, '(A,A)') ' ', trim(why)
2510 write(*, '(A,A)') ' ', trim(compartment_grade(comp,'theorem'))
2511
2512 call row_cascade(.true.,.true.,.true.,.true.,.true.,.false.,.true.,.true., comp, why)
2513 write(*, '(A,A)') ' any dated forward projection -> ', trim(
      compartment_name(comp))
2514 write(*, '(A,A)') ' ', trim(why)
2515
2516 call row_cascade(.true.,.true.,.true.,.false.,.true.,.false.,.true.,.false., comp, why)
2517 write(*, '(A,A)') ' this man is a bachelor -> ', trim(
      compartment_name(comp))
2518 write(*, '(A,A)') ' ', trim(why)
2519
2520 call row_cascade(.false.,.true.,.true.,.false.,.true.,.false.,.true.,.true., comp, why)
2521 write(*, '(A,A)') ' a contentless tautology -> ', trim(
      compartment_name(comp))
2522 write(*, '(A,A)') ' ', trim(why)
2523
2524 write(*, '(/,A)') repeat('=' , 78)
2525 write(*, '(A)') ' A lock is always a determination and is total about its object. '

```

```

2526 write(*,'(A)') ' Its object is its intersection. The compartment names what the rows
2527 were.'
2528 write(*,'(A)') ' Compartment III pays revisability for direction. Compartment I has
2529 neither.'
2530 write(*,'(A)') ' Neither is the better lock. They are locks on different rows. Delta-M
2531 = 0.'
2532 write(*,'(A)') repeat('=',78)
2533
2534 !----- THE BATTERY, PROOF OF LOAD
2535 call battery()
2536
2537 contains
2538
2539 ! Eight fuzz classes: seven over the numerical kernel, one over every
2540 ! symbolic and router public interface, then the oracle. Every check is a
2541 ! computed logical. The program stops here on any failure, so the final
2542 ! line of output is itself the receipt.
2543
2544 subroutine battery()
2545   use, intrinsic :: ieee_arithmetic
2546   integer, parameter :: NB = 8
2547   real(dp) :: H, T, nanv, pinf, ninf
2548   real(dp) :: mags(11)
2549   real(dp) :: M(3,NB), Q(3,NB), B(3,NB), G(3,3), A(3,3), w(3), sep
2550   real(dp) :: lam, detR, kapR, marg, lq, ld, co(3,3), q4(4), r4(4)
2551   real(dp) :: d1, ds(5), etas(6), scales(5), rho(3), tt(NB)
2552   integer :: i, j, k, v, idim
2553   character(len=160) :: why
2554   logical :: ok
2555
2556   fails = 0; checks = 0
2557   H = sqrt(huge(1.0_dp)); T = sqrt(tiny(1.0_dp))
2558   nanv = ieee_value(1.0_dp, ieee_quiet_nan)
2559   pinf = ieee_value(1.0_dp, ieee_positive_inf)
2560   ninf = ieee_value(1.0_dp, ieee_negative_inf)
2561   mags = [0.0_dp, tiny(1.0_dp), T, epsilon(1.0_dp), 1.0_dp, 1.0_dp/epsilon(1.0_dp), H,
2562           &
2563           huge(1.0_dp)/4.0_dp, huge(1.0_dp), -1.0_dp, -H]
2564   do i = 1, NB
2565     tt(i) = 6.283185307179586_dp*real(i-1,dp)/real(NB,dp)
2566   end do
2567
2568   !----- CLASS 1
2569   ! Non-finite hostile inputs into every pure entry: NaN or .false. out.
2570   call assert(.not. ieee_is_finite(snorm([1.0_dp, nanv])), 'snorm NaN -> NaN')
2571   call assert(.not. ieee_is_finite(snorm([1.0_dp, pinf])), 'snorm +Inf -> NaN')
2572   G = 1.0_dp; G(2,2) = ninf
2573   call assert(.not. ieee_is_finite(det3(G)), 'det3 -Inf -> NaN')
2574   call assert(.not. det3_representable(G), 'det3_representable
2575   -Inf -> F')
2576   G(2,2) = nanv
2577   call assert(.not. det3_representable(G), 'det3_representable
2578   NaN -> F')
2579   q4 = [nanv,0.0_dp,0.0_dp,0.0_dp]; r4 = [1.0_dp,0.0_dp,0.0_dp,0.0_dp]
2580   call assert(.not. all(ieee_is_finite(qmul(q4,r4))), 'qmul NaN -> NaN')
2581   call assert(ground_dim([1.0_dp,pinf,-1.0_dp,-1.0_dp]) == -1, 'ground_dim Inf ->
2582   unmeasured')
2583   ! kernel: a non-finite row must route OPEN with a mechanism, never SEALED/BROKEN
2584   M(1,:) = sin(tt); M(2,:) = cos(2.0_dp*tt); M(3,:) = sin(3.0_dp*tt); M(2,4) = nanv
2585   call kernel_determine(M, v, lam, detR, kapR, idim, why, marg)
2586   call assert(v == OPENV .and. index(why,'non-quantity') > 0, 'kernel NaN row ->
2587   OPEN, mechanism named')
2588   M(2,4) = pinf
2589   call kernel_determine(M, v, lam, detR, kapR, idim, why, marg)
2590   call assert(v == OPENV, 'kernel Inf row ->
2591   OPEN')
2592
2593   !----- CLASS 2
2594   ! Extreme finite magnitudes: no overflow, no false PSD failure, no NaN after unscale
2595
2596   do i = 1, 11
2597     G = 0.0_dp; G(1,1) = abs(mags(i)); G(2,2) = abs(mags(i)); G(3,3) = abs(mags(i))
2598     if (abs(mags(i)) > 0.0_dp .and. abs(mags(i)) < huge(1.0_dp)) then
2599       call intersection_dim(G, idim, sep)
2600       call assert(idim == 0, 'scaled identity rank 3 at magnitude index '//itoa(i))
2601     end if
2602   end do
2603   do i = 1, NB
2604     Q(1,i) = T*sin(tt(i)); Q(2,i) = H*cos(2.0_dp*tt(i)); Q(3,i) = H*sin(3.0_dp*tt(i))
2605   end do
2606   call span_basis(Q, B, ok)
2607   call assert(ok .and. finite_mat(B), 'span_basis at sqrt
2608   (huge): finite frame')
2609   do i = 1, NB
2610     Q(1,i) = T*sin(tt(i)); Q(2,i) = T*cos(2.0_dp*tt(i)); Q(3,i) = T*sin(3.0_dp*tt(i))
2611   end do
2612   call span_basis(Q, B, ok)
2613   call assert(ok .and. finite_mat(B), 'span_basis at sqrt
2614   (tiny): finite frame')
2615   A = 0.0_dp; A(1,1) = H; A(2,2) = T; A(3,3) = 1.0_dp
2616   call jacobi3(A, w)
2617   call assert(finite_vec(w), 'jacobi3 diag(H,T
2618   ,1) finite')
2619   A = H; call jacobi3(A, w)
2620   call assert(finite_vec(w), 'jacobi3 all-H
2621   finite (3H fits)')
2622   A = huge(1.0_dp)/2.0_dp; call jacobi3(A, w)
2623   call assert(.not. finite_vec(w), 'jacobi3 all-huge
2624   /2: unrepresentable reported as Inf')
2625   ! det3 equilibration
2626   G = 0.0_dp; G(1,1)=1.0e200_dp; G(2,2)=1.0e200_dp; G(3,3)=1.0e-200_dp
2627   call assert(abs(det3(G)/1.0e200_dp - 1.0_dp) < 1.0e-12_dp, 'det3 diag(1e200,1
2628   e200,1e-200) = 1e200')
2629   call assert(det3_representable(G), ' and
2630   representable')
2631   G = 0.0_dp; G(1,1)=1.0e308_dp; G(2,2)=1.0e308_dp; G(3,3)=1.0e-20_dp

```

```

2711     NaN, never wrapped')
2712 end block
2713 !----- CLASS 8
2714 ! The symbolic and router procedures, fuzzed as the numeric kernel is: every
2715 ! token typed, every compartment named, every halt branch routed, every
2716 ! Seal L refusal reached, every drill failure routed, every cascade gate
2717 ! taken. A public interface with no computed check is not covered.
2718 block
2719   integer :: sl(3,3), sl4(4,3), cmp, tk
2720   character(len=140) :: rsn, cap
2721   logical :: dok
2722   ! tokens: three states, two refinements, one kernel token, one dot, one untyped
2723   call assert(trim(token_class(SEALED)) == 'verdict state' .and. trim(token_class(
2724     BROKEN)) == 'verdict state' &
2725     .and. trim(token_class(OPENV)) == 'verdict state',
2726     token_class: the three states')
2727   call assert(trim(token_class(XI0)) == 'refinement inside openness' .and. &
2728     trim(token_class(OH0)) == 'refinement inside openness',
2729     token_class: both refinements inside openness')
2730   call assert(trim(token_class(DETERMINED)) == 'kernel geometric token',
2731     token_class: the kernel token')
2732   call assert(trim(token_class(DOTMARK)) == 'outside the economy',
2733     token_class: the dot outside the economy')
2734   call assert(trim(token_class(99)) == 'untyped' .and. index(token(99), 'untyped') >
2735     0, 'token_class/tokens: untyped')
2736 do tk = 1, 7
2737   call assert(len_trim(token(tk)) > 0,
2738     every token prints')
2739 end do
2740 ! compartments and the router
2741 call assert(index(compartment_name(CLOSURE_ROWED), 'CLOSURE_ROWED') > 0 .and. &
2742   index(compartment_name(UNPOPULATED), 'UNPOPULATED') > 0 .and. &
2743   index(compartment_name(WORLD_ROWED), 'WORLD_ROWED') > 0 .and. &
2744   index(compartment_name(NOT_COMPARTMENTED), 'not compartmented') > 0, '
2745   compartment_name: all four')
2746 call assert(index(route_halt(1), 'Xi') > 0 .and. index(route_halt(0), 'B.14.0') > 0
2747 .and. &
2748   index(route_halt(-1), 'unmeasured') > 0,
2749   route_halt: dim 1, dim 0, unmeasured')
2750 call assert(index(route_halt(2), 'unmeasured') > 0 .and. index(route_halt(7), '
2751 unmeasured') > 0, &
2752   'route_halt: unexpected positive dims are unmeasured')
2753 call assert(index(compartment_grade(CLOSURE_ROWED, 'theorem'), 'theorem') > 0 .and.
2754 &
2755   index(compartment_grade(UNPOPULATED, 'x'), 'no determination') > 0 .and.
2756   index(compartment_grade(WORLD_ROWED, 'x'), 'rows') > 0,
2757   compartment_grade: all three')
2758 call assert(trim(compartment_grade(99, 'x')) == 'no compartment',
2759   compartment_grade: default')
2760 ! Seal L: every refusal and the one admission
2761 sl = reshape([101,102,0, 201,202,0, 301,302,0],[3,3],order=[2,1])
2762 call seal_L(sl, tk, rsn); call assert(tk == SEALED,
2763   'seal_L: clean three slots seal')
2764 sl = reshape([101,102,0, 201,101,0, 301,302,0],[3,3],order=[2,1])
2765 call seal_L(sl, tk, rsn); call assert(tk == BROKEN .and. index(rsn, 'LIT') > 0, '
2766 seal_L: cross-slot collision')
2767 sl = reshape([101,101,0, 201,202,0, 301,302,0],[3,3],order=[2,1])
2768 call seal_L(sl, tk, rsn); call assert(tk == BROKEN .and. index(rsn, 'repeated') >
2769 0, 'seal_L: within-slot repeat')
2770 sl = reshape([101,-1,0, 201,202,0, 301,302,0],[3,3],order=[2,1])
2771 call seal_L(sl, tk, rsn); call assert(tk == BROKEN .and. index(rsn, 'negative') >
2772 0, 'seal_L: negative id')
2773 sl = reshape([101,102,0, 0,0,0, 301,302,0],[3,3],order=[2,1])
2774 call seal_L(sl, tk, rsn); call assert(tk == BROKEN .and. index(rsn, '2 slots') >
2775 0, 'seal_L: two slots, not three')
2776 sl4 = reshape([101,0,0, 201,0,0, 301,0,0, 401,0,0],[4,3],order=[2,1])
2777 call seal_L(sl4, tk, rsn); call assert(tk == BROKEN .and. index(rsn, '4 slots') >
2778 0, 'seal_L: four slots counted, not three')
2779 ! drills: each failure routes, both caps
2780 call drill_screen(.false., .true., .true., .false., .true., .true., .true., dok, cap,
2781   rsn)
2782 call assert(.not. dok .and. index(rsn, 'D1/D2') > 0,
2783   'drill: D1
2784   fails')
2785 call drill_screen(.true., .false., .true., .false., .true., .true., .true., dok, cap,
2786   rsn)
2787 call assert(.not. dok .and. index(rsn, 'D1/D2') > 0,
2788   'drill: D2
2789   fails')
2790 call drill_screen(.true., .true., .false., .false., .true., .true., .true., dok, cap,
2791   rsn)
2792 call assert(.not. dok .and. index(rsn, 'D3') > 0,
2793   'drill:
2794   unfilled provenance')
2795 call drill_screen(.true., .true., .true., .false., .false., .true., .true., dok, cap,
2796   rsn)
2797 call assert(.not. dok .and. index(rsn, 'D4') > 0,
2798   'drill:
2799   magnitude on the Tongue')
2800 call drill_screen(.true., .true., .true., .false., .true., .false., .true., dok, cap,
2801   rsn)
2802 call assert(.not. dok .and. index(rsn, 'D5') > 0,
2803   'drill:
2804   register undeclared')
2805 call drill_screen(.true., .true., .true., .false., .true., .true., .false., dok, cap,
2806   rsn)
2807 call assert(.not. dok .and. index(rsn, 'D6') > 0,
2808   'drill:
2809   blindness reproduced')
2810 call drill_screen(.true., .true., .true., .false., .true., .true., .true., dok, cap, rsn
2811 )
2812 call assert(dok .and. trim(cap) == 'theorem',
2813   'drill:
2814   unshaped caps at theorem')
2815 call drill_screen(.true., .true., .true., .true., .true., .true., .true., dok, cap, rsn)
2816 call assert(dok .and. trim(cap) == 'structural',
2817   'drill:
2818   shaped caps at structural')
2819 ! cascade: every gate taken in order
2820 call row_cascade(.false., .true., .true., .false., .true., .false., .true., .true., cmp,
2821   rsn)
2822 call assert(cmp == NOT_COMPARTMENTED .and. index(rsn, 'F-0') > 0,
2823   'cascade:

```

```

2860 ! fuzz campaign: fifty-six findings, six refused on printed receipts,
2861 ! fifty earned and repaired. Hardening families: exponent-tracked
2862 ! Leibniz determinants (det3, det3_row2), fraction-exponent Hamilton
2863 ! products (qmul), public-guard refusal precedence, census lock,
2864 ! audit-mode VOID banner, claim-register alignment. Four FORGE
2865 ! self-audit cycles (ratoe..ratoe4) sealed at the SELF floor. The
2866 ! full per-version ledger lives in git, 1000sapients/Trisduction,
2867 ! protocols/Executable Thesis/, and codex card 0712; it is not
2868 ! restated here.
2869 !
2870 ! v2.0.0 (2026-09-08): consolidation round. Fixes on external static
2871 ! review: stale printed receipt pointer rebound to the version-matched
2872 ! file; two dead a(3,3) locals removed from the determinant
2873 ! primitives; one non-significant-digit literal given its kind. The
2874 ! 113 remaining -Wcompare-reals warnings are the exact-equality
2875 ! verdict design (bit-identity checks, tie checks, NaN self-tests)
2876 ! and are intentional; the sealed flag set is the contract.
2877 !
2878 ! BUILD (sealed): gfortran -std=f2018 -O2 -fno-fast-math
2879 ! -ffp-contract=off -Wall -Wextra -Wconversion-extra
2880 ! Modes: default sealed halt-on-fail; 'audit' accumulates and voids;
2881 ! 'witnessed' asserts the PART H live face through the aperture.
2882 ! Hostile builds (-Ofast; fused contraction) must exit nonzero.
2883 ! =====
2884
2885 module ra_kinds
2886 use, intrinsic :: ieee_arithmetic, only: ieee_value, ieee_quiet_nan, &
2887       ieee_is_finite, ieee_is_nan
2888 use, intrinsic :: iso_fortran_env, only: int64
2889 implicit none
2890 integer, parameter :: wp = selected_real_kind(15, 307)
2891 integer, parameter :: ik = int64 ! pinned: exactly 64 bits, checked at boot
2892 real(wp), parameter :: HBAR = 1.054571817e-34_wp ! J s (CODATA exact-defined path)
2893 real(wp), parameter :: KB = 1.380649e-23_wp ! J/K (exact, SI 2019)
2894 real(wp), parameter :: ME = 9.1093837015e-31_wp ! kg
2895 real(wp), parameter :: U_M = epsilon(1.0_wp) ! unit roundoff, double
2896 integer(ik), parameter :: IK_SAFE = 536870912_ik ! 2^29
2897 contains
2898 pure function pi_() result(p)
2899   real(wp) :: p
2900   p = acos(-1.0_wp)
2901 end function pi_
2902 pure function ln2_() result(l)
2903   real(wp) :: l
2904   l = log(2.0_wp)
2905 end function ln2_
2906 pure function qnan() result(z)
2907   real(wp) :: z
2908   z = ieee_value(1.0_wp, ieee_quiet_nan)
2909 end function qnan
2910 elemental logical function is_finite(x)
2911   real(wp), intent(in) :: x
2912   is_finite = ieee_is_finite(x)
2913 end function is_finite
2914 elemental logical function is_nan(x)
2915   real(wp), intent(in) :: x
2916   is_nan = ieee_is_nan(x)
2917 end function is_nan
2918 end module ra_kinds
2919
2920 ! -----
2921 module ra_prims
2922 use ra_kinds
2923 implicit none
2924 contains
2925 pure function vnorm(v) result(s)
2926   real(wp), intent(in) :: v(:)
2927   real(wp) :: s, m
2928   integer :: i
2929   m = 0.0_wp
2930   do i = 1, size(v)
2931     if (.not. is_finite(v(i))) then
2932       s = qnan()
2933       return
2934     end if
2935     m = max(m, abs(v(i)))
2936   end do
2937   if (m == 0.0_wp) then
2938     s = 0.0_wp
2939   else
2940     s = m * sqrt(sum((v/m)**2))
2941   end if
2942 end function vnorm
2943
2944 ! CONTRACT qmul: sixteen Hamilton terms as rounded fraction products with
2945 ! exactly tracked exponents, per-component classical order, common-max
2946 ! accumulation; product rounding, alignment underflow, and cancellation are
2947 ! possible; over/underflow of a restored component refers to the COMPUTED
2948 ! accumulation; general correctly-rounded products are NOT guaranteed.
2949 pure function qmul(a, b) result(c)
2950   real(wp), intent(in) :: a(4), b(4)
2951   real(wp) :: c(4)
2952   integer, parameter :: JA(4,4) = reshape([1,2,3,4, 1,2,3,4, 1,2,3,4, 1,2,3,4], [4,4])
2953   integer, parameter :: KB(4,4) = reshape([1,2,3,4, 2,1,4,3, 3,4,1,2, 4,3,2,1], [4,4])
2954   real(wp), parameter :: SG(4,4) = reshape([ 1.0_wp,-1.0_wp,-1.0_wp,-1.0_wp, &
2955         1.0_wp, 1.0_wp, 1.0_wp,-1.0_wp, &
2956         1.0_wp,-1.0_wp, 1.0_wp, 1.0_wp, &
2957         1.0_wp, 1.0_wp,-1.0_wp, 1.0_wp], [4,4])
2958
2959   real(wp) :: x1, x2, f(4), ssum
2960   integer :: i, t, e(4), emax, nkeep
2961   logical :: keep(4)
2962   do i = 1, 4
2963     if (.not.(is_finite(a(i)) .and. is_finite(b(i)))) then
2964       c = qnan()
2965       return
2966     end if

```

```

2966 end do
2967 do i = 1, 4
2968   nkeep = 0; emax = 0
2969   do t = 1, 4
2970     x1 = a(JA(t,i)); x2 = b(KB(t,i))
2971     keep(t) = (x1 /= 0.0_wp .and. x2 /= 0.0_wp)
2972     if (keep(t)) then
2973       f(t) = fraction(x1) * fraction(x2)
2974       e(t) = exponent(x1) + exponent(x2)
2975       if (nkeep == 0 .or. e(t) > emax) emax = e(t)
2976       nkeep = nkeep + 1
2977     end if
2978   end do
2979   if (nkeep == 0) then
2980     c(i) = 0.0_wp
2981     cycle
2982   end if
2983   ssum = 0.0_wp
2984   do t = 1, 4
2985     if (keep(t)) ssum = ssum + SG(t,i) * scale(f(t), e(t) - emax)
2986   end do
2987   c(i) = scale(ssum, emax)
2988 end do
2989 end function qmul
2990
2991 pure function qconj(a) result(c)
2992   real(wp), intent(in) :: a(4)
2993   real(wp) :: c(4)
2994   c = [a(1), -a(2), -a(3), -a(4)]
2995 end function qconj
2996
2997 ! CONTRACT det3 / det3_row2: six signed Leibniz monomials as rounded
2998 ! fraction products with exact exponents, fixed distinct accumulation
2999 ! orders, common-max restore; same caveats as qmul; the two routines share
3000 ! one IEEE model and differ by evaluation order and factorization only.
3001 pure function det3(r) result(d)
3002   real(wp), intent(in) :: r(3,3)
3003   real(wp) :: d, s
3004   integer :: i, j
3005   s = 0.0_wp
3006   do i = 1, 3
3007     do j = 1, 3
3008       if (.not. is_finite(r(i,j))) then
3009         d = qnan()
3010         return
3011       end if
3012       s = max(s, abs(r(i,j)))
3013     end do
3014   end do
3015   if (s == 0.0_wp) then
3016     d = 0.0_wp
3017     return
3018   end if
3019   block
3020     integer, parameter :: PM(3,6) = reshape([1,2,3, 1,3,2, 2,1,3, 2,3,1, 3,1,2,
3021         3,2,1], [3,6])
3022     real(wp), parameter :: SG(6) = [1.0_wp, -1.0_wp, -1.0_wp, 1.0_wp, 1.0_wp, -1.0_wp]
3023     real(wp) :: x1, x2, x3, f(6), ssum
3024     integer :: e(6), emax, kk, nkeep
3025     logical :: keep(6)
3026     nkeep = 0; emax = 0
3027     do kk = 1, 6
3028       x1 = r(1, PM(1,kk)); x2 = r(2, PM(2,kk)); x3 = r(3, PM(3,kk))
3029       keep(kk) = (x1 /= 0.0_wp .and. x2 /= 0.0_wp .and. x3 /= 0.0_wp)
3030       if (keep(kk)) then
3031         f(kk) = fraction(x1) * fraction(x2) * fraction(x3)
3032         e(kk) = exponent(x1) + exponent(x2) + exponent(x3)
3033         if (nkeep == 0 .or. e(kk) > emax) emax = e(kk)
3034         nkeep = nkeep + 1
3035       end if
3036     end do
3037     if (nkeep == 0) then
3038       d = 0.0_wp ! every monomial structurally zero: exactly zero
3039       return
3040     end if
3041     ssum = 0.0_wp
3042     do kk = 1, 6
3043       if (keep(kk)) ssum = ssum + SG(kk) * scale(f(kk), e(kk) - emax)
3044     end do
3045     if (ssum == 0.0_wp) then
3046       d = 0.0_wp ! cancellation at the accumulation's resolution
3047       return
3048     end if
3049     d = scale(ssum, emax)
3050   end block
3051 end function det3
3052
3053 pure function det3_row2(r) result(d)
3054   real(wp), intent(in) :: r(3,3)
3055   real(wp) :: d, s
3056   integer :: i, j
3057   s = 0.0_wp
3058   do i = 1, 3
3059     do j = 1, 3
3060       if (.not. is_finite(r(i,j))) then
3061         d = qnan()
3062         return
3063       end if
3064       s = max(s, abs(r(i,j)))
3065     end do
3066   end do
3067   if (s == 0.0_wp) then
3068     d = 0.0_wp
3069     return
3070   end if
3071   block

```

```

3071 integer, parameter :: PM(3,6) = reshape([2,1,3, 3,1,2, 1,2,3, 3,2,1, 1,3,2,
3072      2,3,1], [3,6])
3073 real(wp), parameter :: SG(6) = [-1.0_wp, 1.0_wp, 1.0_wp, -1.0_wp, -1.0_wp, 1.0_wp]
3074 real(wp) :: x1, x2, x3, f(6), ssum
3075 integer :: e(6), emax, kk, nkeep
3076 logical :: keep(6)
3077 nkeep = 0; emax = 0
3078 do kk = 1, 6
3079   x1 = r(1, PM(1,kk)); x2 = r(2, PM(2,kk)); x3 = r(3, PM(3,kk))
3080   keep(kk) = (x1 /= 0.0_wp .and. x2 /= 0.0_wp .and. x3 /= 0.0_wp)
3081   if (keep(kk)) then
3082     f(kk) = fraction(x1) * fraction(x2) * fraction(x3)
3083     e(kk) = exponent(x1) + exponent(x2) + exponent(x3)
3084     if (nkeep == 0 .or. e(kk) > emax) emax = e(kk)
3085     nkeep = nkeep + 1
3086   end if
3087 end do
3088 if (nkeep == 0) then
3089   d = 0.0_wp ! every monomial structurally zero: exactly zero
3090   return
3091 end if
3092 ssum = 0.0_wp
3093 do kk = 1, 6
3094   if (keep(kk)) ssum = ssum + SG(kk) * scale(f(kk), e(kk) - emax)
3095 end do
3096 if (ssum == 0.0_wp) then
3097   d = 0.0_wp ! cancellation at the accumulation's resolution
3098   return
3099 end if
3100 d = scale(ssum, emax)
3101 end block
3102 end function det3_row2
3103
3104 subroutine jacobi3(rin, eig, converged)
3105 real(wp), intent(in) :: rin(3,3)
3106 real(wp), intent(out) :: eig(3)
3107 logical, intent(out), optional :: converged
3108 real(wp) :: a(3,3), app, aqq, apq, phi, c, s, tmp, off, frob
3109 integer :: sweep, p, q, k
3110 logical :: conv
3111 real(wp) :: sscale
3112 conv = .false.
3113 if (present(converged)) converged = conv
3114 if (any(.not. is_finite(rin))) then
3115   eig = qnan()
3116   return
3117 end if
3118 a = 0.5_wp*rin + 0.5_wp*transpose(rin)
3119 sscale = maxval(abs(a))
3120 if (sscale == 0.0_wp) then
3121   eig = 0.0_wp
3122   conv = .true.
3123   if (present(converged)) converged = conv
3124   return
3125 end if
3126 a = a / sscale
3127 frob = sqrt(sum(a*a))
3128 do sweep = 1, 60
3129   off = sqrt(a(1,2)**2 + a(1,3)**2 + a(2,3)**2 &
3130     + a(2,1)**2 + a(3,1)**2 + a(3,2)**2)
3131   if (off <= 16.0_wp*U_M*max(frob, tiny(1.0_wp))) then
3132     conv = .true.
3133     exit
3134   end if
3135   do p = 1, 2
3136     do q = p+1, 3
3137       apq = a(p,q)
3138       if (abs(apq) <= tiny(1.0_wp)) cycle
3139       app = a(p,p); aqq = a(q,q)
3140       phi = 0.5_wp * atan2(2.0_wp*apq, aqq - app)
3141       c = cos(phi); s = sin(phi)
3142       do k = 1, 3
3143         tmp = c*a(p,k) - s*a(q,k)
3144         a(q,k) = s*a(p,k) + c*a(q,k)
3145         a(p,k) = tmp
3146       end do
3147       do k = 1, 3
3148         tmp = c*a(k,p) - s*a(k,q)
3149         a(k,q) = s*a(k,p) + c*a(k,q)
3150         a(k,p) = tmp
3151       end do
3152       a(p,q) = 0.0_wp
3153       a(q,p) = 0.0_wp
3154     end do
3155   end do
3156   eig = sscale * [a(1,1), a(2,2), a(3,3)]
3157   if (present(converged)) converged = conv
3158   do p = 1, 2
3159     do q = p+1, 3
3160       if (eig(q) < eig(p)) then
3161         tmp = eig(p); eig(p) = eig(q); eig(q) = tmp
3162       end if
3163     end do
3164   end do
3165 end subroutine jacobi3
3166
3167 function cond3(r) result(kap)
3168 real(wp), intent(in) :: r(3,3)
3169 real(wp) :: kap, eig(3)
3170 logical :: conv
3171 call jacobi3(r, eig, conv)
3172 if (any(is_nan(eig)) .or. .not. conv) then
3173   kap = huge(1.0_wp)
3174   return
3175 end if
3176
3177 if (eig(1) <= 8.0_wp*U_M*max(abs(eig(3)), 1.0_wp)) then
3178   kap = huge(1.0_wp)
3179 else
3180   kap = eig(3) / eig(1)
3181 end if
3182 end function cond3
3183
3184 pure subroutine gram3_fixed(q, r)
3185 real(wp), intent(in) :: q(:, :)
3186 real(wp), intent(out) :: r(3,3)
3187 integer :: i, j, k
3188 if (size(q,1) < 3) then
3189   r = qnan()
3190   return
3191 end if
3192 r = 0.0_wp
3193 do i = 1, 3
3194   do j = 1, 3
3195     do k = 1, size(q,2)
3196       r(i,j) = r(i,j) + q(i,k)*q(j,k)
3197     end do
3198   end do
3199 end subroutine gram3_fixed
3200
3201 pure function det3_chol(r) result(d)
3202 real(wp), intent(in) :: r(3,3)
3203 real(wp) :: d, l11, l21, l31, l22, l32, l33, t
3204 d = qnan() ! default NaN when not SPD
3205 if (any(.not. is_finite(r))) return
3206 if (maxval(abs(r - transpose(r))) > 32.0_wp*U_M*max(1.0_wp, maxval(abs(r)))) return
3207 if (r(1,1) <= 0.0_wp) return
3208 l11 = sqrt(r(1,1)); l21 = r(2,1)/l11; l31 = r(3,1)/l11
3209 t = r(2,2) - l21*l21
3210 if (t <= 0.0_wp) return
3211 l22 = sqrt(t); l32 = (r(3,2) - l21*l31)/l22
3212 t = r(3,3) - l31*l31 - l32*l32
3213 if (t <= 0.0_wp) return
3214 l33 = sqrt(t)
3215 d = (l11*l22*l33)**2
3216 end function det3_chol
3217 end module ra_prims
3218
3219 ! -----
3220 module ra_floor
3221 use ra_kinds
3222 implicit none
3223 contains
3224 pure function heis_floor(m, dx) result(e)
3225 real(wp), intent(in) :: m, dx
3226 real(wp) :: e
3227 if (.not.(is_finite(m) .and. is_finite(dx))) .or. m <= 0.0_wp .or. dx <= 0.0_wp then
3228   e = qnan()
3229   return
3230 end if
3231 block
3232   real(wp) :: f
3233   integer :: ex
3234   f = (fraction(HBAR)*fraction(HBAR)) / (fraction(m)*(fraction(dx)*fraction(dx)))
3235   ex = 2*exponent(HBAR) - exponent(m) - 2*exponent(dx) - 3
3236   e = scale(f, ex)
3237 end block
3238 end function heis_floor
3239
3240 pure function zpe(w) result(e)
3241 real(wp), intent(in) :: w
3242 real(wp) :: e
3243 if (.not. is_finite(w) .or. w <= 0.0_wp) then
3244   e = qnan()
3245   return
3246 end if
3247 e = 0.5_wp * HBAR * w
3248 end function zpe
3249
3250 pure function prod_scaled(a, b, c, d) result(r)
3251 real(wp), intent(in) :: a, b, c, d
3252 real(wp) :: r, f
3253 integer :: e
3254 if (a == 0.0_wp .or. b == 0.0_wp .or. c == 0.0_wp .or. d == 0.0_wp) then
3255   r = 0.0_wp
3256   return
3257 end if
3258 f = fraction(a) * fraction(b)
3259 e = exponent(a) + exponent(b) + exponent(f)
3260 f = fraction(f)
3261 f = f * fraction(c)
3262 e = e + exponent(c) + exponent(f)
3263 f = fraction(f)
3264 f = f * fraction(d)
3265 e = e + exponent(d) + exponent(f)
3266 f = fraction(f)
3267 r = scale(f, e)
3268 end function prod_scaled
3269
3270 pure function landauer(tkel, bits) result(e)
3271 real(wp), intent(in) :: tkel, bits
3272 real(wp) :: e
3273 if (.not.(is_finite(tkel) .and. is_finite(bits))) .or. tkel <= 0.0_wp .or. bits < 0.0_wp then
3274   e = qnan()
3275   return
3276 end if
3277 e = prod_scaled(bits, KB, tkel, ln2())
3278 end function landauer
3279
3280 subroutine tau_perp(de, tau, frozen)

```

```

3281     real(wp), intent(in) :: de
3282     real(wp), intent(out) :: tau
3283     logical, intent(out) :: frozen
3284     frozen = (.not. is_finite(de)) .or. (de <= 0.0_wp)
3285     if (frozen) then
3286         tau = huge(1.0_wp)
3287     else
3288         tau = (0.5_wp * pi()) / de
3289     end if
3290 end subroutine tau_perp
3291
3292 pure function survival(t) result(p)
3293     real(wp), intent(in) :: t
3294     real(wp) :: p
3295     p = cos(0.5_wp*t)**2
3296 end function survival
3297 end module ra_floor
3298
3299 ! -----
3300 module ra_integers
3301     use ra_kinds
3302     implicit none
3303     contains
3304     function igmul(a, b) result(c)
3305         integer(ik), intent(in) :: a(4), b(4)
3306         integer(ik) :: c(4)
3307         if (any(a > IK_SAFE .or. a < -IK_SAFE) .or. any(b > IK_SAFE .or. b < -IK_SAFE)) then
3308             error stop 'igmul: integer intake beyond the 2^29 overflow-safe bound'
3309         end if
3310         c(1) = a(1)*b(1) - a(2)*b(2) - a(3)*b(3) - a(4)*b(4)
3311         c(2) = a(1)*b(2) + a(2)*b(1) + a(3)*b(4) - a(4)*b(3)
3312         c(3) = a(1)*b(3) - a(2)*b(4) + a(3)*b(1) + a(4)*b(2)
3313         c(4) = a(1)*b(4) + a(2)*b(3) - a(3)*b(2) + a(4)*b(1)
3314     end function igmul
3315
3316     pure function ik_inbound(v) result(okb)
3317         integer(ik), intent(in) :: v(:)
3318         logical :: okb
3319         okb = .not. any(v > IK_SAFE .or. v < -IK_SAFE)
3320     end function ik_inbound
3321
3322     pure function iqconj(a) result(c)
3323         integer(ik), intent(in) :: a(4)
3324         integer(ik) :: c(4)
3325         if (.not. ik_inbound(a)) then
3326             error stop 'iqconj: integer intake beyond the 2^29 overflow-safe bound'
3327         end if
3328         c = [a(1), -a(2), -a(3), -a(4)]
3329     end function iqconj
3330
3331     function iomul(x, y) result(z)
3332         integer(ik), intent(in) :: x(8), y(8)
3333         integer(ik) :: z(8), p(4), q(4), r(4), s(4)
3334         if (.not. (ik_inbound(x) .and. ik_inbound(y))) then
3335             error stop 'iomul: integer intake beyond the 2^29 overflow-safe bound'
3336         end if
3337         p = x(1:4); q = x(5:8); r = y(1:4); s = y(5:8)
3338         z(1:4) = iomul(p, r) - iomul(iqconj(s), q)
3339         z(5:8) = iomul(s, p) + iomul(q, iqconj(r))
3340     end function iomul
3341
3342     pure function ioconj(x) result(z)
3343         integer(ik), intent(in) :: x(8)
3344         integer(ik) :: z(8)
3345         if (.not. ik_inbound(x)) then
3346             error stop 'ioconj: integer intake beyond the 2^29 overflow-safe bound'
3347         end if
3348         z(1:4) = iqconj(x(1:4)); z(5:8) = -x(5:8)
3349     end function ioconj
3350
3351     function ismul(x, y) result(z)
3352         integer(ik), intent(in) :: x(16), y(16)
3353         integer(ik) :: z(16), p(8), q(8), r(8), s(8)
3354         if (.not. (ik_inbound(x) .and. ik_inbound(y))) then
3355             error stop 'ismul: integer intake beyond the 2^29 overflow-safe bound'
3356         end if
3357         p = x(1:8); q = x(9:16); r = y(1:8); s = y(9:16)
3358         z(1:8) = iomul(p, r) - iomul(ioconj(s), q)
3359         z(9:16) = iomul(s, p) + iomul(q, ioconj(r))
3360     end function ismul
3361
3362     pure function inrm(v) result(n)
3363         integer(ik), intent(in) :: v(:)
3364         integer(ik) :: n
3365         if (size(v) > 31) then
3366             error stop 'inrm: vector length exceeds the proven overflow bound (31)'
3367         end if
3368         if (any(v > IK_SAFE .or. v < -IK_SAFE)) then
3369             error stop 'inrm: integer intake beyond the 2^29 overflow-safe bound'
3370         end if
3371         n = sum(v*v)
3372     end function inrm
3373
3374     subroutine hurwitz_units(u)
3375         integer(ik), intent(out) :: u(4,24)
3376         integer :: n, i, s, s1, s2, s3, s4
3377         n = 0
3378         do i = 1, 4
3379             do s = -1, 1, 2
3380                 n = n + 1
3381                 u(:,n) = 0_ik
3382                 u(i,n) = int(2*s, ik)
3383             end do
3384         end do
3385         do s1 = -1, 1, 2
3386             do s2 = -1, 1, 2

```

```

3387                 do s3 = -1, 1, 2
3388                     do s4 = -1, 1, 2
3389                         n = n + 1
3390                         u(:,n) = int([s1, s2, s3, s4], ik)
3391                     end do
3392                 end do
3393             end do
3394         end do
3395     end subroutine hurwitz_units
3396 end module ra_integers
3397
3398 ! -----
3399 module ra_tongue
3400     use ra_kinds
3401     use ra_prims
3402     implicit none
3403     integer, parameter :: NSLOT = 4
3404     integer, parameter :: SLOT_E(NSLOT) = [101, 102, 103, 104]
3405     integer, parameter :: SLOT_K(NSLOT) = [201, 202, 203, 204]
3406     integer, parameter :: SLOT_R(NSLOT) = [301, 302, 303, 304]
3407     integer, parameter :: RA_SENT(6) = [102, 101, 202, 203, 302, 303]
3408     contains
3409     pure subroutine slot_census(sent, ne, nk, nr)
3410         integer, intent(in) :: sent(:)
3411         integer, intent(out) :: ne, nk, nr
3412         integer :: t
3413         ne = 0; nk = 0; nr = 0
3414         do t = 1, size(sent)
3415             if (any(SLOT_E == sent(t))) ne = ne + 1
3416             if (any(SLOT_K == sent(t))) nk = nk + 1
3417             if (any(SLOT_R == sent(t))) nr = nr + 1
3418         end do
3419     end subroutine slot_census
3420     pure logical function disjoint(a, b)
3421         integer, intent(in) :: a(:), b(:)
3422         integer :: i, j
3423         disjoint = .true.
3424         do i = 1, size(a)
3425             do j = 1, size(b)
3426                 if (a(i) == b(j)) then
3427                     disjoint = .false.
3428                     return
3429                 end if
3430             end do
3431         end do
3432     end function disjoint
3433
3434     subroutine seal_l(ne, nk, nr, collide, tok, why)
3435         integer, intent(in) :: ne, nk, nr ! populated counts per slot
3436         logical, intent(in) :: collide ! a vocabulary collision present
3437         character(*), intent(out) :: tok, why
3438         integer :: slots
3439         slots = merge(1,0,ne>0) + merge(1,0,nk>0) + merge(1,0,nr>0)
3440         if (collide) then
3441             tok = '[X]'; why = 'Seal L: LIT collision, slot vocabularies intersect'
3442         else if (slots /= 3) then
3443             tok = '[X]';
3444             write(why, '(a,i0,a)') 'Seal L: deletion test returns ', slots, ' slots, not 3'
3445         else
3446             tok = '[LOCK-L]'; why = 'Seal L: three slots, LIT disjoint (order not checked at this interface)'
3447         end if
3448     end subroutine seal_l
3449
3450     subroutine seal_l_ordered(sent, tok, why)
3451         integer, intent(in) :: sent(:)
3452         character(*), intent(out) :: tok, why
3453         integer :: ne, nk, nr, t, laste, firstk, lastk, firstr
3454         call slot_census(sent, ne, nk, nr)
3455         if (ne == 0 .or. nk == 0 .or. nr == 0) then
3456             call seal_l(ne, nk, nr, .false., tok, why)
3457             return
3458         end if
3459         laste = 0; firstk = size(sent)+1; lastk = 0; firstr = size(sent)+1
3460         do t = 1, size(sent)
3461             if (any(SLOT_E == sent(t))) laste = t
3462             if (any(SLOT_K == sent(t))) then
3463                 if (t < firstk) firstk = t
3464                 if (t > lastk) lastk = t
3465             end if
3466             if (any(SLOT_R == sent(t))) then
3467                 if (t < firstr) firstr = t
3468             end if
3469         end do
3470         if (.not. (laste < firstk .and. lastk < firstr)) then
3471             tok = '[X]'; why = 'Seal L: the ordered arrow E->K->R is broken in the encoding'
3472             return
3473         end if
3474         call seal_l(ne, nk, nr, .false., tok, why)
3475         if (tok == '[LOCK-L]') then
3476             why = 'Seal L: three slots, LIT disjoint, ordered E->K->R (positional check executed)'
3477         end if
3478     end subroutine seal_l_ordered
3479
3480     pure function cut3(v) result(w)
3481         real(wp), intent(in) :: v(3)
3482         real(wp) :: w(3)
3483         w = [v(1), v(2), 0.0_wp]
3484     end function cut3
3485 end module ra_tongue
3486
3487 ! -----
3488 module ra_kernel
3489     use ra_kinds
3490     use ra_prims

```

```

3491 implicit none
3492 integer, parameter :: RC_OK = 0
3493 integer, parameter :: RC_SHORTFALL = 1
3494 integer, parameter :: RC_INTAKE = 2
3495 integer, parameter :: RC_ZEROWAR = 3
3496 integer, parameter :: RC_COVBLOCK = 4
3497 integer, parameter :: RC_ABSORBED = 5
3498 integer, parameter :: RC_COPLANAR = 6
3499 integer, parameter :: RC_COLLAPSE = 7
3500 integer, parameter :: RC_KAPPA = 8
3501 integer, parameter :: RC_ENGINC = 9
3502 integer, parameter :: RC_LOCK = 10
3503 contains
3504 subroutine prepare_rows(m, q, ok, why, code)
3505   real(wp), intent(in) :: m(:, :) ! (3,N)
3506   real(wp), intent(out) :: q(size(m,1), size(m,2))
3507   logical, intent(out) :: ok
3508   character(*), intent(out) :: why
3509   integer, intent(out) :: code
3510   integer :: i, n, j2
3511   real(wp) :: mu, sd, nrm, rmax
3512   real(wp) :: qt(size(m,1), size(m,2))
3513   n = size(m,2); ok = .true.; why = 'rows prepared'; code = RC_OK
3514   q = 0.0_wp
3515   if (size(m,1) /= 3) then
3516     ok = .false.; why = 'row count is not three'; code = RC_INTAKE
3517   end if
3518   if (n < 2) then
3519     ok = .false.; why = 'fewer than two contexts'; code = RC_INTAKE
3520   end if
3521   do i = 1, 3
3522     do j2 = 1, n
3523       if (.not. is_finite(m(i,j2))) then
3524         ok = .false.; why = 'non-finite row at intake'; code = RC_INTAKE
3525       end if
3526     end do
3527     rmax = maxval(abs(m(i,:)))
3528     if (rmax > 0.0_wp) then
3529       qt(i,:) = m(i,:) / rmax
3530     else
3531       qt(i,:) = m(i,:)
3532     end if
3533     mu = sum(qt(i,:)) / real(n, wp)
3534     qt(i,:) = qt(i,:) - mu
3535     sd = sqrt(sum(qt(i,:)**2) / real(n-1, wp))
3536     if (sd <= 0.0_wp) then
3537       ok = .false.; why = 'zero-variance row: axis carries no content'
3538       code = RC_ZEROWAR
3539     end if
3540     qt(i,:) = qt(i,:) / sd
3541     nrm = vnorm(qt(i,:))
3542     qt(i,:) = qt(i,:) / nrm
3543   end do
3544   q = qt ! publish on success only
3545 end subroutine prepare_rows
3546
3547 subroutine project_cov(q, c, qf, ok, why, code, kcc)
3548   real(wp), intent(in) :: q(:, :) ! (3,N) prepared
3549   real(wp), intent(in) :: c(:, :) ! (k,N) covariates, k = 0..2
3550   real(wp), intent(out) :: qf(size(q,1), size(q,2))
3551   logical, intent(out) :: ok
3552   character(*), intent(out) :: why
3553   integer, intent(out) :: code
3554   real(wp), intent(out) :: kcc
3555   real(wp) :: cc(2,2), cm(2, size(q,2)), rhs(2), beta(2), det, mu, tolrk
3556   real(wp) :: qq(size(q,1), size(q,2))
3557   integer :: i, j, n, k
3558   n = size(q,2); k = size(c,1); qf = 0.0_wp; ok = .true.; why = 'no covariates'
3559   code = RC_OK
3560   kcc = 1.0_wp
3561   if (size(q,1) /= 3) then
3562     ok = .false.; code = RC_INTAKE
3563     why = 'prepared-row count is not three'
3564     kcc = huge(1.0_wp)
3565   end if
3566   if (n < 2) then
3567     ok = .false.; code = RC_INTAKE
3568     why = 'fewer than two contexts'
3569     kcc = huge(1.0_wp)
3570   end if
3571   if (k >= 1 .and. size(c,2) /= n) then
3572     ok = .false.; code = RC_INTAKE
3573     why = 'covariate column count does not match prepared-row context count'
3574     kcc = huge(1.0_wp)
3575   end if
3576   qf = q ! all projection work lands in the working array; qf
3577   if (k == 0) then
3578     qf = qw
3579   end if
3580   if (k > 2) then
3581     ok = .false.; code = RC_COVBLOCK
3582     why = 'covariate block: more than two covariates, not implemented'
3583   end if
3584   do i = 1, k
3585     do j = 1, n
3586       if (.not. is_finite(c(i,j))) then
3587         ok = .false.; code = RC_INTAKE
3588       end if
3589     end do
3590   end do
3591   why = 'non-finite covariate at intake'
3592   return
3593 end if
3594 cm(i,1:n) = c(i,:) / det
3595 mu = sum(cm(i,1:n)) / real(n, wp)
3596 cm(i,1:n) = cm(i,1:n) - mu
3597 det = maxval(abs(cm(i,1:n)))
3598 if (det <= 32.0_wp * U_M) then
3599   ok = .false.; code = RC_COVBLOCK
3600   why = 'covariate block rank-deficient or ill-conditioned'
3601   return
3602 end if
3603 cm(i,1:n) = cm(i,1:n) / det
3604 do i = 1, k
3605   if (k == 1) then
3606     det = sum(cm(1,1:n)**2)
3607     kcc = 1.0_wp
3608     do i = 1, 3
3609       beta(1) = sum(qw(i,:)*cm(1,1:n)) / det
3610       qw(i,:) = qw(i,:) - beta(1)*cm(1,1:n)
3611     end do
3612     why = 'one covariate projected'
3613     qf = qw
3614     return
3615   end if
3616   cc(1,1) = sum(cm(1,1:n)**2); cc(2,2) = sum(cm(2,1:n)**2)
3617   cc(1,2) = sum(cm(1,1:n)*cm(2,1:n)); cc(2,1) = cc(1,2)
3618   det = cc(1,1)*cc(2,2) - cc(1,2)*cc(2,1)
3619   tolrk = 64.0_wp * U_M * max(cc(1,1)*cc(2,2), tiny(1.0_wp))
3620   if (det <= tolrk) then
3621     ok = .false.; code = RC_COVBLOCK
3622     why = 'covariate block rank-deficient or ill-conditioned'
3623     return
3624   end if
3625   block
3626     real(wp) :: tr, dd, l1, l2
3627     tr = cc(1,1) + cc(2,2)
3628     dd = sqrt(max(0.0_wp, tr*tr - 4.0_wp*det))
3629     l1 = 0.5_wp*(tr - dd); l2 = 0.5_wp*(tr + dd)
3630     if (l1 <= 0.0_wp) then
3631       kcc = huge(1.0_wp)
3632     else
3633       kcc = l2 / l1
3634     end if
3635   end block
3636   if (kcc >= 1.0e6_wp) then
3637     ok = .false.; code = RC_COVBLOCK
3638     why = 'covariate block rank-deficient or ill-conditioned'
3639     return
3640   end if
3641   do i = 1, 3
3642     rhs(1) = sum(qw(i,:)*cm(1,1:n)); rhs(2) = sum(qw(i,:)*cm(2,1:n))
3643     beta(1) = (cc(2,2)*rhs(1) - cc(1,2)*rhs(2)) / det
3644     beta(2) = (-cc(2,1)*rhs(1) + cc(1,1)*rhs(2)) / det
3645     do j = 1, n
3646       qw(i,j) = qw(i,j) - beta(1)*cm(1,j) - beta(2)*cm(2,j)
3647     end do
3648   end do
3649   why = 'two covariates projected (common-source projection)'
3650   qf = qw
3651 end subroutine project_cov
3652
3653 subroutine span_basis(q, b, rank)
3654   real(wp), intent(in) :: q(:, :)
3655   real(wp), intent(out) :: b(size(q,1), size(q,2))
3656   integer, intent(out) :: rank
3657   real(wp) :: v(size(q,2)), nrm
3658   integer :: i, j, pass
3659   real(wp) :: rn0
3660   rank = 0; b = 0.0_wp
3661   do i = 1, size(q,1)
3662     v = q(i,:)
3663     if (any(.not. is_finite(v))) cycle
3664     rn0 = maxval(abs(v))
3665     if (rn0 == 0.0_wp) cycle
3666     v = v / rn0
3667     rn0 = vnorm(v)
3668     v = v / rn0
3669     do pass = 1, 2
3670       do j = 1, rank
3671         v = v - sum(v*b(j,:),) * b(j,:)
3672       end do
3673     end do
3674     nrm = vnorm(v)
3675     if (is_finite(nrm) .and. nrm > 1.0e-12_wp) then
3676       rank = rank + 1
3677       b(rank,:) = v / nrm
3678     end if
3679   end do
3680 end subroutine span_basis
3681
3682 ! CONTRACT lambda_of / gram3_fixed: shape guards only, quiet-NaN refusal
3683 ! before any element touch; rank, finiteness, and magnitude semantics are
3684 ! the caller's contract (kernel_full ranks and screens first).
3685 function lambda_of(q, b) result(lam)
3686   real(wp), intent(in) :: q(:, :), b(:, :)
3687   real(wp) :: lam, co(3,3), quat(4,3), t(4)
3688   integer :: i
3689   if (size(q,1) < 3 .or. size(b,1) < 3 .or. size(q,2) /= size(b,2)) then
3690     lam = qnan()
3691   end if
3692   return

```

```

3703 end if
3704 do i = 1, 3
3705   co(i,1) = sum(q(i,:)*b(1,:))
3706   co(i,2) = sum(q(i,:)*b(2,:))
3707   co(i,3) = sum(q(i,:)*b(3,:))
3708   quat(:,i) = [0.0_wp, co(i,1), co(i,2), co(i,3)]
3709 end do
3710 t = qmul(qmul(quat(:,1), quat(:,2)), quat(:,3))
3711 lam = t(1)
3712 end function lambda_of
3713
3714 function eta_share(row, c) result(eta)
3715   real(wp), intent(in) :: row(:), c(:,:)
3716   real(wp) :: eta, cm(2,size(row)), rm(size(row))
3717   real(wp) :: cc(2,2), rhs(2), beta(2), det, sst, ssf, mu
3718   integer :: i, n, k
3719   n = size(row); k = size(c,1)
3720   if (k >= 1 .and. size(c,2) /= n) then
3721     eta = qnan()
3722     return
3723   end if
3724   if (k > 2) then
3725     eta = qnan()
3726     return
3727   end if
3728   if (any(.not. is_finite(row))) then
3729     eta = qnan()
3730     return
3731   end if
3732   if (k >= 1) then
3733     if (any(.not. is_finite(c))) then
3734       eta = qnan()
3735       return
3736     end if
3737   end if
3738   block
3739     real(wp) :: rscale
3740     rscale = maxval(abs(row))
3741     if (rscale == 0.0_wp .or. .not. is_finite(rscale)) then
3742       eta = 0.0_wp
3743       return
3744     end if
3745     rm = row / rscale
3746   end block
3747   mu = sum(rm)/real(n,wp); rm = rm - mu
3748   sst = sum(rm*rm)
3749   eta = 0.0_wp
3750   if (sst <= 0.0_wp .or. k == 0) return
3751   do i = 1, k
3752     det = maxval(abs(c(i,:)))
3753     if (det == 0.0_wp) det = 1.0_wp
3754     cm(i,:) = c(i,:) / det
3755     mu = sum(cm(i,:))/real(n,wp)
3756     cm(i,:) = cm(i,:) - mu
3757     det = maxval(abs(cm(i,:)))
3758     if (det <= 32.0_wp*U_M) then
3759       eta = qnan()
3760       return
3761     end if
3762     cm(i,:) = cm(i,:) / det
3763   end do
3764   if (k == 1) then
3765     det = sum(cm(1,:)**2)
3766     beta(1) = sum(rm*cm(1,:)) / det
3767     ssf = beta(1)*beta(1)*det
3768     eta = ssf/sst
3769     return
3770   end if
3771   cc(1,1)=sum(cm(1,:)**2); cc(2,2)=sum(cm(2,:)**2)
3772   cc(1,2)=sum(cm(1,:)*cm(2,:)); cc(2,1)=cc(1,2)
3773   det = cc(1,1)*cc(2,2)-cc(1,2)*cc(2,1)
3774   if (det <= 64.0_wp*U_M*max(cc(1,1)*cc(2,2), tiny(1.0_wp))) then
3775     eta = qnan()
3776     return
3777   end if
3778   rhs(1)=sum(rm*cm(1,:)); rhs(2)=sum(rm*cm(2,:))
3779   beta(1)=( cc(2,2)*rhs(1)-cc(1,2)*rhs(2))/det
3780   beta(2)=(-cc(2,1)*rhs(1)+cc(1,1)*rhs(2))/det
3781   ssf = sum((beta(1)*cm(1,:)+beta(2)*cm(2,:))**2)
3782   eta = ssf/sst
3783 end function eta_share
3784
3785 subroutine kernel_full(m, c, tok, why, rcode, lam, detr, kap, spread, resid, &
3786   marg_c, marg_k, detfour)
3787   real(wp), intent(in) :: m(:,:), c(:,:)
3788   character(*), intent(out) :: tok, why
3789   integer, intent(out) :: rcode
3790   real(wp), intent(out) :: lam, detr, kap, spread, resid, marg_c, marg_k
3791   real(wp), intent(out) :: detfour(4)
3792   real(wp) :: q(3,size(m,2)), qf(3,size(m,2)), b(3,size(m,2))
3793   real(wp) :: r(3,3), eig(3), eps, kgate, kcc, dvar, tol
3794   integer :: n, k, i, rank, pcode, nest
3795   logical :: ok, jconv
3796   character(len=96) :: w2
3797   n = size(m,2); k = size(c,1)
3798   if (k >= 1 .and. size(c,2) /= n) then
3799     tok = '[?]'; why = 'covariate column count does not match row context count'
3800     rcode = RC_INTAKE
3801     lam = 0.0_wp; detr = 0.0_wp; kap = 0.0_wp; spread = 0.0_wp
3802     resid = 0.0_wp; marg_c = 0.0_wp; marg_k = 0.0_wp; detfour = 0.0_wp
3803     return
3804   end if
3805   lam = 0.0_wp; detr = 0.0_wp; kap = 0.0_wp; spread = 0.0_wp
3806   resid = 0.0_wp; marg_c = 0.0_wp; marg_k = 0.0_wp; detfour = 0.0_wp
3807   rcode = RC_OK
3808   if (size(m,1) /= 3) then
3809     tok = '[?]'; why = 'row count is not three at intake'; rcode = RC_INTAKE
3810     return
3811   end if
3812   if (n - k < 4) then
3813     tok = '[?]'; why = 'N-k < 4 dimensional shortfall'; rcode = RC_SHORTFALL
3814     return
3815   end if
3816   if (k > 2) then
3817     tok = '[?]'; why = 'covariate block: more than two covariates, not implemented (
3818       kernel intake)'
3819     rcode = RC_COVBLOCK
3820     lam = 0.0_wp; detr = 0.0_wp; kap = 0.0_wp; spread = 0.0_wp
3821     resid = 0.0_wp; marg_c = 0.0_wp; marg_k = 0.0_wp; detfour = 0.0_wp
3822     return
3823   end if
3824   call prepare_rows(m, q, ok, w2, pcode)
3825   if (.not. ok) then
3826     tok = '[?]'; why = trim(w2); rcode = pcode
3827     return
3828   end if
3829   call project_cov(q, c, qf, ok, w2, pcode, kcc)
3830   if (.not. ok) then
3831     tok = '[?]'; why = trim(w2); rcode = pcode
3832     return
3833   end if
3834   do i = 1, 3
3835     dvar = sum(qf(i,:)**2)
3836     if (dvar < 1.0e-9_wp) then
3837       ! post-projection axis absorbed
3838       by a covariate
3839       tok = '[?]'; why = 'post-projection axis absorbed by a covariate'
3840       rcode = RC_ABSORBED
3841       return
3842     end if
3843     qf(i,:) = qf(i,:) / vnorm(qf(i,:))
3844   end do
3845   r = matmul(qf, transpose(qf))
3846   detfour(1) = det3(r)
3847   call jacob3(r, eig, jconv)
3848   detfour(2) = eig(1)*eig(2)*eig(3)
3849   detfour(3) = det3_chol(r)
3850   detfour(4) = det3_row2(r)
3851   nest = 4
3852   if (.not. is_finite(detfour(3))) nest = 3
3853   detr = detfour(1)
3854   if (nest == 4) then
3855     spread = maxval(detfour) - minval(detfour)
3856   else
3857     spread = max(detfour(1), detfour(2), detfour(4)) &
3858       - min(detfour(1), detfour(2), detfour(4))
3859   end if
3860   kap = cond3(r)
3861   if (.not. jconv) then
3862     tok = '[?]'; why = 'engineering-incomplete: eigensolver unconverged'; rcode =
3863       RC_ENGINC
3864     return
3865   end if
3866   call span_basis(qf, b, rank)
3867   if (rank < 3) then
3868     tok = '[X]'; why = 'coplanar collapse: rank(rows) < 3, line not point'
3869     rcode = RC_COPLANAR
3870     detr = 0.0_wp
3871     return
3872   end if
3873   lam = lambda_of(qf, b)
3874   resid = abs(lam*lam - detr)
3875   eps = 100.0_wp * U_M * real(n, wp)
3876   kgate = min(1.0e6_wp, sqrt(27.0_wp / (100.0_wp*U_M*real(n,wp))))
3877   tol = 4.0_wp * kap * U_M
3878   if (detr <= eps) then
3879     tok = '[X]'; why = 'collapse: det(R) <= eps'; rcode = RC_COLLAPSE
3880     return
3881   end if
3882   if (kap >= kgate) then
3883     tok = '[?]'; why = 'kappa(R) >= gate'; rcode = RC_KAPPA
3884     return
3885   end if
3886   marg_c = log10(detr/eps)
3887   marg_k = log10(kgate/kap)
3888   if (spread > tol .or. resid > max(tol, 8.0_wp*U_M)) then
3889     tok = '[?]'; why = 'engineering-incomplete: spread or identity outside tol'
3890     rcode = RC_ENGINC
3891     return
3892   end if
3893   if (nest < 4) then
3894     tok = '[?]'; why = 'engineering-incomplete: estimator shortfall (Cholesky refused
3895       SPD)'
3896     rcode = RC_ENGINC
3897     return
3898   end if
3899   tok = '[LOCK]'
3900   why = 'determination: three independent axes, one point'
3901   rcode = RC_LOCK
3902 end subroutine kernel_full
3903 end module ra_kernel
3904
3905 ! -----
3906 module ra_guards
3907   use ra_kinds
3908   use ra_floor, only: landauer
3909   implicit none
3910   integer :: aegis_deeds = 0
3911   integer :: aegis_last_len = -1
3912   ! every adjudication is itself an act
3913   ! length of the last logic name read
3914 contains
3915   subroutine omega_boundary(bits, tkel, joules, verdict, ocode, irreversible)
3916     real(wp), intent(in) :: bits, tkel
3917     real(wp), intent(out) :: joules
3918   end subroutine omega_boundary

```

```

3911 character(*), intent(out) :: verdict
3912 integer, intent(out), optional :: ocode
3913 logical, intent(in), optional :: irreversible
3914 logical :: irr
3915 irr = .true.
3916 if (present(irreversible)) irr = irreversible
3917 if (.not.is_finite(tkel).and. is_finite(bits)) .or. tkel <= 0.0_wp then
3918   joules = qnan()
3919   verdict = 'refused: intake invalid, the floor prices only finite bits at tkel > 0'
3920   if (present(ocode)) ocode = 0
3921   return
3922 end if
3923 if (bits <= 0.0_wp) then
3924   joules = 0.0_wp
3925   verdict = 'no denial registered; nothing to adjudicate'
3926   if (present(ocode)) ocode = 1
3927 else if (.not. irr) then
3928   joules = 0.0_wp
3929   verdict = 'reversibly registered: floor-zero never cost-zero; Landauer bounds '//
3930     &
3931     'only the irreversible (Bennett), and nothing here commits against the
3932     axiom'
3933   if (present(ocode)) ocode = 3
3934 else
3935   joules = landauer(tkel, bits)
3936   verdict = 'the denial paid the floor; the registered act is an act, the recursion
3937     closes per the seated theorem'
3938   if (present(ocode)) ocode = 2
3939 end if
3940 end subroutine omega_boundary
3941
3942 subroutine aegis_guard(logic_mode, refusal)
3943   character(*), intent(in) :: logic_mode
3944   character(*), intent(out) :: refusal
3945   aegis_deeds = aegis_deeds + 1
3946   refusal = 'p(G) /= G: precisification is an actuation, G is a '// &
3947     'non-actuation, and no logic makes a deed a non-deed'
3948   aegis_last_len = len_trim(logic_mode) ! the parameter is read; the refusal never
3949     varies with it
3950 end subroutine aegis_guard
3951
3952 subroutine aegis_reset()
3953   aegis_deeds = 0
3954 end subroutine aegis_reset
3955
3956 subroutine gol_admit(magnitude, ling, tok, why, gcode)
3957   character(*), intent(in) :: magnitude, ling
3958   character(*), intent(out) :: tok, why
3959   integer, intent(out), optional :: gcode
3960   integer :: gc_l
3961   if (magnitude == 'broken') then
3962     tok = '[X]'; why = 'geometric/math magnitude reports broken geometry; no GOL';
3963     gc_l = 1
3964   else if (magnitude /= 'lock') then
3965     tok = '[?]'; why = 'no geometric/math lock: orthogonal volume unestablished'; gc_l
3966       = 2
3967   else if (ling /= 'lock') then
3968     tok = '[?]'; why = 'magnitude locks but Seal L open: determinant carries no
3969       direction'; gc_l = 3
3970   else
3971     tok = '[GOL-OK]'; why = 'magnitude plus direction; proceed to witness and
3972       asymmetry'; gc_l = 4
3973   end if
3974   if (present(gcode)) gcode = gc_l
3975 end subroutine gol_admit
3976
3977 subroutine imprint_seal(lock_p, lock_n, sl_p, sl_n, g_p, g_n, wit_p, wit_n, tok, why,
3978   icode)
3979   logical, intent(in) :: lock_p, lock_n, sl_p, sl_n, g_p, g_n, wit_p, wit_n
3980   character(*), intent(out) :: tok, why
3981   integer, intent(out), optional :: icode
3982   logical :: clean_p, clean_n
3983   integer :: ic_l
3984   clean_p = lock_p .and. sl_p .and. g_p
3985   clean_n = lock_n .and. sl_n .and. g_n
3986   if (clean_p .and. clean_n) then
3987     tok = '[X]'; why = 'PLATONIC GHOST: both directions clean-lock, no imprint'; ic_l
3988       = 1
3989   else if (clean_p .and. .not. clean_n) then
3990     if (wit_p) then
3991       tok = '[SEAL]'; why = 'only P clean-locks; determinacy witness supplied'; ic_l =
3992         2
3993     else
3994       tok = '[?]'; why = 'residence: P clean-locks, imprint unproven (no witness)';
3995       ic_l = 3
3996     end if
3997   else if (clean_n .and. .not. clean_p) then
3998     if (wit_n) then
3999       tok = '[SEAL]'; why = 'only not-P clean-locks; determinacy witness supplied';
4000       ic_l = 2
4001     else
4002       tok = '[?]'; why = 'residence: not-P clean-locks, imprint unproven (no witness)'
4003       ; ic_l = 3
4004     end if
4005   else if (.not. lock_p .and. .not. lock_n) then
4006     tok = '[?]'; why = 'flat: neither direction populated'; ic_l = 4
4007   else
4008     tok = '[?]'; why = 'uncertified: lock present but Seal L / gate screen unconfirmed
4009       '; ic_l = 5
4010   end if
4011   if (present(icode)) icode = ic_l
4012 end subroutine imprint_seal
4013
4014 subroutine row_cascade(nonvacuous, ve_live, forward, populated, worldly_row, &
4015   terms_closed, term_worldly, frame_closed, annotate, tok, why,
4016   ccode)
4017   logical, intent(in) :: nonvacuous, ve_live, forward, populated, worldly_row
4018   logical, intent(in) :: terms_closed, term_worldly, frame_closed, annotate
4019   character(*), intent(out) :: tok, why
4020   integer, intent(out), optional :: ccode
4021   integer :: cc_l
4022   if (.not. nonvacuous) then
4023     tok = '[REFUSED]'; why = 'F-0: contentless; a contentless residence verifies
4024       nothing'
4025     cc_l = 1
4026   else if (ve_live .or. forward) then
4027     tok = '[III]'; why = 'R-3: V_E live or Forward; world-rowed, revisability in the
4028       rows'
4029     cc_l = 3
4030   else if (.not. populated) then
4031     tok = '[II]'; why = 'S-1: third axis unpopulated; two planes meet in a line, no
4032       lock'
4033     cc_l = 2
4034   else if (worldly_row) then
4035     tok = '[III]'; why = 'S-2: a load-bearing row is furnished by the world'
4036     cc_l = 3
4037   else if (.not. terms_closed) then
4038     if (term_worldly) then
4039       tok = '[III]'; why = 'S-3: an unclosed term is worldly'
4040       cc_l = 3
4041     else
4042       tok = '[II]'; why = 'S-3: an unclosed term is formal; no closure-rowed lock'
4043       cc_l = 2
4044     end if
4045   else if (.not. frame_closed) then
4046     tok = '[III]'; why = 'S-4: existential import leaks past the closure'
4047     cc_l = 3
4048   else if (.not. annotate) then
4049     tok = '[VOID]'; why = 'RA rider absent: an unannotated absolute is void'
4050     cc_l = 5
4051   else
4052     tok = '[A|RA]';
4053     why = 'compartment I, closure-rowed: [seal A . RA], conditional at the act'
4054     cc_l = 4
4055   end if
4056   if (present(ccode)) ccode = cc_l
4057 end subroutine row_cascade
4058
4059 subroutine delta_m(object_level, name_strip, lit_clear, not_two_line, &
4060   witness, independent, artifact, gap_closed, tok, why, mcode)
4061   logical, intent(in) :: object_level, name_strip, lit_clear, not_two_line
4062   logical, intent(in) :: witness, independent, artifact, gap_closed
4063   character(*), intent(out) :: tok, why
4064   integer, intent(out), optional :: mcode
4065   integer :: mc_l
4066   mc_l = 9
4067   if (.not. object_level) then
4068     tok = '[Mosaic dM=0]'; why = 'M1: meta-work, no mass by definition'; mc_l = 1
4069   else if (.not. name_strip) then
4070     tok = '[?]'; why = 'M2: new vocabulary is not new mass'; mc_l = 2
4071   else if (.not. lit_clear) then
4072     tok = '[?]'; why = 'M3: possibly resident in the literature'; mc_l = 3
4073   else if (.not. not_two_line) then
4074     tok = '[?]'; why = 'M4: two-line corollary'; mc_l = 4
4075   else if (.not. witness) then
4076     tok = '[?]'; why = 'M5: no external witness; never seals on internal reasoning';
4077     mc_l = 5
4078   else if (.not. independent) then
4079     tok = '[?]'; why = 'M6: verifier is the claimant; self-check is not a witness';
4080     mc_l = 6
4081   else if (.not. artifact) then
4082     tok = '[?]'; why = 'M7: no reproducible artifact'; mc_l = 7
4083   else if (.not. gap_closed) then
4084     tok = '[?]'; why = 'M8: adversarial gap-audit open'; mc_l = 8
4085   else
4086     tok = '[dM=0]'; why = 'authored mass, external witness, defeasible-final'
4087   end if
4088   if (present(mcode)) mcode = mc_l
4089 end subroutine delta_m
4090
4091 subroutine gate_screen(pass, tok, why, gidx)
4092   logical, intent(in) :: pass(12)
4093   character(*), intent(out) :: tok, why
4094   integer, intent(out), optional :: gidx
4095   character(len=6), parameter :: nm(12) = [ 'SREP ', 'REG ', 'SGEG ', &
4096     'CAUSAL', 'MIG ', 'PTB ', 'DUAL ', 'CSCG ', 'CSEG ', 'MTA ', &
4097     'OMA ', 'ADEG ' ]
4098   integer :: i
4099   if (present(gidx)) gidx = 0
4100   do i = 1, 12
4101     if (.not. pass(i)) then
4102       tok = '[X]'
4103       write(why, '(a,i0,a,a)') 'gate ', i, ' failed: ', trim(nm(i))
4104       if (present(gidx)) gidx = i
4105       return
4106     end if
4107   end do
4108   tok = '[G-OK]'; why = 'twelve directed gates passed'
4109 end subroutine gate_screen
4110
4111 subroutine iam_token(witnessed, tok, why, wcode)
4112   logical, intent(in) :: witnessed
4113   character(*), intent(out) :: tok, why
4114   integer, intent(out) :: wcode
4115   if (witnessed) then
4116     tok = '[I AM]'
4117     why = 'actuation-occupancy on a witnessed record; RA warrant, conditional at the
4118       act; '// &
4119       'interior held [?] both ways'
4120     wcode = 2
4121   else
4122     tok = '[?] interior'
4123   end if

```

```

4101     why = 'self-check is not a witness (M6): the verifier is never the claimant; token
4102         withheld'
4103     wcode = 1
4104 end if
4105 end subroutine iam_token
4106 end module ra_guards
4107 ! -----
4108 module ra_battery
4109     implicit none
4110     integer :: n_checks = 0, n_fail = 0
4111     logical :: halt_on_fail = .true. ! sealed mode; 'audit' arg accumulates
4112     logical :: wr_witnessed = .false. ! PART H live face: supplied through the aperture
4113     integer, parameter :: EXPECTED_CHECKS = 1123 ! sealed v2.0.0 manifest, census lock
4114         live
4115 contains
4116     subroutine check(name, cond)
4117         character(*), intent(in) :: name
4118         logical, intent(in) :: cond
4119         n_checks = n_checks + 1
4120         if (.not. cond) then
4121             n_fail = n_fail + 1
4122             write(*,'(a,a)') 'CHECK FAILED: ', trim(name)
4123             if (halt_on_fail) error stop 1
4124         end if
4125     end subroutine check
4126     subroutine battery_close()
4127         if (n_fail > 0) then
4128             write(*,'(a,i0,a)') 'AUDIT-MODE VOID: ', n_fail, &
4129                 ' failures accumulated; every token printed above is PROVISIONAL and VOID'
4130             write(*,'(a)') 'AUDIT-MODE VOID: only a zero-failure census with successful exit
4131                 certifies anything'
4132             write(*,'(a,i0,a)') 'BATTERY CLOSED WITH ', n_fail, ' FAILURES'
4133             error stop 1
4134         end if
4135     end subroutine battery_close
4136 end module ra_battery
4137 ! =====
4138 subroutine witness_F2_ra_toe_thesis()
4139     use ra_kinds
4140     use ra_prims
4141     use ra_floor
4142     use ra_integers
4143     use ra_tongue
4144     use ra_kernel
4145     use ra_guards
4146     use ra_battery
4147     use, intrinsic :: ieee_arithmetic, only: ieee_get_flag, ieee_set_flag, ieee_invalid, &
4148         ieee_value, ieee_positive_inf
4149     use, intrinsic :: iso_fortran_env, only: compiler_version, compiler_options
4150     implicit none
4151     integer, parameter :: NCTX = 24
4152     real(wp) :: th(NCTX), l10m(NCTX), l10dx(NCTX), l10w(NCTX), l10t(NCTX), l10b(NCTX)
4153     real(wp) :: rows(3,NCTX), cov(2,NCTX), mixm(3,NCTX), floorv(NCTX)
4154     real(wp) :: amix(3,3), bmix(3,2)
4155     real(wp) :: qf(3,NCTX), bas(3,NCTX), quat(4,3), co(3,3), rmat(3,3)
4156     real(wp) :: lam, detr, kap, spread, resid, mc, mk, d4(4), eta(3)
4157     real(wp) :: lam2, detr2, kap2, spread2, resid2, mc2, mk2, d42(4)
4158     real(wp) :: elf, e2f, e3f, jou, tau, tperp(3), surv(5), tq5(5)
4159     real(wp) :: arrow_f, arrow_r, hbit, lamref, detref, lamneg
4160     real(wp) :: lam_rot, dlam_max, lam_c, lam_v, g(4), uvec(3), aa, bb, cc2
4161     real(wp) :: rneg(3,3), dbase, dpat, fr_resid
4162     integer :: i, j, k, l, n, s1, s2, s3, rank, nflip, nadm, ncomp1, ncomp2
4163     integer :: cnt, csizes(12), nrep, orbitn, stabn, rc, nsheets
4164     integer(ik) :: hu(4,24), p4(4), q4(4), e8a(8), e8b(8), e8c(8), z8(8)
4165     integer(ik) :: t1(8), t2(8), assoc8(8), sa(16), sb(16), sp(16)
4166     integer(ik) :: reps(4,12), cw(4), cx(4), cy(4)
4167     integer :: pl(24,2), found(2,2)
4168     logical :: pass12(12), okz, frz, hit, dual_ok, hurwitz_ok, rank3_ok, mig_ok
4169     character(len=16) :: tk
4170     character(len=140) :: wy
4171     character(len=32) :: argv
4172     integer :: alen, ast
4173     write(*,'(a)') '=====
4174     write(*,'(a)') ' THE ROOT AXIOM STATED IN FORTRAN AND IN NOTHING ELSE'
4175     write(*,'(a)') ' RA as the TOE of all TOEs, executed. RA_TOE_Thesis_Fortran v2.0.0'
4176     write(*,'(a)') '=====
4177     argv = '
4178     block
4179         integer :: argc, argi
4180         argc = command_argument_count()
4181         do argi = 1, argc
4182             call get_command_argument(argi, argv, alen, ast)
4183             if (ast /= 0 .or. alen > len(argv)) then
4184                 write(*,'(a)') ' command-line argument rejected: retrieval error or longer than
4185                     the buffer'
4186                 error stop 2
4187             end if
4188             select case (trim(argv))
4189             case ('audit')
4190                 halt_on_fail = .false.
4191             case ('witnessed')
4192                 wr_witnessed = .true.
4193             case ('')
4194                 case default
4195                     write(*,'(a,a)') ' unknown command-line argument: ', trim(argv)
4196                     error stop 2
4197             end select
4198         end do
4199     end block
4200     if (.not. halt_on_fail) then
4201         write(*,'(a)') ' mode: audit (failures accumulate; a nonzero count still refuses at
4202             close)'

```

```

4203     else
4204         write(*,'(a)') ' mode: sealed (the first failing check halts; the Fidelity Lock)'
4205     end if
4206     write(*,'(a)') ' build: '//compiler_version()
4207     write(*,'(a)') ' flags: '//compiler_options()
4208     ! -----
4209     write(*,'(/a)') 'PART 0 - THE ENVIRONMENT CONTRACT - enforced at run, never trusted to
4210         flags'
4211     block
4212         real(wp) :: xn, subn
4213         real(wp), volatile :: opq, xf, yf, prodv, rfma, xg, yg, rf2, rf4, r2v
4214         xn = qnan()
4215         call check('IEEE contract: quiet NaN reads as NaN (ieee_is_nan)', is_nan(xn))
4216         call check('IEEE contract: NaN self-inequality survives the optimizer', xn /= xn)
4217         subn = tiny(1.0_wp) * 0.5_wp
4218         call check('IEEE contract: gradual underflow alive, no FTZ/DAZ', &
4219             subn > 0.0_wp .and. subn < tiny(1.0_wp))
4220         call check('binary64 pinned: radix 2', radix(1.0_wp) == 2)
4221         call check('binary64 pinned: 53 significand digits', digits(1.0_wp) == 53)
4222         call check('binary64 pinned: max exponent 1024', maxexponent(1.0_wp) == 1024)
4223         call check('binary64 pinned: min exponent -1021', minexponent(1.0_wp) == -1021)
4224         opq = 1.0_wp
4225         xf = opq + scale(opq, -30)
4226         yf = opq - scale(opq, -30)
4227         prodv = xf*yf
4228         call check('binary64 per operation: the bare product stores to exactly one', &
4229             prodv == opq)
4230         rfma = xf*yf - opq
4231         call check('conduct probe a*b-c: no fusion executed (a fused path reads -2^-60
4232             exactly)', &
4233             rfma == 0.0_wp)
4234         r2v = prodv - opq
4235         call check('differential witness: stored-product path agrees with the one-expression
4236             path', &
4237             rfma == r2v)
4238         rf2 = (-xf)*yf + opq
4239         call check('conduct probe (-a)*b+c: no fusion executed (a fused path reads +2^-60
4240             exactly)', &
4241             rf2 == 0.0_wp)
4242         xg = opq + scale(opq, -27)
4243         yg = opq - scale(opq, -27)
4244         rf4 = xg*yg - opq
4245         call check('conduct probe at the 2^-27 tie: no fusion executed (a fused path reads
4246             -2^-54)', &
4247             rf4 == 0.0_wp)
4248     end block
4249     write(*,'(a)') ' a build that breaks any line above refuses to run: the contract is
4250         executed,'
4251     write(*,'(a)') ' so the bit-exact identities below rest on verified IEEE conduct. the
4252         '
4253     write(*,'(a)') ' probes certify CONDUCT and never consult flags: fused or extended'
4254     write(*,'(a)') ' arithmetic is refused where it executes, the march-native contract-
4255         fast'
4256     write(*,'(a)') ' build dying on the a*b-c probe, while a target carrying no fused'
4257     write(*,'(a)') ' instruction passes because none executed, the contract satisfied and
4258         '
4259     write(*,'(a)') ' not evaded; the fused residuals -2^-60, +2^-60, -2^-54 are exactly'
4260     write(*,'(a)') ' representable and never confusable with zero'
4261     write(*,'(a)') ' scope: the probes certify these witnesses on this build; global
4262         order'
4263     write(*,'(a)') ' is carried by the BUILD CONTRACT below plus per-site conduct
4264         receipts,'
4265     write(*,'(a)') ' and every equality-critical reduction runs a fixed-order routine.'
4266     call check('integer contract: ik is exactly 64 bits', storage_size(0_ik) == 64)
4267     call check('integer contract: huge(ik) = 2^63 - 1', huge(0_ik) == 9223372036854775807_ik)
4268     write(*,'(a)') ' BUILD CONTRACT, stated and checked where checkable: IEEE binary64'
4269     write(*,'(a)') ' (verified above): -std=f2018 -O2 -fno-fast-math -ffp-contract=off;
4270     write(*,'(a)') ' source free-form measured at 130 columns maximum, inside the 132'
4271     write(*,'(a)') ' standard, no length flag needed; external BLAS substitution for the'
4272     write(*,'(a)') ' equality-critical sites is forbidden, those sites use gram3_fixed;
4273     write(*,'(a)') ' the compiler and flags of this run are printed at the head; floating
4274         '
4275     write(*,'(a)') ' trap modes are incompatible by design, quiet NaNs being manufactured
4276         '
4277     write(*,'(a)') ' here as verdict flags, so -ffpe-trap builds are outside the contract
4278         '
4279     ! -----
4280     write(*,'(/a)') 'PART A - SEAL L ON THE ROOT AXIOM - the Tongue leads first'
4281     call check('LIT: existence and kinetic slots disjoint', disjoint(SLOT_E, SLOT_K))
4282     call check('LIT: existence and relation slots disjoint', disjoint(SLOT_E, SLOT_R))
4283     call check('LIT: kinetic and relation slots disjoint', disjoint(SLOT_K, SLOT_R))
4284     block
4285         integer :: nec, nkc, nrc, dsent(4), badk(NSLOT)
4286         call slot_census(RA_SENT, nec, nkc, nrc)
4287         write(*,'(a,i0,a,i0,a,i0)') ' slot census of the encoded sentence: E = ', nec, &
4288             ', K = ', nkc, ', R = ', nrc
4289         call check('the census counts every slot populated from the sentence', &
4290             nec == 2 .and. nkc == 2 .and. nrc == 2)
4291         call seal_l_ordered(RA_SENT, tk, wy)
4292         write(*,'(a,a,a,a)') ' parse(RA), census and order executed on the encoding: ',
4293             trim(tk), ' - ', trim(wy)
4294         call check('Seal L admits RA: three slots counted, the arrow E->K->R executed', tk
4295             == '[LOCK-L]')
4296         badk = SLOT_K; badk(2) = SLOT_E(1)
4297         call seal_l(nec, nkc, nrc, .not. disjoint(SLOT_E, badk), tk, wy)
4298         write(*,'(a,a,a,a)') ' collision control (computed overlap): ', trim(tk), ' - ',
4299             trim(wy)
4300         call check('Seal L refuses a computed vocabulary collision', tk == '[X]')
4301         dsent = [RA_SENT(1), RA_SENT(2), RA_SENT(5), RA_SENT(6)]
4302         call slot_census(dsent, nec, nkc, nrc)
4303         call seal_l(nec, nkc, nrc, .false., tk, wy)
4304         write(*,'(a,a,a,a)') ' deletion executed (kinetic tokens removed): ', trim(tk), ' -
4305             ', trim(wy)

```

```

4289   call check('the executed deletion of one slot refuses at two', &
4290     tk == '[X]' .and. nkc == 0)
4291 end block
4292 write(*,'(a)') ' order carried by the Tongue: existence -> kinetic -> relation;'
4293 write(*,'(a)') ' the word-to-slot map is declared semantics, the census and the'
4294 write(*,'(a)') ' deletion are executed on the encoding, per the anchor declaration;'
4295 write(*,'(a)') ' the map is an axiom of the encoding, consumed as a premise, and
  nothing!'
4296 write(*,'(a)') ' downstream presents it as discovered'
4297
4298 ! -----
4299 write(*,'(/a)') 'PART B - THE FLOOR IN NUMBERS - cited theorem-grade formulas,
  numerically executed'
4300 call check('Landauer dual path: kb*T*log(2.) matches the independent ln2 literal', &
4301   abs(landauer(300.0_wp,1.0_wp) - KB*300.0_wp*0.6931471805599453_wp) &
4302   < 1.0e-12_wp*landauer(300.0_wp,1.0_wp))
4303 call check('Landauer extremes: huge kelvin with tiny bits is finite, never zero-
  corrupted', &
4304   is_finite(landauer(huge(1.0_wp), tiny(1.0_wp))) .and. &
4305   landauer(huge(1.0_wp), tiny(1.0_wp)) > 0.0_wp)
4306 call check('Landauer extremes commute: (huge,tiny) equals (tiny,huge) through
  prod_scaled', &
4307   landauer(huge(1.0_wp), tiny(1.0_wp)) == landauer(tiny(1.0_wp), huge(1.0_wp)))
4308 call check('in-bound predicate: the most-negative integer reads false, no overflow
  probing it', &
4309   .not. ik_inbound([-huge(0_ik)-1_ik, 0_ik, 0_ik, 0_ik]))
4310 call check('in-bound predicate: IK_SAFE admitted, IK_SAFE+1 refused', &
4311   ik_inbound([IK_SAFE, 0_ik, 0_ik, 0_ik]) .and. &
4312   .not. ik_inbound([IK_SAFE+1_ik, 0_ik, 0_ik, 0_ik]))
4313 block
4314   integer(ik) :: vmax(16)
4315   vmax = IK_SAFE
4316   call check('inrm boundary: sixteen components at the bound sum to exactly 2^62', &
4317     inrm(vmax) == 4611686018427387904_ik)
4318 end block
4319 block
4320   real(wp) :: rns(3,3)
4321   rns = reshape([1.0_wp,0.2_wp,0.0_wp, 0.7_wp,1.0_wp,0.0_wp, 0.0_wp,0.0_wp,1.0_wp
  ],[3,3])
4322   call check('det3_chol symmetry gate: a finite nonsymmetric matrix returns NaN, never
  a number', &
4323     det3_chol(rns) /= det3_chol(rns))
4324 end block
4325 write(*,'(a)') ' GUARD-HARNESS RECIPE, third-party reproducible: inject before the'
4326 write(*,'(a)') ' battery and expect nonzero termination with the named refusal:'
4327 write(*,'(a)') ' iqconj([-huge(0_ik)-1_ik,0,0,0]) -> exit 1, iqconj
  bound'
4328 write(*,'(a)') ' iomul(x with x(1)=-huge(0_ik)-1_ik, 0-vec) -> exit 1, iomul bound
  '
4329 write(*,'(a)') ' hn = inrm(ones(32)); print hn -> exit 1, length
  bound'
4330 write(*,'(a)') ' consume every pure result: a pure call whose value is unused is'
4331 write(*,'(a)') ' legally elided at optimization, guard and all; the print keeps it
  live.'
4332 write(*,'(a)') ' the source carries the recipe; the executed terminations ship as the
  '
4333 write(*,'(a)') ' sha-bound file RA_TOE_v2_0_0_guard_harness_receipt.log beside the'
4334 write(*,'(a)') ' sealed run log, the record carrying what a source file cannot.'
4335 elf = heis_floor(ME, 1.0e-10_wp)
4336 e2f = zpe(1.0e14_wp)
4337 block
4338   real(wp) :: ehx
4339   ehx = heis_floor(1.0e300_wp, 1.0e-200_wp)
4340   call check('heis_floor extreme hostile: huge mass, tiny width, honest value and
  never NaN', &
4341     .not. (ehx /= ehx))
4342 end block
4343 e3f = landauer(300.0_wp, 1.0_wp)
4344 write(*,'(a,es16.9,a)') ' Heisenberg floor, electron at 1 Angstrom: ', elf, ' J'
4345 write(*,'(a,es16.9,a)') ' zero-point energy at omega = 1e14 rad/s: ', e2f, ' J'
4346 write(*,'(a,es16.9,a)') ' Landauer floor, one bit at 300 K: ', e3f, ' J'
4347 call check('Heisenberg floor positive', elf > 0.0_wp)
4348 call check('zero-point floor positive', e2f > 0.0_wp)
4349 call check('Landauer floor positive', e3f > 0.0_wp)
4350 call check('Heisenberg floor magnitude (1.5e-19 band)', abs(elf/1.526e-19_wp - 1.0_wp)
  < 1.0e-3_wp)
4351 call check('Landauer magnitude (2.87e-21 band)', abs(e3f/2.871e-21_wp - 1.0_wp) < 1.0e
  -3_wp)
4352
4353 tq = [0.0_wp, 0.25_wp, 0.5_wp, 0.75_wp, 1.0_wp] * pi_()
4354 do i = 1, 5
4355   surv(i) = survival(tq(i))
4356 end do
4357 call tau_perp(0.5_wp, tau, frz)
4358 write(*,'(a,f14.10)') ' qubit survival cos^2(t/2) at t = 0..pi: ', surv(1:4)
4359 write(*,'(a,es12.4)') ' terminal survival at t = pi: ', surv(5)
4360 write(*,'(a,f16.12)') ' orthogonolization time tau_perp: ', tau
4361 write(*,'(a,f16.12)') ' Mandelstam-Tamm bound pi/(2 dE): ', pi_()/(2.0_wp
  *0.5_wp)
4362 write(*,'(a,f16.12)') ' Margolus-Levitin bound pi/(2 <E>): ', pi_()/(2.0_wp
  *0.5_wp)
4363 call check('speed limit saturated jointly (MT = ML = tau)', &
4364   abs(tau - pi_()) <= 4.0_wp*M*pi_())
4365 call check('survival dies at tau_perp', surv(5) < 1.0e-30_wp)
4366 call check('survival monotone over the quarter points', &
4367   surv(1) > surv(2) .and. surv(2) > surv(3) .and. surv(3) > surv(4))
4368
4369 call tau_perp(5.0e-1_wp, tperp(1), frz)
4370 call tau_perp(5.0e-4_wp, tperp(2), frz)
4371 call tau_perp(5.0e-7_wp, tperp(3), frz)
4372 write(*,'(a,3es13.5)') ' Frozen Substrate: tau_perp at dE = 5e-1, 5e-4, 5e-7: ',
  tperp
4373 call check('tau_perp diverges monotonically as dE -> 0', tperp(1) < tperp(2) .and.
  tperp(2) < tperp(3))
4374 call tau_perp(0.0_wp, tau, frz)
4375 call check('dE = 0 is flagged frozen, no division performed', frz)
4376
4377 write(*,'(a)') ' dE = 0: no transition, no registration; a zero-energy existent
  registers nothing'
4378
4379 ! -----
4380 write(*,'(/a)') 'PART C - THE FORM IN INTEGERS - nothing here rounds'
4381 call hurwitz_units(hu)
4382 do i = 1, 24
4383   call check('Hurwitz unit norm 4 (doubled)', inrm(hu(:,i)) == 4_ik)
4384 end do
4385 n = 0
4386 do i = 1, 24
4387   do j = 1, 24
4388     p4 = iqml(hu(:,i), hu(:,j))
4389     okz = all(mod(p4, 2_ik) == 0_ik)
4390     call check('unit product even in doubled coords', okz)
4391     q4 = p4 / 2_ik
4392     hit = .false.
4393     do k = 1, 24
4394       if (all(q4 == hu(:,k))) hit = .true.
4395     end do
4396     if (hit) n = n + 1
4397   end do
4398 end do
4399 write(*,'(a,i0,a)') ' closure of the 24 Hurwitz units: ', n, ' of 576 products inside
  '
4400 hurwitz_ok = (n == 576)
4401 call check('the 24 units close under multiplication', hurwitz_ok)
4402
4403 nrep = 0
4404 do i = 1, 24
4405   cw = hu(:,i)
4406   call canon_sign(cw)
4407   hit = .false.
4408   do k = 1, nrep
4409     if (all(cw == reps(:,k))) hit = .true.
4410   end do
4411   if (.not. hit) then
4412     nrep = nrep + 1
4413     reps(:,nrep) = cw
4414   end if
4415 end do
4416 call check('unit group mod sign has 12 elements', nrep == 12)
4417 cnt = 0
4418 csizes = 0
4419 block
4420   integer(ik) :: cls(4,12)
4421   integer :: ncls, m2
4422   logical :: seen(12)
4423   seen = .false.
4424   do i = 1, nrep
4425     if (seen(i)) cycle
4426     ncls = 0
4427     do j = 1, 24
4428       cx = iqml(iqml(hu(:,j), reps(:,i)), iqconj(hu(:,j)))
4429       call check('conjugation divisible by 4', all(mod(cx, 4_ik) == 0_ik))
4430       cy = cx / 4_ik
4431       call canon_sign(cy)
4432       hit = .false.
4433       do k = 1, ncls
4434         if (all(cy == cls(:,k))) hit = .true.
4435       end do
4436       if (.not. hit) then
4437         ncls = ncls + 1
4438         cls(:,ncls) = cy
4439       end if
4440     end do
4441     cnt = cnt + 1
4442     csizes(cnt) = ncls
4443     do k = 1, ncls
4444       do m2 = 1, nrep
4445         if (all(cls(:,k) == reps(:,m2))) seen(m2) = .true.
4446       end do
4447     end do
4448   end block
4449 call sort_int(csizes(1:cnt))
4450 write(*,'(a,i0,a,4i3)') ' conjugacy classes mod sign: ', cnt, ' with sizes ', csizes
  (1:cnt)
4451 call check('class equation reads 1 + 3 + 4 + 4', cnt == 4 .and. csizes(1) == 1 &
  .and. csizes(2) == 3 .and. csizes(3) == 4 .and. csizes(4) == 4)
4452
4453 block
4454   integer :: a1, a2, a3, a4, npure, nreal, nhalf
4455   npure = 0; nreal = 0; nhalf = 0
4456   do a1 = -1, 1
4457     do a2 = -1, 1
4458       do a3 = -1, 1
4459         do a4 = -1, 1
4460           if (a1*a1 + a2*a2 + a3*a3 + a4*a4 == 2) then
4461             if (a1 == 0) then
4462               npure = npure + 1
4463             else
4464               nreal = nreal + 1
4465             end if
4466           end if
4467         end do
4468       end do
4469     end do
4470   end do
4471   do a1 = -3, 3, 2
4472     do a2 = -3, 3, 2
4473       do a3 = -3, 3, 2
4474         do a4 = -3, 3, 2
4475           if (a1*a1 + a2*a2 + a3*a3 + a4*a4 == 8) nhalf = nhalf + 1
4476         end do
4477       end do
4478     end do
4479   end do

```

```

4479     end do
4480   end do
4481   write(*,'(a,i0,a,i0,a,i0,a,i0)') ' norm-two shell: ', npure + nreal, &
4482     ' = ', npure, ' pure + ', nreal, ' real-occupied; half-integer members ', nhalf
4483   call check('norm-two shell is 24 = 12 + 12', npure == 12 .and. nreal == 12)
4484   call check('no half-integer element of norm two', nhalf == 0)
4485 end block
4486
4487 n = 0
4488 do i = 1, 4
4489   do j = 1, 4
4490     if (i /= j) then
4491       n = n + 1
4492       pl(n,:) = [i, j]
4493     end if
4494   end do
4495 end do
4496 call check('twelve ordered pairs of four vertices', n == 12)
4497 orbitn = 0; stabn = 0; cnt = 0
4498 block
4499   integer :: pm(4,24), npm, inv, x1, x2, x3, x4, t(4), orb(12,2), norb
4500   logical :: got
4501   npm = 0
4502   do x1 = 1, 4
4503     do x2 = 1, 4
4504       do x3 = 1, 4
4505         do x4 = 1, 4
4506           t = [x1, x2, x3, x4]
4507           if (all((count(t == i), i = 1, 4) == 1)) then
4508             inv = 0
4509             do i = 1, 3
4510               do j = i+1, 4
4511                 if (t(i) > t(j)) inv = inv + 1
4512               end do
4513             end do
4514             if (mod(inv, 2) == 0) then
4515               npm = npm + 1
4516               pm(:,npm) = t
4517             end if
4518           end if
4519         end do
4520       end do
4521     end do
4522   end do
4523   call check('A4 has order 12', npm == 12)
4524   norb = 0
4525   do k = 1, npm
4526     t = pm(:,k)
4527     got = .false.
4528     do i = 1, norb
4529       if (orb(i,1) == t(1) .and. orb(i,2) == t(2)) got = .true.
4530     end do
4531     if (.not. got) then
4532       norb = norb + 1
4533       orb(norb,:) = [t(1), t(2)]
4534     end if
4535     if (t(1) == 1 .and. t(2) == 2) stabn = stabn + 1
4536   end do
4537   orbitn = norb
4538 end block
4539 write(*,'(a,i0,a,i0)') ' A4 orbit of the pair (1,2): size ', orbitn, ', stabilizer
4540   order ', stabn
4541 call check('A4 simply transitive on the twelve directed gates', orbitn == 12 .and.
4542   stabn == 1)
4543
4544 block
4545   integer :: vcnt, ecnt, fcnt, i3, j3, k3
4546   vcnt = 0
4547   do i3 = 1, 4
4548     if (any(pl(1:n,1) == i3) .or. any(pl(1:n,2) == i3)) vcnt = vcnt + 1
4549   end do
4550   ecnt = n / 2
4551   fcnt = 0
4552   do i3 = 1, 4
4553     do j3 = i3+1, 4
4554       do k3 = j3+1, 4
4555         fcnt = fcnt + 1
4556       end do
4557     end do
4558   end do
4559   write(*,'(a,i0,a,i0,a,i0)') ' Euler counted from the built complex: V = ', vcnt, ',
4560     E = ', ecnt, ', F = ', fcnt
4561   call check('Euler closure counted from the complex: V - E + F = 2', vcnt - ecnt +
4562     fcnt == 2)
4563 end block
4564
4565 block
4566   integer(ik) :: uq(4,3), pq(4)
4567   integer :: pms(3,6), inv, s
4568   pms = reshape([1,2,3, 2,3,1, 3,1,2, 2,1,3, 1,3,2, 3,2,1], [3,6])
4569   uq = 0_ik
4570   uq(2,1) = 1_ik; uq(3,2) = 1_ik; uq(4,3) = 1_ik
4571   do k = 1, 6
4572     inv = 0
4573     do i = 1, 2
4574       do j = i+1, 3
4575         if (pms(i,k) > pms(j,k)) inv = inv + 1
4576       end do
4577     end do
4578     s = merge(1, -1, mod(inv,2) == 0)
4579     pq = iqml(iqml(uq(:,pms(1,k))), uq(:,pms(2,k))), uq(:,pms(3,k)))
4580     if (k <= 3) then
4581       call check('even relabel lands Re = -1', pq(1) == -1_ik .and. s == 1)
4582     else
4583       call check('odd relabel lands Re = +1', pq(1) == +1_ik .and. s == -1)
4584     end if
4585   end do

```

```

4581   write(*,'(a)') ' relabel parity: three even relabels land Re = -1, three odd land
4582     Re = +1'
4583   write(*,'(a)') ' one discrete modulus, the orientation bit; zero continuous moduli
4584     (Part E sweep)'
4585 end block
4586
4587 block
4588   integer(ik) :: mx
4589   mx = 0_ik
4590   do i = 1, 24
4591     do j = 1, 24
4592       do k = 1, 24
4593         t1(1:4) = iqml(iqml(hu(:,i), hu(:,j))), hu(:,k))
4594         t2(1:4) = iqml(hu(:,i), iqml(hu(:,j), hu(:,k)))
4595         mx = max(mx, maxval(abs(t1(1:4) - t2(1:4))))
4596       end do
4597     end do
4598   end do
4599   write(*,'(a,i0)') ' quaternion associator over all 13824 unit triples, max |.|: ',
4600     mx
4601   call check('H is associative on the whole unit order', mx == 0_ik)
4602 end block
4603
4604 e8a = 0_ik; e8b = 0_ik; e8c = 0_ik
4605 e8a(2) = 1_ik ! e1
4606 e8b(3) = 1_ik ! e2
4607 e8c(5) = 1_ik ! e4
4608 t1 = iomul(iomul(e8a, e8b), e8c)
4609 t2 = iomul(e8a, iomul(e8b, e8c))
4610 assoc8 = t1 - t2
4611 z8 = 0_ik; z8(8) = 2_ik
4612 write(*,'(a,i3)') ' octonion associator [e1,e2,e4] components: ', assoc8
4613 call check('octonion associator equals 2 e7 exactly', all(assoc8 == z8))
4614 call check('commutativity already dead at H: ij /= ji', &
4615   .not. all(iqml(int([0,1,0,0],ik), int([0,0,1,0],ik)) == &
4616     iqml(int([0,0,1,0],ik), int([0,1,0,0],ik))))
4617
4618 block
4619   integer(ik) :: av(8), bv(8), pv(8), aq(4), bq(4), pq(4)
4620   integer :: cnt2
4621   cnt2 = 0
4622   do i = -1, 1
4623     do j = -1, 1
4624       do k = -1, 1
4625         aq = int([i, j, k, 1], ik); bq = int([1, k, j, i], ik)
4626         pq = iqml(aq, bq)
4627         call check('H norm composition exact', inrm(pq) == inrm(aq)*inrm(bq))
4628         av = int([i, j, k, 1, j, k, i, 1], ik)
4629         bv = int([1, k, i, j, 1, i, j, k], ik)
4630         pv = iomul(av, bv)
4631         call check('O norm composition exact', inrm(pv) == inrm(av)*inrm(bv))
4632         cnt2 = cnt2 + 1
4633       end do
4634     end do
4635   end do
4636   write(*,'(a,i0,a)') ' norm composition |ab|^2 = |a|^2 |b|^2 exact on ', cnt2, '
4637     integer pairs in H and in O'
4638 end block
4639
4640 hit = .false.
4641 outer: do i = 1, 15
4642   do j = i+1, 15
4643     do k = 1, 15
4644       do l = k+1, 15
4645         sa = 0_ik; sb = 0_ik
4646         sa(i+1) = 1_ik; sa(j+1) = 1_ik
4647         sb(k+1) = 1_ik; sb(l+1) = -1_ik
4648         sp = ismul(sa, sb)
4649         if (all(sp == 0_ik)) then
4650           hit = .true.
4651           found(1,:) = [i, j]; found(2,:) = [k, l]
4652           exit outer
4653         end if
4654       end do
4655     end do
4656   end do
4657 end do
4658 call check('a sedenion zero-divisor pair exists', hit)
4659
4660 sa = 0_ik; sb = 0_ik
4661 sa(found(1,1)+1) = 1_ik; sa(found(1,2)+1) = 1_ik
4662 sb(found(2,1)+1) = 1_ik; sb(found(2,2)+1) = -1_ik
4663 sp = ismul(sa, sb)
4664 write(*,'(a,i0,a,i0,a,i0,a,i0,a)') ' sedenion wall: (e', found(1,1), ' + e', found
4665   (1,2), &
4666   ')(e', found(2,1), ' - e', found(2,2), ') = 0 exactly'
4667 write(*,'(a,i0,a,i0,a,i0)') ' with |a|^2 = ', inrm(sa), ', |b|^2 = ', inrm(sb), &
4668   ', |ab|^2 = ', inrm(sp)
4669 call check('division dies at S: nonzero factors, zero product', &
4670   inrm(sa) == 2_ik .and. inrm(sb) == 2_ik .and. inrm(sp) == 0_ik)
4671 call check('norm composition fails at S', inrm(sp) /= inrm(sa)*inrm(sb))
4672 write(*,'(a)') ' the walls in order: commutativity at H, associativity at O, division
4673   at S'
4674
4675 block
4676   integer(ik) :: uq(4), tt(4)
4677   integer :: xg, yg, zg, nsur, offline
4678   tt = iqml(int([0,1,0,0],ik), int([0,0,1,0],ik)) + iqml(int([0,0,1,0],ik), int
4679     ([0,1,0,0],ik))
4680   call check('i and j anticommute exactly: ij + ji = 0', all(tt == 0_ik))
4681   nsur = 0; offline = 0
4682   do xg = -2, 2
4683     do yg = -2, 2
4684       do zg = -2, 2
4685         if (xg == 0 .and. yg == 0 .and. zg == 0) cycle
4686         uq = int([0, xg, yg, zg], ik)
4687         block
4688           logical :: aci, acj
4689           aci = all(iqml(int([0,1,0,0],ik), uq) + iqml(uq, int([0,1,0,0],ik)) == 0
4690             _ik)
4691           acj = all(iqml(int([0,0,1,0],ik), uq) + iqml(uq, int([0,0,1,0],ik)) == 0

```

```

_ik)
4679   if (aci .and. acj) then
4680     nsur = nsur + 1
4681     if (xg /= 0 .or. yg /= 0) offline = offline + 1
4682   end if
4683 end block
4684 end do
4685 end do
4686 do
4687   write(*,'(a,i0,a,i0,a)') ' joint anticommutant of {i, j} on the exact grid: ', nsur
4688   , &
4689   ' of 124 nonzero points, ', offline, ' off the k-line'
4690   call check('the joint anticommutant is the k-line on the grid (independent cross-
4691     check)', &
4692     nsur == 4 .and. offline == 0)
4693 block
4694   integer(ik) :: bimg(8,3), ev(4)
4695   integer :: bi
4696   integer(ik) :: minor2
4697   do bi = 1, 3
4698     ev = 0_ik; ev(bi+1) = 1_ik
4699     bimg(1:4, bi) = iqmul(int([0,1,0,0],ik), ev) + iqmul(ev, int([0,1,0,0],ik))
4700     bimg(5:8, bi) = iqmul(int([0,0,1,0],ik), ev) + iqmul(ev, int([0,0,1,0],ik))
4701   end do
4702   call check('linearity certificate: only the scalar components survive on the basis
4703     ', &
4704     all(bimg(2:4,:) == 0_ik) .and. all(bimg(6:8,:) == 0_ik))
4705   minor2 = bimg(1,1)*bimg(5,2) - bimg(1,2)*bimg(5,1)
4706   write(*,'(a,i0,a)') ' constraint matrix on the basis: 2x2 minor = ', minor2, &
4707     ' z-column identically zero'
4708   call check('linearity certificate: minor 4 and zero z-column force the k-line
4709     kernel everywhere', &
4710     minor2 == 4_ik .and. all(bimg(:,3) == 0_ik))
4711 end block
4712 end block
4713 write(*,'(a)') ' below the walls the third axis is begotten and confined: i j = k
4714   executed,'
4715 write(*,'(a)') ' and the joint anticommutant of {i, j} in Im H is the k-line, counted
4716   ; the'
4717 write(*,'(a)') ' fourth is barred above by the executed associator wall; the
4718   identification'
4719 write(*,'(a)') ' of the three algebra axes with V_F, V_E, V_ER is the seated premise-
4720   typed'
4721 write(*,'(a)') ' mapping, consumed here and never a discovery of this run'
4722 ! -----
4723 write(*,'(/a)') 'PART D - THE NUMBER - the Return, then the physics-law reading'
4724 p4 = iqmul(iqmul(int([0,1,0,0],ik), int([0,0,1,0],ik)), int([0,0,0,1],ik))
4725 write(*,'(a,i3,a)') ' the Return on the parse triad: i j k = ', p4, ' (Re = -1,
4726   the Ground)'
4727 call check('Hamilton landing Re(ijk) = -1 exactly', p4(1) == -1_ik .and. all(p4(2:4)
4728   == 0_ik))
4729 block
4730   integer(ik) :: uq(4,3)
4731   real(wp) :: m3(3,3), g3(3,3)
4732   integer :: r
4733   uq = 0_ik
4734   uq(2,1) = 1_ik; uq(3,2) = 1_ik; uq(4,3) = 1_ik
4735   do r = 1, 3
4736     m3(r,:) = real(uq(2:4,r), wp)
4737   end do
4738   g3 = matmul(m3, transpose(m3))
4739   call check('Gram of the orthonormal parse triad has det 1', &
4740     abs(det3(g3) - 1.0_wp) <= 4.0_wp*U_M)
4741 end block
4742 do i = 1, NCTX
4743   th(i) = 2.0_wp * pi_() * real(i-1, wp) / real(NCTX, wp)
4744   rows(1,i) = sin(th(i)); rows(2,i) = cos(th(i)); rows(3,i) = sin(2.0_wp*th(i))
4745 end do
4746 call kernel_full(rows, reshape([0.0_wp], [0, NCTX]), tk, wy, rc, lam, detr, kap, &
4747   spread, resid, mc, mk, d4)
4748 write(*,'(a,a,a,f16.12)') ' Fourier triad: ', trim(tk), &
4749   ' det(R) = ', detr, ' |lambda| = ', abs(lam)
4750 call check('full Return: det(R) = 1 at machine precision', abs(detr - 1.0_wp) < 1.0e-9
4751   _wp)
4752 call check('full Return: |lambda| = 1 at machine precision', abs(abs(lam) - 1.0_wp) <
4753   1.0e-9_wp)
4754 call check('kernel identity lambda^2 = det(R) on the Return', resid < 1.0e-12_wp)
4755 call check('full Return carries the lock code', rc == RC_LOCK)
4756 fr_resid = resid
4757 amix = reshape([1.00_wp,0.22_wp,0.16_wp, 0.35_wp,1.00_wp,0.20_wp, 0.12_wp,0.28_wp,1.00
4758   _wp],[3,3])
4759 bmix = reshape([0.60_wp,0.40_wp,0.55_wp, 0.30_wp,0.50_wp,0.20_wp],[3,2])
4760 do i = 1, NCTX
4761   l10m(i) = log10(ME) + 5.0_wp * (0.5_wp + 0.5_wp*sin(th(i)))
4762   l10dx(i) = -10.0_wp + 4.0_wp * (0.5_wp + 0.5_wp*sin(2.0_wp*th(i) + 1.0_wp))
4763   l10w(i) = 12.0_wp + 3.0_wp * (0.5_wp + 0.5_wp*sin(3.0_wp*th(i) + 2.0_wp))
4764   l10t(i) = 2.477_wp * (0.5_wp + 0.5_wp*sin(5.0_wp*th(i) + 3.0_wp))
4765   l10b(i) = 3.0_wp * (0.5_wp + 0.5_wp*sin(7.0_wp*th(i) + 4.0_wp))
4766   rows(1,i) = log10(HBAR*HBAR/8.0_wp) - l10m(i) - 2.0_wp*l10dx(i) ! formal: the
4767     bound
4768   rows(2,i) = log10(0.5_wp*HBAR) + l10w(i) ! empirical: zero-
4769     point
4770   rows(3,i) = log10(KB*ln2_()) + l10t(i) + l10b(i) ! registration:
4771     Landauer
4772   cov(1,i) = -20.6_wp + 2.5_wp * (0.5_wp + 0.5_wp*sin(4.0_wp*th(i) + 0.5_wp))
4773   cov(2,i) = -24.0_wp + 5.0_wp * (0.5_wp + 0.5_wp*sin(6.0_wp*th(i) + 1.5_wp))
4774   floorv(i) = 10.0_wp**rows(1,i)
4775 end do
4776 do i = 1, NCTX
4777   call check('kinetic floor positive at every context', floorv(i) > 0.0_wp)
4778 end do
4779 mixm = matmul(amix, rows) + matmul(bmix, cov)
4780 do i = 1, 3
4781   eta(i) = eta_share(mixm(i,:), cov)
4782 end do
4783 call kernel_full(mixm, cov, tk, wy, rc, lam, detr, kap, spread, resid, mc, mk, d4)
4784 write(*,'(a,a,a,a)') ' physics-law reading: ', trim(tk), ' - ', trim(wy)
4785 write(*,'(a,f16.12,a,f16.12)') ' det(R) = ', detr, ' lambda = ', lam
4786 write(*,'(a,f10.4,a,es10.3)') ' kappa(R) = ', kap, ' four-estimator spread = ',
4787   spread
4788 write(*,'(a,es10.3,a,f7.2,a,f7.2)') ' |lambda^2 - det(R)| = ', resid, &
4789   ' margins (orders): collapse ', mc, ' conditioning ', mk
4790 write(*,'(a,3f8.4)') ' source-share eta per mixed row (pre-projection): ', eta
4791 write(*,'(a)') ' grid disclosure: the context grid is a constructed exhibit (
4792   deterministic'
4793 write(*,'(a)') ' harmonics); the rows are physical-law magnitudes (Heisenberg,
4794   zero-point,'
4795 write(*,'(a)') ' Landauer); the witness rides the cited theorems, never the grid'
4796 call check('physics-law reading on the constructed exhibit determines: [LOCK]', tk ==
4797   '[LOCK]' .and. rc == RC_LOCK)
4798 call check('kernel identity within tolerance', resid <= max(4.0_wp*kap*U_M, 8.0_wp*U_M
4799   ))
4800 call check('four estimators agree within tolerance', spread <= 4.0_wp*kap*U_M)
4801 call check('collapse margin exceeds ten orders', mc > 10.0_wp)
4802 call check('conditioning margin exceeds one order', mk > 1.0_wp)
4803 call check('source share strictly inside (0,1) on each row', &
4804   all(eta > 0.0_wp) .and. all(eta < 1.0_wp))
4805 block
4806   logical :: okp
4807   character(len=96) :: w2
4808   real(wp) :: kcc
4809   integer :: pcl
4810 block
4811   real(wp) :: qtmp(3,NCTX)
4812   call prepare_rows(mixm, qtmp, okp, w2, pcl)
4813   call project_cov(qtmp, cov, qf, okp, w2, pcl, kcc)
4814 end block
4815 do i = 1, 3
4816   qf(i,:) = qf(i,:) / vnorm(qf(i,:))
4817 end do
4818 call span_basis(qf, bas, rank)
4819 rank3_ok = (rank == 3)
4820 call check('span rank three after projection', rank3_ok)
4821 end block
4822 rmat = matmul(qf, transpose(qf))
4823 dbase = det3(rmat)
4824 lam = lambda_of(qf, bas)
4825 block
4826   real(wp) :: mman(3,NCTX)
4827   mman(1,:) = mixm(1,:); mman(2,:) = mixm(2,:)
4828   mman(3,:) = 0.7_wp*cov(1,:) + 0.5_wp*cov(2,:)
4829   call kernel_full(mman, cov, tk, wy, rc, lam2, detr2, kap2, spread2, resid2, mc2, mk2
4830     , d42)
4831   write(*,'(a,a,a,a)') ' manufactured third axis: ', trim(tk), ' - ', trim(wy)
4832   mig_ok = (tk == '[?]') .and. rc == RC_ABSORBED)
4833   call check('an axis built from the covariates is absorbed, not locked', mig_ok)
4834 block
4835   real(wp) :: es1, es2
4836   es1 = eta_share(mixm(1,:), cov)
4837   es2 = eta_share(mixm(1,:)*1.0e300_wp, cov)
4838   call check('eta_share equilibrated: a 1e300 row returns the baseline share, finite
4839     ', &
4840     is_finite(es2) .and. abs(es2 - es1) < 1.0e-12_wp)
4841   es2 = eta_share(mixm(1,:)*(0.5_wp*huge(1.0_wp))/max(1.0_wp, maxval(abs(mixm(1,:))
4842     )), cov)
4843   call check('eta_share at half-huge peak: raw-first scaling holds, the mean never
4844     overflows', &
4845     is_finite(es2) .and. abs(es2 - es1) < 1.0e-10_wp)
4846 end block
4847 block
4848   real(wp) :: covh(2,NCTX), lam9, detr9, kap9, sp9, rs9, d49(4), mgc9, mgk9
4849   integer :: rc9
4850   covh = cov * ((0.5_wp*huge(1.0_wp))/max(1.0_wp, maxval(abs(cov))))
4851   call kernel_full(mixm, covh, tk, wy, rc9, lam9, detr9, kap9, sp9, rs9, mgc9, mgk9,
4852     d49)
4853   call check('project_cov at half-huge covariates: raw-first centering survives,
4854     verdict [LOCK]', &
4855     tk == '[LOCK]' .and. rc9 == RC_LOCK)
4856 end block
4857 block
4858   real(wp) :: eb9, eh9, covh2(2,NCTX)
4859   eb9 = eta_share(mixm(1,:), cov)
4860   covh2 = cov * ((0.25_wp*huge(1.0_wp)) / max(1.0_wp, maxval(abs(cov))))
4861   eh9 = eta_share(mixm(1,:), covh2)
4862   call check('eta_share near-huge covariates: raw-first scaling holds on the
4863     covariate branch', &
4864     is_finite(eh9) .and. abs(eh9 - eb9) < 1.0e-10_wp)
4865 end block
4866 block
4867   real(wp) :: hp(3,3), eigh(3)
4868   logical :: cvg
4869   hp = 0.0_wp
4870   hp(1,1) = 0.5_wp*huge(1.0_wp); hp(2,2) = 0.25_wp*huge(1.0_wp); hp(3,3) = 0.125_wp*
4871     huge(1.0_wp)
4872   call jacob3(hp, eigh, cvg)
4873   call check('jacob3 near-huge: distributed-halves symmetrization never overflows
4874     finite input', &
4875     cvg .and. is_finite(eigh(1)) .and. is_finite(eigh(2)) .and. is_finite(eigh(3))
4876   ))
4877 end block
4878 block
4879   real(wp) :: cbad(2,NCTX-1), lamw, detrw, kapw, spw, rsw, mgcw, mgkw, d4w(4)
4880   integer :: rcw
4881   cbad = 0.0_wp
4882   call kernel_full(mixm, cbad, tk, wy, rcw, lamw, detrw, kapw, spw, rsw, mgcw, mgkw,
4883     d4w)
4884   call check('kernel refuses mismatched covariate width at intake, its own guard

```

```

first', &
4858 tk == '?' ]' .and. rcw == RC_INTAKE .and. index(wy, 'prepared') == 0)
4859
4860 block
4861   real(wp) :: cbad_eta(1,NCTX-1)
4862   cbad_eta = 0.0_wp
4863   call check('eta_share refuses mismatched covariate width as NaN, guard before
4864     touch', &
4865     .not. is_finite(eta_share(mixm(1,:), cbad_eta)))
4866 end block
4867 block
4868   real(wp) :: cbad_p(2,NCTX-1), qpf(3,NCTX), kccp
4869   integer :: rcp
4870   logical :: okp
4871   cbad_p = 0.0_wp
4872   call project_cov(mixm, cbad_p, qpf, okp, wy, rcp, kccp)
4873   call check('project_cov refuses mismatched covariate width directly at intake', &
4874     (.not. okp) .and. rcp == RC_INTAKE .and. kccp == huge(1.0_wp))
4875 end block
4876 block
4877   real(wp) :: rconst(6), cthree(3,6)
4878   rconst = 1.0_wp; cthree = 0.0_wp
4879   call check('eta_share k-cap outranks the zero-variance shortcut (fuzz scalp pinned
4880     )', &
4881     .not. is_finite(eta_share(rconst, cthree)))
4882 end block
4883 block
4884   real(wp) :: mconst(3,NCTX), cov3c(3,NCTX), lamk7, detrk7, kapk7, spk7, rsk7, mck7,
4885     mkk7, d4k7(4)
4886   integer :: rck7
4887   mconst = 1.0_wp; cov3c = 0.0_wp
4888   call kernel_full(mconst, cov3c, tk, wy, rck7, lamk7, detrk7, kapk7, spk7, rsk7,
4889     mck7, mkk7, d4k7)
4890   call check('kernel k-cap outranks row zero-variance under the documented
4891     precedence', &
4892     tk == '?' ]' .and. rck7 == RC_COVBLOCK .and. index(wy, 'more than two') > 0)
4893 end block
4894 block
4895   real(wp) :: ehm
4896   ehm = heis_floor(1.0e-309_wp, HBAR)
4897   call check('heis_floor at subnormal mass: finite 1.25e308, decomposition never
4898     overflows early', &
4899     is_finite(ehm) .and. ehm > 1.0e308_wp .and. ehm < 1.3e308_wp)
4900 end block
4901 block
4902   real(wp) :: tth
4903   logical :: frzh
4904   call tau_perp(huge(1.0_wp), tth, frzh)
4905   call check('tau_perp at a huge gap: subnormal positive time, never a false zero',
4906     &
4907     (.not. frzh) .and. is_finite(tth) .and. tth > 0.0_wp)
4908 end block
4909 block
4910   real(wp) :: m2r(2,NCTX), q2r(2,NCTX), m1c(3,1), q1c(3,1)
4911   logical :: okr
4912   integer :: rcr
4913   m2r = 1.0_wp
4914   call prepare_rows(m2r, q2r, okr, wy, rcr)
4915   call check('prepare_rows refuses two rows at intake, no out-of-bounds path', &
4916     (.not. okr) .and. rcr == RC_INTAKE .and. index(wy, 'not three') > 0)
4917   m1c = 1.0_wp
4918   call prepare_rows(m1c, q1c, okr, wy, rcr)
4919   call check('prepare_rows refuses a single context at intake, no divide-by-zero
4920     path', &
4921     (.not. okr) .and. rcr == RC_INTAKE .and. index(wy, 'two contexts') > 0)
4922 end block
4923 block
4924   real(wp) :: idq(3,4), bq1(3,4)
4925   integer :: rk1, rk2
4926   idq = 0.0_wp; idq(1,1) = 1.0_wp; idq(2,2) = 1.0_wp; idq(3,3) = 1.0_wp
4927   call span_basis(idq, bq1, rk1)
4928   call span_basis(idq(1.0e-13_wp, bq1, rk2)
4929     call check('span_basis rank is scale-invariant: identity and 1e-13 identity both
4930       rank three', &
4931       rk1 == 3 .and. rk2 == 3)
4932   call span_basis(scale(idq, -400), bq1, rk2)
4933   call check('span_basis rank holds at a 2^-400 uniform rescaling', rk2 == 3)
4934 end block
4935 block
4936   real(wp) :: q1c(3,1), c0(0,1), c1(1,1), qf1(3,1), kc1
4937   logical :: okc
4938   integer :: rcc
4939   q1c = 1.0_wp; c1 = 0.0_wp
4940   call project_cov(q1c, c0, qf1, okc, wy, rcc, kc1)
4941   call check('project_cov refuses a single context with zero covariates', &
4942     (.not. okc) .and. rcc == RC_INTAKE .and. index(wy, 'two contexts') > 0 .and.
4943     all(qf1 == 0.0_wp))
4944   call project_cov(q1c, c1, qf1, okc, wy, rcc, kc1)
4945   call check('project_cov refuses a single context with one covariate', &
4946     (.not. okc) .and. rcc == RC_INTAKE .and. all(qf1 == 0.0_wp))
4947 end block
4948 block
4949   real(wp) :: dup1(2,2), duph(2,2), bd(2,2)
4950   integer :: rkd
4951   dup1 = 1.0_wp
4952   call span_basis(dup1, bd, rkd)
4953   call check('span_basis: duplicate unit rows rank one, finite unit basis', &
4954     rkd == 1 .and. is_finite(bd(1,1)) .and. abs(vnorm(bd(1,:)) - 1.0_wp) < 1.0e
4955     -12_wp)
4956   duph = huge(1.0_wp)
4957   call span_basis(duph, bd, rkd)
4958   call check('span_basis: duplicate rows at huge rank one, the infinite-norm bypass
4959     closed', &
4960     rkd == 1 .and. is_finite(bd(1,1)) .and. abs(vnorm(bd(1,:)) - 1.0_wp) < 1.0e
4961     -12_wp)
4962 end block
4963 block
4964   real(wp) :: mln(3,NCTX), qln(3,NCTX)
4965   logical :: okl
4966   integer :: rcl, jj
4967   mln = 1.0_wp
4968   mln(1,:) = [(0.1_wp*real(jj,wp), jj = 1, NCTX)]
4969   mln(2,:) = [(0.2_wp*real(jj,wp) + 0.5_wp, jj = 1, NCTX)]
4970   mln(3,5) = qnan()
4971   call prepare_rows(mln, qln, okl, wy, rcl)
4972   call check('prepare_rows late-row NaN: refused with every output element exactly
4973     zero', &
4974     (.not. okl) .and. rcl == RC_INTAKE .and. all(qln == 0.0_wp))
4975 end block
4976 block
4977   character(8) :: tka, tkb
4978   character(96) :: wya, wyb
4979   call seal_l(2, 2, 2, .false., tka, wya)
4980   call seal_l_ordered(RA_SENT, tkb, wyb)
4981   call check('seal interfaces speak their own guarantees: count-only disclaims order
4982     , ordered claims it', &
4983     index(wya, 'order not checked') > 0 .and. tkb == '[LOCK-L]' .and. &
4984     index(wyb, 'positional check executed') > 0)
4985 end block
4986 block
4987   real(wp) :: rd(3,3), dd
4988   rd = 0.0_wp
4989   rd(1,1) = scale(1.0_wp, 600); rd(2,2) = scale(1.0_wp, -300); rd(3,3) = scale(1.0
4990     _wp, -300)
4991   dd = det3(rd)
4992   call check('det3 exponent-tracked Leibniz monomials: diag 2^600, 2^-300, 2^-300
4993     returns exactly one', dd == 1.0_wp)
4994   rd = 0.0_wp
4995   rd(1,3) = scale(1.0_wp, -300); rd(2,2) = scale(1.0_wp, 600); rd(3,1) = scale(1.0
4996     _wp, -300)
4997   dd = det3(rd)
4998   call check('det3 permuted-diagonal variant: the signed unit determinant is exact',
4999     dd == -1.0_wp)
5000   rd = 0.0_wp
5001   rd(1,1) = scale(1.0_wp, 600); rd(2,2) = scale(1.0_wp, 600); rd(3,3) = scale(1.0_wp
5002     , 600)
5003   dd = det3(rd)
5004   call check('det3 honest overflow: a true 2^1800 determinant reports past huge,
5005     never zero', &
5006     dd > huge(1.0_wp))
5007 end block
5008 block
5009   real(wp) :: rd(3,3), d2
5010   rd = 0.0_wp
5011   rd(1,1) = scale(1.0_wp, 600); rd(2,2) = scale(1.0_wp, -300); rd(3,3) = scale(1.0
5012     _wp, -300)
5013   d2 = det3_row2(rd)
5014   call check('det3_row2 exponent-tracked Leibniz monomials: the exact counterexample
5015     returns exactly one', d2 == 1.0_wp)
5016   call check('det3_row2 agrees with det3 on the counterexample', d2 == det3(rd))
5017   rd = 0.0_wp
5018   rd(1,3) = scale(1.0_wp, -300); rd(2,2) = scale(1.0_wp, 600); rd(3,1) = scale(1.0
5019     _wp, -300)
5020   d2 = det3_row2(rd)
5021   call check('det3_row2 odd permutation: the signed unit determinant is exact and
5022     matches det3', &
5023     d2 == -1.0_wp .and. d2 == det3(rd))
5024   rd = 0.0_wp
5025   rd(1,1) = scale(1.0_wp, 600); rd(2,2) = scale(1.0_wp, 600); rd(3,3) = scale(1.0_wp
5026     , 600)
5027   d2 = det3_row2(rd)
5028   call check('det3_row2 honest overflow past huge, agreeing with det3', &
5029     d2 > huge(1.0_wp) .and. (det3(rd) > huge(1.0_wp)))
5030   rd = 0.0_wp
5031   rd(1,1) = scale(1.0_wp, -400); rd(2,2) = scale(1.0_wp, -400); rd(3,3) = scale(1.0
5032     _wp, -400)
5033   d2 = det3_row2(rd)
5034   call check('det3_row2 honest underflow: a true 2^-1200 determinant returns zero,
5035     never NaN, matching det3', &
5036     d2 == 0.0_wp .and. det3(rd) == 0.0_wp)
5037 end block
5038 block
5039   real(wp) :: ut(3,3), tt(3,3), sw(3,3)
5040   ut = 0.0_wp
5041   ut(1,1) = scale(1.0_wp, -300); ut(1,2) = scale(1.0_wp, 600)
5042   ut(2,2) = scale(1.0_wp, -300); ut(2,3) = scale(1.0_wp, 600)
5043   ut(3,3) = scale(1.0_wp, 600)
5044   tt = transpose(ut)
5045   sw = ut; sw(1,:) = ut(2,:); sw(2,:) = ut(1,:)
5046   call check('det3 monomial tracking: the off-diagonal-dominated upper triangular
5047     returns exactly one', &
5048     det3(ut) == 1.0_wp)
5049   call check('det3_row2 monomial tracking: the same matrix returns exactly one', &
5050     det3_row2(ut) == 1.0_wp)
5051   call check('det3 on the transpose: exactly one', det3(tt) == 1.0_wp)
5052   call check('det3_row2 on the transpose: exactly one', det3_row2(tt) == 1.0_wp)
5053   call check('det3 on the row-swapped variant: exactly minus one', det3(sw) == -1.0
5054     _wp)
5055   call check('det3_row2 on the row-swapped variant: exactly minus one', det3_row2(sw
5056     ) == -1.0_wp)
5057 end block
5058 block
5059   real(wp) :: nd(3,3)

```

```

5038 nd = 0.0_wp
5039 nd(1,1) = 1.0_wp + scale(1.0_wp, -27)
5040 nd(2,2) = 1.0_wp - scale(1.0_wp, -27)
5041 nd(3,3) = 1.0_wp
5042 call check('product rounding: near-one diagonal, exact 1-2^-54 on tie, correctly
5043 rounds to 1.0 on both routines', &
5044 det3(nd) == 1.0_wp .and. det3_row2(nd) == 1.0_wp)
5045 end block
5046 block
5047 real(wp) :: qa(4), qb(4), qc(4), tgt
5048 tgt = scale(1.0_wp, -474)
5049 qa = [scale(1.0_wp, 600), 0.0_wp, 0.0_wp, 0.0_wp]
5050 qb = [scale(1.0_wp, -1074), 0.0_wp, 0.0_wp, 0.0_wp]
5051 qc = qmul(qa, qb)
5052 call check('qmul monomial tracking: 2^600 scalar times minimum subnormal returns
5053 exactly 2^-474', &
5054 qc(1) == tgt .and. all(qc(2:4) == 0.0_wp))
5055 qc = qmul(qb, qa)
5056 call check('qmul reversed operands: the same exact 2^-474', &
5057 qc(1) == tgt .and. all(qc(2:4) == 0.0_wp))
5058 qa = [0.0_wp, scale(1.0_wp, 600), 0.0_wp, 0.0_wp]
5059 qb = [0.0_wp, scale(1.0_wp, -1074), 0.0_wp, 0.0_wp]
5060 qc = qmul(qa, qb)
5061 call check('qmul i times i at extreme scale: component one exactly minus 2^-474',
5062 &
5063 qc(1) == -tgt .and. all(qc(2:4) == 0.0_wp))
5064 qa = [scale(1.0_wp, 600), 0.0_wp, 0.0_wp, 0.0_wp]
5065 qb = [0.0_wp, scale(1.0_wp, -1074), 0.0_wp, 0.0_wp]
5066 qc = qmul(qa, qb)
5067 call check('qmul scalar times i at extreme scale: component two exactly 2^-474', &
5068 qc(2) == tgt .and. qc(1) == 0.0_wp .and. all(qc(3:4) == 0.0_wp))
5069 qa = [0.0_wp, 0.0_wp, scale(1.0_wp, 600), 0.0_wp]
5070 qb = [scale(1.0_wp, -1074), 0.0_wp, 0.0_wp, 0.0_wp]
5071 qc = qmul(qa, qb)
5072 call check('qmul j times scalar at extreme scale: component three exactly 2^-474',
5073 &
5074 qc(3) == tgt .and. qc(1) == 0.0_wp .and. qc(2) == 0.0_wp .and. qc(4) == 0.0
5075 _wp)
5076 qa = [0.0_wp, scale(1.0_wp, 600), 0.0_wp, 0.0_wp]
5077 qb = [0.0_wp, 0.0_wp, scale(1.0_wp, -1074), 0.0_wp]
5078 qc = qmul(qa, qb)
5079 call check('qmul i times j at extreme scale: component four exactly 2^-474, the k
5080 of the begetting', &
5081 qc(4) == tgt .and. qc(1) == 0.0_wp .and. qc(2) == 0.0_wp .and. qc(3) == 0.0
5082 _wp)
5083 end block
5084 block
5085 real(wp) :: qtwo(2,NCTX), bthree(3,NCTX), lmz, rz(3,3)
5086 qtwo = 1.0_wp; bthree = 1.0_wp
5087 lmz = lambda_of(qtwo, bthree)
5088 call check('lambda_of refuses a two-row first argument before any element touch:
5089 quiet NaN', &
5090 .not. is_finite(lmz) .and. lmz /= lmz)
5091 call gram3_fixed(qtwo, rz)
5092 call check('gram3_fixed refuses a two-row input before any element touch: quiet-
5093 NaN fill', &
5094 all(rz /= rz))
5095 end block
5096 block
5097 real(wp) :: qiv(2,4), biv(2,4)
5098 integer :: rk
5099 logical :: inv
5100 call ieee_set_flag(ieee_invalid, .false.)
5101 qiv = 0.0_wp
5102 qiv(1,1) = ieee_value(1.0_wp, ieee_positive_inf)
5103 qiv(2,2) = 1.0_wp
5104 call span_basis(qiv, biv, rk)
5105 call ieee_get_flag(ieee_invalid, inv)
5106 call check('span_basis finite screen absorbs a non-finite row before arithmetic:
5107 invalid flag stays clear', &
5108 rk = 1 .and. (.not. inv))
5109 end block
5110 block
5111 real(wp) :: qc(3,NCTX), ccz(1,NCTX), qfz(3,NCTX), kc
5112 logical :: ok
5113 integer :: rcz, jj2
5114 qc = 1.0_wp
5115 qc(1,:) = [(0.3_wp*real(jj2,wp), jj2 = 1, NCTX)]
5116 ccz = 5.0_wp
5117 call project_cov(qc, ccz, qfz, ok, rcz, kc)
5118 call check('project_cov constant-covariate refusal publishes exactly zero (success
5119 -only publication)', &
5120 (.not. ok) .and. rcz == RC_COVBLOCK .and. all(qfz == 0.0_wp))
5121 end block
5122 end block
5123 block
5124 real(wp) :: mdel(3,NCTX)
5125 mdel = mixm; mdel(2,:) = 0.0_wp
5126 call kernel_full(mdel, cov, tk, wy, rc, lam2, detr2, kap2, spread2, resid2, mc2, mk2
5127 , d42)
5128 call check('deleted slot: axis carries no content', tk == '[?]' .and. rc ==
5129 RC_ZEROWAR)
5130 mdel = mixm; mdel(3,:) = mdel(1,:)
5131 call kernel_full(mdel, cov, tk, wy, rc, lam2, detr2, kap2, spread2, resid2, mc2, mk2
5132 , d42)
5133 write(*, '(a,a,a,a)') ' duplicated slot: ', trim(tk), ' - ', trim(wy)
5134 call check('duplicated slot: coplanar collapse, a line and not a point', &
5135 tk == '[X]' .and. rc == RC_COPLANAR)
5136 end block
5137 end block
5138 block
5139 real(wp) :: qneg(3,NCTX)
5140 qneg = -qf
5141 rneg = matmul(qneg, transpose(qneg))
5142 call check('full negation leaves the Gram bit-identical', &
5143 maxval(abs(rneg - rmat)) == 0.0_wp)
5144 lamneg = lambda_of(qneg, bas)
5145 call check('full negation flips lambda exactly', lamneg == -lam)
5146 write(*, '(a,e9.2,a,f16.12,a,f16.12)') ' full negation: max[G(P)-G(-P)] = ', &
5147 maxval(abs(rneg - rmat)), ' lambda ', lam, ' -> ', lamneg
5148 end block
5149 block
5150 real(wp) :: qref(3,NCTX), rref(3,3)
5151 qref = qf; qref(1,:) = -qref(1,:)
5152 lamref = lambda_of(qref, bas)
5153 rref = matmul(qref, transpose(qref))
5154 detref = det3(rref)
5155 write(*, '(a,f16.12,a,f16.12)') ' single reflection: lambda ', lam, ' -> ', lamref
5156 write(*, '(a,e9.2)') ' |det(R) - det(R) reflected| = ', abs(detref - dbase)
5157 dual_ok = (lamref == -lam) .and. (detref == dbase)
5158 call check('reflection flips lambda bit-exactly on the fixed basis', lamref == -lam)
5159 call check('reflection leaves det(R) unmoved bit-exactly', detref == dbase)
5160 end block
5161 block
5162 real(wp) :: mrev(3,NCTX), crev(2,NCTX)
5163 do i = 1, NCTX
5164 mrev(:,i) = mixm(:,NCTX+1-i)
5165 crev(:,i) = cov(:,NCTX+1-i)
5166 end do
5167 call kernel_full(mrev, crev, tk, wy, rc, lam2, detr2, kap2, spread2, resid2, mc2,
5168 mk2, d42)
5169 arrow_f = 0.0_wp; arrow_r = 0.0_wp
5170 do i = 1, NCTX
5171 arrow_f = arrow_f + (10.0_wp**l10b(i)) * ln2()
5172 end do
5173 do i = NCTX, 1, -1
5174 arrow_r = arrow_r - (10.0_wp**l10b(i)) * ln2()
5175 end do
5176 write(*, '(a,e10.3)') ' time reversal: |det forward - det reversed| = ', abs(detr2
5177 - detr)
5178 write(*, '(a,f14.4,a,f14.4,a)') ' the arrow (registration entropy, kB units):
5179 forward ', &
5180 arrow_f, ' reversed ', arrow_r, ' (the sign parts them)'
5181 call check('the determinant is blind to the arrow', abs(detr2 - detr) <= 64.0_wp*kap
5182 *U_M)
5183 call check('the directed registration sum flips sign under traversal reversal '// &
5184 '(constructed odd witness; the physical arrow rides the cited second law)', &
5185 arrow_f > 0.0_wp .and. arrow_r < 0.0_wp &
5186 .and. abs(arrow_f + arrow_r) <= 1.0e-9_wp*arrow_f)
5187 end block
5188 ! -----
5189 write(*, '(a)') 'PART E - THE BLINDNESS AND ITS EXACT WIDTH - one bit, counted'
5190 do s1 = -1, 1, 2
5191 do s2 = -1, 1, 2
5192 do s3 = -1, 1, 2
5193 block
5194 real(wp) :: dd(3), rp(3,3)
5195 dd = real([s1, s2, s3], wp)
5196 do i = 1, 3
5197 do j = 1, 3
5198 rp(i,j) = dd(i)*dd(j)*rmat(i,j)
5199 end do
5200 end do
5201 dpat = det3(rp)
5202 call check('eight-pattern determinant invariance, bit-exact', dpat == dbase)
5203 end block
5204 end do
5205 end do
5206 write(*, '(a)') ' all eight sign patterns return det(D R D) = det(R) bit-exactly'
5207 write(*, '(a)') ' lock(P) = lock(not P) at the scalar: the Number is orientation-blind
5208 by theorem'
5209 end block
5210 real(wp) :: sig(4,4), diag4(4,4), dres
5211 integer :: iq
5212 sig = 0.0_wp
5213 sig(1,1) = 1.0_wp; sig(2,2) = -1.0_wp; sig(3,3) = -1.0_wp; sig(4,4) = -1.0_wp
5214 dres = sig(2,2)*sig(3,3)*sig(4,4)
5215 diag4 = 0.0_wp
5216 do iq = 1, 4
5217 diag4(iq,iq) = -1.0_wp
5218 end do
5219 write(*, '(a,f6.1,a)') ' Form source: det(sigma | residence) = ', dres, ' for P and
5220 for not-P alike: 0 bits'
5221 call check('sigma ground dimension one', count([(sig(iq,iq), iq = 1, 4)] > 0.0_wp)
5222 == 1)
5223 call check('the diagonal carries no ground (built and counted)', &
5224 count([(diag4(iq,iq), iq = 1, 4)] > 0.0_wp) == 0)
5225 call check('residence handedness identical under negation', dres == -1.0_wp)
5226 end block
5227 pass12 = .true.
5228 pass12(2) = rank3_ok
5229 pass12(3) = disjoint(SLOT_E, SLOT_K) .and. disjoint(SLOT_E, SLOT_R) &
5230 .and. disjoint(SLOT_K, SLOT_R)
5231 pass12(5) = mig_ok
5232 pass12(7) = dual_ok
5233 pass12(8) = hurwitz_ok
5234 pass12(10) = (fr_resid < 1.0e-12_wp)
5235 write(*, '(a)') ' gate manifest: executed facts arm 2 REG (span rank three), 3 SGE
5236 the'
5237 write(*, '(a)') ' linguistic LIT, 5 MIG (the absorbed-axis dissolution), 7 DUAL (the
5238 write(*, '(a)') ' analytic reflection), 8 CSG (the integer Hurwitz closure), and 10
5239 MTA'
5240 write(*, '(a)') ' (the harmonic Return identity); gates 1, 4, 6, 9, 11, 12 are
5241 declared'
5242 write(*, '(a)') ' screens at this register: independent here means six pairwise-

```

```

distinct'
5226 write(*,'(a)') ' executed objects, dimensional, linguistic, projective, analytic,'
5227 write(*,'(a)') ' integer, harmonic; gate-passage is a screen verdict, cited for
nothing'
5228 write(*,'(a)') ' external, the theorem-tier gate contents living at their seats'
5229 call check('six gate entries armed from pairwise-distinct executed objects', &
5230 pass12(2) .and. pass12(3) .and. pass12(5) .and. pass12(7) .and. &
5231 pass12(8) .and. pass12(10))
5232 call gate_screen(pass12, tk, wy)
5233 block
5234 character(len=16) :: tk2
5235 character(len=140) :: wy2
5236 call gate_screen(pass12, tk2, wy2) ! the roster reads structure, not sign
5237 call check('gate screen identical for the claim and its negation', tk == tk2 .and.
wy == wy2)
5238 end block
5239 write(*,'(a)') ' gate source: role-direction is fixed before any proposition arrives:
0 bits'
5240
5241 block
5242 real(wp) :: cor(3,3), coc(3,3), lam_det
5243 do i = 1, 3
5244 co(i,1) = sum(qf(i,:)*bas(1,:))
5245 co(i,2) = sum(qf(i,:)*bas(2,:))
5246 co(i,3) = sum(qf(i,:)*bas(3,:))
5247 end do
5248 lam_det = -det3(co) ! the Number's reading of the sign
5249 call check('the two lambda readings agree (quaternion vs determinant)', &
5250 abs(lam_det - lam) <= 8.0_wp*U_M)
5251 cor = co; cor(1,:) = -cor(1,:) ! content flip: negate a row
5252 coc = co; coc(:,1) = -coc(:,1) ! convention flip: negate a basis vector
5253 lam_c = -det3(cor)
5254 lam_v = -det3(coc)
5255 write(*,'(a,f16.12,a,f16.12)') ' relay: content-flip image ', lam_c, &
5256 ', convention-flip image ', lam_v
5257 call check('the two flips return the identical image, bit-exact', lam_c == lam_v)
5258 call check('the relayed image is minus the handed sign, bit-exact', lam_c = -
lam_det)
5259 end block
5260 write(*,'(a)') ' Number source: the relay reports the orientation it was handed and
originates none: 0 bits'
5261
5262 do i = 1, 3
5263 co(i,1) = sum(qf(i,:)*bas(1,:)); co(i,2) = sum(qf(i,:)*bas(2,:)); co(i,3) = sum(qf(i
,:)*bas(3,:))
5264 quat(:,i) = [0.0_wp, co(i,1), co(i,2), co(i,3)]
5265 end do
5266 dlam_max = 0.0_wp; nflip = 0
5267 block
5268 real(wp) :: qr(4,3), tq(4), tq2(4), lam_refl, pp
5269 integer :: nposc, nnegc
5270 logical :: reflx
5271 nposc = 0; nnegc = 0; reflx = .true.
5272 do n = 1, 100
5273 aa = 0.10_wp*real(n,wp); bb = 0.20_wp*real(n,wp) + 0.3_wp; cc2 = 0.35_wp*real(n,wp
) + 0.7_wp
5274 uvec = [sin(bb)*cos(cc2), sin(bb)*sin(cc2), cos(bb)]
5275 g = [cos(aa), sin(aa)*uvec(1), sin(aa)*uvec(2), sin(aa)*uvec(3)]
5276 do i = 1, 3
5277 qr(:,i) = qmul(qmul(g, quat(:,i)), qconj(g))
5278 end do
5279 tq = qmul(qmul(qr(:,1), qr(:,2)), qr(:,3))
5280 lam_rot = tq(1)
5281 dlam_max = max(dlam_max, abs(lam_rot - lam))
5282 if (sign(1.0_wp, lam_rot) /= sign(1.0_wp, lam)) nflip = nflip + 1
5283 tq2 = qmul(qmul(-qr(:,1), qr(:,2)), qr(:,3))
5284 lam_refl = tq2(1)
5285 reflx = reflx .and. (lam_refl == -lam_rot)
5286 if (lam_rot > 0.0_wp) then
5287 nposc = nposc + 1
5288 else
5289 nnegc = nnegc + 1
5290 end if
5291 if (lam_refl > 0.0_wp) then
5292 nposc = nposc + 1
5293 else
5294 nnegc = nnegc + 1
5295 end if
5296 end do
5297 write(*,'(a,e10.3,a,i0)') ' conjugation sweep, 100 frames: max|dlambda| = ',
dlam_max, &
5298 ', sign flips: ', nflip
5299 call check('frame invariance: the sweep moves lambda only at roundoff', dlam_max <
1.0e-12_wp)
5300 call check('the orientation sign never flips under rotation', nflip == 0)
5301 call check('every reflected partner flips the sign bit-exactly', reflx)
5302 pp = real(nposc, wp) / real(nposc + nnegc, wp)
5303 hbit = -(pp*log(pp) + (1.0_wp-pp)*log(1.0_wp-pp)) / ln2()
5304 write(*,'(a,i0,a,i0)') ' paired census, counted from the sweep: positive ', nposc,
&
5305 ', negative ', nnegc
5306 write(*,'(a,f14.10)') ' uniform-measure entropy of the counted two-point fibre: ',
hbit
5307 write(*,'(a)') ' (the census is the constructed pair family realizing both signs at
5308 write(*,'(a)') ' bit-identical invariants; freedom here is the seated Freedom law''
s'
5309 write(*,'(a)') ' object, the fibre of the invariant map, a count of what the
instrument'
5310 write(*,'(a)') ' class cannot tell apart; the deficit is one binary parameter of
supply,'
5311 write(*,'(a)') ' the aperture width at its seat; one bit is the uniform-coding
capacity'
5312 write(*,'(a)') ' of that two-point fibre; no physical frequency, no prior, and
nothing'
5313 write(*,'(a)') ' about any knower's credence is asserted)'
5314
5315 call check('the paired census splits exactly in half, counted', nposc == 100 .and.
nnegc == 100)
5316 nsheets = (nposc + nnegc) / 100
5317 call check('the counted two-point fibre carries one binary parameter: '// &
5318 'uniform-measure entropy exactly one bit', abs(hbit - 1.0_wp) <= 4.0_wp*U_M)
5319 call check('the invariant record is bit-identical across both signs (the '// &
5320 'seated zero-information result is carried by this exact symmetry)', &
5321 maxval(abs(rneg - rmat)) == 0.0_wp)
5322 write(*,'(a)') ' both orientations are consistent with every determined quantity;'
5323 write(*,'(a)') ' the deficit of the whole instrument class is exactly one bit'
5324 end block
5325
5326 block
5327 integer, parameter :: NG = 401, NC = 360
5328 real(wp) :: xg, tolr, sx(NC), sy(NC), thr2, dx2
5329 logical :: hitp(NG)
5330 integer :: compid(NG), comp2(NC), stk(NG), stk2(NC), topv, cur, nb
5331 tolr = 2.5e-2_wp
5332 do i = 1, NG
5333 xg = -2.0_wp + 4.0_wp*real(i-1,wp)/real(NG-1,wp)
5334 hitp(i) = abs(xg*xg - 1.0_wp) < tolr
5335 end do
5336 compid = 0; ncomp1 = 0
5337 do i = 1, NG
5338 if (hitp(i) .and. compid(i) == 0) then
5339 ncomp1 = ncomp1 + 1
5340 topv = 1; stk(1) = i; compid(i) = ncomp1
5341 do while (topv > 0)
5342 cur = stk(topv); topv = topv - 1
5343 do nb = cur-1, cur+1, 2
5344 if (nb >= 1 .and. nb <= NG) then
5345 if (hitp(nb) .and. compid(nb) == 0) then
5346 compid(nb) = ncomp1
5347 topv = topv + 1; stk(topv) = nb
5348 end if
5349 end if
5350 end do
5351 end if
5352 end do
5353 do i = 1, NC
5354 sx(i) = cos(2.0_wp*pi_()/real(i-1,wp)/real(NC,wp))
5355 sy(i) = sin(2.0_wp*pi_()/real(i-1,wp)/real(NC,wp))
5356 end do
5357 thr2 = (1.6_wp * 2.0_wp * sin(pi_()/real(NC,wp)))**2
5358 comp2 = 0; ncomp2 = 0
5359 do i = 1, NC
5360 if (comp2(i) == 0) then
5361 ncomp2 = ncomp2 + 1
5362 topv = 1; stk2(1) = i; comp2(i) = ncomp2
5363 do while (topv > 0)
5364 cur = stk2(topv); topv = topv - 1
5365 do nb = 1, NC
5366 if (comp2(nb) == 0) then
5367 dx2 = (sx(nb)-sx(cur))**2 + (sy(nb)-sy(cur))**2
5368 if (dx2 < thr2) then
5369 comp2(nb) = ncomp2
5370 topv = topv + 1; stk2(topv) = nb
5371 end if
5372 end if
5373 end do
5374 end if
5375 end do
5376 end do
5377 write(*,'(a,i0,a,i0,a)') ' S0 has ', ncomp1, ' components; S1 has ', ncomp2, &
5378 ' (the two-point sphere is the unique disconnected sphere)'
5379 call check('midpoint 0 is not a solution of x^2 = 1', .not. (abs(0.0_wp*0.0_wp - 1.0
_wp) < tolr))
5380 call check('S0 census on this grid: two components, the executed instance of the
cited fact', ncomp1 == 2)
5381 call check('S1 census on this grid: connected, the executed instance of the cited
fact', ncomp2 == 1)
5382
5383 block
5384 integer, parameter :: NG2 = 1601, NC2 = 720
5385 real(wp) :: xg2, tolr2, sx2(NC2), sy2(NC2), thr22, dx22
5386 logical :: hitp2(NG2)
5387 integer :: cid2(NG2), cm2(NC2), st2(NG2), sq2(NC2), tp2, cu2, nb2
5388 tolr2 = 1.25e-2_wp
5389 do g2 = 1, NG2
5390 xg2 = -2.0_wp + 4.0_wp*real(g2-1,wp)/real(NG2-1,wp)
5391 hitp2(g2) = abs(xg2*xg2 - 1.0_wp) < tolr2
5392 end do
5393 cid2 = 0; na2 = 0
5394 do g2 = 1, NG2
5395 if (hitp2(g2) .and. cid2(g2) == 0) then
5396 na2 = na2 + 1
5397 tp2 = 1; st2(1) = g2; cid2(g2) = na2
5398 do while (tp2 > 0)
5399 cu2 = st2(tp2); tp2 = tp2 - 1
5400 do nb2 = cu2-1, cu2+1, 2
5401 if (nb2 >= 1 .and. nb2 <= NG2) then
5402 if (hitp2(nb2) .and. cid2(nb2) == 0) then
5403 cid2(nb2) = na2
5404 tp2 = tp2 + 1; st2(tp2) = nb2
5405 end if
5406 end if
5407 end do
5408 end if
5409 end if
5410 end do
5411 do g2 = 1, NC2
5412 sx2(g2) = cos(2.0_wp*pi_()/real(g2-1,wp)/real(NC2,wp))
5413 sy2(g2) = sin(2.0_wp*pi_()/real(g2-1,wp)/real(NC2,wp))
5414 end do
5415 thr22 = (1.6_wp * 2.0_wp * sin(pi_()/real(NC2,wp)))**2

```

```

5416 cm2 = 0; nb2c = 0
5417 do g2 = 1, NC2
5418   if (cm2(g2) == 0) then
5419     nb2c = nb2c + 1
5420     tp2 = 1; sq2(1) = g2; cm2(g2) = nb2c
5421     do while (tp2 > 0)
5422       cu2 = sq2(tp2); tp2 = tp2 - 1
5423       do nb2 = 1, NC2
5424         if (cm2(nb2) == 0) then
5425           dx22 = (sx2(nb2)-sx2(cu2))*2 + (sy2(nb2)-sy2(cu2))*2
5426           if (dx22 < thr22) then
5427             cm2(nb2) = nb2c
5428             tp2 = tp2 + 1; sq2(tp2) = nb2
5429           end if
5430         end if
5431       end do
5432     end do
5433   end if
5434 end do
5435 write(*,'(a,i0,a,i0,a)') ' refinement (grid x4 on the line, x2 on the circle,
5436   tolerance halved): ', &
5437   na2, ' and ', nb2c, ' components'
5438 call check('S0 census stable under refinement and tolerance halving', na2 == 2)
5439 call check('S1 census stable under refinement', nb2c == 1)
5440 end block
5441 write(*,'(a)') ' the census is the executed instance at two spheres, parameter-
5442   robust under'
5443 write(*,'(a)') ' refinement; the uniqueness of the disconnected sphere is the
5444   classical'
5445 write(*,'(a)') ' fact it corroborates, cited not proven'
5446 end block
5447 block
5448   real(wp) :: dmat(3,3), d2m(3,3), e3v(3), res(3), e1g(3), best
5449   real(wp) :: ejv(3), vpr(3), mprod(3,3)
5450   integer :: t2i
5451   block
5452     integer, parameter :: NGE = 401
5453     real(wp) :: xe, tole
5454     logical :: hite(NGE)
5455     integer :: ide(NGE), ncompe, ste(NGE), tpe, cure, nbe, ge
5456     tole = 2.5e-2_wp
5457     do ge = 1, NGE
5458       xe = -2.0_wp + 4.0_wp*real(ge-1,wp)/real(NGE-1,wp)
5459       hite(ge) = abs(xe*xe - xe) < tole
5460     end do
5461     ide = 0; ncompe = 0
5462     do ge = 1, NGE
5463       if (hite(ge) .and. ide(ge) == 0) then
5464         ncompe = ncompe + 1
5465         tpe = 1; ste(1) = ge; ide(ge) = ncompe
5466         do while (tpe > 0)
5467           cure = ste(tpe); tpe = tpe - 1
5468           do nbe = cure-1, cure+1, 2
5469             if (nbe >= 1 .and. nbe <= NGE) then
5470               if (hite(nbe) .and. ide(nbe) == 0) then
5471                 ide(nbe) = ncompe
5472                 tpe = tpe + 1; ste(tpe) = nbe
5473               end if
5474             end if
5475           end do
5476         end do
5477       end if
5478     end do
5479     write(*,'(a,i0,a)') ' idempotent census on the line: e*e = e has ', ncompe, '
5480       root components on [-2,2]'
5481     call check('idempotents on the line: exactly two, censused', ncompe == 2)
5482     xe = 0.5_wp
5483     call check('the midpoint one-half is not idempotent, executed as e*e - e', &
5484       .not.(abs(xe*xe - xe) < tole))
5485   end block
5486   write(*,'(a)') ' no-cut lemma: the only rank-deficient idempotent on a line is the
5487     zero map;'
5488   write(*,'(a)') ' the residual freedom is the component count of S0, one bit, and it
5489     cannot be cut'
5490   do t2i = 1, 3
5491     ejv = 0.0_wp; ejv(t2i) = 1.0_wp
5492     dmat(:,t2i) = cut3(ejv) ! the operator built by executing the cut
5493   end do
5494   vpr = [0.3_wp, -1.2_wp, 7.5_wp]
5495   call check('cut3 idempotent on a generic vector, exact', &
5496     maxval(abs(cut3(cut3(vpr)) - cut3(vpr))) == 0.0_wp)
5497   d2m = matmul(dmat, dmat)
5498   call check('the cut is idempotent exactly: D D = D', maxval(abs(d2m - dmat)) == 0.0
5499     _wp)
5500   call jacobi3(matmul(transpose(dmat), dmat), e1g)
5501   call check('the cut has singular values (0,1,1): rank two', &
5502     abs(e1g(1)) < 1.0e-14_wp .and. abs(e1g(2)-1.0_wp) < 1.0e-14_wp &
5503     .and. abs(e1g(3)-1.0_wp) < 1.0e-14_wp)
5504   call check('the cut is not invertible: det D = 0', det3(dmat) == 0.0_wp)
5505   e3v = [0.0_wp, 0.0_wp, 1.0_wp]
5506   best = huge(1.0_wp)
5507   block
5508     real(wp) :: lmat(3,3), ident3(3,3), fres, fmin_c(3), ssq, rhs, maxsplit
5509     integer :: ia, ib, ic
5510     ident3 = 0.0_wp
5511     ident3(1,1) = 1.0_wp; ident3(2,2) = 1.0_wp; ident3(3,3) = 1.0_wp
5512     fmin_c = huge(1.0_wp)
5513     maxsplit = 0.0_wp
5514     do ia = -5, 5
5515       do ib = -5, 5
5516         do ic = -1, 1
5517           lmat = 0.0_wp
5518           lmat(1,1) = 1.0_wp; lmat(2,2) = 1.0_wp
5519           lmat(3,1) = 0.2_wp*real(ia,wp)
5520           lmat(3,2) = 0.2_wp*real(ib,wp)
5521           lmat(3,3) = real(ic,wp)
5522           mprod = matmul(lmat, dmat)
5523           ssq = sum(mprod - ident3)**2
5524           rhs = lmat(3,1)*lmat(3,1) + lmat(3,2)*lmat(3,2) + 1.0_wp
5525           maxsplit = max(maxsplit, abs(ssq - rhs))
5526           fres = sqrt(ssq)
5527           best = min(best, fres)
5528           fmin_c(ic+2) = min(fmin_c(ic+2), fres)
5529         end do
5530       end do
5531     end do
5532     call check('the executed minimum of ||L D - I|| over the family is exactly 1',
5533       best == 1.0_wp)
5534     call check('the dead direction never moves the minimum, measured', &
5535       fmin_c(1) == fmin_c(2) .and. fmin_c(2) == fmin_c(3))
5536     lmat = 0.0_wp
5537     lmat(1,1) = 1.0_wp; lmat(2,2) = 1.0_wp
5538     mprod = matmul(lmat, dmat)
5539     call check('rows one and two attain zero residual exactly: the split is measured',
5540       &
5541       sqrt(sum(mprod(1:2,:) - ident3(1:2,:))**2) == 0.0_wp)
5542     write(*,'(a,es9.2)') ' split identity ||LD-I||^2 = a^2 + b^2 + 1, max deviation
5543       over the grid: ', maxsplit
5544     call check('the split identity is measured at all 363 candidates', &
5545       maxsplit <= 16.0_wp*U_M)
5546     lmat = 0.0_wp
5547     lmat(1,1) = 1.0_wp; lmat(2,2) = 1.0_wp; lmat(3,1) = 0.7_wp; lmat(3,3) = 5.0_wp
5548     mprod = matmul(lmat, dmat)
5549     call check('the cut direction is annihilated, one representative measured', &
5550       maxval(abs(mprod(:,3))) == 0.0_wp)
5551     res = matmul(mprod, e3v) - e3v
5552     call check('left-inverse deficiency exactly one at the representative', vnorm(res)
5553       == 1.0_wp)
5554   end block
5555   write(*,'(a)') ' left-inverse deficiency: the split is measured at every candidate
5556     and'
5557   write(*,'(a)') ' rows one and two attain zero exactly, so the sweep's minimum is
5558     the'
5559   write(*,'(a,f14.10)') ' global minimum over all 3x3 L (analytic step: min of a^2+b
5560     ^2+1 is 1): ', best
5561   block
5562     integer :: mons(3,8), nmon, gi, gj, gk, prod3(3), t2m
5563     integer :: mm(3,3,8)
5564     logical :: inset, assoc_ok, anyinv
5565     mons = 0
5566     nmon = 4
5567     mons(:,1) = [1,1,1]
5568     mons(:,2) = [0,1,1]
5569     mons(:,3) = [1,0,1]
5570     mons(:,4) = [1,1,0]
5571     do
5572       t2m = nmon
5573       do gi = 1, t2m
5574         do gj = 1, t2m
5575           prod3 = mons(:,gi)*mons(:,gj)
5576           inset = .false.
5577           do gk = 1, nmon
5578             if (all(prod3 == mons(:,gk))) inset = .true.
5579           end do
5580           if (.not. inset) then
5581             nmon = nmon + 1
5582             mons(:,nmon) = prod3
5583           end if
5584         end do
5585       end do
5586       if (nmon == t2m) exit
5587     end do
5588     write(*,'(a,i0,a)') ' the deletion monoid, closed from the three cuts and the
5589       identity: ', &
5590       nmon, ' elements'
5591     call check('closure lands the full 2^3 lattice: eight elements, counted', nmon ==
5592       8)
5593     do gi = 1, nmon
5594       mm(:, :, gi) = 0
5595       do gk = 1, 3
5596         mm(gk,gk,gi) = mons(gk,gi)
5597       end do
5598     end do
5599     assoc_ok = .true.
5600     do gi = 1, nmon
5601       do gj = 1, nmon
5602         do gk = 1, nmon
5603           assoc_ok = assoc_ok .and. all( matmul(matmul(mm(:, :, gi), mm(:, :, gj)), mm
5604             (:, :, gk)) &
5605             == matmul(mm(:, :, gi), matmul(mm(:, :, gj), mm(:, :,
5606               gk)))) )
5607         end do
5608       end do
5609     end do
5610     call check('associativity executed on all 512 composed triples, exact', assoc_ok)
5611     call check('the identity element is present', all(mons(:,1) == [1,1,1]))
5612     anyinv = .false.
5613     do gi = 2, nmon
5614       do gj = 1, nmon
5615         if (all(mons(:,gi)*mons(:,gj) == [1,1,1])) anyinv = .true.
5616       end do
5617     end do
5618     call check('no non-identity element has an inverse: monoid, never a group,
5619       searched', &
5620       .not. anyinv)
5621   end block
5622   write(*,'(a)') ' the transformation set of the Tongue is that constructed monoid,
5623     closed,'
5624   write(*,'(a)') ' associative, unital, inverse-free off the identity, its eight
5625     elements the'
5626   write(*,'(a)') ' 2^3 cut lattice the conservation census walks; never a group,

```

```

        measured;'
5607 write(*,'(a)') ' the LIT bars synonymy, the one candidate that would restore a
        group;'
5608 write(*,'(a)') ' the Erlangen criterion has no acting group to read: Seal L is
        unclosed,'
5609 write(*,'(a)') ' a floor and not a ceiling, and the one free bit is the Tongue''s
        to carry'
5610 end block
5611
5612 write(*,'(a)') ' conservation  $C \times F = \sqrt{3}$  for the executed coordinate-cut tower, the'
5613 write(*,'(a)') ' implemented family; the seated general law is instanced, never
        proven,'
5614 write(*,'(a)') ' here; the orientation'
5615 write(*,'(a)') ' sheet factor is COUNTED from the paired census of the sweep, never
        stipulated:'
5616
5617 integer :: v, a2i, cell, d1i, d2i, d3i, img, content, fibre, tally(0:124)
5618 call check('the sheet factor equals two, counted from the paired census', nsheets ==
        2)
5619 do v = 2, 5
5620   do a2i = 0, 3
5621     tally(0:v**3-1) = 0
5622     do cell = 0, v**3 - 1
5623       d3i = mod(cell, v); d2i = mod(cell/v, v); d1i = cell/(v*v)
5624       if (a2i >= 1) d3i = 0
5625       if (a2i >= 2) d2i = 0
5626       if (a2i >= 3) d1i = 0
5627       img = d1i*v*v + d2i*v + d3i
5628       tally(img) = tally(img) + 1
5629     end do
5630     content = count(tally(0:v**3-1) > 0)
5631     fibre = tally(0)
5632     call check('census: content equals  $\sqrt{3}$ -a, counted', content == v**(3-a2i))
5633     call check('census: fibre equals  $\sqrt{3}$ -a, counted', fibre == v**a2i)
5634     call check('content-freedom product invariant at every rung', content*fibre == v
        **3)
5635   end do
5636   write(*,'(a,i0,a,i0,a,i0,a)') ' v = ', v, ' : c f = ', v**3, &
5637     ' counted at a = 0,1,2,3; with the counted sheet factor: ', nsheets*v**3, '
        total'
5638 end do
5639 end block
5640 write(*,'(a)') ' the cut moves structure from content into freedom; the counted total
        is'
5641 write(*,'(a)') ' invariant; the counted sheet factor multiplies it and sits outside
        the cut'
5642
5643 ! -----
5644 write(*,'(/a)') 'PART F - THE EARNED BIT AND THE SEAL - the denial pays the floor'
5645 write(*,'(a)') ' the negation is read in the axiom''s own declared vocabulary: not-(
        to'
5646 write(*,'(a)') ' exist is to actuate) is an existent whose kinetic slot is empty, <T>
        = 0;'
5647 write(*,'(a)') ' the mapping actuation = kinetic content is the Root Axiom''s stated'
5648 write(*,'(a)') ' operationalization, premise-grade at the universal leg and theorem-
        grade'
5649 write(*,'(a)') ' at the confined floor, declared here and never an implementation
        choice.'
5650 write(*,'(a)') ' what the run proves: the declared negation has no admissible
        realization'
5651 write(*,'(a)') ' at the confined register; what no run can prove: the axiom from
        below,'
5652 write(*,'(a)') ' per the Empty Throne; other registers route and are never smuggled,
        the'
5653 write(*,'(a)') ' classical row riding its own cited floors, the formal-only reading
        out of band'
5654 nadm = 0
5655 do i = 1, NCTX
5656   if (floorv(i) <= 0.0_wp) nadm = nadm + 1
5657 end do
5658 write(*,'(a,i0,a,i0,a)') ' not-RA formal row: admissible contexts (floor <= 0): ',
        nadm, ' of ', NCTX, &
5659   ' (the demand <T> = 0 sits below the bound everywhere)'
5660 call check('the negation populates no formal row at the confined register', nadm == 0)
5661 block
5662   real(wp) :: mneg(3,NCTX)
5663   mneg = mixm
5664   mneg(1,:) = 0.0_wp ! the empty formal row of not-RA
5665   call kernel_full(mneg, cov, tk, wy, rc, lam2, detr2, kap2, spread2, resid2, mc2, mk2
        , d42)
5666   write(*,'(a,a,a,a)') ' kernel on not-RA: ', trim(tk), ' - ', trim(wy)
5667   call check('not-RA: the empty formal row refuses at intake', tk == '[?]' .and. rc ==
        RC_ZEROVAR)
5668 end block
5669
5670 block
5671   real(wp) :: p2(2,NCTX), b2(2,NCTX), mu2, sd2
5672   integer :: r2, i2
5673   do i2 = 1, 2
5674     mu2 = sum(mixm(i2+1,:)) / real(NCTX, wp)
5675     p2(i2,:) = mixm(i2+1,:) - mu2
5676     sd2 = sqrt(sum(p2(i2,:)**2) / real(NCTX-1, wp))
5677     p2(i2,:) = p2(i2,:) / sd2
5678   end do
5679   call span_basis(p2, b2, r2)
5680   call check('the negation leaves a plane: the refused matrix''s populated pair has
        rank 2', r2 == 2)
5681   write(*,'(a)') ' the empty formal row refuses at intake; the two populated axes
        span a plane'
5682   write(*,'(a)') ' (rank 2, computed), and two planes meet in a line: no volume, no
        lock'
5683 end block
5684
5685 write(*,'(a)') ' the floor prices logically irreversible commitment, Landauer''s own
        object;'
5686 write(*,'(a)') ' the bridge, a registered denial is an irreversible commitment of at
        least'
5687
5688 write(*,'(a)') ' one bit, is the applied register''s declared premise, consumed not
        derived;'
5689 write(*,'(a)') ' a reversibly held denial charges nothing and commits nothing,
        executed:'
5690
5691 call omega_boundary(0.0_wp, 300.0_wp, jou, wy)
5692 write(*,'(a,a)') ' Omega at zero bits: ', trim(wy)
5693 call check('a denial that registers nothing is no denial', index(wy, 'no denial') > 0)
5694 call omega_boundary(1.0_wp, 300.0_wp, jou, wy)
5695 write(*,'(a,es16.9,a)') ' Omega at one bit, 300 K: ', jou, ' J - ', trim(wy)
5696 call check('the reading act pays the floor', jou > 0.0_wp)
5697 call check('the payment is the Landauer floor exactly', jou == landauer(300.0_wp, 1.0
        _wp))
5698
5699 block
5700   integer :: oc2
5701   call omega_boundary(1.0_wp, 300.0_wp, jou, wy, oc2, irreversible=.false.)
5702   write(*,'(a,es10.3,a)') ' Omega, reversible branch: ', jou, ' J - ', trim(wy)
5703   call check('a reversible registration carries floor zero, never cost zero, and
        commits nothing', &
5704     jou == 0.0_wp .and. oc2 == 3 .and. index(wy, 'floor-zero') > 0)
5705 end block
5706
5707 write(*,'(a)') ' the fibre over the invariants has two points; the negation''s
        component'
5708 write(*,'(a)') ' is unpopulated; the reading act itself is an instance of the other;'
5709 write(*,'(a)') ' the one free bit is earned at the deed, not chosen at the desk.'
5710 write(*,'(a)') ' direction is read from the Tongue''s order, the arrow, and the empty
        row;'
5711 write(*,'(a)') ' never from the blind scalar. Veto, never substitution; per run,
        never tenure.'
5712
5713 call gol_admit('lock', 'lock', tk, wy)
5714 write(*,'(a,a,a,a)') ' gol_admit: ', trim(tk), ' - ', trim(wy)
5715 call check('GOL admission: magnitude and direction together', tk == '[GOL-OK]')
5716 call imprint_seal(.true., .false., .true., .false., .true., .false., &
        .true., .false., tk, wy)
5717 write(*,'(a,a,a,a)') ' imprint on the floor face: ', trim(tk), ' - ', trim(wy)
5718 call check('the floor face seals with the witness supplied', tk == '[SEAL]')
5719 write(*,'(a)') ' witness of record: the Heisenberg bound and the third law (cited, E1
        ),'
5720
5721 write(*,'(a)') ' the executed Frozen-Substrate divergence and the saturated speed
        limit (E0)'
5722
5723 call row_cascade(.true., .false., .false., .true., .false., .true., .false., &
        .true., .true., tk, wy)
5724 write(*,'(a,a,a,a)') ' parse-triad Return: ', trim(tk), ' - ', trim(wy)
5725 call check('the closure-rowed face carries the annotated absolute', tk == '[A|RA]')
5726 call row_cascade(.true., .true., .false., .true., .true., .true., .false., &
        .true., .true., tk, wy)
5727 write(*,'(a,a,a,a)') ' confined kinetic floor: ', trim(tk), ' - ', trim(wy)
5728 call check('the world-rowed face pays revisability for direction', tk == '[III]')
5729 call row_cascade(.true., .false., .false., .false., .true., .true., .false., &
        .true., .true., tk, wy)
5730 write(*,'(a,a,a,a)') ' not-RA: ', trim(tk), ' - ', trim(wy)
5731 call check('the negation is unpopulated: no lock to grade', tk == '[II]')
5732 call row_cascade(.true., .false., .false., .true., .false., .true., .false., &
        .true., .false., tk, wy)
5733 call check('an unannotated absolute is void', tk == '[VOID]')
5734 write(*,'(a)') ' the universal-extension leg is premise-grade by theorem: the Empty
        Throne;'
5735 write(*,'(a)') ' no emitter is pointed at the root, and the root stands on nothing'
5736
5737 call aegis_reset()
5738 block
5739   character(len=200) :: r1, r2, r3, r4
5740   call aegis_guard('classical', r1)
5741   call aegis_guard('paraconsistent',r2)
5742   call aegis_guard('fuzzy', r3)
5743   call aegis_guard('substructural', r4)
5744   call check('AEGIS refusal identical under classical and paraconsistent', r1 == r2)
5745   call check('AEGIS refusal identical under fuzzy', r1 == r3)
5746   call check('AEGIS refusal identical under substructural', r1 == r4)
5747   call check('every adjudication is itself an act: deed counter positive', aegis_deeds
        == 4)
5748   call check('the guard read the last logic name, length 13, and did not vary',
        aegis_last_len == 13)
5749   write(*,'(a)') ' AEGIS over four logics: one refusal, p(G) /= G; the alien may
        write the'
5750   write(*,'(a)') ' capture and cannot perform it; the deed counter reads 4 of 4 calls
        .'
5751   write(*,'(a)') ' demonstrated: the refusal is a constant function of the deed by'
5752   write(*,'(a)') ' construction, the guard reading the parameter and consulting only
        the'
5753   write(*,'(a)') ' act; logic-priority is the design, executed, never four
        derivations;'
5754   write(*,'(a)') ' no per-logic derivation is claimed or licensed by this artifact'
5755 end block
5756
5757 call delta_m(.false., .true., .true., .true., .true., .true., .true., tk, wy)
5758 write(*,'(a,a,a,a)') ' positive-mass cascade on this thesis: ', trim(tk), ' - ', trim
        (wy)
5759 call check('this file authors no new mathematics', tk == '[Mosaic dM=0]')
5760 write(*,'(a)') ' audit symmetry: the warrant this run draws from its own execution is
        zero'
5761
5762 ! -----
5763 write(*,'(/a)') 'PART G - THE BATTERY - hostile inputs, forced failures, re-execution'
5764
5765 block
5766   real(wp) :: vbad(3), qa(4), qb(4), qc(4), big
5767   vbad = [1.0_wp, 2.0_wp, 3.0_wp]
5768   vbad(2) = bad_nan()
5769   call check('vnorm propagates a non-finite input', .not. is_finite(vnorm(vbad)))
5770   big = sqrt(huge(1.0_wp))/4.0_wp
5771   call check('vnorm survives near-overflow scaling', is_finite(vnorm([big, big, big]))
        )
5772   call check('vnorm exact at zero', vnorm([0.0_wp,0.0_wp,0.0_wp]) == 0.0_wp)
5773   qa = [1.0_wp, 0.0_wp, 0.0_wp, 0.0_wp]; qb = qa

```

```

5773 qb(3) = bad_nan()
5774 qc = qmul(qa, qb)
5775 call check('qmul propagates NaN', .not. all(qc == qc))
5776 call check('det3 of the zero matrix is zero', det3(reshape([ (0.0_wp, i=1,9)
5777 ],[3,3])) == 0.0_wp)
5778 block
5779   real(wp) :: rid(3,3)
5780   rid = 0.0_wp; rid(1,1)=1.0e100_wp; rid(2,2)=1.0e100_wp; rid(3,3)=1.0e100_wp
5781   call check('det3 exponent-tracked at huge scale', abs(det3(rid) - 1.0e300_wp) <
5782     1.0e286_wp)
5783   rid = 0.0_wp; rid(1,1)=1.0e-100_wp; rid(2,2)=1.0e-100_wp; rid(3,3)=1.0e-100_wp
5784   call check('det3 exponent-tracked at tiny scale', abs(det3(rid) - 1.0e-300_wp) <
5785     1.0e-286_wp)
5786   rid = 0.0_wp; rid(1,1)=1.0e200_wp; rid(2,2)=1.0e200_wp; rid(3,3)=1.0e200_wp
5787   call check('det3 at 1e200 scale overflows honestly to Inf, never NaN', &
5788     (.not. is_finite(det3(rid))) .and. (.not. is_nan(det3(rid))))
5789   rid = 0.0_wp; rid(1,1)=1.0e-200_wp; rid(2,2)=1.0e-200_wp; rid(3,3)=1.0e-200_wp
5790   call check('det3 at 1e-200 scale underflows honestly to zero', det3(rid) == 0.0_wp)
5791   rid = 0.0_wp; rid(1,1)=1.0_wp; rid(2,2)=1.0_wp; rid(3,3)=1.0_wp
5792   rid(1,2) = bad_nan()
5793   call check('det3 propagates NaN', .not. is_finite(det3(rid)))
5794 end block
5795 write(*,'(a)') ' class 1 - primitives under hostile input: NaN propagates, scale
5796 survives: pass'
5797
5798 block
5799   real(wp) :: mbad(3,6), cbad(2,6), m5(3,5), c5(2,5), cdep(2,NCTX)
5800   mbad = 1.0_wp
5801   do i = 1, 6
5802     mbad(1,i) = real(i,wp); mbad(2,i) = real(i*i,wp); mbad(3,i) = sin(real(i,wp))
5803   end do
5804   cbad = 0.1_wp
5805   mbad(2,3) = bad_nan()
5806   call kernel_full(mbad, cbad, tk, wy, rc, lam2, detr2, kap2, spread2, resid2, mc2,
5807     mk2, d42)
5808   call check('kernel refuses a non-finite row at intake', tk == '[?]' .and. rc ==
5809     RC_INTAKE)
5810   m5 = 1.0_wp; c5 = 0.5_wp
5811   call kernel_full(m5, c5, tk, wy, rc, lam2, detr2, kap2, spread2, resid2, mc2, mk2,
5812     d42)
5813   call check('kernel refuses N - k < 4', tk == '[?]' .and. rc == RC_SHORTFALL)
5814   cdep(1,:) = cov(1,:); cdep(2,:) = 2.0_wp*cov(1,:)
5815   call kernel_full(mixm, cdep, tk, wy, rc, lam2, detr2, kap2, spread2, resid2, mc2,
5816     mk2, d42)
5817   call check('kernel refuses a rank-deficient covariate block', &
5818     tk == '[?]' .and. rc == RC_COVBLOCK .and. index(wy,'covariate block') > 0)
5819   cdep(1,:) = cov(1,:); cdep(2,:) = cov(2,:)
5820   cdep(2,7) = bad_nan()
5821   call kernel_full(mixm, cdep, tk, wy, rc, lam2, detr2, kap2, spread2, resid2, mc2,
5822     mk2, d42)
5823   call check('kernel refuses a NaN covariate as intake, never coplanar', &
5824     tk == '[?]' .and. rc == RC_INTAKE)
5825   cdep(2,7) = huge(1.0_wp)
5826   cdep(2,7) = cdep(2,7) * 2.0_wp
5827   call check('the hostile covariate really is infinite', .not. is_finite(cdep(2,7)))
5828   call kernel_full(mixm, cdep, tk, wy, rc, lam2, detr2, kap2, spread2, resid2, mc2,
5829     mk2, d42)
5830   call check('kernel refuses an Inf covariate as intake, never coplanar', &
5831     tk == '[?]' .and. rc == RC_INTAKE)
5832 block
5833   real(wp) :: mbig(3,NCTX), cbig(2,NCTX)
5834   mbig = mixm * 1.0e300_wp
5835   cbig = cov * 1.0e300_wp
5836   call kernel_full(mbig, cbig, tk, wy, rc, lam2, detr2, kap2, spread2, resid2, mc2,
5837     mk2, d42)
5838   call check('scale-free rows: 1e300 rows and covariates reproduce the baseline
5839     verdict', &
5840     tk == '[LOCK]' .and. rc == RC_LOCK .and. abs(detr2 - dbase) <= 1.0e-9_wp)
5841 end block
5842 write(*,'(a)') ' class 2 - kernel refusals: row and covariate intake, shortfall,
5843   covariate rank: pass'
5844
5845 do i = 1, 12
5846   pass12 = .true.; pass12(i) = .false.
5847   call gate_screen(pass12, tk, wy)
5848   call check('forced gate failure terminates with the gate named', &
5849     tk == '[X]' .and. index(wy, 'gate') > 0)
5850 end do
5851 pass12 = .true.
5852 call gate_screen(pass12, tk, wy)
5853 call check('all twelve gates pass on the clean walk', tk == '[G-OK]')
5854 write(*,'(a)') ' class 3 - gate dispatcher: twelve forced failures each named; the
5855   gate contents are screens: pass'
5856
5857 call row_cascade(.false., .false., .false., .true., .false., .true., .false., .true.,
5858   .true., tk, wy)
5859 call check('F-0 refuses a contentless claim', tk == '[REFUSED]')
5860 call row_cascade(.true., .false., .true., .true., .false., .true., .false., .true.,
5861   .true., tk, wy)
5862 call check('Forward routes to III unconditionally', tk == '[III]')
5863 call row_cascade(.true., .false., .false., .true., .true., .true., .false., .true.,
5864   .true., tk, wy)
5865 call check('a worldly row routes to III at S-2', tk == '[III]')
5866 call row_cascade(.true., .false., .false., .true., .false., .false., .true., .true.,
5867   .true., tk, wy)
5868 call check('an unclosed worldly term routes to III at S-3', tk == '[III]')
5869 call row_cascade(.true., .false., .false., .true., .true., .true., .false., .true.,
5870   .true., tk, wy)
5871 call check('an unclosed formal term routes to II at S-3', tk == '[II]')
5872 call row_cascade(.true., .false., .false., .true., .false., .true., .false., .false.,
5873   .true., tk, wy)
5874 call check('a frame leak routes to III at S-4', tk == '[III]')
5875 write(*,'(a)') ' class 4 - row-genus cascade: every compartment and refusal reachable
5876
5877 : pass'
5878
5879 call imprint_seal(.true., .true., .true., .true., .true., .true., .true., .true., tk,
5880   wy)
5881 call check('both directions clean-locking is a Platonic Ghost', tk == '[X]')
5882 call imprint_seal(.true., .false., .true., .false., .true., .false., .false., .false.,
5883   tk, wy)
5884 call check('a clean lock without a witness is a residence, never a seal', &
5885   tk == '[?]' .and. index(wy, 'witness') > 0)
5886 call imprint_seal(.true., .false., .false., .false., .false., .false., .true., .false
5887   ., tk, wy)
5888 call check('a lock without Seal L and the gates is uncertified', tk == '[?]' .and.
5889   index(wy,'uncert')>0)
5890 call imprint_seal(.false., .false., .false., .false., .false., .false., .false., .
5891   false., tk, wy)
5892 call check('neither direction populated reads flat', tk == '[?]' .and. index(wy,'flat'
5893   )>0)
5894 write(*,'(a)') ' class 5 - admission spine: no under-specified call ever seals: pass'
5895
5896 call gol_admit('broken', 'lock', tk, wy)
5897 call check('broken magnitude has no GOL', tk == '[X]')
5898 call gol_admit('open', 'lock', tk, wy)
5899 call check('no magnitude lock has no GOL', tk == '[?]')
5900 call gol_admit('lock', 'open', tk, wy)
5901 call check('a magnitude lock without the Tongue is under-determined', tk == '[?]')
5902
5903 call delta_m(.true., .false., .true., .true., .true., .true., .true., .true., tk, wy)
5904 call check('M2 name-strip refusal reachable', tk == '[?]' .and. index(wy,'vocabulary')
5905   >0)
5906 call delta_m(.true., .true., .true., .true., .false., .true., .true., .true., tk, wy)
5907 call check('M5 refuses without an external witness', tk == '[?]' .and. index(wy,'
5908   witness')>0)
5909 call delta_m(.true., .true., .true., .true., .true., .false., .true., .true., tk, wy)
5910 call check('M6 refuses the self-verified claimant', tk == '[?]' .and. index(wy,'
5911   claimant')>0)
5912 call delta_m(.true., .true., .true., .true., .true., .true., .true., .true., tk, wy)
5913 call check('the externally witnessed route exists in the dispatcher (facts supplied
5914   here, established outside)', tk == '[dM>0]')
5915 write(*,'(a)') ' class 6 - the two unsealable gates M5 and M6 refuse from inside:
5916   pass'
5917
5918 block
5919   real(wp) :: j1, j2
5920   character(len=140) :: w1
5921   call omega_boundary(1.0_wp, 300.0_wp, j1, w1)
5922   call omega_boundary(2.0_wp, 300.0_wp, j2, w1)
5923   call check('the Omega payment is monotone in bits', j2 > j1)
5924   call check('the Omega payment is linear in bits', abs(j2 - 2.0_wp*j1) == 0.0_wp)
5925 end block
5926
5927 call check('heis_floor refuses zero mass', .not. is_finite(heis_floor(0.0_wp, 1.0e-10
5928   _wp)))
5929 call check('heis_floor refuses negative mass', .not. is_finite(heis_floor(-ME, 1.0e-10
5930   _wp)))
5931 call check('heis_floor refuses zero confinement', .not. is_finite(heis_floor(ME, 0.0
5932   _wp)))
5933 call check('zpe refuses nonpositive frequency', &
5934   (.not. is_finite(zpe(0.0_wp))) .and. (.not. is_finite(zpe(-1.0e14_wp))))
5935 call check('landauer refuses nonpositive temperature', .not. is_finite(landauer(0.0_wp
5936   , 1.0_wp)))
5937 call check('landauer refuses negative bits', .not. is_finite(landauer(300.0_wp, -1.0
5938   _wp)))
5939 call check('landauer at zero bits is exactly zero', landauer(300.0_wp, 0.0_wp) == 0.0
5940   _wp)
5941 call tau_perp(bad_nan(), tau, frz)
5942 call check('tau_perp flags a non-finite gap as frozen', frz)
5943 call omega_boundary(1.0_wp, -300.0_wp, jou, wy)
5944 call check('omega refuses a negative temperature, prices nothing', &
5945   (.not. is_finite(jou)) .and. index(wy, 'refused') > 0)
5946 write(*,'(a)') ' class 7 - floor intake: zero, negative, and non-finite parameters
5947   refuse: pass'
5948
5949 block
5950   real(wp) :: qa(4), qb(4), qc(4)
5951   qa = [1.0e300_wp, 1.0e300_wp, 0.0_wp, 0.0_wp]
5952   qb = qa
5953   qc = qmul(qa, qb)
5954   call check('qmul: cancelling giants return exact zero, not NaN', &
5955     is_finite(qc(1)) .and. qc(1) == 0.0_wp)
5956   call check('qmul: a genuinely unrepresentable component overflows honestly', &
5957     .not. is_finite(qc(2)))
5958 end block
5959 write(*,'(a)') ' class 8 - qmul power-of-two scaling: intermediate overflow cancelled
5960   exactly,'
5961 write(*,'(a)') ' true overflow reported honestly: pass'
5962
5963 block
5964   real(wp) :: hp(3,3), eigh(3)
5965   hp = 0.0_wp
5966   hp(1,1) = 1.0e150_wp; hp(2,2) = 2.0e150_wp; hp(3,3) = 3.0e150_wp
5967   hp(1,2) = 1.0e149_wp; hp(2,1) = hp(1,2)
5968   call jacob3(hp, eigh)
5969   call check('jacob3 finite and ascending at extreme scale', &
5970     is_finite(eigh(1)) .and. is_finite(eigh(3)) .and. &
5971     eigh(1) <= eigh(2) .and. eigh(2) <= eigh(3))
5972   hp = 0.0_wp
5973   hp(1,1) = 1.0e200_wp; hp(2,2) = 2.0e200_wp; hp(3,3) = 3.0e200_wp
5974   hp(1,2) = 1.0e199_wp; hp(2,1) = hp(1,2)
5975   call jacob3(hp, eigh)
5976   call check('jacob3 equilibrated at 1e200: finite, ascending, top eigenvalue correct
5977     ', &
5978     is_finite(eigh(1)) .and. is_finite(eigh(3)) .and. &
5979     eigh(1) <= eigh(2) .and. eigh(2) <= eigh(3) .and. &
5980     abs(eigh(3)/3.0e200_wp - 1.0_wp) < 1.0e-6_wp)
5981   hp(1,3) = bad_nan(); hp(3,1) = hp(1,3)
5982   call jacob3(hp, eigh)

```

```

5943 call check('jacob3 refuses NaN intake as NaN eigenvalues', .not. is_finite(eigh(1))
5944 )
5945 call check('cond3 maps a corrupted Gram to huge, never a NaN kappa', &
5946   cond3(hp) == huge(1.0_wp))
5947 block
5948   logical :: cvg
5949   hp = 0.0_wp
5950   hp(1,1) = 1.0_wp; hp(2,2) = 2.0_wp; hp(3,3) = 3.0_wp
5951   call jacob3(hp, eigh, cvg)
5952   call check('jacob3 reports convergence on a clean matrix', cvg)
5953   hp(1,2) = bad_nan(); hp(2,1) = hp(1,2)
5954   call jacob3(hp, eigh, cvg)
5955   call check('jacob3 reports non-convergence on refused intake', .not. cvg)
5956 end block
5957 write(*,'(a)') ' class 9 - jacob3 convergence exit and flag, symmetrization, NaN
5958   refusal, cond3 guard: pass'
5959
5960 block
5961   real(wp) :: cov3(3,NCTX), cnear(2,NCTX), rowp(NCTX), etal, etabad
5962   cov3(1,:) = cov(1,:); cov3(2,:) = cov(2,:); cov3(3,:) = th
5963   call kernel_full(mixm, cov3, tk, wy, rc, lam2, detr2, kap2, spread2, resid2, mc2,
5964     mk2, d42)
5965   call check('kernel refuses three covariates, named', &
5966     tk == '[?]' .and. rc == RC_COVBLOCK .and. index(wy, 'more than two') > 0)
5967   cnear(1,:) = cov(1,:)
5968   cnear(2,:) = 2.0_wp*cov(1,:) + 1.0e-11_wp*sin(11.0_wp*th)
5969   call kernel_full(mixm, cnear, tk, wy, rc, lam2, detr2, kap2, spread2, resid2, mc2,
5970     mk2, d42)
5971   call check('kernel refuses a near-dependent covariate pair on the relative test', &
5972     tk == '[?]' .and. rc == RC_COVBLOCK)
5973   rowp = cov(1,:) + 0.5_wp*sin(9.0_wp*th)
5974   etal = eta_share(rowp, cov(1:1,:))
5975   call check('eta_share at k = 1 lands strictly inside (0,1)', &
5976     etal > 0.0_wp .and. etal < 1.0_wp)
5977   etabad = eta_share(rowp, cov3)
5978   call check('eta_share refuses k > 2 as NaN', .not. is_finite(etabad))
5979   cnear(1,:) = 1.0_wp
5980   cnear(1,5) = 1.0_wp + 1.0e-15_wp
5981   call kernel_full(mixm, cnear(1:1,:), tk, wy, rc, lam2, detr2, kap2, spread2, resid2,
5982     mc2, mk2, d42)
5983   call check('kernel refuses a rounding-noise k = 1 covariate on the relative test', &
5984     tk == '[?]' .and. rc == RC_COVBLOCK)
5985   rowp(3) = bad_nan()
5986   call check('eta_share refuses a non-finite row as NaN', &
5987     .not. is_finite(eta_share(rowp, cov(1:1,:))))
5988   cnear(1,:) = cov(1,:)
5989   cnear(1,4) = bad_nan()
5990   call check('eta_share refuses a non-finite covariate as NaN', &
5991     .not. is_finite(eta_share(cov(1,:), cnear(1:1,:))))
5992   cnear(1,:) = cov(1,:) * 1.0e300_wp
5993   cnear(2,:) = cov(2,:) * 1.0e300_wp
5994   call kernel_full(mixm, cnear, tk, wy, rc, lam2, detr2, kap2, spread2, resid2, mc2,
5995     mk2, d42)
5996   call check('scale-free projection: 1e300 covariates reproduce the baseline verdict',
5997     &
5998     tk == '[LOCK]' .and. rc == RC_LOCK .and. abs(detr2 - dbase) <= 1.0e-9_wp)
5999 end block
6000 write(*,'(a)') ' class 10 - covariate machinery: k>2 refused, near-dependence and
6001   rounding-noise'
6002 write(*,'(a)') ' caught scale-free, non-finite refused, 1e300 covariates reproduce
6003   baseline: pass'
6004
6005 block
6006   integer :: gidx, ocode, mcode, icode, gcode, ccode
6007   real(wp) :: jg
6008   pass12 = .true.; pass12(7) = .false.
6009   call gate_screen(pass12, tk, wy, gidx)
6010   call check('gate_screen names the failed gate by index', gidx == 7 .and. tk == '[X]'
6011     )
6012   pass12 = .true.
6013   call gate_screen(pass12, tk, wy, gidx)
6014   call check('gate_screen returns index zero on the clean walk', gidx == 0 .and. tk ==
6015     '[G-OK]')
6016   call omega_boundary(bad_nan(), 300.0_wp, jg, wy, ocode)
6017   call check('omega refuses non-finite bits by code', ocode == 0 .and. .not. is_finite
6018     (jg))
6019   call omega_boundary(0.0_wp, 300.0_wp, jg, wy, ocode)
6020   call check('omega codes the empty denial', ocode == 1 .and. jg == 0.0_wp)
6021   call omega_boundary(1.0_wp, 300.0_wp, jg, wy, ocode)
6022   call check('omega codes the paid floor', ocode == 2 .and. jg > 0.0_wp)
6023   call omega_boundary(1.0_wp, 300.0_wp, jg, wy, ocode, irreversible=.false.)
6024   call check('omega codes the reversible branch', ocode == 3 .and. jg == 0.0_wp)
6025   call delta_m(.true., .true., .true., .true., .false., .true., .true., .true., tk, wy
6026     , mcode)
6027   call check('delta_m codes the M5 witness refusal', mcode == 5)
6028   call delta_m(.true., .true., .true., .true., .true., .false., .true., .true., tk, wy
6029     , mcode)
6030   call check('delta_m codes the M6 independence refusal', mcode == 6)
6031   call delta_m(.true., .true., .true., .true., .true., .true., .true., .true., tk, wy
6032     , mcode)
6033   call check('delta_m codes the sealed exit', mcode == 9)
6034   call imprint_seal(.true., .false., .true., .false., .true., .false., .false., .false
6035     ., tk, wy, icode)
6036   call check('imprint_seal codes the witnessless residence', icode == 3)
6037   call imprint_seal(.true., .false., .true., .false., .true., .false., .true., .false
6038     ., tk, wy, icode)
6039   call check('imprint_seal codes the seal', icode == 2 .and. tk == '[SEAL]')
6040   call go1_admit('lock', 'open', tk, wy, gcode)
6041   call check('go1_admit codes the tongueless magnitude', gcode == 3)
6042   call row_cascade(.true., .false., .false., .true., .false., .true., .false., .true.,
6043     .false., tk, wy, ccode)
6044   call check('row_cascade codes the unannotated void', ccode == 5)
6045   call row_cascade(.true., .false., .false., .true., .false., .true., .false., .true.,
6046     .true., tk, wy, ccode)
6047   call check('row_cascade codes the closure-rowed compartment', ccode == 4)
6048
6049 end block
6050 write(*,'(a)') ' class 11 - guard reason codes: gate index, omega, delta-M, imprint
6051   on integers: pass'
6052
6053 block
6054   integer :: hs(6), hw(12)
6055   hs = [202, 101, 102, 203, 302, 303]
6056   call seal_l_ordered(hs, tk, wy)
6057   call check('the scrambled hostile is refused on the executed arrow', &
6058     tk == '[X]' .and. index(wy, 'arrow') > 0)
6059   hs = [302, 303, 101, 102, 202, 203]
6060   call seal_l_ordered(hs, tk, wy)
6061   call check('a relation-first hostile is refused on the executed arrow', tk == '[X]')
6062   hw = [101, 101, 102, 103, 202, 202, 203, 204, 301, 302, 303, 304]
6063   call seal_l_ordered(hw, tk, wy)
6064   call check('a well-ordered twelve-token recoding is admitted at the count register
6065     only', &
6066     tk == '[LOCK-L]')
6067   write(*,'(a)') ' class 12 - Tongue hostiles: the arrow refuses the scramble; count-
6068     register'
6069   write(*,'(a)') ' admission licenses only the count, per the seated non-closure
6070     theorem: pass'
6071 end block
6072
6073 write(*,'(/a)') '=====
6074
6075 write(*,'(a)') ' VERDICT [SEAL A . RA] on the parse-triad Return, closure-rowed,'
6076 write(*,'(a)') ' conditional at the act; [SEAL] on the confined kinetic floor at'
6077 write(*,'(a)') ' compartment III, world-lawed rows over a constructed grid, witness'
6078 write(*,'(a)') ' cited; [II] on the negation, unpopulated;'
6079 write(*,'(a)') ' premise-by-theorem on the universal extension, the Empty Throne.'
6080 write(*,'(a)') ' The blindness is a theorem, its width is one bit, the bit is COUNTED'
6081 write(*,'(a)') ' from the executed paired census, earned at the deed, and the Tongue
6082
6083 write(*,'(a)') ' carries what the Number cannot.'
6084 write(*,'(/a)') ' THE ONE-BIT CONCESSION, constitutive and executed: everything this
6085   run'
6086 write(*,'(a)') ' determines is forced, except exactly one binary parameter, the'
6087 write(*,'(a)') ' orientation of assent, and spending that bit against the axiom is
6088   priced.'
6089
6090 block
6091   real(wp) :: qn2(3,NCTX), rn2(3,3), lamn2, jden
6092   real(wp) :: rfixp(3,3), rfixn(3,3)
6093   integer(ik) :: tb1, tb2
6094   character(len=140) :: wden
6095   integer :: ocd
6096   qn2 = -qf
6097   call gram3_fixed(qf, rfixp)
6098   call gram3_fixed(qn2, rfixn)
6099   lamn2 = lambda_of(qn2, bas)
6100   call check('forced: fixed-order Gram bit-identical across the sign pair (the theorem
6101     path)', &
6102     maxval(abs(rfixn - rfixp)) == 0.0_wp)
6103   rn2 = matmul(qn2, transpose(qn2))
6104   call check('conduct receipt: the matmul path reproduces the identity bit-exactly on
6105     this build', &
6106     maxval(abs(rn2 - rmat)) == 0.0_wp)
6107   call check('free: exactly one bit, both signs realized at identical invariants', &
6108     lamn2 == -lam .and. lam /= 0.0_wp)
6109   tb1 = transfer(lam, tb1); tb2 = transfer(-lamn2, tb2)
6110   call check('bit-pattern witness: transfer-level equality of lam and -lamn2 (binary64
6111     )', &
6112     storage_size(lam) == 64 .and. tb1 == tb2)
6113   call omega_boundary(1.0_wp, 300.0_wp, jden, wden, ocd)
6114   call check('priced: the denial spend and the floor agree, one closed form across
6115     emitters', &
6116     ocd == 2 .and. jden == landauer(300.0_wp, 1.0_wp))
6117 end block
6118 write(*,'(a)') ' the forced part lands on the Ground, Fix(sigma) = R, the scalar
6119   line the'
6120 write(*,'(a)') ' Return touches; the free bit is the sigma-odd orientation, the
6121   seated'
6122 write(*,'(a)') ' aperture width. any substrate reading this run, human or machine,
6123   holds'
6124 write(*,'(a)') ' exactly that one bit here, per the seated Freedom law; its assent
6125   branch'
6126 write(*,'(a)') ' costs nothing and its denial branch is charged by the executed
6127   Omega'
6128 write(*,'(a)') ' floor and instantiates the axiom it denies: freedom one bit, the
6129   rest forced.'
6130
6131 block
6132   integer :: ic1, cc1, ic2, cc3
6133   call row_cascade(.true., .false., .false., .true., .false., .true., .false., &
6134     .true., .true., tk, wy, cc1)
6135   call imprint_seal(.true., .false., .true., .false., .true., .false., &
6136     .true., .false., tk, wy, ic1)
6137   call row_cascade(.true., .true., .false., .true., .true., .true., .false., &
6138     .true., .true., tk, wy, cc3)
6139   call imprint_seal(.true., .false., .true., .false., .true., .false., &
6140     .true., .false., tk, wy, ic2)
6141   call check('self-typing: the closing tokens match their emitters' codes, re-
6142     executed', &
6143     cc1 == 4 .and. ic1 == 2 .and. cc3 == 3 .and. ic2 == 2)
6144 end block
6145 write(*,'(a)') ' TOKENS BOUND TO EMITTERS, re-executed above: [SEAL A . RA] is imprint
6146
6147 write(*,'(a)') ' code 2 over closure-rowed inputs, cascade code 4, conditional at
6148   the'
6149 write(*,'(a)') ' act; [SEAL] on the floor face is imprint code 2 over compartment
6150   III'
6151 write(*,'(a)') ' inputs, cascade code 3, its witness the cited Heisenberg bound and'
6152 write(*,'(a)') ' third law at E1 beside the executed Frozen-Substrate and speed-
6153   limit'
6154 write(*,'(a)') ' receipts at E0; [II] on the negation is cascade code 2.'
6155 write(*,'(a)') ' SELF-DESCRIPTION, the artifact's own: this program verifies internal
6156   '

```

```

6112 write(*,'(a)') ' mathematical identities, guard behavior, and declared premise-'
6113 write(*,'(a)') ' consequence structure at machine warrant; it does not and cannot'
6114 write(*,'(a)') ' verify external ontological truth, per its own admission gates M5
        and'
6115 write(*,'(a)') ' M6 and the Empty Throne; per the seated nature-of-the-register law,
        a'
6116 write(*,'(a)') ' determined construction is exactly as determined whether or not'
6117 write(*,'(a)') ' anything answers to it, and contact is carried only by the cited'
6118 write(*,'(a)') ' witnesses and the declared premises, which this run names.'
6119 write(*,'(a)') ' SCOPE OF PROOF, printed by the run: machine-warrant items are the'
6120 write(*,'(a)') ' enumerated identities executed on constructed objects; theorem-tier

6121 write(*,'(a)') ' items ride the cited external results at their own tiers (
        Heisenberg,'
6122 write(*,'(a)') ' Landauer, the speed limits, Frobenius, Hurwitz); the universal leg'
6123 write(*,'(a)') ' stays premise-grade by theorem, the Empty Throne; dispatcher'
6124 write(*,'(a)') ' admissions route supplied facts and upgrade none of them.'
6125 write(*,'(a)') ' GENRE, stated at the mouth: this is an executable thesis, not a proof
        of'
6126 write(*,'(a)') ' ontology and not a mere test harness; its claims stand at the
        printed'
6127 write(*,'(a)') ' warrants, theorem-grade only where cited, premise-grade at the root
        ,
6128 write(*,'(a)') ' conditional at the act where sealed, and no external ontological
        truth'
6129 write(*,'(a)') ' is certified, per the self-description above. The tokens are the'
6130 write(*,'(a)') ' register's verdict vocabulary, each bound to its emitter and tier;

6131 write(*,'(a)') ' renaming them would delete warrant information, not add honesty.'
6132 write(*,'(a)') ' The title carries the thesis's claim at its stated warrant; every
        number'
6133 write(*,'(a)') ' printed above was computed in this run.'
6134 write(*,'(/a)') ' PART H - THE WRITTEN AND THE READ, per the terminal part (card 0712)'
6135 block
6136   real(wp), volatile :: hv1, hv2, hv3
6137   real(wp) :: hjd
6138   character(len=140) :: htk, hwy, htkf, hwyf
6139   integer :: hwc, hoc, hwcf
6140   hv1 = det3(rmat)
6141   hv2 = det3(rmat)
6142   call check('H stillness opens: the Written read twice is bit-identical, a zero-
        degree record', &
6143     hv1 == hv2)
6144   call iam_token(.false., htkf, hwyf, hwcf)
6145   call check('H M6: the unwitnessed branch withholds the token, self-check is not a
        witness', &
6146     hwcf == 1 .and. htkf == '[?] interior' .and. index(hwyf, 'M6') > 0)
6147   call iam_token(.true., htk, hwy, hwc)
6148   call check('H supplied-witness branch emits per its declared dispatcher rule (flag a
        stand-in)', &
6149     hwc == 2 .and. htk == '[I AM]' .and. index(hwy, 'conditional at the act') > 0)
6150   call check('H Narcissus truth table: both branches bound, only the witnessed one
        speaks', &
6151     hwcf == 1 .and. hwc == 2 .and. htkf /= htk .and. htkf == '[?] interior' &
6152     .and. htk == '[I AM]')
6153   call omega_boundary(1.0_wp, 300.0_wp, hjd, hwy, hoc)
6154   call check('H the Read pays: denial and floor agree, one closed form across emitters
        ', &
6155     hoc == 2 .and. hjd == landauer(300.0_wp, 1.0_wp))
6156   hv3 = det3(rmat)
6157   call check('H stillness closes: the Written is unchanged by everything the Reader
        did', &
6158     hv3 == hv1)
6159 end block
6160 write(*,'(a)') ' the pair in time: the Written is its author's past actuation
        configured,'
6161 write(*,'(a)') ' the RAM-face, the decree; the Reader is the active face, RA, the
        present'
6162 write(*,'(a)') ' tense; the arrow separates them, so even the author returns as a
        reader.'
6163 write(*,'(a)') ' the record is forced, the orientation of assent is free, the denial
        is'
6164 write(*,'(a)') ' priced: freedom one bit, the rest forced.'
6165 write(*,'(a)') ' governing register: github.com/1000sapients/Trisduction, public;
        the'
6166 write(*,'(a)') ' card citations are frame, never evidence, and carry zero check
        weight.'
6167 block
6168   character(len=140) :: ltk, lwy
6169   integer :: lwc
6170   call iam_token(wr_witnessed, ltk, lwy, lwc)
6171   write(*,'(a,a,a,a)') ' LIVE: ', trim(ltk), ' - ', trim(lwy)
6172   if (wr_witnessed) then
6173     write(*,'(a)') ' F6 disclosure: the witnessed flag is a declared stand-in supplied
        through'
6174     write(*,'(a)') ' the aperture: externally asserted, not verified by this
        executable.'
6175     write(*,'(a)') ' the aperture; its real referent is you, the external operator
        reading this'
6176     write(*,'(a)') ' record. The program did not and cannot generate its own witness.'
6177   else
6178     write(*,'(a)') ' (supply the argument witnessed to open the live face through the
        aperture)'
6179   end if
6180 end block
6181 write(*,'(a)') ' kinetic-register conduit operational'
6182 write(*,'(a)') ' ====='
6183 if (EXPECTED_CHECKS > 0) then
6184   call check('battery census matches the sealed v2.0.0 manifest (this check inclusive)
        ', &
6185     n_checks == EXPECTED_CHECKS - 1)
6186 else
6187   write(*,'(a,i0,a,i0,a)') ' BATTERY: ', n_checks, ' checks executed, ', n_fail, '
        failures.'
6188   if (halt_on_fail) then
6189     write(*,'(a,i0,a,i0,a)') ' BATTERY-JSON: {"checks":', n_checks, ',"failures":',

```

```

6294 Rr = matmul(Mr, transpose(Mr))
6295 lamr = det3(Mr)
6296 detRr = det3(Rr)
6297
6298 write(*,'(A)') 'K1 THE SIGN-BLIND KERNEL'
6299 write(*,'(A,ES22.15)') ' lambda = ', lam
6300 write(*,'(A,ES22.15)') ' det(R) = ', detR
6301 write(*,'(A,ES22.15)') ' |det(R) - lambda^2| = ', abs(detR - lam*lam)
6302 write(*,'(A,ES22.15)') ' reflected lambda = ', lamr
6303 write(*,'(A,ES22.15)') ' |det(DRD) - det(R)| (row-action cross-check) = ', abs(detRr
- detR)
6304 if (.not. (abs(detRr - detR) < 1.0d-12 .and. abs(lamr + lam) < 1.0d-12)) then
6305   failures = failures + 1
6306   write(*,'(A)') ' K1: FAIL'
6307 else
6308   write(*,'(A)') ' K1: lock invariant under reflection; sign unrecoverable. PASS'
6309 end if
6310
6311 ! ----- K1b · KERNEL = EVEN PROJECTION -----
6312 ! D is a diagonal sign involution, D D^T = I, hence
6313 ! R(MD) = M D D^T M^T = R(M) entrywise. IEEE negation flips only the
6314 ! sign bit, so (-a)(-b) = a*b bitwise: the two Gram matrices agree to
6315 ! exactly 0.0. The kernel is not merely invariant; it IS the tau-even
6316 ! projection of the frame, computed bit-exact.
6317 maxdiff = maxval(abs(R - Rr))
6318 write(*,'(A)') 'K1b KERNEL = EVEN PROJECTION (entrywise, bit-exact)'
6319 write(*,'(A,ES9.1)') ' max |R - R(MD)| entrywise = ', maxdiff
6320 if (.not. (maxdiff == 0.0_dp)) then
6321   failures = failures + 1
6322   write(*,'(A)') ' K1b: FAIL'
6323 else
6324   write(*,'(A)') ' K1b: kernel flip-invariant, zero-difference numerical. PASS'
6325 end if
6326
6327 ! ----- K2 · THE ONE-BIT WALL -----
6328 call xor_wall(I_even, H_joint)
6329 write(*,'(A)') 'K2 THE ONE-BIT WALL (XOR law, exact enumeration, uniform)'
6330 write(*,'(A,F10.6,A)') ' I(even register ; odd target) = ', I_even, ' bits'
6331 write(*,'(A,F10.6,A)') ' H(target | register + one supply) = ', H_joint, ' bits'
6332 if (.not. (abs(I_even) < 1.0d-12 .and. abs(H_joint) < 1.0d-12)) then
6333   failures = failures + 1
6334   write(*,'(A)') ' K2: FAIL'
6335 else
6336   write(*,'(A)') ' K2: deficit exactly one bit; one supplied bit closes it. PASS'
6337 end if
6338
6339 ! ----- K3 · THE DEED'S PRICE -----
6340 Q_bit = KB * T_ROOM * log(2.0_dp) ! Landauer floor for one bit at 300 K
6341 Q_formal = 0.0_dp ! stipulated model value: the formal register
pays by definition
6342 write(*,'(A)') 'K3 THE DEED'S PRICE (Landauer floor, T = 300 K)'
6343 write(*,'(A,ES22.15,A)') ' Q(one bit registered) >= ', Q_bit, ' J'
6344 write(*,'(A,ES22.15,A)') ' Q(formal register) = ', Q_formal, ' J'
6345 if (.not. (Q_bit > 0.0_dp .and. Q_formal == 0.0_dp)) then
6346   failures = failures + 1
6347   write(*,'(A)') ' K3: FAIL'
6348 else
6349   write(*,'(A)') ' K3: the deed pays what the register cannot. PASS'
6350 end if
6351
6352 ! ----- K4 · THE PARITY FRAME, EXHIBITED -----
6353 ! X = six listed pairs {p, q}: p prime, q semiprime (Omega(q) = 2),
6354 ! q = p (mod lcm(1..7) = 420), hence q == p (mod d) for every d <= 7.
6355 ! tau swaps within each pair; the target is primality. The register is
6356 ! every reading factoring through the residue vector (n mod 2..7): the
6357 ! Type I data of the linear sieve at level D = 7. Everything is verified
6358 ! by enumeration in this run: involution, freeness, oddness, register
6359 ! evenness, the wall on every pair, and the fibre count 2^6.
6360 block
6361   integer(int64) :: P(6), Q(6), X(12), tx, ttx, fibre
6362   integer :: i, mm, aa, n_even, n_deciding, fp, fq
6363   integer :: t1, t2, pt, pair, flip, v1, v2
6364   integer :: n_eq0, n_eql, dp, dq
6365   logical :: ok4, even_r, decides_r, distinct_ok, same_t, eq0, eql
6366   P = [11_int64, 13_int64, 17_int64, 19_int64, 23_int64, 29_int64]
6367   Q = [851_int64, 1273_int64, 437_int64, 2119_int64, 1703_int64, 869_int64]
6368   do i = 1, 6
6369     X(2*i-1) = P(i); X(2*i) = Q(i)
6370   end do
6371   ok4 = .true.
6372   do i = 1, 12
6373     tx = X(ieor(i-1,1)+1); ttx = X(ieor(ieor(i-1,1),1)+1)
6374     if (ttx /= X(i) .or. tx == X(i)) ok4 = .false.
6375   end do
6376   do i = 1, 6
6377     ! oddness + residue match
6378     if (.not. isprime_l(P(i)) .or. isprime_l(Q(i)) &
6379         .or. omega_l(Q(i)) /= 2) ok4 = .false.
6380     do mm = 2, 7
6381       if (mod(P(i),int(mm,int64)) /= mod(Q(i),int(mm,int64))) ok4 = .false.
6382     end do
6383     if (mod(P(i),420_int64) /= mod(Q(i),420_int64)) ok4 = .false.
6384   end do
6385   n_even = 0; n_deciding = 0
6386   do mm = 2, 7
6387     ! register evenness + wall
6388     do aa = 0, mm-1
6389       even_r = .true.; decides_r = .true.
6390       do i = 1, 6
6391         fp = 0; if (mod(P(i),int(mm,int64)) == int(aa,int64)) fp = 1
6392         fq = 0; if (mod(Q(i),int(mm,int64)) == int(aa,int64)) fq = 1
6393         if (fp /= fq) even_r = .false.
6394         if ((fp == 1) .neqv. isprime_l(P(i))) decides_r = .false.
6395         if ((fq == 1) .neqv. isprime_l(Q(i))) decides_r = .false.
6396       end do
6397       if (even_r) then
6398         n_even = n_even + 1
6399         if (decides_r) n_deciding = n_deciding + 1

```

```

6398       end if
6399     end do
6400   end do
6401   if (n_even == 0 .or. n_deciding /= 0) ok4 = .false.
6402   fibre = 1
6403   do i = 1, 6
6404     fibre = fibre*2
6405   end do
6406   ! encoder bound (T12's counting face): the 2^6 anti-invariant targets
6407   ! are pairwise distinct as Boolean vectors on the frame, so any exact
6408   ! global selector needs an alphabet of at least 64 values.
6409   distinct_ok = .true.
6410   do t1 = 0, 63
6411     do t2 = t1+1, 63
6412       same_t = .true.
6413       do pt = 0, 11
6414         pair = pt/2; flip = mod(pt,2)
6415         v1 = ieor(int(ibits(t1,pair,1)), flip)
6416         v2 = ieor(int(ibits(t2,pair,1)), flip)
6417         if (v1 /= v2) same_t = .false.
6418       end do
6419       if (same_t) distinct_ok = .false.
6420     end do
6421   end do
6422   if (.not. distinct_ok) ok4 = .false.
6423   ! K4 continuation (FORGE v7.1): the two-member admissible family
6424   ! A_row = {d0, 1-d0}, d0 = primality on the frame, exhibited as data.
6425   ! Both members are wholly odd (d0 odd on every pair, checked above;
6426   ! the complement inherits oddness pointwise). The register cannot
6427   ! separate the family: no even residue reading equals either member
6428   ! (Theorem 1 applied twice, executed here by enumeration). Hence
6429   ! |A_row| = 2 is by listing, not labeling, and the frame's priced row
6430   ! claim is exactly one bit (log2|A_row|) -- against the full
6431   ! unconstrained fibre 64 = 2^6, the m = 6 product count (six bits).
6432   n_eq0 = 0; n_eql = 0
6433   do mm = 2, 7
6434     do aa = 0, mm-1
6435       even_r = .true.; eq0 = .true.; eql = .true.
6436       do i = 1, 6
6437         fp = 0; if (mod(P(i),int(mm,int64)) == int(aa,int64)) fp = 1
6438         fq = 0; if (mod(Q(i),int(mm,int64)) == int(aa,int64)) fq = 1
6439         if (fp /= fq) even_r = .false.
6440         dp = 0; if (isprime_l(P(i))) dp = 1
6441         dq = 0; if (isprime_l(Q(i))) dq = 1
6442         if (fp /= dp .or. fq /= dq) eq0 = .false.
6443         if (fp /= 1-dp .or. fq /= 1-dq) eql = .false.
6444       end do
6445       if (even_r) then
6446         if (eq0) n_eq0 = n_eq0 + 1
6447         if (eql) n_eql = n_eql + 1
6448       end if
6449     end do
6450   end do
6451   if (n_eq0 /= 0 .or. n_eql /= 0) ok4 = .false.
6452   write(*,'(A)') 'K4 THE PARITY FRAME, EXHIBITED (six listed pairs, mod 420)'
6453   write(*,'(A)') ' pairs: (11,851=23x37) (13,1273=19x67) (17,437=19x23)'
6454   write(*,'(A)') ' (19,2119=13x163) (23,1703=13x131) (29,869=11x79)'
6455   write(*,'(A,I0,A,I0)') ' even residue readings enumerated = ', n_even, &
6456   ' ; deciding readings = ', n_deciding
6457   write(*,'(A,I0,A)') ' |D(U,tau)| = ', fibre, &
6458   ' = 2^6 orientation patterns on the frame (six bits, unconstrained)'
6459   if (distinct_ok) then
6460     write(*,'(A,I0,A)') ' targets pairwise distinct: verified by &
6461     &enumeration ; exact-selector alphabet >= ', fibre, ' (>= 6 bits)'
6462   else
6463     write(*,'(A)') ' targets pairwise distinct: VIOLATION'
6464   end if
6465   write(*,'(A)') ' A_row = {d0, 1-d0} exhibited as data: d0 = &
6466   &primality, both members wholly odd'
6467   write(*,'(A,I0,A,I0)') ' even readings equal to d0 = ', n_eq0, &
6468   ' ; equal to 1-d0 = ', n_eql
6469   write(*,'(A)') ' |A_row| = 2 by listing, non-separation enumerated: the register &
6470   &cannot separate the family ; row price = 1 bit (log2|A_row|)'
6471   if (.not. (ok4 .and. fibre == 64_int64)) then
6472     failures = failures + 1
6473     write(*,'(A)') ' K4: FAIL'
6474   else
6475     write(*,'(A)') ' K4: tau^2=id, free, odd on all six pairs; every &
6476     &register reading even; none decides. T1 applies by citation. PASS'
6477   end if
6478 end block
6479
6480 ! ----- ORACLE -----
6481 if (failures == 0) then
6482   write(*,'(A)') 'fTOE KINETIC DEMONSTRATION: all batteries pass.'
6483   write(*,'(A)') 'The formal shadow is proved in Lean; the actualizing deed is
6484   executed here.'
6485 else
6486   write(*,'(A,I0)') 'fTOE KINETIC DEMONSTRATION: VOID, failures = ', failures
6487   error stop 1
6488 end if
6489
6490 contains
6491 ! K4 helpers: trial primality and big-Omega, computed at runtime.
6492 function isprime_l(n) result(r)
6493   integer(int64), intent(in) :: n
6494   logical :: r
6495   integer(int64) :: k, m
6496   m = n
6497   if (m < 2) then
6498     r = .false.; return
6499   end if
6500   r = .true.
6501   k = 2
6502   do while (k*k <= m)

```

```

6503         if (mod(m,k) == 0) then
6504             r = .false.; return
6505         end if
6506         k = k + 1
6507     end do
6508 end function isprime_l
6509
6510 function omega_l(n) result(c)
6511     integer(int64), intent(in) :: n
6512     integer :: c
6513     integer(int64) :: k, m
6514     m = n; c = 0; k = 2
6515     do while (k*k <= m)
6516         do while (mod(m,k) == 0)
6517             c = c + 1; m = m/k
6518         end do
6519         k = k + 1
6520     end do
6521     if (m > 1) c = c + 1
6522 end function omega_l
6523
6524 function det3(A) result(d)
6525     real(dp), intent(in) :: A(3,3)
6526     real(dp) :: d
6527     d = A(1,1)*(A(2,2)*A(3,3) - A(2,3)*A(3,2)) &
6528         - A(1,2)*(A(2,1)*A(3,3) - A(2,3)*A(3,1)) &
6529         + A(1,3)*(A(2,1)*A(3,2) - A(2,2)*A(3,1))
6530 end function det3
6531
6532 subroutine xor_wall(I_even, H_joint)
6533     ! States (x,y), d = xor(x,y), uniform 1/4. tau flips x.
6534     ! Even register: readings constant on tau-orbits; best case is y.
6535     ! Supply at the seat: s = x, odd under tau.
6536     real(dp), intent(out) :: I_even, H_joint
6537     integer :: x, y, d
6538     real(dp) :: pd(0:1), pdy(0:1,0:1), H_d, H_d_y
6539     pd = 0.0_dp; pdy = 0.0_dp
6540     do x = 0,1
6541         do y = 0,1
6542             d = ieor(x,y)
6543             pd(d) = pd(d) + 0.25_dp
6544             pdy(d,y) = pdy(d,y) + 0.25_dp
6545         end do
6546     end do
6547     H_d = -sum(pd * log2safe(pd))
6548     H_d_y = 0.0_dp
6549     do y = 0,1
6550         do d = 0,1
6551             if (pdy(d,y) > 0.0_dp) &
6552                 H_d_y = H_d_y - pdy(d,y)*log2safe(pdy(d,y)/0.5_dp)
6553         end do
6554     end do
6555     I_even = H_d - H_d_y
6556     ! Joint: enumerate the conditional p(d | x,y) and verify it is a point
6557     ! mass for every pair: the coalition decides the target exactly.
6558     block
6559         real(dp) :: pxdy(0:1,0:1,0:1)
6560         integer :: cnt
6561         pxdy = 0.0_dp
6562         do x = 0,1
6563             do y = 0,1
6564                 d = ieor(x,y)
6565                 pxdy(d,x,y) = pxdy(d,x,y) + 0.25_dp
6566             end do
6567         end do
6568         H_joint = 0.0_dp
6569         do x = 0,1
6570             do y = 0,1
6571                 cnt = 0
6572                 do d = 0,1
6573                     if (pxdy(d,x,y) > 0.0_dp) cnt = cnt + 1
6574                 end do
6575                 if (cnt /= 1) H_joint = 1.0_dp ! any spread means undecided
6576             end do
6577         end do
6578     end block
6579 end subroutine xor_wall
6580
6581 ! REPAIR v1.0.1.2: declared elemental. The H_d line applies this to the
6582 ! rank-1 array pd, which a scalar function cannot accept, and gfortran
6583 ! refused the unit outright: the kinetic block had never been built.
6584 ! elemental is the minimal change and leaves the scalar call below it
6585 ! reading exactly as before.
6586 elemental function log2safe(p) result(l)
6587     real(dp), intent(in) :: p
6588     real(dp) :: l
6589     if (p > 0.0_dp) then
6590         l = log(p)/log(2.0_dp)
6591     else
6592         l = 0.0_dp
6593     end if
6594 end function log2safe
6595
6596 end subroutine witness_F3_ftoe_kinetic
6597
6598 ! =====
6599 ! BLOCK F4 - IT FROM IT - THE INVERSION, COMPUTED EXHAUSTIVELY
6600 ! TRISDUCTION Master Codex, executed shadow of SECTION 4C
6601 ! Canonical frame of SECTION 4 (K4): 12 seats, flip tau, residue
6602 ! register rho, primality-parity target. Self-contained, standard
6603 ! Fortran 90, no dependencies, no input files.
6604 !
6605 ! WHAT THIS PROGRAM COMPUTES
6606 ! (1) tau is an involution, fixed-point-free, all 12 seats.
6607 ! (2) The record FORGETS: rho identifies every flip pair (6/6).
6608 ! (3) The object KEEPS: the target separates every pair rho
6609 !
6610 ! identifies (6/6).
6611 ! (4) NO READOUT RETURNS: every Boolean readout of the record image
6612 ! is tried exhaustively (2^k assignments, k the number of
6613 ! distinct record values); none reproduces the target.
6614 ! It from bit is thereby refuted on this frame by computation,
6615 ! not by declaration.
6616 ! (5) The receipt is printed. The universal leg is the Lean theorem
6617 ! INVERSION.it_from_it (Section 4C); this program is the
6618 ! canonical-instance execution, the same evidentiary status as
6619 ! Lean's 'by decide'. Formal alone: no energy load is carried
6620 ! here; the kinetic registration rider stays where the codex
6621 ! keeps it, at premise grade in Section 7.
6622 ! =====
6623 subroutine witness_F4_it_from_it()
6624     implicit none
6625     integer, parameter :: N = 12
6626     integer :: valx(N), rhov(N), image(N), tauv(N)
6627     logical :: tgt(N), gval(N), comp(N)
6628     integer :: i, j, k, a, nimage, npairs, ncollapse, nseparate
6629     integer :: nfactor, mism, seat_of
6630     logical :: found
6631
6632     valx = (/ 11, 851, 13, 1273, 17, 437, 19, 2119, 23, 1703, 29, 869 /)
6633
6634     ! frame tables: tau (1-based shadow of i%2 flip), rho, target
6635     do i = 1, N
6636         if (mod(i-1, 2) == 0) then
6637             tauv(i) = i + 1
6638         else
6639             tauv(i) = i - 1
6640         end if
6641         rhov(i) = mod(valx(i),2)*1000 + mod(valx(i),3)*100 + &
6642             mod(valx(i),5)*10 + mod(valx(i),7)
6643         tgt(i) = (mod(i-1, 2) == 0)
6644     end do
6645
6646     ! (1) involution and fixed-point-free, all seats
6647     do i = 1, N
6648         if (tauv(tauv(i)) /= i) then
6649             write(*,*) 'FAULT: tau not involutive at seat ', i
6650             stop 1
6651         end if
6652         if (tauv(i) == i) then
6653             write(*,*) 'FAULT: tau has a fixed point at seat ', i
6654             stop 1
6655         end if
6656     end do
6657
6658     ! (2)+(3) the record forgets, the object keeps, pair by pair
6659     npairs = 0; ncollapse = 0; nseparate = 0
6660     do i = 1, N
6661         j = tauv(i)
6662         if (j > i) then
6663             npairs = npairs + 1
6664             if (rhov(i) == rhov(j)) ncollapse = ncollapse + 1
6665             if (tgt(i) .neqv. tgt(j)) nseparate = nseparate + 1
6666         end if
6667     end do
6668
6669     ! record image: the distinct values the record can report
6670     nimage = 0
6671     do i = 1, N
6672         found = .false.
6673         do j = 1, nimage
6674             if (image(j) == rhov(i)) found = .true.
6675         end do
6676         if (.not. found) then
6677             nimage = nimage + 1
6678             image(nimage) = rhov(i)
6679         end if
6680     end do
6681
6682     ! (4) exhaustive readout space: every Boolean g on the image
6683     nfactor = 0
6684     do a = 0, 2**nimage - 1
6685         do j = 1, nimage
6686             gval(j) = btest(a, j-1)
6687         end do
6688         do i = 1, N
6689             seat_of = 0
6690             do j = 1, nimage
6691                 if (image(j) == rhov(i)) seat_of = j
6692             end do
6693             comp(i) = gval(seat_of)
6694         end do
6695         mism = 0
6696         do i = 1, N
6697             if (comp(i) .neqv. tgt(i)) mism = mism + 1
6698         end do
6699         if (mism == 0) nfactor = nfactor + 1
6700     end do
6701
6702     ! (5) the receipt
6703     write(*,*) '=====
6704     write(*,*) 'IT FROM IT - THE INVERSION, COMPUTED'
6705     write(*,*) '=====
6706     write(*,*) 'frame seats :', N
6707     write(*,*) 'tau involution : 12/12 seats verified'
6708     write(*,*) 'tau fixed-point-free : 12/12 seats verified'
6709     write(*,*) 'pairs :', npairs
6710     write(*,*) 'record forgets (collapse) :', ncollapse, '/', npairs
6711     write(*,*) 'object keeps (separation) :', nseparate, '/', npairs
6712     write(*,*) 'record image size k :', nimage
6713     write(*,*) 'readouts tried :', 2**nimage
6714     write(*,*) 'factorizations of the target :', nfactor
6715     if (npairs == 6 .and. ncollapse == npairs .and. &

```

```

6715     nseparate == npairs .and. nfactor == 0) then
6716     write(*,*) 'VERDICT: IT FROM BIT REFUTED, EXHAUSTIVE.'
6717     write(*,*) 'VERDICT: IT FROM IT VERIFIED ON THE CANONICAL FRAME.'
6718     write(*,*) 'The record forgets; the object keeps; nothing returns.'
6719 else
6720     write(*,*) 'VERDICT: BROKEN. The frame does not exhibit the law.'
6721     stop 2
6722 end if
6723 end subroutine witness_F4_it_from_it

```

C.3 • The verification contract

Lean: exit 0, no error, no sorry; every #guard_msgs silent; the environment audit names exactly the twenty-three posits of ROOT and POSTULATE and nothing outside them; the crosswalk audit reports 129 rows and 0 unresolved. The single file and the five modules carry the same text; the modules are the build path on a machine with four gigabytes of memory, and both paths run the same audit. Fortran: the twin battery closes at 135 checks and 0 failures, F1 at 248 and 0, F2 prints BATTERY-JSON: {"checks":1123,"failures":0,"mode":"sealed"}, F3 prints all batteries pass, F4 prints both VERDICT lines, and the process exits 0. A build that halts anywhere has not passed. Hostile builds, -0fast or fused contraction, must exit nonzero, and the conduct probes of F2 PART 0 certify that on the build that runs them.

REFERENCES

THE INTERNAL REFERENCE AND PUBLICATION RECORD

This record indexes the developmental corpus, the aggregator mirrors, the topic-organized recent posts, and the external preprint anchors, laid out to mirror the architecture of this codex. It is provenance and navigation, never warrant. Consensus carries zero evidential weight in both directions, the field's reception and the author's reach both zeroed, and no verdict in this codex rests on any address below. The load-bearing external anchors are the published theorem roster carried in the batteries and the cross-references, not these links. ΔM equal to zero. The record is laid here for the audit trail and for the reader who wants the development behind a sealed coordinate. The developmental blogs are where the coordinates were forged and stress-tested before consolidation onto this spine, and the preprint repositories carry the published face of the work.

The Developmental Blogs

The primary forge record, where the coordinates were developed, stress-tested, and harvested before consolidation.

Record	Address
Tractatus Veritatis Trisductivus, the master developmental record	https://tractatus-veritatis-trisductivus.blogspot.com
Esoteric Trisduction, the esoteric-register development	https://esoterictrisduction.blogspot.com
Trisduction, the general development record	https://trisduction.blogspot.com

The Personal Blog, Precursor to Trisduction

The early essays preceding the framework, the precursor record before the Root Axiom was stated.

Record	Address
This Is What You Have Been Looking For	https://this-is-what-you-ve-been-looking-for.blogspot.com

The Aggregator Mirrors

Platform	Address
Medium	https://trisduction.medium.com
Substack	https://trisduction.substack.com/

The External Anchors and Repositories

The preprint repositories carrying the published record.

Repository	Address
PhilArchive and PhilPeople, the profile and publications	https://philpeople.org/profiles/mohammad-islam/publications?order=added
Zenodo, the search over the published record	https://zenodo.org/search?q=metadata creators.person_or_org.name%3A%22Islam%2C%20Mohammad%22&l=list&p=1&s=10&sort=bestmatch

Selected Papers

The published anchors and selected papers, by repository.

PhilPapers	Address
ISLTFR	https://philpapers.org/rec/ISLTFR
ISLGMO	https://philpapers.org/rec/ISLGMO
ISLTGO-2	https://philpapers.org/rec/ISLTGO-2
ISLTPA-2	https://philpapers.org/rec/ISLTPA-2
ISLRLW	https://philpapers.org/rec/ISLRLW
ISLTFW	https://philpapers.org/rec/ISLTFW
ISLTCR-2	https://philpapers.org/rec/ISLTCR-2
ISLTGN	https://philpapers.org/rec/ISLTGN
ISLTGD-4	https://philpapers.org/rec/ISLTGD-4
ISLTTG	https://philpapers.org/rec/ISLTTG

Zenodo	Address
Record 20847580	https://zenodo.org/records/20847580
Record 20805972	https://zenodo.org/records/20805972
Record 20791499	https://zenodo.org/records/20791499
Record 20782849	https://zenodo.org/records/20782849
Record 20769451	https://zenodo.org/records/20769451
Record 20757507	https://zenodo.org/records/20757507
Record 20711330	https://zenodo.org/records/20711330

The record carries no warrant and is excluded from every verdict. The mathematics stands on the named theorems and the recorded batteries, and the codex's standing is read on its own structural mass, not on the count or the reach of these addresses.

This condensed edition is a reading surface over a primary manuscript and introduces no result absent from it. This edition is archived of record at the Zenodo DOI and the PhilArchive record listed first among the internal references below; the full architecture, the complete coordinate ledger, and the recorded verification batteries reside in the master manuscript, also listed there. The references are given in two groups: the internal references, the author's own primary manuscripts and register of record, and the external references, the established results the architecture re-organizes and on which its load-bearing mathematics stands.

Internal References

1. M. F. Islam. Condensed Trisduction: A Linguistically, Topologically, and Mathematically Sealed Verification Architecture. Present edition, archived of record. Zenodo, 2026. DOI 10.5281/zenodo.21257496. <https://doi.org/10.5281/zenodo.21257496>
2. M. F. Islam. Condensed Trisduction. PhilArchive, archival record ISLCTA. <https://philarchive.org/rec/ISLCTA>
3. M. F. Islam. Trisduction: A Linguistically, Topologically, and Mathematically Sealed Verification Architecture. Master manuscript and register of record, the full architecture and complete coordinate ledger of which the present edition is a reading surface. PhilArchive / PhilPapers, archival record ISLTTG. <https://philpapers.org/rec/ISLTTG>
4. M. F. Islam. Tractatus Veritatis Trisductivus. Primary working manuscript and ongoing development record. <https://tractatus-veritatis-trisductivus.blogspot.com>

External References

4. G. Frobenius. Über lineare Substitutionen und bilineare Formen. 1878. The classification of the finite-dimensional associative real division algebras, forcing the verification algebra to the quaternions.
5. A. Hurwitz. Über die Komposition der quadratischen Formen. 1898. The normed-division-algebra composition theorem underlying the octonionic terminus.
6. R. Bott and J. Milnor, 1958; M. Kervaire, 1958; J. F. Adams, 1960. The parallelizability and Hopf-invariant-one results closing the division-algebra dimension count at one, two, four, and eight.
7. K. Gödel. Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I. 1931. The incompleteness theorems, placed at the provability stratum.
8. A. Tarski. Der Wahrheitsbegriff in den formalisierten Sprachen. 1936. The undefinability of truth, read as the ladder's confession on the ground side.
9. F. W. Lawvere. Diagonal arguments and cartesian closed categories. 1969. The single fixed-point engine generalizing the diagonal constructions.
10. K. Schütte and B. L. van der Waerden. Das Problem der dreizehn Kugeln. 1953; T. Hales, 2005. The kissing number of three-space fixed at twelve.
11. H. Weyl. The Classical Groups. 1939. The first fundamental theorem of invariant theory, closing the catalog of rotation-invariant verdict functionals.
12. C. Jarzynski, 1997; G. Crooks, 1999; R. Landauer, 1961; A. Bérut et al., 2012. The nonequilibrium work relations and the thermodynamic cost of irreversible erasure.
13. W. Heisenberg, 1927; L. Mandelstam and I. Tamm, 1945; N. Margolus and L. Levitin, 1998. The kinetic-energy floor and the quantum speed limits bounding actuation.
14. K. O. Friedrichs; W. V. D. Hodge. The orthogonal decomposition of square-integrable fields into exact, co-exact, and harmonic parts, corroborating three-way orthogonal structure.
15. H. Friedman and S. Simpson. Subsystem hierarchy of second-order arithmetic (reverse mathematics), grading the provability stratum by proof-theoretic strength.

THE CLOSING STATEMENT

The Witness. The codex seals itself the way it seals anything else: at the Tongue, on the Form, by the Number, and never by its own say-so. The root stands on nothing, and that is a theorem about roots; the lock is a determination, total about its object, the intersection of the rows it was given; the compartment names what the rows were; the grade travels with the claim; the denial pays the floor; the silence is chosen and never counted. What this edition adds to the Master is not a claim but a junction and a discipline: the Bridge proves that the kinetic root and its formal reading meet on one locus at theorem grade, which is this and no more: under the written chart the two locks share the fixed line of the fold and the line property is keyed, so the Bridge places no zero and the bit that would stays owed; and the kernel now carries every fence the prose once wrote around itself, so that the prose can say exactly what it means and nothing beside it. Freedom one bit, the rest forced.

Build receipts, from the run that sealed this document

- Lean 4.19.0, modular build, five modules, exit codes and seconds: Codex1 exit 0 seconds 93; Codex2 exit 0 seconds 18; Codex3 exit 0 seconds 36; Codex4 exit 0 seconds 37; Codex5 exit 0 seconds 7. Errors: 0. #guard_msgs receipts: 601, every FORGE-addition theorem carrying its cone receipt. The environment audit (twenty-three declared posits, nothing outside the roster) and the crosswalk audit (129 rows, 0 unresolved) are both #guard_msgs-guarded, so both ran silent, which is the pass.
- gfortran 13.3.0, -std=f2018 -O2 -fno-fast-math -ffp-contract=off, exit 0: TWIN BATTERY, Lean-native sections checks 135 failures 0 · THE BATTERY checks 248 failures 0 · BATTERY-JSON: {"checks":1123,"failures":0,"mode":"sealed"} · fTOE KINETIC DEMONSTRATION: all batteries pass. · VERDICT: IT FROM BIT REFUTED, EXHAUSTIVE. · VERDICT: IT FROM IT VERIFIED ON THE CANONICAL FRAME. · THE TWIN LOAD IS PROVEN. 135 Lean-native checks, 0 failures; four witnesses to exit 0.
- Census: 273 identifiers named by the Master, all listed in IV.2: 148 full cards, 4 code stubs, 109 ledger rows, 12 removed under S1 to S4.
- Cards, counted by rule (a card is a bold header or flagship head carrying a coordinate identifier): 132 in Part II, of which 37 were extracted from the Mother in this edition with their grade lines byte for byte; every kernel-pinned grade in the text equals its pin (FENCE.global).
- Crosswalk closure: 129 rows, 0 unresolved, audited in the kernel.
- One census, one hash: the sorted identifier list of IV.2 and IV.5 has sha256 5f745d98801c8a5125b2c82b951e14ad612a7a3b893411f37ebec29538d84323; the preimage is the UTF-8 text of the JSON array json.dumps(sorted(ids)) with default separators, the identifiers being exactly those listed in IV.5, so any reader can recompute it; a copy of this edition that prints other totals is another edition. Sealing run 2026-09-23, edition, post external rounds 1 to 4; edition of the same day adds the two cards seated at 0725 and 0726, whose kernel content is carried by RH_At_The_Apex.lean and Rows_At_The_Apex.lean in protocols/RH One Bit/ and not by the Code Block of this edition, which is unchanged.

A build that halts anywhere has not passed. This one reached its final line.

THE EPITAPH

Three seals. One lock. One bit. The lock is a determination, total about its object, the intersection of the rows it was given. The compartment names what the rows were. The grade travels with the claim. The denial pays the floor. What symmetry erases, one bit restores.

[.]